Principia Softwarica: The Plan 9 Graphics System `draw`

version 0.1

Yoann Padioleau
yoann.padioleau@gmail.com

with code from
Rob Pike and Russ Cox

February 12, 2026

The text and figures are Copyright © 2014–2026 Yoann Padioleau.
All rights reserved.

The source code is Copyright © 2021 Plan 9 Foundation.
MIT license.

Contents

1 Introduction **13**
- 1.1 Motivations . . . 13
- 1.2 The Plan 9 graphics system: `draw` . . . 14
- 1.3 Other graphics systems . . . 14
- 1.4 Getting started (x86) . . . 16
- 1.5 Requirements . . . 16
- 1.6 About this document . . . 17
- 1.7 Copyright . . . 17
- 1.8 Acknowledgments . . . 17

2 Overview **18**
- 2.1 Graphics system principles . . . 18
  - 2.1.1 Display server . . . 18
  - 2.1.2 Image resolution . . . 18
  - 2.1.3 Color representation . . . 19
  - 2.1.4 Raster graphics (pixels) . . . 19
  - 2.1.5 Vector graphics (shapes) . . . 19
  - 2.1.6 Rasterization . . . 20
  - 2.1.7 `bitblt()` and compositing . . . 20
  - 2.1.8 Fonts . . . 20
  - 2.1.9 Graphical application versus textual application . . . 21
- 2.2 `/dev/draw/` filesystem interface . . . 21
- 2.3 `hellodraw.c` . . . 22
- 2.4 Code organization . . . 24
- 2.5 Software architecture . . . 24
  - 2.5.1 Trace of a drawing operation . . . 27
  - 2.5.2 Data structures relationships . . . 29
- 2.6 Book structure . . . 30

3 Core Data Structures **31**
- 3.1 Geometry . . . 31
  - 3.1.1 `Point` . . . 31
  - 3.1.2 `Rectangle` . . . 32
- 3.2 `Display` (and `Client`) . . . 34
  - 3.2.1 `Display` and `display` . . . 34
  - 3.2.2 `Client` . . . 35
  - 3.2.3 `KDraw` and `sdraw` . . . 36
- 3.3 `Image` (and `Memimage`) . . . 37
  - 3.3.1 `Image` . . . 37
  - 3.3.2 `Memimage` . . . 37

3.3.3 Memdata . . . 38
3.3.4 screenimage, gscreen, and and gscreendata . . . 39
3.3.5 DImage . . . 39
3.4 Channels . . . 40
3.5 Colors . . . 41
3.5.1 imgtorgba() . . . 42
3.5.2 rgbatoimg() . . . 43
3.5.3 memfillcolor() . . . 43
3.6 Drawop . . . 44
3.7 Fonts . . . 45
3.8 Overlapping layers . . . 45

4 Initializations **46**
4.1 Kernel side . . . 46
4.1.1 Binding the VGA device (x86) . . . 46
4.1.2 Setting the screen size: VGA screensize() (x86) . . . 46
4.1.3 Binding the draw device . . . 49
4.1.4 Next steps . . . 55
4.2 Client side . . . 55
4.2.1 initdraw() . . . 55
4.2.2 initdisplay() . . . 55
4.2.3 closedisplay() . . . 59
4.2.4 Next steps . . . 60
4.3 Fonts and overlapping layers . . . 60

5 User/Kernel Bridge **61**
5.1 The draw filesystem . . . 61
5.1.1 drawwalk() . . . 62
5.1.2 drawopen() . . . 65
5.1.3 drawclose() . . . 65
5.1.4 drawread() . . . 66
5.1.5 drawwrite() . . . 67
5.2 /dev/draw/ files . . . 67
5.2.1 Opening /dev/draw/new . . . 67
5.2.2 Reading from /dev/draw/x/ctl . . . 68
5.2.3 Writing in /dev/draw/x/data . . . 69
5.3 The drawing commands protocol . . . 69
5.3.1 Step 1: API function . . . 69
5.3.2 Step 2: marshalling . . . 69
5.3.3 Step 3: unmarshalling and drawmesg() . . . 70
5.3.4 Step 4: algorithm . . . 71
5.3.5 Step 5 (optional): hardware support hook . . . 71

6 Images **72**
6.1 Allocation . . . 72
6.1.1 API . . . 72
6.1.2 Marshalling . . . 72
6.1.3 Unmarshalling . . . 75
6.1.4 Algorithm . . . 76
6.1.5 Client and Memimage connection, DImage . . . 77

6.2 Clipping and replicating . . . 79
6.2.1 API . . . 79
6.2.2 Marshalling . . . 79
6.2.3 Unmarshalling . . . 80
6.2.4 Algorithm . . . 80
6.3 Flushing . . . 80
6.3.1 API . . . 80
6.3.2 Marshalling . . . 81
6.3.3 Unmarshalling . . . 81
6.3.4 Algorithm . . . 81
6.3.5 Hardware support hook (x86) . . . 82
6.4 Naming and sharing . . . 82
6.4.1 API . . . 82
6.4.2 Marshalling . . . 83
6.4.3 Unmarshalling . . . 85
6.4.4 Algorithm . . . 86

7 Manipulating Rectangles **89**
7.1 `draw()` . . . 89
7.1.1 API . . . 89
7.1.2 Marshalling . . . 90
7.1.3 Unmarshalling . . . 91
7.1.4 Algorithm . . . 92
7.1.5 Hardware support hook (x86) . . . 94
7.2 Clipping . . . 95
7.2.1 `drawclip()` . . . 95
7.2.2 Clipping to destination . . . 96
7.2.3 Clipping to source . . . 96
7.2.4 Clipping to mask . . . 96
7.2.5 Clipping and replication . . . 97
7.3 Replicating . . . 98
7.3.1 Source replication . . . 98
7.3.2 Mask replication . . . 99
7.4 Setting or copying pixels . . . 100
7.4.1 `memoptdraw()` . . . 100
7.4.2 Setting pixels . . . 101
7.4.3 Copying pixels . . . 101
7.5 Compositing pixels . . . 102
7.5.1 `alphadraw()` . . . 102
7.5.2 Buffering . . . 104
7.5.3 Clipping and replicating . . . 104
7.5.4 Additional data structures . . . 105
7.5.5 Reading pixels: `readfn()` . . . 108
7.5.6 Writing pixels: `writefn()` . . . 109
7.5.7 Reading the mask . . . 110
7.5.8 Compositing pixels: `alphacalcxx()` . . . 111

8 Drawing Shapes **113**
8.1 Lines . . . 113
8.1.1 API . . . 113
8.1.2 Marshalling . . . 113
8.1.3 Unmarshalling . . . 114
8.1.4 Algorithm . . . 114
8.2 Polygons . . . 118
8.2.1 API . . . 118
8.2.2 Marshalling . . . 118
8.2.3 Unmarshalling . . . 119
8.2.4 Algorithm . . . 121
8.3 Ellipses . . . 128
8.3.1 API . . . 128
8.3.2 Marshalling . . . 129
8.3.3 Unmarshalling . . . 129
8.3.4 Algorithm . . . 130
8.4 Arcs . . . 134
8.4.1 API . . . 134
8.4.2 Unmarshalling . . . 134
8.4.3 Algorithm . . . 135

9 Fonts **137**
9.1 Font files formats . . . 137
9.2 Additional data structures . . . 137
9.2.1 `Font` and `font` . . . 137
9.2.2 `Subfont` and `Fontchar` . . . 138
9.2.3 Font image cache . . . 139
9.3 Additional (user) initializations . . . 139
9.3.1 Default subfont: `getdefont()` . . . 140
9.3.2 Default font . . . 142
9.4 Font allocation . . . 144
9.4.1 `buildfont()` . . . 144
9.4.2 `freefont()` . . . 146
9.5 Drawing text . . . 147
9.5.1 API . . . 147
9.5.2 Marshalling . . . 149
9.5.3 Unmarshalling . . . 151
9.5.4 Algorithm . . . 152
9.6 Computing text width . . . 153
9.6.1 API . . . 153
9.6.2 Algorithm . . . 153
9.7 Character cache . . . 154
9.7.1 `Cacheinfo` . . . 155
9.7.2 Loading characters: `cachechars()` . . . 156
9.7.3 Loading one new character: `loadchar()` . . . 157
9.7.4 Resizing the character cache: `fontresize()` . . . 161
9.7.5 `agefont()` . . . 164
9.8 Subfonts . . . 165
9.8.1 Subfont cache: `Cachesubf` . . . 165
9.8.2 Subfont loading . . . 165

9.8.3 Subfont sharing . . . 169
9.8.4 Subfont IO . . . 170

10 Image IO **173**
10.1 Loading images in the display server . . . 173
10.1.1 API . . . 173
10.1.2 marshalling . . . 174
10.1.3 Unmarshalling . . . 174
10.1.4 Algorithm . . . 175
10.2 Unloading images from the display server . . . 176
10.2.1 API . . . 176
10.2.2 Marshalling . . . 177
10.2.3 Unmarshalling . . . 177
10.2.4 Algorithm . . . 178
10.3 Image format . . . 179
10.4 Reading images from the disk . . . 180
10.5 Writing images to the disk . . . 182

11 Input Devices **183**
11.1 Mouse . . . 183
11.1.1 `/dev/mouse` interface . . . 183
11.1.2 Additional data structures . . . 183
11.1.3 Additional (user) initializations: `initmouse()` . . . 184
11.1.4 IO process and communication channels . . . 184
11.1.5 API . . . 186
11.2 Cursor . . . 186
11.2.1 Additionnal data structures . . . 186
11.2.2 Additionnal (kernel) initializations . . . 188
11.2.3 `swcursorclock()` . . . 189
11.2.4 Drawing rectangles and the cursor (x86) . . . 190
11.2.5 Enabling the cursor (x86) . . . 191
11.2.6 Setting the cursor (x86) . . . 192
11.2.7 `/dev/cursor` . . . 193
11.3 Keyboard . . . 193
11.3.1 `/dev/cons` and `/dev/consctl` interfaces . . . 193
11.3.2 Additional data structures . . . 193
11.3.3 Additional (user) initializations: `initkeyboard()` . . . 194
11.3.4 IO process and communication channels . . . 195

12 Overlapping Layers **197**
12.1 Additional data structures . . . 197
12.1.1 Base layer: `Screen` (and `Memscreen`) . . . 197
12.1.2 Layer (and `Memlayer`) . . . 199
12.1.3 Layer stack . . . 200
12.2 Base layer allocation . . . 200
12.2.1 API . . . 200
12.2.2 Marshalling . . . 201
12.2.3 Unmarshalling . . . 202
12.2.4 `Client` and `Memscreen` connection: `CScreen` . . . 203
12.3 Layers creation . . . 206

12.3.1 API . . . 206
12.3.2 Unmarshalling . . . 207
12.3.3 Algorithm . . . 207
12.3.4 `Client` and `Memimage` connection: `DImage` . . . 210
12.4 Layer stack manipulation . . . 211
12.4.1 API . . . 211
12.4.2 Marshalling . . . 211
12.4.3 Unmarshalling . . . 212
12.4.4 Algorithm . . . 213
12.5 Layer visibility . . . 216
12.5.1 `memlhide()` . . . 216
12.5.2 `memlexpose()` . . . 217
12.6 Layer origin . . . 217
12.6.1 API . . . 217
12.6.2 Marshalling . . . 218
12.6.3 Unmarshalling . . . 218
12.6.4 Algorithm . . . 219
12.7 Drawing and Layers . . . 221
12.7.1 Drawing rectangles on overlapping layers . . . 221
12.7.2 Drawing lines on overlapping layers . . . 227
12.7.3 Loading an image on overlapping layers . . . 230

13 Windowing System Support **232**
13.1 Additional data structures . . . 232
13.1.1 `view` . . . 232
13.1.2 `screen` . . . 232
13.2 Additional (user) initializations . . . 232
13.2.1 `geninitdraw()` . . . 232
13.2.2 `gengetwindow()` . . . 232
13.3 `/dev/winname` and `screenname` . . . 234
13.4 `/dev/label` . . . 235
13.5 Additional mouse functions . . . 235
13.5.1 The resize event . . . 235
13.6 `newwindow()` . . . 236

14 Non 32-Bits Images TODO **237**
14.1 1-Bit images . . . 237
14.2 Small-depth images . . . 239
14.3 8-bits images . . . 244
14.4 16-bits images . . . 245
14.5 24-bits images . . . 246
14.6 Gray-scale images . . . 247
14.7 Images with a color map . . . 247
14.7.1 `RGB` . . . 247
14.7.2 `Memcmap` . . . 247
14.7.3 Conversions . . . 248
14.7.4 API . . . 251
14.7.5 `/dev/draw/x/colormap` . . . 252
14.7.6 Hardware support hook (x86) . . . 253

15 Advanced Topics TODO **256**
15.1 Advanced shapes . . . 256
15.1.1 Special line ends . . . 256
15.1.2 Bezier curves . . . 261
15.2 Advanced compositing . . . 266
15.2.1 Reading replicated pixels . . . 266
15.2.2 Other compositing operators . . . 267
15.3 Advanced mouse settings . . . 271
15.3.1 `/dev/cursor` . . . 271
15.3.2 `mousescrollsize()` . . . 272
15.4 Menus . . . 272
15.4.1 `Menu` . . . 272
15.4.2 Constants . . . 273
15.4.3 Colors . . . 273
15.4.4 `menuhits()` . . . 274
15.4.5 Last hit . . . 279
15.4.6 Scrolling menus . . . 279
15.5 Selection . . . 281
15.5.1 `getrect.c` . . . 281
15.6 Event-based IO . . . 283
15.6.1 Data structures . . . 283
15.6.2 Initialization: `einit()` . . . 285
15.6.3 The slaves . . . 286
15.6.4 Event loop: `event()` . . . 288
15.6.5 The master: `extract()` . . . 290
15.6.6 `ecanxxx()` . . . 291
15.6.7 Timer . . . 292
15.6.8 Misc . . . 293
15.6.9 `egetrects.c` . . . 294
15.6.10 `emenuhits.c` . . . 296
15.7 Graphical text mode . . . 301
15.8 Blank screen . . . 303
15.8.1 Hardware support hook (x86) . . . 305
15.9 Refresh methods . . . 306
15.9.1 `/dev/draw/x/refresh` . . . 307
15.9.2 Window refresh . . . 308
15.10 Concurrency . . . 311
15.10.1 `lockdisplay()` . . . 311
15.10.2 `/dev/draw/x/ctl` concurrent access . . . 311
15.11 Optimisations . . . 312
15.11.1 `memimagedraw()` optimisations . . . 312
15.11.2 `alphadraw()` optimisations . . . 314
15.11.3 `drawchar()` optimisations . . . 324
15.11.4 Flushing bounding boxes . . . 324
15.11.5 Image compression . . . 328
15.12 Other image format . . . 338
15.13 Old image format . . . 338
15.14 `Memimage` duplicated API . . . 338
15.15 Video and animation . . . 346

16 Conclusion **347**

A Debugging **348**
A.1 drawsetdebug() . . . 348
A.2 Dumpers . . . 349
A.2.1 Points and rectangles . . . 349
A.2.2 Channels . . . 350
A.2.3 Drawing commands . . . 350
A.3 Memory tags . . . 351

B Error Management **352**
B.1 User side . . . 352
B.2 Kernel side . . . 352

C Mathematics **354**
C.1 Trigonometric functions . . . 354

D Raspberry Pi VideoCore Driver TODO **361**

E PC VGA driver TODO **362**
E.1 IO ports . . . 363
E.2 VGAdev . . . 365
E.3 /dev/vgaxxx . . . 366
E.4 VGA linear addressing . . . 375
E.5 VGA page . . . 377
E.6 VESA . . . 378
E.7 Cirrus logic 5446 . . . 379
E.7.1 Linear mode . . . 379
E.7.2 Page . . . 380
E.7.3 Cursor . . . 380
E.8 /bin/vga . . . 384

F Examples of Graphical Applications TODO **385**

G Extra Code **386**
G.1 include/ . . . 386
G.1.1 include/graphics/draw.h . . . 386
G.1.2 include/graphics/window.h . . . 391
G.1.3 include/graphics/internals/font.h . . . 391
G.1.4 include/graphics/internals/marshal.h . . . 392
G.1.5 include/graphics/internals/draw_private.h . . . 393
G.1.6 include/graphics/internals/memdraw.h . . . 393
G.1.7 include/graphics/internals/memlayer.h . . . 395
G.1.8 include/graphics/input/keyboard.h . . . 396
G.1.9 include/graphics/input/mouse.h . . . 397
G.1.10 include/graphics/cursor.h . . . 397
G.1.11 include/graphics/input/event.h . . . 397
G.2 lib_graphics/libdraw/ . . . 398
G.2.1 lib_graphics/libdraw/arith.c . . . 398
G.2.2 lib_graphics/libdraw/icossin.c . . . 399
G.2.3 lib_graphics/libdraw/icossin2.c . . . 399

G.2.4 lib_graphics/libdraw/resolution.c . . . 400
G.2.5 lib_graphics/libdraw/chan.c . . . 400
G.2.6 lib_graphics/libdraw/concurrency.c . . . 400
G.2.7 lib_graphics/libdraw/init.c . . . 400
G.2.8 lib_graphics/libdraw/globals.c . . . 401
G.2.9 lib_graphics/libdraw/error.c . . . 401
G.2.10 lib_graphics/libdraw/flush.c . . . 401
G.2.11 lib_graphics/libdraw/alloc.c . . . 402
G.2.12 lib_graphics/libdraw/naming.c . . . 402
G.2.13 lib_graphics/libdraw/misc.c . . . 402
G.2.14 lib_graphics/libdraw/debug.c . . . 402
G.2.15 lib_graphics/libdraw/fmt.c . . . 403
G.2.16 lib_graphics/libdraw/defont.c . . . 403
G.2.17 lib_graphics/libdraw/draw.c . . . 409
G.2.18 lib_graphics/libdraw/line.c . . . 410
G.2.19 lib_graphics/libdraw/poly.c . . . 410
G.2.20 lib_graphics/libdraw/ellipse.c . . . 410
G.2.21 lib_graphics/libdraw/bezier.c . . . 411
G.2.22 lib_graphics/libdraw/fontcache.c . . . 412
G.2.23 lib_graphics/libdraw/font.c . . . 412
G.2.24 lib_graphics/libdraw/colormap.c . . . 412
G.2.25 lib_graphics/libdraw/replclipr.c . . . 413
G.2.26 lib_graphics/libdraw/string.c . . . 413
G.2.27 lib_graphics/libdraw/stringwidth.c . . . 414
G.2.28 lib_graphics/libdraw/subfont.c . . . 414
G.2.29 lib_graphics/libdraw/desktop.c . . . 415
G.2.30 lib_graphics/libdraw/window.c . . . 415
G.2.31 lib_graphics/libdraw/transfer.c . . . 416
G.2.32 lib_graphics/libdraw/io.c . . . 416
G.2.33 lib_graphics/libdraw/compressed.c . . . 417
G.2.34 lib_graphics/libdraw/keyboard.c . . . 417
G.2.35 lib_graphics/libdraw/mouse.c . . . 417
G.2.36 lib_graphics/libdraw/menuhit.c . . . 418
G.2.37 lib_graphics/libdraw/getrect.c . . . 418
G.2.38 lib_graphics/libdraw/event.c . . . 419
G.2.39 lib_graphics/libdraw/egetrect.c . . . 420
G.2.40 lib_graphics/libdraw/emenuhit.c . . . 421
G.3 lib_graphics/libdraw/tests/ . . . 421
G.3.1 lib_graphics/libdraw/tests/drawtest.c . . . 421
G.4 lib_graphics/libmemdraw/ . . . 422
G.4.1 lib_graphics/libmemdraw/resolution.c . . . 422
G.4.2 lib_graphics/libmemdraw/chan.c . . . 422
G.4.3 lib_graphics/libmemdraw/alloc.c . . . 422
G.4.4 lib_graphics/libmemdraw/arc.c . . . 422
G.4.5 lib_graphics/libmemdraw/compressed.c . . . 423
G.4.6 lib_graphics/libmemdraw/cmap.c . . . 423
G.4.7 lib_graphics/libmemdraw/defont.c . . . 428
G.4.8 lib_graphics/libmemdraw/init.c . . . 429
G.4.9 lib_graphics/libmemdraw/alphadraw.c . . . 429

G.4.10 lib_graphics/libmemdraw/color.c . . . 431
G.4.11 lib_graphics/libmemdraw/draw.c . . . 432
G.4.12 lib_graphics/libmemdraw/ellipse.c . . . 433
G.4.13 lib_graphics/libmemdraw/fillpoly.c . . . 433
G.4.14 lib_graphics/libmemdraw/hooks.c . . . 434
G.4.15 lib_graphics/libmemdraw/line.c . . . 435
G.4.16 lib_graphics/libmemdraw/transfer.c . . . 435
G.4.17 lib_graphics/libmemdraw/poly.c . . . 435
G.4.18 lib_graphics/libmemdraw/io.c . . . 436
G.4.19 lib_graphics/libmemdraw/string.c . . . 436
G.4.20 lib_graphics/libmemdraw/subfont.c . . . 436
G.5 lib_graphics/libmemdraw/scripts/ . . . 436
G.5.1 lib_graphics/libmemdraw/scripts/mkcmap.c . . . 436
G.6 lib_graphics/libmemdraw/tests/ . . . 438
G.6.1 lib_graphics/libmemdraw/tests/arctest.c . . . 438
G.6.2 lib_graphics/libmemdraw/tests/memdrawtest.c . . . 439
G.7 lib_graphics/libmemlayer/ . . . 458
G.7.1 lib_graphics/libmemlayer/draw.c . . . 458
G.7.2 lib_graphics/libmemlayer/lalloc.c . . . 458
G.7.3 lib_graphics/libmemlayer/layerop.c . . . 458
G.7.4 lib_graphics/libmemlayer/ldelete.c . . . 458
G.7.5 lib_graphics/libmemlayer/lhide.c . . . 459
G.7.6 lib_graphics/libmemlayer/line.c . . . 459
G.7.7 lib_graphics/libmemlayer/load.c . . . 459
G.7.8 lib_graphics/libmemlayer/lorigin.c . . . 459
G.7.9 lib_graphics/libmemlayer/lsetrefresh.c . . . 460
G.7.10 lib_graphics/libmemlayer/ltofront.c . . . 460
G.7.11 lib_graphics/libmemlayer/ltorear.c . . . 460
G.7.12 lib_graphics/libmemlayer/unload.c . . . 460
G.8 windows/apps/ . . . 461
G.8.1 windows/apps/clock.c . . . 461
G.8.2 windows/apps/colors.c . . . 463
G.8.3 windows/apps/screenlock.c . . . 466
G.9 kernel/devices/screen/ . . . 472
G.9.1 kernel/devices/screen/portscreen.c . . . 472
G.9.2 kernel/devices/screen/portscreen.h . . . 473
G.9.3 kernel/devices/screen/swcursor.c . . . 474
G.9.4 kernel/devices/screen/swconsole.c . . . 475
G.9.5 kernel/devices/screen/devdraw.h . . . 476
G.9.6 kernel/devices/screen/drawinit.c . . . 477
G.9.7 kernel/devices/screen/drawname.c . . . 478
G.9.8 kernel/devices/screen/drawalloc.c . . . 478
G.9.9 kernel/devices/screen/drawwindow.c . . . 479
G.9.10 kernel/devices/screen/drawerror.c . . . 479
G.9.11 kernel/devices/screen/drawmesg.c . . . 480
G.9.12 kernel/devices/screen/drawmisc.c . . . 480
G.9.13 kernel/devices/screen/devdraw.c . . . 481
G.10 kernel/devices/screen/arm/ . . . 483
G.10.1 kernel/devices/screen/arm/screen.c . . . 483

G.11 kernel/devices/screen/386/ 487
G.11.1 kernel/devices/screen/386/screen.h 487
G.11.2 kernel/devices/screen/386/screen.c 488
G.11.3 kernel/devices/screen/386/devvga.c 497
G.11.4 kernel/devices/screen/386/vgaclgd542x.c 498
G.11.5 kernel/devices/screen/386/vgavesa.c 499

Glossary **503**

Index **504**

References **504**

Chapter 1

Introduction

The goal of this book is to present with full details the source code of a graphics system.

1.1 Motivations

Why a graphics system? Because I think you are a better programmer if you fully understand how things work under the hood, and with so many fancy graphics on your screen, there are plenty of things happening under the hood to understand.

A graphics system provides essentially an application programming interface (API) to draw things: lines, rectangles, circles, text, etc. It can also mediate access to a shared resource: the screen. Graphics systems are usually coupled with a windowing system to form a graphical user interface (GUI). GUIs, introduced in the 1970's with the Xerox Alto [TML+79], were a vast improvement over text-based user interfaces, to the point where every mainstream operating systems now come with a GUI (e.g., Microsoft Windows, macOS and iOS, Linux with X Window, or Android).

Graphics systems are usually implemented by a combination of hardware and software. In fact, modern graphics systems rely more and more on specialized hardware, especially for 3D operations. However, in this book, I will cover only 2D operations, implemented mostly in software. I will assume very little from the graphics hardware. Even though 2D operations are usually covered quickly in books about computer graphics (e.g., [FDFH90]), where the focus is more on 3D and advanced rendering techniques, the result of simple 2D operations is what most people see on their screen. Indeed, most graphical applications use simple drawing primitives such as functions to draw lines, rectangles, or text. Those primitives are the foundation of even the windowing systems themselves. Indeed, the Windows book [Pad16b], which covers the Plan 9 windowing system, relies only on the simple functions described in this book.

Here are a few questions I hope this book will answer:

- Which low-level instructions allow to modify the screen?
- How is the picture on the screen represented in the machine? What is a frame buffer?
- How do the kernel and graphics system provide and protect access to the screen?
- How are images represented in memory? How are they stored on the disk? What is the format of a JPEG file?
- How are graphical elements such as rectangles, lines, or circles, rendered on the screen? How does the graphics card help?
- What is the algorithm to draw arbitrary lines (non-vertical and non-horizontal lines) on a rectangular grid of pixels (the screen)?

- How are colors represented in the machine? What does "True Color" mean?
- What is the alpha channel? What is alpha compositing?
- What is the rasterization process? What is vector graphics?
- How are characters drawn on the screen? How fonts are managed? What are "True Type" fonts?
- What is anti-aliasing?
- How is the mouse cursor handled? Where are saved the pixels that were under the cursor? How those pixels are restored on the screen when the mouse moves?
- How does the introduction of graphics and the mouse change the programming model of an application? What are the differences between a command-line application and a graphical application beyond the obvious?

1.2 The Plan 9 graphics system: `draw`

I will explain in this book the code of the Plan 9 graphics system[1], visible to the user primarily through the `/dev/draw/` device directory, and to the programmer mostly through the `draw.h` header file. Because users manipulate graphical applications with a mouse, I will also cover code related to the `/dev/mouse` device file (and the `mouse.h` and `cursor.h` header files). In total, the Plan 9 graphics system, which from now on I will call just `draw`, contains 23 300 lines of code (LOC), including the code of device drivers of two common graphics cards. `draw` is written entirely in C.

Like for most books in Principia Softwarica, I chose a Plan 9 program because those programs are simple, small, elegant, open source, and they form together a coherent set. The ancestor of `draw`, the graphics system of the Blit machine [Pik83a], also designed by Rob Pike, was one of the first graphics systems back in 1982. It was created even before the Macintosh, X Window, and Microsoft Windows.

`draw` services, like many other services in Plan 9, are accessible through files. Indeed, `draw` is a device *and* a filesystem (see the KERNEL book [Pad14] if this sounds confusing). `draw` manages the files under `/dev/draw/`. Because you can export filesystems through the network in Plan 9, `draw` is also a networked graphics system, similar to X Window. Thus, in Plan 9, programs running on one machine can even display their results on another machine.

In Plan 9, as opposed to X Window, the graphics system and windowing system are clearly separated. You can use graphical applications without a windowing system running. The code of the Plan 9 windowing system, called `rio`, will be explained in another book (the WINDOWS book [Pad16b]).

1.3 Other graphics systems

Here are a few graphics systems that I considered for this book, but which I ultimately discarded:

- Xorg[2] is the most popular open-source implementation of the X Window System [SG86], a graphics system (and a windowing system) designed in the 1980's at MIT. It is used by all Linux distributions and has even been ported to other operating systems such as macOS and Microsoft Windows. X Window defines a protocol, X11, for a networked client/server architecture. Clients using a set of libraries (e.g., Xlib) communicate with a display server (`Xorg`) to render graphics. Xorg supports many graphics cards (with hardware acceleration), many extensions (e.g., 3D operations), and many operating systems. However,

[1] http://plan9.bell-labs.com/magic/man2html/3/draw
[2] http://www.x.org

its codebase is enormous with more than 1 million LOC, even when counting just the code of the display server, a few core libraries, and a few device drivers.

X Window was designed at a time where memory was extremely limited, and remote applications were crucial. Time has changed, but this historical baggage means that even the code of simple applications such as a graphical hello world is very long and complicated to write [Ros88]. Some programmers wrote toolkits such as Gtk[3] or Qt[4] to simplify things, but those toolkits add another layer of complexity on something that is already very complicated. Finally, the graphics and windowing system parts of Xorg are strongly coupled; this coupling makes the whole system harder to understand than `draw` and `rio`, which we can study separately.

- Ghostscript[5] is an open source interpreter for Postscript [Inc85], a graphics language invented by Adobe and used initially in printers. Postscript was a very influential programming language; most graphics API today use it as a model. However, Ghostscript is a large program, with more than 750 000 LOC.

  Note that a few graphics systems, for instance, Display Postscript (DPS) in the NeXT computer, or the NeWS windowing system [GRA89] from Sun, were based on Postscript and embedded a Postscript interpreter to render graphics on the screen. One advantage of this approach is that it is easy to print what is on the screen when the printer also understands Postscript; the same code can be used. Moreover, what is printed will match exactly what is on the screen. However, one disadvantage is that you have to use two languages to program a graphical application: one for the business logic (e.g., C), and one for the graphics (Postscript).

- Cairo [WP03][6] is a 2D graphics library used in many applications (e.g., Firefox). However, it is not a complete graphics system; it relies on a "backend" for the final display. It is relatively large with its 235 000 LOC (not including the tests and benchmark code). but supports many drawing primitives and many backends (e.g., X Window via Xlib, macOS via Quartz 2D). Cairo can also generate directly graphic files with backends for Postscript, PDF, SVG, PNG, etc. In Cairo, all drawing operations are expressed in float coordinates, not pixels, and can be transformed by any affine transformations (rotation, scaling, translation), which is very flexible.

  Note that both Cairo and Xorg rely on the Pixman library[7] for low-level pixel manipulation and compositing. This library adds another 50 000 LOC.

- Skia[8] is a 2D graphics library developed by Google that is similar to Cairo. It is the graphics engine of Android and Chrome. It is also large: 485 000 LOC (without the tests).

- Mesa[9] is a 3D graphics library implementing the OpenGL [WNDS99] (Open Graphics Library) specification. OpenGL is a standard API for rendering 2D and 3D vector graphics. Mesa contains a few software-based renderers, as well as drivers supporting hardware-accelerated operations for many graphics cards. However, its codebase is enormous with more than 1.6 million LOC.

  Mesa in some sense is more general than Cairo, Skia, or `draw`. Indeed, if a function can draw a line in 3D, the same function can be used to draw a line in 2D. However, the algorithms and geometry involved are far more complicated to explain. In practice, the graphics used in most applications are 2D graphics, except for some video games and CAD software, which is why I decided to focus on 2D graphics in this book. Moreover, OpenGL does not have a good API to draw text on the screen, even though text is fundamental, even in graphical applications.

[3] http://www.gtk.org/
[4] https://www.qt.io/
[5] http://www.ghostscript.com/
[6] http://cairographics.org/
[7] http://www.pixman.org/
[8] https://skia.org/
[9] http://www.mesa3d.org/

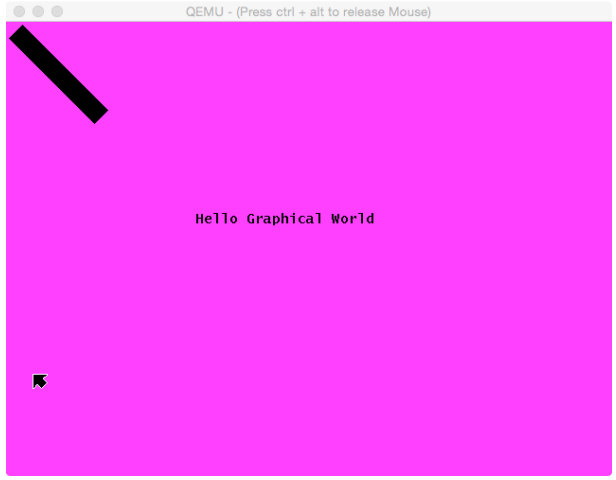

Figure 1.1: `hellodraw` screenshot.

1.4 Getting started (x86)

To play with `draw`, you will first need to install the Plan 9 fork used in Principia Softwarica (see http://www.principia-softwarica.org/wiki/Install). Once installed, you can test `draw` under Plan 9 on an x86 machine by executing the following commands:

```
1   $ bind -a '#v' /dev
2   $ vga -l 640x480x8
# screen should change layout
3   $ bind -a '#i' /dev

4   $ cd /tests/draw
5   $ 8c hellodraw.c
6   $ 8l hellodraw.8 -o hellowdraw
7   $ ./hellodraw
```

You should then see on your screen the graphics in Figure 1.1.

The command in Line 1, above, installs the VGA[10] device driver, which exports the VGA device files in `/dev/` (e.g., `/dev/vgactl`). See the Kernel book [Pad14] for more information on how device drivers work and how to use `bind`. You can also read `docs/man/3/0intro` for an introduction to the Plan 9 devices.

Line 2 runs the program `vga`. This program internally uses the `/dev/vgactl` device file to instruct the graphics card to change the resolution, here to 640 by 480 with 8 bits per pixel. The effect of this command will change the layout of the screen, but you will still be in a terminal textual mode.

Line 3 then installs the `draw` device, which exports the `draw` device files in `/dev` (e.g., `/dev/draw/new`).

Line 5 and Line 6 respectively compiles and links the simple C program `hellodraw.c`, which uses functions from `draw.h`.

Finally, Line 7 executes the program, which will internally use files under `dev/draw/` to create the graphics shown in Figure 1.1.

1.5 Requirements

Because most of this book contains C source code, you will need a good knowledge of the C programming language [KR88] to understand it. I also assume you have a basic understanding of geometry and trigonometry.

[10] VGA (Video Graphics Array) is a graphics standard for x86 machines.

Books on computer graphics such as *Computer Graphics, Principles and Practice* [FDFH90] would be useful, especially for Chapter 8, which contains the algorithms do draw shapes (e.g., lines, ellipses). However, those books are not mandatory; most computer graphics books focus on 3D, which I do not cover at all.

I strongly suggest reading the KERNEL book [Pad14] before reading this book, especially for Chapter 5, which describes the user-to-kernel bridge. Indeed, an important part of `draw` resides in the kernel and implements interfaces such as `Dev` defined by the kernel. `draw` is implemented as a device; I assume you know what is a device file for instance.

It can also be useful to read the many manual pages concerning `draw`. Those pages are located under `docs/man/` in my Plan 9 repository. Here is a partial list of those pages and a short description of their content:

- `2/graphics`: the big picture (no pun intented)
- `2/draw`: the `draw.h` API
- `2/memdraw`: the drawing functions used in the kernel
- `3/draw`: the `draw` protocol (supported by the `/dev/draw/x/data` file)
- `6/image`: the Plan 9 image format

A few other manual pages concern advanced functionalities of `draw` such as the font system (in `6/font`), the mouse (in `2/mouse` and `3/mouse`), or overlapping layers (in `2/window`), but you can read them later.

The historical paper [Pik83b] describes an ancestor of `draw` for the Blit machine. This article is useful to understand some of the design decisions presented in this book.

1.6 About this document

This document is a *literate program* [Knu92]. It derives from a set of files processed by a tool, `syncweb` [Pad09], generating either this book or the actual source code of the program. The code and its documentation are thus strongly connected.

1.7 Copyright

Most of this document is made of source code from Plan 9, so those parts are copyright by Plan 9 Foundation. The prose is mine and is licensed under the All Rights Reserved License.

1.8 Acknowledgments

I would like to acknowledge of course the authors of `draw`: Rob Pike and Russ Cox. They wrote in some sense most of this book.

Chapter 2

Overview

Before showing the source code of `draw` in the following chapters, I first give in this chapter an overview of the general principles of a graphics system. I also describe quickly the interface of the files in `/dev/draw/`, and show the code of a toy program using functions from `draw.h`. I also define terms, explain how the code is organized, and more generally give the background necessary to understand the code I will show later.

2.1 Graphics system principles

As said in the introduction, a *graphics system* provides essentially an API to draw things on a screen: lines, rectangles, circles, characters, etc. The API usually allows also to draw those graphical elements with different colors, widths, fonts, filling patterns, orientations, etc. A *screen* is composed of thousands of picture elements called *pixels*, which can be individually accessed programmatically. Instead of the screen, the programmer can also draw first in off-screen *images*, and then copy parts of those images on the screen.

2.1.1 Display server

Graphics systems are usually implemented as display servers. A *display server* is a program that accepts *drawing commands* from multiple *clients* via a *communication protocol*, and then translates those commands into instructions to the graphics card. Designers of graphics systems often used a client/server architecture because graphics systems are usually coupled with a windowing system. In that case, the graphics system needs to serve many clients: the multiple processes corresponding to the multiple windows on the screen.

In most operating systems (e.g., Linux, macOS, or Microsoft Windows), the graphics system is *strongly* coupled with the windowing system. However, this does not have to be the case. In fact, in Plan 9 you can run a graphical application without any windowing system running.

2.1.2 Image resolution

The *resolution* of a screen or image, for example, `640x480x8`, indicates the width (640), height (480), and depth (8) of the image. The *width* and *height* specify the dimensions of the two-dimensional array of pixels, also called the *grid*. The *depth* represents the number of bits used per pixel. For instance, with a depth of 8 bits, each pixel can take one of 256 different values.

Pixel values usually encode either a color or a shade of gray depending on the monitor. For the shades of gray, 0 can mean black and 255 white. In old black-and-white displays, each pixel was using only one bit: 0 for black and 1 for white. Pictures were then called *bitmaps*.

2.1.3 Color representation

Colors are usually decomposed into three different *components*: red, green, and blue, or *RGB* for short. Note that those colors are *primary colors*, meaning that any color can be expressed as a combination of those three colors. For instance, yellow is an equal mix of red and green.

The pixel value of a colored image can be translated into an actual color by using, for instance, a color map. A *color map* associates to a pixel value between 0 and 255 the three different values for the different RGB components, for example, 127 for red, 127 for green, and 0 for blue (forming a light yellow).

Note that color maps are not necessary. If the resolution of a screen is for instance `1024x768x24`, meaning that 24 bits are used to represent one pixel, then those 24 bits can be decomposed in three different bytes encoding each one of the RGB components. The use of 24 bits per pixel, also known as the *True Color* format, is enough to encode 16 million (2^{24}) different colors.

2.1.4 Raster graphics (pixels)

The term *raster graphics* is often used to talk about graphics in terms of pixels. The word raster comes from the *raster scan* process used in the cathode ray tubes of old television sets. This process painted an image line-by-line by illuminating, with three different electron emitters, thousands of phosphorescent dots triples (one red, one green, one blue) arranged in a grid. In raster graphics, a programmer sees an image as a two-dimensional array.

The software interface of a graphic card can be as simple as a reserved area of the memory address space used to represent the screen as a two-dimensional array. Such an area is often called the *frame buffer*. You can then draw on the screen simply by modifying bytes in the frame buffer. The graphics card then interprets the pixel values in the frame buffer to send the appropriate signals to the monitor.

The graphics card of the Raspberry Pi machine uses such an interface, as explained in Appendix D. When the resolution of the screen is `1024x768x24`, the frame buffer requires around 2.2 MB of address space. If we assume that the frame buffer is memory-mapped at the virtual address `0xA0000000`, then the simple function below can be used to set the color of any pixels on the screen:

⟨*function* `set_pixel` *(imaginary)* 19⟩≡

```
void set_pixel(uint x, uint y, byte red, byte green, byte blue) {
  byte* addr;

  assert(x < 1024);
  assert(y < 768);

  addr = 0xA0000000 + y*1024*3 + x*3;
  *addr = red;
  *(addr+1) = green;
  *(addr+2) = blue;
}
```

2.1.5 Vector graphics (shapes)

Drawing pictures by modifying individual pixels can be very tiresome. We would like a more high-level approach. The term *vector graphics*, used in opposition to raster graphics, expresses graphics in terms of ideal *geometric shapes*: lines, rectangles, circles, ellipsis, arcs, Bezier curves, etc. In vector graphics, a programmer sees an image as a list of shapes. A shape can be defined by a list of vectors specifying the control points of a line or a curve.

Note that vector-graphics displays, developed from the oscilloscope industry, actually predated raster-graphics displays, which were rooted in the television industry. With time, raster displays became more economical to produce and more flexible to program, and so are now dominant on the market.

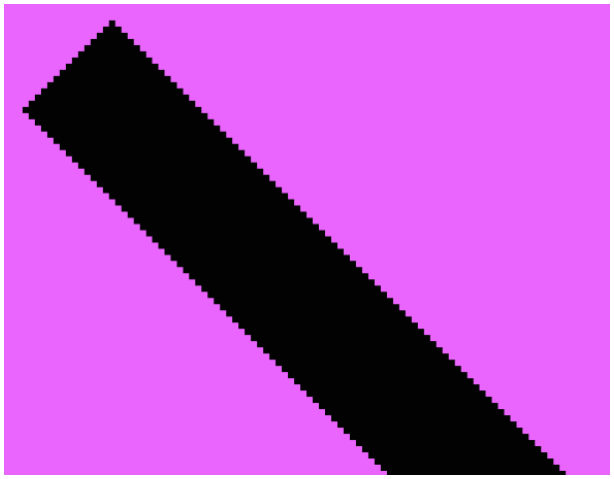

Figure 2.1: Line rasterization.

2.1.6 Rasterization

Rasterization is the process of turning vector graphics, that is geometric shapes, into raster graphics, that is pixels on a grid. Rasterization is a fundamental operation. For instance, Figure 2.1 shows the set of pixels resulting from the rasterization of a line drawn in the `hellodraw.c` program mentioned in Section 1.4. Figure 2.1 shows the magnified left part of the screen of Figure 1.1, where you can then see the individual pixels making up a line. If the resolution of the screen is high enough, and the pixels on the monitor small enough, then the brain can be fooled and can get the illusion that the line is continuous as in Figure 1.1.

2.1.7 `bitblt()` and compositing

`bitblt()`[PGI84], for *bit*-boundary *bl*ock *t*ransfer, is another fundamental graphic operation. It copies a rectangular area of an image, the *source*, to another area of the same or different image, the *destination*, while applying for each pixel involved a *composition operator* `op`. The value of a destination pixel `d` is given by the following formula where `s` is the value of a source pixel:

```
d <- d op s;
```

The name `bitblt()` comes from its use in early graphic systems where pictures were represented as bitmaps. Programmers are using `bitblt()` to highlight regions, draw horizontal and vertical lines, draw rectangles, scroll an image, draw characters, and many more things. In pictures with colors, the `bitblt()` primitive can be used to implement a variety of effects such as translucence by using the appropriate composition operator.

Under Plan 9, the `bitblt()` primitive is called `draw()` and is slightly more general. `draw()` is a very generic and powerful function. In fact, it is such a versatile procedure that, as we will see later, it is the building block used by all the other drawing functions; drawing a line, a polygon, or a circle involves at some point many internal calls to `draw()`.

2.1.8 Fonts

Geometric shapes are important, but so is text, even in a graphical-user-interface context. A graphics system can improve over text-based user interfaces (e.g., terminals) by allowing the programmer to draw text with different sizes, different styles, and even different orientations. To draw characters, the graphics system requires a series of pictures defining the shapes of all the characters of an alphabet. Such a set of pictures is called a *font*.

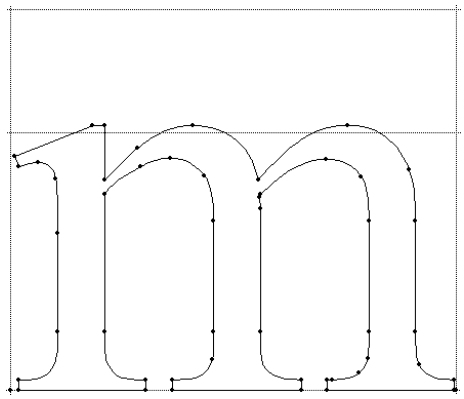

Figure 2.2: Letter `m` defined by a list of shapes.

Figure 2.3: Letter `a` defined by a grid of pixels.

Each character can be represented either using vector graphics, as in Figure 2.2, or raster graphics, as in Figure 2.3. The two approaches illustrate well the pro and cons of the two picture representations:

- In Figure 2.2, the letter `m` is defined as a set of vectors representing the control points of straight lines or Bezier curves. *TrueType* is a font format[1] using this technique. A character is then just another list of geometric shapes to draw. This approach is *scalable* as one can render the letter in different sizes or orientations by rasterizing its different elements. However, such a rasterization can be expensive to compute, and must be done each time the program wants to draw a character.
- In Figure 2.3, the letter `a` is defined instead as a bitmap. This approach supports only one size, but drawing a character can be very fast. Indeed, a programmer can just use `bitblt()` to copy the rectangle defining the character on the screen.

Note that Plan 9 uses Unicode for its characters set (and not simply ASCII), so font files can be large.

2.1.9 Graphical application versus textual application

2.2 `/dev/draw/` filesystem interface

`draw` is a Plan 9 device/filesystem, not a regular program, so its interface is not a set of command-line arguments and input-output files. Instead, its primary interface is the files it serves. Once you have installed `draw`, usually

[1] Such fonts are stored in files with the `.ttf` extension.

with the command `bind -a '#i' /dev` (see Section 1.4), `draw` serves a three-levels file tree that provides an interface to the graphics facilities of Plan 9. Here is the meaning of the main files and directories managed by `draw`:

- `/dev/draw/` is at the first level and is the main directory of `draw`; almost all the files of `draw` reside under it. As opposed to simpler devices such as the mouse with `/dev/mouse`, or the console with `/dev/cons`, the `draw` device uses a directory because it will manage multiple clients at the same time. Because each client is associated with multiple dynamically-created device files, not grouping those files together under `/dev/draw/` would pollute the `/dev/` directory.

- `/dev/draw/new` is at the second level and is the first file that a graphical application must open. It allows the opening a new connection to the `draw` display server. The function `initdraw()`, which we will see in Section 2.3, and which is used by all graphical applications, is using internally this file.

- `/dev/draw/1/`, `/dev/draw/2/`, etc. are directories corresponding to different connections to the display server. A process opening `/dev/draw/new` will create a new dynamic subdirectory under `/dev/draw/`. Those directories will make more sense in the WINDOWS book [Pad16b]. Indeed, the use of the same screen by multiple processes at the same time is useful mainly in a windowing system context where independent graphical applications are running concurrently.

- `/dev/draw/1/ctl` (and `/dev/draw/2/ctl`, etc.) is at the third level and contains mostly information about the screen, for instance, its resolution. Opening `/dev/draw/new` returns a file descriptor that corresponds to a `/dev/draw/x/ctl` file, so the programmer does not need to open this file.

- `/dev/draw/1/data` is the file used by the graphical application to communicate *drawing commands* to the display server. The format of those drawing commands is pretty regular: they start all with a single character, for instance, `'L'` for drawing a line, followed by binary parameters[2]. The code of all the drawing functions in `draw.h` eventually write a drawing command in the `/dev/draw/x/data` file.

The remaining files managed by `draw` are listed below; I will cover them later in this book:

- `/dev/draw/1/colormap` is useful only on 8-bit displays. I will cover this file in Section 14.7.

- `/dev/draw/1/refresh` is discussed in Section 15.9.

- `/dev/winname` is at the first level again and is the only file managed by `draw` that is not under `/dev/draw/`. It is related to the windowing system and I will cover it in Chapter 13.

For more information on the `draw` files and their format, I refer you to the documentation of `draw` in `docs/man/3/draw`.

2.3 hellodraw.c

Programs usually do not access directly `/dev/draw/new` or `/dev/draw/x/data`. They use instead a higher-level API declared in `draw.h`. The program `hellodraw.c` below, which I mentioned in Section 1.4, and whose

[2]Multi-byte integer parameters are transmitted with the low-order byte first.

graphical output is Figure 1.1, uses this API:

⟨lib_graphics/libdraw/tests/hellodraw.c 23⟩≡

```
1   #include <u.h>
2   #include <libc.h>
3   #include <draw.h>
4
5   void main(void) {
6       int result;
7       Image *color;
8
9       result = initdraw(nil, nil, "Hello Draw");
10      if (result < 0) {
11          exits("Error in initdraw");
12      }
13
14      color = allocimage(display, Rect(0,0, 1,1), RGB24, true, DMagenta);
15      draw(display->image, display->image->r, color, nil, ZP);
16
17      line(display->image,
18           Pt(10, 10), Pt(100, 100),
19           Endsquare, Endsquare, 10, display->black, ZP);
20      string(display->image,
21             Pt(200, 200), display->black, ZP, font,
22             "Hello Graphical World");
23      flushimage(display, true);
24
25      sleep(5000); // msec
26  }
```

Here are a few important things to note in this program:

- `draw.h`, included at Line 3, contains all the public data structures and drawing functions of `draw`. `libc.h`, included at Line 2, defines the functions of the Plan 9 core C library.
- `initdraw()`, called at Line 9, must be the first call to the `draw` API. It initializes necessary globals such as `display` (used later at Line 14), which contains information about the screen. `initdraw()` internally opens `/dev/draw/new` to open a new connection to the display server. It also sets the global `font` (used later at Line 21) to the default font of the system.
- Lines 7 and 14 illustrate an important concept in `draw`: *everything is an image*. Indeed, the `Image` type from `draw.h` unifies many things: the screen is represented as an image in `display->image`; fonts are images; windows, as we will see later, are images; even colors are images. A color is simply a `1x1` image (see the second argument of `allocimage()`, Line 14, which creates a rectangle with `Rect(0,0,1,1)`) with the *replicating* bit set to true (the fourth argument of `allocimage()`), and where all pixels have a fixed RGB value (here `DMagenta`, the last argument of `allocimage()`).
- Line 15 uses the very general `draw()` function that I mentioned before in Section 2.1.7. It copies the source image `color` (a replicated `1x1` image) to the destination image `display->image` (the screen) over the whole screen rectangle (`display->image->r`).
- Line 17 draws on the screen a black line of width 10 in diagonal. Line 20 draws some text in the font specified by the global `font`, which was set previously by `initdraw()`.
- Line 23 *flushes* the *drawing commands buffer* by writing all the previous commands in `/dev/draw/x/data`. Line 23 also instructs the `draw` server to make the result visible on the screen with the second parameter of `flushimage()`.

The following chapters will clarify the code of `hellodraw.c`, especially the meaning of the different parameters of the different functions of `draw.h`. The goal here was just to give a quick idea on how to use `draw.h`.

See Appendix F for more examples of graphical applications using `draw.h`.

2.4 Code organization

The code of `draw` can be decomposed in multiple components that correspond to different directories in the Plan 9 repository:

- `include/graphics/draw.h` contains the visible part of `draw` for the programmer. This header file contains the data structures (e.g., `Image`) and function declarations (e.g., `draw()`, `line()`, or `string()`) to draw different shapes on the screen.
- `lib_graphics/libdraw/` is a user-space library implementing the functions defined in `draw.h`. However, those functions are just thin wrappers that write into `/dev/draw/x/data` to delegate the actual rendering to code in the kernel.
- `kernel/devices/screen/` contains the kernel code that implements the `draw` filesystem, as well as a few device drivers for graphics cards. The `devdraw.c` file, in particular, contains the different filesystem methods (e.g., `drawopen()`, `drawread()`, or `drawwrite()`) that will be executed when a program accesses files from `/dev/draw/`. As we will see in Section 5.2.3, the `drawwrite()` method calls `drawmesg()` when the file involved is a `/dev/draw/x/data` file. `drawmesg()` is a large function that processes all incoming drawing commands in a giant `switch()`. This function looks at the first letter of the drawing command to decide which graphic operation to perform.
- `lib_graphics/libmemdraw/` is a library used by the kernel that contains most of the generic graphics-related code. `drawmesg()` calls mostly functions from `libmemdraw/` (e.g., `memimagedraw()`, `memimageline()`). Most of those functions operate on the `Memimage` type defined in `include/graphics/internals/memdraw.h`.
- `lib_graphics/libmemlayer/` is another library used by the kernel to manage overlapping layers. I will cover it later in Chapter 12.

Table 2.1 presents short descriptions of the source files used by `draw`. I organized those files according to the directories I mentioned above, except for the files in `lib_graphics/libmemlayer/`, which I will cover in Chapter 12. The table also shows the main entities (e.g., structures, functions, globals) the files define and the corresponding chapter in this document in which the code contained in a file is primarily discussed.

2.5 Software architecture

From the programmer's point of view, `draw` is just a library. Its software architecture is first a flat list of functions you can call (e.g., `line()`, `string()`), and data structures you can use (e.g., `Image`), to draw things on a screen. All those entities are declared in `draw.h`. Those are the *entry points* of `draw`. However, what is under those entry points is rather complicated. Indeed, as you have seen in Section 2.4, `draw` has many components:

- A user-space library (`libdraw`)
- Two kernel-space libraries (`libmemdraw` and `libmemlayer`)
- A filesystem (accessible though `/dev/draw/`, thanks mainly to `devdraw.c`)
- A protocol (for user-space programs to communicate with the kernel via a `/dev/draw/x/data` device file)

Function	Ch.	libdraw/ Files	libmemdraw/ Files	kernel/ Files	Er
data structures	3	draw.h arith.c	memdraw.h	devdraw.h screen.h	Po
image resolution	3	chan.c resolution.c	color.c resolution.c		by
initializations	4	init.c	init.c	drawinit.c	in
user/kernel bridge	5	marshal.h flush.c		devdraw.c drawmesg.c	dr
image allocation	6	alloc.c	alloc.c	drawalloc.c	al
clipping and replication	6	replclipr.c			
image naming	6	naming.c		drawname.c	na
manipulating rectangles	7	draw.c	draw.c alphadraw.c		dr
drawing lines	8	line.c	line.c		li
drawing polygons	8	poly.c	poly.c fillpoly.c		po
drawing ellipses/arcs	8	ellipse.c	ellipse.c arc.c		el
font data structures	9	font.h			Su
font initialization	9	defont.c font.c			de
text drawing	9	string.c			st
characters cache	9	fontcache.c			ca
subfont management	9	subfont.c			re
transfering an image	10	transfer.c	transfer.c		lo
image IO	10	io.c	io.c		re
mouse management	11	mouse.h mouse.c			Mo
cursor management	11	cursor.h		CURSOR.h swcursor.c	sw
keyboard management	11	keyboard.h keyboard.c			Kx
overlapping layers	12	window.h window.c		DRAWDESKTOP.c	al
color maps	14	colormap.c	cmap.c		rg
drawing Bezier curves	15	bezier.c			be
basic menus	15	menuhit.c			Me
event-based IO	15	event.c			Ev
graphical text mode	15		defont.c string.c	swconsole.c	gr
image compression	15	compressed.c	compressed.c		cl
debugging	A	debug.c			dr
dumpers	A	fmt.c			Pf
error management	B	error.c		drawerror.c	Er
trigonometry	C	icossin.c icossin2.c			si
VideoCore driver	D				
VGA generic driver	E			vga.h devvga.c	vg
Total LOC		8700	5800	7300	

Table 2.1: Chapters and source files of `draw`.

- A few device drivers (for different graphics cards)

A good way to see how those components work together is to *trace* a call to one of the functions of the `draw` API through those different components.

```
      hellodraw.c-----+    clock.c--+    colors.c--+
      |   ...         |    |        |    |         |    APPLICATIONS
U     |   line(...);  |    |        |    |         |
S     +---------------+    +--------+    +---------+
E  include/graphics/draw.h----------------------------+
R  |    void line(...);        ...                    |
   +--------------------------------------------------+
S  lib_graphics/libdraw/------------------------------+
P  |  line.c------------------------+ ellipse.c+     |    USER
A  |  |void line(...) {              | |        |     |  LIBRARIES
C  |  |  ...                         | |        |     |
E  |  |  write(display->fd, "L ...");| |        | ... |
   |  |}                             | |        |     |
   |  +------------------------------+ +--------+     |
   +--------------------------------------------------+
---------------------- /dev/draw/1/data  -----------------------
    kernel/---------------------------------------------+
    |+---+ files/------+ devices/screen/--------------+ |
K   || s | | syswrite()| | devdraw.c----------------+ | |
E   || y | | sysread() | | |void drawwrite() { ... }| | |
R   || s | |           | | |void drawmesg(...) {    | | |
N   || c | +-----------+ | |  ...                   | | |  KERNEL
E   || a |               | |  switch(c) {           | | |
L   || l |               | |  ...                   | | |
    || l |               | |  case 'L':             | | |
S   || s |               | |     memline(...);      | | |
P   || / |               | +------------------------+ | |
A   |+---+               +----------------------------+ |
C   +--------------------------------------------------+
E   lib_graphics/libmemlayer/--------------------------+
    | line.c-------------------------------------+       |
    | |void memline(...) { memimageline(...); } |  ...  |
    | +-------------------------------------------+       |
    +-----------------------------------------------------+   KERNEL
    lib_graphics/libmemdraw/-----------------------------+ LIBRARIES
    | line.c-----------------------+ ellipse.c-+        |
    | |void memimageline() { ... } | |         |   ...  |
    | +----------------------------+ +---------+        |
    +-----------------------------------------------------+
```

Figure 2.4: Tracing of `line()` through the `draw` components.

2.5.1 Trace of a drawing operation

Figure 2.4 illustrates a trace of a call to `line()`. This trace starts at the top of Figure 2.4 with the call to `line()` in the program `hellodraw.c` that you saw in Section 2.3. I will now go gradually downwards in Figure 2.4 to explain the trace:

1. `line()` is declared in `draw.h`. Its slightly simplified prototype is given below:

   ```
   void line(Image* dst, Point p0, Point p1, ...,
             int width, Image* color, ...);
   ```

2. `line()` is defined in `libdraw/`. However, as I said in Section 2.4, `libdraw` is just a thin wrapper that delegates most of the work to the kernel. The code of `line()` just writes a *drawing command* (starting with the letter `'L'`) in the file descriptor `Display.fd` of the global `display`. Remember that this global has been set previously by a call to `initdraw()`, as we saw in Section 2.3.

3. `display->fd`, as we will see in Chapter 3, is a file descriptor that corresponds to a `/dev/draw/x/data` device file. This device file is the primary interface between a graphical application and the graphics code in the kernel.

4. The call to `write()` in `libdraw/line.c` is a *system call*. It triggers a *software interrupt* that jumps to code in the kernel. This software interrupt is handled by the system-call dispatcher in `kernel/syscalls/`. For more information on how system calls work, I refer you to the KERNEL book [Pad14].

5. The system-call dispatcher calls `syswrite()` in `kernel/files/` because the original system call was `write()`.

6. `syswrite()` dispatches to the appropriate *filesystem method*. Because the argument to `write()` is a file descriptor corresponding to a file under `/dev/draw/`, `syswrite()` calls the `drawwrite()` method in `kernel/devices/screen/devdraw.c`. Again, I refer you to the KERNEL book [Pad14] to understand how filesystems work.

7. `drawwrite()` calls `drawmesg()` because the file descriptor corresponds to a `/dev/draw/x/data` device file.

8. `drawmesg()` is the kernel function that handles all the drawing commands written in `/dev/draw/x/data`. Because the first letter of the command is `'L'`, `drawmesg()` calls the `memline()` function.

9. `memline()` is a function from the `libmemlayer` library. This library, which I will describe in Chapter 12, handles images that are overlapping layers. The important thing for now is that `memline()` eventually calls `memimageline()`.

10. `memimageline()` is a function from the `libmemdraw` library. Its sligtly simplified prototype is given below:

    ```
    void memimageline(Memimage *dst, Point p0, Point p1, ...,
                      int radius, Memimage *color, ...);
    ```

    `memimageline()` contains the algorithm to draw lines in images in memory. Those images have the type `Memimage`. As we will see in Section 2.5.2, one of those images corresponds to the framebuffer; changing bytes in this image translates to changes on the screen.

`memimageline()` then returns to the caller `memline()`, which does the same. Eventually, the whole call stack is popped, leading back to code in `hellodraw.c`.

```
   display <Display>
U  |.fd = ...      |
S  |.image = --+   |
E  |           |   |
R  +-----------+---+
               v
S   <Image>---------------+  <Image>----------+
P   |.id = 1              |  |.id = 2         |
A   |.r = (0,0),(1024,768)|  |.r = (0,0),(1,1)|
C   |.chan = RGB24        |  |.chan = RGB24   |
E   +---------------------+  +----------------+
               ^                       ^
---------------+-----------------------+-------------------------
               v                       v
       <DImage>--------+      <DImage>--------+  imagmem <Pool>-+
       |.id = 1        |      | .id = 2       |  |  +--------+  |
  +--->|.image = -+    |      | .image = -+   |  |  |        |  |
  |    +----------+----+      +-----------+---+  |  |        |  |
K |               v                       v      |  +--------+  |
E | drawscreen <Memimage>-+  <Memimage>-------+  |  +-+         |
R | |.r = (0,0),(1024,768)|  |.r = (0,0),(1,1)| ++->| |         |
N | |.chan = RGB24        |  |.chan = RGB24   | ||  +-+         |
E | |.data = |            |  |.data = --------+-+|              |
L | +--------+------------+  +----------------+  |  +---------+ |
  |          +-----------------------------+     |  |         | |
S | <Client>--------+  <Client>---------+  |     |  +---------+ |
P | |.clientid = 1  |  |.clientid = ... |  |     +--------------+
A | |.dimage =|     |  |   ...          |  |
C | +---------+-----+  +----------------+  |     0xA0000000 (FB)+
E +-----------+  ^        ^                |     |              |
                 +--------+                |     |              |
  sdraw <KDraw>--++                        +---->|              |
  |              ||                              |              |
  | .clients =---+|                              |              |
  +---------------+                              +--------------+
```

Figure 2.5: Data structures of `draw`.

2.5.2 Data structures relationships

Figure 2.4 focused on the functions and the control flow of `draw`. However, the control flow is only one part of the story. A few questions are remaining: How are images represented in memory? What is the difference between an `Image` and a `Memimage`? Where do images reside in memory? In user space? In kernel space? How can the frame buffer, usually a protected area of memory, be modified by user-space programs? To answer those questions, Figure 2.5 focuses on the data structures of `draw`, as well as their relationships. This time, I will go from the top and bottom of Figure 2.5 towards its center.

At the top of Figure 2.5 is the global `display` I mentioned a few times before. Its type is the structure `Display` defined in `draw.h`. In Figure 2.4, you saw the use of `display->fd` to write drawing commands in `/dev/draw/x/data`. Another important field of `display` is `display->image`, which, as I said in Section 2.3, *represents* the screen. The type of `display->image` is the structure `Image`, also defined in `draw.h`. The global `display` and `display->image` are both initialized by `initdraw()`.

An `Image` has three important fields:

- An identifier (`Image.id`)
- A rectangle (`Image.r`)
- An image format (called `Image.chan` because it is made of a set of *channels*, which I will define in Section 3.4)

The last two fields together specify the resolution of the image. The first field provides the mean for user-space programs to reference data in the kernel. Indeed, an `Image` identifier is nothing more than a *handle* to the real image (of type `Memimage`) that resides in kernel space. The identifier is what is transmitted through `/dev/draw/x/data`. The call to `line(display->image, ...)` in Section 2.3 eventually writes `'L'` (a character), and the identifier for the screen image `1` (an integer) in `/dev/draw/x/data`. Identifiers allow programs to tell on which image to apply the drawing command.

The first `Image` allocated by `initdraw()` has always the identifier `1`, and always corresponds to the screen. A program can allocate more off-screen images, for instance with `allocimage()`, as shown in Section 2.3. As we will see in Chapter 6, calls to `allocimage()` lead to memory allocations both in user space (for `Image`) and kernel space (for `Memimage` and the data containing all the pixels).

The kernel-space side of Figure 2.5, at the bottom, is a bit more complicated than the user-space side. At the very bottom, the kernel global `sdraw` keeps track of the set of clients the `draw` display server serves. In the examples of this book, the display server will serve mostly a single client. In the WINDOWS book [Pad16b], it will serve many clients: one per window, and one for the windowing system program itself. In any case, each graphical application must first call `initdraw()`, which, by opening `/dev/draw/new`, opens a new connexion to the display server. Each connexion adds a new entry in `sdraw.clients`, which is an array of pointers to `Client`, a structure defined in `devdraw.h`.

Each `Client` has an integer identifier that corresponds to one of the `/dev/draw/x/` subdirectory. Moreover, the field `Client.dimage` contains a set of `Dimage` (for `Draw` image), a structure also defined in `devdraw.h`.

A `DImage` mirrors in kernel space part of an `Image` that resides in user-space. The `Dimage` and associated `Image` share the same identifier. However, a `DImage` also maps an image identifier to a `Memimage`, a structured defined in `memdraw.h`.

A `Memimage` is similar to an `Image`; they have similar fields (`r` and `chan`). An important difference is that the new field `Memimage.data` points to an area in kernel space where resides the actual pixels of the image. This area can be either the framebuffer (shortened `FB` in Figure 2.5) or an off-screen image allocated from the *memory pool* `imagmem` (see the KERNEL book [Pad14] and the LIBCORE book [Pad16a] for more information on memory pools). The `Memimage` representing the screen is also stored in the global `drawscreen`.

2.6 Book structure

You now have enough background to understand the source code of `draw`. The rest of the book is organized as follows. I will start by describing the core data structures of `draw` in Chapter 3. Then, in Chapter 4, I will explain how `draw` is initialized, both on the kernel side, where the graphics card needs to be configured for a particular resolution, and on the user side, where client applications need to connect to the display server. Chapter 5 will focus on the bridge between the user and the kernel. I will explain how client applications communicate with the display server.

The following chapters will describe the different functions of the `draw` API: Chapter 6 the functions to allocate images (e.g. `allocimage()`), Chapter 7 the central function `draw()` to manipulate rectangles, Chapter 8 the functions to draw shapes (e.g. `line()`), Chapter 9 the functions to draw text (e.g. `string()`, and finally Chapter 10 the functions to load and save images (e.g. `loadimage()`).

In Chapter 11, I will describe the functions of `draw` that programs can use to interact with input devices, especially the mouse. The mouse is the primary device used in graphical applications, and its use leads to the animation of some graphics on the screen: the mouse cursor.

Chapter 12 and Chapter 13 will present a major image extension: overlapping layers. They are one of the building blocks to implement a windowing system, but they can also be used in regular applications (e.g., the Plan 9 editor `acme`).

Some of the code in Chapter 6 through 10 will be simplified to show only the parts of the code that handle 32-bits images. However, a real graphics system must support different image formats: 24-bits, 16-bits, 8 bits, images using color maps, or even bitmaps. A graphics system must be able to copy the pixels from one image using one format, to another image using an entirely different format. I will cover all those cases in Chapter 14. Chapter 15 presents other features of `draw` which I did not show before to simplify the explanations, for instance, the support for Bezier curves, or optimizations in certain drawing functions.

Finally, Chapter 16 concludes and gives pointers to other books in the Principia Softwarica series.

Some appendices present the code of non-functional properties: code to help debug `draw` in Appendix A, and code to manage errors in Appendix B. Appendix C contains the code of generic mathematic functions used by `draw`, but which are not unique to `draw`. Appendix D and E will present the code for the device drivers of two traditional graphics cards. Finally, Appendix F contains more examples of graphical applications to get a better feel of how to use `draw` in practice.

Chapter 3

Core Data Structures

Show me your code and conceal your data structures, and I shall continue to be mystified. Show me your data structures, and I won't usually need your code; it'll be obvious.

Fred Brooks

In this chapter, I will present the most important data structures of `draw`. Most of them are defined in `draw.h`; the others are defined in the kernel, but they relate to structures from `draw.h`. For instance, `Client`, which is defined in `devdraw.c`, is the kernel equivalent of `Display`, which is defined in `draw.h`.

3.1 Geometry

3.1.1 Point

⟨*struct* `Point` 31a⟩≡ (386d)

```
struct  Point
{
    int x;
    int y;
};
```

⟨*function* `Pt` 31b⟩≡ (398)

```
Point
Pt(int x, int y)
{
    Point p;

    p.x = x;
    p.y = y;
    return p;
}
```

⟨*function* `eqpt` 31c⟩≡ (398)

```
bool
eqpt(Point p, Point q)
{
    return p.x==q.x && p.y==q.y;
}
```

⟨*global* `ZP` 31d⟩≡ (398)

```
Point ZP;
```

⟨*type* `Vector` 31e⟩≡ (386d)

```
typedef Point Vector;
```

⟨*function* addpt 32a⟩≡ (398)

```
Point
addpt(Point a, Point b)
{
    a.x += b.x;
    a.y += b.y;
    return a;
}
```

⟨*function* subpt 32b⟩≡ (398)

```
Point
subpt(Point a, Point b)
{
    a.x -= b.x;
    a.y -= b.y;
    return a;
}
```

⟨*function* mulpt 32c⟩≡ (398)

```
Point
mulpt(Point a, int b)
{
    a.x *= b;
    a.y *= b;
    return a;
}
```

⟨*function* divpt 32d⟩≡ (398)

```
Point
divpt(Point a, int b)
{
    a.x /= b;
    a.y /= b;
    return a;
}
```

3.1.2 Rectangle

⟨*struct* Rectangle 32e⟩≡ (386d)

```
struct Rectangle
{
    Point   min;
    Point   max;
};
```

⟨*function* canonrect 32f⟩≡ (398)

```
Rectangle
canonrect(Rectangle r)
{
    int t;
    if (r.max.x < r.min.x) {
        t = r.min.x;
        r.min.x = r.max.x;
        r.max.x = t;
    }
    if (r.max.y < r.min.y) {
        t = r.min.y;
        r.min.y = r.max.y;
        r.max.y = t;
```

```
    }
    return r;
}
```

⟨*function* Rect 33a⟩≡ (398)

```
Rectangle
Rect(int ax, int ay, int bx, int by)
{
    Rectangle r;

    r.min.x = ax;
    r.min.y = ay;
    r.max.x = bx;
    r.max.y = by;
    return r;
}
```

⟨*function* Rpt 33b⟩≡ (398)

```
Rectangle
Rpt(Point min, Point max)
{
    Rectangle r;

    r.min = min;
    r.max = max;
    return r;
}
```

⟨*function* eqrect 33c⟩≡ (398)

```
bool
eqrect(Rectangle r, Rectangle s)
{
    return r.min.x==s.min.x && r.max.x==s.max.x &&
           r.min.y==s.min.y && r.max.y==s.max.y;
}
```

⟨*global* ZR 33d⟩≡ (398)

```
Rectangle ZR;
```

⟨*function* Dx 33e⟩≡ (386d)

```
#define Dx(r)   ((r).max.x - (r).min.x)
```

⟨*function* Dy 33f⟩≡ (386d)

```
#define Dy(r)   ((r).max.y - (r).min.y)
```

⟨*function* rectaddpt 33g⟩≡ (398)

```
Rectangle
rectaddpt(Rectangle r, Point p)
{
    r.min.x += p.x;
    r.min.y += p.y;
    r.max.x += p.x;
    r.max.y += p.y;
    return r;
}
```

⟨*function* rectsubpt 34a⟩≡ (398)

```
Rectangle
rectsubpt(Rectangle r, Point p)
{
    r.min.x -= p.x;
    r.min.y -= p.y;
    r.max.x -= p.x;
    r.max.y -= p.y;
    return r;
}
```

⟨*function* ptinrect 34b⟩≡ (398)

```
bool
ptinrect(Point p, Rectangle r)
{
    return p.x >= r.min.x && p.x < r.max.x &&
           p.y >= r.min.y && p.y < r.max.y;
}
```

⟨*function* rectinrect 34c⟩≡ (398)

```
bool
rectinrect(Rectangle r, Rectangle s)
{
    return s.min.x <= r.min.x && r.max.x <= s.max.x &&
           s.min.y <= r.min.y && r.max.y <= s.max.y;
}
```

⟨*function* rectXrect 34d⟩≡ (398)

```
bool
rectXrect(Rectangle r, Rectangle s)
{
    return r.min.x < s.max.x && s.min.x < r.max.x &&
           r.min.y < s.max.y && s.min.y < r.max.y;
}
```

3.2 Display (and Client)

3.2.1 Display and display

⟨*struct* Display 34e⟩≡ (386d)

```
struct Display
{
    // ref_own<Image>, the full screen
    Image   *image;

    ⟨Display devdraw connection fields 35a⟩
    ⟨Display buffer fields 69j⟩
    ⟨Display basic images fields 35c⟩
    ⟨Display font fields 137c⟩
    ⟨Display other fields 37d⟩

    // Extra
    ⟨Display concurrency fields 311a⟩
};
```

⟨*global* display 34f⟩≡ (401a)

```
// option<Display>
Display *display;
```

⟨Display *devdraw connection fields* 35a⟩≡ (34e) 35b ▷

```
int     dirno; // /dev/draw/x
fdt     ctlfd; // /dev/draw/x/ctl (opened via /dev/draw/new)
fdt     fd;    // /dev/draw/x/data
```

⟨Display *devdraw connection fields* 35b⟩+≡ (34e) ◁ 35a 307c ▷

```
char    *devdir; // /dev in general
char    *windir; // /dev in general
```

⟨Display *basic images fields* 35c⟩≡ (34e)

```
// ref_own<Image>
Image   *white;
// ref_own<Image>
Image   *black;

// ref_own<Image>
Image   *opaque;
// ref_own<Image>
Image   *transparent;
```

3.2.2 Client

⟨*struct* Client 35d⟩≡ (476)

```
struct Client
{
    int     clientid; // /dev/draw/x/

    // hash<Image.id, ref_own<Dimage>> (next = Dimage.next in bucket)
    DImage*     dimage[NHASH];

    ⟨Client drawing state fields 44e⟩
    ⟨Client layer fields 198b⟩
    ⟨Client other fields 36c⟩

    // Extra
    ⟨Client concurrency fields 312c⟩
    Ref     r;
};
```

⟨*constant NHASH bis* 35e⟩≡ (476)

```
#define NHASH       (1<<5)
```

⟨*constant* HASHMASK 35f⟩≡ (476)

```
#define HASHMASK    (NHASH-1)
```

⟨DImage *extra fields* 35g⟩≡ (39f)

```
DImage*     next;
```

⟨*function* drawlookup 35h⟩≡ (481)

```
DImage*
drawlookup(Client *client, int id, bool checkname)
{
    DImage *d;

    d = client->dimage[id&HASHMASK];
    while(d){
        if(d->id == id){
            ⟨drawlookup() if checkname 88i⟩
            return d;
```

```
        }
        d = d->next;
    }
    return nil;
}
```

3.2.3 KDraw and sdraw

⟨*struct* KDraw 36a⟩≡ (476)

```
struct KDraw
{
    // growing_array<option<Client>> (size = KDraw.nclient)
    Client**    client;
    int     nclient;

    // gensym
    int     clientid;

    ⟨KDraw other fields 50g⟩
};
```

⟨*global* sdraw 36b⟩≡ (481)

```
KDraw        sdraw;
```

⟨Client *other fields* 36c⟩≡ (35d) 68f▷

```
// index in KDraw.clients[]
int     slot;
```

⟨*function* drawnewclient 36d⟩≡ (481)

```
Client*
drawnewclient(void)
{
    Client *cl, **cp;
    int i;

    // find free slot
    for(i=0; i<sdraw.nclient; i++){
        cl = sdraw.client[i];
        // found one? then i contains the free slot number.
        if(cl == nil)
            break;
    }
    ⟨drawnewclient() grow array if necessary 36e⟩

    cl = malloc(sizeof(Client));
    ⟨drawnewclient() sanity check cl 37a⟩
    memset(cl, 0, sizeof(Client));
    cl->slot = i;
    cl->clientid = ++sdraw.clientid;
    cl->op = SoverD; // The classic
    sdraw.client[i] = cl;
    return cl;
}
```

⟨drawnewclient() *grow array if necessary* 36e⟩≡ (36d)

```
// growing array
if(i == sdraw.nclient){
    cp = malloc((sdraw.nclient+1)*sizeof(Client*));
    ⟨drawnewclient() sanity check cp 37b⟩
```

```
    memmove(cp, sdraw.client, sdraw.nclient*sizeof(Client*));
    free(sdraw.client);
    sdraw.client = cp;
    sdraw.nclient++;
    cp[i] = nil;
}
```

⟨drawnewclient() *sanity check cl* 37a⟩≡ (36d)

```
if(cl == nil)
    return nil;
```

⟨drawnewclient() *sanity check cp* 37b⟩≡ (36e)

```
if(cp == nil)
    return nil;
```

3.3 Image (and Memimage)

3.3.1 Image

⟨*struct* Image 37c⟩≡ (386d)

```
struct Image
{
    int         id;     /* id of system-held Image */

    Rectangle   r;      /* rectangle in data area, local coords */

    channels        chan; // image format
    // derives from Image.chan
    int         depth;      /* number of bits per pixel */

    Rectangle   clipr;  /* clipping region */
    bool        repl;   /* flag: data replicates to tile clipr */

    Display     *display;   /* display holding data */

    ⟨Image layer fields 199b⟩

    // Extra
    ⟨Image extra fields 199h⟩
};
```

⟨Display *other fields* 37d⟩≡ (34e) 199g ▷

```
// gensym
int     imageid;
```

3.3.2 Memimage

⟨*struct* Memimage 37e⟩≡ (393b)

```
struct Memimage
{
    Rectangle r;  /* rectangle in data area, local coords */

    channels chan; /* channel descriptions */
    // derives from Memimage.chan
    int  depth; /* number of bits of storage per pixel */
    int  nchan; /* number of channels */
```

```
    Rectangle clipr;  /* clipping region */
    //bitset<enum<fxxx>, // includes if replicate via Frepl
    ulong flags;

    // finally, the raw pixels
    // ref_own<Memdata>
    Memdata *data; /* pointer to data; shared by windows in this image */

    ⟨MemImage layer fields 199d⟩
    ⟨MemImage other fields 39a⟩
};
```

⟨*enum* fxxx 38a⟩≡ (393b)

```
enum {
    Frepl = 1<<0, /* is replicated */

    // derives from Memimage.chan
    Fgrey = 1<<2, /* is grey */
    Falpha = 1<<3, /* has explicit alpha */
    Fcmap = 1<<4, /* has cmap channel */
    Fbytes = 1<<5, /* has only 8-bit channels */
};
```

⟨*global* memones 38b⟩≡ (429a)

```
static Memimage* memones;
```

⟨*global* memzeros 38c⟩≡ (429a)

```
static Memimage* memzeros;
```

⟨*global* memwhite 38d⟩≡ (429a)

```
Memimage *memwhite;
```

⟨*global* memblack 38e⟩≡ (429a)

```
Memimage *memblack;
```

⟨*global* memtransparent 38f⟩≡ (429a)

```
Memimage *memtransparent;
```

⟨*global* memopaque 38g⟩≡ (429a)

```
Memimage *memopaque;
```

3.3.3 Memdata

⟨*struct* Memdata 38h⟩≡ (393b)

```
/*
 * Memdata is allocated from main pool, but .data from the image pool.
 * Memdata is allocated separately to permit patching its pointer after
 * compaction when windows share the image data.
 * The first word of data is a back pointer to the Memdata, to find
 * The word to patch.
 */
struct Memdata
{
    ulong *base; /* allocated data pointer */
    // the pixels!
    byte *bdata; /* pointer to first byte of actual data; word-aligned */

    ⟨Memdata other fields 50h⟩
    // Extra
    int  ref;  /* number of Memimages using this data */
};
```

⟨MemImage *other fields* 39a⟩≡ (37e) 53d ▷
```
int  zero;  /* data->bdata+zero==&byte containing (0,0) */
ulong width; /* width in words of a single scan line */
```

⟨*function* byteaddr 39b⟩≡ (422a)
```
byte*
byteaddr(Memimage *i, Point p)
{
    byte *a;

    a = (i->data->bdata + i->zero) + (sizeof(ulong) * p.y * i->width);

    ⟨byteaddr() if depth less than 8 239d⟩
    else
        return a + p.x * (i->depth/8);
}
```

3.3.4 screenimage, gscreen, and and gscreendata

⟨*global* screenimage 39c⟩≡ (477)
```
Memimage    *screenimage;
```

⟨*global* gscreen 39d⟩≡ (472)
```
Memimage *gscreen;
```

⟨*global* gscreendata 39e⟩≡ (472)
```
Memdata gscreendata;
```

3.3.5 DImage

⟨*struct* DImage 39f⟩≡ (476)
```
/*
 * Reference counts in DImages:
 *  one per open by original client
 *  one per screen image or fill
 *  one per image derived from this one by name
 */
struct DImage
{
    int     id;
    Memimage*   image;

    ⟨DImage layer fields 199c⟩
    ⟨DImage font fields 139b⟩
    ⟨DImage other fields 87e⟩

    // Extra
    ⟨DImage extra fields 35g⟩
    int     ref;
};
```

⟨*function* allocdimage 39g⟩≡ (478b)
```
/// makescreenimage | drawinstall -> <>
DImage*
allocdimage(Memimage *i)
{
    DImage *d;
```

```
    d = malloc(sizeof(DImage));
    ⟨allocdimage() sanity check d 40a⟩
    d->image = i;
    d->ref = 1;

    d->name = nil;
    d->fromname = nil;
    d->vers = 0;
    d->nfchar = 0;
    d->fchar = 0;

    return d;
}
```

⟨allocdimage() *sanity check d* 40a⟩≡ (39g)

```
if(d == nil)
    return nil;
```

3.4 Channels

⟨*type* channels 40b⟩≡ (386d)

```
typedef ulong channels;
```

⟨*enum* ImageChan 40c⟩≡ (386d)

```
/*
 * image channel descriptors
 */
// coupling: chantostr and channames
enum ImageChan {
    CRed = 0,
    CGreen,
    CBlue,

    CGrey,
    CAlpha,

    CMap,
    ⟨ImageChan cases 237b⟩

    NChan,
};
```

⟨*function* __DC 40d⟩≡ (386d)

```
#define __DC(type, nbits)   ((((type)&15)<<4)|((nbits)&15))
```

⟨*function* NBITS 40e⟩≡ (386d)

```
#define NBITS(c) ((c)&15)
```

⟨*function* TYPE 40f⟩≡ (386d)

```
#define TYPE(c) (((c)>>4)&15)
```

⟨*function* CHAN1 40g⟩≡ (386d)

```
#define CHAN1(a,b)  __DC(a,b)
```

⟨*function* CHAN2 40h⟩≡ (386d)

```
#define CHAN2(a,b,c,d)  (CHAN1((a),(b))<<8|__DC((c),(d)))
```

⟨*function* CHAN3 40i⟩≡ (386d)

```
#define CHAN3(a,b,c,d,e,f)  (CHAN2((a),(b),(c),(d))<<8|__DC((e),(f)))
```

⟨*function* CHAN4 41a⟩≡ (386d)

```
#define CHAN4(a,b,c,d,e,f,g,h)  (CHAN3((a),(b),(c),(d),(e),(f))<<8|__DC((g),(h)))
```

⟨*enum* ImageType 41b⟩≡ (386d)

```
enum ImageType {
    GREY1   = CHAN1(CGrey, 1), // used for masks, fonts (black and white)
    CMAP8   = CHAN1(CMap, 8), // PC graphics mode by default, 1 byte per pixel
    RGB16   = CHAN3(CRed, 5, CGreen, 6, CBlue, 5), // Raspberry mode by default
    RGBA32  = CHAN4(CRed, 8, CGreen, 8, CBlue, 8, CAlpha, 8), // flexible
    ARGB32  = CHAN4(CAlpha, 8, CRed, 8, CGreen, 8, CBlue, 8),/* stupid VGAs */
    ⟨ImageType cases 237a⟩
};
```

⟨*function* chantodepth 41c⟩≡ (400b)

```
int
chantodepth(ulong c)
{
    int n;

    for(n=0; c; c>>=8){
        ⟨chantodepth() sanity check c 41d⟩
        n += NBITS(c);
    }
    ⟨chantodepth() sanity check n 41e⟩
    return n;
}
```

⟨chantodepth() *sanity check c* 41d⟩≡ (41c)

```
if(TYPE(c) >= NChan || NBITS(c) > 8 || NBITS(c) <= 0)
    return 0; // warning?
```

⟨chantodepth() *sanity check n* 41e⟩≡ (41c)

```
if(n==0 || (n>8 && n%8) || (n<8 && 8%n))
    return 0; // warning?
```

3.5 Colors

⟨*type* rgba 41f⟩≡ (386d)

```
typedef ulong rgba;
```

⟨*enum* Colors 41g⟩≡ (386d)

```
enum
{
    DOpaque         = 0xFFFFFFFF,
    DTransparent    = 0x00000000,/* only useful for allocimage, memfillcolor */

    DBlack      = 0x000000FF,
    DWhite      = 0xFFFFFFFF,

    DRed        = 0xFF0000FF,
    DGreen      = 0x00FF00FF,
    DBlue       = 0x0000FFFF,

    DCyan       = 0x00FFFFFF,
    DMagenta    = 0xFF00FFFF,
    DYellow     = 0xFFFF00FF,

    DPaleyellow = 0xFFFFAAFF,
```

```
    DDarkyellow = 0xEEEE9EFF,
    DDarkgreen  = 0x448844FF,
    DPalegreen  = 0xAAFFAAFF,
    DMedgreen   = 0x88CC88FF,
    DDarkblue   = 0x000055FF,
    DPalebluegreen= 0xAAFFFFFF,
    DPaleblue   = 0x0000BBFF,
    DBluegreen  = 0x008888FF,
    DGreygreen  = 0x55AAAAFF,
    DPalegreygreen  = 0x9EEEEEFF,
    DYellowgreen    = 0x99994CFF,
    DMedblue        = 0x000099FF,
    DGreyblue       = 0x005DBBFF,
    DPalegreyblue   = 0x4993DDFF,
    DPurpleblue     = 0x8888CCFF,

    DNotacolor  = 0xFFFFFF00, // Alpha = 0, transparent white
    DNofill = DNotacolor,

};
```

3.5.1 imgtorgba()

⟨*function* imgtorgba 42⟩≡ (431)

```
rgba
imgtorgba(Memimage *img, ulong val)
{
    byte r, g, b, a;
    ulong chan;
    int nb, v;
    ⟨imgtorgba() other locals 248d⟩

    a = 0xFF;
    r = g = b = 0xAA; /* garbage */

    for(chan=img->chan; chan; chan>>=8){
        nb = NBITS(chan);
        v = val&((1<<nb)-1);
        val >>= nb;
        ov = v;
        ⟨imgtorgba() duplicate bits if nb less than 8 239c⟩

        switch(TYPE(chan)){
        case CRed:
            r = v;
            break;
        case CGreen:
            g = v;
            break;
        case CBlue:
            b = v;
            break;
        case CAlpha:
            a = v;
            break;
        ⟨imgtorgba() switch chan type cases 247a⟩
        }
    }
    return (r<<24)|(g<<16)|(b<<8)|a;
```

```
}
```

3.5.2 rgbatoimg()

⟨*function* rgbatoimg 43a⟩≡ (431)

```
ulong
rgbatoimg(Memimage *img, rgba rgba)
{
    byte r, g, b, a;
    ulong chan;
    int d, nb;
    ulong v;
    ⟨rgbatoimg() other locals 248f⟩

    v = 0;
    r = rgba>>24;
    g = rgba>>16;
    b = rgba>>8;
    a = rgba;
    d = 0;
    for(chan=img->chan; chan; chan>>=8){
        nb = NBITS(chan);
        switch(TYPE(chan)){
        case CRed:
            v |= (r>>(8-nb))<<d;
            break;
        case CGreen:
            v |= (g>>(8-nb))<<d;
            break;
        case CBlue:
            v |= (b>>(8-nb))<<d;
            break;
        case CAlpha:
            v |= (a>>(8-nb))<<d;
            break;
        ⟨rgbatoimg() switch chan type cases 247b⟩
        }
        d += nb;
    }
    DBG1("rgba2img %.8lux = %.*lux\n", rgba, 2*d/8, v);
    return v;
}
```

3.5.3 memfillcolor()

⟨*function* memfillcolor 43b⟩≡ (431)

```
void
memfillcolor(Memimage *i, rgba val)
{
    ulong bits;
    ⟨memfillcolor() other colors 44c⟩

    if(val == DNofill)
        return;

    bits = rgbatoimg(i, val);

    switch(i->depth){
```

```
    ⟨memfillcolor() switch depth cases 246e⟩
    default: /* 1, 2, 4, 8, 16, 32 */
        ⟨memfillcolor() duplicate bits if depth less than 32 44d⟩
        memsetl(wordaddr(i, i->r.min), bits, i->width * Dy(i->r));
        break;
    }
}
```

⟨*function* wordaddr 44a⟩≡ (422a)

```
/*
 * Wordaddr is deprecated.
 */
ulong*
wordaddr(Memimage *i, Point p)
{
    return (ulong*) ((uintptr)byteaddr(i, p) & ~(sizeof(ulong)-1));
}
```

⟨*function* memsetl 44b⟩≡ (432)

```
void
memsetl(void *vp, ulong val, int n)
{
    ulong *p, *ep;

    p = vp;
    ep = p+n;
    while(p < ep)
        *p++ = val;
}
```

⟨memfillcolor() *other colors* 44c⟩≡ (43b) 246d ▷

```
int d;
```

⟨memfillcolor() *duplicate bits if depth less than 32* 44d⟩≡ (43b)

```
for(d=i->depth; d<32; d*=2)
    // duplicate bits
    bits = (bits << d) | bits;
```

3.6 Drawop

⟨Client *drawing state fields* 44e⟩≡ (35d)

```
// enum<Drawop>
int     op;
```

⟨*enum* drawop 44f⟩≡ (386d)

```
enum Drawop
{
    /* Porter-Duff compositing operators */
    Clear   = 0,

    SinD    = 8,
    DinS    = 4,
    SoutD   = 2,
    DoutS   = 1,

    S       = SinD|SoutD,
    SoverD  = SinD|SoutD|DoutS, // classic Source over Destination
    SatopD  = SinD|DoutS,
```

```
    SxorD   = SoutD|DoutS,

    D       = DinS|DoutS,
    DoverS  = DinS|DoutS|SoutD,
    DatopS  = DinS|SoutD,
    DxorS   = DoutS|SoutD,  /* == SxorD */

    Ncomp = 12,
};
```

3.7 Fonts

3.8 Overlapping layers

Chapter 4

Initializations

4.1 Kernel side

4.1.1 Binding the VGA device (x86)

4.1.2 Setting the screen size: VGA `screensize()` (x86)

⟨*function* `screensize`*(x86)* 46a⟩≡ (492)

```
int
screensize(int x, int y, int z, ulong chan)
{
    VGAscr *scr = &vgascreen;
    ⟨screensize() other locals 47g⟩

    ⟨screensize() lock 46c⟩
    ⟨screensize() initializations part1 47h⟩

    ⟨screensize() set gscreendata.bdata 47d⟩

    ⟨screensize() free previous gscreen 47b⟩
    // Setting gscreen!!
    gscreen = allocmemimaged(Rect(0,0,x,y), chan, &gscreendata);
    ⟨screensize() sanity check gscreen 47c⟩

    ⟨screensize() vga settings 255a⟩
    ⟨screensize() unlock 47a⟩

    // initial draw
    memimagedraw(gscreen, gscreen->r, memblack, ZP, nil, ZP, S);
    arch_flushmemscreen(gscreen->r);

    ⟨screensize() initializations part2 47i⟩
    return OK_0;
}
```

⟨*global* `vgascreenlock`*(x86)* 46b⟩≡ (492)

```
Lock vgascreenlock;
```

⟨`screensize()` *lock* 46c⟩≡ (46a)

```
lock(&vgascreenlock);
if(waserror()){
    unlock(&vgascreenlock);
    nexterror();
}
```

⟨screensize() *unlock* 47a⟩≡ (46a)
```
unlock(&vgascreenlock);
poperror();
```

⟨screensize() *free previous gscreen* 47b⟩≡ (46a)
```
if(gscreen)
    freememimage(gscreen);
```

⟨screensize() *sanity check gscreen* 47c⟩≡ (46a)
```
if(gscreen == nil)
    error("no memory for vga memimage");
```

Soft screen vs frame buffer (x86)

⟨screensize() *set* gscreendata.bdata 47d⟩≡ (46a)
```
if(scr->paddr == 0){
    int width = (x*z)/BI2BY; // width in bytes
    void *p;

    // !!the alloc!!
    p = xalloc(width*y);
    ⟨screensize() sanity check p 47j⟩
    gscreendata.bdata = p;
    scr->useflush = true;
    ⟨screensize() when use softscreen, other settings 47f⟩
}
else{
    // direct frame buffer
    gscreendata.bdata = scr->vaddr;
    scr->useflush = (scr->dev && scr->dev->flush);
}
```

⟨*global* softscreen*(x86)* 47e⟩≡ (492)
```
static void *softscreen;
```

⟨screensize() *when use softscreen, other settings* 47f⟩≡ (47d) 377a ▷
```
softscreen = gscreendata.bdata;
```

⟨screensize() *other locals* 47g⟩≡ (46a)
```
void *oldsoft;
```

⟨screensize() *initializations part1* 47h⟩≡ (46a) 47k ▷
```
oldsoft = softscreen;
```

⟨screensize() *initializations part2* 47i⟩≡ (46a) 188d ▷
```
if(oldsoft)
    xfree(oldsoft);
```

⟨screensize() *sanity check p* 47j⟩≡ (47d)
```
if(p == nil)
    error("no memory for vga soft screen");
```

memimageinit()

⟨screensize() *initializations part1* 47k⟩+≡ (46a) ◁ 47h
```
memimageinit();
```

⟨*function* memimageinit 48a⟩≡ (429a)

```
void
memimageinit(void)
{
    ⟨memimageinit() only once guard 48c⟩

    ⟨memimageinit() set image pool allocator move 48d⟩
    ⟨memimageinit() initializations 248b⟩
    ⟨memimageinit() install dumpers 349e⟩

    memzeros = allocmemimage(Rect(0,0,1,1), GREY1);
    memzeros->flags |= Frepl;
    memzeros->clipr = Rect(-0x3FFFFFF, -0x3FFFFFF, 0x3FFFFFF, 0x3FFFFFF);
    *byteaddr(memzeros, ZP) = 0;

    memones = allocmemimage(Rect(0,0,1,1), GREY1);
    memones->flags |= Frepl;
    memones->clipr = Rect(-0x3FFFFFF, -0x3FFFFFF, 0x3FFFFFF, 0x3FFFFFF);
    *byteaddr(memones, ZP) = ~0;

    ⟨memimageinit() sanity check memxxx 48b⟩

    memwhite = memones;
    memblack = memzeros;
    memopaque = memones;
    memtransparent = memzeros;
}
```

⟨memimageinit() *sanity check memxxx* 48b⟩≡ (48a)

```
if(memones == nil || memzeros == nil)
    assert(0 /*cannot initialize memimage library */); /* RSC BUG */
```

⟨memimageinit() *only once guard* 48c⟩≡ (48a)

```
static bool didinit = false;
if(didinit)
    return;
didinit = true;
```

memimagemove()

⟨memimageinit() *set image pool allocator move* 48d⟩≡ (48a)

```
if(  strcmp(imagmem->name, "Image") == 0
  || strcmp(imagmem->name, "image") == 0
  )
    imagmem->move = memimagemove;
```

⟨*function* memimagemove 48e⟩≡ (422c)

```
void
memimagemove(void *from, void *to)
{
    Memdata *md;

    md = *(Memdata**)to;
    ⟨memimagemove() sanity check md base 49a⟩
    md->base = to;

    /* if allocmemimage changes this must change too */
    md->bdata = (byte*)md->base + sizeof(Memdata*) + sizeof(ulong);
}
```

⟨memimagemove() *sanity check md base* 49a⟩≡ (48e)

```
if(md->base != from){
    print("compacted data not right: #%p\n", md->base);
    abort();
}
```

4.1.3 Binding the draw device

⟨*function* drawattach 49b⟩≡ (481)

```
static Chan*
drawattach(char *spec)
{
    dlock();
    if(!initscreenimage()){
        dunlock();
        error("no frame buffer");
    }
    dunlock();

    return devattach('i', spec);
}
```

⟨*global* drawlock 49c⟩≡ (481)

```
QLock   drawlock;
```

⟨*function* dlock 49d⟩≡ (481)

```
void
dlock(void)
{
    qlock(&drawlock);
}
```

⟨*function* candlock 49e⟩≡ (481)

```
int
candlock(void)
{
    return canqlock(&drawlock);
}
```

⟨*function* dunlock 49f⟩≡ (481)

```
void
dunlock(void)
{
    qunlock(&drawlock);
}
```

initscreenimage()

⟨*function* initscreenimage 49g⟩≡ (477)

```
error0
initscreenimage(void)
{

    ⟨initscreenimage() only once guard 50d⟩

    screendimage = makescreenimage();
    ⟨initscreenimage() sanity check screendimage 50e⟩
    screenimage = screendimage->image;
```

```
    ⟨initscreenimage() other initializations 235h⟩
    return OK_1;
}
```

⟨*global* screendimage 50a⟩≡ (477)

```
static  DImage* screendimage;
```

⟨*function* makescreenimage 50b⟩≡ (477)

```
static DImage*
makescreenimage(void)
{
    ⟨makescreenimage() locals 50c⟩

    ⟨makescreenimage() allocate Memdata md 50f⟩
    ⟨makescreenimage() allocate Memimage i 51c⟩
    ⟨makescreenimage() allocate DImage di 54c⟩

    ⟨makescreenimage() drawaddname 234h⟩

    return di;
}
```

⟨makescreenimage() *locals* 50c⟩≡ (50b) 50i ▷

```
Memdata *md;
Memimage *i;
DImage *di;
```

⟨initscreenimage() *only once guard* 50d⟩≡ (49g)

```
if(screenimage != nil)
    return OK_1;
```

⟨initscreenimage() *sanity check screendimage* 50e⟩≡ (49g)

```
if(screendimage == nil)
    return ERROR_0;
```

Memdata

⟨makescreenimage() *allocate Memdata md* 50f⟩≡ (50b)

```
// allocate Memdata
md = malloc(sizeof(Memdata));
⟨makescreenimage() sanity check md 51a⟩
md->allocd = true;

md->bdata = arch_attachscreen(&r, &chan, &depth, &width, &sdraw.softscreen);
⟨makescreenimage() sanity check md bdata 51b⟩
md->base = nil; // not allocated by poolalloc
md->ref = 1;
```

⟨KDraw *other fields* 50g⟩≡ (36a) 86c ▷

```
bool     softscreen;
```

⟨Memdata *other fields* 50h⟩≡ (38h)

```
bool allocd; /* is this malloc'd? */
```

⟨makescreenimage() *locals* 50i⟩+≡ (50b) ◁ 50c

```
int width, depth;
Rectangle r;
ulong chan;
```

⟨makescreenimage() *sanity check md* 51a⟩≡ (50f)

```
if(md == nil)
    return nil;
```

⟨makescreenimage() *sanity check md bdata* 51b⟩≡ (50f)

```
if(md->bdata == nil){
    free(md);
    return nil;
}
```

Memimage

⟨makescreenimage() *allocate Memimage i* 51c⟩≡ (50b)

```
// allocate Memimage
i = allocmemimaged(r, chan, md);
⟨makescreenimage() sanity check i 51d⟩
```

⟨makescreenimage() *sanity check i* 51d⟩≡ (51c)

```
if(i == nil){
    free(md);
    return nil;
}
```

⟨*function* allocmemimaged 51e⟩≡ (422c)

```
Memimage*
allocmemimaged(Rectangle r, ulong chan, Memdata *md)
{
    int d;
    ulong l;
    Memimage *i;

    ⟨allocmemimaged() sanity check r 52b⟩
    d = chantodepth(chan);
    ⟨allocmemimaged() sanity check d 52c⟩
    l = wordsperline(r, d);

    i = mallocz(sizeof(Memimage), true);
    ⟨allocmemimaged() sanity check i 52d⟩

    i->data = md;
    ⟨allocmemimaged() set zero field 52a⟩
    i->width = l;
    i->r = r;
    i->clipr = r;
    i->flags = 0;

    i->layer = nil;
    i->cmap = memdefcmap;

    if(memsetchan(i, chan) < 0){
        free(i);
        return nil;
    }
    return i;
}
```

⟨allocmemimaged() *set zero field* 52a⟩≡ (51e)

```
i->zero = sizeof(ulong) * l * r.min.y;
if(r.min.x >= 0)
    i->zero += (r.min.x*d)/8;
else
    i->zero -= (-r.min.x*d+7)/8;
i->zero = -i->zero;
```

⟨allocmemimaged() *sanity check r* 52b⟩≡ (51e)

```
if(Dx(r) <= 0 || Dy(r) <= 0){
    werrstr("bad rectangle %R", r);
    return nil;
}
```

⟨allocmemimaged() *sanity check d* 52c⟩≡ (51e)

```
if(d == 0) {
    werrstr("bad channel descriptor %.8lux", chan);
    return nil;
}
```

⟨allocmemimaged() *sanity check i* 52d⟩≡ (51e)

```
if(i == nil)
    return nil;
```

⟨*function* wordsperline 52e⟩≡ (400a)

```
int
wordsperline(Rectangle r, int d)
{
    return unitsperline(r, d, 8*sizeof(ulong));
}
```

⟨*function* bytesperline 52f⟩≡ (400a)

```
int
bytesperline(Rectangle r, int d)
{
    return unitsperline(r, d, 8);
}
```

⟨*function* unitsperline 52g⟩≡ (400a)

```
static
int
unitsperline(Rectangle r, int d, int bitsperunit)
{
    ulong l, t;

    ⟨unitsperline() sanity check d 52h⟩

    if(r.min.x >= 0){
        l = (r.max.x*d+bitsperunit-1)/bitsperunit;
        l -= (r.min.x*d)/bitsperunit;
    }else{   /* make positive before divide */
        t = (-r.min.x*d+bitsperunit-1)/bitsperunit;
        l = t+(r.max.x*d+bitsperunit-1)/bitsperunit;
    }
    return l;
}
```

⟨unitsperline() *sanity check d* 52h⟩≡ (52g)

```
if(d <= 0 || d > 32) /* being called wrong.  d is image depth. */
    abort();
```

⟨*function* memsetchan 53a⟩≡ (422b)

```
errorneg1
memsetchan(Memimage *i, ulong chan)
{
    int d;
    int t, j, k;
    ulong cc;
    bool bytes;

    d = chantodepth(chan);
    ⟨memsetchan() sanity check d 53b⟩
    i->chan = chan;
    i->depth = d;
    i->flags &= ~(Fgrey|Falpha|Fcmap|Fbytes);
    bytes = true;

    for(cc=chan, j=0, k=0; cc; j+=NBITS(cc), cc>>=8, k++){
        t=TYPE(cc);
        ⟨memsetchan() sanity check t 53c⟩
        if(t == CGrey)
            i->flags |= Fgrey;
        if(t == CAlpha)
            i->flags |= Falpha;
        if(t == CMap && i->cmap == nil){
            i->cmap = memdefcmap;
            i->flags |= Fcmap;
        }

        i->shift[t] = j;
        i->mask[t] = (1<<NBITS(cc))-1;
        i->nbits[t] = NBITS(cc);

        if(NBITS(cc) != 8)
            bytes = false;
    }
    i->nchan = k;
    if(bytes)
        i->flags |= Fbytes;
    return OK_0;
}
```

⟨memsetchan() *sanity check d* 53b⟩≡ (53a)

```
if(d == 0) {
    werrstr("bad channel descriptor");
    return ERROR_NEG1;
}
```

⟨memsetchan() *sanity check t* 53c⟩≡ (53a)

```
if(t < 0 || t >= NChan){
    werrstr("bad channel string");
    return -1;
}
```

⟨MemImage *other fields* 53d⟩+≡ (37e) ◁39a 247h▷

```
// map<enum<ImageChan>, int>
int  shift[NChan];
// map<enum<ImageChan>, int>
int  mask[NChan];
// map<enum<ImageChan>, int>
int  nbits[NChan];
```

⟨*function* freememimage 54a⟩≡ (422c)

```
void
freememimage(Memimage *i)
{
    ⟨freememimage() sanity check i 54b⟩
    // free the Memdata
    if(i->data->ref-- == 1 && i->data->allocd){
        if(i->data->base)
            poolfree(imagmem, i->data->base);
        free(i->data);
    }
    free(i);
}
```

⟨freememimage() *sanity check i* 54b⟩≡ (54a)

```
if(i == nil)
    return;
```

DImage

⟨makescreenimage() *allocate DImage di* 54c⟩≡ (50b)

```
// allocate DImage
di = allocdimage(i);
⟨makescreenimage() sanity check di 54d⟩
```

⟨makescreenimage() *sanity check di* 54d⟩≡ (54c)

```
if(di == nil){
    freememimage(i);    /* frees md */
    return nil;
}
```

Screen name

VGA attachscreen() (x86)

⟨*function* attachscreen*(x86)* 54e⟩≡ (492)

```
byte*
arch_attachscreen(Rectangle* r, ulong* chan, int* d, int* width, bool *softscreen)
{

    ⟨attachscreen() sanity check gscreen 54f⟩

    *r          = gscreen->clipr;
    *chan       = gscreen->chan;
    *d          = gscreen->depth;
    *width      = gscreen->width;

    *softscreen = vgascreen.useflush;

    return gscreendata.bdata;
}
```

⟨attachscreen() *sanity check gscreen* 54f⟩≡ (54e)

```
if(gscreen == nil || gscreendata.bdata == nil)
    return nil;
```

4.1.4 Next steps

4.2 Client side

4.2.1 initdraw()

⟨*function* initdraw 55a⟩≡ (400d)

```
errorneg1
initdraw(Errorfn error, char *fontname , char *label)
{

    if(access("/dev/draw/new", AEXIST)<0 && bind("#i", "/dev", MAFTER)<0){
        fprint(2, "imageinit: can't bind /dev/draw: %r\n");
        return ERROR_NEG1;
    }
    return geninitdraw("/dev", error, fontname, label, "/dev", Refnone);
}
```

⟨*function* geninitdraw 55b⟩≡ (400d)

```
errorneg1
geninitdraw(char *devdir, Errorfn error, char *fontname, char *label, char *windir, int ref)
{
    ⟨geninitdraw() locals 139e⟩

    display = initdisplay(devdir, windir, error);
    ⟨geninitdraw() sanity check display 55c⟩

    ⟨geninitdraw() set up font 139f⟩
    ⟨geninitdraw() write new label 235a⟩
    ⟨geninitdraw() get window 232c⟩
    atexit(drawshutdown);

    return OK_1;
}
```

⟨geninitdraw() *sanity check display* 55c⟩≡ (55b)

```
if(display == nil)
    return ERROR_NEG1;
```

4.2.2 initdisplay()

⟨initdisplay() *locals* 55d⟩≡ (56) 57b ▷

```
fdt ctlfd;
fdt datafd;
fdt reffd;

char info[NINFO+1];
char buf[128];

Display *disp;
Image *image;

int n;
```

⟨*constant* NINFO 55e⟩≡ (400d)

```
#define NINFO 12*12
```

⟨*function* initdisplay 56⟩≡ (400d)

```
Display*
initdisplay(char *dev, char *win, Errorfn error)
{
    ⟨initdisplay() locals 55d⟩

    ⟨initdisplay() install dumpers 349d⟩
    ⟨initdisplay() sanity check arguments 57c⟩

    sprint(buf, "%s/draw/new", dev);
    ctlfd = open(buf, ORDWR|OCEXEC);
    ⟨initdisplay() sanity check ctlfd 57d⟩

    n=read(ctlfd, info, sizeof info);
    ⟨initdisplay() sanity check read ctlfd 57e⟩
    info[n] = '\0';

    sprint(buf, "%s/draw/%d/data", dev, atoi(info+0*12));
    datafd = open(buf, ORDWR|OCEXEC);
    ⟨initdisplay() sanity check datafd 58a⟩

    sprint(buf, "%s/draw/%d/refresh", dev, atoi(info+0*12));
    reffd = open(buf, OREAD|OCEXEC);
    ⟨initdisplay() sanity check reffd 58b⟩

    // our display!
    disp = mallocz(sizeof(Display), true);
    ⟨initdisplay() sanity check disp 58c⟩
    disp->dirno = atoi(info+0*12);

    image = nil;
    ⟨initdisplay() sanity check image part1 58d⟩
    if(n >= NINFO){
        image = mallocz(sizeof(Image), true);
        ⟨initdisplay() sanity check image part2 58e⟩

        image->display = disp;
        image->id = 0; // info+1*12 but should always be 0
        image->chan = strtochan(info+2*12);
        image->depth = chantodepth(image->chan);
        image->repl = atoi(info+3*12);

        image->r.min.x = atoi(info+4*12);
        image->r.min.y = atoi(info+5*12);
        image->r.max.x = atoi(info+6*12);
        image->r.max.y = atoi(info+7*12);

        image->clipr.min.x = atoi(info+8*12);
        image->clipr.min.y = atoi(info+9*12);
        image->clipr.max.x = atoi(info+10*12);
        image->clipr.max.y = atoi(info+11*12);
    }
    disp->image = image;
    disp->fd    = datafd;
    disp->ctlfd = ctlfd;
    disp->reffd = reffd;

    ⟨initdisplay() set display bufsize 57a⟩
    disp->buf = malloc(disp->bufsize+5); /* +5 for flush message */
    ⟨initdisplay() sanity check buf 58g⟩
```

```
    disp->bufp = disp->buf;

    disp->error = error;
    disp->windir = t;
    disp->devdir = strdup(dev);

    qlock(&disp->qlock); // released in closedisplay (why not earlier? first API call?)

    // first API calls!
    disp->white = allocimage(disp, Rect(0, 0, 1, 1), GREY1, true, DWhite);
    disp->black = allocimage(disp, Rect(0, 0, 1, 1), GREY1, true, DBlack);
    ⟨initdisplay() sanity check white and black 58h⟩
    disp->opaque = disp->white;
    disp->transparent = disp->black;

    return disp;
}
```

⟨initdisplay() *set display bufsize* 57a⟩≡ (56)

```
disp->bufsize = iounit(datafd);
if(disp->bufsize <= 0)
    disp->bufsize = 8000;
⟨initdisplay() sanity check bufsize 58f⟩
```

⟨initdisplay() *locals* 57b⟩+≡ (56) ◁55d

```
char *t;
```

⟨initdisplay() *sanity check arguments* 57c⟩≡ (56)

```
if(dev == nil)
    dev = "/dev";
if(win == nil)
    win = "/dev";

if(strlen(dev)>sizeof buf-25 || strlen(win)>sizeof buf-25){
    werrstr("initdisplay: directory name too long");
    return nil;
}
t = strdup(win);
if(t == nil)
    return nil;
```

⟨initdisplay() *sanity check ctlfd* 57d⟩≡ (56)

```
if(ctlfd < 0){
    if(bind("#i", dev, MAFTER) < 0){
Error1:
        free(t);
        werrstr("initdisplay: %s: %r", buf);
        return nil;
    }
    // try again
    ctlfd = open(buf, ORDWR|OCEXEC);
}
if(ctlfd < 0)
    goto Error1;
```

⟨initdisplay() *sanity check read ctlfd* 57e⟩≡ (56)

```
if(n < 12){
Error2:
    close(ctlfd);
    goto Error1;
```

```
  }
  if(n==NINFO+1)
      n = NINFO;
```

⟨initdisplay() *sanity check datafd* 58a⟩≡ (56)

```
  if(datafd < 0)
      goto Error2;
```

⟨initdisplay() *sanity check reffd* 58b⟩≡ (56)

```
  if(reffd < 0){
  Error3:
      close(datafd);
      goto Error2;
  }
```

⟨initdisplay() *sanity check disp* 58c⟩≡ (56)

```
  if(disp == nil){
  Error4:
      close(reffd);
      goto Error3;
  }
```

⟨initdisplay() *sanity check image part1* 58d⟩≡ (56)

```
  if(0){
  Error5:
      free(image);
      free(disp);
      goto Error4;
  }
```

⟨initdisplay() *sanity check image part2* 58e⟩≡ (56)

```
  if(image == nil)
      goto Error5;
```

⟨initdisplay() *sanity check bufsize* 58f⟩≡ (57a)

```
  if(disp->bufsize < 512){
      werrstr("iounit %d too small", disp->bufsize);
      goto Error5;
  }
```

⟨initdisplay() *sanity check buf* 58g⟩≡ (56)

```
  if(disp->buf == nil)
      goto Error5;
```

⟨initdisplay() *sanity check white and black* 58h⟩≡ (56)

```
  if(disp->white == nil || disp->black == nil){
      free(disp->devdir);
      free(disp->white);
      free(disp->black);
      goto Error5;
  }
```

4.2.3 closedisplay()

⟨*function* closedisplay 59a⟩≡ (400d)

```
/*
 * Call with d unlocked.
 * Note that disp->defaultfont and defaultsubfont are not freed here.
 */
void
closedisplay(Display *disp)
{
    _closedisplay(disp, false);
}
```

⟨*function* drawshutdown 59b⟩≡ (400d)

```
/* note handler */
static void
drawshutdown(void)
{
    Display *d;

    d = display;
    if(d){
        display = nil;
        _closedisplay(d, true);
    }
}
```

⟨*function* _closedisplay 59c⟩≡ (400d)

```
static void
_closedisplay(Display *disp, bool isshutdown)
{
    fdt fd;
    char buf[128];

    ⟨_closedisplay() sanity check disp 60⟩
    if(disp == display)
        display = nil;

    ⟨_closedisplay() restore oldlabel 235c⟩

    /*
     * if we're shutting down, don't free all the resources.
     * if other procs are getting shot down by notes too,
     * one might get shot down while holding the malloc lock.
     * just let the kernel clean things up when we exit.
     */
    if(isshutdown)
        return;

    free(disp->devdir);
    free(disp->windir);
    freeimage(disp->white);
    freeimage(disp->black);
    close(disp->fd);
    close(disp->ctlfd);
    /* should cause refresh slave to shut down */
    close(disp->reffd);

    qunlock(&disp->qlock);
    free(disp);
}
```

⟨_closedisplay() *sanity check disp* 60⟩≡ (59c)

```
if(disp == nil)
    return;
```

4.2.4 Next steps

4.3 Fonts and overlapping layers

Chapter 5

User/Kernel Bridge

5.1 The draw filesystem

⟨*global* drawdevtab 61a⟩≡ (481)

```
Dev drawdevtab = {
    .dc       =    'i',
    .name     =    "draw",

    .attach   =    drawattach,
    .walk     =    drawwalk,
    .stat     =    drawstat,

    .open     =    drawopen,
    .close    =    drawclose,
    .read     =    drawread,
    .write    =    drawwrite,

    // generic
    .create   =    devcreate,
    .remove   =    devremove,
    .bread    =    devbread,
    .bwrite   =    devbwrite,
    .wstat    =    devwstat,
    .reset    =    devreset,
    .init     =    devinit,
    .shutdown =    devshutdown,
};
```

⟨*enum* QxxxDraw 61b⟩≡ (481)

```
enum
{
    // Directories

    Qtopdir     = 0, // /dev
    Q2nd,            // /dev/draw/
    Q3rd,            // /dev/draw/x/

    // Files

    Qnew,
    Qwinname,

    // /dev/draw/x/y, third level device files associated to a client
    Qctl,  // used in 'x < Qctl' code, so must be the first!
    Qdata, // used for drawing commands, see drawmesg()
```

```
    Qcolormap,
    Qrefresh,
};
```

⟨*constant* QSHIFT 62a⟩≡ (481)

```
/*
 * Qid path is:
 *   4 bits of file type (qids above)
 *  24 bits of mux slot number +1; 0 means not attached to client
 */
#define QSHIFT  4   /* location in qid of client # */
```

⟨*function QID bis* 62b⟩≡ (481)

```
#define QID(q)      ((((ulong)(q).path)&0x0000000F)>>0)
```

⟨*function* CLIENTPATH 62c⟩≡ (481)

```
#define CLIENTPATH(q)   ((((ulong)q)&0x7FFFFFF0)>>QSHIFT)
```

5.1.1 drawwalk()

⟨*function* drawwalk 62d⟩≡ (481)

```
static Walkqid*
drawwalk(Chan *c, Chan *nc, char **name, int nname)
{
    ⟨drawwalk() sanity check 62e⟩
    return devwalk(c, nc, name, nname, 0, 0, drawgen);
}
```

⟨drawwalk() *sanity check* 62e⟩≡ (62d)

```
if(screenimage == nil)
    error("no frame buffer");
```

⟨*function* drawstat 62f⟩≡ (481)

```
static int
drawstat(Chan *c, uchar *db, int n)
{
    return devstat(c, db, n, 0, 0, drawgen);
}
```

⟨*function* drawgen 62g⟩≡ (481)

```
static int
drawgen(Chan *c, char*, Dirtab*, int, int s, Dir *dp)
{
    int t;
    Qid q;
    ulong path;
    Client *cl;

    q.vers = 0;

    ⟨drawgen() if dotdot 64b⟩
    // else
    t = QID(c->qid);

    /*
     * Top level directory contains the name of the device.
     */
    ⟨drawgen() toplevel directory listing 63a⟩
```

```
    // else
    /*
     * Second level contains "new" plus all the clients.
     */
    ⟨drawgen() second level directory listing 63b⟩
    // else
    /*
     * Third level.
     */
    ⟨drawgen() third level directory listing 64a⟩
}
```

⟨drawgen() *toplevel directory listing* 63a⟩≡ (62g)

```
if(t == Qtopdir){
    switch(s){
    case 0:
        mkqid(&q, Q2nd, 0, QTDIR);
        devdir(c, q, "draw", 0, eve, 0555, dp);
        break;
    case 1:
        mkqid(&q, Qwinname, 0, QTFILE);
        devdir(c, q, "winname", 0, eve, 0444, dp);
        break;
    default:
        return -1;
    }
    return 1;
}
// else

if(t == Qwinname){
    mkqid(&q, Qwinname, 0, QTFILE);
    devdir(c, q, "winname", 0, eve, 0444, dp);
    return 1;
}
```

⟨drawgen() *second level directory listing* 63b⟩≡ (62g)

```
if(t == Q2nd || t == Qnew){
    if(s == 0){
        mkqid(&q, Qnew, 0, QTFILE);
        devdir(c, q, "new", 0, eve, 0666, dp);
    }
    else if(s <= sdraw.nclient){
        cl = sdraw.client[s-1];
        ⟨drawgen() in second level directory listing, sanity check cl 63c⟩
        snprint(up->genbuf, sizeof up->genbuf, "%d",
            cl->clientid);
        mkqid(&q, (s<<QSHIFT)|Q3rd, 0, QTDIR);
        devdir(c, q, up->genbuf, 0, eve, 0555, dp);
        return 1;
    }
    else
        return -1;
    return 1;
}
```

⟨drawgen() *in second level directory listing, sanity check cl* 63c⟩≡ (63b)

```
if(cl == nil)
    return 0;
```

⟨drawgen() *third level directory listing* 64a⟩≡ (62g)

```
path = c->qid.path & ~((1<<QSHIFT)-1);    /* slot component */
q.vers = c->qid.vers;
q.type = QTFILE;
switch(s){
case 0:
    q.path = path|Qcolormap;
    devdir(c, q, "colormap", 0, eve, 0600, dp);
    break;
case 1:
    q.path = path|Qctl;
    devdir(c, q, "ctl", 0, eve, 0600, dp);
    break;
case 2:
    q.path = path|Qdata;
    devdir(c, q, "data", 0, eve, 0600, dp);
    break;
case 3:
    q.path = path|Qrefresh;
    devdir(c, q, "refresh", 0, eve, 0400, dp);
    break;
default:
    return -1;
}
return 1;
```

⟨drawgen() *if dotdot* 64b⟩≡ (62g)

```
if(s == DEVDOTDOT){
    switch(QID(c->qid)){
    case Qtopdir:
    case Q2nd:
        mkqid(&q, Qtopdir, 0, QTDIR);
        devdir(c, q, "#i", 0, eve, 0500, dp);
        break;
    case Q3rd:
        cl = drawclientofpath(c->qid.path);
        if(cl == nil) // when can this happen?
            strncpy(up->genbuf, "??", sizeof up->genbuf);
        else
            snprint(up->genbuf, sizeof up->genbuf,
                "%d", cl->clientid);
        mkqid(&q, Q2nd, 0, QTDIR);
        devdir(c, q, up->genbuf, 0, eve, 0500, dp);
        break;
    default:
        panic("drawwalk %llux", c->qid.path);
    }
    return 1;
}
```

⟨*function* drawclientofpath 64c⟩≡ (481)

```
Client*
drawclientofpath(ulong path)
{
    Client *cl;
    int slot;

    slot = CLIENTPATH(path);
    if(slot == 0)
        return nil;
```

```
    cl = sdraw.client[slot-1];
    if(cl == nil || cl->clientid == 0)
        return nil;
    return cl;
}
```

5.1.2 drawopen()

⟨*function* drawopen 65a⟩≡ (481)

```
static Chan*
drawopen(Chan *c, int omode)
{
    ⟨drawopen() locals 67d⟩

    if(c->qid.type & QTDIR){
        c = devopen(c, omode, 0, 0, drawgen);
        c->iounit = IOUNIT;
    }

    ⟨drawxxx() lock 65c⟩
    ⟨drawopen() if Qnew 68a⟩
    switch(QID(c->qid)){
    ⟨drawopen() switch qid cases 68b⟩
    }
    ⟨drawxxx() unlock 65d⟩

    c->mode = openmode(omode);
    c->flag |= COPEN;
    c->offset = 0;
    c->iounit = IOUNIT;

    return c;
}
```

⟨*constant* IOUNIT 65b⟩≡ (481)

```
#define IOUNIT      (64*1024)
```

⟨drawxxx() *lock* 65c⟩≡ (66d 65)

```
dlock();
if(waserror()){
    dunlock();
    nexterror();
}
```

⟨drawxxx() *unlock* 65d⟩≡ (67a 66d 65)

```
dunlock();
poperror();
```

5.1.3 drawclose()

⟨*function* drawclose 65e⟩≡ (481)

```
static void
drawclose(Chan *c)
{
    Client *cl;
    ⟨drawclose() other locals 66b⟩

    if(QID(c->qid) < Qctl) /* Qtopdir, Qnew, Q3rd, Q2nd have no client */
```

```
        return;

    ⟨drawxxx() lock 65c⟩

    cl = drawclient(c);
    ⟨drawclose() if Qctl 312b⟩

    if((c->flag&COPEN) && (decref(&cl->r)==0)){
        ⟨drawclose() free refresh 307b⟩
        ⟨drawclose() free names 88f⟩
        ⟨drawclose() free screens 205b⟩
        /* all screens are freed, so now we can free images */
        ⟨drawclose() free dimages 66c⟩

        sdraw.client[cl->slot] = nil;

        drawflush();    /* to erase visible, now dead windows */
        free(cl);
    }
    ⟨drawxxx() unlock 65d⟩
}
```

⟨*function* drawclient 66a⟩≡ (481)

```
Client*
drawclient(Chan *c)
{
    Client *client;

    client = drawclientofpath(c->qid.path);
    if(client == nil)
        error(Enoclient);
    return client;
}
```

⟨drawclose() *other locals* 66b⟩≡ (65e) 307a ▷

```
DImage *d, **dp;
int i;
```

⟨drawclose() *free dimages* 66c⟩≡ (65e)

```
dp = cl->dimage;
for(i=0; i<NHASH; i++){
    while((d = *dp) != nil){
        *dp = d->next;
        drawfreedimage(d);
    }
    dp++;
}
```

5.1.4 drawread()

⟨*function* drawread 66d⟩≡ (481)

```
long
drawread(Chan *c, void *a, long n, vlong off)
{
    Client *cl;
    ⟨drawread() other locals 68d⟩

    if(c->qid.type & QTDIR)
        return devdirread(c, a, n, 0, 0, drawgen);
```

```
    ⟨drawread() if Qwinname 234f⟩
    // else

    cl = drawclient(c);

    ⟨drawxxx() lock 65c⟩
    switch(QID(c->qid)){
    ⟨drawread() switch qid cases 68e⟩
    }
    ⟨drawxxx() unlock 65d⟩

    return n;
}
```

5.1.5 drawwrite()

⟨*function* drawwrite 67a⟩≡ (481)

```
static long
drawwrite(Chan *c, void *a, long n, vlong)
{
    Client *cl;
    ⟨drawwrite() other locals 252c⟩

    ⟨drawwrite() sanity check c 67b⟩
    cl = drawclient(c);

    ⟨drawwrite() lock 67c⟩
    switch(QID(c->qid)){
    ⟨drawwrite() switch qid cases 68g⟩
    default:
        error(Ebadusefd);
    }
    ⟨drawxxx() unlock 65d⟩

    return n;
}
```

⟨drawwrite() *sanity check c* 67b⟩≡ (67a)

```
if(c->qid.type & QTDIR)
    error(Eisdir);
```

⟨drawwrite() *lock* 67c⟩≡ (67a)

```
dlock();
if(waserror()){

    drawwakeall();

    dunlock();
    nexterror();
}
```

5.2 /dev/draw/ files

5.2.1 Opening /dev/draw/new

⟨drawopen() *locals* 67d⟩≡ (65a) 311e ▷

```
Client *cl;
```

⟨drawopen() *if Qnew* 68a⟩≡ (65a)

```
if(QID(c->qid) == Qnew){
    cl = drawnewclient();
    ⟨drawopen() when Qnew, sanity check cl 68c⟩
    c->qid.path = Qctl|((cl->slot+1)<<QSHIFT);
}
```

⟨drawopen() *switch qid cases* 68b⟩≡ (65a) 69d ▷

```
case Qnew:
    break;
```

⟨drawopen() *when Qnew, sanity check cl* 68c⟩≡ (68a)

```
if(cl == nil)
    error(Enodev);
```

5.2.2 Reading from /dev/draw/x/ctl

⟨drawread() *other locals* 68d⟩≡ (66d) 68h ▷

```
Memimage *i;
char buf[16];
```

⟨drawread() *switch qid cases* 68e⟩≡ (66d) 178e ▷

```
case Qctl:
    if(n < 12*12) // NINFO
        error(Eshortread);

    ⟨drawread() switch qid cases, when Qctl, set i 68i⟩

    // mostly for initdisplay!
    n = snprint(a, n,
        "%11d %11d %11s %11d %11d %11d %11d %11d %11d %11d %11d %11d ",
        cl->clientid, cl->infoid, chantostr(buf, i->chan),
        (i->flags&Frepl)==Frepl,
        i->r.min.x, i->r.min.y, i->r.max.x, i->r.max.y,
        i->clipr.min.x, i->clipr.min.y, i->clipr.max.x, i->clipr.max.y
        );
    cl->infoid = -1;
    break;
```

⟨Client *other fields* 68f⟩+≡ (35d) ◁36c 178d ▷

```
int     infoid;
```

⟨drawwrite() *switch qid cases* 68g⟩≡ (67a) 69e ▷

```
case Qctl:
    if(n != 4)
        error("unknown draw control request");
    cl->infoid = BGLONG((uchar*)a);
    break;
```

⟨drawread() *other locals* 68h⟩+≡ (66d) ◁68d 252a ▷

```
DImage *di;
```

⟨drawread() *switch qid cases, when Qctl, set i* 68i⟩≡ (68e)

```
⟨drawread() switch qid cases, when Qctl, sanity check infoid 69a⟩
if(cl->infoid == 0){
    i = screenimage;
    ⟨drawread() switch qid cases, when Qctl, sanity check i 69b⟩
}else{
    di = drawlookup(cl, cl->infoid, true);
    ⟨drawread() switch qid cases, when Qctl, sanity check di 69c⟩
    i = di->image;
}
```

⟨drawread() *switch qid cases, when Qctl, sanity check infoid* 69a⟩≡ (68i)

```
if(cl->infoid < 0)
    error(Enodrawimage);
```

⟨drawread() *switch qid cases, when Qctl, sanity check i* 69b⟩≡ (68i)

```
if(i == nil)
    error(Enodrawimage);
```

⟨drawread() *switch qid cases, when Qctl, sanity check di* 69c⟩≡ (68i)

```
if(di == nil)
    error(Enodrawimage);
```

5.2.3 Writing in /dev/draw/x/data

⟨drawopen() *switch qid cases* 69d⟩+≡ (65a) ◁68b 234e▷

```
case Qdata:
case Qcolormap:
case Qrefresh:
    cl = drawclient(c);
    incref(&cl->r);
    break;
```

⟨drawwrite() *switch qid cases* 69e⟩+≡ (67a) ◁68g 253a▷

```
case Qdata:
    // The big dispatch!!
    drawmesg(cl, a, n);

    drawwakeall();
    break;
```

5.3 The drawing commands protocol

5.3.1 Step 1: API function

5.3.2 Step 2: marshalling

⟨*function* BGSHORT 69f⟩≡ (392)

```
#define BGSHORT(p)  (((p)[0]<<0) | ((p)[1]<<8))
```

⟨*function* BGLONG 69g⟩≡ (392)

```
#define BGLONG(p)  ((BGSHORT(p)<<0) | (BGSHORT(p+2)<<16))
```

⟨*function* BPSHORT 69h⟩≡ (392)

```
#define BPSHORT(p, v)  ((p)[0]=(v), (p)[1]=((v)>>8))
```

⟨*function* BPLONG 69i⟩≡ (392)

```
#define BPLONG(p, v)  (BPSHORT(p, (v)), BPSHORT(p+2, (v)>>16))
```

⟨Display *buffer fields* 69j⟩≡ (34e)

```
// drawing operations to write in /dev/draw/x/data until flush
// array<byte> (length = Display.bufsize)
byte    *buf;
int     bufsize;
// index in Display.buf array
byte    *bufp;
```

⟨*function* bufimage 70a⟩≡ (401c)

```
byte*
bufimage(Display *d, int n)
{
    byte *p;

    ⟨bufimage() sanity check n 70c⟩
    ⟨bufimage() flush if buffer full 70b⟩
    p = d->bufp;
    d->bufp += n;
    return p;
}
```

⟨bufimage() *flush if buffer full* 70b⟩≡ (70a)

```
if(d->bufp + n  >  d->buf + d->bufsize)
    if(doflush(d) < 0)
        return nil;
```

⟨bufimage() *sanity check n* 70c⟩≡ (70a)

```
if(n < 0 || n > d->bufsize){
    werrstr("bad count in bufimage");
    return nil;
}
```

⟨*function* doflush 70d⟩≡ (401c)

```
static
errorneg1
doflush(Display *d)
{
    int n, nn;

    n = d->bufp - d->buf;
    if(n <= 0)
        return OK_1; // warning?

    nn=write(d->fd, d->buf, n);
    ⟨doflush() sanity check nn 70e⟩
    d->bufp = d->buf;
    return OK_1;
}
```

⟨doflush() *sanity check nn* 70e⟩≡ (70d)

```
if(nn != n){
    ⟨doflush() if _drawdebug 348b⟩
    d->bufp = d->buf; /* might as well; chance of continuing */
    return ERROR_NEG1;
}
```

5.3.3 Step 3: unmarshalling and drawmesg()

⟨drawmesg() *locals* 70f⟩≡ (71) 75b ▷

```
byte *a;
int m = 0;
char *fmt = nil;
```

⟨*function* drawmesg 71⟩≡ (480a)

```
void
drawmesg(Client *client, void *av, int n)
{
    ⟨drawmesg() locals 70f⟩

    a = av;
    if(waserror()){
        if(fmt)
            printmesg(fmt, a, false);
        nexterror();
    }
    while((n-=m) > 0){
        USED(fmt);
        a += m;
        // dispatch on first letter of the drawing command
        switch(*a){
        ⟨drawmesg() cases 75c⟩
        default:
            error("bad draw command");
        }
    }
    poperror();
}
```

5.3.4 Step 4: algorithm

5.3.5 Step 5 (optional): hardware support hook

Chapter 6

Images

6.1 Allocation

6.1.1 API

⟨*function* allocimage 72a⟩≡ (402a)

```
Image*
allocimage(Display *d, Rectangle r, ulong chan, bool repl, rgba val)
{
    Image* i;

    i =  _allocimage(nil, d, r, chan, repl, val, 0, 0);
    ⟨allocimage() set malloc tag for debug 351a⟩
    return i;
}
```

⟨*function* freeimage 72b⟩≡ (402a)

```
errorneg1
freeimage(Image *i)
{
    errorneg1 ret;

    ret = _freeimage1(i);
    free(i);
    return ret;
}
```

6.1.2 Marshalling

⟨*function* _allocimage 72c⟩≡ (402a)

```
Image*
_allocimage(Image *ai, Display *d, Rectangle r, ulong chan, bool repl, rgba val, int screenid, int refresh)
{
    Image *i = nil;
    char *err = nil;
    byte *a;
    Rectangle clipr;
    int id;
    int depth;

    ⟨_allocimage() sanity check chan 73b⟩
    depth = chantodepth(chan);
    ⟨_allocimage() sanity check depth 74a⟩
```

```
    /* flush pending data so we don't get error allocating the image */
    flushimage(d, false);

    // new allocate: 'b' id[4] screenid[4] refresh[1] chan[4] repl[1] R[4*4] clipR[4*4] rrggbbaa[4]
    a = bufimage(d, 1+4+4+1+4+1+4*4+4*4+4);
    ⟨_allocimage() sanity check a 74b⟩

    d->imageid++;
    id = d->imageid;

    a[0] = 'b';
    BPLONG(a+1, id);

    BPLONG(a+5, screenid);
    a[9] = refresh;

    BPLONG(a+10, chan);
    a[14] = repl;
    BPLONG(a+15, r.min.x);
    BPLONG(a+19, r.min.y);
    BPLONG(a+23, r.max.x);
    BPLONG(a+27, r.max.y);
    ⟨_allocimage() set clipr 79a⟩
    BPLONG(a+31, clipr.min.x);
    BPLONG(a+35, clipr.min.y);
    BPLONG(a+39, clipr.max.x);
    BPLONG(a+43, clipr.max.y);
    BPLONG(a+47, val);

    if(flushimage(d, false) < 0)
        goto Error;

    ⟨_allocimage() if passed image 73a⟩
    else{
        i = malloc(sizeof(Image)); // client side allocation
        ⟨_allocimage() sanity check i 74c⟩
    }

    i->display = d;
    i->id = id;
    i->depth = depth;
    i->chan = chan;
    i->r = r;
    i->clipr = clipr;
    i->repl = repl;

    i->screen = nil;
    i->next = nil;

    return i;
}
```

⟨_allocimage() *if passed image* 73a⟩≡ (72c)

```
if(ai)
    i = ai;
```

⟨_allocimage() *sanity check chan* 73b⟩≡ (72c)

```
if(chan == 0){
    werrstr("bad channel descriptor");
    return nil;
}
```

⟨_allocimage() *sanity check depth* 74a⟩≡ (72c)

```
if(depth == 0){
    err = "bad channel descriptor";
Error:
    if(err)
        werrstr("allocimage: %s", err);
    else
        werrstr("allocimage: %r");
    free(i);
    return nil;
}
```

⟨_allocimage() *sanity check a* 74b⟩≡ (72c)

```
if(a == nil)
    goto Error;
```

⟨_allocimage() *sanity check i* 74c⟩≡ (72c)

```
if(i == nil){
    // free: 'f' id[4]
    a = bufimage(d, 1+4);
    if(a){
        a[0] = 'f';
        BPLONG(a+1, id);
        flushimage(d, false);
    }
    goto Error;
}
```

⟨*function* _freeimage1 74d⟩≡ (402a)

```
errorneg1
_freeimage1(Image *i)
{
    byte *a;
    Display *d;
    Image *w;

    ⟨_freeimage1() sanity check i 74e⟩

    /* make sure no refresh events occur on this if we block in the write */
    d = i->display;
    /* flush pending data so we don't get error deleting the image */
    flushimage(d, false);

    // free: 'f' id[4]
    a = bufimage(d, 1+4);
    ⟨_freeimage1() sanity check a 75a⟩
    a[0] = 'f';
    BPLONG(a+1, i->id);
    ⟨_freeimage1() if screen 206e⟩

    if(flushimage(d, i->screen != nil) < 0)
        return ERROR_NEG1;

    return OK_0;
}
```

⟨_freeimage1() *sanity check i* 74e⟩≡ (74d)

```
if(i == nil || i->display == nil)
    return OK_0;
```

⟨_freeimage1() *sanity check a* 75a⟩≡ (74d)

```
if(a == nil)
    return ERROR_NEG1;
```

6.1.3 Unmarshalling

⟨drawmesg() *locals* 75b⟩+≡ (71) ◁70f 76c▷

```
int dstid, scrnid;
byte refresh;
ulong chan;
int repl;
Rectangle r, clipr;
// rgba
ulong value;

Memimage *i;
```

⟨drawmesg() *cases* 75c⟩≡ (71) 76d▷

```
/* new allocate: 'b' id[4] screenid[4] refresh[1] chan[4] repl[1] R[4*4] clipR[4*4] rrggbbaa[4] */
case 'b':
    printmesg(fmt="LLbLbRRL", a, false);
    m = 1+4+4+1+4+1+4*4+4*4+4;
     ⟨drawmesg() sanity check n with m 75d⟩
    dstid = BGLONG(a+1);

    scrnid = BGSHORT(a+5);
    refresh = a[9];

    chan = BGLONG(a+10);
    repl = a[14];
    drawrectangle(&r, a+15);
    drawrectangle(&clipr, a+31);
    value = BGLONG(a+47);

    ⟨drawmesg() allocate image case, sanity check dstid 76a⟩
    ⟨drawmesg() allocate image case, if screen id 207b⟩
    // else

    i = allocmemimage(r, chan); // server side allocation
    ⟨drawmesg() allocate image case, sanity check i 75e⟩
    if(repl)
        i->flags |= Frepl;
    i->clipr = clipr;
    ⟨drawmesg() allocate image case, clip clipr 79b⟩

    if(drawinstall(client, dstid, i, nil) == nil){
        freememimage(i);
        error(Edrawmem);
    }
    memfillcolor(i, value);
    continue;
```

⟨drawmesg() *sanity check n with m* 75d⟩≡ (349a 218c 212 203 202g 177g 174g 163d 160d 151 129c 120a 114c 92d 91e 85 80b 76d 75c)

```
if(n < m)
    error(Eshortdraw);
```

⟨drawmesg() *allocate image case, sanity check i* 75e⟩≡ (75c)

```
if(i == nil)
    error(Edrawmem);
```

⟨drawmesg() *allocate image case, sanity check dstid* 76a⟩≡ (75c)

```
if(drawlookup(client, dstid, false))
    error(Eimageexists);
```

⟨*function* drawrectangle 76b⟩≡ (480a)

```
void
drawrectangle(Rectangle *r, uchar *a)
{
    r->min.x = BGLONG(a+0*4);
    r->min.y = BGLONG(a+1*4);
    r->max.x = BGLONG(a+2*4);
    r->max.y = BGLONG(a+3*4);
}
```

⟨drawmesg() *locals* 76c⟩+≡ (71) ◁75b 80a▷

```
DImage *ll;
```

⟨drawmesg() *cases* 76d⟩+≡ (71) ◁75c 80b▷

```
/* free: 'f' id[4] */
case 'f':
    printmesg(fmt="L", a, false);
    m = 1+4;
    ⟨drawmesg() sanity check n with m 75d⟩
    ll = drawlookup(client, BGLONG(a+1), false);
    ⟨drawmesg() free image case, if dscreen 308f⟩
    drawuninstall(client, BGLONG(a+1)); // The call

    continue;
```

6.1.4 Algorithm

⟨*function* allocmemimage 76e⟩≡ (422c)

```
Memimage*
allocmemimage(Rectangle r, ulong chan)
{
    int d;
    byte *p;
    ulong l, nw;
    Memdata *md;
    Memimage *i;

    d = chantodepth(chan);
    ⟨allocmemimage() sanity check d 77a⟩

    l = wordsperline(r, d);
    nw = l * Dy(r);

    md = malloc(sizeof(Memdata));
    ⟨allocmemimage() sanity check md 77b⟩
    md->ref = 1;
    // the big alloc!
    md->base = poolalloc(imagmem, sizeof(Memdata*)+(1+nw)*sizeof(ulong));
    ⟨allocmemimage() sanity check md base 77c⟩

    p = (byte*)md->base;
    *(Memdata**)p = md;
    p += sizeof(Memdata*);
    *(ulong*)p = getcallerpc(&r);
    p += sizeof(ulong);
```

```
    /* if this changes, memimagemove must change too */
    md->bdata = p;
    md->allocd = true;

    i = allocmemimaged(r, chan, md);
    ⟨allocmemimage() sanity check i 77d⟩

    return i;
}
```

⟨allocmemimage() *sanity check d* 77a⟩≡ (76e)

```
if(d == 0) {
    werrstr("bad channel descriptor %.8lux", chan);
    return nil;
}
```

⟨allocmemimage() *sanity check md* 77b⟩≡ (76e)

```
if(md == nil)
    return nil;
```

⟨allocmemimage() *sanity check md base* 77c⟩≡ (76e)

```
if(md->base == nil){
    free(md);
    return nil;
}
```

⟨allocmemimage() *sanity check i* 77d⟩≡ (76e)

```
if(i == nil){
    poolfree(imagmem, md->base);
    free(md);
    return nil;
}
```

6.1.5 Client and Memimage connection, DImage

⟨*function* drawinstall 77e⟩≡ (478b)

```
Memimage*
drawinstall(Client *client, int id, Memimage *i, DScreen *dscreen)
{
    DImage *d;

    d = allocdimage(i);
    ⟨drawinstall() sanity check d 77f⟩
    d->id = id;
    ⟨drawinstall() install dscreen 210b⟩

    // insert_hash(d, client->dimage)
    d->next = client->dimage[id&HASHMASK];
    client->dimage[id&HASHMASK] = d;

    return i;
}
```

⟨drawinstall() *sanity check d* 77f⟩≡ (77e)

```
if(d == nil)
    return nil;
```

⟨*function* drawuninstall 78a⟩≡ (478b)

```
void
drawuninstall(Client *client, int id)
{
    DImage *d, *next;

    d = client->dimage[id&HASHMASK];
    ⟨drawuninstall() sanity check d 78b⟩

    // hash_remove(client->dimage, id)
    if(d->id == id){
        client->dimage[id&HASHMASK] = d->next;

        drawfreedimage(d);
        return;
    }
    while(next = d->next){  /* assign = */
        if(next->id == id){
            d->next = next->next;

            drawfreedimage(next);
            return;
        }
        d = next;
    }

    error(Enodrawimage);
}
```

⟨drawuninstall() *sanity check d* 78b⟩≡ (78a)

```
if(d == nil)
    error(Enodrawimage);
```

⟨*function* drawfreedimage 78c⟩≡ (478b)

```
void
drawfreedimage(DImage *dimage)
{
    int i;
    Memimage *l;
    DScreen *ds;

    dimage->ref--;
    ⟨drawfreedimage() sanity check dimage ref 78d⟩
    if(dimage->ref > 0)
        return;
    // else

    ⟨drawfreedimage() free names 88g⟩
    ⟨drawfreedimage() if dscreen 210c⟩
    else
        freememimage(dimage->image);
    Return:
    free(dimage->fchar);
    free(dimage);
}
```

⟨drawfreedimage() *sanity check dimage ref* 78d⟩≡ (78c)

```
if(dimage->ref < 0)
    print("negative ref in drawfreedimage\n");
```

6.2 Clipping and replicating

⟨_allocimage() *set clipr* 79a⟩≡ (72c)

```
if(repl)
    /* huge but not infinite, so various offsets will leave it huge, not overflow */
    clipr = Rect(-0x3FFFFFFF, -0x3FFFFFFF, 0x3FFFFFFF, 0x3FFFFFFF);
else
    clipr = r;
```

⟨drawmesg() *allocate image case, clip clipr* 79b⟩≡ (75c)

```
if(!repl)
    rectclip(&i->clipr, r);
```

⟨*function* rectclip 79c⟩≡ (413a)

```
bool
rectclip(Rectangle *rp, Rectangle b) /* first by reference, second by value */
{
    Rectangle *bp = &b;
    /*
     * Expand rectXrect() in line for speed
     */
    if(!(rp->min.x < bp->max.x && bp->min.x < rp->max.x &&
        rp->min.y < bp->max.y && bp->min.y < rp->max.y))
        return false;

    /* They must overlap */
    if(rp->min.x < bp->min.x)
        rp->min.x = bp->min.x;
    if(rp->min.y < bp->min.y)
        rp->min.y = bp->min.y;
    if(rp->max.x > bp->max.x)
        rp->max.x = bp->max.x;
    if(rp->max.y > bp->max.y)
        rp->max.y = bp->max.y;
    return true;
}
```

6.2.1 API

⟨*function* replclipr 79d⟩≡ (413a)

```
void
replclipr(Image *i, bool repl, Rectangle clipr)
{
    ⟨replclipr() body 79e⟩
}
```

6.2.2 Marshalling

⟨replclipr() *body* 79e⟩≡ (79d)

```
byte *b;

// set repl and clip: 'c' dstid[4] repl[1] clipR[4*4]
b = bufimage(i->display, 1+4+1+4*4);
b[0] = 'c';
BPLONG(b+1, i->id);
repl = (repl != 0);
b[5] = repl;
BPLONG(b+6, clipr.min.x);
```

```
BPLONG(b+10, clipr.min.y);
BPLONG(b+14, clipr.max.x);
BPLONG(b+18, clipr.max.y);
i->repl = repl;
i->clipr = clipr;
```

6.2.3 Unmarshalling

⟨drawmesg() *locals* 80a⟩+≡ (71) ◁76c 85a▷

```
DImage *ddst;
Memimage *dst;
```

⟨drawmesg() *cases* 80b⟩+≡ (71) ◁76d 81b▷

```
/* set repl and clip: 'c' dstid[4] repl[1] clipR[4*4] */
case 'c':
    printmesg(fmt="LbR", a, false);
    m = 1+4+1+4*4;
    ⟨drawmesg() sanity check n with m 75d⟩
    ddst = drawlookup(client, BGLONG(a+1), true);
    ⟨drawmesg() clipping case, sanity check ddst 80c⟩
    dst = ddst->image;

    if(a[5])
        dst->flags |= Frepl;
    drawrectangle(&dst->clipr, a+6); // The call

    continue;
```

⟨drawmesg() *clipping case, sanity check ddst* 80c⟩≡ (80b) 88e▷

```
if(ddst == nil)
    error(Enodrawimage);
```

6.2.4 Algorithm

6.3 Flushing

6.3.1 API

⟨*function* flushimage 80d⟩≡ (401c)

```
errorneg1
flushimage(Display *d, bool visible)
{
    ⟨flushimage() sanity check d 80e⟩
    ⟨flushimage() if visible 81a⟩
    return doflush(d);
}
```

⟨flushimage() *sanity check d* 80e⟩≡ (80d)

```
if(d == nil)
    return OK_0;
```

6.3.2 Marshalling

⟨flushimage() *if visible* 81a⟩≡ (80d)

```
// visible: 'v'
if(visible){
    *d->bufp++ = 'v'; /* five bytes always reserved for this */
}
```

6.3.3 Unmarshalling

⟨drawmesg() *cases* 81b⟩+≡ (71) ◁80b 85b▷

```
/* visible: 'v' */
case 'v':
    printmesg(fmt="", a, false);
    m = 1;

    drawflush(); // The call

    continue;
```

6.3.4 Algorithm

⟨*global* flushrect 81c⟩≡ (481)

```
static  Rectangle   flushrect;
```

⟨*function* drawflush 81d⟩≡ (481)

```
void
drawflush(void)
{
    if(flushrect.min.x < flushrect.max.x)
        arch_flushmemscreen(flushrect);
    flushrect = Rect(10000, 10000, -10000, -10000);
}
```

⟨*function* dstflush 81e⟩≡ (481)

```
void
dstflush(int dstid, Memimage *dst, Rectangle r)
{
    ⟨dstflush() locals 326f⟩

    if(dstid == 0){ // the screen
        combinerect(&flushrect, r);
        return;
    }
    // else
    ⟨dstflush() sanity check dst 81f⟩
    ⟨dstflush() if layer 326g⟩
}
```

⟨dstflush() *sanity check dst* 81f⟩≡ (81e)

```
/* how can this happen? -rsc, dec 12 2002 */
if(dst == nil){
    print("nil dstflush\n");
    return;
}
```

⟨*function* combinerect 82a⟩≡ (398)

```
void
combinerect(Rectangle *r1, Rectangle r2)
{
    if(r1->min.x > r2.min.x)
        r1->min.x = r2.min.x;
    if(r1->min.y > r2.min.y)
        r1->min.y = r2.min.y;
    if(r1->max.x < r2.max.x)
        r1->max.x = r2.max.x;
    if(r1->max.y < r2.max.y)
        r1->max.y = r2.max.y;
}
```

6.3.5 Hardware support hook (x86)

⟨*function* flushmemscreen*(x86)* 82b⟩≡ (492)

```
/*
 * It would be fair to say that this doesn't work for >8-bit screens.
 */
void
arch_flushmemscreen(Rectangle r)
{
    VGAscr *scr;
    ⟨flushmemscreen() other locals 377b⟩

    scr = &vgascreen;

    // call the device driver flush hook
    if(scr->dev && scr->dev->flush){
        scr->dev->flush(scr, r);
        return;
    }
    // else
    if(gscreen == nil || !scr->useflush)
        return;
    if(scr->dev == nil || scr->dev->page == nil)
        return;
    if(!rectclip(&r, gscreen->r))
        return;

    ⟨flushmemscreen() use VGA page 377c⟩
}
```

6.4 Naming and sharing

6.4.1 API

⟨*function* nameimage 82c⟩≡ (402b)

```
error0
nameimage(Image *i, char *name, bool in)
{
    ⟨nameimage() body 83b⟩
}
```

⟨*function* namedimage 83a⟩≡ (402b)

```
Image*
namedimage(Display *d, char *name)
{
    ⟨namedimage() body 83d⟩
}
```

6.4.2 Marshalling

⟨nameimage() *body* 83b⟩≡ (82c)

```
byte *a;
int n;

n = strlen(name);
// name an image: 'N' dstid[4] in[1] j[1] name[j]
a = bufimage(i->display, 1+4+1+1+n);
⟨nameimage() sanity check a 83c⟩
a[0] = 'N';
BPLONG(a+1, i->id);
a[5] = in;
a[6] = n;
memmove(a+7, name, n);

if(flushimage(i->display, false) < 0)
    return ERROR_0;
return OK_1;
```

⟨nameimage() *sanity check a* 83c⟩≡ (83b)

```
if(a == nil)
    return ERROR_0;
```

⟨namedimage() *body* 83d⟩≡ (83a)

```
byte *a;
char *err = nil;
char buf[12*12+1]; // NINFO+1
Image *i = nil;
int id, n;
ulong chan;

n = strlen(name);
⟨namedimage() sanity check n 84a⟩
/* flush pending data so we don't get error allocating the image */
flushimage(d, false);

// attach to a named image: 'n' dstid[4] j[1] name[j]
a = bufimage(d, 1+4+1+n);
⟨namedimage() sanity check a 84b⟩

d->imageid++;
id = d->imageid;

a[0] = 'n';
BPLONG(a+1, id);
a[5] = n;
memmove(a+6, name, n);

if(flushimage(d, false) < 0)
    goto Error;
```

```
if(pread(d->ctlfd, buf, sizeof buf, 0) < 12*12)
    goto Error;
buf[12*12] = '\0';

i = malloc(sizeof(Image));
⟨namedimage() sanity check i 84c⟩

i->display = d;
i->id = id;
chan=strtochan(buf+2*12);
⟨namedimage() sanity check chan 84d⟩
i->chan = chan;
i->depth = chantodepth(chan);
i->repl = atoi(buf+3*12);
i->r.min.x = atoi(buf+4*12);
i->r.min.y = atoi(buf+5*12);
i->r.max.x = atoi(buf+6*12);
i->r.max.y = atoi(buf+7*12);
i->clipr.min.x = atoi(buf+8*12);
i->clipr.min.y = atoi(buf+9*12);
i->clipr.max.x = atoi(buf+10*12);
i->clipr.max.y = atoi(buf+11*12);

i->screen = nil;
i->next = nil;

return i;
```

⟨namedimage() *sanity check n* 84a⟩≡ (83d)

```
if(n >= 256){
    err = "name too long";
Error:
    if(err)
        werrstr("namedimage: %s", err);
    else
        werrstr("namedimage: %r");
    if(i)
        free(i);
    return nil;
}
```

⟨namedimage() *sanity check a* 84b⟩≡ (83d)

```
if(a == nil)
    goto Error;
```

⟨namedimage() *sanity check i* 84c⟩≡ (83d)

```
if(i == nil){
Error1:
    a = bufimage(d, 1+4);
    if(a){
        a[0] = 'f';
        BPLONG(a+1, id);
        flushimage(d, false);
    }
    goto Error;
}
```

⟨namedimage() *sanity check chan* 84d⟩≡ (83d)

```
if(chan == 0){
    werrstr("bad channel '%.12s' from devdraw", buf+2*12);
    goto Error1;
}
```

6.4.3 Unmarshalling

⟨drawmesg() *locals* 85a⟩+≡ (71) ◁80a 91d▷

```
int c, j;
DImage *di;
DName *dn;
```

⟨drawmesg() *cases* 85b⟩+≡ (71) ◁81b 85f▷

```
/* name an image: 'N' dstid[4] in[1] j[1] name[j] */
case 'N':
    printmesg(fmt="Lbz", a, false);
    m = 1+4+1+1;
     ⟨drawmesg() sanity check n with m 75d⟩
    c = a[5];
    j = a[6];
     ⟨drawmesg() name an image case, sanity check j 85c⟩
    m += j;
     ⟨drawmesg() sanity check n with m 75d⟩

    di = drawlookup(client, BGLONG(a+1), false);
    ⟨drawmesg() name an image case, sanity check di 85d⟩

    if(c)
        drawaddname(client, di, j, (char*)a+7); // The call
    else{
        dn = drawlookupname(j, (char*)a+7);
        ⟨drawmesg() name an image case, sanity check dn 85e⟩
        drawdelname(dn);
    }
    continue;
```

⟨drawmesg() *name an image case, sanity check j* 85c⟩≡ (85)

```
if(j == 0)  /* give me a non-empty name please */
    error(Eshortdraw);
```

⟨drawmesg() *name an image case, sanity check di* 85d⟩≡ (85b)

```
if(di == nil)
    error(Enodrawimage);
if(di->name)
    error(Enamed);
```

⟨drawmesg() *name an image case, sanity check dn* 85e⟩≡ (85b)

```
if(dn == nil)
    error(Enoname);
if(dn->dimage != di)
    error(Ewrongname);
```

⟨drawmesg() *cases* 85f⟩+≡ (71) ◁85b 91e▷

```
/* attach to a named image: 'n' dstid[4] j[1] name[j] */
case 'n':
    printmesg(fmt="Lz", a, false);
    m = 1+4+1;
    ⟨drawmesg() sanity check n with m 75d⟩
    j = a[5];
    ⟨drawmesg() name an image case, sanity check j 85c⟩
    m += j;
    ⟨drawmesg() sanity check n with m 75d⟩
    dstid = BGLONG(a+1);

    if(drawlookup(client, dstid, false))
```

```
        error(Eimageexists);

    dn = drawlookupname(j, (char*)a+6); // The call
     ⟨drawmesg() attach to an image case, sanity check dn 86a⟩
    // create new DImage with shared underlying Memimage of dn
    if(drawinstall(client, dstid, dn->dimage->image, nil) == 0)
        error(Edrawmem);
    di = drawlookup(client, dstid, false);
    ⟨drawmesg() attach to an image case, sanity check di 86b⟩

    ⟨drawmesg() attach to an image case, set di name fields using dn 88a⟩

    // for further read on /dev/draw/x/ctl
    client->infoid = dstid;
    continue;
```

⟨drawmesg() *attach to an image case, sanity check dn* 86a⟩≡ (85f)

```
if(dn == nil)
    error(Enoname);
```

⟨drawmesg() *attach to an image case, sanity check di* 86b⟩≡ (85f)

```
if(di == nil)
    error("draw: cannot happen");
```

6.4.4 Algorithm

⟨KDraw *other fields* 86c⟩+≡ (36a) ◁50g 88b▷

```
// growing_array<DName>, size = KDraw.nname
DName*  name;
int     nname;
```

⟨*struct* DName 86d⟩≡ (476)

```
struct DName
{
    // key
    char        *name;

    // value
    Client      *client; // the owner
    DImage*     dimage;
    ⟨DName other fields 88c⟩
};
```

⟨*function* drawlookupname 86e⟩≡ (478a)

```
DName*
drawlookupname(int n, char *str)
{
    DName *name, *ename;

    name = sdraw.name;
    ename = &name[sdraw.nname];
    for(; name<ename; name++)
        if(drawcmp(name->name, str, n) == 0)
            return name;
    return nil;
}
```

⟨*function* drawcmp 87a⟩≡ (478a)

```
static
int
drawcmp(char *a, char *b, int n)
{
    if(strlen(a) != n)
        return 1;
    return memcmp(a, b, n);
}
```

⟨*function* drawaddname 87b⟩≡ (478a)

```
void
drawaddname(Client *client, DImage *di, int n, char *str)
{
    DName *name, *ename, *new, *t;

    name = sdraw.name;
    ename = &name[sdraw.nname];
    for(; name<ename; name++)
        if(drawcmp(name->name, str, n) == 0)
            error(Enameused);

    // grow array
    ⟨drawaddname() grow array by 1 element 87c⟩

    new = &sdraw.name[sdraw.nname++];
    new->name = smalloc(n+1);
    memmove(new->name, str, n);
    new->name[n] = '\0';

    new->dimage = di;
    new->client = client;

    new->vers = ++sdraw.vers;
}
```

⟨drawaddname() *grow array by 1 element* 87c⟩≡ (87b)

```
t = smalloc((sdraw.nname+1)*sizeof(DName));
memmove(t, sdraw.name, sdraw.nname*sizeof(DName));
free(sdraw.name);
sdraw.name = t;
```

⟨*function* drawdelname 87d⟩≡ (478a)

```
void
drawdelname(DName *name)
{
    int i;

    i =  name - sdraw.name;
    memmove(name, name+1, (sdraw.nname-(i+1))*sizeof(DName));
    sdraw.nname--;
}
```

⟨DImage *other fields* 87e⟩≡ (39f) 88d ▷

```
// option<string>, Some when DImage derives from a named image
char    *name;
// option<ref<DImage>>
DImage*     fromname;   /* image this one is derived from, by name */
```

⟨drawmesg() *attach to an image case, set di name fields using dn* 88a⟩≡ (85f)

```
di->name = smalloc(j+1);
memmove(di->name, a+6, j);
di->name[j] = '\0';
di->fromname = dn->dimage;
di->fromname->ref++;
di->vers = dn->vers;
```

⟨KDraw *other fields* 88b⟩+≡ (36a) ◁86c 304b▷

```
int     vers;
```

⟨DName *other fields* 88c⟩≡ (86d)

```
int     vers;
```

⟨DImage *other fields* 88d⟩+≡ (39f) ◁87e

```
int     vers;
```

⟨drawmesg() *clipping case, sanity check ddst* 88e⟩+≡ (80b) ◁80c

```
if(ddst->name)
    error("cannot change repl/clipr of shared image");
```

⟨drawclose() *free names* 88f⟩≡ (65e)

```
/* free names */
for(i=0; i<sdraw.nname; )
    if(sdraw.name[i].client == cl)
        drawdelname(sdraw.name+i);
    else
        i++;
```

⟨drawfreedimage() *free names* 88g⟩≡ (78c)

```
/* any names? */
for(i=0; i<sdraw.nname; )
    if(sdraw.name[i].dimage == dimage)
        drawdelname(sdraw.name+i);
    else
        i++;

if(dimage->fromname){   /* acquired by name; owned by someone else*/
    drawfreedimage(dimage->fromname);
    goto Return;
}
```

⟨*function* drawgoodname 88h⟩≡ (478a)

```
bool
drawgoodname(DImage *d)
{
    DName *n;

    ⟨drawgoodname() if DImage d is a window 210d⟩

    if(d->name == nil)
        return true;
    n = drawlookupname(strlen(d->name), d->name);
    if(n == nil || n->vers != d->vers)
        return false;
    return true;
}
```

⟨drawlookup() *if checkname* 88i⟩≡ (35h)

```
if(checkname && !drawgoodname(d))
    error(Eoldname);
```

Chapter 7

Manipulating Rectangles

7.1 draw()

7.1.1 API

⟨*function* draw 89a⟩≡ (409b)

```
void
draw(Image *dst, Rectangle r, Image *src, Image *mask, Point p1)
{
    draw1(dst, &r, src, &p1, mask, &p1, SoverD);
}
```

⟨*function* drawop 89b⟩≡ (409b)

```
void
drawop(Image *dst, Rectangle r, Image *src, Image *mask, Point p1, Drawop op)
{
    draw1(dst, &r, src, &p1, mask, &p1, op);
}
```

⟨*function* gendraw 89c⟩≡ (409b)

```
void
gendraw(Image *dst, Rectangle r, Image *src, Point p0, Image *mask, Point p1)
{
    draw1(dst, &r, src, &p0, mask, &p1, SoverD);
}
```

⟨*function* gendrawop 89d⟩≡ (409b)

```
void
gendrawop(Image *dst, Rectangle r, Image *src, Point p0, Image *mask, Point p1, Drawop op)
{
    draw1(dst, &r, src, &p0, mask, &p1, op);
}
```

⟨*function* border 89e⟩≡ (409b)

```
void
border(Image *im, Rectangle r, int i, Image *color, Point sp)
{
    ⟨border() if negative i, border goes outside 90a⟩
    // horizontal bars
    draw(im, Rect(r.min.x, r.min.y,   r.max.x, r.min.y+i),
        color, nil, sp);
    draw(im, Rect(r.min.x, r.max.y-i,   r.max.x, r.max.y),
        color, nil, Pt(sp.x, sp.y+Dy(r)-i));
    // vertical bars
    draw(im, Rect(r.min.x, r.min.y+i,   r.min.x+i, r.max.y-i),
```

```
        color, nil, Pt(sp.x, sp.y+i));
    draw(im, Rect(r.max.x-i, r.min.y+i,   r.max.x, r.max.y-i),
        color, nil, Pt(sp.x+Dx(r)-i, sp.y+i));
}
```

⟨border() *if negative i, border goes outside* 90a⟩≡ (89e)

```
if(i < 0){
    r = insetrect(r, i);
    sp = addpt(sp, Pt(i,i));
    i = -i;
}
```

⟨*function* insetrect 90b⟩≡ (398)

```
Rectangle
insetrect(Rectangle r, int n)
{
    r.min.x += n;
    r.min.y += n;
    r.max.x -= n;
    r.max.y -= n;
    return r;
}
```

7.1.2 Marshalling

⟨*function* draw1 90c⟩≡ (409b)

```
static void
draw1(Image *dst, Rectangle *r, Image *src, Point *p0, Image *mask, Point *p1, Drawop op)
{
    byte *a;

    _setdrawop(dst->display, op);

    // draw: 'd' dstid[4] srcid[4] maskid[4] R[4*4] P[2*4] P[2*4]
    a = bufimage(dst->display, 1+4+4+4+4*4+2*4+2*4);
    ⟨draw1() sanity check a 91a⟩
    ⟨draw1() sanity check src and mask 91c⟩
    a[0] = 'd';
    BPLONG(a+1, dst->id);
    BPLONG(a+5, src->id);
    BPLONG(a+9, mask->id);
    BPLONG(a+13, r->min.x);
    BPLONG(a+17, r->min.y);
    BPLONG(a+21, r->max.x);
    BPLONG(a+25, r->max.y);
    BPLONG(a+29, p0->x);
    BPLONG(a+33, p0->y);
    BPLONG(a+37, p1->x);
    BPLONG(a+41, p1->y);
}
```

⟨*function* _setdrawop 90d⟩≡ (409b)

```
void
_setdrawop(Display *d, Drawop op)
{
    uchar *a;

    if(op != SoverD){
        a = bufimage(d, 1+1);
```

```
        ⟨_setdrawop() sanity check a 91b⟩
        a[0] = 'O';
        a[1] = op;
    }
}
```

⟨draw1() *sanity check a* 91a⟩≡ (90c)

```
if(a == nil)
    return; // warning?
```

⟨_setdrawop() *sanity check a* 91b⟩≡ (90d)

```
if(a == nil)
    return; // warning?
```

⟨draw1() *sanity check src and mask* 91c⟩≡ (90c)

```
if(src == nil)
    src = dst->display->black;
if(mask == nil)
    mask = dst->display->opaque;
```

7.1.3 Unmarshalling

⟨drawmesg() *locals* 91d⟩+≡ (71) ◁85a 114b▷

```
Memimage *src, *mask;
Point p, q;
// enum<Drawop>
int op;
```

⟨drawmesg() *cases* 91e⟩+≡ (71) ◁85f 92d▷

```
/* draw: 'd' dstid[4] srcid[4] maskid[4] R[4*4] P[2*4] P[2*4] */
case 'd':
    printmesg(fmt="LLLRPP", a, false);
    m = 1+4+4+4+4*4+2*4+2*4;
    ⟨drawmesg() sanity check n with m 75d⟩
    dstid = BGLONG(a+1);
    dst = drawimage(client, a+1);
    src = drawimage(client, a+5);
    mask = drawimage(client, a+9);
    drawrectangle(&r, a+13);
    drawpoint(&p, a+29);
    drawpoint(&q, a+37);

    op = drawclientop(client);
    memdraw(dst, r, src, p, mask, q, op); // the call!

    dstflush(dstid, dst, r);
    continue;
```

⟨*function* drawpoint 91f⟩≡ (480a)

```
void
drawpoint(Point *p, uchar *a)
{
    p->x = BGLONG(a+0*4);
    p->y = BGLONG(a+1*4);
}
```

⟨*function* drawimage 92a⟩≡ (480a)

```
Memimage*
drawimage(Client *client, byte *a)
{
    DImage *d;

    d = drawlookup(client, BGLONG(a), true);
    ⟨drawimage() sanity check d 92b⟩
    return d->image;
}
```

⟨drawimage() *sanity check d* 92b⟩≡ (92a)

```
if(d == nil)
    error(Enodrawimage);
```

⟨*function* drawclientop 92c⟩≡ (480a)

```
static int
drawclientop(Client *cl)
{
    int op;

    op = cl->op;
    cl->op = SoverD;
    return op;
}
```

⟨drawmesg() *cases* 92d⟩+≡ (71) ◁91e 114c▷

```
/* set compositing operator for next draw operation: 'O' op */
case 'O':
    printmesg(fmt="b", a, false);
    m = 1+1;
    ⟨drawmesg() sanity check n with m 75d⟩
    client->op = a[1];
    continue;
```

7.1.4 Algorithm

memdraw()

⟨*function* memdraw 92e⟩≡ (458a)

```
void
memdraw(Memimage *dst, Rectangle r, Memimage *src, Point p0, Memimage *mask, Point p1, int op)
{
    ⟨memdraw() locals 221d⟩

    DBG("memdraw %p %R %p %P %p %P\n", dst, r, src, p0, mask, p1);
    ⟨memdraw() sanity check mask 92f⟩

    ⟨memdraw() if mask has layer 221c⟩
    Top:
    if(dst->layer == nil && src->layer == nil){
        memimagedraw(dst, r, src, p0, mask, p1, op); // back to memdraw
        return;
    }
    ⟨memdraw() when have layers 221e⟩
}
```

⟨memdraw() *sanity check mask* 92f⟩≡ (93b 92e)

```
if(mask == nil)
    mask = memopaque;
```

Memdrawparam

⟨*struct* Memdrawparam 93a⟩≡ (393b)

```
struct Memdrawparam
{
    Memimage *dst;
    Rectangle r;

    Memimage *src;
    Rectangle sr;

    Memimage *mask;
    Rectangle mr;

    // enum<Drawop>
    int op;

    ⟨Memdrawparam other fields 98d⟩
};
```

memimagedraw()

⟨*function* memimagedraw 93b⟩≡ (432)

```
void
memimagedraw(Memimage *dst, Rectangle r, Memimage *src, Point p0, Memimage *mask, Point p1, int op)
{
    Memdrawparam par;

    DBG1("memimagedraw %p/%luX %R @ %p %p/%luX %P %p/%luX %P... ", dst, dst->chan, r, dst->data->bdata, src, sr
    ⟨memdraw() sanity check mask 92f⟩
    ⟨memimagedraw() sanity check op 94a⟩

    // Clipping
    ⟨memimagedraw() call drawclip, if empty rectangle return 95b⟩

    par.op = op;
    par.dst = dst;
    par.r = r;
    par.src = src;
    par.mask = mask;
    /* par.sr set by drawclip */
    /* par.mr set by drawclip */

    // Replicating (and adjust par.state)
    par.state = 0;
    ⟨memimagedraw() if src is repl 99a⟩
    ⟨memimagedraw() if mask is repl 100a⟩

    DBG1("dr %R sr %R mr %R...", r, par.sr, par.mr);
    DBG1("draw dr %R sr %R mr %R %lux\n", r, par.sr, par.mr, par.state);

    // Optimizing
    /*
     * Now that we've clipped the parameters down to be consistent, we
     * simply try sub-drawing routines in order until we find one that was able
     * to handle us.  If the sub-drawing routine returns zero, it means it was
     * unable to satisfy the request, so we do not return.
     */
    ⟨memimagedraw() try hwdraw 94b⟩
    ⟨memimagedraw() try memoptdraw 100b⟩
```

```
    ⟨memimagedraw() try chardraw 312e⟩
    // else

    // Compositing
    /*
     * General calculation-laden case that does alpha for each pixel.
     */
    alphadraw(&par);
    DBG("alphadraw handled\n");
}
```

⟨memimagedraw() *sanity check op* 94a⟩≡ (93b)

```
if(op < Clear || op > SoverD){
    DBG1("op out of range: %d\n", op);
    return;
}
```

7.1.5 Hardware support hook (x86)

⟨memimagedraw() *try hwdraw* 94b⟩≡ (93b)

```
/*
 * Hardware support.  Each video driver provides this function,
 * which checks to see if there is anything it can help with.
 * There could be an if around this checking to see if dst is in video memory.
 */
if(hwdraw(&par)){
    DBG1("hwdraw handled\n");
    return;
}
```

⟨*global* hwaccel*(x86)* 94c⟩≡ (492)

```
bool hwaccel = true;
```

⟨*function* hwdraw*(x86)* 94d⟩≡ (492)

```
//@Scheck: not dead, actually this is overriding some def in libmemdraw!! ugly
bool hwdraw(Memdrawparam *par)
{
    VGAscr *scr;
    Memimage *dst, *src, *mask;
    int m;

    if(!hwaccel)
        return false;

    scr = &vgascreen;

    dst=par->dst;
    src=par->src;
    mask=par->mask;
    ⟨hwdraw() sanity check parameters 95a⟩

    ⟨hwdraw() if software cursor(x86) 190d⟩

    if(dst->data->bdata != gscreendata.bdata)
        return false;
    // else
    ⟨hwdraw() when dst is the screen 312d⟩
}
```

⟨hwdraw() *sanity check parameters* 95a⟩≡ (94d)

```
if(dst == nil || dst->data == nil)
    return false;
if(src == nil || src->data == nil)
    return false;
if(mask == nil || mask->data == nil)
    return false;
```

7.2 Clipping

7.2.1 drawclip()

⟨memimagedraw() *call drawclip, if empty rectangle return* 95b⟩≡ (93b)

```
if(!drawclip(dst, &r, src, &p0, mask, &p1,   &par.sr, &par.mr)){
    DBG1("empty clipped rectangle\n");
    return;
}
```

⟨*function* drawclip 95c⟩≡ (432)

```
/*
 * Clip the destination rectangle further based on the properties of the
 * source and mask rectangles.  Once the destination rectangle is properly
 * clipped, adjust the source and mask rectangles to be the same size.
 * Then if source or mask is replicated, move its clipped rectangle
 * so that its minimum point falls within the repl rectangle.
 *
 * Return zero if the final rectangle is null.
 */
bool
drawclip(Memimage *dst, Rectangle *r, Memimage *src, Point *p0, Memimage *mask, Point *p1,   Rectangle *sr, Rec
{
    Point rmin = r->min; // save old min
    ⟨drawclip() other locals 96d⟩

    // empty rectangle? nothing to do then.
    if(r->min.x >= r->max.x || r->min.y >= r->max.y)
        return false;

    ⟨drawclip() clipping to destination 96a⟩
    ⟨drawclip() clipping to source 96c⟩
    ⟨drawclip() clipping to mask 96f⟩

    /* move source clipping back to destination */
    ⟨drawclip() adjust r if sr or mr changed 96e⟩

    ⟨drawclip() clipping and replication handling 97c⟩

    assert(Dx(*sr) == Dx(*mr) && Dx(*mr) == Dx(*r));
    assert(Dy(*sr) == Dy(*mr) && Dy(*mr) == Dy(*r));
    assert(ptinrect(*p0, src->r));
    assert(ptinrect(*p1, mask->r));
    assert(ptinrect(r->min, dst->r));

    return true;
}
```

7.2.2 Clipping to destination

⟨drawclip() *clipping to destination* 96a⟩≡ (95c)

```
/* clip to destination */ // can modify r
if(!rectclip(r, dst->r) || !rectclip(r, dst->clipr))
    return false;
⟨drawclip() adjust p0 and p1 if r changed 96b⟩
```

⟨drawclip() *adjust p0 and p1 if r changed* 96b⟩≡ (96a)

```
/* move source point */
// p0 = addpt(p0, subpb(r->min, rmin))
p0->x += r->min.x - rmin.x;
p0->y += r->min.y - rmin.y;
/* move mask point */
// p1 = addpt(p1, subpb(r->min, rmin))
p1->x += r->min.x - rmin.x;
p1->y += r->min.y - rmin.y;
```

7.2.3 Clipping to source

⟨drawclip() *clipping to source* 96c⟩≡ (95c)

```
/* map destination rectangle into source */
sr->min = *p0;
sr->max.x = p0->x+Dx(*r);
sr->max.y = p0->y+Dy(*r);

/* sr is r in source coordinates; clip to source */ // can modify sr
if(!(src->flags&Frepl) && !rectclip(sr, src->r))
    return false;
if(!rectclip(sr, src->clipr))
    return false;
```

⟨drawclip() *other locals* 96d⟩≡ (95c) 96g ▷

```
Point delta;
```

⟨drawclip() *adjust r if sr or mr changed* 96e⟩≡ (95c)

```
delta.x = r->min.x - p0->x;
delta.y = r->min.y - p0->y;
// r = rectaddpt(sr, delta)
r->min.x = sr->min.x + delta.x;
r->min.y = sr->min.y + delta.y;
r->max.x = sr->max.x + delta.x;
r->max.y = sr->max.y + delta.y;
```

7.2.4 Clipping to mask

⟨drawclip() *clipping to mask* 96f⟩≡ (95c)

```
/* compute and clip rectangle in mask */
⟨drawclip() if splitcoords 97b⟩
else{
    if(!(mask->flags&Frepl) && !rectclip(sr, mask->r)) // can modify sr
        return false;
    if(!rectclip(sr, mask->clipr))
        return false;
    *p1 = sr->min;
}
```

⟨drawclip() *other locals* 96g⟩+≡ (95c) ◁ 96d 97a ▷

```
bool splitcoords = (p0->x != p1->x) || (p0->y != p1->y);
```

⟨drawclip() *other locals* 97a⟩+≡ (95c) ◁96g

```
Rectangle omr;
```

⟨drawclip() *if splitcoords* 97b⟩≡ (96f)

```
if(splitcoords){
    /* move mask point with source */
    // p1 = addpt(p1, subpt(sr->min, p0))
    p1->x += sr->min.x - p0->x;
    p1->y += sr->min.y - p0->y;

    mr->min = *p1;
    mr->max.x = p1->x + Dx(*sr);
    mr->max.y = p1->y + Dy(*sr);
    omr = *mr;

    /* mr is now rectangle in mask; clip it */
    if(!(mask->flags&Frepl) && !rectclip(mr, mask->r))
        return false;
    if(!rectclip(mr, mask->clipr))
        return false;

    /* reflect any clips back to source */
    // sr = rectsub(mr, omr)
    sr->min.x += mr->min.x - omr.min.x;
    sr->min.y += mr->min.y - omr.min.y;
    sr->max.x += mr->max.x - omr.max.x;
    sr->max.y += mr->max.y - omr.max.y;

    *p1 = mr->min;
}
```

7.2.5 Clipping and replication

⟨drawclip() *clipping and replication handling* 97c⟩≡ (95c)

```
/* move source rectangle so sr->min is in src->r */
⟨drawclip() if src is repl 97d⟩
/* move mask point so it is in mask->r */
⟨drawclip() mask move 98a⟩
```

Clipping with replicated source

⟨drawclip() *if src is repl* 97d⟩≡ (97c)

```
if(src->flags&Frepl) {
    delta.x = drawreplxy(src->r.min.x, src->r.max.x, sr->min.x) - sr->min.x;
    delta.y = drawreplxy(src->r.min.y, src->r.max.y, sr->min.y) - sr->min.y;
    // sr = rectaddpt(sr, delta)
    sr->min.x += delta.x;
    sr->min.y += delta.y;
    sr->max.x += delta.x;
    sr->max.y += delta.y;
}
*p0 = sr->min;
```

⟨*function* drawreplxy 97e⟩≡ (413a)

```
int
drawreplxy(int min, int max, int x)
{
    int sx;
```

```
    sx = (x-min) % (max-min);
    if(sx < 0)
        sx += max - min;
    return sx + min;
}
```

Clipping with replicated mask

⟨drawclip() *mask move* 98a⟩≡ (97c)

```
*p1 = drawrepl(mask->r, *p1);

mr->min = *p1;
mr->max.x = p1->x+Dx(*sr);
mr->max.y = p1->y+Dy(*sr);
```

⟨*function* drawrepl 98b⟩≡ (413a)

```
Point
drawrepl(Rectangle r, Point p)
{
    p.x = drawreplxy(r.min.x, r.max.x, p.x);
    p.y = drawreplxy(r.min.y, r.max.y, p.y);
    return p;
}
```

7.3 Replicating

⟨*enum* _anon_ (include/memdraw.h)2 98c⟩≡ (393b)

```
/*
 * Encapsulated parameters and information for sub-draw routines.
 */
enum Drawparams {
    Simplesrc =1<<0, // 1x1 src
    Simplemask=1<<1, // 1x1 mask

    Replsrc =1<<2,
    Replmask=1<<3,

    Fullmask=1<<4, // fully opaque mask
};
```

⟨Memdrawparam *other fields* 98d⟩≡ (93a) 98e ▷

```
// enum<Drawparams>
ulong state;
```

7.3.1 Source replication

⟨Memdrawparam *other fields* 98e⟩+≡ (93a) ◁ 98d 99e ▷

```
ulong sval;  /* if Simplesrc, the source pixel in src format */
rgba  srgba; /* sval in rgba */
ulong sdval; /* sval in dst format */
```

⟨memimagedraw() *if src is repl* 99a⟩≡ (93b)

```
if(src->flags&Frepl){
    par.state |= Replsrc;
    if(Dx(src->r)==1 && Dy(src->r)==1){
        par.state |= Simplesrc;
        par.sval = pixelbits(src, src->r.min);

        par.srgba = imgtorgba(src, par.sval);
        par.sdval = rgbatoimg(dst, par.srgba);

        ⟨memimagedraw() when src is repl, sanity check pixel value 99b⟩
    }
}
```

⟨memimagedraw() *when src is repl, sanity check pixel value* 99b⟩≡ (99a)

```
if((par.srgba&0xFF) == 0 && (op&DoutS)){
    DBG1("fill with transparent source\n");
    return; /* no-op successfully handled */
}
```

⟨*function* pixelbits 99c⟩≡ (422a)

```
ulong
pixelbits(Memimage *i, Point pt)
{
    byte *p;
    ulong val = 0;
    int bpp = i->depth; // bits per pixel
    ⟨pixelbits() other locals 239a⟩

    p = byteaddr(i, pt);

    switch(bpp){
    ⟨pixelbits() switch bpp cases 239b⟩
    case 8:
        val = p[0];
        break;
    case 32:
        val = p[0]|(p[1]<<8)|(p[2]<<16)|(p[3]<<24);
        break;
    }
    ⟨pixelbits() duplicate byte if depth less than 32 99d⟩
    return val;
}
```

⟨pixelbits() *duplicate byte if depth less than 32* 99d⟩≡ (99c)

```
// duplicate byte in the whole word
while(bpp<32){
    val |= val<<bpp;
    bpp *= 2;
}
```

7.3.2 Mask replication

⟨Memdrawparam *other fields* 99e⟩+≡ (93a) ◁98e

```
ulong mval;  /* if Simplemask, the mask pixel in mask format */
rgba  mrgba; /* mval in rgba */
```

⟨memimagedraw() *if mask is repl* 100a⟩≡ (93b)

```
if(mask->flags & Frepl){
    par.state |= Replmask;
    if(Dx(mask->r)==1 && Dy(mask->r)==1){
        par.state |= Simplemask;
        par.mval = pixelbits(mask, mask->r.min);

        if(par.mval == 0 && (op&DoutS)){
            DBG1("fill with zero mask\n");
            return; /* no-op successfully handled */
        }

        if(par.mval == ~0)
            par.state |= Fullmask;
        par.mrgba = imgtorgba(mask, par.mval);
    }
}
```

7.4 Setting or copying pixels

7.4.1 memoptdraw()

⟨memimagedraw() *try memoptdraw* 100b⟩≡ (93b)

```
/*
 * Optimizations using memmove and memset.
 */
if(memoptdraw(&par)){
    DBG1("memopt handled\n");
    return;
}
```

⟨*function* memoptdraw 100c⟩≡ (432)

```
static bool
memoptdraw(Memdrawparam *par)
{
    int dx, dy;
    // enum<Drawop>
    int op;
    Memimage *src, *dst;

    int m, y;
    ulong v;

    dx = Dx(par->r);
    dy = Dy(par->r);
    src = par->src;
    dst = par->dst;
    op = par->op;

    DBG1("state %lux mval %lux dd %d\n", par->state, par->mval, dst->depth);

    ⟨memoptdraw() if condition for memset 101a⟩
    ⟨memoptdraw() if condition for memmove 101b⟩
    ⟨memoptdraw() if 1 bit mask, src, and dest 237d⟩
    return false;
}
```

7.4.2 Setting pixels

⟨memoptdraw() *if condition for memset* 101a⟩≡ (100c)

```
/*
 * If we have an opaque mask and source is one opaque pixel we can
 * convert to the destination format and just replicate with memset.
 */
m = Simplesrc|Simplemask|Fullmask;
if((par->state&m)==m &&
   (par->srgba&0xFF) == 0xFF &&
   (op == S || op == SoverD)){
    byte p[4]; // source
    byte *dp;  // destination
    int dwid;
    ⟨memoptdraw() locals for memset case 240b⟩

    dwid = dst->width * sizeof(ulong);
    dp = byteaddr(dst, par->r.min);
    v = par->sdval;

    switch(dst->depth){
    ⟨memoptdraw() when condition for memset, switch depth of dst cases 240c⟩
    case 32:
        p[0] = v;  /* make little endian */
        p[1] = v>>8;
        p[2] = v>>16;
        p[3] = v>>24;
        v = *(ulong*)p;

        for(y=0; y < dy; y++, dp += dwid)
            memsetl(dp, v, dx);
        return true;
    }
}
```

7.4.3 Copying pixels

⟨memoptdraw() *if condition for memmove* 101b⟩≡ (100c)

```
/*
 * If no source alpha, an opaque mask, we can just copy the
 * source onto the destination.  If the channels are the same and
 * the source is not replicated, memmove suffices.
 */
m = Simplemask|Fullmask;
if((par->state&(m|Replsrc))==m &&
    src->depth >= 8 &&
    src->chan == dst->chan &&
    !(src->flags&Falpha) &&
    (op == S || op == SoverD)){

    byte *sp, *dp;
    long swid, dwid;
    long nb;
    int dir = 1;

    ⟨memoptdraw() when condition for memmove, change possibly dir 102a⟩

    swid = src->width * sizeof(ulong);
    dwid = dst->width * sizeof(ulong);
```

```
    sp = byteaddr(src, par->sr.min);
    dp = byteaddr(dst, par->r.min);
    ⟨memoptdraw() when condition for memmove, if negative dir 102b⟩
    nb = (dx * src->depth)/8;
    for(y=0; y<dy; y++, sp+=swid, dp+=dwid)
        memmove(dp, sp, nb);
    return true;
}
```

⟨memoptdraw() *when condition for memmove, change possibly dir* 102a⟩≡ (101b)

```
if(src->data == dst->data &&
   byteaddr(dst, par->r.min) > byteaddr(src, par->sr.min))
    dir = -1;
```

⟨memoptdraw() *when condition for memmove, if negative dir* 102b⟩≡ (101b)

```
if(dir == -1){
    sp += (dy-1)*swid;
    dp += (dy-1)*dwid;
    swid = -swid;
    dwid = -dwid;
}
```

7.5 Compositing pixels

7.5.1 alphadraw()

⟨alphadraw() *locals* 102c⟩≡ (103) 102d ▷

```
Memimage *src, *mask, *dst;
Rectangle r, sr, mr;
int dx, dy;
// enum<Drawop>
int op;
```

⟨alphadraw() *locals* 102d⟩+≡ (103) ◁ 102c 102e ▷

```
bool isgrey;
```

⟨alphadraw() *locals* 102e⟩+≡ (103) ◁ 102d 102f ▷

```
Readfn *rdsrc, *rdmask, *rddst;
Calcfn *calc;
Writefn *wrdst;
```

⟨alphadraw() *locals* 102f⟩+≡ (103) ◁ 102e 102g ▷

```
Buffer bsrc, bdst, bmask;
```

⟨alphadraw() *locals* 102g⟩+≡ (103) ◁ 102f 104a ▷

```
int starty, endy;
int dsty, srcy, masky;
int dir;
int y; // iteration variable, we iterate over lines
```

⟨*function* alphadraw 103⟩≡ (429b)

```
/*
 * For each scan line, we expand the pixels from source, mask, and destination
 * into byte-aligned red, green, blue, alpha, and grey channels.  If buffering
 * is not needed and the channels were already byte-aligned
 * (grey8, rgb24, rgba32, rgb32),
 * the readers need not copy the data: they can simply return pointers to the
 * data.
 * If the destination image is grey and the source is not, it is converted
 * using the NTSC formula.
 *
 * Once we have all the channels, we call either rgbcalc or greycalc,
 * depending on whether the destination image is color.  This is allowed to
 * overwrite the dst buffer (perhaps the actual data, perhaps a copy) with
 * its result.  It should only overwrite the dst buffer
 * with the same format (i.e. red bytes with red bytes, etc.)  A new buffer
 * is returned from the calculator, and that buffer is passed to a function
 * to write it to the destination.
 * If the buffer is already pointing at the destination, the writing function
 * is a no-op.
 */
int
alphadraw(Memdrawparam *par)
{
    ⟨alphadraw() locals 102c⟩

    r = par->r;
    dx = Dx(r);
    dy = Dy(r);
    src = par->src;
    mask = par->mask;
    dst = par->dst;
    sr = par->sr;
    mr = par->mr;
    op = par->op;

    starty = 0;
    endy = dy;
    dir = 1;

    isgrey = dst->flags&Fgrey;
    ⟨alphadraw() set needbuf and possibly adjust starty and endy 104b⟩

    ⟨alphadraw() set Dbuf z part1 105d⟩

    ⟨alphadraw() if source has no alpha and simple bit mask 315f⟩
    else{
        /* usual alphadraw parameter fetching */
        rdsrc = readfn(src);
        rddst = readfn(dst);
        wrdst = writefn(dst);
        calc = alphacalc[op];
        ⟨alphadraw() set rdmask 110c⟩
    }

    ⟨alphadraw() when small repl rectangle optimization 315b⟩
    ⟨alphadraw() set Dbuf z part2 106c⟩

    ⟨alphadraw() setting srcy, masky, and dsty, and clipping part1 104c⟩
    assert(0 <= srcy  && srcy  < Dy(src->r));
```

```
    assert(0 <= masky && masky < Dy(mask->r));
    assert(0 <= dsty  && dsty  < Dy(dst->r));

    // the big loop!
    for(y=starty; y!=endy; y+=dir, srcy+=dir, masky+=dir, dsty+=dir){
        ⟨alphadraw() clipping part2 104d⟩

        bsrc  = rdsrc (&z->spar, z->spar.bufbase, srcy);
        bmask = rdmask(&z->mpar, z->mpar.bufbase, masky);
        bdst  = rddst (&z->dpar, z->dpar.bufbase, dsty);

        // !!The calc dispatch!!
        bdst = calc(bdst, bsrc, bmask, dx, isgrey, op);
        wrdst(&z->dpar, z->dpar.bytermin + dsty * z->dpar.bwidth, bdst);
    }
    ⟨alphadraw() free z 106a⟩
    return 1;
}
```

7.5.2 Buffering

⟨alphadraw() *locals* 104a⟩+≡ (103) ◁102g 105c▷

```
bool needbuf;
```

⟨alphadraw() *set needbuf and possibly adjust starty and endy* 104b⟩≡ (103)

```
/*
 * Buffering when src and dst are the same bitmap is sufficient but not
 * necessary.  There are stronger conditions we could use.  We could
 * check to see if the rectangles intersect, and if simply moving in the
 * correct y direction can avoid the need to buffer.
 */
needbuf = (src->data == dst->data);

if (needbuf && byteaddr(dst, r.min) > byteaddr(src, sr.min)) {
    dir = -1;
    starty = dy-1;
    endy = -1;
}
```

7.5.3 Clipping and replicating

⟨alphadraw() *setting srcy, masky, and dsty, and clipping part1* 104c⟩≡ (103)

```
/*
 * srcy, masky, and dsty are offsets from the top of their
 * respective Rectangles.  they need to be contained within
 * the rectangles, so clipy can keep them there without division.
 */
srcy  = (starty + sr.min.y - src->r.min.y)  % Dy(src->r);
masky = (starty + mr.min.y - mask->r.min.y) % Dy(mask->r);
dsty  =  starty + r.min.y  - dst->r.min.y;
```

⟨alphadraw() *clipping part2* 104d⟩≡ (103)

```
clipy(src, &srcy);
clipy(dst, &dsty);
clipy(mask, &masky);
```

⟨*function* clipy 105a⟩≡ (429b)

```
static void
clipy(Memimage *img, int *y)
{
    int dy;

    // *y = *y % Dy(img->r)

    dy = Dy(img->r);
    if(*y == dy)
        *y = 0;
    else if(*y == -1)
        *y = dy-1;
    assert(0 <= *y && *y < dy);
}
```

7.5.4 Additional data structures

Dbuf

⟨*struct* Dbuf 105b⟩≡ (429b)

```
struct Dbuf
{
    Param spar, mpar, dpar;
    // array<byte> (length = Dbuf.n)
    byte *p;
    int n;
    ⟨Dbuf other fields 105g⟩
};
```

⟨alphadraw() *locals* 105c⟩+≡ (103) ◁104a 106b▷

```
Dbuf *z;
```

⟨alphadraw() *set Dbuf z part1* 105d⟩≡ (103)

```
z = allocdbuf();
⟨alphadraw() sanity check z 106e⟩
⟨alphadraw() set z params part1 107a⟩
```

⟨*global* dbuf 105e⟩≡ (429b)

```
static Dbuf dbuf[10];
```

⟨*function* allocdbuf 105f⟩≡ (429b)

```
static Dbuf*
allocdbuf(void)
{
    int i;

    for(i=0; i<nelem(dbuf); i++){
        if(dbuf[i].inuse)
            continue;
        if(!_tas(&dbuf[i].inuse))
            return &dbuf[i];
    }
    return nil;
}
```

⟨Dbuf *other fields* 105g⟩≡ (105b)

```
bool inuse;
```

⟨alphadraw() *free z* 106a⟩≡ (103)

```
z->inuse = false;
```

⟨alphadraw() *locals* 106b⟩+≡ (103) ◁105c

```
byte *drawbuf;
int ndrawbuf;
```

⟨alphadraw() *set Dbuf z part2* 106c⟩≡ (103)

```
⟨alphadraw() grow drawbuf if needed 106d⟩
drawbuf = z->p;
⟨alphadraw() set z params part2 107c⟩
```

⟨alphadraw() *grow drawbuf if needed* 106d⟩≡ (106c)

```
if(z->n < ndrawbuf){
    free(z->p);
    z->p = mallocz(ndrawbuf, 0);
    ⟨alphadraw() sanity check z->p 106f⟩
    z->n = ndrawbuf;
}
```

⟨alphadraw() *sanity check z* 106e⟩≡ (105d)

```
if(z == nil)
    return 0;
```

⟨alphadraw() *sanity check* z->p 106f⟩≡ (106d)

```
if(z->p == nil){
    z->inuse = false;
    return 0;
}
```

ParamDraw

⟨*struct* ParamDraw 106g⟩≡ (429b)

```
/* giant rathole to customize functions with */
struct ParamDraw {

    Memimage *img;
    Rectangle r;
    int dx; /* of r */ // size of a line of a rectangle in pixels
    int  bwidth; // image width in bytes

    // source bytes, always same y but x varies
    byte *bytey0s; /* byteaddr(Pt(img->r.min.x, img->r.min.y)) */
    byte *bytermin; /* byteaddr(Pt(r.min.x,      img->r.min.y)) */
    byte *bytey0e; /* byteaddr(Pt(img->r.max.x, img->r.min.y)) */

    bool needbuf;
    // destination bytes
    int bufoff;
    int bufdelta;
    // array<byte>, (address = Dbuf.p + bufoff, length = bufdelta)
    byte *bufbase;

    int dir; // -1 or 1

    ⟨ParamDraw mask fields 111a⟩
    ⟨ParamDraw conversion fields 316b⟩
    ⟨ParamDraw replication fields 314a⟩
};
```

⟨alphadraw() *set z params part1* 107a⟩≡ (105d)

```
ndrawbuf = 0;
getparam(&z->spar, src, sr,  isgrey, needbuf, &ndrawbuf);
getparam(&z->dpar, dst, r,   isgrey, needbuf, &ndrawbuf);
getparam(&z->mpar, mask, mr, false,  needbuf, &ndrawbuf);
z->spar.dir = z->mpar.dir = z->dpar.dir = dir;
```

⟨*function* getparam 107b⟩≡ (429b)

```
static void
getparam(Param *p, Memimage *img, Rectangle r, bool convgrey, bool needbuf, int *ndrawbuf)
{
    int nbuf;

    memset(p, 0, sizeof(Param));

    p->img = img;
    p->r = r;
    p->dx = Dx(r);
    p->needbuf = needbuf;
    p->convgrey = convgrey;

    assert(img->r.min.x <= r.min.x && r.min.x < img->r.max.x);

    p->bytey0s  = byteaddr(img, Pt(img->r.min.x, img->r.min.y));
    p->bytermin = byteaddr(img, Pt(r.min.x,      img->r.min.y));
    p->bytey0e  = byteaddr(img, Pt(img->r.max.x, img->r.min.y));

    assert(p->bytey0s <= p->bytermin && p->bytermin <= p->bytey0e);
    if(p->r.min.x == p->img->r.min.x)
        assert(p->bytermin == p->bytey0s);

    p->bwidth   = sizeof(ulong) * img->width;

    nbuf = 1;
    ⟨getparam() if small replicated rectangle 314b⟩

    p->bufdelta = 4 * p->dx;
    p->bufoff = *ndrawbuf;
    *ndrawbuf += p->bufdelta * nbuf;
}
```

⟨alphadraw() *set z params part2* 107c⟩≡ (106c) 317b ▷

```
/*
 * Before we were saving only offsets from drawbuf in the parameter
 * structures; now that drawbuf has been grown to accomodate us,
 * we can fill in the pointers.
 */
z->spar.bufbase = drawbuf + z->spar.bufoff;
z->mpar.bufbase = drawbuf + z->mpar.bufoff;
z->dpar.bufbase = drawbuf + z->dpar.bufoff;
```

Buffer

⟨*struct* Buffer 107d⟩≡ (429b)

```
struct Buffer {
    /* used by most routines */
    byte *red;
    byte *grn;
    byte *blu;
```

```
    byte *alpha; // can be &ones

    ulong *rgba; // bad name, just start of pixel
    int delta; /* number of bytes to add to pointer to get next pixel to the right */

    byte *grey;

    ⟨Buffer boolcalc fields 320b⟩
};
```

7.5.5 Reading pixels: readfn()

⟨*function* readfn 108a⟩≡ (429b)

```
static Readfn*
readfn(Memimage *img)
{
    ⟨readfn() if depth less than 8 241c⟩
    ⟨readfn() if cmap 249c⟩
    return readbyte;
}
```

⟨*function* readbyte 108b⟩≡ (429b)

```
static Buffer
readbyte(Param *p, byte *buf, int y)
{
    Memimage *img;
    byte *begin, *end, *r;
    Buffer b;
    byte *w;
    int dx, nb;

    bool isgrey, convgrey;
    bool alphaonly, copyalpha;
    ⟨readbyte other locals 266a⟩

    img = p->img;
    begin = p->bytey0s  + y*p->bwidth;
    r     = p->bytermin + y*p->bwidth;
    end   = p->bytey0e  + y*p->bwidth;

    w = buf;
    dx = p->dx;
    nb = img->depth/8;

    convgrey = p->convgrey; /* convert rgb to grey */
    isgrey   = img->flags&Fgrey;
    copyalpha = img->flags&Falpha;
    alphaonly = p->alphaonly;

    ⟨readbyte() simple case when no repl, no grey, depth 8 108c⟩
    // else
    ⟨readbyte() more complex cases, possible repl, grey, and small depth 266b⟩
}
```

⟨readbyte() *simple case when no repl, no grey, depth 8* 108c⟩≡ (108b)

```
/* if we can, avoid processing everything */
if(!(img->flags&Frepl) && !convgrey && (img->flags&Fbytes)){
    memset(&b, 0, sizeof(Buffer));
    if(p->needbuf){
```

```
        memmove(buf, r, dx*nb);
        r = buf;
    }
    b.rgba = (ulong*)r;

    if(copyalpha)
        b.alpha = r + img->shift[CAlpha]/8;
    else
        b.alpha = &ones;
    ⟨readbyte() in simple case, if isgrey 247d⟩
    }else{
        b.red = r + img->shift[CRed]/8;
        b.grn = r + img->shift[CGreen]/8;
        b.blu = r + img->shift[CBlue]/8;
    }
    b.delta = nb;
    return b;
}
```

⟨*global* ones 109a⟩≡ (429b)

```
static uchar ones = 0xff;
```

7.5.6 Writing pixels: writefn()

⟨*function* writefn 109b⟩≡ (429b)

```
static Writefn*
writefn(Memimage *img)
{
    ⟨writefn() if depth less than 8 243a⟩
    ⟨writefn() if cmap 249d⟩
    return writebyte;
}
```

⟨*function* writebyte 109c⟩≡ (429b)

```
static void
writebyte(Param *p, byte *w, Buffer src)
{
    Memimage *img;
    int dx, nb;
    byte *red, *grn, *blu, *grey, *alpha;
    int delta;
    // will iterate over all the pixels in one line in src
    int i;

    bool isalpha, isgrey;
    byte ff;
    int adelta;
    ulong u, mask;

    img = p->img;

    red = src.red;
    grn = src.grn;
    blu = src.blu;
    alpha = src.alpha;
    delta = src.delta;
    grey = src.grey;
    dx = p->dx;
```

```
    nb = img->depth/8;
    mask = (nb==4) ? 0 : ~((1<<img->depth)-1); // >>

    isalpha = img->flags&Falpha;
    isgrey = img->flags&Fgrey;

    ⟨writebyte() alpha handling part1 110a⟩

    for(i=0; i<dx; i++){
        u = w[0] | (w[1]<<8) | (w[2]<<16) | (w[3]<<24);
        u &= mask;
        ⟨writebyte() if isgrey 247e⟩
        else{
            u |= ((*red >> (8-img->nbits[CRed])) & img->mask[CRed]) << img->shift[CRed];
            u |= ((*grn >> (8-img->nbits[CGreen])) & img->mask[CGreen]) << img->shift[CGreen];
            u |= ((*blu >> (8-img->nbits[CBlue])) & img->mask[CBlue]) << img->shift[CBlue];
            red += delta;
            grn += delta;
            blu += delta;
        }
        ⟨writebyte() alpha handling part2 110b⟩

        w[0] = u;
        w[1] = u>>8;
        w[2] = u>>16;
        w[3] = u>>24;
        w += nb;
    }
}
```

⟨writebyte() *alpha handling part1* 110a⟩≡ (109c)

```
adelta = src.delta;
if(isalpha && (alpha == nil || alpha == &ones)){
    ff = 0xFF;
    alpha = &ff;
    adelta = 0;
}
```

⟨writebyte() *alpha handling part2* 110b⟩≡ (109c)

```
if(isalpha){
    u |= ((*alpha >> (8-img->nbits[CAlpha])) & img->mask[CAlpha]) << img->shift[CAlpha];
    alpha += adelta;
}
```

7.5.7 Reading the mask

⟨alphadraw() *set rdmask* 110c⟩≡ (103)

```
if(mask->flags&Falpha){
    rdmask = readalphafn(mask);
    z->mpar.alphaonly = true;
}
/*
 * If there is no alpha channel, we'll ask for a grey channel
 * and pretend it is the alpha.
 */
else{
    z->mpar.greymaskcall = readfn(mask);
    z->mpar.convgrey = true;
    rdmask = greymaskread;
```

```
    ⟨alphadraw() when mask and source have no alpha, possibly adapt calc 316a⟩
}
```

⟨ParamDraw *mask fields* 111a⟩≡ (106g)

```
bool alphaonly;
bool convgrey;
Readfn *greymaskcall;
```

⟨*function* readalphafn 111b⟩≡ (429b)

```
static Readfn*
readalphafn(Memimage *m)
{
    USED(m);
    return readbyte;
}
```

⟨*function* greymaskread 111c⟩≡ (429b)

```
/*
 * Alpha reading function that simply relabels the grey pointer.
 */
static Buffer
greymaskread(Param *p, uchar *buf, int y)
{
    Buffer b;

    b = p->greymaskcall(p, buf, y);
    b.alpha = b.grey;
    return b;
}
```

7.5.8 Compositing pixels: alphacalcxx()

⟨*global* alphacalc 111d⟩≡ (429b)

```
static Calcfn *alphacalc[Ncomp] =
{
    alphacalc0,         /* Clear */
    alphacalc14,        /* DoutS */
    alphacalc2810,      /* SoutD */
    alphacalc3679,      /* DxorS */
    alphacalc14,        /* DinS */
    alphacalc5,         /* D */
    alphacalc3679,      /* DatopS */
    alphacalc3679,      /* DoverS */
    alphacalc2810,      /* SinD */
    alphacalc3679,      /* SatopD */
    alphacalc2810,      /* S */
    alphacalc11,        /* SoverD */ // the classic
};
```

⟨*function* alphacalc11 111e⟩≡ (429b)

```
static Buffer
alphacalc11(Buffer bdst, Buffer bsrc, Buffer bmask, int dx, bool grey, int op)
{
    int sa, ma;
    int fd;
    int sadelta;
    int i; // iterate over pixels of the line
    bool q;
```

```
    // temporaries for MUL
    ulong s, t, u, v;
    // ??
    Buffer obdst;

    USED(op);
    obdst = bdst;
    ⟨alphacalc11() alpha handling part1 112b⟩
    q = (bsrc.delta == 4) && (bdst.delta == 4);

    // pixel iteration of the line
    for(i=0; i<dx; i++){
        sa = *bsrc.alpha;
        ma = *bmask.alpha;
        fd = 255 - MUL(sa, ma, t);

        ⟨alphacalc11() if isgrey 247f⟩
        else{
            ⟨alphacalc11() special case for 32bits source and dest 323b⟩
            // else

            *bdst.red = MUL(ma, *bsrc.red, s) + MUL(fd, *bdst.red, t);
            *bdst.grn = MUL(ma, *bsrc.grn, s) + MUL(fd, *bdst.grn, t);
            *bdst.blu = MUL(ma, *bsrc.blu, s) + MUL(fd, *bdst.blu, t);

            bsrc.red += bsrc.delta;
            bsrc.blu += bsrc.delta;
            bsrc.grn += bsrc.delta;

            bdst.red += bdst.delta;
            bdst.blu += bdst.delta;
            bdst.grn += bdst.delta;
        }

        ⟨alphacalc11() alpha handling part2 112c⟩
    }
    return obdst;
}
```

⟨*function* MUL 112a⟩≡ (429b)

```
#define MUL(x, y, t) (t = (x)*(y)+128, (t+(t>>8))>>8)
```

⟨alphacalc11() *alpha handling part1* 112b⟩≡ (111e)

```
sadelta = (bsrc.alpha == &ones) ? 0 : bsrc.delta;
```

⟨alphacalc11() *alpha handling part2* 112c⟩≡ (111e)

```
if(bdst.alpha != &ones){
    *bdst.alpha = MUL(ma, sa, s) + MUL(fd, *bdst.alpha, t);
    bdst.alpha += bdst.delta;
}
bmask.alpha += bmask.delta;
bsrc.alpha += sadelta;
```

Chapter 8

Drawing Shapes

8.1 Lines

8.1.1 API

⟨*function* `line` 113a⟩≡ (410a)

```
void
line(Image *dst, Point p0, Point p1, int end0, int end1, int radius, Image *src, Point sp)
{
    lineop(dst, p0, p1, end0, end1, radius, src, sp, SoverD);
}
```

⟨*enum* `Endline` 113b⟩≡ (386d)

```
enum EndLine
{
    /* line ends */
    Endsquare   = 0, // default
    Enddisc     = 1,
    Endarrow    = 2,

    Endmask     = 0x1F
};
```

8.1.2 Marshalling

⟨*function* `lineop` 113c⟩≡ (410a)

```
void
lineop(Image *dst, Point p0, Point p1, int end0, int end1, int radius, Image *src, Point sp, Drawop op)
{
    byte *a;

    _setdrawop(dst->display, op);

    // draw line: 'L' dstid[4] p0[2*4] p1[2*4] end0[4] end1[4] radius[4] srcid[4] sp[2*4]
    a = bufimage(dst->display, 1+4+2*4+2*4+4+4+4+4+2*4);
    ⟨lineop() sanity check a 114a⟩
    a[0] = 'L';
    BPLONG(a+1, dst->id);
    BPLONG(a+5, p0.x);
    BPLONG(a+9, p0.y);
    BPLONG(a+13, p1.x);
    BPLONG(a+17, p1.y);
    BPLONG(a+21, end0);
    BPLONG(a+25, end1);
```

```
    BPLONG(a+29, radius);
    BPLONG(a+33, src->id);
    BPLONG(a+37, sp.x);
    BPLONG(a+41, sp.y);
}
```

⟨lineop() *sanity check a* 114a⟩≡ (113c)

```
if(a == nil){
    fprint(2, "image line: %r\n");
    return;
}
```

8.1.3 Unmarshalling

⟨drawmesg() *locals* 114b⟩+≡ (71) ◁91d 119e▷

```
Point sp;
// enum<EndLine>
int e0, e1;
```

⟨drawmesg() *cases* 114c⟩+≡ (71) ◁92d 120a▷

```
/* draw line: 'L' dstid[4] p0[2*4] p1[2*4] end0[4] end1[4] radius[4] srcid[4] sp[2*4] */
case 'L':
    printmesg(fmt="LPPlllLP", a, false);
    m = 1+4+2*4+2*4+4+4+4+4+2*4;
    ⟨drawmesg() sanity check n with m 75d⟩
    dst = drawimage(client, a+1);
    dstid = BGLONG(a+1);
    drawpoint(&p, a+5);
    drawpoint(&q, a+13);
    e0 = BGLONG(a+21);
    e1 = BGLONG(a+25);
    j = BGLONG(a+29);
    ⟨drawmesg() when draw line, sanity check j 114d⟩
    src = drawimage(client, a+33);
    drawpoint(&sp, a+37);
    op = drawclientop(client);

    memline(dst, p, q, e0, e1, j, src, sp, op); // The call
    ⟨drawmesg() when draw line, possible flush 324f⟩
    continue;
```

⟨drawmesg() *when draw line, sanity check j* 114d⟩≡ (114c)

```
if(j < 0)
    error("negative line width");
```

8.1.4 Algorithm

⟨*function* memline 114e⟩≡ (459b)

```
void
memline(Memimage *dst, Point p0, Point p1, int end0, int end1, int radius, Memimage *src, Point sp, int op)
{
    _memline(dst, p0, p1, end0, end1, radius, src, sp, dst->clipr, op);
}
```

⟨*function* _memline 115a⟩≡ (459b)

```
static
void
_memline(Memimage *dst, Point p0, Point p1, int end0, int end1, int radius, Memimage *src, Point sp, Rectangle
{
    Memlayer *dl;
    ⟨_memline() other locals 228b⟩

    ⟨_memline() sanity check radius 115b⟩
    ⟨_memline() sanity check no src layer 227c⟩

    Top:
    dl = dst->layer;
    if(dl == nil){
        // back to memdraw/
        _memimageline(dst, p0, p1, end0, end1, radius, src, sp, clipr, op);
        return;
    }
    ⟨_memline() when have layers 228a⟩
}
```

⟨_memline() *sanity check radius* 115b⟩≡ (115)

```
if(radius < 0)
    return;
```

⟨*function* _memimageline 115c⟩≡ (435a)

```
void
_memimageline(Memimage *dst, Point p0, Point p1, int end0, int end1, int radius, Memimage *src, Point sp, Recta
{

    bool hor;
    ⟨_memimageline() other locals 116c⟩

    ⟨_memline() sanity check radius 115b⟩
    ⟨_memimageline() clipping clipr 115d⟩

    /* this means that only verline() handles degenerate lines (p0==p1) */
    hor = (abs(p1.x - p0.x) > abs(p1.y - p0.y));

    /*
     * Clipping is a little peculiar.  We can't use Sutherland-Cohen
     * clipping because lines are wide.  But this is probably just fine:
     * we do all math with the original p0 and p1, but clip when deciding
     * what pixels to draw.  This means the layer code can call this routine,
     * using clipr to define the region being written, and get the same set
     * of pixels regardless of the dicing.
     */
     // ????

    ⟨_memimageline() swap p0 and p1 to have p0 before p1 116a⟩

    // easy case
    ⟨_memimageline() when vertical or horizontal lines 116b⟩
    // else
    /*    Hard: */
    ⟨_memimageline() when arbitrary lines 117c⟩
}
```

⟨_memimageline() *clipping clipr* 115d⟩≡ (115c)

```
// clip to dst
```

```
  if(!rectclip(&clipr, dst->r))
      return;
  if(!rectclip(&clipr, dst->clipr))
      return;

  // clip to src
  d = subpt(sp, p0);
  if(!rectclip(&clipr, rectsubpt(src->clipr, d)))
      return;
  if(!(src->flags&Frepl) && !rectclip(&clipr, rectsubpt(src->r, d)))
      return;
```

⟨_memimageline() *swap p0 and p1 to have p0 before p1* 116a⟩≡ (115c)

```
  if((hor && p0.x > p1.x) || (!hor && p0.y > p1.y)){
      // swap(p0, p1)
      q = p0;
      p0 = p1;
      p1 = q;
      // swap(end0, end1)
      t = end0;
      end0 = end1;
      end1 = t;
  }
```

Simple case: vertical or horizontal lines

⟨_memimageline() *when vertical or horizontal lines* 116b⟩≡ (115c)

```
  if((p0.x == p1.x || p0.y == p1.y)
  && (end0&0x1F) == Endsquare
  && (end1&0x1F) == Endsquare){
      r.min = p0;
      r.max = p1;
      // vertical line
      if(p0.x == p1.x){
          r.min.x -= radius;
          r.max.x += radius+1;
      }
      // horizontal line
      else{
          r.min.y -= radius;
          r.max.y += radius+1;
      }
      ⟨_memimageline() change dst clipr 116d⟩
      sp = addpt(r.min, d);
      memimagedraw(dst, r, src, sp, memopaque, sp, op);
      ⟨_memimageline() restore dst clipr 116e⟩
      return;
  }
```

⟨_memimageline() *other locals* 116c⟩≡ (115c) 117a ▷

```
  Rectangle oclipr;
```

⟨_memimageline() *change dst clipr* 116d⟩≡ (117c 116b)

```
  oclipr = dst->clipr;
  dst->clipr = clipr;
```

⟨_memimageline() *restore dst clipr* 116e⟩≡ (117c 116b)

```
  dst->clipr = oclipr;
```

General case

⟨_memimageline() *other locals* 117a⟩+≡ (115c) ◁116c

```
int sin, cos, dx, dy, t;
Rectangle r;
Point q;
Point pts[10], *pp;
Point d;
```

⟨*constant* ICOSSCALE 117b⟩≡ (386c)

```
ICOSSCALE   = 1024,
```

⟨_memimageline() *when arbitrary lines* 117c⟩≡ (115c)

```
/* draw thick line using polygon fill */
icossin2(p1.x - p0.x, p1.y - p0.y, &cos, &sin);
dx = (sin*(2*radius+1))/2;
dy = (cos*(2*radius+1))/2;
pp = pts;
⟨_memimageline() change dst clipr 116d⟩

q.x = ICOSSCALE*p0.x + ICOSSCALE/2- cos/2;
q.y = ICOSSCALE*p0.y + ICOSSCALE/2- sin/2;
switch(end0 & 0x1F){
⟨_memimageline() switch end0 cases 257c⟩
⟨_memimageline() switch end0 cases, fallthrough to Endsquare 256a⟩
case Endsquare:
default:
    pp->x = q.x - dx;
    pp->y = q.y + dy;
    pp++;
    pp->x = q.x + dx;
    pp->y = q.y - dy;
    pp++;
    break;
}
// 2 points

q.x = ICOSSCALE*p1.x+ICOSSCALE/2+cos/2;
q.y = ICOSSCALE*p1.y+ICOSSCALE/2+sin/2;
switch(end1 & 0x1F){
⟨_memimageline() switch end1 cases 257d⟩
⟨_memimageline() switch end1 cases, fallthrough to Endsquare 256b⟩
case Endsquare:
default:
    pp->x = q.x + dx;
    pp->y = q.y - dy;
    pp++;
    pp->x = q.x - dx;
    pp->y = q.y + dy;
    pp++;
    break;
}
// 2 more points

_memfillpolysc(dst, pts, pp-pts, ~0, src,
               addpt(pts[0], mulpt(d, ICOSSCALE)),
               false, 10, true, op);
⟨_memimageline() restore dst clipr 116e⟩
return;
```

8.2 Polygons

8.2.1 API

⟨*function* poly 118a⟩≡ (410b)

```
void
poly(Image *dst, Point *p, int np, int end0, int end1, int radius, Image *src, Point sp)
{
    dopoly('p', dst, p, np, end0, end1, radius, src, &sp, SoverD);
}
```

⟨*function* polyop 118b⟩≡ (410b)

```
void
polyop(Image *dst, Point *p, int np, int end0, int end1, int radius, Image *src, Point sp, Drawop op)
{
    dopoly('p', dst, p, np, end0, end1, radius, src, &sp, op);
}
```

⟨*function* fillpoly 118c⟩≡ (410b)

```
void
fillpoly(Image *dst, Point *p, int np, int wind, Image *src, Point sp)
{
    dopoly('P', dst, p, np, wind, 0, 0, src, &sp, SoverD);
}
```

⟨*function* fillpolyop 118d⟩≡ (410b)

```
void
fillpolyop(Image *dst, Point *p, int np, int wind, Image *src, Point sp, Drawop op)
{
    dopoly('P', dst, p, np, wind, 0, 0, src, &sp, op);
}
```

8.2.2 Marshalling

⟨*function* dopoly 118e⟩≡ (410b)

```
static
void
dopoly(int cmd, Image *dst, Point *pp, int np, int end0, int end1, int radius, Image *src, Point *sp, Drawop op
{
    byte *a, *t, *u;
    int i, ox, oy;

    ⟨dopoly() sanity check n 119a⟩
    t = malloc(np*2*3);
    ⟨dopoly() sanity check t 119b⟩
    u = t;
    ox = oy = 0;
    for(i=0; i<np; i++){
        u = addcoord(u, ox, pp[i].x);
        ox = pp[i].x;
        u = addcoord(u, oy, pp[i].y);
        oy = pp[i].y;
    }

    _setdrawop(dst->display, op);

    /// polygon: 'p' dstid[4] n[2] end0[4] end1[4] radius[4] srcid[4] sp[2*4] p0[2*4] dp[2*2*n]
    a = bufimage(dst->display, 1+4+2+4+4+4+4+2*4+(u-t));
    ⟨dopoly() sanity check a 119c⟩
```

```
    a[0] = cmd;
    BPLONG(a+1, dst->id);
    BPSHORT(a+5, np-1);
    BPLONG(a+7, end0);
    BPLONG(a+11, end1);
    BPLONG(a+15, radius);
    BPLONG(a+19, src->id);
    BPLONG(a+23, sp->x);
    BPLONG(a+27, sp->y);
    memmove(a+31, t, u-t);
    free(t);
}
```

⟨dopoly() *sanity check n* 119a⟩≡ (118e)

```
if(np == 0)
    return;
```

⟨dopoly() *sanity check t* 119b⟩≡ (118e)

```
if(t == nil)
    return;
```

⟨dopoly() *sanity check a* 119c⟩≡ (118e)

```
if(a == nil){
    free(t);
    fprint(2, "image poly: %r\n");
    return;
}
```

⟨*function* addcoord 119d⟩≡ (410b)

```
static
uchar*
addcoord(byte *p, int oldx, int newx)
{
    int dx;

    dx = newx-oldx;
    /* does dx fit in 7 signed bits? */
    if((unsigned)(dx - -0x40) <= 0x7F)
        *p++ = dx&0x7F;
    else{
        *p++ = 0x80 | (newx&0x7F);
        *p++ = newx>>7;
        *p++ = newx>>15;
    }
    return p;
}
```

8.2.3 Unmarshalling

⟨drawmesg() *locals* 119e⟩+≡ (71) ◁114b 151b▷

```
int ni;
Point *pp;
bool doflush;
int ox, oy;
int esize, oesize;
byte *u;
int y;
```

⟨drawmesg() *cases* 120a⟩+≡ (71) ◁114c 129c▷

```
/* filled polygon: 'P' dstid[4] n[2] wind[4] ignore[2*4] srcid[4] sp[2*4] p0[2*4] dp[2*2*n] */
/* polygon: 'p' dstid[4] n[2] end0[4] end1[4] radius[4] srcid[4] sp[2*4] p0[2*4] dp[2*2*n] */
case 'p':
case 'P':
    printmesg(fmt="LslllLPP", a, false);
    m = 1+4+2+4+4+4+4+2*4;
    ⟨drawmesg() sanity check n with m 75d⟩
    dstid = BGLONG(a+1);
    dst = drawimage(client, a+1);
    ni = BGSHORT(a+5);
    ⟨drawmesg() when draw polygon, sanity check ni 120b⟩
    e0 = BGLONG(a+7);
    e1 = BGLONG(a+11);
    j = 0;
    if(*a == 'p'){
        j = BGLONG(a+15);
        ⟨drawmesg() when draw polygon, sanity check j 120c⟩
    }
    src = drawimage(client, a+19);
    drawpoint(&sp, a+23);
    drawpoint(&p, a+31);
    ni++;
    pp = malloc(ni*sizeof(Point));
    ⟨drawmesg() when draw polygon, sanity check pp 121a⟩

    ⟨drawmesg() when draw polygon, set doflush 325d⟩

    ox = oy = 0;
    esize = 0;
    u = a+m;
    for(y=0; y<ni; y++){
        q = p;
        oesize = esize;
        u = drawcoord(u, a+n, ox, &p.x);
        u = drawcoord(u, a+n, oy, &p.y);
        ox = p.x;
        oy = p.y;

        ⟨drawmesg() when draw polygon, if doflush 326a⟩
        pp[y] = p;
    }
    ⟨drawmesg() when draw polygon, special flush if y is 1 326b⟩
    op = drawclientop(client);

    if(*a == 'p')
        mempoly(dst, pp, ni, e0, e1, j, src, sp, op); // The call
    else
        memfillpoly(dst, pp, ni, e0, src, sp, op); // The call

    free(pp);
    m = u-a;
    continue;
```

⟨drawmesg() *when draw polygon, sanity check ni* 120b⟩≡ (120a)

```
if(ni < 0)
    error("negative count in polygon");
```

⟨drawmesg() *when draw polygon, sanity check j* 120c⟩≡ (120a)

```
if(j < 0)
    error("negative polygon line width");
```

⟨drawmesg() *when draw polygon, sanity check pp* 121a⟩≡ (120a)

```
if(pp == nil)
    error(Enomem);
```

⟨*function* drawcoord 121b⟩≡ (480a)

```
uchar*
drawcoord(uchar *p, uchar *maxp, int oldx, int *newx)
{
    int b, x;

    if(p >= maxp)
        error(Eshortdraw);
    b = *p++;
    x = b & 0x7F;
    if(b & 0x80){
        if(p+1 >= maxp)
            error(Eshortdraw);
        x |= *p++ << 7;
        x |= *p++ << 15;
        if(x & (1<<22))
            x |= ~0<<23;
    }else{
        if(b & 0x40)
            x |= ~0<<7;
        x += oldx;
    }
    *newx = x;
    return p;
}
```

8.2.4 Algorithm

⟨*function* mempoly 121c⟩≡ (435c)

```
void
mempoly(Memimage *dst, Point *vert, int nvert, int end0, int end1, int radius, Memimage *src, Point sp, int op)
{
    int i;
    //enum<EndLine>>
    int e0, e1;
    Point d;

    ⟨mempoly() sanity check nvert 122a⟩
    d = subpt(sp, vert[0]); // to compensate for addpt below
    for(i=1; i<nvert; i++){
        ⟨mempoly() set endlines e0 and e1 121d⟩
        memline(dst, vert[i-1], vert[i], e0, e1, radius, src, addpt(d, vert[i-1]), op);
    }
}
```

⟨mempoly() *set endlines e0 and e1* 121d⟩≡ (121c)

```
e0 = e1 = Enddisc; // for smooth junctions
if(i == 1)
    e0 = end0;
if(i == nvert-1)
    e1 = end1;
```

⟨mempoly() *sanity check nvert* 122a⟩≡ (121c)

```
if(nvert < 2)
    return;
```

⟨*function* memfillpoly 122b⟩≡ (433b)

```
void
memfillpoly(Memimage *dst, Point *vert, int nvert, int w, Memimage *src, Point sp, int op)
{
    _memfillpolysc(dst, vert, nvert, w, src, sp, false, 0, false, op);
}
```

Seg

⟨*struct* Seg 122c⟩≡ (433b)

```
struct Seg
{
    Point p0;
    Point p1;

    long d; // direction? 1 or -1?

    ⟨Seg other fields 123f⟩
};
```

_memfillpolysc()

⟨*function* _memfillpolysc 122d⟩≡ (433b)

```
void
_memfillpolysc(Memimage *dst, Point *vert, int nvert, int w, Memimage *src, Point sp, bool detail, int fixshift
{
    // array<Seg> (length = nvert+1)
    Seg *segtab;
    // array<Seg*> (length = nvert+2)
    Seg **seg;
    Point p0;
    int i;

    ⟨_memfillpolysc() sanity check nvert 123a⟩

    segtab = malloc((nvert+1)*sizeof(Seg));
    ⟨_memfillpolysc() sanity check segtab 123c⟩
    seg = malloc((nvert+2)*sizeof(Seg*));
    ⟨_memfillpolysc() sanity check seg 123b⟩

    sp.x = (sp.x - vert[0].x) >> fixshift;
    sp.y = (sp.y - vert[0].y) >> fixshift;

    p0 = vert[nvert-1]; // start from the end
    ⟨_memfillpolysc() adjust p0 if no fixshift 123d⟩
    for(i = 0; i < nvert; i++) {
        segtab[i].p0 = p0;
        p0 = vert[i];
        ⟨_memfillpolysc() adjust p0 if no fixshift 123d⟩
        segtab[i].p1 = p0;
        segtab[i].d = 1;
    }
    ⟨_memfillpolysc() adjust fixshift if no fixshift 123e⟩
```

```
    xscan(dst, seg, segtab, nvert, w, src, sp, detail, fixshift, clipped, op);
    ⟨_memfillpolysc() if detail 258b⟩

    free(seg);
    free(segtab);
}
```

⟨_memfillpolysc() *sanity check nvert* 123a⟩≡ (122d)

```
if(nvert == 0)
    return;
```

⟨_memfillpolysc() *sanity check seg* 123b⟩≡ (122d)

```
if(seg == nil)
    return;
```

⟨_memfillpolysc() *sanity check segtab* 123c⟩≡ (122d)

```
if(segtab == nil) {
    free(seg);
    return;
}
```

Fix shift arithmetics

⟨_memfillpolysc() *adjust p0 if no fixshift* 123d⟩≡ (122)

```
if(!fixshift) {
    p0.x <<= 1;
    p0.y <<= 1;
}
```

⟨_memfillpolysc() *adjust fixshift if no fixshift* 123e⟩≡ (122d)

```
if(!fixshift)
    fixshift = 1;
```

xscan()

⟨Seg *other fields* 123f⟩≡ (122c) 123g ▷

```
long num;
long den;
long dz;
long dzrem;
```

⟨Seg *other fields* 123g⟩+≡ (122c) ◁ 123f

```
long z;
long zerr;
```

⟨*function* xscan 123h⟩≡ (433b)

```
static void
xscan(Memimage *dst, Seg **seg, Seg *segtab, int nseg, int wind, Memimage *src, Point sp, bool detail, int fixs
{
    Seg **ep, **next, **p, **q, *s;
    long y, maxy, x, x2, xerr, xden, onehalf;
    long n, i, iy, cnt, ix, ix2, minx, maxx;
    Point pt;
    void (*fill)(Memimage*, int, int, int, Memimage*, Point, int);

    fill = fillline;
    USED(clipped);

    for(i=0, s=segtab, p=seg; i<nseg; i++, s++) {
```

```
    *p = s;
    if(s->p0.y == s->p1.y)
        continue;
    if(s->p0.y > s->p1.y) {
        // swap(s->p0, s->p1)
        pt = s->p0;
        s->p0 = s->p1;
        s->p1 = pt;

        s->d = -s->d;
    }
    s->num = s->p1.x - s->p0.x;
    s->den = s->p1.y - s->p0.y;

    s->dz = sdiv(s->num, s->den) << fixshift;
    s->dzrem = mod(s->num, s->den) << fixshift;
    s->dz += sdiv(s->dzrem, s->den);
    s->dzrem = mod(s->dzrem, s->den);
    p++;
}
n = p-seg;
if(n == 0)
    return;
*p = nil;
qsort(seg, n , sizeof(Seg*), ycompare);

onehalf = 0;
if(fixshift)
    onehalf = 1 << (fixshift-1);

minx = dst->clipr.min.x;
maxx = dst->clipr.max.x;

y = seg[0]->p0.y;
if(y < (dst->clipr.min.y << fixshift))
    y = dst->clipr.min.y << fixshift;
iy = (y + onehalf) >> fixshift;
y = (iy << fixshift) + onehalf;
maxy = dst->clipr.max.y << fixshift;

ep = next = seg;

while(y<maxy) {
    for(q = p = seg; p < ep; p++) {
        s = *p;
        if(s->p1.y < y)
            continue;
        s->z += s->dz;
        s->zerr += s->dzrem;
        if(s->zerr >= s->den) {
            s->z++;
            s->zerr -= s->den;
            if(s->zerr < 0 || s->zerr >= s->den)
                print("bad ratzerr1: %ld den %ld dzrem %ld\n", s->zerr, s->den, s->dzrem);
        }
        *q++ = s;
    }

    for(p = next; *p; p++) {
        s = *p;
```

```
        if(s->p0.y >= y)
            break;
        if(s->p1.y < y)
            continue;
        s->z = s->p0.x;
        s->z += smuldivmod(y - s->p0.y, s->num, s->den, &s->zerr);
        if(s->zerr < 0 || s->zerr >= s->den)
            print("bad ratzerr2: %ld den %ld ratdzrem %ld\n", s->zerr, s->den, s->dzrem);
        *q++ = s;
    }
    ep = q;
    next = p;

    if(ep == seg) {
        if(*next == 0)
            break;
        iy = (next[0]->p0.y + onehalf) >> fixshift;
        y = (iy << fixshift) + onehalf;
        continue;
    }

    zsort(seg, ep);

    for(p = seg; p < ep; p++) {
        cnt = 0;

        x = p[0]->z;
        xerr = p[0]->zerr;
        xden = p[0]->den;

        ix = (x + onehalf) >> fixshift;
        if(ix >= maxx)
            break;
        if(ix < minx)
            ix = minx;

        cnt += p[0]->d;
        p++;
        for(;;) {
            if(p == ep) {
                print("xscan: fill to infinity");
                return;
            }
            cnt += p[0]->d;
            if((cnt & wind) == 0)
                break;
            p++;
        }

        x2 = p[0]->z;
        ix2 = (x2 + onehalf) >> fixshift;
        if(ix2 <= minx)
            continue;
        if(ix2 > maxx)
            ix2 = maxx;

        ⟨xscan() if detail 258a⟩

        // the call
        (*fill)(dst, ix, ix2, iy, src, sp, op);
```

```
        }
        y += (1<<fixshift);
        iy++;
    }
}
```

fillxxx()

⟨*function* fillline 126a⟩≡ (433b)

```
static void
fillline(Memimage *dst, int xleft, int xright, int y, Memimage *src, Point p, int op)
{
    Rectangle r;

    r.min.x = xleft;
    r.min.y = y;
    r.max.x = xright;
    r.max.y = y+1;

    p.x += xleft;
    p.y += y;

    memdraw(dst, r, src, p, memopaque, p, op);
}
```

Arithmetics

⟨*function* mod 126b⟩≡ (433b)

```
static long
mod(long x, long y)
{
    long z;

    z = x%y;
    if((long)(((ulong)z)^((ulong)y)) > 0 || z == 0)
        return z;
    return z + y;
}
```

⟨*function* sdiv 126c⟩≡ (433b)

```
static long
sdiv(long x, long y)
{
    if((long)(((ulong)x)^((ulong)y)) >= 0 || x == 0)
        return x/y;

    return (x+((y>>30)|1))/y-1;
}
```

⟨*function* smuldivmod 126d⟩≡ (433b)

```
static long
smuldivmod(long x, long y, long z, long *mod)
{
    vlong vx;

    if(x == 0 || y == 0){
        *mod = 0;
        return 0;
    }
```

```
    vx = x;
    vx *= y;
    *mod = vx % z;
    if(*mod < 0)
        *mod += z; /* z is always >0 */
    if((vx < 0) == (z < 0))
        return vx/z;
    return -((-vx)/z);
}
```

Sorting

⟨*function* zsort 127a⟩≡ (433b)

```
static void
zsort(Seg **seg, Seg **ep)
{
    int done;
    Seg **q, **p, *s;

    if(ep-seg < 20) {
        /* bubble sort by z - they should be almost sorted already */
        q = ep;
        do {
            done = 1;
            q--;
            for(p = seg; p < q; p++) {
                if(p[0]->z > p[1]->z) {
                    s = p[0];
                    p[0] = p[1];
                    p[1] = s;
                    done = 0;
                }
            }
        } while(!done);
    } else {
        q = ep-1;
        for(p = seg; p < q; p++) {
            if(p[0]->z > p[1]->z) {
                qsort(seg, ep-seg, sizeof(Seg*), zcompare);
                break;
            }
        }
    }
}
```

⟨*function* ycompare 127b⟩≡ (433b)

```
static int
ycompare(void *a, void *b)
{
    Seg **s0 = a, **s1 = b;
    long y0, y1;

    y0 = (*s0)->p0.y;
    y1 = (*s1)->p0.y;

    if(y0 < y1)
        return -1;
    if(y0 == y1)
        return 0;
```

```
    return 1;
}
```

⟨*function* zcompare 128a⟩≡ (433b)

```
static int
zcompare(void *a, void *b)
{
    Seg **s0 = a, **s1 = b;
    long z0, z1;

    z0 = (*s0)->z;
    z1 = (*s1)->z;

    if(z0 < z1)
        return -1;
    if(z0 == z1)
        return 0;
    return 1;
}
```

8.3 Ellipses

8.3.1 API

⟨*function* ellipse 128b⟩≡ (410c)

```
void
ellipse(Image *dst, Point c, int a, int b, int thick, Image *src, Point sp)
{
    doellipse('e', dst, &c, a, b, thick, src, &sp, 0, 0, SoverD);
}
```

⟨*function* ellipseop 128c⟩≡ (410c)

```
void
ellipseop(Image *dst, Point c, int a, int b, int thick, Image *src, Point sp, Drawop op)
{
    doellipse('e', dst, &c, a, b, thick, src, &sp, 0, 0, op);
}
```

⟨*function* fillellipse 128d⟩≡ (410c)

```
void
fillellipse(Image *dst, Point c, int a, int b, Image *src, Point sp)
{
    doellipse('E', dst, &c, a, b, 0, src, &sp, 0, 0, SoverD);
}
```

⟨*function* fillellipseop 128e⟩≡ (410c)

```
void
fillellipseop(Image *dst, Point c, int a, int b, Image *src, Point sp, Drawop op)
{
    doellipse('E', dst, &c, a, b, 0, src, &sp, 0, 0, op);
}
```

8.3.2 Marshalling

⟨*function* doellipse 129a⟩≡ (410c)

```
static
void
doellipse(int cmd, Image *dst, Point *c, int xr, int yr, int thick, Image *src, Point *sp, int alpha, int phi,
{
    byte *a;

    _setdrawop(dst->display, op);

    // ellipse: 'e' dstid[4] srcid[4] center[2*4] a[4] b[4] thick[4] sp[2*4] alpha[4] phi[4]
    a = bufimage(dst->display, 1+4+4+2*4+4+4+4+2*4+2*4);
    ⟨doellipse() sanity check a 129b⟩
    a[0] = cmd;
    BPLONG(a+1, dst->id);
    BPLONG(a+5, src->id);
    BPLONG(a+9, c->x);
    BPLONG(a+13, c->y);
    BPLONG(a+17, xr);
    BPLONG(a+21, yr);
    BPLONG(a+25, thick);
    BPLONG(a+29, sp->x);
    BPLONG(a+33, sp->y);
    BPLONG(a+37, alpha);
    BPLONG(a+41, phi);
}
```

⟨doellipse() *sanity check a* 129b⟩≡ (129a)

```
if(a == nil){
    fprint(2, "image ellipse: %r\n");
    return;
}
```

8.3.3 Unmarshalling

⟨drawmesg() *cases* 129c⟩+≡ (71) ◁120a 151c▷

```
/* ellipse: 'e' dstid[4] srcid[4] center[2*4] a[4] b[4] thick[4] sp[2*4] alpha[4] phi[4]*/
case 'e':
case 'E':
    printmesg(fmt="LLPlllPll", a, false);
    m = 1+4+4+2*4+4+4+4+2*4+2*4;
    ⟨drawmesg() sanity check n with m 75d⟩
    dst = drawimage(client, a+1);
    dstid = BGLONG(a+1);
    src = drawimage(client, a+5);
    drawpoint(&p, a+9);
    e0 = BGLONG(a+17);
    e1 = BGLONG(a+21);
    ⟨drawmesg() when draw ellipse, sanity check widths 130a⟩
    j = BGLONG(a+25);
    ⟨drawmesg() when draw ellipse, sanity check j 130b⟩
    drawpoint(&sp, a+29);
    c = j;
    if(*a == 'E')
        c = -1;
    ox = BGLONG(a+37);
    oy = BGLONG(a+41);
    op = drawclientop(client);
```

```
    ⟨drawmesg() when draw ellipse, if ox 134e⟩
     else
         memellipse(dst, p, e0, e1, c, src, sp, op); // The call

    ⟨drawmesg() when draw ellipse, dstflush 326c⟩

     continue;
```

⟨drawmesg() *when draw ellipse, sanity check widths* 130a⟩≡ (129c)

```
  if(e0<0 || e1<0)
      error("invalid ellipse semidiameter");
```

⟨drawmesg() *when draw ellipse, sanity check j* 130b⟩≡ (129c)

```
  if(j < 0)
      error("negative ellipse thickness");
```

8.3.4 Algorithm

⟨*struct* ParamEllipse 130c⟩≡ (433a)

```
  struct ParamEllipse {
      Memimage *dst;
      Memimage *src;
      Point   c;
      int   t;
      Point   sp;
      Memimage *disc;
      int   op;
  };
```

⟨*struct* State 130d⟩≡ (433a)

```
  /*
   * denote residual error by e(x,y) = b^2*x^2 + a^2*y^2 - a^2*b^2
   * e(x,y) = 0 on ellipse, e(x,y) < 0 inside, e(x,y) > 0 outside
   */
  struct State {
      int a;
      int x;
      vlong a2; /* a^2 */
      vlong b2; /* b^2 */
      vlong b2x; /* b^2 * x */
      vlong a2y; /* a^2 * y */
      vlong c1;
      vlong c2; /* test criteria */
      vlong ee; /* ee = e(x+1/2,y-1/2) - (a^2+b^2)/4 */
      vlong dxe;
      vlong dye;
      vlong d2xe;
      vlong d2ye;
  };
```

⟨*function* newstate 130e⟩≡ (433a)

```
  static
  State*
  newstate(State *s, int a, int b)
  {
      s->x = 0;
      s->a = a;
      s->a2 = (vlong)(a*a);
```

```
    s->b2 = (vlong)(b*b);
    s->b2x = (vlong)0;
    s->a2y = s->a2*(vlong)b;
    s->c1 = -((s->a2>>2) + (vlong)(a&1) + s->b2);
    s->c2 = -((s->b2>>2) + (vlong)(b&1));
    s->ee = -s->a2y;
    s->dxe = (vlong)0;
    s->dye = s->ee<<1;
    s->d2xe = s->b2<<1;
    s->d2ye = s->a2<<1;
    return s;
}
```

⟨*function* memellipse 131⟩≡ (433a)

```
void
memellipse(Memimage *dst, Point c, int a, int b, int t, Memimage *src, Point sp, int op)
{
    State in, out;
    int y, inb, inx, outx, u;
    Param p;

    if(a < 0)
        a = -a;
    if(b < 0)
        b = -b;
    p.dst = dst;
    p.src = src;
    p.c = c;
    p.t = t;
    p.sp = subpt(sp, c);
    p.disc = nil;
    p.op = op;

    u = (t<<1)*(a-b);
    if(b<a && u>b*b || a<b && -u>a*a) {
/* if(b<a&&(t<<1)>b*b/a || a<b&&(t<<1)>a*a/b) # very thick */
        bellipse(b, newstate(&in, a, b), &p);
        return;
    }

    if(t < 0) {
        inb = -1;
        newstate(&out, a, y = b);
    } else {
        inb = b - t;
        newstate(&out, a+t, y = b+t);
    }
    if(t > 0)
        newstate(&in, a-t, inb);
    inx = 0;
    for( ; y>=0; y--) {
        outx = step(&out);
        if(y > inb) {
            erect(-outx, y, outx, y, &p);
            if(y != 0)
                erect(-outx, -y, outx, -y, &p);
            continue;
        }
        if(t > 0) {
            inx = step(&in);
```

```
            if(y == inb)
                inx = 0;
        } else if(inx > outx)
            inx = outx;
        erect(inx, y, outx, y, &p);
        if(y != 0)
            erect(inx, -y, outx, -y, &p);
        erect(-outx, y, -inx, y, &p);
        if(y != 0)
            erect(-outx, -y, -inx, -y, &p);
        inx = outx + 1;
    }
}
```

⟨*global* p00*(*(lib_graphics/libmemdraw/ellipse.c)*)* 132a⟩≡ (433a)

```
static Point p00 = {0, 0};
```

⟨*function* bellipse 132b⟩≡ (433a)

```
/*
 * a brushed ellipse
 */
static
void
bellipse(int y, State *s, Param *p)
{
    int t, ox, oy, x, nx;

    t = p->t;
    p->disc = allocmemimage(Rect(-t,-t,t+1,t+1), GREY1);
    if(p->disc == nil)
        return;
    memfillcolor(p->disc, DTransparent);
    memellipse(p->disc, p00, t, t, -1, memopaque, p00, p->op);
    oy = y;
    ox = 0;
    nx = x = step(s);
    do {
        while(nx==x && y-->0)
            nx = step(s);
        y++;
        eline(-x,-oy,-ox, -y, p);
        eline(ox,-oy,  x, -y, p);
        eline(-x,  y,-ox, oy, p);
        eline(ox,  y,  x, oy, p);
        ox = x+1;
        x = nx;
        y--;
        oy = y;
    } while(oy > 0);
}
```

⟨*function* erect 132c⟩≡ (433a)

```
/*
 * a rectangle with closed (not half-open) coordinates expressed
 * relative to the center of the ellipse
 */
static
void
erect(int x0, int y0, int x1, int y1, Param *p)
{
```

```
    Rectangle r;

/* print("R %d,%d %d,%d\n", x0, y0, x1, y1); /**/
    r = Rect(p->c.x+x0, p->c.y+y0, p->c.x+x1+1, p->c.y+y1+1);
    memdraw(p->dst, r, p->src, addpt(p->sp, r.min), memopaque, p00, p->op);
}
```

⟨*function* step 133a⟩≡ (433a)

```
/*
 * return x coord of rightmost pixel on next scan line
 */
static
int
step(State *s)
{
    while(s->x < s->a) {
        if(s->ee+s->b2x <= s->c1 || /* e(x+1,y-1/2) <= 0 */
           s->ee+s->a2y <= s->c2) { /* e(x+1/2,y) <= 0 (rare) */
            s->dxe += s->d2xe;
            s->ee += s->dxe;
            s->b2x += s->b2;
            s->x++;
            continue;
        }
        s->dye += s->d2ye;
        s->ee += s->dye;
        s->a2y -= s->a2;
        if(s->ee-s->a2y <= s->c2) { /* e(x+1/2,y-1) <= 0 */
            s->dxe += s->d2xe;
            s->ee += s->dxe;
            s->b2x += s->b2;
            return s->x++;
        }
        break;
    }
    return s->x;
}
```

⟨*function* epoint 133b⟩≡ (433a)

```
/*
 * a brushed point similarly specified
 */
static
void
epoint(int x, int y, Param *p)
{
    Point p0;
    Rectangle r;

/* print("P%d %d,%d\n", p->t, x, y); /**/
    p0 = Pt(p->c.x+x, p->c.y+y);
    r = Rpt(addpt(p0, p->disc->r.min), addpt(p0, p->disc->r.max));
    memdraw(p->dst, r, p->src, addpt(p->sp, r.min), p->disc, p->disc->r.min, p->op);
}
```

⟨*function* eline 133c⟩≡ (433a)

```
/*
 * a brushed horizontal or vertical line similarly specified
 */
static
```

```
void
eline(int x0, int y0, int x1, int y1, Param *p)
{
/* print("L%d %d,%d %d,%d\n", p->t, x0, y0, x1, y1); /**/
    if(x1 > x0+1)
        erect(x0+1, y0-p->t, x1-1, y1+p->t, p);
    else if(y1 > y0+1)
        erect(x0-p->t, y0+1, x1+p->t, y1-1, p);
    epoint(x0, y0, p);
    if(x1-x0 || y1-y0)
        epoint(x1, y1, p);
}
```

8.4 Arcs

8.4.1 API

⟨*function* arc 134a⟩≡ (410c)

```
void
arc(Image *dst, Point c, int a, int b, int thick, Image *src, Point sp, int alpha, int phi)
{
    alpha |= 1<<31;
    doellipse('e', dst, &c, a, b, thick, src, &sp, alpha, phi, SoverD);
}
```

⟨*function* arcop 134b⟩≡ (410c)

```
void
arcop(Image *dst, Point c, int a, int b, int thick, Image *src, Point sp, int alpha, int phi, Drawop op)
{
    alpha |= 1<<31;
    doellipse('e', dst, &c, a, b, thick, src, &sp, alpha, phi, op);
}
```

⟨*function* fillarc 134c⟩≡ (410c)

```
void
fillarc(Image *dst, Point c, int a, int b, Image *src, Point sp, int alpha, int phi)
{
    alpha |= 1<<31;
    doellipse('E', dst, &c, a, b, 0, src, &sp, alpha, phi, SoverD);
}
```

⟨*function* fillarcop 134d⟩≡ (410c)

```
void
fillarcop(Image *dst, Point c, int a, int b, Image *src, Point sp, int alpha, int phi, Drawop op)
{
    alpha |= 1<<31;
    doellipse('E', dst, &c, a, b, 0, src, &sp, alpha, phi, op);
}
```

8.4.2 Unmarshalling

⟨drawmesg() *when draw ellipse, if ox* 134e⟩≡ (129c)

```
/* high bit indicates arc angles are present */

if(ox & (1<<31)){
    if((ox & (1<<30)) == 0)
        ox &= ~(1<<31);
    memarc(dst, p, e0, e1, c, src, sp, ox, oy, op); // The call
}
```

8.4.3 Algorithm

⟨*enum* _anon_((lib_graphics/libmemdraw/arc.c)) 135a⟩≡ (422d)

```
/*
 * elarc(dst,c,a,b,t,src,sp,alpha,phi)
 *   draws the part of an ellipse between rays at angles alpha and alpha+phi
 *   measured counterclockwise from the positive x axis. other
 *   arguments are as for ellipse(dst,c,a,b,t,src,sp)
 */
enum
{
    R, T, L, B /* right, top, left, bottom */
};
```

⟨*global* corners 135b⟩≡ (422d)

```
static
Point corners[] = {
    {1,1},
    {-1,1},
    {-1,-1},
    {1,-1}
};
```

⟨*global* p00 135c⟩≡ (422d)

```
static
Point p00;
```

⟨*function* memarc 135d⟩≡ (422d)

```
/*
 * make a "wedge" mask covering the desired angle and contained in
 * a surrounding square; draw a full ellipse; intersect that with the
 * wedge to make a mask through which to copy src to dst.
 */
void
memarc(Memimage *dst, Point c, int a, int b, int t, Memimage *src, Point sp, int alpha, int phi, int op)
{
    int i, w, beta, tmp, c1, c2, m, m1;
    Rectangle rect;
    Point p, bnd[8];
    Memimage *wedge, *figure, *mask;

    if(a < 0)
        a = -a;
    if(b < 0)
        b = -b;
    w = t;
    if(w < 0)
        w = 0;
    alpha = -alpha;  /* compensate for upside-down coords */
    phi = -phi;
    beta = alpha + phi;
    if(phi < 0){
        tmp = alpha;
        alpha = beta;
        beta = tmp;
        phi = -phi;
    }
    if(phi >= 360){
        memellipse(dst, c, a, b, t, src, sp, op);
        return;
```

```
    }
    while(alpha < 0)
        alpha += 360;
    while(beta < 0)
        beta += 360;
    c1 = alpha/90 & 3; /* number of nearest corner */
    c2 = beta/90 & 3;
        /*
         * icossin returns point at radius ICOSSCALE.
         * multiplying by m1 moves it outside the ellipse
        */
    rect = Rect(-a-w, -b-w, a+w+1, b+w+1);
    m = rect.max.x; /* inradius of bounding square */
    if(m < rect.max.y)
        m = rect.max.y;
    m1 = (m+ICOSSCALE-1) >> 10;
    m = m1 << 10;  /* assure m1*cossin is inside */
    i = 0;
    bnd[i++] = Pt(0,0);
    icossin(alpha, &p.x, &p.y);
    bnd[i++] = mulpt(p, m1);
    for(;;) {
        bnd[i++] = mulpt(corners[c1], m);
        if(c1==c2 && phi<180)
            break;
        c1 = (c1+1) & 3;
        phi -= 90;
    }
    icossin(beta, &p.x, &p.y);
    bnd[i++] = mulpt(p, m1);

    figure = nil;
    mask = nil;
    wedge = allocmemimage(rect, GREY1);
    if(wedge == nil)
        goto Return;
    memfillcolor(wedge, DTransparent);
    memfillpoly(wedge, bnd, i, ~0, memopaque, p00, S);
    figure = allocmemimage(rect, GREY1);
    if(figure == nil)
        goto Return;
    memfillcolor(figure, DTransparent);
    memellipse(figure, p00, a, b, t, memopaque, p00, S);
    mask = allocmemimage(rect, GREY1);
    if(mask == nil)
        goto Return;
    memfillcolor(mask, DTransparent);
    memimagedraw(mask, rect, figure, rect.min, wedge, rect.min, S);
    c = subpt(c, dst->r.min);
    memdraw(dst, dst->r, src, subpt(sp, c), mask, subpt(p00, c), op);

    Return:
    freememimage(wedge);
    freememimage(figure);
    freememimage(mask);
}
```

Chapter 9

Fonts

9.1 Font files formats

9.2 Additional data structures

9.2.1 Font and font

⟨*struct* Font 137a⟩≡ (386d)

```
struct Font
{
    // ref_own<filename>, // e.g., /lib/font/bit/lucm/latin1.9.font
    char        *name;
    // ref<Display>
    Display     *display;

    short       height; /* max height of image, interline spacing */
    short       ascent; /* top of image to baseline */

    ⟨Font subfont spec fields 137d⟩
    ⟨Font subfont cache fields 165a⟩
    ⟨Font character cache fields 139a⟩
    ⟨Font other fields 161c⟩
};
```

⟨*global* font 137b⟩≡ (401a)

```
Font *font;
```

⟨Display *font fields* 137c⟩≡ (34e) 138a ▷

```
Font    *defaultfont;
```

⟨Font *subfont spec fields* 137d⟩≡ (137a)

```
// array<ref_own<Cachefont>> (length = Font.nsub)
Cachefont   **sub;  /* as read from file */
short       nsub;   /* number of subfonts */
```

⟨*struct* Cachefont 137e⟩≡ (391b)

```
struct Cachefont
{
    Rune  min; /* lowest rune value to be taken from subfont */
    Rune  max; /* highest rune value+1 to be taken from subfont */
    // option<int>, None = 0
    int  offset; /* position in subfont of character at min */

    // ref_own<string>, relative filename
```

```
    char  *name;   /* stored in font */
    // option<ref_own<filename>>, absolute filename, computed by subfontname()
    char  *subfontname;  /* to access subfont */

};
```

9.2.2 Subfont and Fontchar

⟨Display *font fields* 138a⟩+≡ (34e) ◁137c

```
Subfont *defaultsubfont;
```

⟨*struct* Subfont 138b⟩≡ (391b)

```
/*
 * Subfonts
 *
 * given char c, Subfont *f, Fontchar *i, and Point p, one says
 * i = f->info + c;
 * draw(b, Rect(p.x + i->left, p.y + i->top,
 *  p.x + i->left + ((i+1)->x - i->x), p.y + i->bottom),
 *  color, f->bits, Pt(i->x, i->top));
 * p.x += i->width;
 * to draw characters in the specified color (itself an Image) in Image b.
 */
struct Subfont
{
    // ref_own<string>
    char  *name;
    // ref_own<Image> ?
    Image  *bits; /* of font */

    // array<Fontchar> (size = Subfont.n + 1)
    Fontchar  *info; /* n+1 character descriptors */
    short  n;  /* number of chars in font */

    ⟨Subfont other fields 138d⟩

    // Extra
    int  ref;
};
```

⟨*struct* Fontchar 138c⟩≡ (391b)

```
struct Fontchar
{
    // character coordinates in Subfont.bits
    int  x;  /* left edge of bits */
    uchar top;  /* first non-zero scan-line */
    uchar bottom;  /* last non-zero scan-line + 1 */

    // adjustments to make on drawing point coordinates in destination
    schar left;  /* offset of baseline */
    uchar width;  /* width of baseline */
};
```

⟨Subfont *other fields* 138d⟩≡ (138b)

```
uchar  height;  /* height of image */
char  ascent;  /* top of image to baseline */
```

9.2.3 Font image cache

⟨Font *character cache fields* 139a⟩≡ (137a) 155a ▷
```
// ref_own<Image>, growing image
Image       *cacheimage;
```

⟨DImage *font fields* 139b⟩≡ (39f) 139d ▷
```
// growing_hash<Rune, ref_own<Fchar>> (size = DImage.nfchar)
FChar*      fchar;
int     nfchar;
```

⟨*struct* FChar 139c⟩≡ (476)
```
struct FChar
{
    // Rectangle in Font.cacheimage
    int     minx;   /* left edge of bits */
    int     maxx;   /* right edge of bits */
    uchar   miny;   /* first non-zero scan-line */
    uchar   maxy;   /* last non-zero scan-line + 1 */

    schar   left;   /* offset of baseline */
    uchar   width;  /* width of baseline */
};
```

⟨DImage *font fields* 139d⟩+≡ (39f) ◁ 139b
```
int     ascent;
```

9.3 Additional (user) initializations

⟨geninitdraw() *locals* 139e⟩≡ (55b) 143d ▷
```
Subfont *df;
```

⟨geninitdraw() *set up font* 139f⟩≡ (55b)
```
/*
 * Set up default font
 */

// Set up default subfont
df = getdefont(display);
display->defaultsubfont = df;
⟨geninitdraw() sanity check df 139g⟩

// Set up default font
⟨geninitdraw() read fontname if fontname was nil 143e⟩
⟨geninitdraw() if fontname still nil 143f⟩
else{
    font = openfont(display, fontname); /* BUG: grey fonts */
    ⟨geninitdraw() sanity check font 140a⟩
}
display->defaultfont = font;
```

⟨geninitdraw() *sanity check df* 139g⟩≡ (139f)
```
if(df == nil){
    fprint(2, "imageinit: can't open default subfont: %r\n");
Error:
    closedisplay(display);
    display = nil;
    return ERROR_NEG1;
}
```

⟨geninitdraw() *sanity check font* 140a⟩≡ (139f)

```
if(font == nil){
    fprint(2, "imageinit: can't open font %s: %r\n", fontname);
    goto Error;
}
```

9.3.1 Default subfont: getdefont()

⟨*global* sizeofdefont 140b⟩≡ (409a)

```
int sizeofdefont = sizeof defontdata;
```

⟨*function* getdefont 140c⟩≡ (414b)

```
Subfont*
getdefont(Display *d)
{
    Subfont *f; // TODO sf
    Image *i;
    Rectangle r;
    int ld;
    char *p, *hdr;
    int n;
    Fontchar *fc;

    p = (char*)defontdata;
    ⟨getdefont() adjust p if not word-aligned 141a⟩

    // header of the image
    ld = atoi(p+0*12);
    r.min.x = atoi(p+1*12);
    r.min.y = atoi(p+2*12);
    r.max.x = atoi(p+3*12);
    r.max.y = atoi(p+4*12);

    i = allocimage(d, r, drawld2chan[ld], false, 0);
    ⟨getdefont() sanity check i 141b⟩

    p += 5*12;
    n = loadimage(i, r, (byte*)p, (defontdata + sizeofdefont)-(uchar*)p);
    ⟨getdefont() sanity check n 141c⟩

    // header of the Fontchar
    hdr = p+n;
    n = atoi(hdr+0*12);
    p = hdr+3*12;
    fc = malloc(sizeof(Fontchar)*(n+1));
    ⟨getdefont() sanity check fc 141d⟩
    _unpackinfo(fc, (byte*)p, n);

    f = allocsubfont("*default*", n, atoi(hdr+1*12), atoi(hdr+2*12), fc, i);
    ⟨getdefont() sanity check f 141e⟩
    return f;
}
```

⟨*function* _unpackinfo 140d⟩≡ (409a)

```
void
_unpackinfo(Fontchar *fc, byte *p, int n)
{
    int j;
```

```
    for(j=0;  j<=n;  j++){
        fc->x      = p[0]|(p[1]<<8);
        fc->top    = p[2];
        fc->bottom = p[3];

        fc->left   = p[4];
        fc->width  = p[5];

        fc++;
        p += 6;
    }
}
```

⟨getdefont() *adjust p if not word-aligned* 141a⟩≡ (140c)

```
/*
 * make sure data is word-aligned.  this is true with Plan 9 compilers
 * but not in general.  the byte order is right because the data is
 * declared as char*, not ulong*.
 */
n = (int)(uvlong)p & 3;    /* stupid ape */
if(n != 0){
    memmove(p+(4-n), p, sizeofdefont-n);
    p += 4-n;
}
```

⟨getdefont() *sanity check i* 141b⟩≡ (140c)

```
if(i == nil)
    return nil;
```

⟨getdefont() *sanity check n* 141c⟩≡ (140c)

```
if(n < 0){
    freeimage(i);
    return nil;
}
```

⟨getdefont() *sanity check fc* 141d⟩≡ (140c)

```
if(fc == nil){
    freeimage(i);
    return nil;
}
```

⟨getdefont() *sanity check f* 141e⟩≡ (140c)

```
if(f == nil){
    freeimage(i);
    free(fc);
    return nil;
}
```

⟨*function* allocsubfont 141f⟩≡ (414b)

```
Subfont*
allocsubfont(char *name, int n, int height, int ascent, Fontchar *info, Image *i)
{
    Subfont *f; // TODO sf

    ⟨allocsubfont() sanity check height 142a⟩

    f = malloc(sizeof(Subfont));
    ⟨allocsubfont() sanity check f 142b⟩
    f->n = n;
    f->height = height;
```

```
    f->ascent = ascent;
    f->info = info;
    f->bits = i;
    f->ref = 1;
    ⟨allocsubfont() install subfont if name 169f⟩
    else
        f->name = nil;
    return f;
}
```

⟨allocsubfont() *sanity check height* 142a⟩≡ (141f)

```
assert(height != 0 /* allocsubfont */);
```

⟨allocsubfont() *sanity check f* 142b⟩≡ (141f)

```
if(f == nil)
    return nil;
```

9.3.2 Default font

openfont()

⟨*function* openfont 142c⟩≡ (412b)

```
Font*
openfont(Display *d, char *name)
{
    Font *fnt;
    fdt fd;
    int i, n;
    char *buf;
    Dir *dir;

    fd = open(name, OREAD);
    ⟨openfont() sanity check fd 142d⟩

    // n = filename_size(fd)
    dir = dirfstat(fd);
    ⟨openfont() sanity check dir 143a⟩
    n = dir->length;
    free(dir);

    buf = malloc(n+1);
    ⟨openfont() sanity check buf 143b⟩
    buf[n] = '\0';
    i = read(fd, buf, n);
    close(fd);
    ⟨openfont() sanity check i 143c⟩

    fnt = buildfont(d, buf, name);

    free(buf);
    return fnt;
}
```

⟨openfont() *sanity check fd* 142d⟩≡ (142c)

```
if(fd < 0)
    return nil;
```

⟨openfont() *sanity check dir* 143a⟩≡ (142c)

```
if(dir == nil){
Err0:
    close(fd);
    return nil;
}
```

⟨openfont() *sanity check buf* 143b⟩≡ (142c)

```
if(buf == nil)
    goto Err0;
```

⟨openfont() *sanity check i* 143c⟩≡ (142c)

```
if(i != n){
    free(buf);
    return nil;
}
```

/env/font

⟨geninitdraw() *locals* 143d⟩+≡ (55b) ◁139e

```
fdt fd;
int n;
char buf[128];
```

⟨geninitdraw() *read fontname if fontname was nil* 143e⟩≡ (139f)

```
if(fontname == nil){
    fd = open("/env/font", OREAD);
    if(fd >= 0){
        n = read(fd, buf, sizeof(buf));
        if(n>0 && n < sizeof buf-1){
            buf[n] = '\0';
            fontname = buf;
        }
        close(fd);
    }
}
```

font from default subfont

⟨geninitdraw() *if fontname still nil* 143f⟩≡ (139f)

```
/*
 * Build fonts with caches==depth of screen, for speed.
 * If conversion were faster, we'd use 0 and save memory.
 */
if(fontname == nil){
    snprint(buf, sizeof buf, "%d %d\n0 %d\t%s\n",
        df->height, df->ascent,
        df->n-1, deffontname);
    //BUG: Need something better for this installsubfont("*default*", df);
    font = buildfont(display, buf, deffontname);
    ⟨geninitdraw() sanity check font part2 143h⟩
}
```

⟨*global* deffontname 143g⟩≡ (400d)

```
static char deffontname[] = "*default*";
```

⟨geninitdraw() *sanity check font part2* 143h⟩≡ (143f)

```
if(font == nil){
    fprint(2, "imageinit: can't open default font: %r\n");
    goto Error;
}
```

9.4 Font allocation

9.4.1 buildfont()

⟨*function* buildfont 144⟩≡ (412b)

```
Font*
buildfont(Display *d, char *buf, char *name)
{
    Font *fnt;
    char *s, *t;
    ulong min, max;
    Cachefont *c;
    int offset;
    ⟨buildfont() other locals 145g⟩

    s = buf;

    fnt = malloc(sizeof(Font));
    ⟨buildfont() sanity check fnt 145c⟩
    memset(fnt, 0, sizeof(Font));

    fnt->display = d;
    fnt->name = strdup(name);

    ⟨buildfont() allocate cache 155f⟩
    ⟨buildfont() initialize cache 155b⟩

    fnt->height = strtol(s, &s, 0);
    s = skip(s);
    fnt->ascent = strtol(s, &s, 0);
    s = skip(s);
    ⟨buildfont() sanity check fnt fields part2 145e⟩

    fnt->width = 0;

    fnt->sub = nil;
    fnt->nsub = 0;
    do{
        /* must be looking at a number now */
        ⟨buildfont() sanity check s content 145f⟩
        min = strtol(s, &s, 0);
        s = skip(s);
        /* must be looking at a number now */
        ⟨buildfont() sanity check s content 145f⟩
        max = strtol(s, &s, 0);
        s = skip(s);
        ⟨buildfont() sanity check min and max 146a⟩

        ⟨buildfont() set optional offset 145a⟩

        fnt->sub = realloc(fnt->sub, (fnt->nsub+1)*sizeof(Cachefont*));
        ⟨buildfont() sanity check fnt fields part3 146b⟩

        c = malloc(sizeof(Cachefont));
        ⟨buildfont() sanity check c 146c⟩
        fnt->sub[fnt->nsub] = c;
        fnt->nsub++;

        c->min = min;
        c->max = max;
```

```
        c->offset = offset;

        t = s;
        while(*s && *s!=' ' && *s!='\n' && *s!='\t')
            s++;
        *s++ = '\0';

        c->name = strdup(t);
        c->subfontname = nil; // full filename computed lazily later
        ⟨buildfont() sanity check c name field 146d⟩
        s = skip(s);
    } while(*s);
    return fnt;
}
```

⟨buildfont() *set optional offset* 145a⟩≡ (144)

```
t = s;
offset = strtol(s, &t, 0);
if(t>s && (*t==' ' || *t=='\t' || *t=='\n'))
    s = skip(t);
else
    offset = 0;
```

⟨*function* skip 145b⟩≡ (412b)

```
static char*
skip(char *s)
{
    while(*s==' ' || *s=='\n' || *s=='\t')
        s++;
    return s;
}
```

⟨buildfont() *sanity check fnt* 145c⟩≡ (144)

```
if(fnt == nil)
    return nil;
```

⟨buildfont() *sanity check fnt fields part1* 145d⟩≡ (156a)

```
if(fnt->name==nil || fnt->cache==nil || fnt->subf==nil){
Err2:
    free(fnt->name);
    free(fnt->cache);
    free(fnt->subf);
    free(fnt->sub);
    free(fnt);
    return 0;
}
```

⟨buildfont() *sanity check fnt fields part2* 145e⟩≡ (144)

```
if(fnt->height <= 0 || fnt->ascent <= 0){
    werrstr("bad height or ascent in font file");
    goto Err2;
}
```

⟨buildfont() *sanity check s content* 145f⟩≡ (144)

```
if(*s<'0' || '9'<*s){
    werrstr(badform, s-buf);
    goto Err3;
}
```

⟨buildfont() *other locals* 145g⟩≡ (144)

```
char badform[] = "bad font format: number expected (char position %d)";
```

⟨buildfont() *sanity check min and max* 146a⟩≡ (144)

```
if(*s=='\0' || min>=Runemax || max>=Runemax || min>max){
    werrstr("illegal subfont range");
Err3:
    freefont(fnt);
    return nil;
}
```

⟨buildfont() *sanity check fnt fields part3* 146b⟩≡ (144)

```
if(fnt->sub == nil){
    /* realloc manual says fnt->sub may have been destroyed */
    fnt->nsub = 0;
    goto Err3;
}
```

⟨buildfont() *sanity check c* 146c⟩≡ (144)

```
if(c == nil)
    goto Err3;
```

⟨buildfont() *sanity check c name field* 146d⟩≡ (144)

```
if(c->name == nil){
    free(c);
    goto Err3;
}
```

9.4.2 freefont()

⟨*function* freefont 146e⟩≡ (412b)

```
void
freefont(Font *f)
{
    int i;
    Cachefont *c;
    Subfont *s;

    ⟨freefont() sanity check f 146f⟩

    for(i=0; i<f->nsub; i++){
        c = f->sub[i];
        free(c->name);
        free(c->subfontname);
        free(c);
    }
    free(f->sub);

    ⟨freefont() free cache 155c⟩

    free(f->name);
    free(f);
}
```

⟨freefont() *sanity check f* 146f⟩≡ (146e)

```
if(f == nil)
    return;
```

⟨*function* freesubfont 147a⟩≡ (414b)

```
void
freesubfont(Subfont *f)
{
    ⟨freesubfont() sanity check f 147b⟩
    f->ref--;
    if(f->ref > 0)
        return;

    // else
    uninstallsubfont(f);
    free(f->info); /* note: f->info must have been malloc'ed! */
    freeimage(f->bits);
    free(f);
}
```

⟨freesubfont() *sanity check f* 147b⟩≡ (147a)

```
if(f == nil)
    return;
```

9.5 Drawing text

9.5.1 API

⟨*function* string 147c⟩≡ (413c)

```
Point
string(Image *dst, Point pt, Image *src, Point sp, Font *f, char *s)
{
    return _string(dst, pt, src, sp, f, s, nil, 1<<24, dst->clipr, nil, ZP, SoverD);
}
```

⟨*function* stringop 147d⟩≡ (413c)

```
Point
stringop(Image *dst, Point pt, Image *src, Point sp, Font *f, char *s, Drawop op)
{
    return _string(dst, pt, src, sp, f, s, nil, 1<<24, dst->clipr, nil, ZP, op);
}
```

⟨*function* stringn 147e⟩≡ (413c)

```
Point
stringn(Image *dst, Point pt, Image *src, Point sp, Font *f, char *s, int len)
{
    return _string(dst, pt, src, sp, f, s, nil, len, dst->clipr, nil, ZP, SoverD);
}
```

⟨*function* stringnop 147f⟩≡ (413c)

```
Point
stringnop(Image *dst, Point pt, Image *src, Point sp, Font *f, char *s, int len, Drawop op)
{
    return _string(dst, pt, src, sp, f, s, nil, len, dst->clipr, nil, ZP, op);
}
```

⟨*function* runestring 147g⟩≡ (413c)

```
Point
runestring(Image *dst, Point pt, Image *src, Point sp, Font *f, Rune *r)
{
    return _string(dst, pt, src, sp, f, nil, r, 1<<24, dst->clipr, nil, ZP, SoverD);
}
```

⟨*function* runestringop 148a⟩≡ (413c)

```
Point
runestringop(Image *dst, Point pt, Image *src, Point sp, Font *f, Rune *r, Drawop op)
{
    return _string(dst, pt, src, sp, f, nil, r, 1<<24, dst->clipr, nil, ZP, op);
}
```

⟨*function* runestringn 148b⟩≡ (413c)

```
Point
runestringn(Image *dst, Point pt, Image *src, Point sp, Font *f, Rune *r, int len)
{
    return _string(dst, pt, src, sp, f, nil, r, len, dst->clipr, nil, ZP, SoverD);
}
```

⟨*function* runestringnop 148c⟩≡ (413c)

```
Point
runestringnop(Image *dst, Point pt, Image *src, Point sp, Font *f, Rune *r, int len, Drawop op)
{
    return _string(dst, pt, src, sp, f, nil, r, len, dst->clipr, nil, ZP, op);
}
```

⟨*function* stringbg 148d⟩≡ (413c)

```
Point
stringbg(Image *dst, Point pt, Image *src, Point sp, Font *f, char *s, Image *bg, Point bgp)
{
    return _string(dst, pt, src, sp, f, s, nil, 1<<24, dst->clipr, bg, bgp, SoverD);
}
```

⟨*function* stringbgop 148e⟩≡ (413c)

```
Point
stringbgop(Image *dst, Point pt, Image *src, Point sp, Font *f, char *s, Image *bg, Point bgp, int op)
{
    return _string(dst, pt, src, sp, f, s, nil, 1<<24, dst->clipr, bg, bgp, op);
}
```

⟨*function* stringnbg 148f⟩≡ (413c)

```
Point
stringnbg(Image *dst, Point pt, Image *src, Point sp, Font *f, char *s, int len, Image *bg, Point bgp)
{
    return _string(dst, pt, src, sp, f, s, nil, len, dst->clipr, bg, bgp, SoverD);
}
```

⟨*function* stringnbgop 148g⟩≡ (413c)

```
Point
stringnbgop(Image *dst, Point pt, Image *src, Point sp, Font *f, char *s, int len, Image *bg, Point bgp, int op
{
    return _string(dst, pt, src, sp, f, s, nil, len, dst->clipr, bg, bgp, op);
}
```

⟨*function* runestringbg 148h⟩≡ (413c)

```
Point
runestringbg(Image *dst, Point pt, Image *src, Point sp, Font *f, Rune *r, Image *bg, Point bgp)
{
    return _string(dst, pt, src, sp, f, nil, r, 1<<24, dst->clipr, bg, bgp, SoverD);
}
```

⟨*function* runestringbgop 148i⟩≡ (413c)

```
Point
runestringbgop(Image *dst, Point pt, Image *src, Point sp, Font *f, Rune *r, Image *bg, Point bgp, int op)
{
    return _string(dst, pt, src, sp, f, nil, r, 1<<24, dst->clipr, bg, bgp, op);
}
```

⟨*function* runestringnbg 149a⟩≡ (413c)

```
Point
runestringnbg(Image *dst, Point pt, Image *src, Point sp, Font *f, Rune *r, int len, Image *bg, Point bgp)
{
    return _string(dst, pt, src, sp, f, nil, r, len, dst->clipr, bg, bgp, SoverD);
}
```

⟨*function* runestringnbgop 149b⟩≡ (413c)

```
Point
runestringnbgop(Image *dst, Point pt, Image *src, Point sp, Font *f, Rune *r, int len, Image *bg, Point bgp, in
{
    return _string(dst, pt, src, sp, f, nil, r, len, dst->clipr, bg, bgp, op);
}
```

9.5.2 Marshalling

⟨*constant* Max 149c⟩≡ (413b)

```
Max = 100
```

⟨*function* _string 149d⟩≡ (413c)

```
Point
_string(Image *dst, Point pt, Image *src, Point sp, Font *f, char *s, Rune *r, int len, Rectangle clipr, Image
{
    Rune **rptr;
    char **sptr;
    int m, n, max;
    ushort cbuf[Max], *c, *ec;
    byte *b;
    int wid;
    ⟨_string() other locals 168b⟩

    ⟨_string() non unicode string handling, set sptr and rptr 150a⟩
    else
      rptr = &r;

    while((*s || *r) && len){ // PAD: len > 0 in recent
        ⟨_string() set max 150b⟩

        // will trigger many calls to loadchar() for the same subfont
        n = cachechars(f, sptr, rptr, cbuf, max, &wid, &subfontname);

        if(n > 0){
            _setdrawop(dst->display, op);

            // string: 's' dstid[4] srcid[4] fontid[4] P[2*4] clipr[4*4] sp[2*4] ni[2] ni*(index[2])

            m = 47+2*n;
            ⟨_string() if bg part1 150c⟩
            b = bufimage(dst->display, m);
            ⟨_string() sanity check b 151a⟩
            ⟨_string() if bg part2 150d⟩
            else
                b[0] = 's';
            BPLONG(b+1, dst->id);
            BPLONG(b+5, src->id);
            BPLONG(b+9, f->cacheimage->id);
            BPLONG(b+13, pt.x);
            BPLONG(b+17, pt.y + f->ascent); // + ascent!
            BPLONG(b+21, clipr.min.x);
```

```
            BPLONG(b+25, clipr.min.y);
            BPLONG(b+29, clipr.max.x);
            BPLONG(b+33, clipr.max.y);
            BPLONG(b+37, sp.x);
            BPLONG(b+41, sp.y);
            BPSHORT(b+45, n);
            b += 47;
            ⟨_string() if bg part3 150e⟩
            // index of the set of characters to draw (hashcode of Rune)
            ec = &cbuf[n];
            for(c=cbuf; c<ec; c++, b+=2)
                BPSHORT(b, *c);

            pt.x += wid;
            bgp.x += wid;

            agefont(f);

            len -= n; // progress
        }
        ⟨_string() if subfontname 168c⟩
    }
    return pt;
}
```

⟨_string() *non unicode string handling, set sptr and rptr* 150a⟩≡ (154a 149d)

```
if(s == nil){
    s = "";
    sptr = nil;
}else
    sptr = &s;

if(r == nil){
    r = (Rune*) L"";
    rptr = nil;
}
```

⟨_string() *set max* 150b⟩≡ (154a 149d)

```
max = Max;
if(len < max)
    max = len;
```

⟨_string() *if bg part1* 150c⟩≡ (149d)

```
// stringbg: 'x' dstid[4] srcid[4] fontid[4] P[2*4] clipr[4*4] sp[2*4] ni[2] bgid[4] bgpt[2*4] ni*(index[2])
if(bg)
    m += 4+2*4;
```

⟨_string() *if bg part2* 150d⟩≡ (149d)

```
if(bg)
    b[0] = 'x';
```

⟨_string() *if bg part3* 150e⟩≡ (149d)

```
if(bg){
    BPLONG(b, bg->id);
    BPLONG(b+4, bgp.x);
    BPLONG(b+8, bgp.y);
    b += 12;
}
```

⟨_string() *sanity check b* 151a⟩≡ (149d)

```
if(b == nil){
    fprint(2, "string: %r\n");
    break;
}
```

9.5.3 Unmarshalling

⟨drawmesg() *locals* 151b⟩+≡ (71) ◁119e 160c▷

```
DImage *font;
Memimage  *bg;
int ci;
```

⟨drawmesg() *cases* 151c⟩+≡ (71) ◁129c 160d▷

```
/* string: 's' dstid[4] srcid[4] fontid[4] P[2*4] clipr[4*4] sp[2*4] ni[2] ni*(index[2]) */
/* stringbg: 'x' dstid[4] srcid[4] fontid[4] P[2*4] clipr[4*4] sp[2*4] ni[2] bgid[4] bgpt[2*4] ni*(index[2]) */
case 's':
case 'x':
    printmesg(fmt="LLLPRPs", a, false);
    m = 1+4+4+4+2*4+4*4+2*4+2;
    ⟨drawmesg() when draw string, if bg part1 151f⟩
    ⟨drawmesg() sanity check n with m 75d⟩
    dst = drawimage(client, a+1);
    dstid = BGLONG(a+1);
    src = drawimage(client, a+5);
    font = drawlookup(client, BGLONG(a+9), true);
    ⟨drawmesg() when draw string, sanity check font 152b⟩
    drawpoint(&p, a+13);
    drawrectangle(&r, a+21);
    drawpoint(&sp, a+37);
    ni = BGSHORT(a+45);
    u = a+m;
    m += ni*2;
    ⟨drawmesg() sanity check n with m 75d⟩
    ⟨drawmesg() change dst clipr 151d⟩
    op = drawclientop(client);
    bg = dst;
    ⟨drawmesg() when draw string, if paint background 152a⟩
    q = p;
    while(--ni >= 0){
        ci = BGSHORT(u);
        ⟨drawmesg() when draw string, sanity check ci 152c⟩
        q = drawchar(dst, bg, q, src, &sp, font, ci, op); // The call
        u += 2;
    }
    ⟨drawmesg() restore dst clipr 151e⟩
    ⟨drawmesg() when draw text, dstflush 326d⟩
    continue;
```

⟨drawmesg() *change dst clipr* 151d⟩≡ (151c)

```
clipr = dst->clipr;
dst->clipr = r;
```

⟨drawmesg() *restore dst clipr* 151e⟩≡ (151c)

```
dst->clipr = clipr;
```

⟨drawmesg() *when draw string, if bg part1* 151f⟩≡ (151c)

```
if(*a == 'x')
    m += 4+2*4;
```

⟨drawmesg() *when draw string, if paint background* 152a⟩≡ (151c)

```
if(*a == 'x'){
    /* paint background */
    bg = drawimage(client, a+47);
    drawpoint(&q, a+51);
    r.min.x = p.x;
    r.min.y = p.y - font->ascent;
    r.max.x = p.x;
    r.max.y = r.min.y+Dy(font->image->r);
    j = ni;
    while(--j >= 0){
        ci = BGSHORT(u);
        ⟨drawmesg() when draw string, sanity check ci 152c⟩
        r.max.x += font->fchar[ci].width;
        u += 2;
    }
    memdraw(dst, r, bg, q, memopaque, ZP, op); // The call
    u -= 2*ni;
}
```

⟨drawmesg() *when draw string, sanity check font* 152b⟩≡ (151c)

```
if(font == nil)
    error(Enodrawimage);
if(font->nfchar == 0)
    error(Enotfont);
```

⟨drawmesg() *when draw string, sanity check ci* 152c⟩≡ (152a 151c)

```
if(ci<0 || ci>=font->nfchar){
    dst->clipr = clipr;
    error(Eindex);
}
```

9.5.4 Algorithm

⟨*function* drawchar 152d⟩≡ (480a)

```
Point
drawchar(Memimage *dst, Memimage *rdst, Point p, Memimage *src, Point *sp, DImage *font, int index, int op)
{
    FChar *fc;
    Rectangle r;
    Point sp1;
    ⟨drawchar() other locals 324d⟩

    fc = &font->fchar[index];

    r.min.x = p.x + fc->left;
    r.min.y = p.y - (font->ascent - fc->miny);
    r.max.x = r.min.x + (fc->maxx - fc->minx);
    r.max.y = r.min.y + (fc->maxy - fc->miny);

    sp1.x = sp->x + fc->left;
    sp1.y = sp->y + fc->miny;

    ⟨drawchar() optimization when possible 324e⟩
    else{
    fallback:
        memdraw(dst, r, src, sp1, font->image, Pt(fc->minx, fc->miny), op);
    }
```

```
    p.x   += fc->width;
    sp->x += fc->width;
    return p;
}
```

9.6 Computing text width

9.6.1 API

⟨*function* stringwidth 153a⟩≡ (414a)

```
int
stringwidth(Font *f, char *s)
{
    return _stringnwidth(f, s, nil, 1<<24);
}
```

⟨*function* stringnwidth 153b⟩≡ (414a)

```
int
stringnwidth(Font *f, char *s, int len)
{
    return _stringnwidth(f, s, nil, len);
}
```

⟨*function* stringsize 153c⟩≡ (414a)

```
Point
stringsize(Font *f, char *s)
{
    return Pt(_stringnwidth(f, s, nil, 1<<24), f->height);
}
```

⟨*function* runestringnwidth 153d⟩≡ (414a)

```
int
runestringnwidth(Font *f, Rune *r, int len)
{
    return _stringnwidth(f, nil, r, len);
}
```

⟨*function* runestringwidth 153e⟩≡ (414a)

```
int
runestringwidth(Font *f, Rune *r)
{
    return _stringnwidth(f, nil, r, 1<<24);
}
```

⟨*function* runestringsize 153f⟩≡ (414a)

```
Point
runestringsize(Font *f, Rune *r)
{
    return Pt(_stringnwidth(f, nil, r, 1<<24), f->height);
}
```

9.6.2 Algorithm

⟨*enum* _anon_*(*(lib_graphics/libdraw/stringwidth.c)*)* 153g⟩≡ (414a)

```
enum { Max = 64 };
```

⟨*function* _stringnwidth 154a⟩≡ (414a)

```
int
_stringnwidth(Font *f, char *s, Rune *r, int len)
{
    Rune **rptr;
    char **sptr;
    int wid, twid;
    int n, max, l;
    ushort cbuf[Max];
    char *subfontname;
    ⟨_stringnwidth() other locals 154b⟩

    ⟨_string() non unicode string handling, set sptr and rptr 150a⟩
    else
        rptr = &r;

    twid = 0;
    while(len > 0 && (*s || *r)){
        ⟨_string() set max 150b⟩
        n = 0;
        while((l = cachechars(f, sptr, rptr, cbuf, max, &wid, &subfontname)) <= 0){
            ⟨_stringnwidth() if cachechars failed too much 154c⟩
            ⟨_stringnwidth() if subfontname 168h⟩
        }
        agefont(f);

        twid += wid;
        len -= l; // progress
    }
    return twid;
}
```

⟨_stringnwidth() *other locals* 154b⟩≡ (154a) 168g ▷

```
char *name;
Rune rune;
```

⟨_stringnwidth() *if cachechars failed too much* 154c⟩≡ (154a)

```
if(++n > 10){
    if(*r)
        rune = *r;
    else
        chartorune(&rune, s);
    if(f->name != nil)
        name = f->name;
    else
        name = "unnamed font";
    fprint(2, "stringwidth: bad character set for rune 0x%.4ux in %s\n", rune, name);
    return twid;
}
```

9.7 Character cache

⟨*enum* misc 154d⟩≡ (391b)

```
enum
{
    /* starting values */
    ⟨constant NFCACHE 155g⟩
    ⟨constant NFLOOK 155h⟩
```

```
    ⟨constant NFSUBF 165d⟩

    /* max value */
    ⟨constant MAXFCACHE 155i⟩
    ⟨constant MAXSUBF 165e⟩

    /* deltas */
    ⟨constant DSUBF 165f⟩

    /* expiry ages */
    SUBFAGE = 10000,
    CACHEAGE = 10000
};
```

⟨Font *character cache fields* 155a⟩+≡ (137a) ◁139a 155d▷

```
ulong       age;    /* increasing counter; used for LRU */
```

⟨buildfont() *initialize cache* 155b⟩≡ (144) 156a▷

```
fnt->age = 1;
```

⟨freefont() *free cache* 155c⟩≡ (146e) 156b▷

```
freeimage(f->cacheimage);
```

9.7.1 Cacheinfo

⟨Font *character cache fields* 155d⟩+≡ (137a) ◁155a

```
// growing_hash<Rune, Cacheinfo> (size = Font.ncache, bucketsize = NFLOOK)
Cacheinfo   *cache;
int     ncache; /* size of cache */
```

⟨*struct* Cacheinfo 155e⟩≡ (391b)

```
struct Cacheinfo
{
    // the key
    Rune  value; /* value of character at this slot in cache */

    // the values
    ushort  x;  /* left edge of bits */
    byte  width;  /* width of baseline */
    schar  left;  /* offset of baseline */

    ushort  age;
};
```

⟨buildfont() *allocate cache* 155f⟩≡ (144) 165c▷

```
fnt->ncache = NFCACHE+NFLOOK;
fnt->cache = malloc(fnt->ncache * sizeof(fnt->cache[0]));
```

⟨*constant* NFCACHE 155g⟩≡ (154d)

```
LOG2NFCACHE = 6,
NFCACHE = (1<<LOG2NFCACHE), /* #chars cached */
```

⟨*constant* NFLOOK 155h⟩≡ (154d)

```
NFLOOK = 5,   /* #chars to scan in cache */
```

⟨*constant* MAXFCACHE 155i⟩≡ (154d)

```
MAXFCACHE = 1024+NFLOOK,  /* upper limit */
```

⟨buildfont() *initialize cache* 156a⟩+≡ (144) ◁155b

```
⟨buildfont() sanity check fnt fields part1 145d⟩
memset(fnt->subf, 0, fnt->nsubf * sizeof(fnt->subf[0]));
memset(fnt->cache, 0, fnt->ncache * sizeof(fnt->cache[0]));
```

⟨freefont() *free cache* 156b⟩+≡ (146e) ◁155c 165g▷

```
free(f->cache);
```

9.7.2 Loading characters: cachechars()

⟨*function* cachechars 156c⟩≡ (412a)

```
int
cachechars(Font *f, char **ss, Rune **rr, ushort *cp, int max, int *wp, char **subfontname)
{
    Rune *rp;
    char *sp;
    Rune r;
    int w, rw;
    int wid = 0;
    int i;
    int h; // index in Font.cache
    int sh; // rune hashcode
    Cacheinfo *c, *ec;
    ⟨cachechars() other locals 157b⟩

    ⟨cachechars() non unicode string handling part1 157a⟩
    else{
        sp = "";
        rp = *rr;
    }
    *subfontname = nil;

    for(i=0; i<max && (*sp || *rp); sp+=w, rp+=rw){
        ⟨cachechars() non unicode string handling part2 157c⟩
        else{
            r = *rp;
            w = 0;
            rw = 1;
        }

        // sh = hash_code(r)
        sh = (17 * (uint)r) & (f->ncache-NFLOOK-1);

        // c,h = lookup(r, sh, f->cache)
        c = &f->cache[sh];
        ec = c+NFLOOK;
        h = sh;
        while(c < ec){
            if(c->value==r && c->age)
                goto Found;
            c++;
            h++;
        }
        // this can break out of the loop
        ⟨cachechars() when rune not in cache, loads it 157f⟩

        Found:
        wid += c->width;
        c->age = f->age;
```

```
        cp[i] = h;
        i++;
    }
    ⟨cachechars() non unicode string handling part3 157d⟩
    else
        *rr = rp;
    *wp = wid;

    return i;
}
```

⟨cachechars() *non unicode string handling part1* 157a⟩≡ (156c)

```
if(ss){
    sp = *ss;
    rp = L"";
}
```

⟨cachechars() *other locals* 157b⟩≡ (156c) 157e ▷

```
Rune vr;
```

⟨cachechars() *non unicode string handling part2* 157c⟩≡ (156c)

```
if(ss){
    r = *(uchar*)sp;
    if(r < Runeself)
        w = 1;
    else{
        w = chartorune(&vr, sp);
        r = vr;
    }
    rw = 0;
}
```

⟨cachechars() *non unicode string handling part3* 157d⟩≡ (156c)

```
if(ss)
    *ss = sp;
```

9.7.3 Loading one new character: `loadchar()`

⟨cachechars() *other locals* 157e⟩+≡ (156c) ◁ 157b 158b ▷

```
int ld;
```

⟨cachechars() *when rune not in cache, loads it* 157f⟩≡ (156c)

```
⟨cachechars() find oldest entry c with age a in cache 158c⟩
⟨cachechars() if age too recent then resize cache 161e⟩
⟨cachechars() if same age 161f⟩
// else

ld = loadchar(f, r, c, h, i, subfontname);
⟨cachechars() if could not load char 157g⟩
// else
c = &f->cache[h]; /* may have reallocated f->cache */
```

⟨cachechars() *if could not load char* 157g⟩≡ (157f)

```
if(ld <= 0){
    ⟨cachechars() if loadchar failed 158a⟩
    break;
}
```

⟨cachechars() *if loadchar failed* 158a⟩≡ (157g)

```
if(ld == 0) // when failed, but why can fail?
    continue;
```

⟨cachechars() *other locals* 158b⟩+≡ (156c) ◁157e 161d▷

```
int th;
Cacheinfo *tc;
ulong a;
```

⟨cachechars() *find oldest entry* c *with age* a *in cache* 158c⟩≡ (157f)

```
/*
 * Not found; toss out oldest entry
 */
a = ~0;
th = sh;
tc = &f->cache[th];
while(tc < ec){
    if(tc->age < a){
        a = tc->age;
        h = th;
        c = tc;
    }
    tc++;
    th++;
}
```

⟨*function* loadchar 158d⟩≡ (412a)

```
/* return 1 if load succeeded, 0 if failed, -1 if must retry */
error0
loadchar(Font *f, Rune r, Cacheinfo *c, int h, int noflush, char **subfontname)
{
    Rune pic; // ?? means
    Cachefont *cf;
    int i;
    int oi, wid, top, bottom;
    Fontchar *fi;
    Cachesubf *subf;
    ⟨loadchar() other locals 159f⟩

    pic = r;

    // Find subfont spec cf for Rune r
    Again:
    for(i=0; i < f->nsub; i++){
        cf = f->sub[i];
        if(cf->min <= pic && pic <= cf->max)
            goto Found;
    }
    ⟨loadchar() if rune not handled by the font 159b⟩

    Found:
    // Now let's find the loaded subfont subf with spec cf
    /*
     * Choose exact or oldest
     */
    oi = 0;
    subf = &f->subf[0];
    for(i=0; i < f->nsubf; i++){
        if(cf == subf->cf)
            goto Found2;
```

```
        if(subf->age < f->subf[oi].age)
            oi = i;
        subf++;
    }
    // may load and find the right subf
    ⟨loadchar() when the corresponding subfont is not loaded yet 165i⟩

    Found2:
    subf->age = f->age;

    /* possible overflow here, but works out okay */
    pic += cf->offset;
    pic -= cf->min;
    ⟨loadchar() if rune outside range 159c⟩
    fi = &subf->f->info[pic];
    ⟨loadchar() sanity check fontchar width 159d⟩
    wid = (fi+1)->x - fi->x;
    ⟨loadchar() resize cache if width too big 161g⟩

    c->value = r;
    top    = fi->top    + (f->ascent - subf->f->ascent);
    bottom = fi->bottom + (f->ascent - subf->f->ascent);
    c->width = fi->width;
    c->x = h * f->width;
    c->left = fi->left;

    ⟨loadchar() sanity check display 159e⟩

    ⟨loadchar() marshall Cacheinfo c 160a⟩
}
```

⟨*constant* PJW 159a⟩≡ (412a)

```
#define PJW 0 /* use NUL==pjw for invisible characters */
```

⟨loadchar() *if rune not handled by the font* 159b⟩≡ (158d)

```
TryPJW:
if(pic != PJW){
    pic = PJW;
    goto Again;
}
return ERROR_0;
```

⟨loadchar() *if rune outside range* 159c⟩≡ (158d)

```
if(pic >= subf->f->n)
    goto TryPJW;
```

⟨loadchar() *sanity check fontchar width* 159d⟩≡ (158d)

```
if(fi->width == 0)
    goto TryPJW;
```

⟨loadchar() *sanity check display* 159e⟩≡ (158d)

```
if(f->display == nil)
    return OK_1;
```

Marshalling

⟨loadchar() *other locals* 159f⟩≡ (158d) 165h ▷

```
byte *b;
```

⟨loadchar() *marshall Cacheinfo c* 160a⟩≡ (158d)

```
flushimage(f->display, false); /* flush any pending errors */

// load character: 'l' fontid[4] srcid[4] index[2] R[4*4] P[2*4] left[1] width[1]
b = bufimage(f->display, 37);
⟨loadchar() sanity check b 160b⟩
b[0] = 'l';
BPLONG(b+1, f->cacheimage->id);
BPLONG(b+5, subf->f->bits->id);

// index
BPSHORT(b+9, c - f->cache);

// destination coordinates in f->cacheimage
BPLONG(b+11, c->x);
BPLONG(b+15, top);
BPLONG(b+19, c->x + ((fi+1)->x - fi->x));
BPLONG(b+23, bottom);

// source coordinate in subf->f->bits->id
BPLONG(b+27, fi->x);
BPLONG(b+31, fi->top);
b[35] = fi->left;
b[36] = fi->width;

return OK_1;
```

⟨loadchar() *sanity check b* 160b⟩≡ (160a)

```
if(b == nil)
    return ERROR_0;
```

Unmarshalling

⟨drawmesg() *locals* 160c⟩+≡ (71) ◁151b 202f▷

```
FChar *fc;
```

⟨drawmesg() *cases* 160d⟩+≡ (71) ◁151c 163d▷

```
/* load character: 'l' fontid[4] srcid[4] index[2] R[4*4] P[2*4] left[1] width[1] */
case 'l':
    printmesg(fmt="LLSRPbb", a, false);
    m = 1+4+4+2+4*4+2*4+1+1;
    ⟨drawmesg() sanity check n with m 75d⟩
    font = drawlookup(client, BGLONG(a+1), true);
    ⟨drawmesg() when load character, sanity check font 161a⟩
    src = drawimage(client, a+5);
    ci = BGSHORT(a+9);
    ⟨drawmesg() when load character, sanity check ci 161b⟩
    drawrectangle(&r, a+11);
    drawpoint(&p, a+27);
    memdraw(font->image, r, src, p, memopaque, p, S);

    fc = &font->fchar[ci];

    fc->minx = r.min.x;
    fc->maxx = r.max.x;
    fc->miny = r.min.y;
    fc->maxy = r.max.y;

    fc->left = a[35];
```

```
    fc->width = a[36];

    continue;
```

⟨drawmesg() *when load character, sanity check font* 161a⟩≡ (160d)

```
if(font == nil)
    error(Enodrawimage);
if(font->nfchar == 0)
    error(Enotfont);
```

⟨drawmesg() *when load character, sanity check ci* 161b⟩≡ (160d)

```
if(ci >= font->nfchar)
    error(Eindex);
```

9.7.4 Resizing the character cache: fontresize()

⟨Font *other fields* 161c⟩≡ (137a)

```
short       width;  /* widest so far; used in caching only */
int     maxdepth;   /* maximum depth of all loaded subfonts */
```

⟨cachechars() *other locals* 161d⟩+≡ (156c) ◁158b

```
int nc;
```

⟨cachechars() *if age too recent then resize cache* 161e⟩≡ (157f)

```
if(a && (f->age - a) < 500){ /* kicking out too recent; resize */
    nc = 2*(f->ncache-NFLOOK) + NFLOOK;
    if(nc <= MAXFCACHE){
        if(i == 0)
            fontresize(f, f->width, nc, f->maxdepth);
        /* else flush first; retry will resize */
        break;
    }
    // else, no resize
}
```

⟨cachechars() *if same age* 161f⟩≡ (157f)

```
if(c->age == f->age) /* flush pending string output */
    break;
```

⟨loadchar() *resize cache if width too big* 161g⟩≡ (158d)

```
if(f->width < wid || f->width == 0 || f->maxdepth < subf->f->bits->depth){
    /*
     * Flush, free, reload (easier than reformatting f->b)
     */
    if(noflush)
        return -1;
    // else

    if(f->width < wid)
        f->width = wid;
    if(f->maxdepth < subf->f->bits->depth)
        f->maxdepth = subf->f->bits->depth;
    i = fontresize(f, f->width, f->ncache, f->maxdepth);
    if(i <= 0)
        return i;
    /* c is still valid as didn't reallocate f->cache */
}
```

Marshalling

⟨*function* fontresize 162a⟩≡ (412a)

```
/* return whether resize succeeded && f->cache is unchanged */
static bool
fontresize(Font *f, int wid, int ncache, int depth)
{
    Cacheinfo *i;
    int ret;
    Image *new;
    uchar *b;
    Display *d;

    ret = false;
    ⟨fontresize() sanity check depth and wid 162c⟩
    d = f->display;
    ⟨fontresize() sanity check d 163a⟩

    new = allocimage(d, Rect(0, 0, ncache*wid, f->height), CHAN1(CGrey, depth), false, 0);
    ⟨fontresize() sanity check new 163b⟩
    flushimage(d, false); /* flush any pending errors */

    b = bufimage(d, 1+4+4+1);
    ⟨fontresize() sanity check b 163c⟩
    b[0] = 'i';
    BPLONG(b+1, new->id);
    BPLONG(b+5, ncache);
    b[9] = f->ascent;
    if(flushimage(d, false) < 0){
        fprint(2, "resize: init failed: %r\n");
        freeimage(new);
        goto Return;
    }

    freeimage(f->cacheimage);
    f->cacheimage = new;

    Nodisplay:
    f->width = wid;
    f->maxdepth = depth;
    ret = true;
    ⟨fontresize() if need to resize the cache 162b⟩
    Return:
    memset(f->cache, 0, f->ncache*sizeof f->cache[0]);
    return ret;
}
```

⟨fontresize() *if need to resize the cache* 162b⟩≡ (162a)

```
if(f->ncache != ncache){
    i = malloc(ncache*sizeof f->cache[0]);
    if(i != nil){
        ret = false;
        free(f->cache);
        f->ncache = ncache;
        f->cache = i;
    }
    /* else just wipe the cache clean and things will be ok */
}
```

⟨fontresize() *sanity check depth and wid* 162c⟩≡ (162a)

```
  if(depth <= 0)
      depth = 1;
  if(wid <= 0)
      wid = 1;
```

⟨fontresize() *sanity check d* 163a⟩≡ (162a)

```
  if(d == nil)
      goto Nodisplay;
```

⟨fontresize() *sanity check new* 163b⟩≡ (162a)

```
  if(new == nil){
      fprint(2, "font cache resize failed: %r\n");
      abort();
      goto Return;
  }
```

⟨fontresize() *sanity check b* 163c⟩≡ (162a)

```
  if(b == nil){
      freeimage(new);
      goto Return;
  }
```

Unmarshalling

⟨drawmesg() *cases* 163d⟩+≡ (71) ◁160d 174g▷

```
  /* initialize font: 'i' fontid[4] nchars[4] ascent[1] */
  case 'i':
      printmesg(fmt="Llb", a, false);
      m = 1+4+4+1;
      ⟨drawmesg() sanity check n with m 75d⟩
      dstid = BGLONG(a+1);
      ⟨drawmesg() when initialize font image and cache, sanity check dstid 163e⟩
      font = drawlookup(client, dstid, true);
      ⟨drawmesg() when initialize font image and cache, sanity check font 163f⟩
      ni = BGLONG(a+5);
      ⟨drawmesg() when initialize font image and cache, sanity check ni 163g⟩
      free(font->fchar);  /* should we complain if non-zero? */
      font->fchar = malloc(ni * sizeof(FChar));
      ⟨drawmesg() when initialize font image and cache, sanity check fchar 164a⟩
      memset(font->fchar, 0, ni*sizeof(FChar));
      font->nfchar = ni;
      font->ascent = a[9];
      continue;
```

⟨drawmesg() *when initialize font image and cache, sanity check dstid* 163e⟩≡ (163d)

```
  if(dstid == 0)
      error("cannot use display as font");
```

⟨drawmesg() *when initialize font image and cache, sanity check font* 163f⟩≡ (163d)

```
  if(font == nil)
      error(Enodrawimage);
  if(font->image->layer)
      error("cannot use window as font");
```

⟨drawmesg() *when initialize font image and cache, sanity check ni* 163g⟩≡ (163d)

```
  if(ni<=0 || ni>4096)
      error("bad font size (4096 chars max)");
```

⟨drawmesg() *when initialize font image and cache, sanity check fchar* 164a⟩≡ (163d)

```
if(font->fchar == nil)
    error("no memory for font");
```

9.7.5 agefont()

⟨*function* agefont 164b⟩≡ (412a)

```
void
agefont(Font *f)
{
    ⟨agefont() locals 164c⟩

    f->age++;
    ⟨agefont() if age overflow 164d⟩
}
```

⟨agefont() *locals* 164c⟩≡ (164b)

```
Cacheinfo *c, *ec;
Cachesubf *s, *es;
```

⟨agefont() *if age overflow* 164d⟩≡ (164b)

```
if(f->age == 65536){
    /*
     * Renormalize ages
     */
    c = f->cache;
    ec = c+f->ncache;
    while(c < ec){
        if(c->age){
            c->age >>= 2;
            c->age++;
        }
        c++;
    }

    s = f->subf;
    es = s + f->nsubf;
    while(s < es){
        if(s->age){
            if(s->age < SUBFAGE && s->cf->name != nil){
                /* clean up */
                if(display &&
                    s->f != display->defaultsubfont)
                    freesubfont(s->f);
                s->cf = nil;
                s->f = nil;
                s->age = 0;
            }else{
                s->age >>= 2;
                s->age++;
            }
        }
        s++;
    }
    f->age = (65536>>2) + 1;
}
```

9.8 Subfonts

9.8.1 Subfont cache: Cachesubf

⟨Font *subfont cache fields* 165a⟩≡ (137a)
```
// growing_array<Cachesubf> (size = Font.nsubf, init = NFSUBF, max = MAXSUBF)
Cachesubf   *subf;
int     nsubf;  /* size of subfont list */
```

⟨*struct* Cachesubf 165b⟩≡ (391b)
```
struct Cachesubf
{
    // ref<Cachefont>, the key
    Cachefont *cf; /* font info that owns us */
    // ref_own<Subfont>, the value
    Subfont  *f; /* attached subfont */

    ulong  age; /* for replacement */
};
```

⟨buildfont() *allocate cache* 165c⟩+≡ (144) ◁155f
```
fnt->nsubf = NFSUBF;
fnt->subf = malloc(fnt->nsubf * sizeof(fnt->subf[0]));
```

⟨*constant* NFSUBF 165d⟩≡ (154d)
```
NFSUBF = 2,   /* #subfonts to cache */
```

⟨*constant* MAXSUBF 165e⟩≡ (154d)
```
MAXSUBF = 50,   /* generous upper limit */
```

⟨*constant* DSUBF 165f⟩≡ (154d)
```
DSUBF =  4,
```

⟨freefont() *free cache* 165g⟩+≡ (146e) ◁156b
```
for(i=0; i<f->nsubf; i++){
    s = f->subf[i].f;
    if(s && display && s != display->defaultsubfont)
        freesubfont(s);
}
free(f->subf);
```

9.8.2 Subfont loading

Subfont loading part 1

⟨loadchar() *other locals* 165h⟩+≡ (158d) ◁159f
```
Cachesubf *of;
```

⟨loadchar() *when the corresponding subfont is not loaded yet* 165i⟩≡ (158d)
```
subf = &f->subf[oi];
⟨loadchar() if old subfont entry valid, free it or expand subfont cache 166a⟩
subf->age = 0;
subf->cf = nil;
subf->f = cf2subfont(cf, f);
⟨loadchar() sanity check subfont f 168a⟩
subf->cf = cf;
⟨loadchar() sanity check ascents 166c⟩
```

⟨loadchar() *if old subfont entry valid, free it or expand subfont cache* 166a⟩≡ (165i)

```
if(subf->f){
    if(f->age - subf->age > SUBFAGE || f->nsubf > MAXSUBF){
Toss:
        /* ancient data; toss */
        freesubfont(subf->f);
        subf->cf = nil;
        subf->f = nil;
        subf->age = 0;
    }else{    /* too recent; grow instead */
        of = f->subf;
        f->subf = malloc((f->nsubf+DSUBF) * sizeof(Subfont));
        ⟨loadchar() sanity check new malloced subf 166b⟩
        memmove(f->subf, of, (f->nsubf+DSUBF) * sizeof(Subfont));
        memset(f->subf+f->nsubf, 0, DSUBF*sizeof *subf);
        subf = &f->subf[f->nsubf];
        f->nsubf += DSUBF;
        free(of);
    }
}
```

⟨loadchar() *sanity check new malloced subf* 166b⟩≡ (166a)

```
if(f->subf == nil){
    f->subf = of;
    goto Toss;
}
```

⟨loadchar() *sanity check ascents* 166c⟩≡ (165i)

```
if(subf->f->ascent > f->ascent && f->display){
    /* should print something? this is a mistake in the font file */
    /* must prevent c->top from going negative when loading cache */
    Image *b;
    int d, t;
    d = subf->f->ascent - f->ascent;
    b = subf->f->bits;
    draw(b, b->r, b, nil, addpt(b->r.min, Pt(0, d)));
    draw(b, Rect(b->r.min.x, b->r.max.y-d, b->r.max.x, b->r.max.y), f->display->black, nil, b->r.min);
    for(i=0; i<subf->f->n; i++){
        t = subf->f->info[i].top-d;
        if(t < 0)
            t = 0;
        subf->f->info[i].top = t;
        t = subf->f->info[i].bottom-d;
        if(t < 0)
            t = 0;
        subf->f->info[i].bottom = t;
    }
    subf->f->ascent = f->ascent;
}
```

⟨*function* cf2subfont 166d⟩≡ (412a)

```
static Subfont*
cf2subfont(Cachefont *cf, Font *f)
{
    int depth;
    char *name;

    name = cf->subfontname;
    if(name == nil){
        ⟨cf2subfont() set depth 167a⟩
```

```
        name = subfontname(cf->name, f->name, depth);
        ⟨cf2subfont() sanity check name 167b⟩
        cf->subfontname = name;
    }
    return lookupsubfont(f->display, name);
}
```

⟨cf2subfont() *set depth* 167a⟩≡ (166d)

```
if(f->display && f->display->screenimage)
    depth = f->display->screenimage->depth;
else
    depth = 8;
```

⟨cf2subfont() *sanity check name* 167b⟩≡ (166d)

```
if(name == nil)
    return nil;
```

⟨*function* subfontname 167c⟩≡ (414b)

```
/*
 * Default version: convert to file name
 */
char*
subfontname(char *cfname, char *fname, int maxdepth)
{
    char *t, *u, *tmp1, *tmp2;
    int i;

    t = strdup(cfname);  /* t is the return string */
    if(strcmp(cfname, "*default*") == 0)
        return t;
    if(t[0] != '/'){
        tmp2 = strdup(fname);
        u = utfrrune(tmp2, '/');
        if(u)
            u[0] = 0;
        else
            strcpy(tmp2, ".");
        tmp1 = smprint("%s/%s", tmp2, t);
        free(tmp2);
        free(t);
        t = tmp1;
    }

    if(maxdepth > 8)
        maxdepth = 8;

    for(i=3; i>=0; i--){
        if((1<<i) > maxdepth)
            continue;
        /* try i-bit grey */
        tmp2 = smprint("%s.%d", t, i);
        if(access(tmp2, AREAD) == 0) {
            free(t);
            return tmp2;
        }
        free(tmp2);
    }

    /* try default */
    if(access(t, AREAD) == 0)
```

```
        return t;

    free(t);
    return nil;
}
```

Subfont loading part 2

⟨loadchar() *sanity check subfont f* 168a⟩≡ (165i)

```
if(subf->f == nil){
    if(cf->subfontname == nil)
        goto TryPJW;
    *subfontname = cf->subfontname;
    return -1; // caller must retry
}
```

⟨_string() *other locals* 168b⟩≡ (149d) 168e ▷

```
char *subfontname;
Subfont *sf = nil;
```

⟨_string() *if subfontname* 168c⟩≡ (149d)

```
if(subfontname){
    ⟨_string() free previous sf 168d⟩
    // populate Font.subf global so next loadchar will find the subfont
    sf=_getsubfont(f->display, subfontname);
    ⟨_string() sanity check sf and possibly adjust f 168f⟩
    /*
     * must not free sf until cachechars has found it in the cache
     * and picked up its own reference.
     */
}
```

⟨_string() *free previous sf* 168d⟩≡ (168c)

```
freesubfont(sf);
```

⟨_string() *other locals* 168e⟩+≡ (149d) ◁ 168b

```
Font *def;
```

⟨_string() *sanity check sf and possibly adjust f* 168f⟩≡ (168c)

```
if(sf == nil){
    def = f->display ? f->display->defaultfont : nil;
    if(def && f != def)
        f = def;
    else
        break;
}
```

⟨_stringnwidth() *other locals* 168g⟩+≡ (154a) ◁ 154b

```
Font *def;
```

⟨_stringnwidth() *if subfontname* 168h⟩≡ (154a)

```
if(subfontname){
    if(_getsubfont(f->display, subfontname) == 0){
        def = f->display->defaultfont;
        if(def && f!=def)
            f = def;
        else
            break;
    }
}
```

⟨*function* _getsubfont 169a⟩≡ (414b)

```
/*
 * Default version: treat as file name
 */
Subfont*
_getsubfont(Display *d, char *name)
{
    fdt fd;
    Subfont *f;

    fd = open(name, OREAD);
    ⟨_getsubfont() sanity check fd 169b⟩

    ⟨_getsubfont() locking part1 169c⟩
    f = readsubfont(d, name, fd, d && !d->locking);
    ⟨_getsubfont() locking part2 169d⟩
    ⟨_getsubfont() sanity check f 169e⟩
    close(fd);
    ⟨_getsubfont() set malloc tag for debug 351b⟩

    return f;
}
```

⟨_getsubfont() *sanity check fd* 169b⟩≡ (169a)

```
if(fd < 0){
    fprint(2, "getsubfont: can't open %s: %r\n", name);
    return nil;
}
```

⟨_getsubfont() *locking part1* 169c⟩≡ (169a)

```
/*
 * unlock display so i/o happens with display released, unless
 * user is doing his own locking, in which case this could break things.
 * _getsubfont is called only from string.c and stringwidth.c,
 * which are known to be safe to have this done.
 */
if(d && !d->locking)
    unlockdisplay(d);
```

⟨_getsubfont() *locking part2* 169d⟩≡ (169a)

```
if(d && !d->locking)
    lockdisplay(d);
```

⟨_getsubfont() *sanity check f* 169e⟩≡ (169a)

```
if(f == nil)
    fprint(2, "getsubfont: can't read %s: %r\n", name);
```

9.8.3 Subfont sharing

⟨allocsubfont() *install subfont if name* 169f⟩≡ (141f)

```
if(name){
    f->name = strdup(name);
    if(lookupsubfont(i->display, name) == nil)
        installsubfont(name, f);
}
```

⟨*global* lastname 169g⟩≡ (414b)

```
static char *lastname;
```

⟨*global* lastsubfont 170a⟩≡ (414b)

```
Subfont *lastsubfont;
```

⟨*function* lookupsubfont 170b⟩≡ (414b)

```
Subfont*
lookupsubfont(Display *d, char *name)
{
    if(d && strcmp(name, "*default*") == 0)
        return d->defaultsubfont;
    if(lastname && strcmp(name, lastname)==0)
      if(d == lastsubfont->bits->display){
        lastsubfont->ref++;
        return lastsubfont;
    }
    return nil;
}
```

⟨*function* installsubfont 170c⟩≡ (414b)

```
void
installsubfont(char *name, Subfont *subfont)
{
    free(lastname);
    lastname = strdup(name);
    lastsubfont = subfont; /* notice we don't free the old one; that's your business */
}
```

⟨*function* uninstallsubfont 170d⟩≡ (414b)

```
void
uninstallsubfont(Subfont *subfont)
{
    if(subfont == lastsubfont){
        lastname = nil;
        lastsubfont = nil;
    }
}
```

9.8.4 Subfont IO

Reading a subfont

⟨*function* readsubfont 170e⟩≡ (414b)

```
Subfont*
readsubfont(Display *d, char *name, fdt fd, bool dolock)
{
    Image *i;
    Fontchar *fc;
    char hdr[3*12 + 4+1];
    int n;
    byte *p;
    Subfont *f;

    // reading the image

    i = readimage(d, fd, dolock); // the call!
    ⟨readsubfont() sanity check i 171a⟩

    // reading the fontchars

    if(read(fd, hdr, 3*12) != 3*12){
```

```
        freeimage(i);
        werrstr("rdsubfonfile: header read error: %r");
        return nil;
    }
    n = atoi(hdr);
    p = malloc(6 * (n+1));
    ⟨readsubfont() sanity check p 171b⟩
    if(read(fd, p, 6 * (n+1)) != 6 * (n+1)){
        werrstr("rdsubfonfile: fontchar read error: %r");
    Err:
        freeimage(i);
        free(p);
        return nil;
    }
    fc = malloc(sizeof(Fontchar) * (n+1));
    ⟨readsubfont() sanity check fc 171c⟩
    _unpackinfo(fc, p, n);
    ⟨readsubfont() if lock part1 171e⟩
    f = allocsubfont(name, n, atoi(hdr+12), atoi(hdr+24), fc, i);
    ⟨readsubfont() if lock part2 171f⟩
    ⟨readsubfont() sanity check f 171d⟩
    free(p);
    return f;
}
```

⟨readsubfont() *sanity check i* 171a⟩≡ (170e)

```
if(i == nil)
    return nil;
```

⟨readsubfont() *sanity check p* 171b⟩≡ (170e)

```
if(p == nil)
    goto Err;
```

⟨readsubfont() *sanity check fc* 171c⟩≡ (170e)

```
if(fc == nil)
    goto Err;
```

⟨readsubfont() *sanity check f* 171d⟩≡ (170e)

```
if(f == nil){
    free(fc);
    goto Err;
}
```

⟨readsubfont() *if lock part1* 171e⟩≡ (170e)

```
if(dolock)
    lockdisplay(d);
```

⟨readsubfont() *if lock part2* 171f⟩≡ (170e)

```
if(dolock)
    unlockdisplay(d);
```

Writing a subfont

⟨*function* writesubfont 171g⟩≡ (414b)

```
errorneg1
writesubfont(fdt fd, Subfont *f)
{
    char hdr[3*12+1];
    byte *data;
```

```
    int nb;

    sprint(hdr, "%11d %11d %11d ", f->n, f->height, f->ascent);
    if(write(fd, hdr, 3*12) != 3*12){
    Err:
        werrstr("writesubfont: bad write: %r");
        return -1;
    }
    nb = 6*(f->n+1);
    data = malloc(nb);
    ⟨writesubfont() sanity check data 172a⟩
    packinfo(f->info, data, f->n);
    if(write(fd, data, nb) != nb)
        goto Err;
    free(data);
    return OK_0;
}
```

⟨writesubfont() *sanity check data* 172a⟩≡ (171g)

```
if(data == nil)
    return ERROR_NEG1;
```

⟨*function* packinfo 172b⟩≡ (414b)

```
static
void
packinfo(Fontchar *fc, uchar *p, int n)
{
    int j;

    for(j=0;  j<=n;  j++){
        p[0] = fc->x;
        p[1] = fc->x>>8;
        p[2] = fc->top;
        p[3] = fc->bottom;
        p[4] = fc->left;
        p[5] = fc->width;
        fc++;
        p += 6;
    }
}
```

Chapter 10

Image IO

10.1 Loading images in the display server

10.1.1 API

⟨*function* loadimage 173a⟩≡ (416a)

```
errorneg1
loadimage(Image *i, Rectangle r, byte *data, int ndata)
{
    int chunk;
    int n, bpl;
    long dy;
    ⟨loadimage() other locals 174c⟩

    chunk = i->display->bufsize - 64; // a little room for header

    ⟨loadimage() sanity check r 173b⟩
    bpl = bytesperline(r, i->depth);
    n = bpl * Dy(r);
    ⟨loadimage() sanity check ndata 174a⟩

    ndata = 0;
    while(r.max.y > r.min.y){
        dy = r.max.y - r.min.y;
        if(dy * bpl > chunk)
            dy = chunk / bpl;
        ⟨loadimage() sanity check dy 174b⟩
        n = dy * bpl;

        ⟨loadimage() marshall one chunk of size n with height dy 174d⟩

        ndata += n;
        data += n;
        r.min.y += dy; // progress
    }
    ⟨loadimage() flush and sanity check no error 174f⟩
    return ndata;
}
```

⟨loadimage() *sanity check r* 173b⟩≡ (173a)

```
if(!rectinrect(r, i->r)){
    werrstr("loadimage: bad rectangle");
    return ERROR_NEG1;
}
```

⟨loadimage() *sanity check ndata* 174a⟩≡ (173a)

```
if(n > ndata){
    werrstr("loadimage: insufficient data");
    return ERROR_NEG1;
}
```

⟨loadimage() *sanity check dy* 174b⟩≡ (173a)

```
if(dy <= 0){
    werrstr("loadimage: image too wide for buffer");
    return ERROR_NEG1;
}
```

10.1.2 marshalling

⟨loadimage() *other locals* 174c⟩≡ (173a)

```
byte *a;
```

⟨loadimage() *marshall one chunk of size n with height dy* 174d⟩≡ (173a)

```
// write: 'y' id[4] R[4*4] data[x*1]
a = bufimage(i->display, 21+n);
⟨loadimage() sanity check a 174e⟩
a[0] = 'y';
BPLONG(a+1, i->id);
BPLONG(a+5, r.min.x);
BPLONG(a+9, r.min.y);
BPLONG(a+13, r.max.x);
BPLONG(a+17, r.min.y + dy);
memmove(a+21, data, n);
```

⟨loadimage() *sanity check a* 174e⟩≡ (174d)

```
if(a == nil){
    werrstr("bufimage failed");
    return ERROR_NEG1;
}
```

⟨loadimage() *flush and sanity check no error* 174f⟩≡ (173a)

```
if(flushimage(i->display, false) < 0)
    return ERROR_NEG1;
```

10.1.3 Unmarshalling

⟨drawmesg() *cases* 174g⟩+≡ (71) ◁163d 177g▷

```
/* write: 'y' id[4] R[4*4] data[x*1] */
/* write from compressed data: 'Y' id[4] R[4*4] data[x*1] */
case 'y':
case 'Y':
    printmesg(fmt="LR", a, false);
    m = 1+4+4*4;
    ⟨drawmesg() sanity check n with m 75d⟩
    dstid = BGLONG(a+1);
    dst = drawimage(client, a+1);
    drawrectangle(&r, a+5);
    ⟨drawmesg() when load an image, sanity check r 175a⟩
    y = memload(dst, r, a+m, n-m, *a=='Y'); // The call
    ⟨drawmesg() when load an image, sanity check y 175b⟩
    dstflush(dstid, dst, r);
    m += y;
    continue;
```

⟨drawmesg() *when load an image, sanity check r* 175a⟩≡ (174g)

```
if(!rectinrect(r, dst->r))
    error(Ewriteoutside);
```

⟨drawmesg() *when load an image, sanity check y* 175b⟩≡ (174g)

```
if(y < 0)
    error("bad writeimage call");
```

10.1.4 Algorithm

⟨*function* memload 175c⟩≡ (459c)

```
errorneg1
memload(Memimage *dst, Rectangle r, byte *data, int n, bool iscompressed)
{
    errorneg1 (*loadfn)(Memimage*, Rectangle, byte*, int);
    Memlayer *dl;
    ⟨memload() other locals 230a⟩

    loadfn = loadmemimage;
    ⟨memload() if iscompressed 329a⟩

    Top:
    dl = dst->layer;
    if(dl == nil)
        return (*loadfn)(dst, r, data, n);
    // else
    ⟨memload() if dst has a layer 230b⟩
}
```

⟨*function* loadmemimage 175d⟩≡ (435b)

```
errorneg1
loadmemimage(Memimage *i, Rectangle r, byte *data, int ndata)
{
    int l;
    int y;
    byte *q;
    int lpart = 0, rpart = 0;
    ⟨loadmemimage() other locals 243c⟩

    ⟨loadmemimage() sanity check r 176a⟩
    l = bytesperline(r, i->depth);
    ⟨loadmemimage() sanity check ndata 176b⟩
    ndata = l*Dy(r);

    q = byteaddr(i, r.min);

    ⟨loadmemimage() set mx, lpart, rpart, m for small depth images 243d⟩
    ⟨loadmemimage() if 1 byte per line 244a⟩

    if(lpart==0 && rpart==0){ /* easy case */
        for(y = r.min.y; y < r.max.y; y++){
            memmove(q, data, l);
            q += i->width * sizeof(ulong);
            data += l;
        }
        return ndata;
    }
    ⟨loadmemimage() when small depth images 244b⟩
}
```

⟨loadmemimage() *sanity check r* 176a⟩≡ (175d)

```
if(!rectinrect(r, i->r))
    return ERROR_NEG1;
```

⟨loadmemimage() *sanity check ndata* 176b⟩≡ (175d)

```
if(ndata < l*Dy(r))
    return ERROR_NEG1;
```

10.2 Unloading images from the display server

10.2.1 API

⟨*function* unloadimage 176c⟩≡ (416a)

```
errorneg1
unloadimage(Image *i, Rectangle r, byte *data, int ndata)
{
    int bpl, dy;
    int n, ntot;
    Display *d;
    ⟨unloadimage() other locals 177c⟩

    ⟨unloadimage() sanity check r 176d⟩
    bpl = bytesperline(r, i->depth);
    ⟨unloadimage() sanity check ndata 176e⟩

    d = i->display;
    flushimage(d, false); /* make sure subsequent flush is for us only */

    ntot = 0;
    while(r.min.y < r.max.y){

        dy = 8000/bpl;
        ⟨unloadimage() sanity check dy 177b⟩
        if(dy > Dy(r))
            dy = Dy(r);

       ⟨unloadimage() marshall reading request for rectangle of height dy 177d⟩

        n = read(d->fd, data + ntot, ndata - ntot);
       ⟨unloadimage() sanity check n 177a⟩
        ntot += n;
        r.min.y += dy; // progress
    }
    return ntot;
}
```

⟨unloadimage() *sanity check r* 176d⟩≡ (176c)

```
if(!rectinrect(r, i->r)){
    werrstr("unloadimage: bad rectangle");
    return ERROR_NEG1;
}
```

⟨unloadimage() *sanity check ndata* 176e⟩≡ (176c)

```
if(ndata < bpl*Dy(r)){
    werrstr("unloadimage: buffer too small");
    return ERROR_NEG1;
}
```

⟨unloadimage() *sanity check n* 177a⟩≡ (176c)
```
if(n < 0)
    return n;
```

⟨unloadimage() *sanity check dy* 177b⟩≡ (176c)
```
if(dy <= 0){
    werrstr("unloadimage: image too wide");
    return ERROR_NEG1;
}
```

10.2.2 Marshalling

⟨unloadimage() *other locals* 177c⟩≡ (176c)
```
byte *a;
```

⟨unloadimage() *marshall reading request for rectangle of height dy* 177d⟩≡ (176c)
```
// read: 'r' id[4] R[4*4]
a = bufimage(d, 1+4+4*4);
⟨unloadimage() sanity check a 177f⟩
a[0] = 'r';
BPLONG(a+1, i->id);
BPLONG(a+5, r.min.x);
BPLONG(a+9, r.min.y);
BPLONG(a+13, r.max.x);
BPLONG(a+17, r.min.y+dy);
```

⟨unloadimage() *flush and sanity check no error* 177e⟩≡
```
if(flushimage(d, false) < 0)
    return ERROR_NEG1;
```

⟨unloadimage() *sanity check a* 177f⟩≡ (177d)
```
if(a == nil){
    werrstr("unloadimage: %r");
    return -1;
}
```

10.2.3 Unmarshalling

⟨drawmesg() *cases* 177g⟩+≡ (71) ◁174g 202g▷
```
/* read: 'r' id[4] R[4*4] */
case 'r':
    printmesg(fmt="LR", a, false);
    m = 1+4+4*4;
    ⟨drawmesg() sanity check n with m 75d⟩
    i = drawimage(client, a+1);
    drawrectangle(&r, a+5);
    ⟨drawmesg() when read an image, sanity check r 178a⟩
    c = bytesperline(r, i->depth);
    c *= Dy(r);

    free(client->readdata);
    client->readdata = mallocz(c, 0);
    ⟨drawmesg() when read an image, sanity check readdata 178b⟩
    client->nreaddata = memunload(i, r, client->readdata, c); // The call
    ⟨drawmesg() when read an image, sanity check nreaddata 178c⟩
    continue;
```

⟨drawmesg() *when read an image, sanity check r* 178a⟩≡ (177g)

```
if(!rectinrect(r, i->r))
    error(Ereadoutside);
```

⟨drawmesg() *when read an image, sanity check readdata* 178b⟩≡ (177g)

```
if(client->readdata == nil)
    error("readimage malloc failed");
```

⟨drawmesg() *when read an image, sanity check nreaddata* 178c⟩≡ (177g)

```
if(client->nreaddata < 0){
    free(client->readdata);
    client->readdata = nil;
    error("bad readimage call");
}
```

⟨Client *other fields* 178d⟩+≡ (35d) ◁68f 306b▷

```
// array<byte> (size >= Client.nreaddata)
byte*   readdata;
int     nreaddata;
```

⟨drawread() *switch qid cases* 178e⟩+≡ (66d) ◁68e 252b▷

```
case Qdata:
    ⟨drawread() switch qid cases, when Qdata, sanity checks 178f⟩
    n = cl->nreaddata;
    memmove(a, cl->readdata, cl->nreaddata);
    free(cl->readdata);
    cl->readdata = nil;
    break;
```

⟨drawread() *switch qid cases, when Qdata, sanity checks* 178f⟩≡ (178e)

```
if(cl->readdata == nil)
    error("no draw data");
if(n < cl->nreaddata)
    error(Eshortread);
```

10.2.4 Algorithm

⟨*function* memunload 178g⟩≡ (460d)

```
errorneg1
memunload(Memimage *src, Rectangle r, byte *data, int n)
{
    Memlayer *dl;
    ⟨memunload() other locals 230f⟩

    Top:
    dl = src->layer;
    if(dl == nil)
        return unloadmemimage(src, r, data, n);
    // else
    ⟨memunload() if src has layer 231⟩
}
```

⟨*function* unloadmemimage 178h⟩≡ (435b)

```
errorneg1
unloadmemimage(Memimage *i, Rectangle r, byte *data, int ndata)
{
    int y, l;
    byte *q;
```

```
    ⟨unloadmemimage() sanity check r 179a⟩
    l = bytesperline(r, i->depth);
    ⟨unloadmemimage() sanity check ndata 179b⟩
    ndata = l*Dy(r);
    q = byteaddr(i, r.min);
    for(y = r.min.y; y < r.max.y; y++){
        memmove(data, q, l);
        q += i->width * sizeof(ulong);
        data += l;
    }
    return ndata;
}
```

⟨unloadmemimage() *sanity check r* 179a⟩≡ (178h)

```
if(!rectinrect(r, i->r))
    return ERROR_NEG1;
```

⟨unloadmemimage() *sanity check ndata* 179b⟩≡ (178h)

```
if(ndata < l*Dy(r))
    return ERROR_NEG1;
```

10.3 Image format

⟨*global* channames 179c⟩≡ (400b)

```
static char channames[] = "rgbkamx";
```

⟨*function* strtochan 179d⟩≡ (400b)

```
ulong
strtochan(char *s)
{
    char *p, *q;
    int t, n;
    int depth = 0;
    ulong chan = 0;

    p = s;
    while(*p && isspace(*p))
        p++;

    while(*p && !isspace(*p)){
        q = strchr(channames, p[0]);
        ⟨strtochan() sanity check q 180a⟩
        t = q-channames;
        ⟨strtochan() sanity check number after channel type 180b⟩
        n = p[1]-'0';
        depth += n;
        chan = (chan << 8) | __DC(t, n);
        p += 2;
    }
    ⟨strtochan() sanity check depth 180c⟩
    return chan;
}
```

⟨*function* isspace 179e⟩≡ (400b)

```
/* avoid pulling in ctype when using with drawterm etc. */
static int
isspace(char c)
{
    return c==' ' || c== '\t' || c=='\r' || c=='\n';
}
```

⟨strtochan() *sanity check q* 180a⟩≡ (179d)
```
if(q == nil)
    return 0;
```

⟨strtochan() *sanity check number after channel type* 180b⟩≡ (179d)
```
if(p[1] < '0' || p[1] > '9')
    return 0;
```

⟨strtochan() *sanity check depth* 180c⟩≡ (179d)
```
if(depth == 0 || (depth > 8 && depth % 8) || (depth < 8 && 8 % depth))
    return 0;
```

⟨*global* drawld2chan 180d⟩≡ (402c)
```
ulong drawld2chan[] = {
    GREY1,
    GREY2,
    GREY4,

    CMAP8,
};
```

10.4 Reading images from the disk

⟨*function* readimage 180e⟩≡ (416b)
```
Image*
readimage(Display *d, fdt fd, bool dolock)
{
    char hdr[5*12+1]; // 5 numbers as 11 chars plus space
    ulong chan;
    Rectangle r;
    Image *i;

    int miny, maxy;
    int dy;
    uint l, n;
    int m, j, chunk;
    byte *tmp;
    ⟨readimage() other locals 338a⟩

    // Reading the header

    if(readn(fd, hdr, 11) != 11)
        return nil;
    ⟨readimage() if first 11 characters are compressed string 330a⟩
    // else
    if(readn(fd, hdr+11, 5*12-11) != 5*12-11)
        return nil;
    if(d)
        chunk = d->bufsize - 32; /* a little room for header */
    ⟨readimage() set chunk if no display 182a⟩

    ⟨readimage() check if new or old format, set new 338b⟩
    ⟨readimage() sanity check hdr 182c⟩
    ⟨readimage() if old format 338d⟩
    else{
        hdr[11] = '\0';
        chan = strtochan(hdr);
        ⟨readimage() sanity check chan 182d⟩
```

```
    }

    r.min.x = atoi(hdr+1*12);
    r.min.y = atoi(hdr+2*12);
    r.max.x = atoi(hdr+3*12);
    r.max.y = atoi(hdr+4*12);
    ⟨readimage() sanity check r 182e⟩

    miny = r.min.y;
    maxy = r.max.y;

    // Allocating the image

    l = bytesperline(r, chantodepth(chan));
    if(d){
        ⟨readimage() lock display 181a⟩
        i = allocimage(d, r, chan, false, -1);
        ⟨readimage() unlock display 181b⟩
        ⟨readimage() sanity check i 182f⟩
    }
    ⟨readimage() alloc image if no display 182b⟩

    // Read from disk and load the image

    tmp = malloc(chunk);
    ⟨readimage() sanity check tmp 182g⟩

    while(maxy > miny){
        dy = maxy - miny;
        if(dy * l > chunk)
            dy = chunk/l;
        ⟨readimage() sanity check dy 182h⟩
        n = dy * l;

        m = readn(fd, tmp, n);

        ⟨readimage() sanity check m 182i⟩
        ⟨readimage() if old format, flip all bits 338e⟩
        if(d){
            ⟨readimage() lock display 181a⟩

            if(loadimage(i, Rect(r.min.x, miny, r.max.x, miny+dy), tmp,
                         chunk) <= 0)
                goto Err1;
            ⟨readimage() unlock display 181b⟩
        }
        miny += dy; // progress
    }
    free(tmp);
    return i;
}
```

⟨readimage() *lock display* 181a⟩≡ (180)

```
if(dolock)
    lockdisplay(d);
```

⟨readimage() *unlock display* 181b⟩≡ (180)

```
if(dolock)
    unlockdisplay(d);
```

⟨readimage() *set chunk if no display* 182a⟩≡ (180e)

```
else
    chunk = 8192;
```

⟨readimage() *alloc image if no display* 182b⟩≡ (180e)

```
else{
    i = mallocz(sizeof(Image), 1);
    if(i == nil)
        return nil;
}
```

⟨readimage() *sanity check hdr* 182c⟩≡ (180e)

```
if(hdr[11] != ' '){
    werrstr("readimage: bad format");
    return nil;
}
```

⟨readimage() *sanity check chan* 182d⟩≡ (180e)

```
if(chan == 0){
    werrstr("readimage: bad channel string %s", hdr);
    return nil;
}
```

⟨readimage() *sanity check r* 182e⟩≡ (180e)

```
if(r.min.x > r.max.x || r.min.y > r.max.y){
    werrstr("readimage: bad rectangle");
    return nil;
}
```

⟨readimage() *sanity check i* 182f⟩≡ (180e)

```
if(i == nil)
    return nil;
```

⟨readimage() *sanity check tmp* 182g⟩≡ (180e)

```
if(tmp == nil)
    goto Err;
```

⟨readimage() *sanity check dy* 182h⟩≡ (180e)

```
if(dy <= 0){
    werrstr("readimage: image too wide for buffer");
    goto Err;
}
```

⟨readimage() *sanity check m* 182i⟩≡ (180e)

```
if(m != n){
    werrstr("readimage: read count %d not %d: %r", m, n);
Err:
    if(dolock)
        lockdisplay(d);
Err1:
    freeimage(i);
    if(dolock)
        unlockdisplay(d);
    free(tmp);
    return nil;
}
```

10.5 Writing images to the disk

Chapter 11

Input Devices

11.1 Mouse

11.1.1 /dev/mouse interface

11.1.2 Additional data structures

⟨*struct* Mouse 183a⟩≡ (397a)

```
struct Mouse
{
    Point xy;
    // bitset<enum<Click>>
    buttons buttons; /* bit array: LMR=124 */

    ulong msec;
};
```

⟨*type* buttons 183b⟩≡ (397)

```
// bitset<enum<Click>>
typedef int buttons;
```

⟨*enum* Click 183c⟩≡ (397)

```
enum Click {
    CLICK_LEFT = 1,
    CLICK_MIDDLE = 2,
    CLICK_RIGHT = 4,
};
```

⟨*struct* Mousectl 183d⟩≡ (397a)

```
struct Mousectl
{
    Mouse;

    // /dev/mouse
    fdt  mfd;  /* to mouse file */
    // ref_own<filename>
    char *file; // "/dev/mouse" usually

    // ref<Image>
    Image* image; /* of associated window/display */

    ⟨Mousectl channel fields 184b⟩
    ⟨Mousectl IO process field 184a⟩
    ⟨Mousectl cursor field 271b⟩
};
```

⟨Mousectl *IO process field* 184a⟩≡ (183d)
```
int  pid; /* of slave proc */
```

⟨Mousectl *channel fields* 184b⟩≡ (183d) 235d ▷
```
// chan<Mouse> (listener = user program, sender = _ioproc(mouse.c))
Channel *c;   /* chan(Mouse) */
```

11.1.3 Additional (user) initializations: initmouse()

⟨*function* initmouse 184c⟩≡ (417c)
```
Mousectl*
initmouse(char *file, Image *i)
{
    Mousectl *mc;
    ⟨initmouse() other locals 271c⟩

    mc = mallocz(sizeof(Mousectl), true);

    ⟨initmouse() sanitize file 184d⟩
    mc->mfd = open(file, ORDWR|OCEXEC);
    ⟨initmouse() sanity check mfd 184e⟩
    mc->file = strdup(file);

    ⟨initmouse() set cursor field 271d⟩
    mc->image = i;
    ⟨initmouse() set channels 184f⟩
    ⟨initmouse() create process 184g⟩

    return mc;
}
```

⟨initmouse() *sanitize file* 184d⟩≡ (184c)
```
if(file == nil)
    file = "/dev/mouse";
```

⟨initmouse() *sanity check mfd* 184e⟩≡ (184c)
```
if(mc->mfd < 0 && strcmp(file, "/dev/mouse")==0){
    bind("#m", "/dev", MAFTER);
    mc->mfd = open(file, ORDWR|OCEXEC);
}
if(mc->mfd < 0){
    free(mc);
    return nil;
}
```

11.1.4 IO process and communication channels

⟨initmouse() *set channels* 184f⟩≡ (184c) 235e ▷
```
mc->c       = chancreate(sizeof(Mouse), 0);
```

⟨initmouse() *create process* 184g⟩≡ (184c)
```
proccreate(_ioproc, mc, 4096);
```

⟨*function* _ioproc((lib_graphics/libdraw/mouse.c)) 185a⟩≡ (417c)

```
static
void
_ioproc(void *arg)
{
    int n, nerr;
    char buf[1+5*12]; // /dev/mouse first code char and 5 numbers
    Mouse m;
    Mousectl *mc = arg;
    ⟨_ioproc() other locals (mouse.c) 235f⟩

    threadsetname("mouseproc");
    memset(&m, 0, sizeof m);
    mc->pid = getpid();

    nerr = 0;
    for(;;){
        // blocking call
        n = read(mc->mfd, buf, sizeof buf);
        ⟨_ioproc() sanity check n from read (mouse.c) 185b⟩
        nerr = 0;
        switch(buf[0]){
        ⟨_ioproc() switch buf, resize case, fallthrough m case (mouse.c) 235g⟩
        case 'm':

            m.xy.x    = atoi(buf+1+0*12);
            m.xy.y    = atoi(buf+1+1*12);
            m.buttons = atoi(buf+1+2*12);
            m.msec    = atoi(buf+1+3*12);

            // send it!
            send(mc->c, &m);

            /*
             * mc->Mouse is updated after send so it doesn't have wrong value
             * if we block during send.
             * This means that programs should receive into mc->Mouse
             * (see readmouse() above) if they want full synchrony.
             */
            mc->Mouse = m;
            break;
        }
    }
}
```

⟨_ioproc() *sanity check n from read (mouse.c)* 185b⟩≡ (185a)

```
if(n != 1+4*12){
    yield(); /* if error is due to exiting, we'll exit here */
    fprint(2, "mouse: bad count %d not 49: %r\n", n);
    if(n<0 || ++nerr>10)
        threadexits("read error");
    continue;
}
```

⟨*function* closemouse 185c⟩≡ (417c)

```
void
closemouse(Mousectl *mc)
{
    ⟨closemouse() sanity check mc 186a⟩
```

```
    postnote(PNPROC, mc->pid, "kill");

    do ; while(nbrecv(mc->c, &mc->Mouse) > 0);

    close(mc->mfd);
    close(mc->cfd);
    free(mc->file);
    free(mc->c);
    free(mc->resizec);
    free(mc);
}
```

⟨closemouse() *sanity check mc* 186a⟩≡ (185c)

```
if(mc == nil)
    return;
```

11.1.5 API

⟨*function* readmouse 186b⟩≡ (417c)

```
errorneg1
readmouse(Mousectl *mc)
{
    ⟨readmouse() flush image before reading mouse 186c⟩
    if(recv(mc->c, &mc->Mouse) < 0){
        fprint(2, "readmouse: %r\n");
        return ERROR_NEG1;
    }
    return OK_0;
}
```

⟨readmouse() *flush image before reading mouse* 186c⟩≡ (186b)

```
if(mc->image)
    flushimage(mc->image->display, true);
```

⟨*function* moveto 186d⟩≡ (417c)

```
void
moveto(Mousectl *m, Point pt)
{
    fprint(m->mfd, "m%d %d", pt.x, pt.y);
    m->xy = pt;
}
```

11.2 Cursor

11.2.1 Additionnal data structures

Cursor

⟨*struct* Cursor 186e⟩≡ (397b)

```
struct Cursor
{
    Point offset;

    byte clr[2*16];
    byte set[2*16];
};
```

⟨*global* swcursor_arrow 187a⟩≡ (474)

```
Cursor  swcursor_arrow = {
    .offset = { -1, -1 },
    .clr = {
      0xFF, 0xFF, 0x80, 0x01, 0x80, 0x02, 0x80, 0x0C,
      0x80, 0x10, 0x80, 0x10, 0x80, 0x08, 0x80, 0x04,
      0x80, 0x02, 0x80, 0x01, 0x80, 0x02, 0x8C, 0x04,
      0x92, 0x08, 0x91, 0x10, 0xA0, 0xA0, 0xC0, 0x40,
    },
    .set = {
      0x00, 0x00, 0x7F, 0xFE, 0x7F, 0xFC, 0x7F, 0xF0,
      0x7F, 0xE0, 0x7F, 0xE0, 0x7F, 0xF0, 0x7F, 0xF8,
      0x7F, 0xFC, 0x7F, 0xFE, 0x7F, 0xFC, 0x73, 0xF8,
      0x61, 0xF0, 0x60, 0xE0, 0x40, 0x40, 0x00, 0x00,
    },
};
```

⟨*struct* Cursorinfo 187b⟩≡ (473)

```
struct Cursorinfo {
  Cursor;
  Lock;
};
```

⟨*global signature cursor* 187c⟩≡ (473)

```
// devmouse.c (set in <arch>/screen.c)
extern Cursorinfo  cursor;
```

Cursor rendering in software, cursor images

⟨*global* swback 187d⟩≡ (474)

```
Memimage*   swback; /* screen under cursor */
```

⟨*global* swimg1 187e⟩≡ (474)

```
Memimage*   swimg1;
```

⟨*global* swmask1 187f⟩≡ (474)

```
Memimage*   swmask1;
```

Cursor coordinates and visibility

⟨*global* swpt 187g⟩≡ (474)

```
Point   swpt;   /* desired cursor location */
```

⟨*global* swvispt 187h⟩≡ (474)

```
Point   swvispt;    /* actual cursor location */
```

⟨*global* swvers 187i⟩≡ (474)

```
int swvers; /* incremented each time cursor image changes */
```

⟨*global* swvisvers 187j⟩≡ (474)

```
int swvisvers;  /* the version on the screen */
```

⟨*global* swenabled 187k⟩≡ (474)

```
bool swenabled;  /* is the cursor supposed to be on the screen? */
```

⟨*global* swvisible 187l⟩≡ (474)

```
bool swvisible;  /* is the cursor visible? */
```

VGAcur and swcursor (x86)

⟨VGAscr *cursor fields* 188a⟩≡ (362a)
```
Cursor;
// the cursor device methods (software cursor or hardware support)
VGAcur* cur;
```

⟨*struct* VGAcur*(x86)* 188b⟩≡ (487)
```
struct VGAcur {
  char* name;

  void  (*enable)(VGAscr*);
  void  (*disable)(VGAscr*);

  int   (*move)(VGAscr*, Point);

  void  (*load)(VGAscr*, Cursor*);

  // optional
  int doespanning;
};
```

⟨*global* swcursor*(x86)* 188c⟩≡ (492)
```
VGAcur swcursor =
{
    .name = "soft",

    .enable  = swenable,
    .disable = swdisable,

    .load = swload,
    .move = swmove,
};
```

11.2.2 Additionnal (kernel) initializations

⟨screensize() *initializations part2* 188d⟩+≡ (46a) ◁47i 254a▷
```
if(didswcursorinit)
    swcursorinit_wrapper();
```

⟨*global* didswcursorinit*(x86)* 188e⟩≡ (492)
```
bool didswcursorinit;
```

⟨*function* swcursorinit 188f⟩≡ (474)
```
void
swcursor_init(void)
{
    static bool init;

    if(!init){
        init = true;
        addclock0link(swcursor_clock, 10);
        swenabled = 1; //bcm: was not in pc (but maybe bug?)
    }
    ⟨swcursorinit() free old versions of cursor images if any 189c⟩

    swback  = allocmemimage(Rect(0,0,32,32), gscreen->chan);

    swmask  = allocmemimage(Rect(0,0,16,16), GREY8);
```

```
    swmask1 = allocmemimage(Rect(0,0,16,16), GREY1);
    swimg   = allocmemimage(Rect(0,0,16,16), GREY8);
    swimg1  = allocmemimage(Rect(0,0,16,16), GREY1);

    ⟨swcursorinit() sanity check cursor images 189d⟩

    memfillcolor(swmask,  DOpaque);
    memfillcolor(swmask1, DOpaque);
    memfillcolor(swimg,   DBlack);
    memfillcolor(swimg1,  DBlack);
}
```

⟨*global* swimg 189a⟩≡ (474)

```
Memimage*   swimg;  /* cursor image */
```

⟨*global* swmask 189b⟩≡ (474)

```
Memimage*   swmask; /* cursor mask */
```

⟨swcursorinit() *free old versions of cursor images if any* 189c⟩≡ (188f)

```
if(swback){
    freememimage(swback);
    freememimage(swmask);
    freememimage(swmask1);
    freememimage(swimg);
    freememimage(swimg1);
}
```

⟨swcursorinit() *sanity check cursor images* 189d⟩≡ (188f)

```
if(swback == nil || swmask == nil || swmask1 == nil || swimg == nil || swimg1 == nil){
    print("software cursor: allocmemimage fails");
    return;
}
```

11.2.3 swcursorclock()

⟨*function* swcursorclock 189e⟩≡ (474)

```
void
swcursor_clock(void)
{
    int x;

    if(!swenabled)
        return;
    swcursor_move(mousexy()); //bcm: only, was not in pc
    if(swvisible && eqpt(swpt, swvispt) && swvers==swvisvers)
        return;

    x = arch_splhi();
    // check again, might have changed in between
    if(swenabled)
     if(!swvisible || !eqpt(swpt, swvispt) || swvers!=swvisvers)
      if(canqlock(&drawlock)){

        swcursor_hide();
        swcursor_draw();

        qunlock(&drawlock);
    }
    arch_splx(x);
}
```

⟨*global* swrect 190a⟩≡ (474)

```
Rectangle   swrect; /* screen rectangle in swback */
```

⟨*function* swcursordraw 190b⟩≡ (474)

```
void
swcursor_draw(void)
{
    bool dounlock;

    if(swvisible)
        return;
    if(!swenabled)
        return;
    if(swback == nil || swimg1 == nil || swmask1 == nil)
        return;
    dounlock = canqlock(&drawlock); //bcm:
    //assert(!canqlock(&drawlock)); // pc:

    swvispt = swpt;
    swvisvers = swvers;
    // cursor is 16x16 picture
    swrect = rectaddpt(Rect(0,0,16,16), swvispt);
    // save what is under the cursor
    memimagedraw(swback, swback->r, gscreen, swpt, memopaque, ZP, S);
    // draw cursor
    memimagedraw(gscreen, swrect, swimg1, ZP, swmask1, ZP, SoverD);
    arch_flushmemscreen(swrect);
    swvisible = true;
    //bcm:
    if(dounlock)
        qunlock(&drawlock);

}
```

⟨*function* swcursorhide 190c⟩≡ (474)

```
/*
 * called with drawlock locked for us, most of the time.
 * kernel prints at inopportune times might mean we don't
 * hold the lock, but memimagedraw is now reentrant so
 * that should be okay: worst case we get cursor droppings.
 */
void
swcursor_hide(void)
{
    if(!swvisible)
        return;
    if(swback == nil)
        return;

    swvisible = false;
    // restore what was under the cursor
    memimagedraw(gscreen, swrect, swback, ZP, memopaque, ZP, S);
    arch_flushmemscreen(swrect);
}
```

11.2.4 Drawing rectangles and the cursor (x86)

⟨hwdraw() *if software cursor(x86)* 190d⟩≡ (94d)

```
if(scr->cur == &swcursor){
```

```
    /*
     * always calling swcursorhide here doesn't cure
     * leaving cursor tracks nor failing to refresh menus
     * with the latest libmemdraw/draw.c.
     */
    if(dst->data->bdata == gscreendata.bdata)
        swcursoravoid(par->r);
    if(src->data->bdata == gscreendata.bdata)
        swcursoravoid(par->sr);
    if(mask->data->bdata == gscreendata.bdata)
        swcursoravoid(par->mr);
}
```

⟨*function* swcursoravoid 191a⟩≡ (474)

```
void
swcursor_avoid(Rectangle r)
{
    if(swvisible && rectXrect(r, swrect))
        swcursor_hide();
}
```

11.2.5 Enabling the cursor (x86)

⟨*function* cursoron*(x86)* 191b⟩≡ (492)

```
int
arch_cursoron(bool dolock)
{
    VGAscr *scr;
    int v;

    scr = &vgascreen;
    if(scr->cur == nil || scr->cur->move == nil)
        return 0;

    if(dolock)
        lock(&cursor);

    v = scr->cur->move(scr, mousexy());

    if(dolock)
        unlock(&cursor);

    return v;
}
```

⟨*function* cursoroff*(x86)* 191c⟩≡ (492)

```
void
arch_cursoroff(int)
{
}
```

⟨*function* swenable*(x86)* 191d⟩≡ (492)

```
/*
 * Need to lock drawlock for ourselves.
 */
void
swenable(VGAscr*)
{
    swenabled = true;
```

```
    if(canqlock(&drawlock)){
        swcursordraw();
        qunlock(&drawlock);
    }
}
```

⟨*function* swdisable*(x86)* 192a⟩≡ (492)

```
void
swdisable(VGAscr*)
{
    swenabled = false;
    if(canqlock(&drawlock)){
        swcursorhide();
        qunlock(&drawlock);
    }
}
```

11.2.6 Setting the cursor (x86)

⟨*function* ksetcursor*(x86)* 192b⟩≡ (492)

```
void
arch_ksetcursor(Cursor* curs)
{
    VGAscr *scr;

    scr = &vgascreen;
    if(scr->cur == nil || scr->cur->load == nil)
        return;

    scr->cur->load(scr, curs);
}
```

⟨*function* swload 192c⟩≡ (474)

```
void
swcursor_load(Cursor *curs)
{
    byte *ip, *mp;
    int i, j, set, clr;

    if(!swimg || !swmask || !swimg1 || !swmask1)
        return;
    /*
     * Build cursor image and mask.
     * Image is just the usual cursor image
     * but mask is a transparent alpha mask.
     *
     * The 16x16x8 memimages do not have
     * padding at the end of their scan lines.
     */
    ip = byteaddr(swimg, ZP);
    mp = byteaddr(swmask, ZP);

    for(i=0; i<32; i++){
        set = curs->set[i];
        clr = curs->clr[i];
        for(j=0x80; j; j>>=1){
            *ip++ = set & j ? 0x00 : 0xFF;
            *mp++ = (clr|set) & j ? 0xFF : 0x00;
        }
```

```
    }
    swoffset = curs->offset;
    swvers++;

    memimagedraw(swimg1,  swimg1->r,  swimg,  ZP, memopaque, ZP, S);
    memimagedraw(swmask1, swmask1->r, swmask, ZP, memopaque, ZP, S);
}
```

⟨*global* swoffset 193a⟩≡ (474)

```
Point   swoffset;
```

⟨*function* swmove 193b⟩≡ (474)

```
int
swcursor_move(Point p)
{
    swpt = addpt(p, swoffset);
    return 0;
}
```

11.2.7 /dev/cursor

11.3 Keyboard

11.3.1 /dev/cons and /dev/consctl interfaces

11.3.2 Additional data structures

⟨*enum* Kxxx 193c⟩≡ (396)

```
enum {
    KF= 0xF000, /* Rune: beginning of private Unicode space */
    Spec= 0xF800,

    /* KF|1, KF|2, ..., KF|0xC is F1, F2, ..., F12 */
    Khome= KF|0x0D,
    Kpgup= KF|0x0F,
    Kpgdown= KF|0x13,
    Kprint= KF|0x10,

    Kup= KF|0x0E,
    Kdown= Spec|0x00,
    Kleft= KF|0x11,
    Kright= KF|0x12,

    Kview= Spec|0x00,
    Kins= KF|0x14,
    Kend= KF|0x18,

    Kalt= KF|0x15,
    Kshift= KF|0x16,
    Kctl= KF|0x17,

    Kdel= 0x7f,
    Kesc= 0x1b,
    Keof= 0x04, // C-d
    Kbs= 0x08, // C-h (backspace)

    // from plan9front
  Ksoh= 0x01,
```

```
  Kstx= 0x02,
  Ketx= 0x03,
  //Keof= 0x04,
  Kenq= 0x05,
  Kack= 0x06,
  //Kbs= 0x08,
  Knack= 0x15,
  Ketb= 0x17,
  //Kdel= 0x7f,
  //Kesc= 0x1b,

 };
```

⟨*struct* Keyboardctl 194a⟩≡ (396)

```
 struct Keyboardctl
 {
     // /dev/cons
     fdt  consfd;  /* to cons file */
     char *file;

     // dev/consctl
     fdt  ctlfd;  /* to ctl file */

     // chan<Rune, 20> (listener = user program, sender = _ioproc(keyboard.c))
     Channel *c; /* chan(Rune)[20] */

     int  pid;  /* of slave proc */
 };
```

11.3.3 Additional (user) initializations: initkeyboard()

⟨*function* initkeyboard 194b⟩≡ (417b)

```
 Keyboardctl*
 initkeyboard(char *file)
 {
     Keyboardctl *kc;
     char *t;

     kc = mallocz(sizeof(Keyboardctl), 1);
     ⟨initkeyboard() sanity check kc 195c⟩

     ⟨initkeyboard() sanitize file 195a⟩
     kc->file = strdup(file);
     kc->consfd = open(file, ORDWR|OCEXEC);

     t = malloc(strlen(file)+16);
     ⟨initkeyboard() sanity check t and consfd 195d⟩
     sprint(t, "%sctl", file);
     kc->ctlfd = open(t, OWRITE|OCEXEC);
     ⟨initkeyboard() sanity check ctlfd 195e⟩

     if(ctlkeyboard(kc, "rawon") < 0){
         fprint(2, "initkeyboard: can't turn on raw mode on %s: %r\n", t);
         close(kc->ctlfd);
         goto Error2;
     }
     free(t);

     kc->c = chancreate(sizeof(Rune), 20);
```

```
    proccreate(_ioproc, kc, 4096);

    return kc;
}
```

⟨initkeyboard() *sanitize file* 195a⟩≡ (194b)

```
if(file == nil)
    file = "/dev/cons";
```

⟨*function* ctlkeyboard 195b⟩≡ (417b)

```
int
ctlkeyboard(Keyboardctl *kc, char *m)
{
    return write(kc->ctlfd, m, strlen(m));
}
```

⟨initkeyboard() *sanity check kc* 195c⟩≡ (194b)

```
if(kc == nil)
    return nil;
```

⟨initkeyboard() *sanity check t and consfd* 195d⟩≡ (194b)

```
if(kc->consfd<0 || t == nil){
Error1:
    free(kc);
    return nil;
}
```

⟨initkeyboard() *sanity check ctlfd* 195e⟩≡ (194b)

```
if(kc->ctlfd < 0){
    fprint(2, "initkeyboard: can't open %s: %r\n", t);
Error2:
    close(kc->consfd);
    free(t);
    goto Error1;
}
```

11.3.4 IO process and communication channels

⟨*function* _ioproc 195f⟩≡ (417b)

```
static
void
_ioproc(void *arg)
{
    Keyboardctl *kc = arg;
    int m, n;
    char buf[20];
    Rune r;

    threadsetname("kbdproc");
    kc->pid = getpid();

    n = 0;
    for(;;){
        while(n>0 && fullrune(buf, n)){
            m = chartorune(&r, buf);
            n -= m;
            memmove(buf, buf+m, n);
            // sent it!
            send(kc->c, &r);
```

```
        }
        // blocking call
        m = read(kc->consfd, buf+n, sizeof buf-n);
        ⟨_ioproc() sanity check m from read (keyboard.c) 196a⟩
        n += m;
    }
}
```

⟨_ioproc() *sanity check m from read (keyboard.c)* 196a⟩≡ (195f)

```
if(m <= 0){
    yield(); /* if error is due to exiting, we'll exit here */
    fprint(2, "keyboard read error: %r\n");
    threadexits("error");
}
```

⟨*function* closekeyboard 196b⟩≡ (417b)

```
void
closekeyboard(Keyboardctl *kc)
{
    if(kc == nil)
        return;

    postnote(PNPROC, kc->pid, "kill");
    close(kc->ctlfd);
    close(kc->consfd);
    free(kc->file);
    free(kc->c);
    free(kc);
}
```

Chapter 12

Overlapping Layers

12.1 Additional data structures

12.1.1 Base layer: Screen (and Memscreen)

Screen

⟨*struct* Screen 197a⟩≡ (391a)

```
struct Screen
{
    Display *display;   /* display holding data */
    int     id;         /* id of system-held Screen */

    Image   *image;     /* unused; for reference only */
    Image   *fill;      /* color to paint behind windows */
};
```

DScreen

⟨*struct* DScreen 197b⟩≡ (476)

```
struct DScreen
{
    int     id;

    DImage      *dimage;
    DImage      *dfill;

    ⟨DScreen other fields 197c⟩

    // Extra
    int     ref;
    ⟨DScreen extra fields 197e⟩
};
```

⟨DScreen *other fields* 197c⟩≡ (197b) 198e ▷

```
bool     public;
```

⟨*global* dscreen 197d⟩≡ (479a)

```
// list<ref<DScreen>> (next = DScreen.next)
static  DScreen*    dscreen;
```

⟨DScreen *extra fields* 197e⟩≡ (197b)

```
// list<ref<DScreen>> (head = dscreen)
DScreen*    next;
```

⟨*function* drawlookupdscreen 198a⟩≡ (479a)

```
DScreen*
drawlookupdscreen(int id)
{
    DScreen *s;

    s = dscreen;
    while(s){
        if(s->id == id)
            return s;
        s = s->next;
    }
    return nil;
}
```

CScreen

⟨Client *layer fields* 198b⟩≡ (35d)

```
// list<ref_own<DScreen>> (next = CScreen.next)
CScreen*    cscreen;
```

⟨*struct* CScreen 198c⟩≡ (476)

```
struct CScreen
{
    // ref_shared<DScreen>
    DScreen*    dscreen;

    CScreen*    next;
};
```

⟨*function* drawlookupscreen 198d⟩≡ (479a)

```
DScreen*
drawlookupscreen(Client *client, int id, CScreen **cs)
{
    CScreen *s;

    s = client->cscreen;
    while(s){
        if(s->dscreen->id == id){
            *cs = s;
            return s->dscreen;
        }
        s = s->next;
    }
    error(Enodrawscreen);
    return nil;
}
```

⟨DScreen *other fields* 198e⟩+≡ (197b) ◁197c 198f▷

```
Client*     owner;
```

Memscreen

⟨DScreen *other fields* 198f⟩+≡ (197b) ◁198e

```
// ref_own<Memscreen>
Memscreen*  screen;
```

⟨*struct* Memscreen 199a⟩≡ (395)

```
struct Memscreen
{
    Memimage *image;  /* upon which all layers are drawn */
    Memimage *fill;  /* if non-zero, picture to use when repainting */

    ⟨Memscreen stack windows fields 200a⟩
};
```

12.1.2 Layer (and Memlayer)

⟨Image *layer fields* 199b⟩≡ (37c)

```
Screen      *screen;    /* nil if not a window */
```

⟨DImage *layer fields* 199c⟩≡ (39f)

```
DScreen*    dscreen;    /* nil if not a window */
```

Memlayer

⟨MemImage *layer fields* 199d⟩≡ (37e)

```
// ref_own<Memlayer>
Memlayer *layer; /* nil if not a layer*/
```

⟨*struct* Memlayer 199e⟩≡ (395)

```
struct Memlayer
{
    Rectangle  screenr; /* true position of layer on screen */
    Point   delta; /* add delta to go from image coords to screen */

    // ref<Memscreen>
    Memscreen *screen; /* screen this layer belongs to */

    // ref_own<Memimage> (in image coords)
    Memimage *save; /* save area for obscured parts */


    ⟨Memlayer stack windows fields 200b⟩
    ⟨Memlayer refresh fields 309f⟩
    ⟨Memlayer other fields 199f⟩
};
```

⟨Memlayer *other fields* 199f⟩≡ (199e)

```
bool  clear; /* layer is fully visible */
```

Display.windows

⟨Display *other fields* 199g⟩+≡ (34e) ◁37d 233b▷

```
// list<ref<Window>> (next = Image.next)
Image   *windows;
```

⟨Image *extra fields* 199h⟩≡ (37c)

```
Image       *next;  /* next in list of windows */
```

12.1.3 Layer stack

⟨Memscreen *stack windows fields* 200a⟩≡ (199a)

```
// list<ref<Memimage>> (next = Memimage.layer->rear)
Memimage *frontmost; /* frontmost layer on screen */
// list<ref<Memimage>> (next = Memimage.layer->front)
Memimage *rearmost; /* rearmost layer on screen */
```

⟨Memlayer *stack windows fields* 200b⟩≡ (199e)

```
Memimage *front; /* window in front of this one */
Memimage *rear; /* window behind this one*/
```

12.2 Base layer allocation

12.2.1 API

⟨*function* allocscreen 200c⟩≡ (415a)

```
Screen*
allocscreen(Image *image, Image *fill, bool public)
{
    Display *d;
    Screen *s;
    int id, try;
    byte *a;

    d = image->display;
    ⟨allocscreen() sanity check images have same display 200d⟩
    s = malloc(sizeof(Screen));
    ⟨allocscreen() sanity check s 200e⟩
    SET(id);
    for(try=0; try<25; try++){
        /* loop until find a free id */
        ⟨allocscreen() marshall allocate screen message 201f⟩
    }
    s->display = d;
    s->id = id;
    s->image = image;
    s->fill = fill;
    ⟨allocscreen() sanity check screen image 200f⟩
    return s;
}
```

⟨allocscreen() *sanity check images have same display* 200d⟩≡ (200c)

```
if(d != fill->display){
    werrstr("allocscreen: image and fill on different displays");
    return nil;
}
```

⟨allocscreen() *sanity check s* 200e⟩≡ (200c)

```
if(s == nil)
    return nil;
```

⟨allocscreen() *sanity check screen image* 200f⟩≡ (200c)

```
assert(s->image && s->image->chan != 0);
```

⟨*function* freescreen 201a⟩≡ (415a)

```
errorneg1
freescreen(Screen *s)
{
    Display *d;
    byte *a;

    ⟨freescreen() sanity check s 201b⟩
    d = s->display;
    ⟨freescreen() marshall free screen message 202b⟩
    free(s);
    return OK_1;
}
```

⟨freescreen() *sanity check s* 201b⟩≡ (201a)

```
if(s == nil)
    return 0;
```

⟨*function* publicscreen 201c⟩≡ (415a)

```
Screen*
publicscreen(Display *d, int id, ulong chan)
{
    Screen *s;
    byte *a;

    s = malloc(sizeof(Screen));
    ⟨publicscreen() sanity check s 201d⟩

    ⟨publicscreen() marshall use public screen message 202d⟩

    s->display = d;
    s->id = id;

    s->image = nil;
    s->fill = nil;

    return s;
}
```

⟨publicscreen() *sanity check s* 201d⟩≡ (201c)

```
if(s == nil)
    return nil;
```

12.2.2 Marshalling

⟨*global* screenid 201e⟩≡ (415a)

```
static int screenid;
```

⟨allocscreen() *marshall allocate screen message* 201f⟩≡ (200c)

```
// allocate screen: 'A' id[4] imageid[4] fillid[4] public[1]
a = bufimage(d, 1+4+4+4+1);
⟨allocscreen() sanity check a 202a⟩
id = ++screenid;
a[0] = 'A';
BPLONG(a+1, id);
BPLONG(a+5, image->id);
BPLONG(a+9, fill->id);
a[13] = public;
if(flushimage(d, false) != -1)
    break;
```

⟨allocscreen() *sanity check a* 202a⟩≡ (201f)

```
if(a == nil){
    free(s);
    return nil;
}
```

⟨freescreen() *marshall free screen message* 202b⟩≡ (201a)

```
// free screen: 'F' id[4]
a = bufimage(d, 1+4);
⟨freescreen() sanity check a 202c⟩
a[0] = 'F';
BPLONG(a+1, s->id);
/*
 * flush(true) because screen is likely holding last reference to
 * window, and want it to disappear visually.
 */
if(flushimage(d, true) < 0)
    return ERROR_NEG1;
```

⟨freescreen() *sanity check a* 202c⟩≡ (202b)

```
if(a == nil)
    return ERROR_NEG1;
```

⟨publicscreen() *marshall use public screen message* 202d⟩≡ (201c)

```
// use public screen: 'S' id[4] chan[4]
a = bufimage(d, 1+4+4);
⟨publicscreen() sanity check a 202e⟩
a[0] = 'S';
BPLONG(a+1, id);
BPLONG(a+5, chan);
if(flushimage(d, false) < 0)
    goto Error;
```

⟨publicscreen() *sanity check a* 202e⟩≡ (202d)

```
if(a == nil){
Error:
    free(s);
    return nil;
}
```

12.2.3 Unmarshalling

⟨drawmesg() *locals* 202f⟩+≡ (71) ◁160c 203c▷

```
DImage *dsrc;
```

⟨drawmesg() *cases* 202g⟩+≡ (71) ◁177g 203d▷

```
/* allocate screen: 'A' id[4] imageid[4] fillid[4] public[1] */
case 'A':
    printmesg(fmt="LLLb", a, false);
    m = 1+4+4+4+1;
    ⟨drawmesg() sanity check n with m 75d⟩
    dstid = BGLONG(a+1);
    ⟨drawmesg() when allocate screen, sanity check dstid 203a⟩
    ddst = drawlookup(client, BGLONG(a+5), true);
    dsrc = drawlookup(client, BGLONG(a+9), true);
    ⟨drawmesg() when allocate screen, sanity check ddst and dsrc 203b⟩

    if(drawinstallscreen(client, nil, dstid, ddst, dsrc, a[13]) == 0)// The call
        error(Edrawmem);
    continue;
```

⟨drawmesg() *when allocate screen, sanity check dstid* 203a⟩≡ (202g)

```
if(dstid == 0)
    error(Ebadarg);
if(drawlookupdscreen(dstid))
    error(Escreenexists);
```

⟨drawmesg() *when allocate screen, sanity check ddst and dsrc* 203b⟩≡ (202g)

```
if(ddst==nil || dsrc==nil)
    error(Enodrawimage);
```

⟨drawmesg() *locals* 203c⟩+≡ (71) ◁202f 203e▷

```
CScreen *cs;
```

⟨drawmesg() *cases* 203d⟩+≡ (71) ◁202g 203f▷

```
/* free screen: 'F' id[4] */
case 'F':
    printmesg(fmt="L", a, false);
    m = 1+4;
    ⟨drawmesg() sanity check n with m 75d⟩
    drawlookupscreen(client, BGLONG(a+1), &cs);
    drawuninstallscreen(client, cs); // The call
    continue;
```

⟨drawmesg() *locals* 203e⟩+≡ (71) ◁203c 207a▷

```
DScreen *dscrn;
```

⟨drawmesg() *cases* 203f⟩+≡ (71) ◁203d 212d▷

```
/* use public screen: 'S' id[4] chan[4] */
case 'S':
    printmesg(fmt="Ll", a, false);
    m = 1+4+4;
    ⟨drawmesg() sanity check n with m 75d⟩
    dstid = BGLONG(a+1);
    ⟨drawmesg() when use public screen, sanity check dstid 203g⟩
    dscrn = drawlookupdscreen(dstid);
    ⟨drawmesg() when use public screen, sanity check dscrn 203h⟩
    if(drawinstallscreen(client, dscrn, 0, nil, nil, false) == 0) // The call
        error(Edrawmem);
    continue;
```

⟨drawmesg() *when use public screen, sanity check dstid* 203g⟩≡ (203f)

```
if(dstid == 0)
    error(Ebadarg);
```

⟨drawmesg() *when use public screen, sanity check dscrn* 203h⟩≡ (203f)

```
if(dscrn == nil || (!dscrn->public && dscrn->owner != client))
    error(Enodrawscreen);
if(dscrn->screen->image->chan != BGLONG(a+5))
    error("inconsistent chan");
```

12.2.4 Client and Memscreen connection: CScreen

⟨*function* drawinstallscreen 203i⟩≡ (479a)

```
Memscreen*
drawinstallscreen(Client *client, DScreen *d, int id, DImage *dimage, DImage *dfill, bool public)
{
    Memscreen *s;
    CScreen *c;
```

```
    c = malloc(sizeof(CScreen));
    ⟨drawinstallscreen() sanity check dimage 204a⟩
    ⟨drawinstallscreen() sanity check c 204b⟩

    if(d == nil){
        d = malloc(sizeof(DScreen));
        ⟨drawinstallscreen() sanity check d 204c⟩
        s = malloc(sizeof(Memscreen));
        ⟨drawinstallscreen() sanity check s 205a⟩

        d->id = id;
        d->dimage = dimage;
        d->dfill = dfill;

        d->screen = s;
        if(dimage){
            s->image = dimage->image;
            dimage->ref++;
        }
        if(dfill){
            s->fill = dfill->image;
            dfill->ref++;
        }
        // no windows yet
        s->frontmost = nil;
        s->rearmost = nil;

        d->public = public;
        d->owner = client;
        d->ref = 0;

        // add_list(d, dscreen)
        d->next = dscreen;
        dscreen = d;
    }

    c->dscreen = d;
    d->ref++;

    // add_list(c, client->cscreen)
    c->next = client->cscreen;
    client->cscreen = c;

    return d->screen;
}
```

⟨drawinstallscreen() *sanity check dimage* 204a⟩≡ (203i)

```
if(dimage && dimage->image && dimage->image->chan == 0)
    panic("bad image %p in drawinstallscreen", dimage->image);
```

⟨drawinstallscreen() *sanity check c* 204b⟩≡ (203i)

```
if(c == nil)
    return nil;
```

⟨drawinstallscreen() *sanity check d* 204c⟩≡ (203i)

```
if(d == nil){
    free(c);
    return nil;
}
```

⟨drawinstallscreen() *sanity check s* 205a⟩≡ (203i)

```
if(s == nil){
    free(c);
    free(d);
    return nil;
}
```

⟨drawclose() *free screens* 205b⟩≡ (65e)

```
while(cl->cscreen)
    drawuninstallscreen(cl, cl->cscreen);
```

⟨*function* drawuninstallscreen 205c⟩≡ (479a)

```
void
drawuninstallscreen(Client *client, CScreen *this)
{
    CScreen *cs, *next;

    // remove_list(this, client->cscreen)
    cs = client->cscreen;
    if(cs == this){
        client->cscreen = this->next;

        drawfreedscreen(this->dscreen);
        free(this);
        return;
    }
    while(next = cs->next){ /* assign = */
        if(next == this){
            cs->next = this->next;

            drawfreedscreen(this->dscreen);
            free(this);
            return;
        }
        cs = next;
    }
}
```

⟨*function* drawfreedscreen 205d⟩≡ (479a)

```
void
drawfreedscreen(DScreen *this)
{
    DScreen *ds, *next;

    this->ref--;
    ⟨drawfreedscreen() sanity check reference count 206a⟩
    if(this->ref > 0)
        return;
    // else

    // remove_list(this, dscreen)
    ds = dscreen;
    if(ds == this){
        dscreen = this->next;
        goto Found;
    }
    while(next = ds->next){ /* assign = */
        if(next == this){
            ds->next = this->next;
            goto Found;
```

```
        }
        ds = next;
    }
    error(Enodrawimage);

    Found:
    if(this->dimage)
        drawfreedimage(this->dimage);
    if(this->dfill)
        drawfreedimage(this->dfill);
    free(this->screen);
    free(this);
}
```

⟨drawfreedscreen() *sanity check reference count* 206a⟩≡ (205d)

```
if(this->ref < 0)
    print("negative ref in drawfreedscreen\n");
```

12.3 Layers creation

12.3.1 API

⟨*function* allocwindow 206b⟩≡ (415b)

```
Image*
allocwindow(Screen *s, Rectangle r, int ref, rgba val)
{
    return _allocwindow(nil, s, r, ref, val);
}
```

⟨*function* _allocwindow 206c⟩≡ (415b)

```
Image*
_allocwindow(Image *i, Screen *s, Rectangle r, int ref, rgba val)
{
    Display *d;

    d = s->display;
    i = _allocimage(i, d, r, d->screenimage->chan, false, val, s->id, ref);
    ⟨_allocwindow() sanity check i 206d⟩

    i->screen = s;
    // add_list(i, display->windows)
    i->next = s->display->windows;
    s->display->windows = i;

    return i;
}
```

⟨_allocwindow() *sanity check i* 206d⟩≡ (206c)

```
if(i == nil)
    return nil;
```

⟨_freeimage1() *if screen* 206e⟩≡ (74d)

```
if(i->screen){
    // remove_list(i, d->windows)
    w = d->windows;
    if(w == i)
        d->windows = i->next;
    else
```

```
        while(w){
            if(w->next == i){
                w->next = i->next;
                break;
            }
            w = w->next;
        }
}
```

12.3.2 Unmarshalling

⟨drawmesg() *locals* 207a⟩+≡ (71) ◁203e 212c▷

```
Memscreen *scrn;
Memimage *l;
```

⟨drawmesg() *allocate image case, if screen id* 207b⟩≡ (75c)

```
if(scrnid){
    dscrn = drawlookupscreen(client, scrnid, &cs);
    scrn = dscrn->screen;
    ⟨drawmesg() when allocate window, sanity check repl and chan 207c⟩
    ⟨drawmesg() when allocate window, set reffn 309i⟩
    l = memlalloc(scrn, r, reffn, nil, value); // The call
    ⟨drawmesg() when allocate window, sanity check l 207d⟩
    ⟨drawmesg() when allocate window, addflush 327a⟩

    // similar to regular allocate image case
    l->clipr = clipr;
    rectclip(&l->clipr, r);

    if(drawinstall(client, dstid, l, dscrn) == 0){
        memldelete(l);
        error(Edrawmem);
    }
    dscrn->ref++;

    ⟨drawmesg() when allocate window, if reffn 310a⟩
    continue;
}
```

⟨drawmesg() *when allocate window, sanity check repl and chan* 207c⟩≡ (207b)

```
if(repl || chan != scrn->image->chan)
    error("image parameters incompatible with screen");
```

⟨drawmesg() *when allocate window, sanity check l* 207d⟩≡ (207b)

```
if(l == nil)
    error(Edrawmem);
```

12.3.3 Algorithm

⟨*function* memlalloc 207e⟩≡ (458b)

```
Memimage*
memlalloc(Memscreen *s, Rectangle screenr, Refreshfn refreshfn, void *refreshptr, rgba val)
{
    Memimage *n;
    Memlayer *l;
    ⟨memlalloc() other locals 208c⟩

    ⟨memlalloc() set paint once 209a⟩
```

```
    // share pixels data with screen image
    n = allocmemimaged(screenr, s->image->chan, s->image->data);
    ⟨memlalloc() sanity check n 209c⟩
    // overwrite zero and width derived from screenr in allocmemimaged
    // because the rectangle here does not match the data, it's a
    // subrectangle. Zero and width should remain the same.
    n->zero = s->image->zero;
    n->width = s->image->width;

    l = malloc(sizeof(Memlayer));
    ⟨memlalloc() sanity check l 209d⟩

    l->screen = s;
    l->screenr = screenr;
    l->delta = Pt(0,0); // can be changed later by originwindow()
    l->clear = false;
    ⟨memlalloc() allocate save image 208a⟩

    n->layer = l;
    // ok no more sanity check, we can make the connection
    n->data->ref++;

    ⟨memlalloc() set refresh fields part1 310c⟩
    ⟨memlalloc() manage stack of windows 208b⟩
    ⟨memlalloc() set refresh fields part2 310d⟩

    ⟨memlalloc() paint with requested color 209b⟩
    return n;
}
```

⟨memlalloc() *allocate save image* 208a⟩≡ (207e)

```
⟨memlalloc() if refreshfn 310b⟩
else{
    l->save = allocmemimage(screenr, s->image->chan);
    ⟨memlalloc() sanity check l save 209e⟩
    /* allocmemimage doesn't initialize memory; this paints save area */
    if(val != DNofill)
        memfillcolor(l->save, val);
}
```

⟨memlalloc() *manage stack of windows* 208b⟩≡ (207e)

```
/* start with new window behind all existing ones */

// add_end_double_list(l, s->frontmost, s->rearmost)
l->front = s->rearmost;
l->rear = nil;
if(s->rearmost)
    s->rearmost->layer->rear = n;
s->rearmost = n;
if(s->frontmost == nil)
    s->frontmost = n;

/* now pull new window to front */
_memltofrontfill(n, val != DNofill);
```

⟨memlalloc() *other locals* 208c⟩≡ (207e)

```
static Memimage *paint;
```

⟨memlalloc() *set paint once* 209a⟩≡ (207e)

```
if(paint == nil){
    paint = allocmemimage(Rect(0,0,1,1), RGBA32);
    ⟨memlalloc() sanity check paint 209f⟩
    paint->flags |= Frepl;
    paint->clipr = Rect(-0x3FFFFFF, -0x3FFFFFF, 0x3FFFFFF, 0x3FFFFFF);
}
```

⟨memlalloc() *paint with requested color* 209b⟩≡ (207e)

```
/*
 * paint with requested color; previously exposed areas are already right
 * if this window has backing store, but just painting the whole thing is simplest.
 */
if(val != DNofill){
    memsetchan(paint, n->chan);
    memfillcolor(paint, val);
    memdraw(n, n->r, paint, n->r.min, nil, n->r.min, S);
}
```

⟨memlalloc() *sanity check n* 209c⟩≡ (207e)

```
if(n == nil)
    return nil;
```

⟨memlalloc() *sanity check l* 209d⟩≡ (207e)

```
if(l == nil){
    free(n);
    return nil;
}
```

⟨memlalloc() *sanity check l save* 209e⟩≡ (208a)

```
if(l->save == nil){
    free(l);
    free(n);
    return nil;
}
```

⟨memlalloc() *sanity check paint* 209f⟩≡ (209a)

```
if(paint == nil)
    return nil;
```

⟨*function* memldelete 209g⟩≡ (458d)

```
void
memldelete(Memimage *i)
{
    Memlayer *l;
    Memscreen *s;

    l = i->layer;
    s = i->layer->screen;

    /* free backing store and disconnect refresh, to make pushback fast */
    freememimage(l->save);
    l->save = nil;
    l->refreshptr = nil;
    memltorear(i);

    /* window is now the rearmost;  clean up screen structures and deallocate */

    if(s->fill){
        i->clipr = i->r;
```

```
        memdraw(i, i->r, s->fill, i->r.min, nil, i->r.min, S);
    }
    ⟨memldelete() manage stack of windows 210a⟩

    free(l);
    freememimage(i);
}
```

⟨memldelete() *manage stack of windows* 210a⟩≡ (209g)

```
// remove_double_list(l, s->frontmost, s->rearmost)
if(l->front){
    l->front->layer->rear = nil;
    s->rearmost = l->front;
}else{
    s->frontmost = nil;
    s->rearmost = nil;
}
```

12.3.4 Client and Memimage connection: DImage

⟨drawinstall() *install dscreen* 210b⟩≡ (77e)

```
d->dscreen = dscreen;
```

⟨drawfreedimage() *if dscreen* 210c⟩≡ (78c)

```
ds = dimage->dscreen;
if(ds){
    l = dimage->image;
    ⟨drawfreedimage() addflush 326e⟩
    ⟨drawfreedimage() free refreshptr 308d⟩

    if(drawgoodname(dimage))
        memldelete(l);
    else
        memlfree(l);

    drawfreedscreen(ds); // one less reference
}
```

⟨drawgoodname() *if DImage* d *is a window* 210d⟩≡ (88h)

```
/* if window, validate the screen's own images */
if(d->dscreen)
    if(!drawgoodname(d->dscreen->dimage)
    || !drawgoodname(d->dscreen->dfill))
        return false;
```

⟨*function* memlfree 210e⟩≡ (458d)

```
/*
 * Just free the data structures, don't do graphics
 */
void
memlfree(Memimage *i)
{
    Memlayer *l;

    l = i->layer;
    freememimage(l->save);
    free(l);
    freememimage(i);
}
```

12.4 Layer stack manipulation

12.4.1 API

⟨*function* `topwindow` 211a⟩≡ (415b)

```
void
topwindow(Image *w)
{
    ⟨xxxmwindow() sanity check window w 211e⟩
    topbottom(&w, 1, true);
}
```

⟨*function* `bottomwindow` 211b⟩≡ (415b)

```
void
bottomwindow(Image *w)
{
    ⟨xxxmwindow() sanity check window w 211e⟩
    topbottom(&w, 1, false);
}
```

⟨*function* `topnwindows` 211c⟩≡ (415b)

```
void
topnwindows(Image **w, int n)
{
    topbottom(w, n, true);
}
```

⟨*function* `bottomnwindows` 211d⟩≡ (415b)

```
void
bottomnwindows(Image **w, int n)
{
    topbottom(w, n, false);
}
```

⟨`xxxmwindow()` *sanity check window w* 211e⟩≡ (211)

```
if(w->screen == nil)
    return;
```

12.4.2 Marshalling

⟨*function* `topbottom` 211f⟩≡ (415b)

```
static
void
topbottom(Image **w, int n, bool top)
{
    int i;
    byte *b;
    Display *d;

    ⟨topbottom() sanity check n 212a⟩
    ⟨topbottom() sanity check images have same display 212b⟩

    // top or bottom windows: 't' top[1] nw[2] n*id[4]
    b = bufimage(d, 1+1+2+4*n);
    b[0] = 't';
    b[1] = top;
    BPSHORT(b+2, n);
    for(i=0; i<n; i++)
        BPLONG(b+4+4*i, w[i]->id);
}
```

⟨topbottom() *sanity check n* 212a⟩≡ (211f)

```
if(n < 0){
Ridiculous:
    fprint(2, "top/bottom: ridiculous number of windows\n");
    return;
}
if(n == 0)
    return;
if(n > (w[0]->display->bufsize - 100)/4)
    goto Ridiculous;
```

⟨topbottom() *sanity check images have same display* 212b⟩≡ (211f)

```
/*
 * this used to check that all images were on the same screen.
 * we don't know the screen associated with images we acquired
 * by name.  instead, check that all images are on the same display.
 * the display will check that they are all on the same screen.
 */
d = w[0]->display;
for(i=1; i<n; i++)
    if(w[i]->display != d){
        fprint(2, "top/bottom: windows not on same screen\n");
        return;
    }
```

12.4.3 Unmarshalling

⟨drawmesg() *locals* 212c⟩+≡ (71) ◁207a 309g▷

```
int nw;
Memimage **lp;
```

⟨drawmesg() *cases* 212d⟩+≡ (71) ◁203f 218c▷

```
/* top or bottom windows: 't' top[1] nw[2] n*id[4] */
case 't':
    printmesg(fmt="bsL", a, false);
    m = 1+1+2;
    ⟨drawmesg() sanity check n with m 75d⟩
    nw = BGSHORT(a+2);
    ⟨drawmesg() when top or bottom windows, sanity check nw 213a⟩
    m += nw*4;
    ⟨drawmesg() sanity check n with m 75d⟩
    lp = malloc(nw * sizeof(Memimage*));
    ⟨drawmesg() when top or bottom windows, sanity check lp 213b⟩
    for(j=0; j<nw; j++)
        lp[j] = drawimage(client, a+1+1+2+j*4);
    ⟨drawmesg() when top or bottom windows, sanity check windows 213c⟩

    if(a[1])
        memltofrontn(lp, nw); // The call
    else
        memltorearn(lp, nw); // The call

    ⟨drawmesg() when top or bottom windows, addflush 327b⟩
    ⟨drawmesg() when top or bottom windows, refresh 309c⟩

    poperror();
    free(lp);
    continue;
```

⟨drawmesg() *when top or bottom windows, sanity check nw* 213a⟩≡ (212d)

```
if(nw < 0)
    error(Ebadarg);
if(nw == 0)
    continue;
```

⟨drawmesg() *when top or bottom windows, sanity check lp* 213b⟩≡ (212d)

```
if(lp == nil)
    error(Enomem);
if(waserror()){
    free(lp);
    nexterror();
}
```

⟨drawmesg() *when top or bottom windows, sanity check windows* 213c⟩≡ (212d)

```
if(lp[0]->layer == nil)
    error("images are not windows");
for(j=1; j<nw; j++)
    if(lp[j]->layer->screen != lp[0]->layer->screen)
        error("images not on same screen");
```

12.4.4 Algorithm

To front

⟨*function* memltofrontn 213d⟩≡ (460b)

```
void
memltofrontn(Memimage **ip, int n)
{
    Memimage *i, *front;

    ⟨memltofrontn() sanity check n 213e⟩
    front = nil;
    while(--n >= 0){
        i = *ip++;
        _memltofront(i, front, true);
        front = i;
    }
    _memlsetclear(front->layer->screen);
}
```

⟨memltofrontn() *sanity check n* 213e⟩≡ (215b 213d)

```
if(n == 0)
    return;
```

⟨*function* memltofront 213f⟩≡ (460b)

```
void
memltofront(Memimage *i)
{
    _memltofront(i, nil, true);
    _memlsetclear(i->layer->screen);
}
```

⟨*function* _memltofront 213g⟩≡ (460b)

```
/*
 * Pull i towards top of screen, just behind front
*/
static
void
```

```
_memltofront(Memimage *i, Memimage *front, bool fill)
{
    Memlayer *l;
    Memscreen *s;
    Memimage *f;
    Rectangle x;
    bool overlap;
    ⟨_memltofront() other locals 214a⟩

    l = i->layer;
    s = l->screen;

    while(l->front != front){
        f = l->front;
        x = l->screenr;

        // i will now pass in front of f, so hide f
        overlap = rectclip(&x, f->layer->screenr);
        if(overlap){
            memlhide(f, x);
            f->layer->clear = false;
        }
        ⟨_memltofront() put f behind i 214b⟩
        if(overlap && fill)
            memlexpose(i, x);
    }
}
```

⟨_memltofront() *other locals* 214a⟩≡ (213g)

```
Memimage *ff, *rr;
```

⟨_memltofront() *put f behind i* 214b⟩≡ (213g)

```
/* swap l and f in screen's list */
// swap_double_list(l, f, s->frontmost, s->rearmost)
ff = f->layer->front;
rr = l->rear;

if(ff == nil)
    s->frontmost = i;
else
    ff->layer->rear = i;

if(rr == nil)
    s->rearmost = f;
else
    rr->layer->front = f;

l->front = ff;
l->rear = f; // f is now behind i
f->layer->front = i;
f->layer->rear = rr;
```

⟨*function* _memlsetclear 214c⟩≡ (458d)

```
void
_memlsetclear(Memscreen *s)
{
    Memimage *i, *j;
    Memlayer *l;

    for(i = s->rearmost; i; i = i->layer->front){
```

```
        l = i->layer;
        l->clear = rectinrect(l->screenr, l->screen->image->clipr);
        if(l->clear)
            for(j = l->front; j; j = j->layer->front)
                if(rectXrect(l->screenr, j->layer->screenr)){
                    l->clear = false;
                    break;
                }
    }
}
```

⟨*function* _memltofrontfill 215a⟩≡ (460b)

```
void
_memltofrontfill(Memimage *i, bool fill)
{
    _memltofront(i, nil, fill);
    _memlsetclear(i->layer->screen);
}
```

To rear

⟨*function* memltorearn 215b⟩≡ (460c)

```
void
memltorearn(Memimage **ip, int n)
{
    Memimage *i, *rear;

    ⟨memltofrontn() sanity check n 213e⟩
    rear = nil;
    while(--n >= 0){
        i = *ip++;
        _memltorear(i, rear);
        rear = i;
    }
    _memlsetclear(rear->layer->screen);
}
```

⟨*function* memltorear 215c⟩≡ (460c)

```
void
memltorear(Memimage *i)
{
    _memltorear(i, nil);
    _memlsetclear(i->layer->screen);
}
```

⟨*function* _memltorear 215d⟩≡ (460c)

```
void
_memltorear(Memimage *i, Memimage *rear)
{
    Memlayer *l;
    Memscreen *s;
    Memimage *f, *r, *rr;
    Rectangle x;
    bool overlap;

    l = i->layer;
    s = l->screen;

    while(l->rear != rear){
```

```
        r = l->rear;
        x = l->screenr;

        overlap = rectclip(&x, r->layer->screenr);
        if(overlap){
            memlhide(i, x);
            l->clear = false;
        }
        ⟨_memltorear() put i behind r 216a⟩
        if(overlap)
            memlexpose(r, x);
    }
}
```

⟨_memltorear() *put i behind r* 216a⟩≡ (215d)

```
/* swap l and r in screen's list */
// swap_end_double_list(i, r, s->frontmost, s->rearmost)
rr = r->layer->rear;
f = l->front;

if(rr == nil)
    s->rearmost = i;
else
    rr->layer->front = i;

if(f == nil)
    s->frontmost = r;
else
    f->layer->rear = r;

l->rear = rr;
l->front = r; // i is now behind r
r->layer->rear = i;
r->layer->front = f;
```

12.5 Layer visibility

12.5.1 memlhide()

⟨*function* memlhide 216b⟩≡ (459a)

```
void
memlhide(Memimage *i, Rectangle screenr)
{
    if(i->layer->save == nil)
        return;
    if(!rectclip(&screenr, i->layer->screen->image->r))
        return;

    _memlayerop(lhideop, i, screenr, screenr, i->layer);
}
```

⟨*function* lhideop 216c⟩≡ (459a)

```
/*
 * Hide puts that portion of screenr now on the screen into the window's
 * save area.
 * Expose puts that portion of screenr now in the save area onto the screen.
 *
 * Hide and Expose both require that the layer structures in the screen
```

```
 * match the geometry they are being asked to update, that is, they update the
 * save area (hide) or screen (expose) based on what those structures tell them.
 * This means they must be called at the correct time during window shuffles.
 */
static
void
lhideop(Memimage *src, Rectangle screenr, Rectangle clipr, void *etc, bool insave)
{
    Memlayer *l = etc;
    Rectangle r;

    USED(clipr.min.x);
    USED(insave);

    if(src != l->save){ /* do nothing if src is already in save area */
        r = rectsubpt(screenr, l->delta);
        memdraw(l->save, r, src, screenr.min, nil, screenr.min, S);
    }
}
```

12.5.2 memlexpose()

⟨*function* memlexpose 217a⟩≡ (459a)

```
void
memlexpose(Memimage *i, Rectangle screenr)
{
    if(!rectclip(&screenr, i->layer->screen->image->r))
        return;
    _memlayerop(lexposeop, i, screenr, screenr, i->layer);
}
```

⟨*function* lexposeop 217b⟩≡ (459a)

```
static
void
lexposeop(Memimage *dst, Rectangle screenr, Rectangle clipr, void *etc, bool insave)
{
    Memlayer *l = etc;
    Rectangle r;

    USED(clipr.min.x);
    if(insave) /* if dst is save area, don't bother */
        return;
    r = rectsubpt(screenr, l->delta);
    if(l->save)
        memdraw(dst, screenr, l->save, r.min, nil, r.min, S);
    ⟨lexposeop() if not save but refresh method 309b⟩
}
```

12.6 Layer origin

12.6.1 API

⟨*function* originwindow 217c⟩≡ (415b)

```
errorneg1
originwindow(Image *w, Point log, Point scr)
{
    byte *b;
```

```
    Point delta;

    flushimage(w->display, false);

    ⟨originwindow() marshall position window message 218a⟩

    delta = subpt(log, w->r.min);
    // new image coords
    w->r     = rectaddpt(w->r, delta);
    w->clipr = rectaddpt(w->clipr, delta);
    return OK_1;
}
```

12.6.2 Marshalling

⟨originwindow() *marshall position window message* 218a⟩≡ (217c)

```
// position window: 'o' id[4] r.min [2*4] screenr.min [2*4]
b = bufimage(w->display, 1+4+2*4+2*4);
⟨originwindow() sanity check b 218b⟩
b[0] = 'o';
BPLONG(b+1, w->id);
BPLONG(b+5, log.x);
BPLONG(b+9, log.y);
BPLONG(b+13, scr.x);
BPLONG(b+17, scr.y);
if(flushimage(w->display, true) < 0)
    return ERROR_NEG1;
```

⟨originwindow() *sanity check b* 218b⟩≡ (218a)

```
if(b == nil)
    return 0; // really? not -1?
```

12.6.3 Unmarshalling

⟨drawmesg() *cases* 218c⟩+≡ (71) ◁212d 349a▷

```
/* position window: 'o' id[4] r.min [2*4] screenr.min [2*4] */
case 'o':
    printmesg(fmt="LPP", a, false);
    m = 1+4+2*4+2*4;
    ⟨drawmesg() sanity check n with m 75d⟩
    dst = drawimage(client, a+1);
    if(dst->layer){
        drawpoint(&p, a+5);
        drawpoint(&q, a+13);
        r = dst->layer->screenr;

        ni = memlorigin(dst, p, q); // The call

        ⟨drawmesg() when position window, sanity check ni 219a⟩
        if(ni > 0){
            ⟨drawmesg() when position window, addflush 327c⟩
            ⟨drawmesg() when position window, refresh 309d⟩
        }
    }
    // else, could display error
    continue;
```

⟨drawmesg() *when position window, sanity check ni* 219a⟩≡ (218c)

```
if(ni < 0)
    error("image origin failed");
```

12.6.4 Algorithm

⟨*function* memlorigin 219b⟩≡ (459d)

```
/*
 * Place i so i->r.min = log, i->layer->screenr.min == scr.
*/
errorneg1
memlorigin(Memimage *i, Point log, Point scr)
{
    Memlayer *l;
    Memscreen *s;
    Rectangle newr, oldr;
    Point delta;
    bool eqlog, eqscr, wasclear;
    ⟨memlorigin() other locals 219c⟩

    l = i->layer;
    s = l->screen;

    oldr = l->screenr;
    newr = Rect(scr.x, scr.y, scr.x + Dx(oldr), scr.y + Dy(oldr));

    eqscr = eqpt(scr, oldr.min);
    eqlog = eqpt(log, i->r.min);

    if(eqscr && eqlog)
        return 0;

    ⟨memlorigin() allocate new save image if log changed 220a⟩
    /*
     * Bring it to front and move logical coordinate system.
     */
    memltofront(i);
    wasclear = l->clear;
    ⟨memlorigin() set new save image if log changed 220b⟩

    // like in user side
    delta = subpt(log, i->r.min);
    // new image coords
    i->r     = rectaddpt(i->r, delta);
    i->clipr = rectaddpt(i->clipr, delta);

    l->delta = subpt(l->screenr.min, i->r.min);

    if(eqscr)
        return 0;
    // else

    ⟨memlorigin() move window 220e⟩
}
```

New origin

⟨memlorigin() *other locals* 219c⟩≡ (219b) 220d ▷

```
Memimage *nsave;
```

⟨memlorigin() *allocate new save image if log changed* 220a⟩≡ (219b)

```
nsave = nil;
if(!eqlog && l->save != nil){
    nsave = allocmemimage(Rect(log.x, log.y, log.x+Dx(oldr), log.y+Dy(oldr)), i->chan);
    ⟨memlorigin() sanity check nsave 220c⟩
}
```

⟨memlorigin() *set new save image if log changed* 220b⟩≡ (219b)

```
if(nsave){
    if(!wasclear)
        memimagedraw(nsave, nsave->r, l->save, l->save->r.min, nil, ZP, S);
    freememimage(l->save);
    l->save = nsave;
}
```

⟨memlorigin() *sanity check nsave* 220c⟩≡ (220a)

```
if(nsave == nil)
    return ERROR_NEG1;
```

New screen position

⟨memlorigin() *other locals* 220d⟩+≡ (219b) ◁219c

```
Memimage *shad, *t;
bool overlap;
Rectangle x;
```

⟨memlorigin() *move window* 220e⟩≡ (219b)

```
/*
 * To clean up old position, make a shadow window there, don't paint it,
 * push it behind this one, and (later) delete it.  Because the refresh
 * function for this fake window is a no-op, this will cause no graphics
 * action except to restore the background and expose the windows
 * previously hidden.
 */
shad = memlalloc(s, oldr, memlnorefresh, nil, DNofill);
⟨memlorigin() sanity check shad 221b⟩
⟨memlorigin() manage stack of windows, put shad after front 221a⟩
shad->layer->clear = false;

/*
 * Shadow is now holding down the fort at the old position.
 * Move the window and hide things obscured by new position.
 */
for(t = l->rear->layer->rear; t != nil; t = t->layer->rear){
    x = newr;
    overlap = rectclip(&x, t->layer->screenr);
    if(overlap){
        memlhide(t, x);
        t->layer->clear = false;
    }
}
l->screenr = newr;
l->delta = subpt(scr, i->r.min);
l->clear = rectinrect(newr, l->screen->image->clipr);

/*
 * Everything's covered.  Copy to new position and delete shadow window.
 */
if(wasclear)
```

```
    memdraw(s->image, newr, s->image, oldr.min, nil, ZP, S);
  else
    memlexpose(i, newr);

  memldelete(shad);

  return 1;
```

⟨memlorigin() *manage stack of windows, put shad after front* 221a⟩≡ (220e)

```
  // add_after_double_list(shad, i, s->frontmost, s->rearmost)
  s->frontmost = i; // useless since memltofront(i) above
  if(s->rearmost == i)
    s->rearmost = shad;
  else
    l->rear->layer->front = shad;
  shad->layer->front = i;
  shad->layer->rear = l->rear;
  l->rear = shad;
  l->front = nil;
```

⟨memlorigin() *sanity check shad* 221b⟩≡ (220e)

```
  if(shad == nil)
    return ERROR_NEG1;
```

12.7 Drawing and Layers

12.7.1 Drawing rectangles on overlapping layers

memdraw()

⟨memdraw() *if mask has layer* 221c⟩≡ (92e)

```
  if(mask->layer){
    DBG1("mask->layer != nil\n");
    return; /* too hard, at least for now */
  }
```

⟨memdraw() *locals* 221d⟩≡ (92e) 222a ▷

```
  Memlayer *dl, *sl;
```

⟨memdraw() *when have layers* 221e⟩≡ (92e)

```
  ⟨memdraw() call drawclip, if empty rectangle return 222b⟩

  /*
   Convert to screen coordinates.
   */
  dl = dst->layer;
  ⟨memdraw() convert r if dst is a window 222c⟩

  ⟨memdraw() if dst is fully visible can optimize 223a⟩
  // else

  sl = src->layer;
  ⟨memdraw() convert p0 and srcr if src is a window 222d⟩

  /*
   * Now everything is in screen coordinates.
   * mask is an image.  dst and src are images or obscured layers.
   */
```

```
⟨memdraw() if dst and src are the same window 223c⟩
// else

⟨memdraw() make src an image 222e⟩

/*
 * src is now an image.  dst may be an image or a clear layer
 */
if(dl == nil)
    goto Top;
⟨memdraw() after src is an image, if dst is fully visible can optimize 224b⟩
// else

⟨memdraw() general case where dst is an obscured window 224e⟩
```

⟨memdraw() *locals* 222a⟩+≡ (92e) ◁221d 223b▷

```
Rectangle srcr, mr; // set by drawclip
```

⟨memdraw() *call drawclip, if empty rectangle return* 222b⟩≡ (221e)

```
if(!drawclip(dst, &r, src, &p0, mask, &p1,   &srcr, &mr)){
    return;
}
```

Coordinate conversions

⟨memdraw() *convert* r *if dst is a window* 222c⟩≡ (221e)

```
if(dl != nil){
    // r = rectaddpt(r, dl->delta)
    r.min.x += dl->delta.x;
    r.min.y += dl->delta.y;
    r.max.x += dl->delta.x;
    r.max.y += dl->delta.y;
}
```

⟨memdraw() *convert* p0 *and* srcr *if src is a window* 222d⟩≡ (221e)

```
if(sl != nil){
    // p0 = addpt(p0, sl->delta)
    p0.x += sl->delta.x;
    p0.y += sl->delta.y;
    // srcr = rectaddpt(srcr, sl->delta)
    srcr.min.x += sl->delta.x;
    srcr.min.y += sl->delta.y;
    srcr.max.x += sl->delta.x;
    srcr.max.y += sl->delta.y;
}
```

Source conversion

⟨memdraw() *make src an image* 222e⟩≡ (221e)

```
if(sl){
    ⟨memdraw() if src is fully visible can optimize 224a⟩

    /* relatively rare case; use save area */
    ⟨memdraw() if refresh function for src 308b⟩
    memlhide(src, srcr); // draw the needed pixels in sl->save

    /* convert back to logical coordinates */
    // p0 = subpt(p0, sl->delta)
    p0.x -= sl->delta.x;
```

```
    p0.y -= sl->delta.y;
    // srcr = rectsubpt(srcr, sl->delta)
    srcr.min.x -= sl->delta.x;
    srcr.min.y -= sl->delta.y;
    srcr.max.x -= sl->delta.x;
    srcr.max.y -= sl->delta.y;

    src = src->layer->save; // progress
}
```

Shortcuts

⟨memdraw() *if dst is fully visible can optimize* 223a⟩≡ (221e)

```
Clearlayer:
if(dl!=nil && dl->clear){
    if(src == dst){
        // p0 = addpt(p0, dl->delta)
        p0.x += dl->delta.x;
        p0.y += dl->delta.y;

        src = dl->screen->image; // progress
    }
    dst = dl->screen->image; // progress
    goto Top;
}
```

⟨memdraw() *locals* 223b⟩+≡ (92e) ◁222a 224c▷

```
Rectangle tr;
```

⟨memdraw() *if dst and src are the same window* 223c⟩≡ (221e)

```
/*
 * if dst and src are the same layer, just draw in save area and expose.
 */
if(dl != nil && dst == src){
    ⟨memdraw() if refresh function for dst 308c⟩
    if(rectXrect(r, srcr)){
        ⟨memdraw() set tr to union of r and srcr and adapt p1 223d⟩
        memlhide(dst, tr);
    }else{
        memlhide(dst, r);
        memlhide(dst, srcr);
    }
    memdraw(dl->save, rectsubpt(r,    dl->delta),
            dl->save, subpt(srcr.min, dl->delta),
            mask, p1, op);
    memlexpose(dst, r);
    return;
}
```

⟨memdraw() *set tr to union of r and srcr and adapt p1* 223d⟩≡ (223c)

```
tr = r;
if(srcr.min.x < tr.min.x){
    p1.x += tr.min.x - srcr.min.x;
    tr.min.x = srcr.min.x;
}
if(srcr.min.y < tr.min.y){
    p1.y += tr.min.x - srcr.min.x;
    tr.min.y = srcr.min.y;
}
```

```
if(srcr.max.x > tr.max.x)
    tr.max.x = srcr.max.x;
if(srcr.max.y > tr.max.y)
    tr.max.y = srcr.max.y;
```

⟨memdraw() *if src is fully visible can optimize* 224a⟩≡ (222e)

```
if(sl->clear){
    src = sl->screen->image;
    if(dl != nil){
        // r = rectsubpt(r, dl->delta)
        r.min.x -= dl->delta.x;
        r.min.y -= dl->delta.y;
        r.max.x -= dl->delta.x;
        r.max.y -= dl->delta.y;
    }
    goto Top;
}
```

⟨memdraw() *after src is an image, if dst is fully visible can optimize* 224b⟩≡ (221e)

```
if(dl->clear)
    goto Clearlayer;
```

_ldrawop()

⟨memdraw() *locals* 224c⟩+≡ (92e) ◁223b

```
struct Draw d;
```

⟨*struct* Draw 224d⟩≡ (458a)

```
struct Draw
{
    Memlayer    *dstlayer;

    Memimage *src;
    Point deltas;

    Memimage *mask;
    Point deltam;

    // enum<Drawop>
    int op;
};
```

⟨memdraw() *general case where dst is an obscured window* 224e⟩≡ (221e)

```
/*
 * dst is an obscured layer
 */
d.dstlayer = dl;
d.src = src;
d.mask = mask;
d.op = op;
d.deltas = subpt(p0, r.min);
d.deltam = subpt(p1, r.min);

_memlayerop(ldrawop, dst, r, r, &d);
```

⟨*function* ldrawop 225a⟩≡ (458a)

```
static
void
ldrawop(Memimage *dst, Rectangle screenr, Rectangle clipr, void *etc, bool insave)
{
    struct Draw *d = etc;
    Point p0, p1;
    Rectangle r;
    ⟨ldrawop() other locals 225b⟩

    ⟨ldrawop() return if no save in dst 308a⟩

    p0 = addpt(screenr.min, d->deltas);
    p1 = addpt(screenr.min, d->deltam);

    if(insave){
        r     = rectsubpt(screenr, d->dstlayer->delta);
        clipr = rectsubpt(clipr,   d->dstlayer->delta);
    }else
        r = screenr;

    /* now in logical coordinates */

    ⟨ldrawop() if r not in clipr 225c⟩
    memdraw(dst, r, d->src, p0, d->mask, p1, d->op); // will call memimagedraw
}
```

⟨ldrawop() *other locals* 225b⟩≡ (225a)

```
Rectangle oclipr, srcr, mr;
bool ok;
```

⟨ldrawop() *if r not in clipr* 225c⟩≡ (225a)

```
/* clipr may have narrowed what we should draw on, so clip if necessary */
if(!rectinrect(r, clipr)){
    ⟨ldrawop() change dst clipr 225d⟩
    ok = drawclip(dst, &r, d->src, &p0, d->mask, &p1, &srcr, &mr);
    ⟨ldrawop() restore dst clipr 225e⟩
    if(!ok)
        return;
}
```

⟨ldrawop() *change dst clipr* 225d⟩≡ (225c)

```
oclipr = dst->clipr;
dst->clipr = clipr;
```

⟨ldrawop() *restore dst clipr* 225e⟩≡ (225c)

```
dst->clipr = oclipr;
```

_memlayerop()

⟨*function* _memlayerop 225f⟩≡ (458c)

```
/*
 * Assumes incoming rectangle has already been clipped to i's logical r and clipr
 */
void
_memlayerop(
    void (*fn)(Memimage*, Rectangle, Rectangle, void*, int),
    Memimage *i,
    Rectangle screenr, /* clipped to window boundaries */
```

```
    Rectangle clipr, /* clipped also to clipping rectangles of hierarchy */
    void *etc)
{
    Memlayer *l;
    Rectangle r, scr;

    l = i->layer;
    if(!rectclip(&screenr, l->screenr))
        return;

    if(l->clear){
        fn(l->screen->image, screenr, clipr, etc, false);
        return;
    }

    r = screenr; // original value before rectclip below
    scr = l->screen->image->clipr;

    /*
     * Do the piece on the screen
     */
    if(rectclip(&screenr, scr))
        _layerop(fn, i, screenr, clipr, etc, l->screen->frontmost);

    if(rectinrect(r, scr))
        return;

    /*
     * Do the piece off the screen
    */
    if(!rectXrect(r, scr)){
        /* completely offscreen; easy */
        fn(l->save, r, clipr, etc, true);
        return;
    }
    // else

    if(r.min.y < scr.min.y){
        /* above screen */
        fn(l->save, Rect(r.min.x, r.min.y, r.max.x, scr.min.y), clipr, etc, true);
        r.min.y = scr.min.y;
    }
    if(r.max.y > scr.max.y){
        /* below screen */
        fn(l->save, Rect(r.min.x, scr.max.y, r.max.x, r.max.y), clipr, etc, true);
        r.max.y = scr.max.y;
    }
    if(r.min.x < scr.min.x){
        /* left of screen */
        fn(l->save, Rect(r.min.x, r.min.y, scr.min.x, r.max.y), clipr, etc, true);
        r.min.x = scr.min.x;
    }
    if(r.max.x > scr.max.x){
        /* right of screen */
        fn(l->save, Rect(scr.max.x, r.min.y, r.max.x, r.max.y), clipr, etc, true);
    }
}
```

_layerop()

⟨*function* _layerop 227a⟩≡ (458c)

```
static void
_layerop(
    void (*fn)(Memimage*, Rectangle, Rectangle, void*, int),
    Memimage *i,
    Rectangle r,
    Rectangle clipr,
    void *etc,
    Memimage *front)
{
    Rectangle fr;

    Top:
    if(front == i){
        /* no one is in front of this part of window; use the screen */
        fn(i->layer->screen->image, r, clipr, etc, false);
        return;
    }
    // else

    fr = front->layer->screenr;
    if(!rectXrect(r, fr)){
        /* r doesn't touch this window; continue on next rearmost */
        front = front->layer->rear;
        goto Top;
    }
    // else, r touches this window somewhere

    if(fr.max.y < r.max.y){
        // rectangle below front
        RECUR(r.min.x, fr.max.y, r.max.x, r.max.y);
        r.max.y = fr.max.y;
    }
    if(r.min.y < fr.min.y){
        // rectangle above front
        RECUR(r.min.x, r.min.y, r.max.x, fr.min.y);
        r.min.y = fr.min.y;
    }
    if(fr.max.x < r.max.x){
        // rectangle right of front
        RECUR(fr.max.x, r.min.y, r.max.x, r.max.y);
        r.max.x = fr.max.x;
    }
    if(r.min.x < fr.min.x){
        // rectangle left of front
        RECUR(r.min.x, r.min.y, fr.min.x, r.max.y);
        r.min.x = fr.min.x;
    }
    /* r is covered by front, so put in save area */
    (*fn)(i->layer->save, r, clipr, etc, true);
}
```

⟨*function* RECUR 227b⟩≡ (458c)

```
#define RECUR(a,b,c,d) _layerop(fn, i, Rect(a, b, c, d), clipr, etc, front->layer->rear);
```

12.7.2 Drawing lines on overlapping layers

⟨_memline() *sanity check no src layer* 227c⟩≡ (115a)

```
if(src->layer) /* can't draw line with layered source */
    return;
```

⟨_memline() *when have layers* 228a⟩≡ (115a)

```
⟨_memline() clip source 228c⟩

// Convert to screen coordinates
⟨_memline() convert p0, p1, clipr coordinates 228d⟩

⟨_memline() if dst is fully visible can optimize 228e⟩

⟨_memline() set r bounding box and clip it 228g⟩

⟨_memline() general case, call _memlayerop 229c⟩
```

⟨_memline() *other locals* 228b⟩≡ (115a) 228f▷

```
bool srcclipped = false;
Point d;
```

⟨_memline() *clip source* 228c⟩≡ (228a)

```
if(!srcclipped){
    d = subpt(sp, p0);
    if(!rectclip(&clipr, rectsubpt(src->clipr, d)))
        return;
    if(!(src->flags&Frepl) && !rectclip(&clipr, rectsubpt(src->r, d)))
        return;
    srcclipped = true;
}
```

⟨_memline() *convert p0, p1, clipr coordinates* 228d⟩≡ (228a)

```
/* dst is known to be a layer */
// p0 = addpt(p0, dl->delta)
p0.x += dl->delta.x;
p0.y += dl->delta.y;
// p1 = addpt(p1, dl->delta)
p1.x += dl->delta.x;
p1.y += dl->delta.y;
// clipr = rectaddpt(clipr, dl->elta)
clipr.min.x += dl->delta.x;
clipr.min.y += dl->delta.y;
clipr.max.x += dl->delta.x;
clipr.max.y += dl->delta.y;
```

⟨_memline() *if dst is fully visible can optimize* 228e⟩≡ (228a)

```
if(dl->clear){
    dst = dst->layer->screen->image;
    goto Top;
}
```

⟨_memline() *other locals* 228f⟩+≡ (115a) ◁228b 229a▷

```
Rectangle r;
```

⟨_memline() *set r bounding box and clip it* 228g⟩≡ (228a)

```
/* can't use sutherland-cohen clipping because lines are wide */
r = memlinebbox(p0, p1, end0, end1, radius);
/*
 * r is now a bounding box for the line;
 * use it as a clipping rectangle for subdivision
 */
if(!rectclip(&r, clipr))
    return;
```

⟨_memline() *other locals* 229a⟩+≡ (115a) ◁228f

```
struct Lline ll;
```

⟨*struct* Lline 229b⟩≡ (459b)

```
struct Lline
{
    Point   p0;
    Point   p1;
    Point   delta;
    int   end0;
    int   end1;
    int   radius;
    Point   sp;

    Memlayer  *dstlayer;
    Memimage *src;
    int   op;
};
```

⟨_memline() *general case, call* _memlayerop 229c⟩≡ (228a)

```
ll.p0 = p0;
ll.p1 = p1;
ll.end0 = end0;
ll.end1 = end1;
ll.sp = sp;
ll.dstlayer = dst->layer;
ll.src = src;
ll.radius = radius;
ll.delta = dl->delta;
ll.op = op;

_memlayerop(llineop, dst, r, r, &ll);
```

⟨*function* llineop 229d⟩≡ (459b)

```
static
void
llineop(Memimage *dst, Rectangle screenr, Rectangle clipr, void *etc, bool insave)
{
    struct Lline *ll = etc;
    Point p0, p1;

    USED(screenr.min.x);

    if(insave && ll->dstlayer->save == nil)
        return;
    if(!rectclip(&clipr, screenr))
        return;

    if(insave){
        p0 = subpt(ll->p0, ll->delta);
        p1 = subpt(ll->p1, ll->delta);
        clipr = rectsubpt(clipr, ll->delta);
    }else{
        p0 = ll->p0;
        p1 = ll->p1;
    }

    _memline(dst, p0, p1, ll->end0, ll->end1, ll->radius, ll->src, ll->sp, clipr, ll->op);
}
```

12.7.3 Loading an image on overlapping layers

memload()

⟨memload() *other locals* 230a⟩≡ (175c)
```
Rectangle lr; // saved logical r
int dx;
Memimage *tmp;
```

⟨memload() *if dst has a layer* 230b⟩≡ (175c)
```
/*
 * Convert to screen coordinates.
 */
lr = r;
// r = rectaddpt(r, dl->delta)
r.min.x += dl->delta.x;
r.min.y += dl->delta.y;
r.max.x += dl->delta.x;
r.max.y += dl->delta.y;

dx = dl->delta.x & (7 / dst->depth);

⟨memload() if dst is fully visible can optimize 230c⟩

/*
 * dst is an obscured layer or data is unaligned
 */
⟨memload() if data is aligned, load on save and expose 230d⟩
// else
tmp = allocmemimage(lr, dst->chan);
⟨memload() sanity check tmp 230e⟩
n = loadfn(tmp, lr, data, n);
memdraw(dst, lr, tmp, lr.min, nil, lr.min, S);
freememimage(tmp);
return n;
```

⟨memload() *if dst is fully visible can optimize* 230c⟩≡ (230b)
```
if(dl->clear && dx==0){
    dst = dl->screen->image;
    goto Top;
}
```

⟨memload() *if data is aligned, load on save and expose* 230d⟩≡ (230b)
```
if(dl->save && dx==0){
    n = loadfn(dl->save, lr, data, n);
    if(n > 0)
        memlexpose(dst, r);
    return n;
}
```

⟨memload() *sanity check tmp* 230e⟩≡ (230b)
```
if(tmp == nil)
    return ERROR_NEG1;
```

memunload()

⟨memunload() *other locals* 230f⟩≡ (178g)
```
Rectangle lr;
int dx;
Memimage *tmp;
```

⟨memunload() *if src has layer* 231⟩≡ (178g)

```
/*
 Convert to screen coordinates.
 */
lr = r;
// r = rectaddpt(r, dl->delta)
r.min.x += dl->delta.x;
r.min.y += dl->delta.y;
r.max.x += dl->delta.x;
r.max.y += dl->delta.y;

dx = dl->delta.x&(7/src->depth);
if(dl->clear && dx==0){
    src = dl->screen->image;
    goto Top;
}

/*
 * src is an obscured layer or data is unaligned
 */
if(dl->save && dx==0){
    if(dl->refreshfn != nil)
        return ERROR_NEG1; /* can't unload window if it's not Refbackup */
    if(n > 0)
        memlhide(src, r);
    n = unloadmemimage(dl->save, lr, data, n);
    return n;
}

tmp = allocmemimage(lr, src->chan);
if(tmp == nil)
    return ERROR_NEG1;
memdraw(tmp, lr, src, lr.min, nil, lr.min, S);
n = unloadmemimage(tmp, lr, data, n);
freememimage(tmp);
return n;
```

Chapter 13

Windowing System Support

13.1 Additional data structures

13.1.1 view

⟨*global* view 232a⟩≡ (401a)

```
// ref<Image>, a window
Image *view;
```

13.1.2 screen

⟨*global* screen 232b⟩≡ (415b)

```
Screen *screen;
```

13.2 Additional (user) initializations

13.2.1 geninitdraw()

⟨geninitdraw() *get window* 232c⟩≡ (55b)

```
snprint(buf, sizeof buf, "%s/winname", display->windir);
if(gengetwindow(display, buf, &view, &screen, ref) < 0)
    goto Error;
```

⟨*function* getwindow 232d⟩≡ (415b)

```
errorneg1
getwindow(Display *d, int ref)
{
    char winname[128];

    snprint(winname, sizeof winname, "%s/winname", d->windir);
    return gengetwindow(d, winname, &view, &screen, ref);
}
```

13.2.2 gengetwindow()

⟨*function* gengetwindow 232e⟩≡ (415b)

```
/*
 * Attach, or possibly reattach, to window.
 * If reattaching, maintain value of screen pointer.
 */
errorneg1
```

```
gengetwindow(Display *d, char *winname, Image **winp, Screen **scrp, int ref)
{
    int n;
    fdt fd;
    char buf[64+1]; // /dev/winname content
    Image *image;
    Rectangle r;

    fd = open(winname, OREAD);
    if(fd<0 || (n=read(fd, buf, sizeof buf-1))<=0){
        // no /dev/winname, image is then the full screen
        image = d->image;
        ⟨gengetwindow() sanity check image from display 233c⟩
        strcpy(buf, "noborder");
    }else{
        close(fd);
        buf[n] = '\0';
        ⟨gengetwindow() if already had a view, free it 234d⟩
        // get our window!
        image = namedimage(d, buf);
        ⟨gengetwindow() sanity check image from namedimage 234a⟩
    }

    d->screenimage = image;
    *scrp = allocscreen(image, d->white, false);
    ⟨gengetwindow() sanity check srcp 234b⟩

    r = image->r;
    if(strncmp(buf, "noborder", 8) != 0)
        r = insetrect(image->r, Borderwidth);
    *winp = _allocwindow(*winp, *scrp, r, ref, DWhite);
    ⟨gengetwindow() sanity check winp 234c⟩
    // PAD! commented for now cos buggy
    // get logical coordinates when run under rio
    //if(originwindow(*winp, Pt(0,0), r.min) <= 0)
    //  return ERROR_NEG1;

    d->screenimage = *winp;
    assert((*winp)->chan != 0);
    return OK_1;
}
```

⟨*constant* Borderwidth 233a⟩≡ (386c)

```
//coupling: must be equal to Selborder in rio
Borderwidth =   4,
```

⟨Display *other fields* 233b⟩+≡ (34e) ◁199g 235b▷

```
// option<Image>, same than view, the window (or desktop image)
Image   *screenimage;
```

⟨gengetwindow() *sanity check image from display* 233c⟩≡ (232e)

```
if(image == nil){
    fprint(2, "gengetwindow: %r\n");
    ⟨gengetwindow() return error 233d⟩
}
```

⟨gengetwindow() *return error* 233d⟩≡ (234 233c)

```
*winp = nil;
d->screenimage = nil;
return ERROR_NEG1;
```

⟨gengetwindow() *sanity check image from namedimage* 234a⟩≡ (232e)

```
if(image == nil){
    fprint(2, "namedimage %s failed: %r\n", buf);
    ⟨gengetwindow() return error 233d⟩
}
assert(image->chan != 0);
```

⟨gengetwindow() *sanity check srcp* 234b⟩≡ (232e)

```
if(*scrp == nil){
    freeimage(d->screenimage);
    ⟨gengetwindow() return error 233d⟩
}
```

⟨gengetwindow() *sanity check winp* 234c⟩≡ (232e)

```
if(*winp == nil){
    freescreen(*scrp);
    *scrp = nil;
    freeimage(image);
    ⟨gengetwindow() return error 233d⟩
}
```

⟨gengetwindow() *if already had a view, free it* 234d⟩≡ (232e)

```
if(*winp != nil){
    _freeimage1(*winp);
    freeimage((*scrp)->image);
    freescreen(*scrp);
    *scrp = nil;
}
```

13.3 /dev/winname and screenname

⟨drawopen() *switch qid cases* 234e⟩+≡ (65a) ◁69d 311f▷

```
case Qwinname:
    break;
```

⟨drawread() *if Qwinname* 234f⟩≡ (66d)

```
if(QID(c->qid) == Qwinname)
    return readstr(off, a, n, screenname);
```

⟨*global* screenname 234g⟩≡ (477)

```
char    screenname[40];
```

⟨makescreenimage() *drawaddname* 234h⟩≡ (50b)

```
if(!waserror()){
    snprint(screenname, sizeof screenname, "noborder.screen.%d", ++screennameid);
    drawaddname(nil, di, strlen(screenname), screenname);
    poperror();
}
```

⟨*global* screennameid 234i⟩≡ (477)

```
// gensym
static  int screennameid;
```

13.4 /dev/label

⟨geninitdraw() *write new label* 235a⟩≡ (55b)

```
/*
 * Write label; ignore errors (we might not be running under rio)
 */
if(label){
    snprint(buf, sizeof buf, "%s/label", display->windir);
    fd = open(buf, OREAD);
    if(fd >= 0){
        read(fd, display->oldlabel, (sizeof display->oldlabel)-1);
        close(fd);
        fd = create(buf, OWRITE, 0666);
        if(fd >= 0){
            write(fd, label, strlen(label));
            close(fd);
        }
    }
}
```

⟨Display *other fields* 235b⟩+≡ (34e) ◁233b 352b▷

```
char    oldlabel[64];
```

⟨_closedisplay() *restore oldlabel* 235c⟩≡ (59c)

```
if(disp->oldlabel[0]){
    snprint(buf, sizeof buf, "%s/label", disp->windir);
    fd = open(buf, OWRITE);
    if(fd >= 0){
        write(fd, disp->oldlabel, strlen(disp->oldlabel));
        close(fd);
    }
}
```

13.5 Additional mouse functions

13.5.1 The resize event

⟨Mousectl *channel fields* 235d⟩+≡ (183d) ◁184b

```
Channel *resizec; /* chan(int)[2] */
/* buffered in case client is waiting for a mouse action before handling resize */
```

⟨initmouse() *set channels* 235e⟩+≡ (184c) ◁184f

```
mc->resizec = chancreate(sizeof(int), 2);
```

⟨_ioproc() *other locals (mouse.c)* 235f⟩≡ (185a)

```
int one = 1;
```

⟨_ioproc() *switch buf, resize case, fallthrough m case (mouse.c)* 235g⟩≡ (185a)

```
case 'r':
    send(mc->resizec, &one);
    /* fall through */
```

⟨initscreenimage() *other initializations* 235h⟩≡ (49g)

```
mouseresize();
```

13.6 newwindow()

⟨*function* newwindow 236a⟩≡ (415b)

```
/* Connect us to new window, if possible */
errorneg1
newwindow(char *str)
{
    fdt fd;
    char *wsys;
    char buf[256];

    wsys = getenv("wsys");
    ⟨newwindow() sanity check wsys 236b⟩
    fd = open(wsys, ORDWR);
    free(wsys);
    ⟨newwindow() sanity check fd 236c⟩
    rfork(RFNAMEG);

    if(str)
        snprint(buf, sizeof buf, "new %s", str);
    else
        strcpy(buf, "new");

    return mount(fd, -1, "/dev", MBEFORE, buf);
}
```

⟨newwindow() *sanity check wsys* 236b⟩≡ (236a)

```
if(wsys == nil)
    return ERROR_NEG1;
```

⟨newwindow() *sanity check fd* 236c⟩≡ (236a)

```
if(fd < 0)
    return ERROR_NEG1;
```

Chapter 14

Non 32-Bits Images TODO

⟨ImageType *cases* 237a⟩≡ (41b) 237c ▷

```
  GREY2   = CHAN1(CGrey, 2),
  GREY4   = CHAN1(CGrey, 4),
  GREY8   = CHAN1(CGrey, 8),

  RGB15   = CHAN4(CIgnore, 1, CRed, 5, CGreen, 5, CBlue, 5),
  RGB24   = CHAN3(CRed, 8, CGreen, 8, CBlue, 8),
  BGR24   = CHAN3(CBlue, 8, CGreen, 8, CRed, 8),
```

⟨ImageChan *cases* 237b⟩≡ (40c)

```
  CIgnore,
```

⟨ImageType *cases* 237c⟩+≡ (41b) ◁ 237a

```
  ABGR32  = CHAN4(CAlpha, 8, CBlue, 8, CGreen, 8, CRed, 8),

  XRGB32  = CHAN4(CIgnore, 8, CRed, 8, CGreen, 8, CBlue, 8),
  XBGR32  = CHAN4(CIgnore, 8, CBlue, 8, CGreen, 8, CRed, 8),
```

14.1 1-Bit images

⟨memoptdraw() *if 1 bit mask, src, and dest* 237d⟩≡ (100c)

```
  /*
   * If we have a 1-bit mask, 1-bit source, and 1-bit destination, and
   * they're all bit aligned, we can just use bit operators.  This happens
   * when we're manipulating boolean masks, e.g., in the arc code.
   */
  if((par->state&(Simplemask|Simplesrc|Replmask|Replsrc))==0
  && dst->chan==GREY1
  && src->chan==GREY1
  && par->mask->chan==GREY1
  && (par->r.min.x&7)==(par->sr.min.x&7)
  && (par->r.min.x&7)==(par->mr.min.x&7)){
      uchar *sp, *dp, *mp;
      uchar lm, rm;
      long swid, dwid, mwid;
      int i, x, dir;

      sp = byteaddr(src, par->sr.min);
      dp = byteaddr(dst, par->r.min);
      mp = byteaddr(par->mask, par->mr.min);
      swid = src->width*sizeof(ulong);
      dwid = dst->width*sizeof(ulong);
      mwid = par->mask->width*sizeof(ulong);
```

```
if(src->data == dst->data
&& byteaddr(dst, par->r.min) > byteaddr(src, par->sr.min)){
    dir = -1;
}else
    dir = 1;

lm = 0xFF>>(par->r.min.x&7);
rm = 0xFF<<(8-(par->r.max.x&7));
dx -= (8-(par->r.min.x&7)) + (par->r.max.x&7);

if(dx < 0){ /* one byte wide */
    lm &= rm;
    if(dir == -1){
        dp += dwid*(dy-1);
        sp += swid*(dy-1);
        mp += mwid*(dy-1);
        dwid = -dwid;
        swid = -swid;
        mwid = -mwid;
    }
    for(y=0; y<dy; y++){
        *dp ^= (*dp ^ *sp) & *mp & lm;
        dp += dwid;
        sp += swid;
        mp += mwid;
    }
    return 1;
}

dx /= 8;
if(dir == 1){
    i = (lm!=0)+dx+(rm!=0);
    mwid -= i;
    swid -= i;
    dwid -= i;
    for(y=0; y<dy; y++, dp+=dwid, sp+=swid, mp+=mwid){
        if(lm){
            *dp ^= (*dp ^ *sp++) & *mp++ & lm;
            dp++;
        }
        for(x=0; x<dx; x++){
            *dp ^= (*dp ^ *sp++) & *mp++;
            dp++;
        }
        if(rm){
            *dp ^= (*dp ^ *sp++) & *mp++ & rm;
            dp++;
        }
    }
    return 1;
}else{
/* dir == -1 */
    i = (lm!=0)+dx+(rm!=0);
    dp += dwid*(dy-1)+i-1;
    sp += swid*(dy-1)+i-1;
    mp += mwid*(dy-1)+i-1;
    dwid = -dwid+i;
    swid = -swid+i;
    mwid = -mwid+i;
```

```
        for(y=0; y<dy; y++, dp+=dwid, sp+=swid, mp+=mwid){
            if(rm){
                *dp ^= (*dp ^ *sp--) & *mp-- & rm;
                dp--;
            }
            for(x=0; x<dx; x++){
                *dp ^= (*dp ^ *sp--) & *mp--;
                dp--;
            }
            if(lm){
                *dp ^= (*dp ^ *sp--) & *mp-- & lm;
                dp--;
            }
        }
    }
    return 1;
}
```

14.2 Small-depth images

⟨pixelbits() *other locals* 239a⟩≡ (99c)

```
int off, npack;
```

⟨pixelbits() *switch bpp cases* 239b⟩≡ (99c) 245c ▷

```
case 1:
case 2:
case 4:
    npack = 8/bpp;
    off = pt.x % npack;
    val = p[0] >> bpp*(npack-1-off);
    val &= (1<<bpp)-1;
    break;
```

⟨imgtorgba() *duplicate bits if nb less than 8* 239c⟩≡ (42)

```
// duplicate bits
while(nb < 8){
    v |= v<<nb;
    nb *= 2;
}
v >>= (nb-8);
```

⟨byteaddr() *if depth less than 8* 239d⟩≡ (39b)

```
if(i->depth < 8){
    /*
     * We need to always round down,
     * but C rounds toward zero.
     */
    int np;
    np = 8/i->depth;
    if(p.x < 0)
        return a+(p.x-np+1)/np;
    else
        return a+p.x/np;
}
```

⟨allocimagemix() *if depth less than 8* 240a⟩≡ (274c)

```
if(d->screenimage->depth <= 8){ /* create a 2x2 texture */
    t = allocimage(d, Rect(0,0,1,1), d->screenimage->chan, false, color1);
    if(t == nil)
        return nil;

    b = allocimage(d, Rect(0,0,2,2), d->screenimage->chan, true, color3);
    if(b == nil){
        freeimage(t);
        return nil;
    }

    draw(b, Rect(0,0,1,1), t, nil, ZP);
    freeimage(t);
    return b;
}
```

⟨memoptdraw() *locals for memset case* 240b⟩≡ (101a)

```
int ppb, np, nb;
uchar lm, rm;
int d;
```

⟨memoptdraw() *when condition for memset, switch depth of dst cases* 240c⟩≡ (101a) 241a ▷

```
case 1:
case 2:
case 4:
    for(d=dst->depth; d<8; d*=2)
        v |= (v<<d);
    ppb = 8/dst->depth; /* pixels per byte */
    m = ppb-1;
    /* left edge */
    np = par->r.min.x&m;  /* no. pixels unused on left side of word */
    dx -= (ppb-np);
    nb = 8 - np * dst->depth;  /* no. bits used on right side of word */
    lm = (1<<nb)-1;
    DBG1("np %d x %d nb %d lm %ux ppb %d m %ux\n", np, par->r.min.x, nb, lm, ppb, m);

    /* right edge */
    np = par->r.max.x&m; /* no. pixels used on left side of word */
    dx -= np;
    nb = 8 - np * dst->depth;  /* no. bits unused on right side of word */
    rm = ~((1<<nb)-1);
    DBG1("np %d x %d nb %d rm %ux ppb %d m %ux\n", np, par->r.max.x, nb, rm, ppb, m);

    DBG1("dx %d Dx %d\n", dx, Dx(par->r));
    /* lm, rm are masks that are 1 where we should touch the bits */
    if(dx < 0){ /* just one byte */
        lm &= rm;
        for(y=0; y<dy; y++, dp+=dwid)
            *dp ^= (v ^ *dp) & lm;
    }else if(dx == 0){ /* no full bytes */
        if(lm)
            dwid--;

        for(y=0; y<dy; y++, dp+=dwid){
            if(lm){
              DBG1("dp %p v %lux lm %ux (v ^ *dp) & lm %lux\n", dp, v, lm, (v^*dp)&lm);
                *dp ^= (v ^ *dp) & lm;
                dp++;
            }
```

```
            *dp ^= (v ^ *dp) & rm;
        }
    }else{  /* full bytes in middle */
        dx /= ppb;
        if(lm)
            dwid--;
        dwid -= dx;

        for(y=0; y<dy; y++, dp+=dwid){
            if(lm){
                *dp ^= (v ^ *dp) & lm;
                dp++;
            }
            memset(dp, v, dx);
            dp += dx;
            *dp ^= (v ^ *dp) & rm;
        }
    }
    return true;
```

⟨memoptdraw() *when condition for memset, switch depth of dst cases* 241a⟩+≡ (101a) ◁240c 244c▷

```
 default:
     assert(0 /* bad dest depth in memoptdraw */);
```

⟨setcolor() *switch depth cases* 241b⟩≡ (254c) 245b▷

```
 case 1:
 case 2:
 case 4:
     x = 0x0F;
     break;
```

⟨readfn() *if depth less than 8* 241c⟩≡ (108a)

```
 if(img->depth < 8)
     return readnbit;
```

⟨*function* readnbit 241d⟩≡ (429b)

```
 static Buffer
 readnbit(Param *p, uchar *buf, int y)
 {
     Buffer b;
     Memimage *img;
     uchar *repl, *r, *w, *ow, bits;
     int i, n, sh, depth, x, dx, npack, nbits;

     b.rgba = (ulong*)buf;
     b.grey = w = buf;
     b.red = b.blu = b.grn = w;
     b.alpha = &ones;
     b.delta = 1;

     dx = p->dx;
     img = p->img;
     depth = img->depth;
     repl = &replbit[depth][0];
     npack = 8/depth;
     sh = 8-depth;

     /* copy from p->r.min.x until end of repl rectangle */
     x = p->r.min.x;
     n = dx;
```

```
if(n > p->img->r.max.x - x)
    n = p->img->r.max.x - x;

r = p->bytermin + y*p->bwidth;
DBG1("readnbit dx %d %p=%p+%d*%d, *r=%d fetch %d ", dx, r, p->bytermin, y, p->bwidth, *r, n);
bits = *r++;
nbits = 8;
if(i=x&(npack-1)){
    DBG1("throwaway %d...", i);
    bits <<= depth*i;
    nbits -= depth*i;
}
for(i=0; i<n; i++){
    if(nbits == 0){
        DBG1("(%.2ux)...", *r);
        bits = *r++;
        nbits = 8;
    }
    *w++ = repl[bits>>sh];
    DBG1("bit %x...", repl[bits>>sh]);
    bits <<= depth;
    nbits -= depth;
}
dx -= n;
if(dx == 0)
    return b;

assert(x+i == p->img->r.max.x);

/* copy from beginning of repl rectangle until where we were before. */
x = p->img->r.min.x;
n = dx;
if(n > p->r.min.x - x)
    n = p->r.min.x - x;

r = p->bytey0s + y*p->bwidth;
DBG1("x=%d r=%p...", x, r);
bits = *r++;
nbits = 8;
if(i=x&(npack-1)){
    bits <<= depth*i;
    nbits -= depth*i;
}
DBG1("nbits=%d...", nbits);
for(i=0; i<n; i++){
    if(nbits == 0){
        bits = *r++;
        nbits = 8;
    }
    *w++ = repl[bits>>sh];
    DBG1("bit %x...", repl[bits>>sh]);
    bits <<= depth;
    nbits -= depth;
    DBG1("bits %x nbits %d...", bits, nbits);
}
dx -= n;
if(dx == 0)
    return b;

assert(dx > 0);
```

```
    /* now we have exactly one full scan line: just replicate the buffer itself until we are done */
    ow = buf;
    while(dx--)
        *w++ = *ow++;

    return b;
}
```

⟨writefn() *if depth less than 8* 243a⟩≡ (109b)

```
if(img->depth < 8)
    return writenbit;
```

⟨*function* writenbit 243b⟩≡ (429b)

```
static void
writenbit(Param *p, uchar *w, Buffer src)
{
    uchar *r;
    ulong bits;
    int i, sh, depth, npack, nbits, x, ex;

    assert(src.grey != nil && src.delta == 1);

    x = p->r.min.x;
    ex = x+p->dx;
    depth = p->img->depth;
    npack = 8/depth;

    i=x&(npack-1);
    bits = i ? (*w >> (8-depth*i)) : 0;
    nbits = depth*i;
    sh = 8-depth;
    r = src.grey;

    for(; x<ex; x++){
        bits <<= depth;
        DBG1(" %x", *r);
        bits |= (*r++ >> sh);
        nbits += depth;
        if(nbits == 8){
            *w++ = bits;
            nbits = 0;
        }
    }

    if(nbits){
        sh = 8-nbits;
        bits <<= sh;
        bits |= *w & ((1<<sh)-1);
        *w = bits;
    }
    DBG1("\n");
    return;
}
```

⟨loadmemimage() *other locals* 243c⟩≡ (175d)

```
int mx, m, mr;
```

⟨loadmemimage() *set mx, lpart, rpart, m for small depth images* 243d⟩≡ (175d)

```
mx = 7 / i->depth;
lpart = (r.min.x & mx) * i->depth;
rpart = (r.max.x & mx) * i->depth;
m = 0xFF >> lpart;
```

⟨loadmemimage() *if 1 byte per line* 244a⟩≡ (175d)

```
/* may need to do bit insertion on edges */
if(l == 1){ /* all in one byte */
    if(rpart)
        m ^= 0xFF >> rpart;
    for(y = r.min.y; y < r.max.y; y++){
        *q ^= (*data ^ *q) & m;
        q += i->width*sizeof(ulong);
        data++;
    }
    return ndata;
}
```

⟨loadmemimage() *when small depth images* 244b⟩≡ (175d)

```
mr = 0xFF ^ (0xFF >> rpart);
if(lpart!=0 && rpart==0){
    for(y=r.min.y; y<r.max.y; y++){
        *q ^= (*data^*q) & m;
        if(l > 1)
            memmove(q+1, data+1, l-1);
        q += i->width*sizeof(ulong);
        data += l;
    }
    return ndata;
}
if(lpart==0 && rpart!=0){
    for(y=r.min.y; y<r.max.y; y++){
        if(l > 1)
            memmove(q, data, l-1);
        q[l-1] ^= (data[l-1]^q[l-1]) & mr;
        q += i->width*sizeof(ulong);
        data += l;
    }
    return ndata;
}
for(y=r.min.y; y<r.max.y; y++){
    *q ^= (*data^*q) & m;
    if(l > 2)
        memmove(q+1, data+1, l-2);
    q[l-1] ^= (data[l-1]^q[l-1]) & mr;
    q += i->width*sizeof(ulong);
    data += l;
}
return ndata;
```

14.3 8-bits images

⟨memoptdraw() *when condition for memset, switch depth of dst cases* 244c⟩+≡ (101a) ◁241a 245d▷

```
case 8:
    for(y=0; y < dy; y++, dp += dwid)
        memset(dp, v, dx);
    return true;
```

⟨chardraw() *switch depth cases* 244d⟩≡ (313) 245f▷

```
case 8:
    DBG1("8loop...");
    wc = wp;
    for(x=bx; x>ex; x--, wc++){
```

```
        i = x&7;
        if(i == 8-1)
            bits = *q++;
        DBG1("bits %lux sh %d...", bits, i);
        if((bits>>i)&1)
            *wc = v;
    }
    break;
```

⟨getcolor() *switch depth cases* 245a⟩≡ (253b)

```
  case 8:
      x = 0xFF;
      break;
```

⟨setcolor() *switch depth cases* 245b⟩+≡ (254c) ◁241b

```
  case 8:
      x = 0xFF;
      break;
```

14.4 16-bits images

⟨pixelbits() *switch bpp cases* 245c⟩+≡ (99c) ◁239b 246a▷

```
  case 16:
      val = p[0]|(p[1]<<8);
      break;
```

⟨memoptdraw() *when condition for memset, switch depth of dst cases* 245d⟩+≡ (101a) ◁244c 246b▷

```
  case 16:
      p[0] = v;  /* make little endian */
      p[1] = v>>8;
      v = *(ushort*)p;
      DBG("dp=%p; dx=%d; for(y=0; y<%d; y++, dp+=%d)\nmemsets(dp, v, dx);\n",          dp, dx, dy, dwid);
      for(y=0; y<dy; y++, dp+=dwid)
          memsets(dp, v, dx);
      return 1;
```

⟨*function* memsets 245e⟩≡ (432)

```
  static void
  memsets(void *vp, ushort val, int n)
  {
      ushort *p, *ep;

      p = vp;
      ep = p+n;
      while(p<ep)
          *p++ = val;
  }
```

⟨chardraw() *switch depth cases* 245f⟩+≡ (313) ◁244d 246f▷

```
  case 16:
      ws = (ushort*)wp;
      v = *(ushort*)sp;
      for(x=bx; x>ex; x--, ws++){
          i = x&7;
          if(i == 8-1)
              bits = *q++;
          DBG1("bits %lux sh %d...", bits, i);
          if((bits>>i)&1)
              *ws = v;
      }
      break;
```

14.5 24-bits images

⟨pixelbits() *switch bpp cases* 246a⟩+≡ (99c) ◁245c

```
case 24:
    val = p[0]|(p[1]<<8)|(p[2]<<16);
    break;
```

⟨memoptdraw() *when condition for memset, switch depth of dst cases* 246b⟩+≡ (101a) ◁245d

```
case 24:
    for(y=0; y<dy; y++, dp+=dwid)
        memset24(dp, v, dx);
    return 1;
```

⟨*function* memset24 246c⟩≡ (432)

```
void
memset24(void *vp, ulong val, int n)
{
    uchar *p, *ep;
    uchar a,b,c;

    p = vp;
    ep = p+3*n;

    a = val;
    b = val>>8;
    c = val>>16;
    while(p<ep){
        *p++ = a;
        *p++ = b;
        *p++ = c;
    }
}
```

⟨memfillcolor() *other colors* 246d⟩+≡ (43b) ◁44c

```
int y;
```

⟨memfillcolor() *switch depth cases* 246e⟩≡ (43b)

```
case 24: /* 24-bit images suck */
    for(y=i->r.min.y; y<i->r.max.y; y++)
        memset24(byteaddr(i, Pt(i->r.min.x, y)), bits, Dx(i->r));
    break;
```

⟨chardraw() *switch depth cases* 246f⟩+≡ (313) ◁245f

```
case 24:
    wc = wp;
    for(x=bx; x>ex; x--, wc+=3){
        i = x&7;
        if(i == 8-1)
            bits = *q++;
        DBG1("bits %lux sh %d...", bits, i);
        if((bits>>i)&1){
            wc[0] = sp[0];
            wc[1] = sp[1];
            wc[2] = sp[2];
        }
    }
    break;
```

14.6 Gray-scale images

⟨imgtorgba() *switch chan type cases* 247a⟩≡ (42) 248e ▷

```
case CGrey:
    r = g = b = v;
    break;
```

⟨rgbatoimg() *switch chan type cases* 247b⟩≡ (43a) 248g ▷

```
case CGrey:
    m = RGB2K(r,g,b);
    v |= (m>>(8-nb))<<d;
    break;
```

⟨*function* RGB2K 247c⟩≡ (393b)

```
/* perfect approximation to NTSC = .299r+.587g+.114b when 0 <= r,g,b < 256 */
#define RGB2K(r,g,b) ((156763*(r)+307758*(g)+59769*(b))>>19)
```

⟨readbyte() *in simple case, if isgrey* 247d⟩≡ (108c)

```
if(isgrey){
    b.grey = r + img->shift[CGrey]/8;
    b.red = b.grn = b.blu = b.grey;
```

⟨writebyte() *if isgrey* 247e⟩≡ (109c)

```
if(isgrey){
    u |= ((*grey >> (8-img->nbits[CGrey])) & img->mask[CGrey]) << img->shift[CGrey];
    grey += delta;
}
```

⟨alphacalc11() *if isgrey* 247f⟩≡ (111e)

```
if(grey){
    *bdst.grey = MUL(ma, *bsrc.grey, s) + MUL(fd, *bdst.grey, t);
    bsrc.grey += bsrc.delta;
    bdst.grey += bdst.delta;
}
```

14.7 Images with a color map

14.7.1 RGB

⟨*struct* RGB 247g⟩≡ (386d)

```
struct RGB
{
    ulong   red;
    ulong   green;
    ulong   blue;
};
```

14.7.2 Memcmap

⟨MemImage *other fields* 247h⟩+≡ (37e) ◁ 53d

```
Memcmap *cmap;
```

⟨*struct* Memcmap 247i⟩≡ (393b)

```
struct Memcmap
{
    // map<colorcmap8, (reg8, green8, blue8)>
    byte cmap2rgb[3*256];
    // map<(red4,green4,blue4), colorcmap8)
    byte rgb2cmap[16*16*16];
};
```

⟨*global* memdefcmap 248a⟩≡ (428a)

```
Memcmap *memdefcmap = &def;
```

⟨memimageinit() *initializations* 248b⟩≡ (48a) 267a ▷

```
_memmkcmap();
```

⟨*function* _memmkcmap 248c⟩≡ (428a)

```
void _memmkcmap(void){}
```

14.7.3 Conversions

⟨imgtorgba() *other locals* 248d⟩≡ (42)

```
byte *p;
int ov;
```

⟨imgtorgba() *switch chan type cases* 248e⟩+≡ (42) ◁ 247a

```
case CMap:
    p = img->cmap->cmap2rgb + 3*ov;
    r = *p++;
    g = *p++;
    b = *p;
    break;
```

⟨rgbatoimg() *other locals* 248f⟩≡ (43a)

```
byte *p;
byte m;
```

⟨rgbatoimg() *switch chan type cases* 248g⟩+≡ (43a) ◁ 247b

```
case CMap:
    p = img->cmap->rgb2cmap;
    m = p[(r>>4)*256+(g>>4)*16+(b>>4)];
    v |= (m>>(8-nb))<<d;
    break;
```

⟨*function* rgb2cmap 248h⟩≡ (412c)

```
int
rgb2cmap(int cr, int cg, int cb)
{
    int i, r, g, b, sq;
    ulong rgb;
    int best, bestsq;

    best = 0;
    bestsq = 0x7FFFFFFF;
    // find best candidate
    for(i=0; i<256; i++){
        rgb = cmap2rgb(i);
        r = (rgb>>16) & 0xFF;
        g = (rgb>>8) & 0xFF;
        b = (rgb>>0) & 0xFF;
        sq = (r-cr)*(r-cr)+(g-cg)*(g-cg)+(b-cb)*(b-cb);
        if(sq < bestsq){
            bestsq = sq;
            best = i;
        }
    }
    return best;
}
```

⟨*function* cmap2rgb 249a⟩≡ (412c)

```
int
cmap2rgb(int c)
{
    int j, num, den, r, g, b, v, rgb;

    r = c>>6;
    v = (c>>4)&3;
    j = (c-v+r)&15;
    g = j>>2;
    b = j&3;
    den=r;
    if(g>den)
        den=g;
    if(b>den)
        den=b;
    if(den==0) {
        v *= 17;
        rgb = (v<<16)|(v<<8)|v;
    }
    else{
        num=17*(4*den+v);
        rgb = ((r*num/den)<<16)|((g*num/den)<<8)|(b*num/den);
    }
    return rgb;
}
```

⟨*function* cmap2rgba 249b⟩≡ (412c)

```
int
cmap2rgba(int c)
{
    return (cmap2rgb(c)<<8)|0xFF;
}
```

⟨readfn() *if cmap* 249c⟩≡ (108a)

```
if(img->nbits[CMap] == 8)
    return readcmap;
```

⟨writefn() *if cmap* 249d⟩≡ (109b)

```
if(img->chan == CMAP8)
    return writecmap;
```

⟨*function* readcmap 249e⟩≡ (429b)

```
static Buffer
readcmap(Param *p, uchar *buf, int y)
{
    Buffer b;
    int a, convgrey, copyalpha, dx, i, m;
    uchar *q, *cmap, *begin, *end, *r, *w;

    begin = p->bytey0s + y*p->bwidth;
    r = p->bytermin + y*p->bwidth;
    end = p->bytey0e + y*p->bwidth;

    cmap = p->img->cmap->cmap2rgb;
    convgrey = p->convgrey;
    copyalpha = (p->img->flags&Falpha) ? 1 : 0;

    w = buf;
    dx = p->dx;
```

```
    if(copyalpha){
        b.alpha = buf++;
        a = p->img->shift[CAlpha]/8;
        m = p->img->shift[CMap]/8;
        for(i=0; i<dx; i++){
            *w++ = r[a];
            q = cmap+r[m]*3;
            r += 2;
            if(r == end)
                r = begin;
            if(convgrey){
                *w++ = RGB2K(q[0], q[1], q[2]);
            }else{
                *w++ = q[2]; /* blue */
                *w++ = q[1]; /* green */
                *w++ = q[0]; /* red */
            }
        }
    }else{
        b.alpha = &ones;
        for(i=0; i<dx; i++){
            q = cmap+*r++*3;
            if(r == end)
                r = begin;
            if(convgrey){
                *w++ = RGB2K(q[0], q[1], q[2]);
            }else{
                *w++ = q[2]; /* blue */
                *w++ = q[1]; /* green */
                *w++ = q[0]; /* red */
            }
        }
    }

    b.rgba = (ulong*)(buf-copyalpha);

    if(convgrey){
        b.grey = buf;
        b.red = b.blu = b.grn = buf;
        b.delta = 1+copyalpha;
    }else{
        b.blu = buf;
        b.grn = buf+1;
        b.red = buf+2;
        b.grey = nil;
        b.delta = 3+copyalpha;
    }
    return b;
}
```

⟨*function* writecmap 250⟩≡ (429b)

```
static void
writecmap(Param *p, uchar *w, Buffer src)
{
    uchar *cmap, *red, *grn, *blu;
    int i, dx, delta;

    cmap = p->img->cmap->rgb2cmap;

    delta = src.delta;
```

```
    red = src.red;
    grn = src.grn;
    blu = src.blu;

    dx = p->dx;
    for(i=0; i<dx; i++, red+=delta, grn+=delta, blu+=delta)
        *w++ = cmap[(*red>>4)*256+(*grn>>4)*16+(*blu>>4)];
}
```

14.7.4 API

⟨*function* readcolmap 251a⟩≡ (412c)

```
void
readcolmap(Display *d, RGB *colmap)
{
    int i;
    char *p, *q;
    Biobuf *b;
    char buf[128];

    sprint(buf, "/dev/draw/%d/colormap", d->dirno);
    b = Bopen(buf, OREAD);
    if(b == 0)
        drawerror(d, "rdcolmap: can't open colormap device");

    for(;;) {
        p = Brdline(b, '\n');
        if(p == 0)
            break;
        i = strtoul(p, &q, 0);
        if(i < 0 || i > 255) {
            fprint(2, "rdcolmap: bad index\n");
            exits("bad");
        }
        p = q;
        colmap[255-i].red = getval(&p);
        colmap[255-i].green = getval(&p);
        colmap[255-i].blue = getval(&p);
    }
    Bterm(b);
}
```

⟨*function* getval 251b⟩≡ (412c)

```
static ulong
getval(char **p)
{
    ulong v;
    char *q;

    v = strtoul(*p, &q, 0);
    v |= v<<8;
    v |= v<<16;
    *p = q;
    return v;
}
```

⟨*function* writecolmap 251c⟩≡ (412c)

```
/*
 * This code (and the devdraw interface) will have to change
```

```
 * if we ever get bitmaps with ldepth > 3, because the
 * colormap will have to be written in chunks
 */
void
writecolmap(Display *d, RGB *m)
{
    int i, n, fd;
    char buf[64], *t;
    ulong r, g, b;

    sprint(buf, "/dev/draw/%d/colormap", d->dirno);
    fd = open(buf, OWRITE);
    if(fd < 0)
        drawerror(d, "writecolmap: open colormap failed");
    t = malloc(8192);
    if (t == nil)
        drawerror(d, "writecolmap: no memory");
    n = 0;
    for(i = 0; i < 256; i++) {
        r = m[i].red>>24;
        g = m[i].green>>24;
        b = m[i].blue>>24;
        n += sprint(t+n, "%d %lud %lud %lud\n", 255-i, r, g, b);
    }
    i = write(fd, t, n);
    free(t);
    close(fd);
    if(i != n)
        drawerror(d, "writecolmap: bad write");
}
```

14.7.5 /dev/draw/x/colormap

⟨drawread() *other locals* 252a⟩+≡ (66d) ◁68h 307d▷

```
uchar *p;
int index, m;
ulong red, green, blue;
ulong offset = off;
```

⟨drawread() *switch qid cases* 252b⟩+≡ (66d) ◁178e 307e▷

```
case Qcolormap:
    drawactive(true);  /* to restore map from backup */
    p = malloc(4*12*256+1);
    if(p == nil)
        error(Enomem);
    m = 0;
    for(index = 0; index < 256; index++){
        arch_getcolor(index, &red, &green, &blue);
        m += snprint((char*)p+m, 4*12*256+1 - m,
            "%11d %11lud %11lud %11lud\n", index,
            red>>24, green>>24, blue>>24);
    }
    n = readstr(offset, a, n, (char*)p);
    free(p);
    break;
```

⟨drawwrite() *other locals* 252c⟩≡ (67a)

```
char buf[128], *fields[4], *q;
int i, m, red, green, blue, x;
```

⟨drawwrite() *switch qid cases* 253a⟩+≡ (67a) ◁69e

```
case Qcolormap:
    drawactive(true);  /* to restore map from backup */
    m = n;
    n = 0;
    while(m > 0){
        x = m;
        if(x > sizeof(buf)-1)
            x = sizeof(buf)-1;
        q = memccpy(buf, a, '\n', x);
        if(q == 0)
            break;
        i = q-buf;
        n += i;
        a = (char*)a + i;
        m -= i;
        *q = 0;
        if(tokenize(buf, fields, nelem(fields)) != 4)
            error(Ebadarg);
        i = strtoul(fields[0], 0, 0);
        red = strtoul(fields[1], 0, 0);
        green = strtoul(fields[2], 0, 0);
        blue = strtoul(fields[3], &q, 0);
        if(fields[3] == q)
            error(Ebadarg);
        if(red>255 || green>255 || blue>255 || i<0 || i>255)
            error(Ebadarg);
        red |= red<<8;
        red |= red<<16;
        green |= green<<8;
        green |= green<<16;
        blue |= blue<<8;
        blue |= blue<<16;
        arch_setcolor(i, red, green, blue);
    }
    break;
```

14.7.6 Hardware support hook (x86)

⟨*function* getcolor*(x86)* 253b⟩≡ (492)

```
void
arch_getcolor(ulong p, ulong* pr, ulong* pg, ulong* pb)
{
    VGAscr *scr;
    ulong x;

    scr = &vgascreen;
    if(gscreen == nil)
        return;

    switch(gscreen->depth){
    default:
        x = 0x0F;
        break;
    ⟨getcolor() switch depth cases 245a⟩
    }
    p &= x;

    lock(&cursor);
```

```
    *pr = scr->colormap[p][0];
    *pg = scr->colormap[p][1];
    *pb = scr->colormap[p][2];
    unlock(&cursor);
}
```

drawcmap() and setcolor()

⟨screensize() *initializations part2* 254a⟩+≡ (46a) ◁188d 368b▷

```
drawcmap();
```

⟨*function* drawcmap 254b⟩≡ (480b)

```
/*
 * On 8 bit displays, load the default color map
 */
void
drawcmap(void)
{
    int r, g, b;
    int cr, cg, cb, v;
    int num, den;
    int i, j;

    drawactive(true);  /* to restore map from backup */
    for(r=0,i=0; r!=4; r++)
        for(v=0; v!=4; v++,i+=16){
        for(g=0,j=v-r; g!=4; g++)
            for(b=0;b!=4;b++,j++){
            den = r;
            if(g > den)
                den = g;
            if(b > den)
                den = b;
            if(den == 0)    /* divide check -- pick grey shades */
                cr = cg = cb = v*17;
            else{
                num = 17*(4*den+v);
                cr = r*num/den;
                cg = g*num/den;
                cb = b*num/den;
            }
            arch_setcolor(i+(j&15),
                cr*0x01010101, cg*0x01010101, cb*0x01010101);
            }
    }
}
```

⟨*function* setcolor*(x86)* 254c⟩≡ (492)

```
/*
 * On some video cards (e.g., Mach64), the palette is used as the
 * DAC registers for >8-bit modes.  We don't want to set them when the user
 * is trying to set a colormap and the card is in one of these modes.
 */
int
arch_setcolor(ulong p, ulong r, ulong g, ulong b)
{
    int x;

    if(gscreen == nil)
```

```
        return 0;

    switch(gscreen->depth){
    ⟨setcolor() switch depth cases 241b⟩
    default:
        return 0;
    }
    p &= x;

    return setpalette(p, r, g, b);
}
```

setpalette()

⟨screensize() *vga settings* 255a⟩≡ (46a) 488c ▷

```
scr->palettedepth = 6;  /* default */
```

⟨*function* setpalette*(x86)* 255b⟩≡ (492)

```
int
setpalette(ulong p, ulong r, ulong g, ulong b)
{
    VGAscr *scr;
    int d;

    scr = &vgascreen;
    d = scr->palettedepth;

    lock(&cursor);
    scr->colormap[p][0] = r;
    scr->colormap[p][1] = g;
    scr->colormap[p][2] = b;
    vgao(PaddrW, p);
    vgao(Pdata, r>>(32-d));
    vgao(Pdata, g>>(32-d));
    vgao(Pdata, b>>(32-d));
    unlock(&cursor);

    return ~0;
}
```

Chapter 15

Advanced Topics TODO

15.1 Advanced shapes

15.1.1 Special line ends

Disc

⟨_memimageline() *switch end0 cases, fallthrough to Endsquare* 256a⟩≡ (117c)

```
case Enddisc:
    discend(p0, radius, dst, src, d, op);
    /* fall through */
```

⟨_memimageline() *switch end1 cases, fallthrough to Endsquare* 256b⟩≡ (117c)

```
case Enddisc:
    discend(p1, radius, dst, src, d, op);
    /* fall through */
```

⟨*function* discend 256c⟩≡ (435a)

```
static
void
discend(Point p, int radius, Memimage *dst, Memimage *src, Point dsrc, int op)
{
    Memimage *disc;
    Rectangle r;

    disc = membrush(radius);
    if(disc != nil){
        r.min.x = p.x - radius;
        r.min.y = p.y - radius;
        r.max.x = p.x + radius+1;
        r.max.y = p.y + radius+1;
        memdraw(dst, r, src, addpt(r.min, dsrc), disc, Pt(0,0), op);
    }
}
```

⟨*function* membrush 256d⟩≡ (435a)

```
static Memimage*
membrush(int radius)
{
    static Memimage *brush;
    static int brushradius;

    if(brush == nil || brushradius != radius){
        freememimage(brush);
        brush = allocmemimage(Rect(0, 0, 2*radius+1, 2*radius+1), memopaque->chan);
```

```
        if(brush != nil){
            memfillcolor(brush, DTransparent); /* zeros */
            memellipse(brush, Pt(radius, radius), radius, radius, -1, memopaque, Pt(radius, radius), S);
        }
        brushradius = radius;
    }

    return brush;
}
```

Arrows

ARROW()

⟨*function* ARROW 257a⟩≡ (386d)

```
#define ARROW(a, b, c)  (Endarrow|((a)<<5)|((b)<<14)|((c)<<23))
```

⟨*enum* _anon_*(*(lib_graphics/libmemdraw/line.c)*)* 257b⟩≡ (435a)

```
enum
{
    Arrow1 = 8,
    Arrow2 = 10,
    Arrow3 = 3,
};
```

_memimageline() and arrows

⟨_memimageline() *switch end0 cases* 257c⟩≡ (117c)

```
case Endarrow:
    arrowend(q, pp, end0, -sin, -cos, radius);
    _memfillpolysc(dst, pts, 5, ~0, src,
                   addpt(pts[0], mulpt(d, ICOSSCALE)),
                   true, 10, true, op);
    pp[1] = pp[4];
    pp += 2;
    break;
```

⟨_memimageline() *switch end1 cases* 257d⟩≡ (117c)

```
case Endarrow:
    arrowend(q, pp, end1, sin, cos, radius);
    _memfillpolysc(dst, pp, 5, ~0, src,
                   addpt(pp[0], mulpt(d, ICOSSCALE)),
                   true, 10, true, op);
    pp[1] = pp[4];
    pp += 2;
    break;
```

⟨*function* arrowend 257e⟩≡ (435a)

```
static
void
arrowend(Point tip, Point *pp, int end, int sin, int cos, int radius)
{
    int x1, x2, x3;

    /* before rotation */
    if(end == Endarrow){
        x1 = Arrow1;
        x2 = Arrow2;
        x3 = Arrow3;
    }else{
```

```
        x1 = (end>>5) & 0x1FF; /* distance along line from end of line to tip */
        x2 = (end>>14) & 0x1FF; /* distance along line from barb to tip */
        x3 = (end>>23) & 0x1FF; /* distance perpendicular from edge of line to barb */
    }

    /* comments follow track of right-facing arrowhead */
    pp->x = tip.x+((2*radius+1)*sin/2-x1*cos);  /* upper side of shaft */
    pp->y = tip.y-((2*radius+1)*cos/2+x1*sin);
    pp++;
    pp->x = tip.x+((2*radius+2*x3+1)*sin/2-x2*cos);  /* upper barb */
    pp->y = tip.y-((2*radius+2*x3+1)*cos/2+x2*sin);
    pp++;
    pp->x = tip.x;
    pp->y = tip.y;
    pp++;
    pp->x = tip.x+(-(2*radius+2*x3+1)*sin/2-x2*cos); /* lower barb */
    pp->y = tip.y-(-(2*radius+2*x3+1)*cos/2+x2*sin);
    pp++;
    pp->x = tip.x+(-(2*radius+1)*sin/2-x1*cos);  /* lower side of shaft */
    pp->y = tip.y+((2*radius+1)*cos/2-x1*sin);
}
```

detail and yscan()

⟨xscan() *if detail* 258a⟩≡ (123h)

```
if(ix == ix2 && detail) {
    if(xerr*p[0]->den + p[0]->zerr*xden > p[0]->den*xden)
        x++;
    ix = (x + x2) >> (fixshift+1);
    ix2 = ix+1;
}
```

⟨_memfillpolysc() *if detail* 258b⟩≡ (122d)

```
if(detail)
    yscan(dst, seg, segtab, nvert, w, src, sp, fixshift, op);
```

⟨*function* xcompare 258c⟩≡ (433b)

```
static int
xcompare(void *a, void *b)
{
    Seg **s0, **s1;
    long x0, x1;

    s0 = a;
    s1 = b;
    x0 = (*s0)->p0.x;
    x1 = (*s1)->p0.x;

    if(x0 < x1)
        return -1;
    if(x0 == x1)
        return 0;
    return 1;
}
```

⟨*function* yscan 258d⟩≡ (433b)

```
static void
yscan(Memimage *dst, Seg **seg, Seg *segtab, int nseg, int wind, Memimage *src, Point sp, int fixshift, int op)
{
    long x, maxx, y, y2, yerr, yden, onehalf;
```

```
Seg **ep, **next, **p, **q, *s;
int n, i, ix, cnt, iy, iy2, miny, maxy;
Point pt;

for(i=0, s=segtab, p=seg; i<nseg; i++, s++) {
    *p = s;
    if(s->p0.x == s->p1.x)
        continue;
    if(s->p0.x > s->p1.x) {
        pt = s->p0;
        s->p0 = s->p1;
        s->p1 = pt;
        s->d = -s->d;
    }
    s->num = s->p1.y - s->p0.y;
    s->den = s->p1.x - s->p0.x;
    s->dz = sdiv(s->num, s->den) << fixshift;
    s->dzrem = mod(s->num, s->den) << fixshift;
    s->dz += sdiv(s->dzrem, s->den);
    s->dzrem = mod(s->dzrem, s->den);
    p++;
}
n = p-seg;
if(n == 0)
    return;
*p = 0;
qsort(seg, n , sizeof(Seg*), xcompare);

onehalf = 0;
if(fixshift)
    onehalf = 1 << (fixshift-1);

miny = dst->clipr.min.y;
maxy = dst->clipr.max.y;

x = seg[0]->p0.x;
if(x < (dst->clipr.min.x << fixshift))
    x = dst->clipr.min.x << fixshift;
ix = (x + onehalf) >> fixshift;
x = (ix << fixshift) + onehalf;
maxx = dst->clipr.max.x << fixshift;

ep = next = seg;

while(x<maxx) {
    for(q = p = seg; p < ep; p++) {
        s = *p;
        if(s->p1.x < x)
            continue;
        s->z += s->dz;
        s->zerr += s->dzrem;
        if(s->zerr >= s->den) {
            s->z++;
            s->zerr -= s->den;
            if(s->zerr < 0 || s->zerr >= s->den)
                print("bad ratzerr1: %ld den %ld ratdzrem %ld\n", s->zerr, s->den, s->dzrem);
        }
        *q++ = s;
    }
```

```
for(p = next; *p; p++) {
    s = *p;
    if(s->p0.x >= x)
        break;
    if(s->p1.x < x)
        continue;
    s->z = s->p0.y;
    s->z += smuldivmod(x - s->p0.x, s->num, s->den, &s->zerr);
    if(s->zerr < 0 || s->zerr >= s->den)
        print("bad ratzerr2: %ld den %ld ratdzrem %ld\n", s->zerr, s->den, s->dzrem);
    *q++ = s;
}
ep = q;
next = p;

if(ep == seg) {
    if(*next == 0)
        break;
    ix = (next[0]->p0.x + onehalf) >> fixshift;
    x = (ix << fixshift) + onehalf;
    continue;
}

zsort(seg, ep);

for(p = seg; p < ep; p++) {
    cnt = 0;
    y = p[0]->z;
    yerr = p[0]->zerr;
    yden = p[0]->den;
    iy = (y + onehalf) >> fixshift;
    if(iy >= maxy)
        break;
    if(iy < miny)
        iy = miny;
    cnt += p[0]->d;
    p++;
    for(;;) {
        if(p == ep) {
            print("yscan: fill to infinity");
            return;
        }
        cnt += p[0]->d;
        if((cnt&wind) == 0)
            break;
        p++;
    }
    y2 = p[0]->z;
    iy2 = (y2 + onehalf) >> fixshift;
    if(iy2 <= miny)
        continue;
    if(iy2 > maxy)
        iy2 = maxy;
    if(iy == iy2) {
        if(yerr*p[0]->den + p[0]->zerr*yden > p[0]->den*yden)
            y++;
        iy = (y + y2) >> (fixshift+1);
        fillpoint(dst, ix, iy, src, sp, op);
    }
}
```

```
        x += (1<<fixshift);
        ix++;
    }
}
```

⟨*function* fillpoint 261a⟩≡ (433b)

```
static void
fillpoint(Memimage *dst, int x, int y, Memimage *src, Point p, int op)
{
    Rectangle r;

    r.min.x = x;
    r.min.y = y;
    r.max.x = x+1;
    r.max.y = y+1;
    p.x += x;
    p.y += y;
    memdraw(dst, r, src, p, memopaque, p, op);
}
```

15.1.2 Bezier curves

Plist

⟨*constant* PINC 261b⟩≡ (411)

```
#define PINC 32  /* realloc granularity */
```

⟨*struct* Plist 261c⟩≡ (411)

```
struct Plist
{
    Point *p;
    int np;   /* -1 if malloc/realloc failed */
};
```

⟨*function* appendpt 261d⟩≡ (411)

```
static void
appendpt(Plist *l, Point p)
{
    if(l->np == -1)
        return;

    if(l->np == 0)
        l->p = malloc(PINC*sizeof(Point));
    else if(l->np % PINC == 0)
        l->p = realloc(l->p, (l->np+PINC)*sizeof(Point));

    if(l->p == nil){
        l->np = -1;
        return;
    }
    l->p[l->np++] = p;
}
```

API

⟨*function* bezier 261e⟩≡ (411)

```
error0
bezier(Image *dst, Point p0, Point p1, Point p2, Point p3, int end0, int end1, int radius, Image *src, Point sp
```

```
{
    return bezierop(dst, p0, p1, p2, p3, end0, end1, radius, src, sp, SoverD);
}
```

⟨*function* bezierop 262a⟩≡ (411)

```
error0
bezierop(Image *dst, Point p0, Point p1, Point p2, Point p3, int end0, int end1, int radius, Image *src, Point
{
    Plist l;

    l.np = 0;
    bezierpts(&l, p0, p1, p2, p3);
    if(l.np == -1)
        return ERROR_0;
    if(l.np != 0){
        polyop(dst, l.p, l.np, end0, end1, radius, src, addpt(subpt(sp, p0), l.p[0]), op);
        free(l.p);
    }
    return OK_1;
}
```

⟨*function* bezspline 262b⟩≡ (411)

```
error0
bezspline(Image *dst, Point *pt, int npt, int end0, int end1, int radius, Image *src, Point sp)
{
    return bezsplineop(dst, pt, npt, end0, end1, radius, src, sp, SoverD);
}
```

⟨*function* bezsplineop 262c⟩≡ (411)

```
error0
bezsplineop(Image *dst, Point *pt, int npt, int end0, int end1, int radius, Image *src, Point sp, Drawop op)
{
    Plist l;

    l.np = 0;
    _bezsplinepts(&l, pt, npt);
    if(l.np==-1)
        return ERROR_0;
    if(l.np != 0){
        polyop(dst, l.p, l.np, end0, end1, radius, src, addpt(subpt(sp, pt[0]), l.p[0]), op);
        free(l.p);
    }
    return OK_1;
}
```

⟨*function* fillbezier 262d⟩≡ (411)

```
error0
fillbezier(Image *dst, Point p0, Point p1, Point p2, Point p3, int w, Image *src, Point sp)
{
    return fillbezierop(dst, p0, p1, p2, p3, w, src, sp, SoverD);
}
```

⟨*function* fillbezierop 262e⟩≡ (411)

```
error0
fillbezierop(Image *dst, Point p0, Point p1, Point p2, Point p3, int w, Image *src, Point sp, Drawop op)
{
    Plist l;

    l.np = 0;
    bezierpts(&l, p0, p1, p2, p3);
```

```
    if(l.np == -1)
        return ERROR_0;
    if(l.np != 0){
        fillpolyop(dst, l.p, l.np, w, src, addpt(subpt(sp, p0), l.p[0]), op);
        free(l.p);
    }
    return OK_1;
}
```

⟨*function* fillbezspline 263a⟩≡ (411)

```
error0
fillbezspline(Image *dst, Point *pt, int npt, int w, Image *src, Point sp)
{
    return fillbezsplineop(dst, pt, npt, w, src, sp, SoverD);
}
```

⟨*function* fillbezsplineop 263b⟩≡ (411)

```
error0
fillbezsplineop(Image *dst, Point *pt, int npt, int w, Image *src, Point sp, Drawop op)
{
    Plist l;

    l.np = 0;
    _bezsplinepts(&l, pt, npt);
    if(l.np == -1)
        return ERROR_0;
    if(l.np > 0){
        fillpolyop(dst, l.p, l.np, w, src, addpt(subpt(sp, pt[0]), l.p[0]), op);
        free(l.p);
    }
    return OK_1;
}
```

⟨*function* bezsplinepts 263c⟩≡ (411)

```
int
bezsplinepts(Point *pt, int npt, Point **pp)
{
    Plist l;
    l.np = 0;
    l.p = nil;
    _bezsplinepts(&l, pt, npt);
    *pp  = l.p;
    return l.np;
}
```

Algorithm

beziepts()

⟨*function* beziepts 263d⟩≡ (411)

```
static void
beziepts(Plist *l, Point p0, Point p1, Point p2, Point p3)
{
    bpts(l, p0, p1, p2, p3);
    appendpt(l, p3);
}
```

⟨*function* bpts 264a⟩≡ (411)

```
static void
bpts(Plist *l, Point p0, Point p1, Point p2, Point p3)
{
    bpts1(l, p0, p1, p2, p3, 1);
}
```

⟨*function* bpts1 264b⟩≡ (411)

```
/*
 * Convert cubic Bezier curve control points to polyline
 * vertices.  Leaves the last vertex off, so you can continue
 * with another curve.
 */
static void
bpts1(Plist *l, Point p0, Point p1, Point p2, Point p3, int scale)
{
    Point p01, p12, p23, p012, p123, p0123;
    Point tp0, tp1, tp2, tp3;
    tp0=divpt(p0, scale);
    tp1=divpt(p1, scale);
    tp2=divpt(p2, scale);
    tp3=divpt(p3, scale);
    if(psdist(tp1, tp0, tp3)<=1 && psdist(tp2, tp0, tp3)<=1){
        appendpt(l, tp0);
        appendpt(l, tp1);
        appendpt(l, tp2);
    }
    else{
        /*
         * if scale factor is getting too big for comfort,
         * rescale now & concede the rounding error
         */
        if(scale>(1<<12)){
            p0=tp0;
            p1=tp1;
            p2=tp2;
            p3=tp3;
            scale=1;
        }
        p01=addpt(p0, p1);
        p12=addpt(p1, p2);
        p23=addpt(p2, p3);
        p012=addpt(p01, p12);
        p123=addpt(p12, p23);
        p0123=addpt(p012, p123);
        bpts1(l, mulpt(p0, 8), mulpt(p01, 4), mulpt(p012, 2), p0123, scale*8);
        bpts1(l, p0123, mulpt(p123, 2), mulpt(p23, 4), mulpt(p3, 8), scale*8);
    }
}
```

⟨*function* normsq 264c⟩≡ (411)

```
static int
normsq(Point p)
{
    return p.x*p.x+p.y*p.y;
}
```

⟨*function* psdist 264d⟩≡ (411)

```
static int
psdist(Point p, Point a, Point b)
```

```
{
    int num, den;

    p = subpt(p, a);
    b = subpt(b, a);
    num = p.x*b.x + p.y*b.y;
    if(num <= 0)
        return normsq(p);
    den = normsq(b);
    if(num >= den)
        return normsq(subpt(b, p));
    return normsq(subpt(divpt(mulpt(b, num), den), p));
}
```

_bezsplinepts()

⟨*function* _bezsplinepts 265⟩≡ (411)

```
static void
_bezsplinepts(Plist *l, Point *pt, int npt)
{
    Point *p, *ep;
    Point a, b, c, d;
    int periodic;

    if(npt<3)
        return;
    ep = &pt[npt-3];
    periodic = eqpt(pt[0], ep[2]);
    if(periodic){
        a = divpt(addpt(ep[1], pt[0]), 2);
        b = divpt(addpt(ep[1], mulpt(pt[0], 5)), 6);
        c = divpt(addpt(mulpt(pt[0], 5), pt[1]), 6);
        d = divpt(addpt(pt[0], pt[1]), 2);
        bpts(l, a, b, c, d);
    }
    for(p=pt; p<=ep; p++){
        if(p==pt && !periodic){
            a = p[0];
            b = divpt(addpt(p[0], mulpt(p[1], 2)), 3);
        }
        else{
            a = divpt(addpt(p[0], p[1]), 2);
            b = divpt(addpt(p[0], mulpt(p[1], 5)), 6);
        }
        if(p==ep && !periodic){
            c = divpt(addpt(mulpt(p[1], 2), p[2]), 3);
            d = p[2];
        }
        else{
            c = divpt(addpt(mulpt(p[1], 5), p[2]), 6);
            d = divpt(addpt(p[1], p[2]), 2);
        }
        bpts(l, a, b, c, d);
    }
    appendpt(l, d);
}
```

15.2 Advanced compositing

15.2.1 Reading replicated pixels

⟨readbyte *other locals* 266a⟩≡ (108b)

```
int i;
byte *rrepl, *grepl, *brepl, *arepl, *krepl;
byte ured, ugrn, ublu;
ulong u;
```

⟨readbyte() *more complex cases, possible repl, grey, and small depth* 266b⟩≡ (108b)

```
DBG1("2\n");
rrepl = replbit[img->nbits[CRed]];
grepl = replbit[img->nbits[CGreen]];
brepl = replbit[img->nbits[CBlue]];
arepl = replbit[img->nbits[CAlpha]];

krepl = replbit[img->nbits[CGrey]];

for(i=0; i<dx; i++){
    u = r[0] | (r[1]<<8) | (r[2]<<16) | (r[3]<<24);
    if(copyalpha) {
        *w++ = arepl[(u>>img->shift[CAlpha]) & img->mask[CAlpha]];
        //DBG print("a %x\n", w[-1]);
    }

    if(isgrey)
        *w++ = krepl[(u >> img->shift[CGrey]) & img->mask[CGrey]];
    else if(!alphaonly){
        ured = rrepl[(u >> img->shift[CRed]) & img->mask[CRed]];
        ugrn = grepl[(u >> img->shift[CGreen]) & img->mask[CGreen]];
        ublu = brepl[(u >> img->shift[CBlue]) & img->mask[CBlue]];
        if(convgrey){
            DBG1("g %x %x %x\n", ured, ugrn, ublu);
            *w++ = RGB2K(ured, ugrn, ublu);
            DBG1("%x\n", w[-1]);
        }else{
            *w++ = brepl[(u >> img->shift[CBlue]) & img->mask[CBlue]];
            *w++ = grepl[(u >> img->shift[CGreen]) & img->mask[CGreen]];
            *w++ = rrepl[(u >> img->shift[CRed]) & img->mask[CRed]];
        }
    }
    r += nb;
    if(r == end)
        r = begin;
}

b.alpha = copyalpha ? buf : &ones;
b.rgba = (ulong*)buf;
if(alphaonly){
    b.red = b.grn = b.blu = b.grey = nil;
    if(!copyalpha)
        b.rgba = nil;
    b.delta = 1;
}else if(isgrey || convgrey){
    b.grey = buf+copyalpha;
    b.red = b.grn = b.blu = buf+copyalpha;
    b.delta = copyalpha+1;
    DBG1("alpha %x grey %x\n", b.alpha ? *b.alpha : 0xFF, *b.grey);
}else{
```

```
    b.blu = buf+copyalpha;
    b.grn = buf+copyalpha+1;
    b.grey = nil;
    b.red = buf+copyalpha+2;
    b.delta = copyalpha+3;
  }
  return b;
```

mktables

⟨memimageinit() *initializations* 267a⟩+≡ (48a) ◁248b

```
  mktables();
```

⟨*global* replbit 267b⟩≡ (429b)

```
  /*
   * Conversion tables.
   */
  uchar replbit[1+8][256];  /* replbit[x][y] is the replication of the x-bit quantity y to 8-bit depth */
```

⟨*function* mktables 267c⟩≡ (429a)

```
  static void
  mktables(void)
  {
      int i, j, small;
      ⟨mktables() only once guard 267d⟩

      /* bit replication up to 8 bits */
      for(i=0; i<256; i++){
          for(j=0; j<=8; j++){ /* j <= 8 [sic] */
              small = i & ((1<<j)-1);
              replbit[j][i] = (small*replmul[j])>>8;
          }
      }

  }
```

⟨mktables() *only once guard* 267d⟩≡ (267c)

```
  static bool tablesbuilt = false;
  if(tablesbuilt)
      return;
  tablesbuilt = true;
```

15.2.2 Other compositing operators

Clear

⟨*function* alphacalc0 267e⟩≡ (429b)

```
  static Buffer
  alphacalc0(Buffer bdst, Buffer b1, Buffer b2, int dx, int grey, int op)
  {
      USED(grey);
      USED(op);
      USED(b1);
      USED(b2);

      memset(bdst.rgba, 0, dx * bdst.delta);
      return bdst;
  }
```

DoutS

⟨*function* alphacalc14 268a⟩≡ (429b)

```
static Buffer
alphacalc14(Buffer bdst, Buffer bsrc, Buffer bmask, int dx, int grey, int op)
{
    Buffer obdst;
    int fd, sadelta;
    int i, sa, ma, q;
    ulong s, t;

    obdst = bdst;
    sadelta = bsrc.alpha == &ones ? 0 : bsrc.delta;
    q = bsrc.delta == 4 && bdst.delta == 4;

    for(i=0; i<dx; i++){
        sa = *bsrc.alpha;
        ma = *bmask.alpha;
        fd = MUL(sa, ma, t);
        if(op == DoutS)
            fd = 255-fd;

        if(grey){
            *bdst.grey = MUL(fd, *bdst.grey, t);
            bsrc.grey += bsrc.delta;
            bdst.grey += bdst.delta;
        }else{
            if(q){
                *bdst.rgba = MUL0123(fd, *bdst.rgba, s, t);
                bsrc.rgba++;
                bdst.rgba++;
                bsrc.alpha += sadelta;
                bmask.alpha += bmask.delta;
                continue;
            }
            *bdst.red = MUL(fd, *bdst.red, t);
            *bdst.grn = MUL(fd, *bdst.grn, t);
            *bdst.blu = MUL(fd, *bdst.blu, t);
            bsrc.red += bsrc.delta;
            bsrc.blu += bsrc.delta;
            bsrc.grn += bsrc.delta;
            bdst.red += bdst.delta;
            bdst.blu += bdst.delta;
            bdst.grn += bdst.delta;
        }
        if(bdst.alpha != &ones){
            *bdst.alpha = MUL(fd, *bdst.alpha, t);
            bdst.alpha += bdst.delta;
        }
        bmask.alpha += bmask.delta;
        bsrc.alpha += sadelta;
    }
    return obdst;
}
```

SoutD

⟨*function* alphacalc2810 268b⟩≡ (429b)

```
static Buffer
alphacalc2810(Buffer bdst, Buffer bsrc, Buffer bmask, int dx, int grey, int op)
```

```
{
    Buffer obdst;
    int fs, sadelta;
    int i, ma, da, q;
    ulong s, t;

    obdst = bdst;
    sadelta = bsrc.alpha == &ones ? 0 : bsrc.delta;
    q = bsrc.delta == 4 && bdst.delta == 4;

    for(i=0; i<dx; i++){
        ma = *bmask.alpha;
        da = *bdst.alpha;
        if(op == SoutD)
            da = 255-da;
        fs = ma;
        if(op != S)
            fs = MUL(fs, da, t);

        if(grey){
            *bdst.grey = MUL(fs, *bsrc.grey, t);
            bsrc.grey += bsrc.delta;
            bdst.grey += bdst.delta;
        }else{
            if(q){
                *bdst.rgba = MUL0123(fs, *bsrc.rgba, s, t);
                bsrc.rgba++;
                bdst.rgba++;
                bmask.alpha += bmask.delta;
                bdst.alpha += bdst.delta;
                continue;
            }
            *bdst.red = MUL(fs, *bsrc.red, t);
            *bdst.grn = MUL(fs, *bsrc.grn, t);
            *bdst.blu = MUL(fs, *bsrc.blu, t);
            bsrc.red += bsrc.delta;
            bsrc.blu += bsrc.delta;
            bsrc.grn += bsrc.delta;
            bdst.red += bdst.delta;
            bdst.blu += bdst.delta;
            bdst.grn += bdst.delta;
        }
        if(bdst.alpha != &ones){
            *bdst.alpha = MUL(fs, *bsrc.alpha, t);
            bdst.alpha += bdst.delta;
        }
        bmask.alpha += bmask.delta;
        bsrc.alpha += sadelta;
    }
    return obdst;
}
```

DxorS

⟨*function* alphacalc3679 269⟩≡ (429b)

```
static Buffer
alphacalc3679(Buffer bdst, Buffer bsrc, Buffer bmask, int dx, bool grey, int op)
{
    Buffer obdst;
```

```
    int fs, fd, sadelta;
    int i, sa, ma, da, q;
    ulong s, t, u, v;

    obdst = bdst;
    sadelta = (bsrc.alpha == &ones) ? 0 : bsrc.delta;
    q = (bsrc.delta == 4) && (bdst.delta == 4);

    for(i=0; i<dx; i++){
        sa = *bsrc.alpha;
        ma = *bmask.alpha;
        da = *bdst.alpha;
        if(op == SatopD)
            fs = MUL(ma, da, t);
        else
            fs = MUL(ma, 255-da, t);
        if(op == DoverS)
            fd = 255;
        else{
            fd = MUL(sa, ma, t);
            if(op != DatopS)
                fd = 255-fd;
        }

        if(grey){
            *bdst.grey = MUL(fs, *bsrc.grey, s) + MUL(fd, *bdst.grey, t);
            bsrc.grey += bsrc.delta;
            bdst.grey += bdst.delta;
        }else{
            if(q){
                *bdst.rgba = MUL0123(fs, *bsrc.rgba, s, t)+MUL0123(fd, *bdst.rgba, u, v);
                bsrc.rgba++;
                bdst.rgba++;
                bsrc.alpha += sadelta;
                bmask.alpha += bmask.delta;
                bdst.alpha += bdst.delta;
                continue;
            }
            *bdst.red = MUL(fs, *bsrc.red, s)+MUL(fd, *bdst.red, t);
            *bdst.grn = MUL(fs, *bsrc.grn, s)+MUL(fd, *bdst.grn, t);
            *bdst.blu = MUL(fs, *bsrc.blu, s)+MUL(fd, *bdst.blu, t);
            bsrc.red += bsrc.delta;
            bsrc.blu += bsrc.delta;
            bsrc.grn += bsrc.delta;
            bdst.red += bdst.delta;
            bdst.blu += bdst.delta;
            bdst.grn += bdst.delta;
        }
        if(bdst.alpha != &ones){
            *bdst.alpha = MUL(fs, sa, s)+MUL(fd, da, t);
            bdst.alpha += bdst.delta;
        }
        bmask.alpha += bmask.delta;
        bsrc.alpha += sadelta;
    }
    return obdst;
}
```

DinS

D

⟨*function* alphacalc5 271a⟩≡ (429b)

```
static Buffer
alphacalc5(Buffer bdst, Buffer b1, Buffer b2, int dx, int grey, int op)
{
    USED(dx);
    USED(grey);
    USED(op);
    USED(b1);
    USED(b2);

    return bdst;
}
```

DatopS

DoverS

SinD

SatopD

S

SoverD

15.3 Advanced mouse settings

15.3.1 /dev/cursor

⟨Mousectl *cursor field* 271b⟩≡ (183d)

```
// /dev/cursor
fdt  cfd;  /* to cursor file */
```

⟨initmouse() *other locals* 271c⟩≡ (184c)

```
char *t;
char *sl;
```

⟨initmouse() *set cursor field* 271d⟩≡ (184c)

```
⟨initmouse() set t to /dev/cursor 271e⟩
mc->cfd = open(t, ORDWR|OCEXEC);
free(t);
```

⟨initmouse() *set t to /dev/cursor* 271e⟩≡ (271d)

```
// t = "{basename(file)}/cursor"
t = malloc(strlen(file)+16);
⟨initmouse() sanity check t 272a⟩
strcpy(t, file);
sl = utfrrune(t, '/');
if(sl)
    strcpy(sl, "/cursor");
else
    strcpy(t, "/dev/cursor");
```

⟨initmouse() *sanity check t* 272a⟩≡ (271e)

```
if (t == nil) {
    close(mc->mfd);
    free(mc);
    return nil;
}
```

⟨*function* setcursor 272b⟩≡ (417c)

```
void
setcursor(Mousectl *mc, Cursor *c)
{
    char curs[2*4 + 2*2*16]; // sizeof Cursor

    if(c == nil)
        write(mc->cfd, curs, 0);
    else{
        BPLONG(curs+0*4, c->offset.x);
        BPLONG(curs+1*4, c->offset.y);
        memmove(curs+2*4, c->clr, 2*2*16);
        write(mc->cfd, curs, sizeof curs);
    }
}
```

15.3.2 mousescrollsize()

⟨*function* mousescrollsize 272c⟩≡ (417c)

```
int
mousescrollsize(int maxlines)
{
    static int lines, pcnt;
    char *mss;

    if(lines == 0 && pcnt == 0){
        mss = getenv("mousescrollsize");
        if(mss){
            if(strchr(mss, '%') != nil)
                pcnt = atof(mss);
            else
                lines = atoi(mss);
            free(mss);
        }
        if(lines == 0 && pcnt == 0)
            lines = 1;
        if(pcnt>=100)
            pcnt = 100;
    }

    if(lines)
        return lines;
    return pcnt * maxlines/100.0;
}
```

15.4 Menus

15.4.1 Menu

⟨*struct* Menu 272d⟩≡ (397a)

```
struct Menu
{
    char **item;
    ⟨Menu other fields 275b⟩
};
```

15.4.2 Constants

⟨*enum* _anon_*(*(lib_graphics/libdraw/menuhit.c)*)* 273a⟩≡ (418a)

```
enum
{
    Margin = 4,  /* outside to text */
    Border = 2,  /* outside to selection boxes */
    Blackborder = 2, /* width of outlining border */
    Vspacing = 2,  /* extra spacing between lines of text */
    ⟨menuhit.c constants 279h⟩
};
```

15.4.3 Colors

⟨*global* back*(*(lib_graphics/libdraw/menuhit.c)*)* 273b⟩≡ (418a)

```
static Image *back;
```

⟨*global* high*(*(lib_graphics/libdraw/menuhit.c)*)* 273c⟩≡ (418a)

```
static Image *high;
```

⟨*global* bord*(*(lib_graphics/libdraw/menuhit.c)*)* 273d⟩≡ (418a)

```
static Image *bord;
```

⟨*global* text*(*(lib_graphics/libdraw/menuhit.c)*)* 273e⟩≡ (418a)

```
static Image *text;
```

⟨*global* htext*(*(lib_graphics/libdraw/menuhit.c)*)* 273f⟩≡ (418a)

```
static Image *htext;
```

⟨*global* menutxt*(*(lib_graphics/libdraw/menuhit.c)*)* 273g⟩≡ (418a)

```
static Image *menutxt;
```

⟨menuhit() *initialize colors* 273h⟩≡ (274f)

```
if(back == nil)
    menucolors();
```

⟨*function* menucolors*(*(lib_graphics/libdraw/menuhit.c)*)* 273i⟩≡ (418a)

```
static
void
menucolors(void)
{
    /* Main tone is greenish, with negative selection */
    back = allocimagemix(display, DPalegreen, DWhite);
    high = allocimage(display, Rect(0,0,1,1), view->chan, true, DDarkgreen); /* dark green */
    bord = allocimage(display, Rect(0,0,1,1), view->chan, true, DMedgreen); /* not as dark green */
    ⟨menucolors() sanity check images 274a⟩
    text = display->black;
    htext = back;
    return;

⟨menucolors() Error label 274b⟩
}
```

⟨menucolors() *sanity check images* 274a⟩≡ (273i)

```
if(back == nil || high == nil || bord == nil)
    goto Error;
```

⟨menucolors() *Error label* 274b⟩≡ (273i)

```
Error:
freeimage(back);
freeimage(high);
freeimage(bord);
back = display->white;
high = display->black;
bord = display->black;
text = display->black;
htext = display->white;
```

⟨*function* allocimagemix 274c⟩≡ (402a)

```
Image*
allocimagemix(Display *d, ulong color1, ulong color3)
{
    Image *t, *b;
    static Image *qmask;

    if(qmask == nil)
        qmask = allocimage(d, Rect(0,0,1,1), GREY8, true, 0x3F3F3FFF);

    ⟨allocimagemix() if depth less than 8 240a⟩
    else{ /* use a solid color, blended using alpha */
        t = allocimage(d, Rect(0,0,1,1), d->screenimage->chan, true, color1);
        ⟨allocimagemix() sanity check t 274d⟩
        b = allocimage(d, Rect(0,0,1,1), d->screenimage->chan, true, color3);
        ⟨allocimagemix() sanity check b 274e⟩
        draw(b, b->r, t, qmask, ZP);
        freeimage(t);
        return b;
    }
}
```

⟨allocimagemix() *sanity check t* 274d⟩≡ (274c)

```
if(t == nil)
    return nil;
```

⟨allocimagemix() *sanity check b* 274e⟩≡ (274c)

```
if(b == nil){
    freeimage(t);
    return nil;
}
```

15.4.4 menuhits()

⟨*function* menuhit 274f⟩≡ (418a)

```
int
menuhit(int button, Mousectl *mc, Menu *menu, Screen *scr)
{
    int i;
    int lasti;
    char *item;
    int nitem, nitemdrawn;
    int maxwid, wid;
    Rectangle r, menur, textr;
```

```
    Image *b, *backup, *save;
    Point pt;
    bool scrolling;
    int off;
    ⟨menuhit() other locals 279b⟩

    ⟨menuhit() initialize colors 273h⟩
    ⟨menuhit() set repl and clip on view 279c⟩

    ⟨menuhit() compute maxwid and nitem 275a⟩
    ⟨menuhit() set lasthit 279e⟩
    ⟨menuhit() if need a scroller for items 280a⟩
    else{
        scrolling = false;
        nitemdrawn = nitem;
        wid = maxwid;
        off = 0;
        lasti = menu->lasthit;
    }
    ⟨menuhit() set r, the temporary rectangle to draw the menu 275c⟩
    ⟨menuhit() set menur, the adjusted rectangle to draw the menu 276a⟩
    ⟨menuhit() set textr, the rectangle to draw the text 276c⟩
    ⟨menuhit() set scrollr, the rectangle to draw maybe a scrollbar 280c⟩

    ⟨menuhit() set b the image to draw on and right backup 276f⟩
    ⟨menuhit() paint on b 277b⟩

    ⟨menuhit() allocate save image 278b⟩
    ⟨menuhit() loop while button still pressed and return lasti 278a⟩

    ⟨menuhit() restore what was under the menu in backup 277a⟩
    ⟨menuhit() free save image 278e⟩
    ⟨menuhit() restore clip on view 279d⟩
    flushimage(display, true);
    if(lasti >= 0){
        ⟨menuhit() save lasthit before returning 279g⟩
        return menu->lasthit;
    }
    return -1;
}
```

The items size

⟨menuhit() *compute* maxwid *and* nitem 275a⟩≡ (274f)

```
maxwid = 0;
for(nitem = 0;
    item = menu->item? menu->item[nitem] : (*menu->gen)(nitem);
    nitem++){
    i = stringwidth(font, item);
    if(i > maxwid)
        maxwid = i;
}
```

⟨Menu *other fields* 275b⟩≡ (272d) 279f▷

```
char *(*gen)(int);
```

The rectangles

⟨menuhit() *set* r, *the temporary rectangle to draw the menu* 275c⟩≡ (274f)

```
r = Rect(0, 0, wid, nitemdrawn*(font->height+Vspacing));
r = insetrect(r, -Margin); // enlarge
r = rectsubpt(r, Pt(wid/2, lasti*(font->height+Vspacing)+font->height/2));
r = rectaddpt(r, mc->xy);
```

⟨menuhit() *set* menur, *the adjusted rectangle to draw the menu* 276a⟩≡ (274f)

```
pt = ZP;
⟨menuhit() adjust pt if r outside view 276b⟩
menur = rectaddpt(r, pt);
```

⟨menuhit() *adjust* pt *if* r *outside view* 276b⟩≡ (276a)

```
if(r.max.x > view->r.max.x)
    pt.x = view->r.max.x - r.max.x;
if(r.max.y > view->r.max.y)
    pt.y = view->r.max.y - r.max.y;
if(r.min.x < view->r.min.x)
    pt.x = view->r.min.x-r.min.x;
if(r.min.y < view->r.min.y)
    pt.y = view->r.min.y-r.min.y;
```

⟨menuhit() *set* textr, *the rectangle to draw the text* 276c⟩≡ (274f)

```
textr.max.x = menur.max.x - Margin;
textr.min.x = textr.max.x - maxwid;
textr.min.y = menur.min.y + Margin;
textr.max.y = textr.min.y + nitemdrawn*(font->height+Vspacing);
```

⟨*function* menurect*(*(lib_graphics/libdraw/menuhit.c)*)* 276d⟩≡ (418a)

```
/*
 * r is a rectangle holding the text elements.
 * return the rectangle, including its black edge, holding element i.
 */
static Rectangle
menurect(Rectangle r, int i)
{
    ⟨menurect() sanity check i 276e⟩
    r.min.y += (font->height+Vspacing)*i;
    r.max.y = r.min.y + font->height+Vspacing;
    return insetrect(r, Border-Margin);
}
```

⟨menurect() *sanity check i* 276e⟩≡ (276d)

```
if(i < 0)
    return Rect(0, 0, 0, 0);
```

The image

⟨menuhit() *set* b *the image to draw on and right* backup 276f⟩≡ (274f)

```
if(scr){
    b = allocwindow(scr, menur, Refbackup, DWhite);
    ⟨menuhit() sanitize b 276g⟩
    backup = nil;
}else{
    b = view;
    backup = allocimage(display, menur, view->chan, false, -1);
    if(backup)
        draw(backup, menur, view, nil, menur.min);
}
```

⟨menuhit() *sanitize b* 276g⟩≡ (276f)

```
if(b == nil)
    b = view;
```

⟨menuhit() *restore what was under the menu in* backup 277a⟩≡ (274f)

```
if(b != view)
    freeimage(b);
if(backup){
    draw(view, menur, backup, nil, menur.min);
    freeimage(backup);
}
```

The paint

⟨menuhit() *paint on* b 277b⟩≡ (274f)

```
draw(b, menur, back, nil, ZP);
border(b, menur, Blackborder, bord, ZP);
⟨menuhit() move cursor to lasti entry 277f⟩
menupaint(b, menu, textr, off, nitemdrawn);
⟨menuhit() if scrolling, paint the scroll 280e⟩
```

⟨*function* menupaint*(*(lib_graphics/libdraw/menuhit.c)*)* 277c⟩≡ (418a)

```
static void
menupaint(Image *m, Menu *menu, Rectangle textr, int off, int nitemdrawn)
{
    int i;

    draw(m, insetrect(textr, Border-Margin), back, nil, ZP);
    for(i = 0; i<nitemdrawn; i++)
        paintitem(m, menu, textr, off, i, false, nil, nil);
}
```

⟨*function* paintitem*(*(lib_graphics/libdraw/menuhit.c)*)* 277d⟩≡ (418a)

```
static
void
paintitem(Image *m, Menu *menu, Rectangle textr, int off, int i, bool highlight, Image *save, Image *restore)
{
    char *item;
    Rectangle r;
    Point pt;

    ⟨paintitem() sanity check i 277e⟩
    r = menurect(textr, i);
    ⟨paintitem() if restore 278d⟩
    ⟨paintitem() if save 278c⟩
    item = menu->item? menu->item[i+off] : (*menu->gen)(i+off);
    pt.x = (textr.min.x + textr.max.x - stringwidth(font, item))/2;
    pt.y = textr.min.y + i*(font->height+Vspacing);
    draw(m, r, highlight? high : back, nil, pt);
    string(m, pt, highlight? htext : text, pt, font, item);
}
```

⟨paintitem() *sanity check i* 277e⟩≡ (277d)

```
if(i < 0)
    return;
```

⟨menuhit() *move cursor to lasti entry* 277f⟩≡ (277b)

```
r = menurect(textr, lasti);
moveto(mc, divpt(addpt(r.min, r.max), 2));
```

The loop

⟨menuhit() *loop while button still pressed and return* lasti 278a⟩≡ (274f)

```
while(mc->buttons & (1<<(button-1))){
    lasti = menuscan(b, menu, button, mc, textr, off, lasti, save);
    if(lasti >= 0)
        break;
    while(!ptinrect(mc->xy, textr) && (mc->buttons & (1<<(button-1)))){
        ⟨menuhit() if scrolling and mouse in scrollr 281a⟩
        readmouse(mc);
    }
}
```

⟨menuhit() *allocate* save *image* 278b⟩≡ (274f)

```
save = allocimage(display, menurect(textr, 0), view->chan, false, -1);
```

⟨paintitem() *if save* 278c⟩≡ (277d)

```
if(save)
    draw(save, save->r, m, nil, r.min);
```

⟨paintitem() *if restore* 278d⟩≡ (277d)

```
if(restore){
    draw(m, r, restore, nil, restore->r.min);
    return;
}
```

⟨menuhit() *free* save *image* 278e⟩≡ (274f)

```
freeimage(save);
```

The scan

⟨*function* menuscan*(*(lib_graphics/libdraw/menuhit.c)*)* 278f⟩≡ (418a)

```
/*
 * menur is a rectangle holding all the highlightable text elements.
 * track mouse while inside the box, return what's selected when button
 * is raised, -1 as soon as it leaves box.
 * invariant: nothing is highlighted on entry or exit.
 */
static int
menuscan(Image *m, Menu *menu, int button, Mousectl *mc, Rectangle textr, int off, int lasti, Image *save)
{
    int i;

    paintitem(m, menu, textr, off, lasti, true, save, nil);
    for(readmouse(mc); mc->buttons & (1<<(button-1)); readmouse(mc)){
        i = menusel(textr, mc->xy);
        if(i != -1 && i == lasti)
            continue;
        // else (i == -1 || i != lasti)
        paintitem(m, menu, textr, off, lasti, false, nil, save); // restore
        if(i == -1)
            return i;
        lasti = i;
        paintitem(m, menu, textr, off, lasti, true, save, nil);
    }
    return lasti;
}
```

⟨*function* menusel*(*(lib_graphics/libdraw/menuhit.c)*)* 279a⟩≡ (418a)

```
/*
 * r is a rectangle holding the text elements.
 * return the element number containing p.
 */
static int
menusel(Rectangle r, Point p)
{
    r = insetrect(r, Margin);
    if(!ptinrect(p, r))
        return -1;
    return (p.y - r.min.y)/(font->height+Vspacing);
}
```

Misc

⟨menuhit() *other locals* 279b⟩≡ (274f) 279i ▷

```
Rectangle sc; // save clipr
```

⟨menuhit() *set repl and clip on view* 279c⟩≡ (274f)

```
sc = view->clipr;
replclipr(view, false, view->r);
```

⟨menuhit() *restore clip on view* 279d⟩≡ (274f)

```
replclipr(view, false, sc);
```

15.4.5 Last hit

⟨menuhit() *set lasthit* 279e⟩≡ (274f)

```
if(menu->lasthit<0 || menu->lasthit>=nitem)
    menu->lasthit = 0;
```

⟨Menu *other fields* 279f⟩+≡ (272d) ◁ 275b

```
int lasthit;
```

⟨menuhit() *save lasthit before returning* 279g⟩≡ (274f)

```
menu->lasthit = lasti+off;
```

15.4.6 Scrolling menus

⟨menuhit.c *constants* 279h⟩≡ (273a)

```
Maxunscroll = 25, /* maximum #entries before scrolling turns on */
Nscroll = 20,  /* number entries in scrolling part */
Scrollwid = 14,  /* width of scroll bar */
Gap = 4,   /* between text and scroll bar */
```

⟨menuhit() *other locals* 279i⟩+≡ (274f) ◁ 279b 280b ▷

```
int screenitem;
```

⟨menuhit() *if need a scroller for items* 280a⟩≡ (274f)

```
screenitem = (Dy(view->r) - 10) / (font->height + Vspacing);
if(nitem>Maxunscroll || nitem>screenitem){
    scrolling = 1;
    nitemdrawn = Nscroll;
    if(nitemdrawn > screenitem)
        nitemdrawn = screenitem;
    wid = maxwid + Gap + Scrollwid;
    off = menu->lasthit - nitemdrawn/2;
    if(off < 0)
        off = 0;
    if(off > nitem-nitemdrawn)
        off = nitem-nitemdrawn;
    lasti = menu->lasthit-off;
}
```

⟨menuhit() *other locals* 280b⟩+≡ (274f) ◁279i 280g▷

```
Rectangle scrollr;
```

⟨menuhit() *set* scrollr, *the rectangle to draw maybe a scrollbar* 280c⟩≡ (274f)

```
⟨menuhit() if scrolling set scrollr 280d⟩
else
    scrollr = Rect(0, 0, 0, 0);
```

⟨menuhit() *if scrolling set scrollr* 280d⟩≡ (280c)

```
if(scrolling){
    scrollr = insetrect(menur, Border);
    scrollr.max.x = scrollr.min.x+Scrollwid;
}
```

⟨menuhit() *if scrolling, paint the scroll* 280e⟩≡ (277b)

```
if(scrolling)
    menuscrollpaint(b, scrollr, off, nitem, nitemdrawn);
```

⟨*function* menuscrollpaint*(*(lib_graphics/libdraw/menuhit.c)*)* 280f⟩≡ (418a)

```
static void
menuscrollpaint(Image *m, Rectangle scrollr, int off, int nitem, int nitemdrawn)
{
    Rectangle r;

    draw(m, scrollr, back, nil, ZP);
    r.min.x = scrollr.min.x;
    r.max.x = scrollr.max.x;
    r.min.y = scrollr.min.y + (Dy(scrollr)*off)/nitem;
    r.max.y = scrollr.min.y + (Dy(scrollr)*(off+nitemdrawn))/nitem;
    if(r.max.y < r.min.y+2)
        r.max.y = r.min.y+2;
    border(m, r, 1, bord, ZP);
    if(menutxt == nil)
        menutxt = allocimage(display, Rect(0, 0, 1, 1), view->chan, true, DDarkgreen); /* border color; BUG? */
    if(menutxt)
        draw(m, insetrect(r, 1), menutxt, nil, ZP);
}
```

⟨menuhit() *other locals* 280g⟩+≡ (274f) ◁280b

```
int noff;
```

⟨menuhit() *if scrolling and mouse in* scrollr 281a⟩≡ (278a)

```
if(scrolling && ptinrect(mc->xy, scrollr)){
    noff = ((mc->xy.y-scrollr.min.y)*nitem)/Dy(scrollr);
    noff -= nitemdrawn/2;
    if(noff < 0)
        noff = 0;
    if(noff > nitem-nitemdrawn)
        noff = nitem-nitemdrawn;
    if(noff != off){
        off = noff;
        menupaint(b, menu, textr, off, nitemdrawn);
        menuscrollpaint(b, scrollr, off, nitem, nitemdrawn);
    }
}
```

15.5 Selection

15.5.1 getrect.c

⟨*constant* W*(*(lib_graphics/libdraw/getrect.c)*)* 281b⟩≡ (418b)

```
#define W Borderwidth
```

⟨*global* tmp*(*(lib_graphics/libdraw/getrect.c)*)* 281c⟩≡ (418b)

```
static Image *tmp[4];
```

⟨*global* red*(*(lib_graphics/libdraw/getrect.c)*)* 281d⟩≡ (418b)

```
static Image *red;
```

⟨*global* sweep*(*(lib_graphics/libdraw/getrect.c)*)* 281e⟩≡ (418b)

```
static Cursor sweep={
    {-7, -7},
    {0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xE0, 0x07,
     0xE0, 0x07, 0xE0, 0x07, 0xE3, 0xF7, 0xE3, 0xF7,
     0xE3, 0xE7, 0xE3, 0xF7, 0xE3, 0xFF, 0xE3, 0x7F,
     0xE0, 0x3F, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF,},
    {0x00, 0x00, 0x7F, 0xFE, 0x40, 0x02, 0x40, 0x02,
     0x40, 0x02, 0x40, 0x02, 0x40, 0x02, 0x41, 0xE2,
     0x41, 0xC2, 0x41, 0xE2, 0x41, 0x72, 0x40, 0x38,
     0x40, 0x1C, 0x40, 0x0E, 0x7F, 0xE6, 0x00, 0x00,}
};
```

⟨*function* brects*(*(lib_graphics/libdraw/getrect.c)*)* 281f⟩≡ (418b)

```
static
void
brects(Rectangle r, Rectangle rp[4])
{
    if(Dx(r) < 2*W)
        r.max.x = r.min.x+2*W;
    if(Dy(r) < 2*W)
        r.max.y = r.min.y+2*W;
    rp[0] = Rect(r.min.x, r.min.y, r.max.x, r.min.y+W);
    rp[1] = Rect(r.min.x, r.max.y-W, r.max.x, r.max.y);
    rp[2] = Rect(r.min.x, r.min.y+W, r.min.x+W, r.max.y-W);
    rp[3] = Rect(r.max.x-W, r.min.y+W, r.max.x, r.max.y-W);
}
```

⟨*function* getrect 282a⟩≡ (418b)

```
Rectangle
getrect(int but, Mousectl *mc)
{
    Rectangle r, rc;

    but = 1<<(but-1);
    setcursor(mc, &sweep);
    while(mc->buttons)
        readmouse(mc);
    while(!(mc->buttons & but)){
        readmouse(mc);
        if(mc->buttons & (7^but))
            goto Return;
    }
    r.min = mc->xy;
    r.max = mc->xy;
    do{
        rc = canonrect(r);
        drawgetrect(rc, 1);
        readmouse(mc);
        drawgetrect(rc, 0);
        r.max = mc->xy;
    }while(mc->buttons == but);

    Return:
    setcursor(mc, nil);
    if(mc->buttons & (7^but)){
        rc.min.x = rc.max.x = 0;
        rc.min.y = rc.max.y = 0;
        while(mc->buttons)
            readmouse(mc);
    }
    return rc;
}
```

⟨*function* drawgetrect 282b⟩≡ (418b)

```
void
drawgetrect(Rectangle rc, int up)
{
    int i;
    Rectangle r, rects[4];

    /*
     * BUG: if for some reason we have two of these going on at once
     * when we must grow the tmp buffers, we lose data.  Also if tmp
     * is unallocated and we ask to restore the screen, it would be nice
     * to complain, but we silently make a mess.
     */
    if(up && tmp[0]!=nil)
        if(Dx(tmp[0]->r)<Dx(rc) || Dy(tmp[2]->r)<Dy(rc))
            freetmp();
    if(tmp[0] == 0){
        r = Rect(0, 0, max(Dx(display->screenimage->r), Dx(rc)), W);
        tmp[0] = allocimage(display, r, view->chan, false, -1);
        tmp[1] = allocimage(display, r, view->chan, false, -1);
        r = Rect(0, 0, W, max(Dy(display->screenimage->r), Dy(rc)));
        tmp[2] = allocimage(display, r, view->chan, false, -1);
        tmp[3] = allocimage(display, r, view->chan, false, -1);
        red = allocimage(display, Rect(0,0,1,1), view->chan, true, DRed);
```

```
        if(tmp[0]==0 || tmp[1]==0 || tmp[2]==0 || tmp[3]==0 || red==0){
            freetmp();
            drawerror(display, "getrect: allocimage failed");
        }
    }
    brects(rc, rects);
    if(!up){
        for(i=0; i<4; i++)
            draw(view, rects[i], tmp[i], nil, ZP);
        return;
    }
    for(i=0; i<4; i++){
        draw(tmp[i], Rect(0, 0, Dx(rects[i]), Dy(rects[i])), view, nil, rects[i].min);
        draw(view, rects[i], red, nil, ZP);
    }
}
```

⟨*function* freetmp*(*(lib_graphics/libdraw/getrect.c)*)* 283a⟩≡ (418b)

```
static
void
freetmp(void)
{
    freeimage(tmp[0]);
    freeimage(tmp[1]);
    freeimage(tmp[2]);
    freeimage(tmp[3]);
    freeimage(red);
    tmp[0] = tmp[1] = tmp[2] = tmp[3] = red = nil;
}
```

⟨*function* max 283b⟩≡ (418b)

```
static
int
max(int a, int b)
{
    if(a > b)
        return a;
    return b;
}
```

15.6 Event-based IO

15.6.1 Data structures

⟨*enum* Key 283c⟩≡ (397c)

```
enum
{
    Emouse = 1,
    Ekeyboard = 2,
};
```

⟨*type* keys 283d⟩≡ (397c)

```
// bitset<Key>
typedef ulong keys;
```

⟨*struct* Event 284a⟩≡ (397c)

```
struct Event
{
    int  kbdc;
    Mouse mouse;

    int  n;  /* number of characters in message */
    void *v;  /* data unpacked by general event-handling function */
    byte data[EMAXMSG]; /* message from an arbitrary file descriptor */
};
```

⟨*struct Mouse (include/event.h)* 284b⟩≡ (397c)

```
struct Mouse
{
    int  buttons; /* bit array: LMR=124 */
    Point xy;
    ulong msec;
};
```

⟨*struct Menu (include/event.h)* 284c⟩≡ (397c)

```
struct Menu
{
    char **item;
    char *(*gen)(int);
    int lasthit;
};
```

⟨*enum* _anon_ (include/event.h)2 284d⟩≡ (397c)

```
enum
{
    MAXSLAVE = 32,
    EMAXMSG = 128+8192, /* size of 9p header+data */
};
```

⟨*struct* Slave 284e⟩≡ (419)

```
struct Slave
{
    int pid;

    Ebuf *head;  /* queue of messages for this descriptor */
    Ebuf *tail;

    int (*fn)(int, Event*, uchar*, int);
};
```

⟨*struct* Ebuf 284f⟩≡ (419)

```
struct Ebuf
{
    // Extra
    Ebuf *next;

    int n;  /* number of bytes in buf */
    // this must be the last field
    byte buf[EMAXMSG];

};
```

⟨*global* eslave 284g⟩≡ (419)

```
static Slave eslave[MAXSLAVE];
```

⟨*global* nslave 285a⟩≡ (419)
```
static int nslave;
```

⟨*global* Skeyboard 285b⟩≡ (419)
```
static int Skeyboard = -1;
```

⟨*global* Smouse 285c⟩≡ (419)
```
static int Smouse = -1;
```

⟨*global* Stimer 285d⟩≡ (419)
```
static int Stimer = -1;
```

⟨*global* parentpid 285e⟩≡ (419)
```
static int parentpid;
```

⟨*global* logfid 285f⟩≡ (419)
```
static fdt logfid;
```

⟨*global* epipe 285g⟩≡ (419)
```
static fdt epipe[2];
```

⟨*global* mousefd 285h⟩≡ (419)
```
static fdt mousefd;
```

⟨*global* cursorfd 285i⟩≡ (419)
```
static fdt cursorfd;
```

15.6.2 Initialization: einit()

⟨*function* einit 285j⟩≡ (419)
```
void
einit(keys keys)
{
    fdt ctl, fd;
    char buf[256];

    parentpid = getpid();
    if(pipe(epipe) < 0)
        drawerror(display, "events: einit pipe");

    atexit(ekill);
    atnotify(enote, 1);

    snprint(buf, sizeof buf, "%s/mouse", display->devdir);
    mousefd = open(buf, ORDWR|OCEXEC);
    if(mousefd < 0)
        drawerror(display, "einit: can't open mouse\n");

    snprint(buf, sizeof buf, "%s/cursor", display->devdir);
    cursorfd = open(buf, ORDWR|OCEXEC);
    if(cursorfd < 0)
        drawerror(display, "einit: can't open cursor\n");

    if(keys&Ekeyboard){
        snprint(buf, sizeof buf, "%s/cons", display->devdir);
        fd = open(buf, OREAD);
        if(fd < 0)
            drawerror(display, "events: can't open console");
```

```
        snprint(buf, sizeof buf, "%s/consctl", display->devdir);
        ctl = open("/dev/consctl", OWRITE|OCEXEC);
        if(ctl < 0)
            drawerror(display, "events: can't open consctl");
        write(ctl, "rawon", 5);

        for(Skeyboard=0; Ekeyboard & ~(1<<Skeyboard); Skeyboard++)
            ;
        ekeyslave(fd);
    }
    if(keys&Emouse){
        estart(Emouse, mousefd, 1+4*12);
        for(Smouse=0; Emouse & ~(1<<Smouse); Smouse++)
            ;
    }
}
```

⟨*function* ekill 286a⟩≡ (419)

```
static void
ekill(void)
{
    enote(0, 0);
}
```

⟨*function* enote 286b⟩≡ (419)

```
static int
enote(void *v, char *s)
{
    char t[1];
    int i, pid;

    USED(v, s);
    pid = getpid();
    if(pid != parentpid){
        for(i=0; i<nslave; i++){
            if(pid == eslave[i].pid){
                t[0] = MAXSLAVE;
                write(epipe[1], t, 1);
                break;
            }
        }
        return 0;
    }
    close(epipe[0]);
    epipe[0] = -1;
    close(epipe[1]);
    epipe[1] = -1;
    for(i=0; i<nslave; i++){
        if(pid == eslave[i].pid)
            continue; /* don't kill myself */
        postnote(PNPROC, eslave[i].pid, "die");
    }
    return 0;
}
```

15.6.3 The slaves

⟨*function* ekeyslave 286c⟩≡ (419)

```
static void
```

```
ekeyslave(fdt fd)
{
    Rune r;
    char t[3], k[10];
    int kr, kn, w;

    if(eforkslave(Ekeyboard) < MAXSLAVE)
        // parent (master)
        return;

    // child (slave) code
    kn = 0;
    t[0] = Skeyboard;
    for(;;){
        while(!fullrune(k, kn)){
            // blocking call
            kr = read(fd, k+kn, sizeof k - kn);
            if(kr <= 0)
                goto breakout;
            kn += kr;
        }
        w = chartorune(&r, k);
        kn -= w;
        memmove(k, &k[w], kn);
        t[1] = r;
        t[2] = r>>8; // TODO what about other parts of the Rune????
        // send to master
        if(write(epipe[1], t, 3) != 3)
            break;
    }
breakout:;
    t[0] = MAXSLAVE;
    write(epipe[1], t, 1);
    _exits(0);
}
```

⟨*function* eforkslave 287⟩≡ (419)

```
static int
eforkslave(ulong key)
{
    int i, pid;

    for(i=0; i<MAXSLAVE; i++)
        if((key & ~(1<<i)) == 0 && eslave[i].pid == 0){
            if(nslave <= i)
                nslave = i + 1;
            /*
             * share the file descriptors so the last child
             * out closes all connections to the window server.
             */
            switch(pid = rfork(RFPROC)){
            // child
            case 0:
                return MAXSLAVE+i;
            case -1:
                fprint(2, "events: fork error\n");
                exits("fork");
            }
            // parent
            eslave[i].pid = pid;
```

```
            eslave[i].head = eslave[i].tail = nil;
            return i;
        }
    drawerror(display, "events: bad slave assignment");
    return 0;
}
```

⟨*function* estart 288a⟩≡ (419)

```
ulong
estart(ulong key, fdt fd, int n)
{
    return estartfn(key, fd, n, nil);
}
```

⟨*function* estartfn 288b⟩≡ (419)

```
ulong
estartfn(ulong key, fdt fd, int n, int (*fn)(int, Event*, uchar*, int))
{
    char buf[EMAXMSG+1];
    int i, r;

    if(fd < 0)
        drawerror(display, "events: bad file descriptor");
    if(n <= 0 || n > EMAXMSG)
        n = EMAXMSG;

    i = eforkslave(key);
    // master
    if(i < MAXSLAVE){
        eslave[i].fn = fn;
        return 1<<i;
    }
    // slave
    buf[0] = i - MAXSLAVE; // slave identifier
    while((r = read(fd, buf+1, n))>0)
        if(write(epipe[1], buf, r+1)!=r+1)
            break;

    buf[0] = MAXSLAVE;
    write(epipe[1], buf, 1);
    _exits(nil);
    return 0;
}
```

15.6.4 Event loop: `event()`

⟨*function* event 288c⟩≡ (419)

```
keys
event(Event *e)
{
    return eread(~0UL, e);
}
```

⟨*function* eread 288d⟩≡ (419)

```
keys
eread(keys keys, Event *e)
{
    Ebuf *eb;
    int i, id;
```

```
    if(keys == 0)
        return 0;
    for(;;){
        // check if pending message
        for(i=0; i<nslave; i++) {
            if((keys & (1<<i)) && eslave[i].head){
                id = 1<<i;
                if(i == Smouse)
                    e->mouse = emouse();
                else if(i == Skeyboard)
                    e->kbdc = ekbd();
                else if(i == Stimer)
                    eslave[i].head = nil;
                else{
                    eb = ebread(&eslave[i]);
                    e->n = eb->n;
                    if(eslave[i].fn)
                        id = (*eslave[i].fn)(id, e, eb->buf, eb->n);
                    else
                        memmove(e->data, eb->buf, eb->n);
                    free(eb);
                }
                return id;
            }
        }
        // read from pipe new messages from the slaves
        extract();
    }
    return 0; // unreachable
}
```

⟨*function* emouse 289a⟩≡ (419)

```
Mouse
emouse(void)
{
    Mouse m;
    Ebuf *eb;
    static int but[2];
    int b;

    if(Smouse < 0)
        drawerror(display, "events: mouse not initialized");
    eb = ebread(&eslave[Smouse]);
    m.xy.x = atoi((char*)eb->buf+1+0*12);
    m.xy.y = atoi((char*)eb->buf+1+1*12);
    b = atoi((char*)eb->buf+1+2*12);
    m.buttons = b;
    m.msec = atoi((char*)eb->buf+1+3*12);
    if (logfid)
        fprint(logfid, "b: %d xy: %P\n", m.buttons, m.xy);
    free(eb);
    return m;
}
```

⟨*function* ekbd 289b⟩≡ (419)

```
int
ekbd(void)
{
    Ebuf *eb;
```

```
    int c;

    if(Skeyboard < 0)
        drawerror(display, "events: keyboard not initialzed");
    eb = ebread(&eslave[Skeyboard]);
    c = eb->buf[0] + (eb->buf[1]<<8);
    free(eb);
    return c;
}
```

⟨*function* ebread 290a⟩≡ (419)

```
static
Ebuf*
ebread(Slave *s)
{
    Ebuf *eb;
    Dir *d;
    ulong l;

    for(;;){
        d = dirfstat(epipe[0]);
        if(d == nil)
            drawerror(display, "events: eread stat error");
        l = d->length;
        free(d);
        if(s->head && l==0)
            break;
        // else
        extract();
    }
    // eb = pop_queue(s->head, s->tail);
    eb = s->head;
    s->head = s->head->next;
    if(s->head == nil)
        s->tail = nil;

    return eb;
}
```

15.6.5 The master: extract()

⟨*function* extract 290b⟩≡ (419)

```
static void
extract(void)
{
    Slave *s;
    Ebuf *eb;
    int i, n;
    byte ebuf[EMAXMSG+1];

    /* avoid generating a message if there's nothing to show. */
    /* this test isn't perfect, though; could do flushimage(display, 0) then call extract */
    /* also: make sure we don't interfere if we're multiprocessing the display */
    if(display->locking){
        /* if locking is being done by program, this means it can't depend on automatic flush in emouse() etc.
        if(canqlock(&display->qlock)){
            if(display->bufp > display->buf)
                flushimage(display, true);
            unlockdisplay(display);
```

```
        }
    }else
        if(display->bufp > display->buf)
            flushimage(display, true);

loop:
    if((n=read(epipe[0], ebuf, EMAXMSG+1)) < 0
    || ebuf[0] >= MAXSLAVE)
        drawerror(display, "eof on event pipe");
    if(n == 0)
        goto loop;

    i = ebuf[0];
    if(i >= nslave || n <= 1)
        drawerror(display, "events: protocol error: short read");
    s = &eslave[i];

    if(i == Stimer){
        s->head = (Ebuf *)1;
        return;
    }
    if(i == Skeyboard && n != 3)
        drawerror(display, "events: protocol error: keyboard");
    if(i == Smouse){
        if(n < 1+1+2*12)
            drawerror(display, "events: protocol error: mouse");
        if(ebuf[1] == 'r')
            eresized(1);

        /* squash extraneous mouse events */
        if((eb=s->tail) && memcmp(eb->buf+1+2*12, ebuf+1+1+2*12, 12)==0){
            memmove(eb->buf, &ebuf[1], n - 1);
            return;
        }
    }

    /* try to save space by only allocating as much buffer as we need */
    eb = malloc(sizeof(*eb) - sizeof(eb->buf) + n - 1);
    if(eb == nil)
        drawerror(display, "events: protocol error 4");
    eb->n = n - 1;
    memmove(eb->buf, &ebuf[1], n - 1);

    // add_queue(eb, s->head, s->tail)
    eb->next = nil;
    if(s->head)
        s->tail = s->tail->next = eb;
    else
        s->head = s->tail = eb;
}
```

15.6.6 ecanxxx()

⟨*function* ecanmouse 291⟩≡ (419)

```
int
ecanmouse(void)
{
    if(Smouse < 0)
        drawerror(display, "events: mouse not initialized");
```

```
    return ecanread(Emouse);
}
```

⟨*function* ecankbd 292a⟩≡ (419)

```
int
ecankbd(void)
{
    if(Skeyboard < 0)
        drawerror(display, "events: keyboard not initialzed");
    return ecanread(Ekeyboard);
}
```

⟨*function* ecanread 292b⟩≡ (419)

```
int
ecanread(ulong keys)
{
    Dir *d;
    int i;
    ulong l;

    for(;;){
        for(i=0; i<nslave; i++)
            if((keys & (1<<i)) && eslave[i].head)
                return 1;
        d = dirfstat(epipe[0]);
        if(d == nil)
            drawerror(display, "events: ecanread stat error");
        l = d->length;
        free(d);
        if(l == 0)
            return 0;
        extract();
    }
    return 0; // unreachable
}
```

15.6.7 Timer

⟨*function* etimer 292c⟩≡ (419)

```
ulong
etimer(ulong key, int n)
{
    char t[2];
    int cnt;

    if(Stimer != -1)
        drawerror(display, "events: timer started twice");
    Stimer = eforkslave(key);
    // master
    if(Stimer < MAXSLAVE)
        return 1<<Stimer;
    // slave
    if(n <= 0)
        n = 1000;
    t[0] = t[1] = Stimer - MAXSLAVE;
    do {
        sleep(n);
        cnt = write(epipe[1], t, 2);
    } while(cnt == 2);
```

```
    t[0] = MAXSLAVE;
    write(epipe[1], t, 1);
    _exits(nil);
    return 0;
}
```

15.6.8 Misc

⟨*function* emoveto 293a⟩≡ (419)

```
void
emoveto(Point pt)
{
    char buf[2*12+2];
    int n;

    n = sprint(buf, "m%d %d", pt.x, pt.y);
    write(mousefd, buf, n);
}
```

⟨*function* esetcursor 293b⟩≡ (419)

```
void
esetcursor(Cursor *c)
{
    uchar curs[2*4+2*2*16];

    if(c == 0)
        write(cursorfd, curs, 0);
    else{
        BPLONG(curs+0*4, c->offset.x);
        BPLONG(curs+1*4, c->offset.y);
        memmove(curs+2*4, c->clr, 2*2*16);
        write(cursorfd, curs, sizeof curs);
    }
}
```

⟨*function* ereadmouse 293c⟩≡ (419)

```
int
ereadmouse(Mouse *m)
{
    int n;
    char buf[128];

    do{
        n = read(mousefd, buf, sizeof(buf));
        if(n < 0) /* probably interrupted */
            return -1;
        n = eatomouse(m, buf, n);
    }while(n == 0);
    return n;
}
```

⟨*function* eatomouse 293d⟩≡ (419)

```
int
eatomouse(Mouse *m, char *buf, int n)
{
    if(n != 1+4*12){
        werrstr("atomouse: bad count");
        return -1;
```

```
    }

    if(buf[0] == 'r')
        eresized(1);
    m->xy.x = atoi(buf+1+0*12);
    m->xy.y = atoi(buf+1+1*12);
    m->buttons = atoi(buf+1+2*12);
    m->msec = atoi(buf+1+3*12);
    return n;
}
```

15.6.9 egetrects.c

⟨*constant* W 294a⟩≡ (420)

```
#define W Borderwidth
```

⟨*global* tmp 294b⟩≡ (420)

```
static Image *tmp[4];
```

⟨*global* red 294c⟩≡ (420)

```
static Image *red;
```

⟨*global* sweep 294d⟩≡ (420)

```
static Cursor sweep={
    {-7, -7},
    {0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xE0, 0x07,
     0xE0, 0x07, 0xE0, 0x07, 0xE3, 0xF7, 0xE3, 0xF7,
     0xE3, 0xE7, 0xE3, 0xF7, 0xE3, 0xFF, 0xE3, 0x7F,
     0xE0, 0x3F, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF,},
    {0x00, 0x00, 0x7F, 0xFE, 0x40, 0x02, 0x40, 0x02,
     0x40, 0x02, 0x40, 0x02, 0x40, 0x02, 0x41, 0xE2,
     0x41, 0xC2, 0x41, 0xE2, 0x41, 0x72, 0x40, 0x38,
     0x40, 0x1C, 0x40, 0x0E, 0x7F, 0xE6, 0x00, 0x00,}
};
```

⟨*function* brects 294e⟩≡ (420)

```
static
void
brects(Rectangle r, Rectangle rp[4])
{
    if(Dx(r) < 2*W)
        r.max.x = r.min.x+2*W;
    if(Dy(r) < 2*W)
        r.max.y = r.min.y+2*W;
    rp[0] = Rect(r.min.x, r.min.y, r.max.x, r.min.y+W);
    rp[1] = Rect(r.min.x, r.max.y-W, r.max.x, r.max.y);
    rp[2] = Rect(r.min.x, r.min.y+W, r.min.x+W, r.max.y-W);
    rp[3] = Rect(r.max.x-W, r.min.y+W, r.max.x, r.max.y-W);
}
```

⟨*function* egetrect 294f⟩≡ (420)

```
Rectangle
egetrect(int but, Mouse *m)
{
    Rectangle r, rc;

    but = 1<<(but-1);
    esetcursor(&sweep);
    while(m->buttons)
```

```
        *m = emouse();
    while(!(m->buttons & but)){
        *m = emouse();
        if(m->buttons & (7^but))
            goto Return;
    }
    r.min = m->xy;
    r.max = m->xy;
    do{
        rc = canonrect(r);
        edrawgetrect(rc, 1);
        *m = emouse();
        edrawgetrect(rc, 0);
        r.max = m->xy;
    }while(m->buttons == but);

    Return:
    esetcursor(0);
    if(m->buttons & (7^but)){
        rc.min.x = rc.max.x = 0;
        rc.min.y = rc.max.y = 0;
        while(m->buttons)
            *m = emouse();
    }
    return rc;
}
```

⟨*function* freetmp 295a⟩≡ (420)

```
static
void
freetmp(void)
{
    freeimage(tmp[0]);
    freeimage(tmp[1]);
    freeimage(tmp[2]);
    freeimage(tmp[3]);
    freeimage(red);
    tmp[0] = tmp[1] = tmp[2] = tmp[3] = red = nil;
}
```

⟨*function* edrawgetrect 295b⟩≡ (420)

```
void
edrawgetrect(Rectangle rc, int up)
{
    int i;
    Rectangle r, rects[4];

    if(up && tmp[0]!=nil)
        if(Dx(tmp[0]->r)<Dx(rc) || Dy(tmp[2]->r)<Dy(rc))
            freetmp();

    if(tmp[0] == 0){
        r = Rect(0, 0, Dx(view->r), W);
        tmp[0] = allocimage(display, r, view->chan, 0, -1);
        tmp[1] = allocimage(display, r, view->chan, 0, -1);
        r = Rect(0, 0, W, Dy(view->r));
        tmp[2] = allocimage(display, r, view->chan, 0, -1);
        tmp[3] = allocimage(display, r, view->chan, 0, -1);
        red = allocimage(display, Rect(0,0,1,1), view->chan, 1, DRed);
        if(tmp[0]==0 || tmp[1]==0 || tmp[2]==0 || tmp[3]==0 || red==0)
```

```
            drawerror(display, "getrect: allocimage failed");
        }
        brects(rc, rects);
        if(!up){
            for(i=0; i<4; i++)
                draw(view, rects[i], tmp[i], nil, ZP);
            return;
        }
        for(i=0; i<4; i++){
            draw(tmp[i], Rect(0, 0, Dx(rects[i]), Dy(rects[i])), view, nil, rects[i].min);
            draw(view, rects[i], red, nil, ZP);
        }
    }
```

15.6.10 emenuhits.c

⟨*enum* _anon_*(*(lib_graphics/libdraw/emenuhit.c)*)* 296a⟩≡ (421a)

```
    enum
    {
        Margin = 4,  /* outside to text */
        Border = 2,  /* outside to selection boxes */
        Blackborder = 2, /* width of outlining border */
        Vspacing = 2,  /* extra spacing between lines of text */
        Maxunscroll = 25, /* maximum #entries before scrolling turns on */
        Nscroll = 20,  /* number entries in scrolling part */
        Scrollwid = 14,  /* width of scroll bar */
        Gap = 4,   /* between text and scroll bar */
    };
```

⟨*global* menutxt 296b⟩≡ (421a)

```
    static Image *menutxt;
```

⟨*global* back 296c⟩≡ (421a)

```
    static Image *back;
```

⟨*global* high 296d⟩≡ (421a)

```
    static Image *high;
```

⟨*global* bord 296e⟩≡ (421a)

```
    static Image *bord;
```

⟨*global* text 296f⟩≡ (421a)

```
    static Image *text;
```

⟨*global* htext 296g⟩≡ (421a)

```
    static Image *htext;
```

⟨*function* menucolors 296h⟩≡ (421a)

```
    static
    void
    menucolors(void)
    {
        /* Main tone is greenish, with negative selection */
        back = allocimagemix(display, DPalegreen, DWhite);
        high = allocimage(display, Rect(0,0,1,1), CMAP8, 1, DDarkgreen); /* dark green */
        bord = allocimage(display, Rect(0,0,1,1), CMAP8, 1, DMedgreen); /* not as dark green */
        if(back == nil || high == nil || bord == nil)
            goto Error;
        text = display->black;
```

```
    htext = back;
    return;

    Error:
    freeimage(back);
    freeimage(high);
    freeimage(bord);
    back = display->white;
    high = display->black;
    bord = display->black;
    text = display->black;
    htext = display->white;
}
```

⟨*function* menurect 297a⟩≡ (421a)

```
/*
 * r is a rectangle holding the text elements.
 * return the rectangle, including its black edge, holding element i.
 */
static Rectangle
menurect(Rectangle r, int i)
{
    if(i < 0)
        return Rect(0, 0, 0, 0);
    r.min.y += (font->height+Vspacing)*i;
    r.max.y = r.min.y+font->height+Vspacing;
    return insetrect(r, Border-Margin);
}
```

⟨*function* menusel 297b⟩≡ (421a)

```
/*
 * r is a rectangle holding the text elements.
 * return the element number containing p.
 */
static int
menusel(Rectangle r, Point p)
{
    r = insetrect(r, Margin);
    if(!ptinrect(p, r))
        return -1;
    return (p.y-r.min.y)/(font->height+Vspacing);
}
```

⟨*function* paintitem 297c⟩≡ (421a)

```
static
void
paintitem(Menu *menu, Rectangle textr, int off, int i, int highlight, Image *save, Image *restore)
{
    char *item;
    Rectangle r;
    Point pt;

    if(i < 0)
        return;
    r = menurect(textr, i);
    if(restore){
        draw(view, r, restore, nil, restore->r.min);
        return;
    }
    if(save)
```

```
        draw(save, save->r, view, nil, r.min);
    item = menu->item? menu->item[i+off] : (*menu->gen)(i+off);
    pt.x = (textr.min.x+textr.max.x-stringwidth(font, item))/2;
    pt.y = textr.min.y+i*(font->height+Vspacing);
    draw(view, r, highlight? high : back, nil, pt);
    string(view, pt, highlight? htext : text, pt, font, item);
}
```

⟨*function* menuscan 298a⟩≡ (421a)

```
/*
 * menur is a rectangle holding all the highlightable text elements.
 * track mouse while inside the box, return what's selected when button
 * is raised, -1 as soon as it leaves box.
 * invariant: nothing is highlighted on entry or exit.
 */
static int
menuscan(Menu *menu, int but, Mouse *m, Rectangle textr, int off, int lasti, Image *save)
{
    int i;

    paintitem(menu, textr, off, lasti, 1, save, nil);
    flushimage(display, true); /* in case display->locking is set */
    *m = emouse();
    while(m->buttons & (1<<(but-1))){
        flushimage(display, true); /* in case display->locking is set */
        *m = emouse();
        i = menusel(textr, m->xy);
        if(i != -1 && i == lasti)
            continue;
        paintitem(menu, textr, off, lasti, 0, nil, save);
        if(i == -1)
            return i;
        lasti = i;
        paintitem(menu, textr, off, lasti, 1, save, nil);
    }
    return lasti;
}
```

⟨*function* menupaint 298b⟩≡ (421a)

```
static void
menupaint(Menu *menu, Rectangle textr, int off, int nitemdrawn)
{
    int i;

    draw(view, insetrect(textr, Border-Margin), back, nil, ZP);
    for(i = 0; i<nitemdrawn; i++)
        paintitem(menu, textr, off, i, 0, nil, nil);
}
```

⟨*function* menuscrollpaint 298c⟩≡ (421a)

```
static void
menuscrollpaint(Rectangle scrollr, int off, int nitem, int nitemdrawn)
{
    Rectangle r;

    draw(view, scrollr, back, nil, ZP);
    r.min.x = scrollr.min.x;
    r.max.x = scrollr.max.x;
    r.min.y = scrollr.min.y + (Dy(scrollr)*off)/nitem;
    r.max.y = scrollr.min.y + (Dy(scrollr)*(off+nitemdrawn))/nitem;
```

```
    if(r.max.y < r.min.y+2)
        r.max.y = r.min.y+2;
    border(view, r, 1, bord, ZP);
    if(menutxt == 0)
        menutxt = allocimage(display, Rect(0, 0, 1, 1), CMAP8, 1, DDarkgreen);
    if(menutxt)
        draw(view, insetrect(r, 1), menutxt, nil, ZP);
}
```

⟨*function* emenuhit 299⟩≡ (421a)

```
int
emenuhit(int but, Mouse *m, Menu *menu)
{
    int i, nitem, nitemdrawn, maxwid, lasti, off, noff, wid, screenitem;
    int scrolling;
    Rectangle r, menur, sc, textr, scrollr;
    Image *b, *save;
    Point pt;
    char *item;

    if(back == nil)
        menucolors();
    sc = view->clipr;
    replclipr(view, 0, view->r);
    maxwid = 0;
    for(nitem = 0;
        item = menu->item? menu->item[nitem] : (*menu->gen)(nitem);
        nitem++){
        i = stringwidth(font, item);
        if(i > maxwid)
            maxwid = i;
    }
    if(menu->lasthit<0 || menu->lasthit>=nitem)
        menu->lasthit = 0;
    screenitem = (Dy(view->r)-10)/(font->height+Vspacing);
    if(nitem>Maxunscroll || nitem>screenitem){
        scrolling = 1;
        nitemdrawn = Nscroll;
        if(nitemdrawn > screenitem)
            nitemdrawn = screenitem;
        wid = maxwid + Gap + Scrollwid;
        off = menu->lasthit - nitemdrawn/2;
        if(off < 0)
            off = 0;
        if(off > nitem-nitemdrawn)
            off = nitem-nitemdrawn;
        lasti = menu->lasthit-off;
    }else{
        scrolling = 0;
        nitemdrawn = nitem;
        wid = maxwid;
        off = 0;
        lasti = menu->lasthit;
    }
    r = insetrect(Rect(0, 0, wid, nitemdrawn*(font->height+Vspacing)), -Margin);
    r = rectsubpt(r, Pt(wid/2, lasti*(font->height+Vspacing)+font->height/2));
    r = rectaddpt(r, m->xy);
    pt = ZP;
    if(r.max.x>view->r.max.x)
        pt.x = view->r.max.x-r.max.x;
```

```
if(r.max.y>view->r.max.y)
    pt.y = view->r.max.y-r.max.y;
if(r.min.x<view->r.min.x)
    pt.x = view->r.min.x-r.min.x;
if(r.min.y<view->r.min.y)
    pt.y = view->r.min.y-r.min.y;
menur = rectaddpt(r, pt);
textr.max.x = menur.max.x-Margin;
textr.min.x = textr.max.x-maxwid;
textr.min.y = menur.min.y+Margin;
textr.max.y = textr.min.y + nitemdrawn*(font->height+Vspacing);
if(scrolling){
    scrollr = insetrect(menur, Border);
    scrollr.max.x = scrollr.min.x+Scrollwid;
}else
    scrollr = Rect(0, 0, 0, 0);

b = allocimage(display, menur, view->chan, 0, 0);
if(b == 0)
    b = view;
draw(b, menur, view, nil, menur.min);
draw(view, menur, back, nil, ZP);
border(view, menur, Blackborder, bord, ZP);
save = allocimage(display, menurect(textr, 0), view->chan, 0, -1);
r = menurect(textr, lasti);
emoveto(divpt(addpt(r.min, r.max), 2));
menupaint(menu, textr, off, nitemdrawn);
if(scrolling)
    menuscrollpaint(scrollr, off, nitem, nitemdrawn);
while(m->buttons & (1<<(but-1))){
    lasti = menuscan(menu, but, m, textr, off, lasti, save);
    if(lasti >= 0)
        break;
    while(!ptinrect(m->xy, textr) && (m->buttons & (1<<(but-1)))){
        if(scrolling && ptinrect(m->xy, scrollr)){
            noff = ((m->xy.y-scrollr.min.y)*nitem)/Dy(scrollr);
            noff -= nitemdrawn/2;
            if(noff < 0)
                noff = 0;
            if(noff > nitem-nitemdrawn)
                noff = nitem-nitemdrawn;
            if(noff != off){
                off = noff;
                menupaint(menu, textr, off, nitemdrawn);
                menuscrollpaint(scrollr, off, nitem, nitemdrawn);
            }
        }
        flushimage(display, true); /* in case display->locking is set */
        *m = emouse();
    }
}
draw(view, menur, b, nil, menur.min);
if(b != view)
    freeimage(b);
freeimage(save);
replclipr(view, 0, sc);
if(lasti >= 0){
    menu->lasthit = lasti+off;
    return menu->lasthit;
}
```

```
    return -1;
}
```

15.7 Graphical text mode

⟨*global* curpos 301a⟩≡ (475)

```
Point     swconsole_curpos;
```

⟨*global window bis* 301b⟩≡ (475)

```
Rectangle swconsole_window;
```

⟨*global* conscol 301c⟩≡ (475)

```
Memimage *swconsole_conscol;
```

⟨*global* back2 301d⟩≡ (475)

```
Memimage *swconsole_back;
```

⟨*global* memdefont2 301e⟩≡ (475)

```
Memsubfont *swconsole_memdefont;
```

⟨*function* swconsole_init 301f⟩≡ (475)

```
void swconsole_init(void)
{
    Rectangle r;

    swconsole_back = memwhite;
    swconsole_conscol = memblack;

    swconsole_memdefont = getmemdefont();
    w = swconsole_memdefont->info[' '].width;
    h = swconsole_memdefont->height;

    r = insetrect(gscreen->r, 4);
    memimagedraw(gscreen, r, memblack, ZP, memopaque, ZP, S);
    swconsole_window = insetrect(r, 4);
    memimagedraw(gscreen, swconsole_window, memwhite, ZP, memopaque, ZP, S);
}
```

⟨*global swconsole h w* 301g⟩≡ (475)

```
static int h, w;
```

⟨*global* swconsole_screenlock 301h⟩≡ (475)

```
Lock      swconsole_screenlock;
```

⟨*function* swconsole_scroll 301i⟩≡ (475)

```
void
swconsole_scroll(void)
{
    int o;
    Point p;
    Rectangle r;

    o = Scroll*h;
    r = Rpt(swconsole_window.min,
            Pt(swconsole_window.max.x, swconsole_window.max.y-o));
    p = Pt(swconsole_window.min.x, swconsole_window.min.y+o);
    memimagedraw(gscreen, r, gscreen, p, nil, p, S);
    arch_flushmemscreen(r);
    r = Rpt(Pt(swconsole_window.min.x, swconsole_window.max.y-o),
```

```
            swconsole_window.max);
    memimagedraw(gscreen, r, swconsole_back, ZP, nil, ZP, S);
    arch_flushmemscreen(r);

    swconsole_curpos.y -= o;
}
```

⟨*function* swconsole_screenputc 302⟩≡ (475)

```
void
swconsole_screenputc(char *buf)
{
    int w;
    uint pos;
    Point p;
    Rectangle r;
    static int *xp;
    static int xbuf[256];

    if (xp < xbuf || xp >= &xbuf[sizeof(xbuf)])
        xp = xbuf;

    switch (buf[0]) {
    case '\n':
        if (swconsole_curpos.y + h >= swconsole_window.max.y)
            swconsole_scroll();
        swconsole_curpos.y += h;
        swconsole_screenputc("\r");
        break;
    case '\r':
        xp = xbuf;
        swconsole_curpos.x = swconsole_window.min.x;
        break;
    case '\t':
        p = memsubfontwidth(swconsole_memdefont, " ");
        w = p.x;
        if (swconsole_curpos.x >= swconsole_window.max.x - Tabstop * w)
            swconsole_screenputc("\n");

        pos = (swconsole_curpos.x - swconsole_window.min.x) / w;
        pos = Tabstop - pos % Tabstop;
        *xp++ = swconsole_curpos.x;
        r = Rect(swconsole_curpos.x, swconsole_curpos.y,
                 swconsole_curpos.x + pos * w, swconsole_curpos.y + h);
        memimagedraw(gscreen, r, swconsole_back,
                     swconsole_back->r.min, nil, swconsole_back->r.min, S);
        arch_flushmemscreen(r);
        swconsole_curpos.x += pos * w;
        break;
    case '\b':
        if (xp <= xbuf)
            break;
        xp--;
        r = Rect(*xp, swconsole_curpos.y,
                 swconsole_curpos.x, swconsole_curpos.y + h);
        memimagedraw(gscreen, r, swconsole_back,
                     swconsole_back->r.min, nil, swconsole_back->r.min, S);
        arch_flushmemscreen(r);
        swconsole_curpos.x = *xp;
        break;
    case '\0':
```

```
        break;
    default:
        p = memsubfontwidth(swconsole_memdefont, buf);
        w = p.x;

        if (swconsole_curpos.x >= swconsole_window.max.x - w)
            swconsole_screenputc("\n");

        *xp++ = swconsole_curpos.x;
        r = Rect(swconsole_curpos.x, swconsole_curpos.y,
                 swconsole_curpos.x + w, swconsole_curpos.y + h);
        memimagedraw(gscreen, r, swconsole_back,
                     swconsole_back->r.min, nil, swconsole_back->r.min, S);
        memimagestring(gscreen, swconsole_curpos, swconsole_conscol, ZP,
                       swconsole_memdefont, buf);
        arch_flushmemscreen(r);
        swconsole_curpos.x += w;
        break;
    }
}
```

⟨*function* swconsole_screenputs 303a⟩≡ (475)

```
void
swconsole_screenputs(char *s, int n)
{
    int i;
    Rune r;
    char buf[4];

    if(!arch_islo()) {
        /* don't deadlock trying to print in interrupt */
        if(!canlock(&swconsole_screenlock))
            return;
    }
    else
        lock(&swconsole_screenlock);

    while(n > 0){
        i = chartorune(&r, s);
        if(i == 0){
            s++;
            --n;
            continue;
        }
        memmove(buf, s, i);
        buf[i] = 0;
        n -= i;
        s += i;
        swconsole_screenputc(buf);
    }
    unlock(&swconsole_screenlock);
}
```

15.8 Blank screen

⟨vgactl *cases(x86)* 303b⟩≡ (369) 304a ▷

```
case CMblank:
    drawblankscreen(true);
    return;
```

⟨vgactl *cases(x86)* 304a⟩+≡ (369) ◁303b 304f▷

```
case CMunblank:
    drawblankscreen(false);
    return;
```

⟨KDraw *other fields* 304b⟩+≡ (36a) ◁88b 304d▷

```
bool blanked;    /* screen turned off */
```

⟨*function* drawblankscreen 304c⟩≡ (480b)

```
void
drawblankscreen(bool blank)
{
    int i, nc;
    ulong *p;

    if(blank == sdraw.blanked)
        return;
    if(!candlock())
        return;
    if(screenimage == nil){
        dunlock();
        return;
    }
    p = sdraw.savemap;
    nc = screenimage->depth > 8 ? 256 : 1<<screenimage->depth;

    /*
     * blankscreen uses the hardware to blank the screen
     * when possible.  to help in cases when it is not possible,
     * we set the color map to be all black.
     */
    if(!blank){ /* turn screen on */
        for(i=0; i<nc; i++, p+=3)
            arch_setcolor(i, p[0], p[1], p[2]);
        arch_blankscreen(false);
    }else{  /* turn screen off */
        arch_blankscreen(true);
        for(i=0; i<nc; i++, p+=3){
            arch_getcolor(i, &p[0], &p[1], &p[2]);
            arch_setcolor(i, 0, 0, 0);
        }
    }
    sdraw.blanked = blank;
    dunlock();
}
```

⟨KDraw *other fields* 304d⟩+≡ (36a) ◁304b 304g▷

```
ulong   savemap[3*256];
```

⟨*global* blanktime 304e⟩≡ (480b)

```
ulong blanktime = 30;   /* in minutes; a half hour */
```

⟨vgactl *cases(x86)* 304f⟩+≡ (369) ◁304a 305d▷

```
case CMblanktime:
    blanktime = strtoul(cb->f[1], 0, 0);
    return;
```

⟨KDraw *other fields* 304g⟩+≡ (36a) ◁304d

```
ulong   blanktime;  /* time of last operation */
```

⟨*function* drawactive 305a⟩≡ (480b)

```
/*
 * record activity on screen, changing blanking as appropriate
 */
void
drawactive(bool active)
{
    if(active){
        drawblankscreen(false);
        sdraw.blanktime = CPUS(0)->ticks;
    }else{
        if(blanktime && sdraw.blanktime
           && TK2SEC(CPUS(0)->ticks - sdraw.blanktime)/60 >= blanktime)
            drawblankscreen(true);
    }
}
```

⟨*function* drawidletime 305b⟩≡ (480b)

```
int
drawidletime(void)
{
    return TK2SEC(CPUS(0)->ticks - sdraw.blanktime)/60;
}
```

15.8.1 Hardware support hook (x86)

⟨*global* hwblank*(x86)* 305c⟩≡ (492)

```
bool hwblank = false;    /* turned on by drivers that are known good */
```

⟨vgactl *cases(x86)* 305d⟩+≡ (369) ◁304f 370a▷

```
case CMhwblank:
    if(strcmp(cb->f[1], "on") == 0)
        hwblank = true;
    else if(strcmp(cb->f[1], "off") == 0)
        hwblank = false;
    else
        break;
    return;
```

⟨VGAscr *optional methods(x86)* 305e⟩≡ (362a) 363a▷

```
void  (*blank)(VGAscr*, int);
```

⟨VGAscr *other fields(x86)* 305f⟩≡ (362a) 362d▷

```
int isblank;
```

⟨*function blankscreen bis(x86)* 305g⟩≡ (492)

```
void
arch_blankscreen(int blank)
{
    VGAscr *scr;

    scr = &vgascreen;
    if(hwblank){
        if(scr->blank)
            scr->blank(scr, blank);
        else
            vgablank(scr, blank);
    }
}
```

⟨*function* vgablank*(x86)* 306a⟩≡ (492)

```
/*
 * Supposedly this is the way to turn DPMS
 * monitors off using just the VGA registers.
 * Unfortunately, it seems to mess up the video mode
 * on the cards I've tried.
 */
void
vgablank(VGAscr*, int blank)
{
    uchar seq1, crtc17;

    if(blank) {
        seq1 = 0x00;
        crtc17 = 0x80;
    } else {
        seq1 = 0x20;
        crtc17 = 0x00;
    }

    outs(Seqx, 0x0100);         /* synchronous reset */
    seq1 |= vgaxi(Seqx, 1) & ~0x20;
    vgaxo(Seqx, 1, seq1);
    crtc17 |= vgaxi(Crtx, 0x17) & ~0x80;
    arch_delay(10);
    vgaxo(Crtx, 0x17, crtc17);
    outs(Crtx, 0x0300);             /* end synchronous reset */
}
```

15.9 Refresh methods

⟨Client *other fields* 306b⟩+≡ (35d) ◁178d

```
Refresh*    refresh;
Rendez      refrend;
int     refreshme;
```

⟨*function* drawrefactive 306c⟩≡ (480b)

```
int
drawrefactive(void *a)
{
    Client *c;

    c = a;
    return c->refreshme || c->refresh != nil;
}
```

⟨*struct* Refresh 306d⟩≡ (476)

```
struct Refresh
{
    DImage*     dimage;
    Rectangle   r;
    Refresh*    next;
};
```

⟨*function* drawwakeall 306e⟩≡ (481)

```
void
drawwakeall(void)
{
```

```
    Client *cl;
    int i;

    for(i=0; i<sdraw.nclient; i++){
        cl = sdraw.client[i];
        if(cl && (cl->refreshme || cl->refresh))
            wakeup(&cl->refrend);
    }
}
```

⟨drawclose() *other locals* 307a⟩+≡ (65e) ◁66b

```
Refresh *r;
```

⟨drawclose() *free refresh* 307b⟩≡ (65e)

```
while(r = cl->refresh){ /* assign = */
    cl->refresh = r->next;
    free(r);
}
```

15.9.1 /dev/draw/x/refresh

⟨Display *devdraw connection fields* 307c⟩+≡ (34e) ◁35b

```
fdt     reffd; // /dev/draw/x/refresh
```

⟨drawread() *other locals* 307d⟩+≡ (66d) ◁252a

```
Refresh *r;
```

⟨drawread() *switch qid cases* 307e⟩+≡ (66d) ◁252b

```
case Qrefresh:
    if(n < 5*4)
        error(Ebadarg);
    for(;;){
        if(cl->refreshme || cl->refresh)
            break;

        dunlock();
        if(waserror()){
            dlock();    /* restore lock for waserror() above */
            nexterror();
        }

        sleep(&cl->refrend, drawrefactive, cl);

        poperror();
        dlock();
    }
    p = a;
    while(cl->refresh && n>=5*4){
        r = cl->refresh;
        BPLONG(p+0*4, r->dimage->id);
        BPLONG(p+1*4, r->r.min.x);
        BPLONG(p+2*4, r->r.min.y);
        BPLONG(p+3*4, r->r.max.x);
        BPLONG(p+4*4, r->r.max.y);
        cl->refresh = r->next;
        free(r);
        p += 5*4;
        n -= 5*4;
    }
```

```
    cl->refreshme = 0;
    n = p-(uchar*)a;
    break;
```

15.9.2 Window refresh

⟨ldrawop() *return if no save in dst* 308a⟩≡ (225a)

```
if(insave && d->dstlayer->save == nil)
    return;
```

⟨memdraw() *if refresh function for src* 308b⟩≡ (222e)

```
if(sl->save == nil)
    return; /* refresh function makes this case unworkable */
```

⟨memdraw() *if refresh function for dst* 308c⟩≡ (223c)

```
if(dl->save == nil)
    return; /* refresh function makes this case unworkable */
```

⟨drawfreedimage() *free refreshptr* 308d⟩≡ (210c)

```
if(l->layer->refreshfn == drawrefresh)  /* else true owner will clean up */
    free(l->layer->refreshptr);
l->layer->refreshptr = nil;
```

⟨*function* drawrefresh 308e⟩≡ (480b)

```
void
drawrefresh(Memimage*, Rectangle r, void *v)
{
    Refx *x;
    DImage *d;
    Client *c;
    Refresh *ref;

    if(v == 0)
        return;
    x = v;
    c = x->client;
    d = x->dimage;
    for(ref=c->refresh; ref; ref=ref->next)
        if(ref->dimage == d){
            combinerect(&ref->r, r);
            return;
        }
    ref = malloc(sizeof(Refresh));
    if(ref){
        ref->dimage = d;
        ref->r = r;
        ref->next = c->refresh;
        c->refresh = ref;
    }
}
```

⟨drawmesg() *free image case, if dscreen* 308f⟩≡ (76d)

```
if(ll && ll->dscreen && ll->dscreen->owner != client)
    ll->dscreen->owner->refreshme = 1;
```

⟨*struct* Refx 308g⟩≡ (476)

```
struct Refx
{
    Client*     client;
    DImage*     dimage;
};
```

⟨*function* memlnorefresh 309a⟩≡ (459d)

```
void
memlnorefresh(Memimage *l, Rectangle r, void *v)
{
    USED(l);
    USED(r.min.x);
    USED(v);
}
```

⟨lexposeop() *if not save but refresh method* 309b⟩≡ (217b)

```
else
    l->refreshfn(dst, r, l->refreshptr);
```

⟨drawmesg() *when top or bottom windows, refresh* 309c⟩≡ (212d)

```
ll = drawlookup(client, BGLONG(a+1+1+2), true);
drawrefreshscreen(ll, client);
```

⟨drawmesg() *when position window, refresh* 309d⟩≡ (218c)

```
ll = drawlookup(client, BGLONG(a+1), true);
drawrefreshscreen(ll, client);
```

⟨*function* drawrefreshscreen 309e⟩≡ (480b)

```
void
drawrefreshscreen(DImage *l, Client *client)
{
    while(l != nil && l->dscreen == nil)
        l = l->fromname;
    if(l != nil && l->dscreen->owner != client)
        l->dscreen->owner->refreshme = 1;
}
```

⟨Memlayer *refresh fields* 309f⟩≡ (199e)

```
Refreshfn refreshfn;  /* function to call to refresh obscured parts if save==nil */
void  *refreshptr; /* argument to refreshfn */
```

⟨drawmesg() *locals* 309g⟩+≡ (71) ◁212c

```
Refreshfn reffn;
Refx *refx;
```

⟨*enum* Refresh 309h⟩≡ (386d)

```
enum RefreshMethod
{
    /* refresh methods */
    Refbackup   = 0,
    Refnone     = 1,

    Refmesg     = 2 // incomplete apparently
};
```

⟨drawmesg() *when allocate window, set reffn* 309i⟩≡ (207b)

```
reffn = nil;
switch(refresh){
case Refbackup:
    break;
case Refnone:
    reffn = memlnorefresh;
    break;
case Refmesg:
    reffn = drawrefresh;
    break;
default:
    error("unknown refresh method");
}
```

⟨drawmesg() *when allocate window, if reffn* 310a⟩≡ (207b)

```
if(reffn){
    refx = nil;
    if(reffn == drawrefresh){
        refx = malloc(sizeof(Refx));
        if(refx == 0){
            drawuninstall(client, dstid);
            error(Edrawmem);
        }
        refx->client = client;
        refx->dimage = drawlookup(client, dstid, true);
    }
    memlsetrefresh(l, reffn, refx);
}
```

⟨memlalloc() *if refreshfn* 310b⟩≡ (208a)

```
if(refreshfn)
    l->save = nil;
```

⟨memlalloc() *set refresh fields part1* 310c⟩≡ (207e)

```
l->refreshfn = refreshfn;
l->refreshptr = nil; /* don't set it until we're done */
```

⟨memlalloc() *set refresh fields part2* 310d⟩≡ (207e)

```
l->refreshptr = refreshptr;
```

⟨*function* memlsetrefresh 310e⟩≡ (460a)

```
int
memlsetrefresh(Memimage *i, Refreshfn fn, void *ptr)
{
    Memlayer *l;

    l = i->layer;
    if(l->refreshfn!=nil && fn!=nil){ /* just change functions */
        l->refreshfn = fn;
        l->refreshptr = ptr;
        return 1;
    }

    if(l->refreshfn == nil){ /* is using backup image; just free it */
        freememimage(l->save);
        l->save = nil;
        l->refreshfn = fn;
        l->refreshptr = ptr;
        return 1;
    }

    l->save = allocmemimage(i->r, i->chan);
    if(l->save == nil)
        return 0;
    /* easiest way is just to update the entire save area */
    l->refreshfn(i, i->r, l->refreshptr);
    l->refreshfn = nil;
    l->refreshptr = nil;
    return 1;
}
```

15.10 Concurrency

15.10.1 lockdisplay()

⟨Display *concurrency fields* 311a⟩≡ (34e) 311d ▷
```
QLock   qlock;
```

⟨*function* lockdisplay 311b⟩≡ (400c)
```
void
lockdisplay(Display *disp)
{
    ⟨lockdisplay() if debuglockdisplay 348d⟩
    else
        qlock(&disp->qlock);
}
```

⟨*function* unlockdisplay 311c⟩≡ (400c)
```
void
unlockdisplay(Display *disp)
{
    qunlock(&disp->qlock);
}
```

⟨Display *concurrency fields* 311d⟩+≡ (34e) ◁ 311a
```
bool        locking;    /*program is using lockdisplay */
```

15.10.2 /dev/draw/x/ctl concurrent access

⟨drawopen() *locals* 311e⟩+≡ (65a) ◁ 67d
```
DImage *di;
DName *dn;
```

⟨drawopen() *switch qid cases* 311f⟩+≡ (65a) ◁ 234e
```
case Qctl:
    cl = drawclient(c);

    ⟨drawopen() switch qid cases, when Qctl, set busy 312a⟩

    flushrect = Rect(10000, 10000, -10000, -10000);
    dn = drawlookupname(strlen(screenname), screenname);
    if(dn == nil)
        error("draw: cannot happen 2");

    if(drawinstall(cl, 0, dn->dimage->image, 0) == 0)
        error(Edrawmem);

    di = drawlookup(cl, 0, false);
    if(di == nil)
        error("draw: cannot happen 1");

    ⟨drawopen() switch qid cases, when Qctl, set name 311g⟩

    incref(&cl->r);
    break;
```

⟨drawopen() *switch qid cases, when Qctl, set name* 311g⟩≡ (311f)
```
di->vers = dn->vers;
di->name = smalloc(strlen(screenname)+1);
strcpy(di->name, screenname);
di->fromname = dn->dimage;
di->fromname->ref++;
```

⟨drawopen() *switch qid cases, when Qctl, set busy* 312a⟩≡ (311f)

```
if(cl->busy)
    error(Einuse);
cl->busy = true;
```

⟨drawclose() *if Qctl* 312b⟩≡ (65e)

```
if(QID(c->qid) == Qctl)
    cl->busy = false;
```

⟨Client *concurrency fields* 312c⟩≡ (35d)

```
bool     busy;
```

15.11 Optimisations

15.11.1 memimagedraw() optimisations

hwdraw() (x86)

⟨hwdraw() *when dst is the screen* 312d⟩≡ (94d)

```
if(scr->fill == nil && scr->scroll == nil)
    return false;
/*
 * If we have an opaque mask and source is one opaque
 * pixel we can convert to the destination format and just
 * replicate with memset.
 */
m = Simplesrc|Simplemask|Fullmask;
if(scr->fill
&& (par->state&m)==m
&& ((par->srgba&0xFF) == 0xFF)
&& (par->op&S) == S)
    return scr->fill(scr, par->r, par->sdval);

/*
 * If no source alpha, an opaque mask, we can just copy the
 * source onto the destination.  If the channels are the same and
 * the source is not replicated, memmove suffices.
 */
m = Simplemask|Fullmask;
if(scr->scroll
&& src->data->bdata==dst->data->bdata
&& !(src->flags&Falpha)
&& (par->state&m)==m
&& (par->op&S) == S)
    return scr->scroll(scr, par->r, par->sr);

return false;
```

chardraw()

⟨memimagedraw() *try chardraw* 312e⟩≡ (93b)

```
/*
 * Character drawing.
 * Solid source color being painted through a boolean mask onto a
 * high res image.
 */
if(chardraw(&par)){
```

```
    DBG1("chardraw handled\n");
    return;
}
```

⟨*function* chardraw 313⟩≡ (432)

```
/*
 * Boolean character drawing.
 * Solid opaque color through a 1-bit greyscale mask.
 */
static bool
chardraw(Memdrawparam *par)
{
    Rectangle r, mr;
    Memimage *mask, *src, *dst;
    int op;
    int dx, dy;

    ulong bits;
    int i, ddepth, x, bx, ex, y, npack, bsh, depth;
    ulong v, maskwid, dstwid;
    uchar *wp, *rp, *q, *wc;
    ushort *ws;
    ulong *wl;
    uchar sp[4];

    DBG1("chardraw? mf %lux md %d sf %lux dxs %d dys %d dd %d ddat %p sdat %p\n",
        par->mask->flags, par->mask->depth, par->src->flags,
        Dx(par->src->r), Dy(par->src->r), par->dst->depth, par->dst->data, par->src->data);

    mask = par->mask;
    src = par->src;
    dst = par->dst;
    r = par->r;
    mr = par->mr;
    op = par->op;

    if((par->state&(Replsrc|Simplesrc|Replmask)) != (Replsrc|Simplesrc)
    || mask->depth != 1
    || src->flags&Falpha
    || dst->depth<8
    || dst->data==src->data
    || op != SoverD)
        return false;

    // else

    DBG1("chardraw...");

    depth = mask->depth;
    maskwid = mask->width * sizeof(ulong);
    rp = byteaddr(mask, mr.min);
    npack = 8/depth;
    bsh = (mr.min.x % npack) * depth;

    wp = byteaddr(dst, r.min);
    dstwid = dst->width*sizeof(ulong);
    DBG1("bsh %d\n", bsh);
    dy = Dy(r);
    dx = Dx(r);
```

```
    ddepth = dst->depth;

    /*
     * for loop counts from bsh to bsh+dx
     *
     * we want the bottom bits to be the amount
     * to shift the pixels down, so for n≡0 (mod 8) we want
     * bottom bits 7.  for n≡1, 6, etc.
     * the bits come from -n-1.
     */

    bx = -bsh-1;
    ex = -bsh-1-dx;
    SET(bits);
    v = par->sdval;

    /* make little endian */
    sp[0] = v;
    sp[1] = v>>8;
    sp[2] = v>>16;
    sp[3] = v>>24;

    DBG1("sp %x %x %x %x\n", sp[0], sp[1], sp[2], sp[3]);
    for(y=0; y<dy; y++, rp+=maskwid, wp+=dstwid){
        q = rp;
        if(bsh)
            bits = *q++;
        switch(ddepth){
        ⟨chardraw() switch depth cases 244d⟩
        case 32:
            wl = (ulong*)wp;
            v = *(ulong*)sp;
            for(x=bx; x>ex; x--, wl++){
                i = x&7;
                if(i == 8-1)
                    bits = *q++;
                DBG1("bits %lux sh %d...", bits, i);
                if((bits>>i)&1)
                    *wl = v;
            }
            break;
        }
    }

    DBG1("\n");
    return 1;
}
```

15.11.2 alphadraw() optimisations

Small replicated rectangle

⟨ParamDraw *replication fields* 314a⟩≡ (106g) 315a ▷

```
bool replcache; /* if set, cache buffers */
```

⟨getparam() *if small replicated rectangle* 314b⟩≡ (107b)

```
if((img->flags&Frepl) && Dy(img->r) <= MAXBCACHE && Dy(img->r) < Dy(r)){
    p->replcache = true;
    nbuf = Dy(img->r);
}
```

⟨ParamDraw *replication fields* 315a⟩+≡ (106g) ◁314a 315c▷

```
Readfn *replcall;
```

⟨alphadraw() *when small repl rectangle optimization* 315b⟩≡ (103)

```
/*
 * If the image has a small enough repl rectangle,
 * we can just read each line once and cache them.
 */
if(z->spar.replcache){
    z->spar.replcall = rdsrc;
    rdsrc = replread;
}
if(z->mpar.replcache){
    z->mpar.replcall = rdmask;
    rdmask = replread;
}
```

⟨ParamDraw *replication fields* 315c⟩+≡ (106g) ◁315a

```
Buffer bcache[MAXBCACHE];
ulong bfilled;
```

⟨*enum* _anon_*(*(lib_graphics/libmemdraw/draw.c)*)* 315d⟩≡ (429b)

```
enum {
    MAXBCACHE = 16
};
```

⟨*function* replread 315e⟩≡ (429b)

```
/*
 * Replicated cached scan line read.  Call the function listed in the Param,
 * but cache the result so that for replicated images we only do the work once.
 */
static Buffer
replread(Param *p, uchar *s, int y)
{
    Buffer *b;

    USED(s);
    b = &p->bcache[y];
    if((p->bfilled & (1<<y)) == 0){
        p->bfilled |= 1<<y;
        *b = p->replcall(p, p->bufbase+y*p->bufdelta, y);
    }
    return *b;
}
```

No alpha source

⟨alphadraw() *if source has no alpha and simple bit mask* 315f⟩≡ (103)

```
/*
 * If the mask is purely boolean, we can convert from src to dst format
 * when we read src, and then just copy it to dst where the mask tells us to.
 * This requires a boolean (1-bit grey) mask and lack of a source alpha channel.
 *
 * The computation is accomplished by assigning the function pointers as follows:
 * rdsrc - read and convert source into dst format in a buffer
 *  rdmask - convert mask to bytes, set pointer to it
 *  rddst - fill with pointer to real dst data, but do no reads
 * calc - copy src onto dst when mask says to.
 * wrdst - do nothing
```

```
 * This is slightly sleazy, since things aren't doing exactly what their names say,
 * but it avoids a fair amount of code duplication to make this a case here
 * rather than have a separate booldraw.
 */
if(!(src->flags&Falpha)
&& mask->chan == GREY1
&& dst->depth >= 8
&& op == SoverD){
    rdsrc = convfn(dst, &z->dpar, src, &z->spar, &ndrawbuf);
    rddst = readptr;
    rdmask = readfn(mask);
    calc = boolcopyfn(dst, mask);
    wrdst = nullwrite;
}
```

⟨`alphadraw()` *when mask and source have no alpha, possibly adapt calc* 316a⟩≡ (110c)

```
/*
 * Should really be above, but then boolcopyfns would have
 * to deal with bit alignment, and I haven't written that.
 *
 * This is a common case for things like ellipse drawing.
 * When there's no alpha involved and the mask is boolean,
 * we can avoid all the division and multiplication.
 */
if(mask->chan == GREY1 && !(src->flags&Falpha))
    calc = boolcalc[op];
else if(op == SoverD && !(src->flags&Falpha))
    calc = alphacalcS;
```

Simple bit mask and big dest, `boolcopyxxx()`

⟨`ParamDraw` *conversion fields* 316b⟩≡ (106g) 317a ▷

```
Readfn *convreadcall;
Writefn *convwritecall;
int convbufoff;
Param *convdpar;
int convdx;
```

⟨*function* `convfn` 316c⟩≡ (429b)

```
static Readfn*
convfn(Memimage *dst, Param *dpar, Memimage *src, Param *spar, int *ndrawbuf)
{
    if(dst->chan == src->chan && !(src->flags&Frepl)){
        DBG1("readptr...");
        return readptr;
    }

    if(dst->chan==CMAP8 && (src->chan==GREY1||src->chan==GREY2||src->chan==GREY4)){
        /* cheat because we know the replicated value is exactly the color map entry. */
        DBG1("Readnbit...");
        return readnbit;
    }

    spar->convreadcall = readfn(src);
    spar->convwritecall = writefn(dst);
    spar->convdpar = dpar;

    /* allocate a conversion buffer */
    spar->convbufoff = *ndrawbuf;
```

```
    *ndrawbuf += spar->dx*4;

    if(spar->dx > Dx(spar->img->r)){
        spar->convdx = spar->dx;
        spar->dx = Dx(spar->img->r);
    }

    DBG1("genconv...");
    return genconv;
}
```

⟨ParamDraw *conversion fields* 317a⟩+≡ (106g) ◁316b

```
uchar *convbuf;
```

⟨alphadraw() *set z params part2* 317b⟩+≡ (106c) ◁107c

```
z->spar.convbuf = drawbuf + z->spar.convbufoff;
```

⟨*function* genconv 317c⟩≡ (429b)

```
static Buffer
genconv(Param *p, uchar *buf, int y)
{
    Buffer b;
    int nb;
    uchar *r, *w, *ew;

    /* read from source into RGB format in convbuf */
    b = p->convreadcall(p, p->convbuf, y);

    /* write RGB format into dst format in buf */
    p->convwritecall(p->convdpar, buf, b);

    if(p->convdx){
        nb = p->convdpar->img->depth/8;
        r = buf;
        w = buf+nb*p->dx;
        ew = buf+nb*p->convdx;
        while(w<ew)
            *w++ = *r++;
    }

    b.red = buf;
    b.blu = b.grn = b.grey = b.alpha = nil;
    b.rgba = (ulong*)buf;
    b.delta = 0;

    return b;
}
```

⟨*global* nullwrite 317d⟩≡ (429b)

```
static Writefn nullwrite;
```

⟨*function* nullwrite 317e⟩≡ (429b)

```
static void
nullwrite(Param *p, uchar *s, Buffer b)
{
    USED(p);
    USED(s);
    USED(b);
}
```

⟨*function* readptr 318a⟩≡ (429b)

```
static Buffer
readptr(Param *p, uchar *s, int y)
{
    Buffer b;
    uchar *q;

    USED(s);
    q = p->bytermin + y*p->bwidth;
    b.red = q; /* ptr to data */
    b.grn = b.blu = b.grey = b.alpha = nil;
    b.rgba = (ulong*)q;
    b.delta = p->img->depth/8;
    return b;
}
```

⟨*function* boolcopyfn 318b⟩≡ (429b)

```
static Calcfn*
boolcopyfn(Memimage *img, Memimage *mask)
{
    if(mask->flags&Frepl && Dx(mask->r)==1 && Dy(mask->r)==1 && pixelbits(mask, mask->r.min)==~0)
        return boolmemmove;

    switch(img->depth){
    case 8:
        return boolcopy8;
    case 16:
        return boolcopy16;
    case 24:
        return boolcopy24;
    case 32:
        return boolcopy32;
    default:
        assert(0 /* boolcopyfn */);
    }
    return nil;
}
```

⟨*function* boolmemmove 318c⟩≡ (429b)

```
static Buffer
boolmemmove(Buffer bdst, Buffer bsrc, Buffer b1, int dx, int i, int o)
{
    USED(i);
    USED(o);
    USED(b1);
    USED(bsrc);
    memmove(bdst.red, bsrc.red, dx*bdst.delta);
    return bdst;
}
```

⟨*function* boolcopy8 318d⟩≡ (429b)

```
static Buffer
boolcopy8(Buffer bdst, Buffer bsrc, Buffer bmask, int dx, int i, int o)
{
    uchar *m, *r, *w, *ew;

    USED(i);
    USED(o);
    m = bmask.grey;
    w = bdst.red;
```

```
    r = bsrc.red;
    ew = w+dx;
    for(; w < ew; w++,r++)
        if(*m++)
            *w = *r;
    return bdst; /* not used */
}
```

⟨*function* boolcopy16 319a⟩≡ (429b)

```
static Buffer
boolcopy16(Buffer bdst, Buffer bsrc, Buffer bmask, int dx, int i, int o)
{
    uchar *m;
    ushort *r, *w, *ew;

    USED(i);
    USED(o);
    m = bmask.grey;
    w = (ushort*)bdst.red;
    r = (ushort*)bsrc.red;
    ew = w+dx;
    for(; w < ew; w++,r++)
        if(*m++)
            *w = *r;
    return bdst; /* not used */
}
```

⟨*function* boolcopy24 319b⟩≡ (429b)

```
static Buffer
boolcopy24(Buffer bdst, Buffer bsrc, Buffer bmask, int dx, int i, int o)
{
    uchar *m;
    uchar *r, *w, *ew;

    USED(i);
    USED(o);
    m = bmask.grey;
    w = bdst.red;
    r = bsrc.red;
    ew = w+dx*3;
    while(w < ew){
        if(*m++){
            *w++ = *r++;
            *w++ = *r++;
            *w++ = *r++;
        }else{
            w += 3;
            r += 3;
        }
    }
    return bdst; /* not used */
}
```

⟨*function* boolcopy32 319c⟩≡ (429b)

```
static Buffer
boolcopy32(Buffer bdst, Buffer bsrc, Buffer bmask, int dx, int i, int o)
{
    uchar *m;
    ulong *r, *w, *ew;
```

```
    USED(i);
    USED(o);
    m = bmask.grey;
    w = (ulong*)bdst.red;
    r = (ulong*)bsrc.red;
    ew = w+dx;
    for(; w < ew; w++,r++)
        if(*m++)
            *w = *r;
    return bdst; /* not used */
}
```

Simple bit mask, `boolcalcxxx()`

⟨*global* `boolcalc` 320a⟩≡ (429b)

```
static Calcfn *boolcalc[Ncomp] =
{
    alphacalc0,  /* Clear */
    boolcalc14,  /* DoutS */
    boolcalc236789,  /* SoutD */
    boolcalc236789,  /* DxorS */
    boolcalc14,  /* DinS */
    alphacalc5,  /* D */
    boolcalc236789,  /* DatopS */
    boolcalc236789,  /* DoverS */
    boolcalc236789,  /* SinD */
    boolcalc236789,  /* SatopD */
    boolcalc1011,  /* S */
    boolcalc1011,  /* SoverD */
};
```

⟨`Buffer` *boolcalc fields* 320b⟩≡ (107d)

```
/* used by boolcalc* for mask data */
uchar *m;  /* ptr to mask data r.min byte; like p->bytermin */
int  mskip; /* no. of left bits to skip in *m */
uchar *bm;  /* ptr to mask data img->r.min byte; like p->bytey0s */
int  bmskip; /* no. of left bits to skip in *bm */
uchar *em;  /* ptr to mask data img->r.max.x byte; like p->bytey0e */
int  emskip; /* no. of right bits to skip in *em */
```

⟨*function* `boolcalc14` 320c⟩≡ (429b)

```
static Buffer
boolcalc14(Buffer bdst, Buffer b1, Buffer bmask, int dx, int grey, int op)
{
    Buffer obdst;
    int i, ma, zero;

    USED(b1);

    obdst = bdst;

    for(i=0; i<dx; i++){
        ma = *bmask.alpha;
        zero = ma ? op == DoutS : op == DinS;

        if(grey){
            if(zero)
                *bdst.grey = 0;
            bdst.grey += bdst.delta;
```

```
        }else{
            if(zero)
                *bdst.red = *bdst.grn = *bdst.blu = 0;
            bdst.red += bdst.delta;
            bdst.blu += bdst.delta;
            bdst.grn += bdst.delta;
        }
        bmask.alpha += bmask.delta;
        if(bdst.alpha != &ones){
            if(zero)
                *bdst.alpha = 0;
            bdst.alpha += bdst.delta;
        }
    }
    return obdst;
}
```

⟨*function* boolcalc236789 321⟩≡ (429b)

```
static Buffer
boolcalc236789(Buffer bdst, Buffer bsrc, Buffer bmask, int dx, int grey, int op)
{
    Buffer obdst;
    int fs, fd;
    int i, ma, da, zero;
    ulong s, t;

    obdst = bdst;
    zero = !(op&1);

    for(i=0; i<dx; i++){
        ma = *bmask.alpha;
        da = *bdst.alpha;
        fs = da;
        if(op&2)
            fs = 255-da;
        fd = 0;
        if(op&4)
            fd = 255;

        if(grey){
            if(ma)
                *bdst.grey = MUL(fs, *bsrc.grey, s)+MUL(fd, *bdst.grey, t);
            else if(zero)
                *bdst.grey = 0;
            bsrc.grey += bsrc.delta;
            bdst.grey += bdst.delta;
        }else{
            if(ma){
                *bdst.red = MUL(fs, *bsrc.red, s)+MUL(fd, *bdst.red, t);
                *bdst.grn = MUL(fs, *bsrc.grn, s)+MUL(fd, *bdst.grn, t);
                *bdst.blu = MUL(fs, *bsrc.blu, s)+MUL(fd, *bdst.blu, t);
            }
            else if(zero)
                *bdst.red = *bdst.grn = *bdst.blu = 0;
            bsrc.red += bsrc.delta;
            bsrc.blu += bsrc.delta;
            bsrc.grn += bsrc.delta;
            bdst.red += bdst.delta;
            bdst.blu += bdst.delta;
            bdst.grn += bdst.delta;
```

```
        }
        bmask.alpha += bmask.delta;
        if(bdst.alpha != &ones){
            if(ma)
                *bdst.alpha = fs+MUL(fd, da, t);
            else if(zero)
                *bdst.alpha = 0;
            bdst.alpha += bdst.delta;
        }
    }
    return obdst;
}
```

⟨*function* boolcalc1011 322⟩≡ (429b)

```
static Buffer
boolcalc1011(Buffer bdst, Buffer bsrc, Buffer bmask, int dx, int grey, int op)
{
    Buffer obdst;
    int i, ma, zero;

    obdst = bdst;
    zero = !(op&1);

    for(i=0; i<dx; i++){
        ma = *bmask.alpha;

        if(grey){
            if(ma)
                *bdst.grey = *bsrc.grey;
            else if(zero)
                *bdst.grey = 0;
            bsrc.grey += bsrc.delta;
            bdst.grey += bdst.delta;
        }else{
            if(ma){
                *bdst.red = *bsrc.red;
                *bdst.grn = *bsrc.grn;
                *bdst.blu = *bsrc.blu;
            }
            else if(zero)
                *bdst.red = *bdst.grn = *bdst.blu = 0;
            bsrc.red += bsrc.delta;
            bsrc.blu += bsrc.delta;
            bsrc.grn += bsrc.delta;
            bdst.red += bdst.delta;
            bdst.blu += bdst.delta;
            bdst.grn += bdst.delta;
        }
        bmask.alpha += bmask.delta;
        if(bdst.alpha != &ones){
            if(ma)
                *bdst.alpha = 255;
            else if(zero)
                *bdst.alpha = 0;
            bdst.alpha += bdst.delta;
        }
    }
    return obdst;
}
```

No alpha source, alphacalcS()

⟨*function* alphacalcS 323a⟩≡ (429b)

```
/* source alpha 1 */
static Buffer
alphacalcS(Buffer bdst, Buffer bsrc, Buffer bmask, int dx, int grey, int op)
{
    Buffer obdst;
    int fd;
    int i, ma;
    ulong s, t;

    USED(op);
    obdst = bdst;

    for(i=0; i<dx; i++){
        ma = *bmask.alpha;
        fd = 255-ma;

        if(grey){
            *bdst.grey = MUL(ma, *bsrc.grey, s)+MUL(fd, *bdst.grey, t);
            bsrc.grey += bsrc.delta;
            bdst.grey += bdst.delta;
        }else{
            *bdst.red = MUL(ma, *bsrc.red, s)+MUL(fd, *bdst.red, t);
            *bdst.grn = MUL(ma, *bsrc.grn, s)+MUL(fd, *bdst.grn, t);
            *bdst.blu = MUL(ma, *bsrc.blu, s)+MUL(fd, *bdst.blu, t);
            bsrc.red += bsrc.delta;
            bsrc.blu += bsrc.delta;
            bsrc.grn += bsrc.delta;
            bdst.red += bdst.delta;
            bdst.blu += bdst.delta;
            bdst.grn += bdst.delta;
        }
        if(bdst.alpha != &ones){
            *bdst.alpha = ma+MUL(fd, *bdst.alpha, t);
            bdst.alpha += bdst.delta;
        }
        bmask.alpha += bmask.delta;
    }
    return obdst;
}
```

32 bits SoverD alphacalc()

⟨alphacalc11() *special case for 32bits source and dest* 323b⟩≡ (111e)

```
if(q){
    *bdst.rgba = MUL0123(ma, *bsrc.rgba, s, t)+MUL0123(fd, *bdst.rgba, u, v);
    bsrc.rgba++;
    bdst.rgba++;
    bsrc.alpha += sadelta;
    bmask.alpha += bmask.delta;
    continue;
}
```

⟨*function* MUL0123 323c⟩≡ (429b)

```
#define MUL0123(a, x, s, t) ((MUL13(a, x, s)<<8)|MUL02(a, x, t))
```

⟨*function* MUL13 323d⟩≡ (429b)

```
#define MUL13(a, x, t)  (t = (a)*(((x)&MASK13)>>8)+128, ((t+((t>>8)&MASK02))>>8)&MASK02)
```

⟨*constant* MASK13 324a⟩≡ (429b)
```
#define MASK13 0xFF00FF00
```

⟨*function* MUL02 324b⟩≡ (429b)
```
#define MUL02(a, x, t)  (t = (a)*(((x)&MASK02)>>0)+128, ((t+((t>>8)&MASK02))>>8)&MASK02)
```

⟨*constant* MASK02 324c⟩≡ (429b)
```
#define MASK02 0x00FF00FF
```

15.11.3 drawchar() optimisations

⟨drawchar() *other locals* 324d⟩≡ (152d)
```
static Memimage *tmp;
```

⟨drawchar() *optimization when possible* 324e⟩≡ (152d)
```
/*
 * If we're drawing greyscale fonts onto a VGA screen,
 * it's very costly to read the screen memory to do the
 * alpha blending inside memdraw.  If this is really a stringbg,
 * then rdst is the bg image (in main memory) which we can
 * refer to for the underlying dst pixels instead of reading dst
 * directly.
 */
if(arch_ishwimage(dst) && !arch_ishwimage(rdst) && font->image->depth > 1){
    if(tmp == nil || tmp->chan != dst->chan || Dx(tmp->r) < Dx(r) || Dy(tmp->r) < Dy(r)){
        if(tmp)
            freememimage(tmp);
        tmp = allocmemimage(Rect(0,0,Dx(r),Dy(r)), dst->chan);
        if(tmp == nil)
            goto fallback;
    }
    memdraw(tmp, Rect(0,0,Dx(r),Dy(r)), rdst, r.min, memopaque, ZP, S);
    memdraw(tmp, Rect(0,0,Dx(r),Dy(r)), src, sp1, font->image,
      Pt(fc->minx, fc->miny), op);
    memdraw(dst, r, tmp, ZP, memopaque, ZP, S);
}
```

15.11.4 Flushing bounding boxes

Flushing lines

⟨drawmesg() *when draw line, possible flush* 324f⟩≡ (114c)
```
/* avoid memlinebbox if possible */
if(dstid==0 || dst->layer!=nil){
    /* BUG: this is terribly inefficient: update maximal containing rect*/
    r = memlinebbox(p, q, e0, e1, j);
    dstflush(dstid, dst, insetrect(r, -(1+1+j)));
}
```

⟨*function* memlinebbox 324g⟩≡ (435a)
```
Rectangle
memlinebbox(Point p0, Point p1, int end0, int end1, int radius)
{
    Rectangle r, r1;
    int extra;

    r.min.x = 10000000;
    r.min.y = 10000000;
```

```
    r.max.x = -10000000;
    r.max.y = -10000000;
    extra = lmax(memlineendsize(end0), memlineendsize(end1));
    r1 = insetrect(canonrect(Rpt(p0, p1)), -(radius+extra));
    addbbox(&r, r1.min);
    addbbox(&r, r1.max);
    return r;
}
```

⟨*function* memlineendsize 325a⟩≡ (435a)

```
int
memlineendsize(int end)
{
    int x3;

    if((end&0x3F) != Endarrow)
        return 0;
    if(end == Endarrow)
        x3 = Arrow3;
    else
        x3 = (end>>23) & 0x1FF;
    return x3;
}
```

⟨*function* lmax 325b⟩≡ (435a)

```
static
int
lmax(int a, int b)
{
    if(a > b)
        return a;
    return b;
}
```

⟨*function* addbbox 325c⟩≡ (435a)

```
/*
 * Simple-minded conservative code to compute bounding box of line.
 * Result is probably a little larger than it needs to be.
 */
static
void
addbbox(Rectangle *r, Point p)
{
    if(r->min.x > p.x)
        r->min.x = p.x;
    if(r->min.y > p.y)
        r->min.y = p.y;
    if(r->max.x < p.x+1)
        r->max.x = p.x+1;
    if(r->max.y < p.y+1)
        r->max.y = p.y+1;
}
```

Flushing polygons

⟨drawmesg() *when draw polygon, set doflush* 325d⟩≡ (120a)

```
doflush = false;
if(dstid==0 || (dst->layer && dst->layer->screen->image->data == screenimage->data))
    doflush = true;    /* simplify test in loop */
```

⟨drawmesg() *when draw polygon, if doflush* 326a⟩≡ (120a)

```
if(doflush){
    esize = j;
    if(*a == 'p'){
        if(y == 0){
            c = memlineendsize(e0);
            if(c > esize)
                esize = c;
        }
        if(y == ni-1){
            c = memlineendsize(e1);
            if(c > esize)
                esize = c;
        }
    }
    if(*a=='P' && e0!=1 && e0 !=~0)
        r = dst->clipr;
    else if(y > 0){
        r = Rect(q.x-oesize, q.y-oesize, q.x+oesize+1, q.y+oesize+1);
        combinerect(&r, Rect(p.x-esize, p.y-esize, p.x+esize+1, p.y+esize+1));
    }
    if(rectclip(&r, dst->clipr))  /* should perhaps be an arg to dstflush */
        dstflush(dstid, dst, r);
}
```

⟨drawmesg() *when draw polygon, special flush if y is 1* 326b⟩≡ (120a)

```
if(y == 1)
    dstflush(dstid, dst, Rect(p.x-esize, p.y-esize, p.x+esize+1, p.y+esize+1));
```

Flushing ellipses

⟨drawmesg() *when draw ellipse, dstflush* 326c⟩≡ (129c)

```
dstflush(dstid, dst, Rect(p.x-e0-j, p.y-e1-j, p.x+e0+j+1, p.y+e1+j+1));
```

Flushing text

⟨drawmesg() *when draw text, dstflush* 326d⟩≡ (151c)

```
p.y -= font->ascent;
dstflush(dstid, dst, Rect(p.x, p.y, q.x, p.y+Dy(font->image->r)));
```

Flushing and windows

⟨drawfreedimage() *addflush* 326e⟩≡ (210c)

```
if(l->data == screenimage->data)
    addflush(l->layer->screenr);
```

⟨dstflush() *locals* 326f⟩≡ (81e)

```
Memlayer *l;
```

⟨dstflush() *if layer* 326g⟩≡ (81e)

```
l = dst->layer;
if(l == nil)
    return;

// else
do{
    if(l->screen->image->data != screenimage->data)
        return;
```

```
    r = rectaddpt(r, l->delta);
    l = l->screen->image->layer;
}while(l);

addflush(r);
```

⟨drawmesg() *when allocate window, addflush* 327a⟩≡ (207b)

```
addflush(l->layer->screenr);
```

⟨drawmesg() *when top or bottom windows, addflush* 327b⟩≡ (212d)

```
if(lp[0]->layer->screen->image->data == screenimage->data)
    for(j=0; j<nw; j++)
        addflush(lp[j]->layer->screenr);
```

⟨drawmesg() *when position window, addflush* 327c⟩≡ (218c)

```
addflush(r);
addflush(dst->layer->screenr);
```

⟨*function* addflush 327d⟩≡ (481)

```
void
addflush(Rectangle r)
{
    int abb, ar, anbb;
    Rectangle nbb;

    if(!sdraw.softscreen || !rectclip(&r, screenimage->r))
        return;
    if(flushrect.min.x >= flushrect.max.x){
        flushrect = r;
        waste = 0;
        return;
    }
    // else

    nbb = flushrect;
    combinerect(&nbb, r);
    ar = Dx(r)*Dy(r);
    abb = Dx(flushrect)*Dy(flushrect);
    anbb = Dx(nbb)*Dy(nbb);
    /*
     * Area of new waste is area of new bb minus area of old bb,
     * less the area of the new segment, which we assume is not waste.
     * This could be negative, but that's OK.
     */
    waste += anbb-abb - ar;
    if(waste < 0)
        waste = 0;
    /*
     * absorb if:
     *  total area is small
     *  waste is less than half total area
     *  rectangles touch
     */
    if(anbb<=1024 || waste*2<anbb || rectXrect(flushrect, r)){
        flushrect = nbb;
        return;
    }

    /* emit current state */
```

```
    if(flushrect.min.x < flushrect.max.x)
        arch_flushmemscreen(flushrect);
    flushrect = r;

    waste = 0;
}
```

⟨*global* waste 328a⟩≡ (481)

```
static  int     waste;
```

15.11.5 Image compression

⟨*constant* NMATCH 328b⟩≡ (393a)

```
/*
 * Compressed image file parameters and helper routines
 */
#define NMATCH 3  /* shortest match possible */
```

⟨*constant* NRUN 328c⟩≡ (393a)

```
#define NRUN (NMATCH+31) /* longest match possible */
```

Loading compressed images

⟨*function* cloadimage 328d⟩≡ (417a)

```
int
cloadimage(Image *i, Rectangle r, uchar *data, int ndata)
{
    int m, nb, miny, maxy, ncblock;
    uchar *a;

    if(!rectinrect(r, i->r)){
        werrstr("cloadimage: bad rectangle");
        return -1;
    }

    miny = r.min.y;
    m = 0;
    ncblock = _compblocksize(r, i->depth);
    while(miny != r.max.y){
        maxy = atoi((char*)data+0*12);
        nb = atoi((char*)data+1*12);
        if(maxy<=miny || r.max.y<maxy){
            werrstr("creadimage: bad maxy %d", maxy);
            return -1;
        }
        data += 2*12;
        ndata -= 2*12;
        m += 2*12;
        if(nb<=0 || ncblock<nb || nb>ndata){
            werrstr("creadimage: bad count %d", nb);
            return -1;
        }
        a = bufimage(i->display, 21+nb);
        if(a == nil)
            return -1;
        a[0] = 'Y';
        BPLONG(a+1, i->id);
        BPLONG(a+5, r.min.x);
```

```
        BPLONG(a+9, miny);
        BPLONG(a+13, r.max.x);
        BPLONG(a+17, maxy);
        memmove(a+21, data, nb);
        miny = maxy;
        data += nb;
        ndata += nb;
        m += nb;
    }
    return m;
}
```

⟨memload() *if iscompressed* 329a⟩≡ (175c)

```
if(iscompressed)
    loadfn = cloadmemimage;
```

⟨*function* cloadmemimage 329b⟩≡ (423a)

```
int
cloadmemimage(Memimage *i, Rectangle r, uchar *data, int ndata)
{
    int y, bpl, c, cnt, offs;
    uchar mem[NMEM], *memp, *omemp, *emem, *linep, *elinep, *u, *eu;

    if(!rectinrect(r, i->r))
        return -1;
    bpl = bytesperline(r, i->depth);
    u = data;
    eu = data+ndata;
    memp = mem;
    emem = mem+NMEM;
    y = r.min.y;
    linep = byteaddr(i, Pt(r.min.x, y));
    elinep = linep+bpl;
    for(;;){
        if(linep == elinep){
            if(++y == r.max.y)
                break;
            linep = byteaddr(i, Pt(r.min.x, y));
            elinep = linep+bpl;
        }
        if(u == eu){ /* buffer too small */
            return -1;
        }
        c = *u++;
        if(c >= 128){
            for(cnt=c-128+1; cnt!=0 ;--cnt){
                if(u == eu){  /* buffer too small */
                    return -1;
                }
                if(linep == elinep){ /* phase error */
                    return -1;
                }
                *linep++ = *u;
                *memp++ = *u++;
                if(memp == emem)
                    memp = mem;
            }
        }
        else{
            if(u == eu) /* short buffer */
```

```
                return -1;
            offs = *u++ + ((c&3)<<8)+1;
            if(memp-mem < offs)
                omemp = memp+(NMEM-offs);
            else
                omemp = memp-offs;
            for(cnt=(c>>2)+NMATCH; cnt!=0; --cnt){
                if(linep == elinep) /* phase error */
                    return -1;
                *linep++ = *omemp;
                *memp++ = *omemp++;
                if(omemp == emem)
                    omemp = mem;
                if(memp == emem)
                    memp = mem;
            }
        }
    }
    return u-data;
}
```

Reading compressed images

⟨`readimage()` *if first 11 characters are compressed string* 330a⟩≡ (180e)

```
if(memcmp(hdr, "compressed\n", 11) == 0)
    return creadimage(d, fd, dolock);
```

⟨*function* `creadimage` 330b⟩≡ (417a)

```
Image *
creadimage(Display *d, int fd, int dolock)
{
    char hdr[5*12+1];
    Rectangle r;
    int m, nb, miny, maxy, new, ldepth, ncblock;
    uchar *buf, *a;
    Image *i;
    ulong chan;

    if(readn(fd, hdr, 5*12) != 5*12)
        return nil;

    /*
     * distinguish new channel descriptor from old ldepth.
     * channel descriptors have letters as well as numbers,
     * while ldepths are a single digit formatted as %-11d.
     */
    new = 0;
    for(m=0; m<10; m++){
        if(hdr[m] != ' '){
            new = 1;
            break;
        }
    }
    if(hdr[11] != ' '){
        werrstr("creadimage: bad format");
        return nil;
    }
    if(new){
        hdr[11] = '\0';
```

```
    if((chan = strtochan(hdr)) == 0){
        werrstr("creadimage: bad channel string %s", hdr);
        return nil;
    }
}else{
    ldepth = ((int)hdr[10])-'0';
    if(ldepth<0 || ldepth>3){
        werrstr("creadimage: bad ldepth %d", ldepth);
        return nil;
    }
    chan = drawld2chan[ldepth];
}
r.min.x=atoi(hdr+1*12);
r.min.y=atoi(hdr+2*12);
r.max.x=atoi(hdr+3*12);
r.max.y=atoi(hdr+4*12);
if(r.min.x>r.max.x || r.min.y>r.max.y){
    werrstr("creadimage: bad rectangle");
    return nil;
}

if(d){
    if(dolock)
        lockdisplay(d);
    i = allocimage(d, r, chan, 0, 0);
    ⟨creadimage() set malloc tag for debug 351c⟩
    if(dolock)
        unlockdisplay(d);
    if(i == nil)
        return nil;
}else{
    i = mallocz(sizeof(Image), 1);
    if(i == nil)
        return nil;
}
ncblock = _compblocksize(r, chantodepth(chan));
buf = malloc(ncblock);
if(buf == nil)
    goto Errout;
miny = r.min.y;
while(miny != r.max.y){
    if(readn(fd, hdr, 2*12) != 2*12){
    Errout:
        if(dolock)
            lockdisplay(d);
    Erroutlock:
        freeimage(i);
        if(dolock)
            unlockdisplay(d);
        free(buf);
        return nil;
    }
    maxy = atoi(hdr+0*12);
    nb = atoi(hdr+1*12);
    if(maxy<=miny || r.max.y<maxy){
        werrstr("creadimage: bad maxy %d", maxy);
        goto Errout;
    }
    if(nb<=0 || ncblock<nb){
        werrstr("creadimage: bad count %d", nb);
```

```
            goto Errout;
        }
        if(readn(fd, buf, nb)!=nb)
            goto Errout;
        if(d){
            if(dolock)
                lockdisplay(d);
            a = bufimage(i->display, 21+nb);
            if(a == nil)
                goto Erroutlock;
            a[0] = 'Y';
            BPLONG(a+1, i->id);
            BPLONG(a+5, r.min.x);
            BPLONG(a+9, miny);
            BPLONG(a+13, r.max.x);
            BPLONG(a+17, maxy);
            if(!new) /* old image: flip the data bits */
                _twiddlecompressed(buf, nb);
            memmove(a+21, buf, nb);
            if(dolock)
                unlockdisplay(d);
        }
        miny = maxy;
    }
    free(buf);
    return i;
}
```

⟨*function* _compblocksize 332a⟩≡ (417a)

```
int
_compblocksize(Rectangle r, int depth)
{
    int bpl;

    bpl = bytesperline(r, depth);
    bpl = 2*bpl; /* add plenty extra for blocking, etc. */
    if(bpl < NCBLOCK)
        return NCBLOCK;
    return bpl;
}
```

⟨*function* _twiddlecompressed 332b⟩≡ (417a)

```
/*
 * compressed data are seuences of byte codes.
 * if the first byte b has the 0x80 bit set, the next (b^0x80)+1 bytes
 * are data.  otherwise, it's two bytes specifying a previous string to repeat.
 */
void
_twiddlecompressed(uchar *buf, int n)
{
    uchar *ebuf;
    int j, k, c;

    ebuf = buf+n;
    while(buf < ebuf){
        c = *buf++;
        if(c >= 128){
            k = c-128+1;
            for(j=0; j<k; j++, buf++)
                *buf ^= 0xFF;
```

```
        }else
            buf++;
    }
}
```

Writing compressed images

⟨*constant* HSHIFT 333a⟩≡ (417a)

```
#define HSHIFT 3 /* HSHIFT==5 runs slightly faster, but hash table is 64x bigger */
```

⟨*constant* NHASH 333b⟩≡ (417a)

```
#define NHASH (1<<(HSHIFT*NMATCH))
```

⟨*constant* HMASK 333c⟩≡ (417a)

```
#define HMASK (NHASH-1)
```

⟨*function* hupdate 333d⟩≡ (417a)

```
#define hupdate(h, c) ((((h)<<HSHIFT)^(c))&HMASK)
```

⟨*struct* Hlist 333e⟩≡ (417a)

```
struct Hlist{
    uchar *s;
    Hlist *next, *prev;
};
```

⟨*constant* NMEM 333f⟩≡ (393a)

```
#define NMEM 1024  /* window size */
```

⟨*constant* NDUMP 333g⟩≡ (393a)

```
#define NDUMP 128  /* maximum length of dump */
```

⟨*constant* NCBLOCK 333h⟩≡ (393a)

```
#define NCBLOCK 6000  /* size of compressed blocks */
```

⟨*function* writeimage 333i⟩≡ (417a)

```
int
writeimage(fdt fd, Image *i, bool dolock)
{
    uchar *outbuf, *outp, *eout;  /* encoded data, pointer, end */
    uchar *loutp;    /* start of encoded line */
    Hlist *hash;    /* heads of hash chains of past strings */
    Hlist *chain, *hp;   /* hash chain members, pointer */
    Hlist *cp;    /* next Hlist to fall out of window */
    int h;     /* hash value */
    uchar *line, *eline;   /* input line, end pointer */
    uchar *data, *edata;   /* input buffer, end pointer */
    ulong n;    /* length of input buffer */
    ulong nb;    /* # of bytes returned by unloadimage */
    int bpl;    /* input line length */
    int offs, runlen;   /* offset, length of consumed data */
    uchar dumpbuf[NDUMP];   /* dump accumulator */
    int ndump;    /* length of dump accumulator */
    int miny, dy;    /* y values while unloading input */
    int chunk, ncblock;
    Rectangle r;
    uchar *p, *q, *s, *es, *t;
    char hdr[11+5*12+1];
    char cbuf[20];

    chunk = i->display->bufsize - 32; /* a little room for header */
```

```
r = i->r;
bpl = bytesperline(r, i->depth);
n = Dy(r)*bpl;
data = malloc(n);

ncblock = _compblocksize(r, i->depth);
outbuf = malloc(ncblock);
hash = malloc(NHASH*sizeof(Hlist));
chain = malloc(NMEM*sizeof(Hlist));
if(data == 0 || outbuf == 0 || hash == 0 || chain == 0){
ErrOut:
    free(data);
    free(outbuf);
    free(hash);
    free(chain);
    return -1;
}

for(miny = r.min.y; miny != r.max.y; miny += dy){
    dy = r.max.y-miny;
    if(dy*bpl > chunk)
        dy = chunk/bpl;

    if(dolock)
        lockdisplay(i->display);

    nb = unloadimage(i, Rect(r.min.x, miny, r.max.x, miny+dy),
        data+(miny-r.min.y)*bpl, dy*bpl);

    if(dolock)
        unlockdisplay(i->display);

    if(nb != dy*bpl)
        goto ErrOut;
}

sprint(hdr, "compressed\n%11s %11d %11d %11d %11d ",
    chantostr(cbuf, i->chan), r.min.x, r.min.y, r.max.x, r.max.y);
if(write(fd, hdr, 11+5*12) != 11+5*12)
    goto ErrOut;
edata = data+n;
eout = outbuf+ncblock;
line = data;
r.max.y = r.min.y;
while(line != edata){
    memset(hash, 0, NHASH*sizeof(Hlist));
    memset(chain, 0, NMEM*sizeof(Hlist));
    cp = chain;
    h = 0;
    outp = outbuf;
    for(n = 0; n != NMATCH; n++)
        h = hupdate(h, line[n]);
    loutp = outbuf;
    while(line != edata){
        ndump = 0;
        eline = line+bpl;
        for(p = line; p != eline; ){
            if(eline-p < NRUN)
                es = eline;
            else
```

```
        es = p+NRUN;
    q = 0;
    runlen = 0;
    for(hp = hash[h].next; hp; hp = hp->next){
        s = p + runlen;
        if(s >= es)
            continue;
        t = hp->s + runlen;
        for(; s >= p; s--)
            if(*s != *t--)
                goto matchloop;
        t += runlen+2;
        s += runlen+2;
        for(; s < es; s++)
            if(*s != *t++)
                break;
        n = s-p;
        if(n > runlen){
            runlen = n;
            q = hp->s;
            if(n == NRUN)
                break;
        }
matchloop: ;
    }
    if(runlen < NMATCH){
        if(ndump == NDUMP){
            if(eout-outp < ndump+1)
                goto Bfull;
            *outp++ = ndump-1+128;
            memmove(outp, dumpbuf, ndump);
            outp += ndump;
            ndump = 0;
        }
        dumpbuf[ndump++] = *p;
        runlen = 1;
    }
    else{
        if(ndump != 0){
            if(eout-outp < ndump+1)
                goto Bfull;
            *outp++ = ndump-1+128;
            memmove(outp, dumpbuf, ndump);
            outp += ndump;
            ndump = 0;
        }
        offs = p-q-1;
        if(eout-outp < 2)
            goto Bfull;
        *outp++ = ((runlen-NMATCH)<<2) + (offs>>8);
        *outp++ = offs&255;
    }
    for(q = p+runlen; p != q; p++){
        if(cp->prev)
            cp->prev->next = 0;
        cp->next = hash[h].next;
        cp->prev = &hash[h];
        if(cp->next)
            cp->next->prev = cp;
        cp->prev->next = cp;
```

```
                    cp->s = p;
                    if(++cp == &chain[NMEM])
                        cp = chain;
                    if(edata-p > NMATCH)
                        h = hupdate(h, p[NMATCH]);
                }
            }
            if(ndump != 0){
                if(eout-outp < ndump+1)
                    goto Bfull;
                *outp++ = ndump-1+128;
                memmove(outp, dumpbuf, ndump);
                outp += ndump;
            }
            line = eline;
            loutp = outp;
            r.max.y++;
        }
    Bfull:
        if(loutp == outbuf)
            goto ErrOut;
        n = loutp-outbuf;
        sprint(hdr, "%11d %11ld ", r.max.y, n);
        write(fd, hdr, 2*12);
        write(fd, outbuf, n);
        r.min.y = r.max.y;
    }
    free(data);
    free(outbuf);
    free(hash);
    free(chain);
    return 0;
}
```

Memimage functions

⟨*function* creadmemimage 336⟩≡ (436a)

```
Memimage*
creadmemimage(int fd)
{
    char hdr[5*12+1];
    Rectangle r;
    int m, nb, miny, maxy, new, ldepth, ncblock;
    uchar *buf;
    Memimage *i;
    ulong chan;

    if(readn(fd, hdr, 5*12) != 5*12){
        werrstr("readmemimage: short header (2)");
        return nil;
    }

    /*
     * distinguish new channel descriptor from old ldepth.
     * channel descriptors have letters as well as numbers,
     * while ldepths are a single digit formatted as %-11d.
     */
    new = 0;
    for(m=0; m<10; m++){
```

```
    if(hdr[m] != ' '){
        new = 1;
        break;
    }
}
if(hdr[11] != ' '){
    werrstr("creadimage: bad format");
    return nil;
}
if(new){
    hdr[11] = '\0';
    if((chan = strtochan(hdr)) == 0){
        werrstr("creadimage: bad channel string %s", hdr);
        return nil;
    }
}else{
    ldepth = ((int)hdr[10])-'0';
    if(ldepth<0 || ldepth>3){
        werrstr("creadimage: bad ldepth %d", ldepth);
        return nil;
    }
    chan = drawld2chan[ldepth];
}
r.min.x=atoi(hdr+1*12);
r.min.y=atoi(hdr+2*12);
r.max.x=atoi(hdr+3*12);
r.max.y=atoi(hdr+4*12);
if(r.min.x>r.max.x || r.min.y>r.max.y){
    werrstr("creadimage: bad rectangle");
    return nil;
}

i = allocmemimage(r, chan);
if(i == nil)
    return nil;
ncblock = _compblocksize(r, i->depth);
buf = malloc(ncblock);
if(buf == nil)
    goto Errout;
miny = r.min.y;
while(miny != r.max.y){
    if(readn(fd, hdr, 2*12) != 2*12){
    Shortread:
        werrstr("readmemimage: short read");
    Errout:
        freememimage(i);
        free(buf);
        return nil;
    }
    maxy = atoi(hdr+0*12);
    nb = atoi(hdr+1*12);
    if(maxy<=miny || r.max.y<maxy){
        werrstr("readimage: bad maxy %d", maxy);
        goto Errout;
    }
    if(nb<=0 || ncblock<nb){
        werrstr("readimage: bad count %d", nb);
        goto Errout;
    }
    if(readn(fd, buf, nb)!=nb)
```

```
            goto Shortread;
        if(!new) /* old image: flip the data bits */
            _twiddlecompressed(buf, nb);
        cloadmemimage(i, Rect(r.min.x, miny, r.max.x, maxy), buf, nb);
        miny = maxy;
    }
    free(buf);
    return i;
}
```

15.12 Other image format

15.13 Old image format

⟨readimage() *other locals* 338a⟩≡ (180e) 338c ▷

```
bool new;
```

⟨readimage() *check if new or old format, set new* 338b⟩≡ (180e)

```
/*
 * distinguish new channel descriptor from old ldepth.
 * channel descriptors have letters as well as numbers,
 * while ldepths are a single digit formatted as %-11d.
 */
new = false;
for(m=0; m<10; m++){
    if(hdr[m] != ' '){
        new = true;
        break;
    }
}
```

⟨readimage() *other locals* 338c⟩+≡ (180e) ◁ 338a

```
int ldepth;
```

⟨readimage() *if old format* 338d⟩≡ (180e)

```
if(!new){
    ldepth = ((int)hdr[10])-'0';
    if(ldepth<0 || ldepth>3){
        werrstr("readimage: bad ldepth %d", ldepth);
        return nil;
    }
    chan = drawld2chan[ldepth];
}
```

⟨readimage() *if old format, flip all bits* 338e⟩≡ (180e)

```
if(!new) /* an old image: must flip all the bits */
    for(j=0; j<chunk; j++)
        tmp[j] ^= 0xFF;
```

15.14 Memimage duplicated API

⟨*function* getmemdefont 338f⟩≡ (428b)

```
Memsubfont*
getmemdefont(void)
{
    char *hdr, *p;
```

```
    int n;
    Fontchar *fc;
    Memsubfont *f;
    int ld;
    Rectangle r;
    Memdata *md;
    Memimage *i;

    /*
     * make sure data is word-aligned.  this is true with Plan 9 compilers
     * but not in general.  the byte order is right because the data is
     * declared as char*, not ulong*.
     */
    p = (char*)defontdata;
    n = (uintptr)p & 3;
    if(n != 0){
        memmove(p+(4-n), p, sizeofdefont-n);
        p += 4-n;
    }
    ld = atoi(p+0*12);
    r.min.x = atoi(p+1*12);
    r.min.y = atoi(p+2*12);
    r.max.x = atoi(p+3*12);
    r.max.y = atoi(p+4*12);

    md = mallocz(sizeof(Memdata), 1);
    if(md == nil)
        return nil;

    p += 5*12;

    md->base = nil;  /* so freememimage doesn't free p */
    md->bdata = (uchar*)p; /* ick */
    md->ref = 1;
    md->allocd = true;  /* so freememimage does free md */

    i = allocmemimaged(r, drawld2chan[ld], md);
    if(i == nil){
        free(md);
        return nil;
    }

    hdr = p + Dy(r) * i->width * sizeof(ulong);
    n = atoi(hdr);
    p = hdr+3*12;
    fc = malloc(sizeof(Fontchar)*(n+1));
    if(fc == nil){
        freememimage(i);
        return 0;
    }
    _unpackinfo(fc, (uchar*)p, n);
    f = allocmemsubfont("*default*", n, atoi(hdr+12), atoi(hdr+24), fc, i);
    if(f == nil){
        freememimage(i);
        free(fc);
        return 0;
    }
    return f;
}
```

⟨*struct* Memsubfont 340a⟩≡ (393b)

```
/*
 * Subfonts
 *
 * given char c, Subfont *f, Fontchar *i, and Point p, one says
 * i = f->info+c;
 * draw(b, Rect(p.x+i->left, p.y+i->top,
 *  p.x+i->left+((i+1)->x-i->x), p.y+i->bottom),
 *  color, f->bits, Pt(i->x, i->top));
 * p.x += i->width;
 * to draw characters in the specified color (itself a Memimage) in Memimage b.
 */
struct Memsubfont
{
    char *name;

    Memimage *bits;  /* of font */

    short n;  /* number of chars in font */
    Fontchar *info;  /* n+1 character descriptors */

    uchar height;  /* height of bitmap */
    char ascent;  /* top of bitmap to baseline */
};
```

⟨*function* allocmemsubfont 340b⟩≡ (436c)

```
Memsubfont*
allocmemsubfont(char *name, int n, int height, int ascent, Fontchar *info, Memimage *i)
{
    Memsubfont *f;

    f = malloc(sizeof(Memsubfont));
    if(f == 0)
        return 0;
    f->n = n;
    f->height = height;
    f->ascent = ascent;
    f->info = info;
    f->bits = i;
    if(name)
        f->name = strdup(name);
    else
        f->name = 0;
    return f;
}
```

⟨*function* memsubfontwidth 340c⟩≡ (436b)

```
Point
memsubfontwidth(Memsubfont *f, char *cs)
{
    Rune c;
    Point p;
    uchar *s;
    Fontchar *i;
    int w, width;

    p = Pt(0, f->height);
    s = (uchar*)cs;
    for(; c=*s; p.x+=width){
        width = 0;
```

```
        if(c < Runeself)
            s++;
        else{
            w = chartorune(&c, (char*)s);
            if(w == 0){
                s++;
                continue;
            }
            s += w;
        }
        if(c >= f->n)
            continue;
        i = f->info+c;
        width = i->width;
    }
    return p;
}
```

⟨*function* memimagestring 341a⟩≡ (436b)

```
Point
memimagestring(Memimage *b, Point p, Memimage *color, Point cp, Memsubfont *f, char *cs)
{
    int w, width;
    uchar *s;
    Rune c;
    Fontchar *i;

    s = (uchar*)cs;
    for(; c=*s; p.x+=width, cp.x+=width){
        width = 0;
        if(c < Runeself)
            s++;
        else{
            w = chartorune(&c, (char*)s);
            if(w == 0){
                s++;
                continue;
            }
            s += w;
        }
        if(c >= f->n)
            continue;
        i = f->info+c;
        width = i->width;
        memdraw(b, Rect(p.x+i->left, p.y+i->top, p.x+i->left+(i[1].x-i[0].x), p.y+i->bottom),
            color, cp, f->bits, Pt(i->x, i->top), SoverD);
    }
    return p;
}
```

⟨*function* readmemimage 341b⟩≡ (436a)

```
Memimage*
readmemimage(fdt fd)
{
    char hdr[5*12+1];
    int dy;
    ulong chan;
    uint l, n;
    int m, j;
    int new, miny, maxy;
```

```
Rectangle r;
uchar *tmp;
int ldepth, chunk;
Memimage *i;

if(readn(fd, hdr, 11) != 11){
    werrstr("readimage: short header");
    return nil;
}
if(memcmp(hdr, "compressed\n", 11) == 0)
    return creadmemimage(fd);

if(readn(fd, hdr+11, 5*12-11) != 5*12-11){
    werrstr("readimage: short header (2)");
    return nil;
}

/*
 * distinguish new channel descriptor from old ldepth.
 * channel descriptors have letters as well as numbers,
 * while ldepths are a single digit formatted as %-11d.
 */
new = 0;
for(m=0; m<10; m++){
    if(hdr[m] != ' '){
        new = 1;
        break;
    }
}
if(hdr[11] != ' '){
    werrstr("readimage: bad format");
    return nil;
}
if(new){
    hdr[11] = '\0';
    if((chan = strtochan(hdr)) == 0){
        werrstr("readimage: bad channel string %s", hdr);
        return nil;
    }
}else{
    ldepth = ((int)hdr[10])-'0';
    if(ldepth<0 || ldepth>3){
        werrstr("readimage: bad ldepth %d", ldepth);
        return nil;
    }
    chan = drawld2chan[ldepth];
}

r.min.x = atoi(hdr+1*12);
r.min.y = atoi(hdr+2*12);
r.max.x = atoi(hdr+3*12);
r.max.y = atoi(hdr+4*12);
if(r.min.x>r.max.x || r.min.y>r.max.y){
    werrstr("readimage: bad rectangle");
    return nil;
}

miny = r.min.y;
maxy = r.max.y;
```

```
    l = bytesperline(r, chantodepth(chan));
    i = allocmemimage(r, chan);
    if(i == nil)
        return nil;
    chunk = 32*1024;
    if(chunk < l)
        chunk = l;
    tmp = malloc(chunk);
    if(tmp == nil)
        goto Err;
    while(maxy > miny){
        dy = maxy - miny;
        if(dy*l > chunk)
            dy = chunk/l;
        if(dy <= 0){
            werrstr("readmemimage: image too wide for buffer");
            goto Err;
        }
        n = dy*l;
        m = readn(fd, tmp, n);
        if(m != n){
            werrstr("readmemimage: read count %d not %d: %r", m, n);
   Err:
    freememimage(i);
            free(tmp);
            return nil;
        }
        if(!new) /* an old image: must flip all the bits */
            for(j=0; j<chunk; j++)
                tmp[j] ^= 0xFF;

        if(loadmemimage(i, Rect(r.min.x, miny, r.max.x, miny+dy), tmp, chunk) <= 0)
            goto Err;
        miny += dy;
    }
    free(tmp);
    return i;
}
```

⟨*constant* CHUNK 343a⟩≡ (436a)

```
#define CHUNK 8000
```

⟨*constant* HSHIFT*(*(lib_graphics/libmemdraw/write.c)*)* 343b⟩≡ (436a)

```
#define HSHIFT 3 /* HSHIFT==5 runs slightly faster, but hash table is 64x bigger */
```

⟨*constant* NHASH*(*(lib_graphics/libmemdraw/write.c)*)* 343c⟩≡ (436a)

```
#define NHASH (1<<(HSHIFT*NMATCH))
```

⟨*constant* HMASK*(*(lib_graphics/libmemdraw/write.c)*)* 343d⟩≡ (436a)

```
#define HMASK (NHASH-1)
```

⟨*function* hupdate*(*(lib_graphics/libmemdraw/write.c)*)* 343e⟩≡ (436a)

```
#define hupdate(h, c) ((((h)<<HSHIFT)^(c))&HMASK)
```

⟨*struct* Hlist*(*(lib_graphics/libmemdraw/write.c)*)* 343f⟩≡ (436a)

```
struct Hlist{
    uchar *s;
    Hlist *next, *prev;
};
```

⟨*function* writememimage 344⟩≡ (436a)

```
int
writememimage(int fd, Memimage *i)
{
    uchar *outbuf, *outp, *eout;  /* encoded data, pointer, end */
    uchar *loutp;    /* start of encoded line */
    Hlist *hash;    /* heads of hash chains of past strings */
    Hlist *chain, *hp;   /* hash chain members, pointer */
    Hlist *cp;    /* next Hlist to fall out of window */
    int h;     /* hash value */
    uchar *line, *eline;   /* input line, end pointer */
    uchar *data, *edata;   /* input buffer, end pointer */
    ulong n;    /* length of input buffer */
    ulong nb;    /* # of bytes returned by unloadimage */
    int bpl;    /* input line length */
    int offs, runlen;   /* offset, length of consumed data */
    uchar dumpbuf[NDUMP];   /* dump accumulator */
    int ndump;    /* length of dump accumulator */
    int miny, dy;    /* y values while unloading input */
    int ncblock;    /* size of compressed blocks */
    Rectangle r;
    uchar *p, *q, *s, *es, *t;
    char hdr[11+5*12+1];
    char cbuf[20];

    r = i->r;
    bpl = bytesperline(r, i->depth);
    n = Dy(r)*bpl;
    data = malloc(n);
    ncblock = _compblocksize(r, i->depth);
    outbuf = malloc(ncblock);
    hash = malloc(NHASH*sizeof(Hlist));
    chain = malloc(NMEM*sizeof(Hlist));
    if(data == 0 || outbuf == 0 || hash == 0 || chain == 0){
    ErrOut:
        free(data);
        free(outbuf);
        free(hash);
        free(chain);
        return -1;
    }
    for(miny = r.min.y; miny != r.max.y; miny += dy){
        dy = r.max.y-miny;
        if(dy*bpl > CHUNK)
            dy = CHUNK/bpl;
        nb = unloadmemimage(i, Rect(r.min.x, miny, r.max.x, miny+dy),
            data+(miny-r.min.y)*bpl, dy*bpl);
        if(nb != dy*bpl)
            goto ErrOut;
    }
    sprint(hdr, "compressed\n%11s %11d %11d %11d %11d ",
        chantostr(cbuf, i->chan), r.min.x, r.min.y, r.max.x, r.max.y);
    if(write(fd, hdr, 11+5*12) != 11+5*12)
        goto ErrOut;
    edata = data+n;
    eout = outbuf+ncblock;
    line = data;
    r.max.y = r.min.y;
    while(line != edata){
        memset(hash, 0, NHASH*sizeof(Hlist));
```

```
memset(chain, 0, NMEM*sizeof(Hlist));
cp = chain;
h = 0;
outp = outbuf;
for(n = 0; n != NMATCH; n++)
    h = hupdate(h, line[n]);
loutp = outbuf;
while(line != edata){
    ndump = 0;
    eline = line+bpl;
    for(p = line; p != eline; ){
        if(eline-p < NRUN)
            es = eline;
        else
            es = p+NRUN;
        q = 0;
        runlen = 0;
        for(hp = hash[h].next; hp; hp = hp->next){
            s = p + runlen;
            if(s >= es)
                continue;
            t = hp->s + runlen;
            for(; s >= p; s--)
                if(*s != *t--)
                    goto matchloop;
            t += runlen+2;
            s += runlen+2;
            for(; s < es; s++)
                if(*s != *t++)
                    break;
            n = s-p;
            if(n > runlen){
                runlen = n;
                q = hp->s;
                if(n == NRUN)
                    break;
            }
    matchloop: ;
        }
        if(runlen < NMATCH){
            if(ndump == NDUMP){
                if(eout-outp < ndump+1)
                    goto Bfull;
                *outp++ = ndump-1+128;
                memmove(outp, dumpbuf, ndump);
                outp += ndump;
                ndump = 0;
            }
            dumpbuf[ndump++] = *p;
            runlen = 1;
        }
        else{
            if(ndump != 0){
                if(eout-outp < ndump+1)
                    goto Bfull;
                *outp++ = ndump-1+128;
                memmove(outp, dumpbuf, ndump);
                outp += ndump;
                ndump = 0;
            }
```

```
                offs = p-q-1;
                if(eout-outp < 2)
                    goto Bfull;
                *outp++ = ((runlen-NMATCH)<<2) + (offs>>8);
                *outp++ = offs&255;
            }
            for(q = p+runlen; p != q; p++){
                if(cp->prev)
                    cp->prev->next = 0;
                cp->next = hash[h].next;
                cp->prev = &hash[h];
                if(cp->next)
                    cp->next->prev = cp;
                cp->prev->next = cp;
                cp->s = p;
                if(++cp == &chain[NMEM])
                    cp = chain;
                if(edata-p > NMATCH)
                    h = hupdate(h, p[NMATCH]);
            }
        }
        if(ndump != 0){
            if(eout-outp < ndump+1)
                goto Bfull;
            *outp++ = ndump-1+128;
            memmove(outp, dumpbuf, ndump);
            outp += ndump;
        }
        line = eline;
        loutp = outp;
        r.max.y++;
    }
Bfull:
    if(loutp == outbuf)
        goto ErrOut;
    n = loutp-outbuf;
    sprint(hdr, "%11d %11ld ", r.max.y, n);
    write(fd, hdr, 2*12);
    write(fd, outbuf, n);
    r.min.y = r.max.y;
}
free(data);
free(outbuf);
free(hash);
free(chain);
return 0;
}
```

15.15 Video and animation

Chapter 16

Conclusion

Appendix A

Debugging

⟨*global* _drawdebug 348a⟩≡ (402d)
```
bool _drawdebug = false;
```

⟨doflush() *if* _drawdebug 348b⟩≡ (70e)
```
if(_drawdebug)
    fprint(2, "flushimage fail: d=%p: n=%d nn=%d %r\n", d, n, nn); /**/
```

⟨*global* debuglockdisplay 348c⟩≡ (400c)
```
bool  debuglockdisplay = false;
```

⟨lockdisplay() *if debuglockdisplay* 348d⟩≡ (311b)
```
if(debuglockdisplay){
    /* avoid busy looping; it's rare we collide anyway */
    while(!canqlock(&disp->qlock)){
        fprint(1, "proc %d waiting for display lock...\n", getpid());
        sleep(1000);
    }
}
```

A.1 drawsetdebug()

⟨*function* drawsetdebug 348e⟩≡ (402d)
```
void
drawsetdebug(bool v)
{
    byte *a;
    a = bufimage(display, 1+1);
    ⟨drawsetdebug() sanity check a 348f⟩
    a[0] = 'D';
    a[1] = v;
}
```

⟨drawsetdebug() *sanity check a* 348f⟩≡ (348e)
```
if(a == nil){
    fprint(2, "drawsetdebug: %r\n");
    return;
}
```

⟨*global* drawdebug 348g⟩≡ (432)
```
bool drawdebug;
```

⟨drawmesg() *cases* 349a⟩+≡ (71) ◁218c

```
/* toggle debugging: 'D' val[1] */
case 'D':
    printmesg(fmt="b", a, false);
    m = 1+1;
    ⟨drawmesg() sanity check n with m 75d⟩
    drawdebug = a[1];
    continue;
```

⟨*macro* DBG1 349b⟩≡ (458a 432 431 429b)

```
#define DBG1 if(0) print
```

⟨*macro* DBG 349c⟩≡ (458a 432)

```
#define DBG if(0) print
```

A.2 Dumpers

A.2.1 Points and rectangles

⟨initdisplay() *install dumpers* 349d⟩≡ (56)

```
fmtinstall('P', Pfmt);
fmtinstall('R', Rfmt);
```

⟨memimageinit() *install dumpers* 349e⟩≡ (48a) 349h▷

```
fmtinstall('P', Pfmt);
fmtinstall('R', Rfmt);
```

⟨*function* Pfmt 349f⟩≡ (403a)

```
int
Pfmt(Fmt *f)
{
    Point p;

    p = va_arg(f->args, Point);
    return fmtprint(f, "[%d %d]", p.x, p.y);
}
```

⟨*function* Rfmt 349g⟩≡ (403a)

```
int
Rfmt(Fmt *f)
{
    Rectangle r;

    r = va_arg(f->args, Rectangle);
    return fmtprint(f, "%P %P", r.min, r.max);
}
```

⟨memimageinit() *install dumpers* 349h⟩+≡ (48a) ◁349e

```
fmtinstall('b', _ifmt);
```

A.2.2 Channels

⟨*function* chantostr 350a⟩≡ (400b)

```
char*
chantostr(char *buf, ulong cc)
{
    ulong c, rc;
    char *p;

    if(chantodepth(cc) == 0)
        return nil;

    /* reverse the channel descriptor so we can easily generate the string in the right order */
    rc = 0;
    for(c=cc; c; c>>=8){
        rc <<= 8;
        rc |= c&0xFF;
    }

    p = buf;
    for(c=rc; c; c>>=8) {
        *p++ = channames[TYPE(c)];
        *p++ = '0'+NBITS(c);
    }
    *p = 0;

    return buf;
}
```

A.2.3 Drawing commands

⟨*function* printmesg 350b⟩≡ (480a)

```
static void
printmesg(char *fmt, uchar *a, bool plsprnt)
{
    char buf[256];
    char *p, *q;
    int s, left;

    if(!plsprnt && !drawdebug){ //old: 1 || !plsprnt
        SET(s,q,p);
        USED(fmt, a, buf, p, q, s);
        return;
    }
    // else, plsprnt || drawdebug

    q = buf;
    *q++ = *a++;
    for(p=fmt; *p; p++){
        left = sizeof buf - 2 - (q - buf);  /* 2 for \n\0 */
        switch(*p){
        case 'l':
            q += snprint(q, left, " %ld", (long)BGLONG(a));
            a += 4;
            break;
        case 'L':
            q += snprint(q, left, " %.8lux", (ulong)BGLONG(a));
            a += 4;
            break;
```

```
        case 'R':
            q += snprint(q, left, " [%d %d %d %d]", BGLONG(a),
                BGLONG(a+4), BGLONG(a+8), BGLONG(a+12));
            a += 16;
            break;
        case 'P':
            q += snprint(q, left, " [%d %d]", BGLONG(a), BGLONG(a+4));
            a += 8;
            break;
        case 'b':
            q += snprint(q, left, " %d", *a++);
            break;
        case 's':
            q += snprint(q, left, " %d", BGSHORT(a));
            a += 2;
            break;
        case 'S':
            q += snprint(q, left, " %.4ux", BGSHORT(a));
            a += 2;
            break;
        }
    }
    *q++ = '\n';
    *q = '\0';
    iprint("%.*s", (int)(q-buf), buf);
}
```

A.3 Memory tags

⟨allocimage() *set malloc tag for debug* 351a⟩≡ (72a)

```
if (i)
    setmalloctag(i, getcallerpc(&d));
```

⟨_getsubfont() *set malloc tag for debug* 351b⟩≡ (169a)

```
setmalloctag(f, getcallerpc(&d));
```

⟨creadimage() *set malloc tag for debug* 351c⟩≡ (330b)

```
setmalloctag(i, getcallerpc(&d));
```

Appendix B

Error Management

B.1 User side

⟨*type* Errorfn 352a⟩≡ (386d)
```
typedef void (*Errorfn)(Display*, char*);
```

⟨Display *other fields* 352b⟩+≡ (34e) ◁235b
```
Errorfn error;
```

⟨*function* drawerror 352c⟩≡ (401b)
```
void
drawerror(Display *d, char *s)
{
    char err[ERRMAX];

    if(d && d->error)
        d->error(d, s);
    else{
        errstr(err, sizeof err);
        fprint(STDERR, "draw: %s: %s\n", s, err);
        exits(s); // extreme!
    }
}
```

B.2 Kernel side

⟨*global* Enodrawimage 352d⟩≡ (479b)
```
char Enodrawimage[] =   "unknown id for draw image";
```

⟨*global* Enodrawscreen 352e⟩≡ (479b)
```
char Enodrawscreen[] =  "unknown id for draw screen";
```

⟨*global* Eshortdraw 352f⟩≡ (479b)
```
char Eshortdraw[] = "short draw message";
```

⟨*global* Eshortread 352g⟩≡ (479b)
```
char Eshortread[] = "draw read too short";
```

⟨*global* Eimageexists 352h⟩≡ (479b)
```
char Eimageexists[] =   "image id in use";
```

⟨*global* Escreenexists 352i⟩≡ (479b)
```
char Escreenexists[] =  "screen id in use";
```

⟨*global* Edrawmem 353a⟩≡ (479b)
```
char Edrawmem[] =   "image memory allocation failed";
```

⟨*global* Ereadoutside 353b⟩≡ (479b)
```
char Ereadoutside[] =   "readimage outside image";
```

⟨*global* Ewriteoutside 353c⟩≡ (479b)
```
char Ewriteoutside[] =  "writeimage outside image";
```

⟨*global* Enotfont 353d⟩≡ (479b)
```
char Enotfont[] =   "image not a font";
```

⟨*global* Eindex 353e⟩≡ (479b)
```
char Eindex[] =     "character index out of range";
```

⟨*global* Enoclient 353f⟩≡ (479b)
```
char Enoclient[] =  "no such draw client";
```

⟨*global* Enameused 353g⟩≡ (479b)
```
char Enameused[] =  "image name in use";
```

⟨*global* Enoname 353h⟩≡ (479b)
```
char Enoname[] =    "no image with that name";
```

⟨*global* Eoldname 353i⟩≡ (479b)
```
char Eoldname[] =   "named image no longer valid";
```

⟨*global* Enamed 353j⟩≡ (479b)
```
char Enamed[] =     "image already has name";
```

⟨*global* Ewrongname 353k⟩≡ (479b)
```
char Ewrongname[] =     "wrong name for image";
```

Appendix C

Mathematics

C.1 Trigonometric functions

⟨*global* sinus 354⟩≡ (399a)

```
/*
 * Integer sine and cosine for integral degree argument.
 * Tables computed by (sin,cos)(PI*d/180).
 */
static short sinus[91] = {
    0, /* 0 */
    18, /* 1 */
    36, /* 2 */
    54, /* 3 */
    71, /* 4 */
    89, /* 5 */
    107, /* 6 */
    125, /* 7 */
    143, /* 8 */
    160, /* 9 */
    178, /* 10 */
    195, /* 11 */
    213, /* 12 */
    230, /* 13 */
    248, /* 14 */
    265, /* 15 */
    282, /* 16 */
    299, /* 17 */
    316, /* 18 */
    333, /* 19 */
    350, /* 20 */
    367, /* 21 */
    384, /* 22 */
    400, /* 23 */
    416, /* 24 */
    433, /* 25 */
    449, /* 26 */
    465, /* 27 */
    481, /* 28 */
    496, /* 29 */
    512, /* 30 */
    527, /* 31 */
    543, /* 32 */
    558, /* 33 */
    573, /* 34 */
    587, /* 35 */
```

```
    602, /* 36 */
    616, /* 37 */
    630, /* 38 */
    644, /* 39 */
    658, /* 40 */
    672, /* 41 */
    685, /* 42 */
    698, /* 43 */
    711, /* 44 */
    724, /* 45 */
    737, /* 46 */
    749, /* 47 */
    761, /* 48 */
    773, /* 49 */
    784, /* 50 */
    796, /* 51 */
    807, /* 52 */
    818, /* 53 */
    828, /* 54 */
    839, /* 55 */
    849, /* 56 */
    859, /* 57 */
    868, /* 58 */
    878, /* 59 */
    887, /* 60 */
    896, /* 61 */
    904, /* 62 */
    912, /* 63 */
    920, /* 64 */
    928, /* 65 */
    935, /* 66 */
    943, /* 67 */
    949, /* 68 */
    956, /* 69 */
    962, /* 70 */
    968, /* 71 */
    974, /* 72 */
    979, /* 73 */
    984, /* 74 */
    989, /* 75 */
    994, /* 76 */
    998, /* 77 */
    1002, /* 78 */
    1005, /* 79 */
    1008, /* 80 */
    1011, /* 81 */
    1014, /* 82 */
    1016, /* 83 */
    1018, /* 84 */
    1020, /* 85 */
    1022, /* 86 */
    1023, /* 87 */
    1023, /* 88 */
    1024, /* 89 */
    1024, /* 90 */
  };
```

⟨*function* icossin 355⟩≡ (399a)

```
  void
  icossin(int deg, int *cosp, int *sinp)
```

```
{
    int sinsign, cossign;
    short *stp, *ctp;

    deg %= 360;
    if(deg < 0)
        deg += 360;
    sinsign = 1;
    cossign = 1;
    stp = 0;
    ctp = 0;
    switch(deg/90){
    case 2:
        sinsign = -1;
        cossign = -1;
        deg -= 180;
        /* fall through */
    case 0:
        stp = &sinus[deg];
        ctp = &sinus[90-deg];
        break;
    case 3:
        sinsign = -1;
        cossign = -1;
        deg -= 180;
        /* fall through */
    case 1:
        deg = 180-deg;
        cossign = -cossign;
        stp = &sinus[deg];
        ctp = &sinus[90-deg];
        break;
    }
    *sinp = sinsign*stp[0];
    *cosp = cossign*ctp[0];
}
```

⟨*global* sinus*(*(lib_graphics/libdraw/icossin2.c)*)* 356⟩≡ (399b)

```
/*
 * Sine and Cosine of arctangents, calculated by
 *   (sin(atan(index/100.0))*1024.+0.5)
 *   (cos(atan(index/100.0))*1024.+0.5)
 * To use, get rational tangent between 0<=tan<=1, scale by 100,
 * and look up sin and cos, and use linear interpolation.  divide by 1024.
 * Maximum error is 0.0020.  Without linear interpolation, it's 0.010.
 */
static
short sinus[] = {
    0, /* 0.00 */
    10, /* 0.01 */
    20, /* 0.02 */
    31, /* 0.03 */
    41, /* 0.04 */
    51, /* 0.05 */
    61, /* 0.06 */
    72, /* 0.07 */
    82, /* 0.08 */
    92, /* 0.09 */
    102, /* 0.10 */
    112, /* 0.11 */
```

```
122, /* 0.12 */
132, /* 0.13 */
142, /* 0.14 */
152, /* 0.15 */
162, /* 0.16 */
172, /* 0.17 */
181, /* 0.18 */
191, /* 0.19 */
201, /* 0.20 */
210, /* 0.21 */
220, /* 0.22 */
230, /* 0.23 */
239, /* 0.24 */
248, /* 0.25 */
258, /* 0.26 */
267, /* 0.27 */
276, /* 0.28 */
285, /* 0.29 */
294, /* 0.30 */
303, /* 0.31 */
312, /* 0.32 */
321, /* 0.33 */
330, /* 0.34 */
338, /* 0.35 */
347, /* 0.36 */
355, /* 0.37 */
364, /* 0.38 */
372, /* 0.39 */
380, /* 0.40 */
388, /* 0.41 */
397, /* 0.42 */
405, /* 0.43 */
412, /* 0.44 */
420, /* 0.45 */
428, /* 0.46 */
436, /* 0.47 */
443, /* 0.48 */
451, /* 0.49 */
458, /* 0.50 */
465, /* 0.51 */
472, /* 0.52 */
480, /* 0.53 */
487, /* 0.54 */
493, /* 0.55 */
500, /* 0.56 */
507, /* 0.57 */
514, /* 0.58 */
520, /* 0.59 */
527, /* 0.60 */
533, /* 0.61 */
540, /* 0.62 */
546, /* 0.63 */
552, /* 0.64 */
558, /* 0.65 */
564, /* 0.66 */
570, /* 0.67 */
576, /* 0.68 */
582, /* 0.69 */
587, /* 0.70 */
593, /* 0.71 */
```

```
    598, /* 0.72 */
    604, /* 0.73 */
    609, /* 0.74 */
    614, /* 0.75 */
    620, /* 0.76 */
    625, /* 0.77 */
    630, /* 0.78 */
    635, /* 0.79 */
    640, /* 0.80 */
    645, /* 0.81 */
    649, /* 0.82 */
    654, /* 0.83 */
    659, /* 0.84 */
    663, /* 0.85 */
    668, /* 0.86 */
    672, /* 0.87 */
    676, /* 0.88 */
    681, /* 0.89 */
    685, /* 0.90 */
    689, /* 0.91 */
    693, /* 0.92 */
    697, /* 0.93 */
    701, /* 0.94 */
    705, /* 0.95 */
    709, /* 0.96 */
    713, /* 0.97 */
    717, /* 0.98 */
    720, /* 0.99 */
    724, /* 1.00 */
    728, /* 1.01 */
};
```

⟨*global* cosinus 358⟩≡ (399b)

```
static
short cosinus[] = {
    1024, /* 0.00 */
    1024, /* 0.01 */
    1024, /* 0.02 */
    1024, /* 0.03 */
    1023, /* 0.04 */
    1023, /* 0.05 */
    1022, /* 0.06 */
    1022, /* 0.07 */
    1021, /* 0.08 */
    1020, /* 0.09 */
    1019, /* 0.10 */
    1018, /* 0.11 */
    1017, /* 0.12 */
    1015, /* 0.13 */
    1014, /* 0.14 */
    1013, /* 0.15 */
    1011, /* 0.16 */
    1010, /* 0.17 */
    1008, /* 0.18 */
    1006, /* 0.19 */
    1004, /* 0.20 */
    1002, /* 0.21 */
    1000, /* 0.22 */
    998, /* 0.23 */
    996, /* 0.24 */
```

```
993, /* 0.25 */
991, /* 0.26 */
989, /* 0.27 */
986, /* 0.28 */
983, /* 0.29 */
981, /* 0.30 */
978, /* 0.31 */
975, /* 0.32 */
972, /* 0.33 */
969, /* 0.34 */
967, /* 0.35 */
963, /* 0.36 */
960, /* 0.37 */
957, /* 0.38 */
954, /* 0.39 */
951, /* 0.40 */
947, /* 0.41 */
944, /* 0.42 */
941, /* 0.43 */
937, /* 0.44 */
934, /* 0.45 */
930, /* 0.46 */
927, /* 0.47 */
923, /* 0.48 */
920, /* 0.49 */
916, /* 0.50 */
912, /* 0.51 */
909, /* 0.52 */
905, /* 0.53 */
901, /* 0.54 */
897, /* 0.55 */
893, /* 0.56 */
890, /* 0.57 */
886, /* 0.58 */
882, /* 0.59 */
878, /* 0.60 */
874, /* 0.61 */
870, /* 0.62 */
866, /* 0.63 */
862, /* 0.64 */
859, /* 0.65 */
855, /* 0.66 */
851, /* 0.67 */
847, /* 0.68 */
843, /* 0.69 */
839, /* 0.70 */
835, /* 0.71 */
831, /* 0.72 */
827, /* 0.73 */
823, /* 0.74 */
819, /* 0.75 */
815, /* 0.76 */
811, /* 0.77 */
807, /* 0.78 */
804, /* 0.79 */
800, /* 0.80 */
796, /* 0.81 */
792, /* 0.82 */
788, /* 0.83 */
784, /* 0.84 */
```

```
    780, /* 0.85 */
    776, /* 0.86 */
    773, /* 0.87 */
    769, /* 0.88 */
    765, /* 0.89 */
    761, /* 0.90 */
    757, /* 0.91 */
    754, /* 0.92 */
    750, /* 0.93 */
    746, /* 0.94 */
    742, /* 0.95 */
    739, /* 0.96 */
    735, /* 0.97 */
    731, /* 0.98 */
    728, /* 0.99 */
    724, /* 1.00 */
    720, /* 1.01 */
};
```

⟨*function* icossin2 360⟩≡ (399b)

```
void
icossin2(int x, int y, int *cosp, int *sinp)
{
    int sinsign, cossign, tan, tan10, rem;
    short *stp, *ctp;

    if(x == 0){
        if(y >= 0)
            *sinp = ICOSSCALE, *cosp = 0;
        else
            *sinp = -ICOSSCALE, *cosp = 0;
        return;
    }
    sinsign = cossign = 1;
    if(x < 0){
        cossign = -1;
        x = -x;
    }
    if(y < 0){
        sinsign = -1;
        y = -y;
    }
    if(y > x){
        tan = 1000*x/y;
        tan10 = tan/10;
        stp = &cosinus[tan10];
        ctp = &sinus[tan10];
    }else{
        tan = 1000*y/x;
        tan10 = tan/10;
        stp = &sinus[tan10];
        ctp = &cosinus[tan10];
    }
    rem = tan-(tan10*10);
    *sinp = sinsign*(stp[0]+(stp[1]-stp[0])*rem/10);
    *cosp = cossign*(ctp[0]+(ctp[1]-ctp[0])*rem/10);
}
```

Appendix D

Raspberry Pi VideoCore Driver TODO

Appendix E

PC VGA driver TODO

⟨*struct* VGAscr*(x86)* 362a⟩≡ (487)

```
struct VGAscr {

  ulong paddr;    /* frame buffer */
  void* vaddr;

  int   apsize;

  Memsubfont* memdefont;

  ⟨VGAscr cursor fields 188a⟩

  ulong colormap[Pcolours][3];
  int palettedepth;

  Pcidev* pci;
  ulong io;       /* device specific registers */
  ulong *mmio;

  ulong storage;
  bool useflush;

  ulong id; /* internal identifier for driver use */

  // the vga device methods
  VGAdev* dev;

  // why here? why not in VGAdev?
  ⟨VGAscr optional methods(x86) 305e⟩

  ⟨VGAscr other fields(x86) 305f⟩

  // Extra
  Lock  devlock;
};
```

⟨*global* vgascreen*(x86)* 362b⟩≡ (492)

```
VGAscr vgascreen;
```

⟨*function* VGAMEM*(x86)* 362c⟩≡ (487)

```
#define VGAMEM()  0xA0000
```

⟨VGAscr *other fields(x86)* 362d⟩+≡ (362a) ◁305f

```
int overlayinit;
```

⟨VGAscr *optional methods(x86)* 363a⟩+≡ (362a) ◁305e

```
int (*fill)(VGAscr*, Rectangle, ulong);
int (*scroll)(VGAscr*, Rectangle, Rectangle);
```

⟨*function* ishwimage*(x86)* 363b⟩≡ (492)

```
bool
arch_ishwimage(Memimage* i)
{
  return (i->data->bdata == gscreendata.bdata);
}
```

⟨*enum* vgamisc*(x86)* 363c⟩≡ (487)

```
   Pcolours  = 256,    /* Palette */

   Pred    = 0,
   Pgreen  = 1,
   Pblue   = 2,

   Pblack    = 0x00,
   Pwhite    = 0xFF,
};
```

⟨*enum* _anon_ (kernel/devices/screen/386/screen.h)2 363d⟩≡ (487)

```
enum {
  Backgnd   = 0,  /* black */
};
```

⟨*global* vgadevtab*(x86)* 363e⟩≡ (497)

```
Dev vgadevtab = {
    .dc       =    'v',
    .name     =    "vga",

    .attach   =    vgaattach,
    .walk     =    vgawalk,

    .open     =    vgaopen,
    .close    =    vgaclose,
    .read     =    vgaread,
    .write    =    vgawrite,
    .stat     =    vgastat,

    .reset    =    vgareset,

    // generic
    .create   =    devcreate,
    .remove   =    devremove,
    .wstat    =    devwstat,
    .bread    =    devbread,
    .bwrite   =    devbwrite,
    .init     =    devinit,
    .shutdown =    devshutdown,
};
```

E.1 IO ports

⟨*enum* vgaports*(x86)* 363f⟩≡ (487)

```
/*
 * Generic VGA registers.
```

```
 */
enum {
  MiscW   = 0x03C2, /* Miscellaneous Output (W) */
  MiscR   = 0x03CC, /* Miscellaneous Output (R) */
  Status0   = 0x03C2, /* Input status 0 (R) */
  Status1   = 0x03DA, /* Input Status 1 (R) */
  FeatureR  = 0x03CA, /* Feature Control (R) */
  FeatureW  = 0x03DA, /* Feature Control (W) */

  Seqx    = 0x03C4, /* Sequencer Index, Data at Seqx+1 */
  Crtx    = 0x03D4, /* CRT Controller Index, Data at Crtx+1 */
  Grx   = 0x03CE, /* Graphics Controller Index, Data at Grx+1 */
  Attrx   = 0x03C0, /* Attribute Controller Index and Data */

  PaddrW    = 0x03C8, /* Palette Address Register, write */
  Pdata     = 0x03C9, /* Palette Data Register */
  Pixmask   = 0x03C6, /* Pixel Mask Register */
  PaddrR    = 0x03C7, /* Palette Address Register, read */
  Pstatus   = 0x03C7, /* DAC Status (RO) */
```

⟨*function* vgao*(x86)* 364a⟩≡ (487)

```
#define vgao(port, data)  outb(port, data)
```

⟨*global* vgaxlock*(x86)* 364b⟩≡ (492)

```
static Lock vgaxlock;           /* access to index registers */
```

⟨*function* vgaxi*(x86)* 364c⟩≡ (492)

```
int
vgaxi(long port, uchar index)
{
    uchar data;

    ilock(&vgaxlock);
    switch(port){

    case Seqx:
    case Crtx:
    case Grx:
        outb(port, index);
        data = inb(port+1);
        break;

    case Attrx:
        /*
         * Allow processor access to the colour
         * palette registers. Writes to Attrx must
         * be preceded by a read from Status1 to
         * initialise the register to point to the
         * index register and not the data register.
         * Processor access is allowed by turning
         * off bit 0x20.
         */
        inb(Status1);
        if(index < 0x10){
            outb(Attrx, index);
            data = inb(Attrx+1);
            inb(Status1);
            outb(Attrx, 0x20|index);
        }
        else{
```

```
            outb(Attrx, 0x20|index);
            data = inb(Attrx+1);
        }
        break;

    default:
        iunlock(&vgaxlock);
        return -1;
    }
    iunlock(&vgaxlock);

    return data & 0xFF;
}
```

⟨*function* vgaxo*(x86)* 365a⟩≡ (492)

```
int
vgaxo(long port, uchar index, uchar data)
{
    ilock(&vgaxlock);
    switch(port){

    case Seqx:
    case Crtx:
    case Grx:
        /*
         * We could use an outport here, but some chips
         * (e.g., 86C928) have trouble with that for some
         * registers.
         */
        outb(port, index);
        outb(port+1, data);
        break;

    case Attrx:
        inb(Status1);
        if(index < 0x10){
            outb(Attrx, index);
            outb(Attrx, data);
            inb(Status1);
            outb(Attrx, 0x20|index);
        }
        else{
            outb(Attrx, 0x20|index);
            outb(Attrx, data);
        }
        break;

    default:
        iunlock(&vgaxlock);
        return -1;
    }
    iunlock(&vgaxlock);

    return 0;
}
```

E.2 VGAdev

⟨*struct* VGAdev*(x86)* 365b⟩≡ (487)

```
struct VGAdev {
  char* name;

  void  (*enable)(VGAscr*);
  void  (*disable)(VGAscr*);

  void  (*page)(VGAscr*, int);
  void  (*linear)(VGAscr*, int, int);

  // optional
  void  (*drawinit)(VGAscr*);
  int   (*fill)(VGAscr*, Rectangle, ulong);

  void  (*ovlctl)(VGAscr*, Chan*, void*, int);
  int   (*ovlwrite)(VGAscr*, void*, int, vlong);

  void  (*flush)(VGAscr*, Rectangle);
};
```

E.3 /dev/vgaxxx

⟨*enum* _anon_ (kernel/devices/screen/386/devvga.c) 366a⟩≡ (497)

```
enum {
    Qdir,

    Qvgabios,
    Qvgactl,

    Qvgaovl,
    Qvgaovlctl,
};
```

⟨*global* vgadir*(x86)* 366b⟩≡ (497)

```
static Dirtab vgadir[] = {
    ".",    { Qdir, 0, QTDIR },     0,  0550,
    "vgabios",  { Qvgabios, 0 },    0x100000, 0440,
    "vgactl",       { Qvgactl, 0 },     0,  0660,
    "vgaovl",       { Qvgaovl, 0 },     0,  0660,
    "vgaovlctl",    { Qvgaovlctl, 0 },  0,  0660,
};
```

⟨*enum* _anon_ (kernel/devices/screen/386/devvga.c)2 366c⟩≡ (497)

```
enum {
    CMactualsize,
    CMblank,
    CMblanktime,
    CMdrawinit,
    CMhwaccel,
    CMhwblank,
    CMhwgc,
    CMlinear,
    CMpalettedepth,
    CMpanning,
    CMsize,
    CMtextmode,
    CMtype,
    CMunblank,
};
```

⟨*global* vgactlmsg*(x86)* 367a⟩≡ (497)

```
static Cmdtab vgactlmsg[] = {
    CMactualsize,   "actualsize",   2,
    CMblank,    "blank",    1,
    CMblanktime,    "blanktime",    2,
    CMdrawinit, "drawinit", 1,
    CMhwaccel,  "hwaccel",  2,
    CMhwblank,  "hwblank",  2,
    CMhwgc,     "hwgc",     2,
    CMlinear,   "linear",   0,
    CMpalettedepth, "palettedepth", 2,
    CMpanning,  "panning",  2,
    CMsize,     "size",     3,
    CMtextmode, "textmode", 1,
    CMtype,     "type",     2,
    CMunblank,  "unblank",  1,
};
```

⟨*function* vgareset*(x86)* 367b⟩≡ (497)

```
static void
vgareset(void)
{
    /* reserve the 'standard' vga registers */
    if(ioalloc(0x2b0, 0x2df-0x2b0+1, 0, "vga") < 0)
        panic("vga ports already allocated");
    if(ioalloc(0x3c0, 0x3da-0x3c0+1, 0, "vga") < 0)
        panic("vga ports already allocated");
}
```

⟨*function* vgaattach*(x86)* 367c⟩≡ (497)

```
static Chan*
vgaattach(char* spec)
{
    if(*spec && strcmp(spec, "0"))
        error(Eio);
    return devattach('v', spec);
}
```

⟨*function* vgawalk*(x86)* 367d⟩≡ (497)

```
Walkqid*
vgawalk(Chan* c, Chan *nc, char** name, int nname)
{
    return devwalk(c, nc, name, nname, vgadir, nelem(vgadir), devgen);
}
```

⟨*function* vgastat*(x86)* 367e⟩≡ (497)

```
static int
vgastat(Chan* c, uchar* dp, int n)
{
    return devstat(c, dp, n, vgadir, nelem(vgadir), devgen);
}
```

⟨*function* vgaopen*(x86)* 367f⟩≡ (497)

```
static Chan*
vgaopen(Chan* c, int omode)
{
    VGAscr *scr;
    static char *openctl = "openctl\n";

    scr = &vgascreen;
```

```
    if ((ulong)c->qid.path == Qvgaovlctl) {
        if (scr->dev && scr->dev->ovlctl)
            scr->dev->ovlctl(scr, c, openctl, strlen(openctl));
        else
            error(Enonexist);
    }
    return devopen(c, omode, vgadir, nelem(vgadir), devgen);
}
```

⟨*function* vgaclose*(x86)* 368a⟩≡ (497)

```
static void
vgaclose(Chan* c)
{
    VGAscr *scr;
    static char *closectl = "closectl\n";

    scr = &vgascreen;
    if((ulong)c->qid.path == Qvgaovlctl)
        if(scr->dev && scr->dev->ovlctl){
            if(waserror()){
                print("ovlctl error: %s\n", up->errstr);
                return;
            }
            scr->dev->ovlctl(scr, c, closectl, strlen(closectl));
            poperror();
        }
}
```

⟨screensize() *initializations part2* 368b⟩+≡ (46a) ◁254a

```
physgscreenr = gscreen->r;
```

⟨*global* physgscreenr 368c⟩≡ (472)

```
Rectangle physgscreenr;
```

⟨*function* vgaread*(x86)* 368d⟩≡ (497)

```
static long
vgaread(Chan* c, void* a, long n, vlong off)
{
    int len;
    char *p, *s;
    VGAscr *scr;
    ulong offset = off;
    char chbuf[30];

    switch((ulong)c->qid.path){

    case Qdir:
        return devdirread(c, a, n, vgadir, nelem(vgadir), devgen);

    case Qvgabios:
        if(offset >= 0x100000)
            return 0;
        if(offset+n >= 0x100000)
            n = 0x100000 - offset;
        memmove(a, (uchar*)arch_kaddr(0)+offset, n); // why not KADDR()?
        return n;

    case Qvgactl:
        scr = &vgascreen;
```

```
        p = malloc(READSTR);
        if(p == nil)
            error(Enomem);
        if(waserror()){
            free(p);
            nexterror();
        }

        len = 0;

        if(scr->dev)
            s = scr->dev->name;
        else
            s = "cga";
        len += snprint(p+len, READSTR-len, "type %s\n", s);

        if(gscreen) {
            len += snprint(p+len, READSTR-len, "size %dx%dx%d %s\n",
                gscreen->r.max.x, gscreen->r.max.y,
                gscreen->depth, chantostr(chbuf, gscreen->chan));

            if(Dx(gscreen->r) != Dx(physgscreenr)
            || Dy(gscreen->r) != Dy(physgscreenr))
                len += snprint(p+len, READSTR-len, "actualsize %dx%d\n",
                    physgscreenr.max.x, physgscreenr.max.y);
        }

        len += snprint(p+len, READSTR-len, "blank time %lud idle %d state %s\n",
            blanktime, drawidletime(), scr->isblank ? "off" : "on");
        len += snprint(p+len, READSTR-len, "hwaccel %s\n", hwaccel ? "on" : "off");
        len += snprint(p+len, READSTR-len, "hwblank %s\n", hwblank ? "on" : "off");
        len += snprint(p+len, READSTR-len, "panning %s\n", panning ? "on" : "off");
        len += snprint(p+len, READSTR-len, "addr p 0x%lux v 0x%p size 0x%ux\n", scr->paddr, scr->vaddr, scr->ap
        USED(len);

        n = readstr(offset, a, n, p);
        poperror();
        free(p);

        return n;

    case Qvgaovl:
    case Qvgaovlctl:
        error(Ebadusefd);
        break;

    default:
        error(Egreg);
        break;
    }

    return 0;
}
```

⟨*function* vgactl*(x86)* 369⟩≡ (497)

```
static void
vgactl(Cmdbuf *cb)
{
    int align, i, size, x, y, z;
    char *chanstr, *p;
```

```
    ulong chan;
    Cmdtab *ct;
    VGAscr *scr;
    extern VGAdev *vgadev[];
    extern VGAcur *vgacur[];

    scr = &vgascreen;
    ct = lookupcmd(cb, vgactlmsg, nelem(vgactlmsg));
    switch(ct->index){
    ⟨vgactl cases(x86) 303b⟩
    }

    cmderror(cb, "bad VGA control message");
}
```

⟨vgactl *cases(x86)* 370a⟩+≡ (369) ◁305d 370b▷

```
case CMhwgc:
    if(strcmp(cb->f[1], "off") == 0){
        lock(&cursor);
        if(scr->cur){
            if(scr->cur->disable)
                scr->cur->disable(scr);
            scr->cur = nil;
        }
        unlock(&cursor);
        return;
    }
    if(strcmp(cb->f[1], "soft") == 0){
        lock(&cursor);
        swcursorinit_wrapper();
        if(scr->cur && scr->cur->disable)
            scr->cur->disable(scr);
        scr->cur = &swcursor;
        if(scr->cur->enable)
            scr->cur->enable(scr);
        unlock(&cursor);
        return;
    }
    for(i = 0; vgacur[i]; i++){
        if(strcmp(cb->f[1], vgacur[i]->name))
            continue;
        lock(&cursor);
        if(scr->cur && scr->cur->disable)
            scr->cur->disable(scr);
        scr->cur = vgacur[i];
        if(scr->cur->enable)
            scr->cur->enable(scr);
        unlock(&cursor);
        return;
    }
    break;
```

⟨vgactl *cases(x86)* 370b⟩+≡ (369) ◁370a 371a▷

```
case CMtype:
    for(i = 0; vgadev[i]; i++){
        if(strcmp(cb->f[1], vgadev[i]->name))
            continue;
        if(scr->dev && scr->dev->disable)
            scr->dev->disable(scr);
        scr->dev = vgadev[i];
```

```
        if(scr->dev->enable)
            scr->dev->enable(scr);
        return;
    }
    break;
```

⟨vgactl *cases(x86)* 371a⟩+≡ (369) ◁370b 372a▷

```
  case CMsize:
      x = strtoul(cb->f[1], &p, 0);
      if(x == 0 || x > 10240)
          error(Ebadarg);
      if(*p)
          p++;

      y = strtoul(p, &p, 0);
      if(y == 0 || y > 10240)
          error(Ebadarg);
      if(*p)
          p++;

      z = strtoul(p, &p, 0);

      chanstr = cb->f[2];
      if((chan = strtochan(chanstr)) == 0)
          error("bad channel");

      if(chantodepth(chan) != z)
          error("depth, channel do not match");

      arch_cursoroff(1);
      deletescreenimage();
      if(screensize(x, y, z, chan))
          error(Egreg);
      vgascreenwin(scr);
      resetscreenimage();
      arch_cursoron(1);
      return;
```

⟨*function* resetscreenimage 371b⟩≡ (477)

```
  void
  resetscreenimage(void)
  {
      dlock();
      initscreenimage();
      dunlock();
  }
```

⟨*function* deletescreenimage 371c⟩≡ (477)

```
  void
  deletescreenimage(void)
  {
      dlock();
      if(screenimage){
          /* will be freed via screendimage; disable */
          screenimage->clipr = ZR;
          screenimage = nil;
      }
      if(screendimage){
          drawfreedimage(screendimage);
          screendimage = nil;
```

```
    }
    dunlock();
}
```

⟨vgactl *cases(x86)* 372a⟩+≡ (369) ◁371a 372b▷

```
case CMactualsize:
    if(gscreen == nil)
        error("set the screen size first");

    x = strtoul(cb->f[1], &p, 0);
    if(x == 0 || x > 2048)
        error(Ebadarg);
    if(*p)
        p++;

    y = strtoul(p, nil, 0);
    if(y == 0 || y > 2048)
        error(Ebadarg);

    if(x > gscreen->r.max.x || y > gscreen->r.max.y)
        error("physical screen bigger than virtual");

    physgscreenr = Rect(0,0,x,y);
    gscreen->clipr = physgscreenr;
    return;
```

⟨vgactl *cases(x86)* 372b⟩+≡ (369) ◁372a 372c▷

```
case CMpalettedepth:
    x = strtoul(cb->f[1], &p, 0);
    if(x != 8 && x != 6)
        error(Ebadarg);

    scr->palettedepth = x;
    return;
```

⟨vgactl *cases(x86)* 372c⟩+≡ (369) ◁372b 372d▷

```
case CMdrawinit:
    if(gscreen == nil)
        error("drawinit: no gscreen");
    if(scr->dev && scr->dev->drawinit)
        scr->dev->drawinit(scr);
    return;
```

⟨vgactl *cases(x86)* 372d⟩+≡ (369) ◁372c 373b▷

```
case CMlinear:
    if(cb->nf!=2 && cb->nf!=3)
        error(Ebadarg);
    size = strtoul(cb->f[1], 0, 0);
    if(cb->nf == 2)
        align = 0;
    else
        align = strtoul(cb->f[2], 0, 0);
    if(screenaperture(size, align) < 0)
        error("not enough free address space");
    return;
```

⟨*function* screenaperture*(x86)* 372e⟩≡ (492)

```
int
screenaperture(int size, int align)
{
```

```
    VGAscr *scr;

    scr = &vgascreen;

    if(scr->paddr)  /* set up during enable */
        return 0;

    if(size == 0)
        return 0;

    if(scr->dev && scr->dev->linear){
        scr->dev->linear(scr, size, align);
        return 0;
    }

    /*
     * Need to allocate some physical address space.
     * The driver will tell the card to use it.
     */
    size = PGROUND(size);
    scr->paddr = upaalloc(size, align);
    if(scr->paddr == 0)
        return -1;
    scr->vaddr = vmap(scr->paddr, size);
    if(scr->vaddr == nil)
        return -1;
    scr->apsize = size;

    return 0;
}
```

⟨*global* panning*(x86)* 373a⟩≡ (492)

```
bool panning = false;
```

⟨vgactl *cases(x86)* 373b⟩+≡ (369) ◁372d 373c▷

```
case CMpanning:
    if(strcmp(cb->f[1], "on") == 0){
        if(scr == nil || scr->cur == nil)
            error("set screen first");
        if(!scr->cur->doespanning)
            error("panning not supported");
        gscreen->clipr = gscreen->r;
        panning = true;
    }
    else if(strcmp(cb->f[1], "off") == 0){
        gscreen->clipr = physgscreenr;
        panning = false;
    }else
        break;
    return;
```

⟨vgactl *cases(x86)* 373c⟩+≡ (369) ◁373b 374a▷

```
case CMhwaccel:
    if(strcmp(cb->f[1], "on") == 0)
        hwaccel = 1;
    else if(strcmp(cb->f[1], "off") == 0)
        hwaccel = 0;
    else
        break;
    return;
```

⟨vgactl *cases(x86)* 374a⟩+≡ (369) ◁373c

```
case CMtextmode:
    arch__screeninit();
    return;
```

⟨*global* Enooverlay*(x86)* 374b⟩≡ (497)

```
char Enooverlay[] = "No overlay support";
```

⟨*function* vgawrite*(x86)* 374c⟩≡ (497)

```
static long
vgawrite(Chan* c, void* a, long n, vlong off)
{
    ulong offset = off;
    Cmdbuf *cb;
    VGAscr *scr;

    switch((ulong)c->qid.path){

    case Qdir:
        error(Eperm);

    case Qvgactl:
        if(offset || n >= READSTR)
            error(Ebadarg);
        cb = parsecmd(a, n);
        if(waserror()){
            free(cb);
            nexterror();
        }
        vgactl(cb);
        poperror();
        free(cb);
        return n;

    case Qvgaovl:
        scr = &vgascreen;
        if (scr->dev == nil || scr->dev->ovlwrite == nil) {
            error(Enooverlay);
            break;
        }
        return scr->dev->ovlwrite(scr, a, n, off);

    case Qvgaovlctl:
        scr = &vgascreen;
        if (scr->dev == nil || scr->dev->ovlctl == nil) {
            error(Enooverlay);
            break;
        }
        scr->dev->ovlctl(scr, c, a, n);
        return n;

    default:
        error(Egreg);
        break;
    }

    return 0;
}
```

E.4 VGA linear addressing

⟨*function* vgalinearpciid*(x86)* 375a⟩≡ (492)

```
void
vgalinearpciid(VGAscr *scr, int vid, int did)
{
    Pcidev *p;

    p = nil;
    while((p = pcimatch(p, vid, 0)) != nil){
        if(p->ccrb != 3)    /* video card */
            continue;
        if(did != 0 && p->did != did)
            continue;
        break;
    }
    if(p == nil)
        error("pci video card not found");

    scr->pci = p;
    vgalinearpci(scr);
}
```

⟨*function* vgalinearpci*(x86)* 375b⟩≡ (492)

```
void
vgalinearpci(VGAscr *scr)
{
    ulong paddr;
    int i, size, best;
    Pcidev *p;

    p = scr->pci;
    if(p == nil)
        return;

    /*
     * Scan for largest memory region on card.
     * Some S3 cards (e.g., Savage) have enormous
     * mmio regions (but even larger frame buffers).
     * Some 3dfx cards (e.g., Voodoo3) have mmio
     * buffers the same size as the frame buffer,
     * but only the frame buffer is marked as
     * prefetchable (bar&8).  If a card doesn't fit
     * into these heuristics, its driver will have to
     * call vgalinearaddr directly.
     */
    best = -1;
    for(i=0; i<nelem(p->mem); i++){
        if(p->mem[i].bar&1) /* not memory */
            continue;
        if(p->mem[i].size < 640*480)    /* not big enough */
            continue;
        if(best==-1
        || p->mem[i].size > p->mem[best].size
        || (p->mem[i].size == p->mem[best].size
          && (p->mem[i].bar&8)
          && !(p->mem[best].bar&8)))
            best = i;
    }
    if(best >= 0){
```

```
        paddr = p->mem[best].bar & ~0x0F;
        size = p->mem[best].size;
        vgalinearaddr(scr, paddr, size);
        return;
    }
    error("no video memory found on pci card");
}
```

⟨*function* vgalinearaddr*(x86)* 376⟩≡ (492)

```
void
vgalinearaddr(VGAscr *scr, ulong paddr, int size)
{
    int x, nsize;
    ulong npaddr;

    /*
     * new approach.  instead of trying to resize this
     * later, let's assume that we can just allocate the
     * entire window to start with.
     */

    if(scr->paddr == paddr && size <= scr->apsize)
        return;

    if(scr->paddr){
        /*
         * could call vunmap and vmap,
         * but worried about dangling pointers in devdraw
         */
        error("cannot grow vga frame buffer");
    }

    /* round to page boundary, just in case */
    x = paddr&(BY2PG-1);
    npaddr = paddr-x;
    nsize = PGROUND(size+x);

    /*
     * Don't bother trying to map more than 4000x4000x32 = 64MB.
     * We only have a 256MB window.
     */
    if(nsize > 64*MB)
        nsize = 64*MB;
    scr->vaddr = vmap(npaddr, nsize);
    if(scr->vaddr == 0)
        error("cannot allocate vga frame buffer");
    scr->vaddr = (char*)scr->vaddr+x;
    scr->paddr = paddr;
    scr->apsize = nsize;
    /* let mtrr harmlessly fail on old CPUs, e.g., P54C */
    if(!waserror()){
        //mtrr(npaddr, nsize, "wc"); disabled mtrr
                error("mtrr disabled");
        poperror();
    }
}
```

E.5 VGA page

⟨screensize() *when use softscreen, other settings* 377a⟩+≡ (47d) ◁47f

```
if(scr->dev && scr->dev->page){
    scr->vaddr = KADDR(VGAMEM());
    scr->apsize = 1<<16;
}
```

⟨flushmemscreen() *other locals* 377b⟩≡ (82b)

```
byte *sp, *disp, *sdisp, *edisp;
int y, len, incs, off, page;
```

⟨flushmemscreen() *use VGA page* 377c⟩≡ (82b)

```
incs = gscreen->width * BY2WD;

switch(gscreen->depth){
case 8:
    len = Dx(r);
    break;
default:
    len = 0;
    panic("flushmemscreen: depth\n");
    break;
}
if(len < 1)
    return;

off = r.min.y * gscreen->width * BY2WD
       + (r.min.x * gscreen->depth)/8;
page = off/scr->apsize;
off %= scr->apsize;
disp = scr->vaddr;
sdisp = disp+off;
edisp = disp+scr->apsize;

off = r.min.y * gscreen->width * BY2WD
       + (r.min.x * gscreen->depth)/8;

sp = gscreendata.bdata + off;

// call device driver again, for subpart
scr->dev->page(scr, page);

for(y = r.min.y; y < r.max.y; y++) {
    if(sdisp + incs < edisp) {
        memmove(sdisp, sp, len);
        sp += incs;
        sdisp += incs;
    }
    else {
        off = edisp - sdisp;
        page++;
        if(off <= len){
            if(off > 0)
                memmove(sdisp, sp, off);

            // call device driver again, for subpart
            scr->dev->page(scr, page);
            if(len - off > 0)
                memmove(disp, sp+off, len - off);
```

```
        }
        else {
            memmove(sdisp, sp, len);
            // call device driver again, for subpart
            scr->dev->page(scr, page);
        }
        sp += incs;
        sdisp += incs - scr->apsize;
    }
}
```

E.6 VESA

⟨*global* vgavesadev*(x86)* 378a⟩≡ (501c)

```
VGAdev vgavesadev = {
    "vesa",
    0,
    0,
    0,
    vesalinear,
    0,
    0,
    0,
    0,
    vesaflush,
};
```

⟨*function* vesalinear*(x86)* 378b⟩≡ (501c)

```
static void
vesalinear(VGAscr *scr, int, int)
{
    int i, mode, size, havesize;
    uchar *p;
    ulong paddr;
    Pcidev *pci;

    if(hardscreen) {
        scr->vaddr = 0;
        scr->paddr = scr->apsize = 0;
        return;
    }

    vbecheck();
    mode = vbegetmode();
    /*
     * bochs loses the top bits - cannot use this
    if((mode&(1<<14)) == 0)
        error("not in linear graphics mode");
     */
    mode &= 0x3FFF;
    p = vbemodeinfo(mode);
    if(!(WORD(p+0) & (1<<4)))
        error("not in VESA graphics mode");
    if(!(WORD(p+0) & (1<<7)))
        error("not in linear graphics mode");

    paddr = LONG(p+40);
    size = WORD(p+20)*WORD(p+16);
```

```
    size = PGROUND(size);

    /*
     * figure out max size of memory so that we have
     * enough if the screen is resized.
     */
    pci = nil;
    havesize = 0;
    while(!havesize && (pci = pcimatch(pci, 0, 0)) != nil){
        if(pci->ccrb != Pcibcdisp)
            continue;
        for(i=0; i<nelem(pci->mem); i++)
            if(paddr == (pci->mem[i].bar&~0x0F)){
                if(pci->mem[i].size > size)
                    size = pci->mem[i].size;
                havesize = 1;
                break;
            }
    }

    /* no pci - heuristic guess */
    if (!havesize)
        if(size < 4*1024*1024)
            size = 4*1024*1024;
        else
            size = ROUND(size, 1024*1024);
    if(size > 16*1024*1024)     /* arbitrary */
        size = 16*1024*1024;

    vgalinearaddr(scr, paddr, size);
    if(scr->apsize)
        addvgaseg("vesascreen", scr->paddr, scr->apsize);

    if(Usesoftscreen){
        hardscreen = scr->vaddr;
        scr->vaddr = 0;
        scr->paddr = scr->apsize = 0;
    }
}
```

E.7 Cirrus logic 5446

⟨*global* vgaclgd542xdev*(x86)* 379a⟩≡ (498)

```
VGAdev vgaclgd542xdev = {
    .name = "clgd542x",

    .enable = nil,
    .disable = nil,
    .page = clgd542xpage,
    .linear = clgd542xlinear,
};
```

E.7.1 Linear mode

⟨*function* clgd542xlinear*(x86)* 379b⟩≡ (498)

```
static void
clgd542xlinear(VGAscr* scr, int, int)
```

```
{
    vgalinearpciid(scr, 0x1013, 0);
}
```

E.7.2 Page

⟨*function* clgd542xpage*(x86)* 380a⟩≡ (498)

```
static void
clgd542xpage(VGAscr* scr, int page)
{
    lock(&scr->devlock);
    clgd542xpageset(scr, page);
    unlock(&scr->devlock);
}
```

⟨*function* clgd542xpageset*(x86)* 380b⟩≡ (498)

```
static int
clgd542xpageset(VGAscr*, int page)
{
    uchar gr09;
    int opage;

    if(vgaxi(Seqx, 0x07) & 0xF0)
        page = 0;
    gr09 = vgaxi(Grx, 0x09);
    if(vgaxi(Grx, 0x0B) & 0x20){
        vgaxo(Grx, 0x09, page<<2);
        opage = gr09>>2;
    }
    else{
        vgaxo(Grx, 0x09, page<<4);
        opage = gr09>>4;
    }

    return opage;
}
```

E.7.3 Cursor

⟨*global* vgaclgd542xcur*(x86)* 380c⟩≡ (498)

```
VGAcur vgaclgd542xcur = {
    .name = "clgd542xhwgc",

    .enable = clgd542xenable,
    .disable = clgd542xdisable,
    .load = clgd542xload,
    .move = clgd542xmove,
};
```

⟨*function* clgd542xenable*(x86)* 380d⟩≡ (498)

```
static void
clgd542xenable(VGAscr* scr)
{
    uchar sr12;
    int mem, x;

    /*
     * Disable the cursor.
```

```
 */
sr12 = vgaxi(Seqx, 0x12);
vgaxo(Seqx, 0x12, sr12 & ~0x01);

/*
 * Cursor colours.
 * Can't call setcolor here as cursor is already locked.
 */
vgaxo(Seqx, 0x12, sr12|0x02);
vgao(PaddrW, 0x00);
vgao(Pdata, Pwhite);
vgao(Pdata, Pwhite);
vgao(Pdata, Pwhite);
vgao(PaddrW, 0x0F);
vgao(Pdata, Pblack);
vgao(Pdata, Pblack);
vgao(Pdata, Pblack);
vgaxo(Seqx, 0x12, sr12);

mem = 0;
switch(vgaxi(Crtx, 0x27) & ~0x03){

case 0x88:              /* CL-GD5420 */
case 0x8C:              /* CL-GD5422 */
case 0x94:              /* CL-GD5424 */
case 0x80:              /* CL-GD5425 */
case 0x90:              /* CL-GD5426 */
case 0x98:              /* CL-GD5427 */
case 0x9C:              /* CL-GD5429 */
    /*
     * The BIOS leaves the memory size in Seq0A, bits 4 and 3.
     * See Technical Reference Manual Appendix E1, Section 1.3.2.
     *
     * The storage area for the 64x64 cursors is the last 16Kb of
     * display memory.
     */
    mem = (vgaxi(Seqx, 0x0A)>>3) & 0x03;
    break;

case 0xA0:              /* CL-GD5430 */
case 0xA8:              /* CL-GD5434 */
case 0xAC:              /* CL-GD5436 */
case 0xB8:              /* CL-GD5446 */
case 0x30:              /* CL-GD7543 */
    /*
     * Attempt to intuit the memory size from the DRAM control
     * register. Minimum is 512KB.
     * If DRAM bank switching is on then there's double.
     */
    x = vgaxi(Seqx, 0x0F);
    mem = (x>>3) & 0x03;
    if(x & 0x80)
        mem++;
    break;

case 0xBC:              /* CL-GD5480 */
    mem = 2;            /* 1024 = 256<<2 */
    x = vgaxi(Seqx, 0x0F);
    if((x & 0x18) == 0x18){
        mem <<= 1;      /* 2048 = 256<<3 */
```

```
            if(x & 0x80)
                mem <<= 2;  /* 2048 = 256<<4 */
        }
        if(vgaxi(Seqx, 0x17) & 0x80)
            mem <<= 1;
        break;

    default:                /* uh, ah dunno */
        break;
    }
    scr->storage = ((256<<mem)-16)*1024;

    /*
     * Set the current cursor to index 0
     * and turn the 64x64 cursor on.
     */
    vgaxo(Seqx, 0x13, 0);
    vgaxo(Seqx, 0x12, sr12|0x05);
}
```

⟨*function* clgd542xdisable*(x86)* 382a⟩≡ (498)

```
static void
clgd542xdisable(VGAscr*)
{
    uchar sr12;

    sr12 = vgaxi(Seqx, 0x12);
    vgaxo(Seqx, 0x12, sr12 & ~0x01);
}
```

⟨*function* clgd542xinitcursor*(x86)* 382b⟩≡ (498)

```
static void
clgd542xinitcursor(VGAscr* scr, int xo, int yo, int index)
{
    uchar *p, seq07;
    uint p0, p1;
    int opage, x, y;

    /*
     * Is linear addressing turned on? This will determine
     * how we access the cursor storage.
     */
    seq07 = vgaxi(Seqx, 0x07);
    opage = 0;
    p = scr->vaddr;
    if(!(seq07 & 0xF0)){
        lock(&scr->devlock);
        opage = clgd542xpageset(scr, scr->storage>>16);
        p += (scr->storage & 0xFFFF);
    }
    else
        p += scr->storage;
    p += index*1024;

    for(y = yo; y < 16; y++){
        p0 = scr->set[2*y];
        p1 = scr->set[2*y+1];
        if(xo){
            p0 = (p0<<xo)|(p1>>(8-xo));
            p1 <<= xo;
```

```
        }
        *p++ = p0;
        *p++ = p1;

        for(x = 16; x < 64; x += 8)
            *p++ = 0x00;

        p0 = scr->clr[2*y]|scr->set[2*y];
        p1 = scr->clr[2*y+1]|scr->set[2*y+1];
        if(xo){
            p0 = (p0<<xo)|(p1>>(8-xo));
            p1 <<= xo;
        }
        *p++ = p0;
        *p++ = p1;

        for(x = 16; x < 64; x += 8)
            *p++ = 0x00;
    }
    while(y < 64+yo){
        for(x = 0; x < 64; x += 8){
            *p++ = 0x00;
            *p++ = 0x00;
        }
        y++;
    }

    if(!(seq07 & 0xF0)){
        clgd542xpageset(scr, opage);
        unlock(&scr->devlock);
    }
}
```

⟨*function* clgd542xmove*(x86)* 383⟩≡ (498)

```
static int
clgd542xmove(VGAscr* scr, Point p)
{
    int index, x, xo, y, yo;

    index = 0;
    if((x = p.x+scr->offset.x) < 0){
        xo = -x;
        x = 0;
    }
    else
        xo = 0;
    if((y = p.y+scr->offset.y) < 0){
        yo = -y;
        y = 0;
    }
    else
        yo = 0;

    if(xo || yo){
        clgd542xinitcursor(scr, xo, yo, 1);
        index = 1;
    }
    vgaxo(Seqx, 0x13, index<<2);

    vgaxo(Seqx, 0x10|((x & 0x07)<<5), (x>>3) & 0xFF);
```

```
    vgaxo(Seqx, 0x11|((y & 0x07)<<5), (y>>3) & 0xFF);

    return 0;
}
```

⟨*function* clgd542xload*(x86)* 384⟩≡ (498)

```
static void
clgd542xload(VGAscr* scr, Cursor* curs)
{
    uchar sr12;

    /*
     * Disable the cursor.
     */
    sr12 = vgaxi(Seqx, 0x12);
    vgaxo(Seqx, 0x12, sr12 & ~0x01);

    memmove(&scr->Cursor, curs, sizeof(Cursor));
    clgd542xinitcursor(scr, 0, 0, 0);

    /*
     * Enable the cursor.
     */
    vgaxo(Seqx, 0x13, 0);
    vgaxo(Seqx, 0x12, sr12|0x05);
}
```

E.8 /bin/vga

Appendix F

Examples of Graphical Applications TODO

Appendix G

Extra Code

⟨lib_graphics/libdraw/enter.c 386a⟩≡

⟨lib_graphics/libdraw/eenter.c 386b⟩≡

G.1 include/

G.1.1 include/graphics/draw.h

⟨*enum* constants1 386c⟩≡ (386d)

```
enum
{
    ⟨constant ICOSSCALE 117b⟩
    ⟨constant Borderwidth 233a⟩
};
```

⟨include/graphics/draw.h 386d⟩≡

```
#pragma src "/sys/src/libdraw"
#pragma lib "libdraw.a"

typedef struct  Point Point;
typedef struct  Rectangle Rectangle;
typedef struct  Display Display;
typedef struct  Image Image;
typedef struct  Font Font;
typedef struct  RGB RGB;

// defined in other header files
typedef struct  Screen Screen;
typedef struct  Cachefont Cachefont;
typedef struct  Cacheinfo Cacheinfo;
typedef struct  Cachesubf Cachesubf;
typedef struct  Subfont Subfont;
#pragma incomplete Screen
#pragma incomplete Cachefont
#pragma incomplete Cacheinfo
#pragma incomplete Cachesubf
#pragma incomplete Subfont

// temporary hack, because does not have a memfont.h so need to
// disable safe-linking for Fontchar everywhere
typedef struct  Fontchar Fontchar;
#pragma incomplete Fontchar
```

```
// TODO why need that?
typedef struct  Mouse Mouse;
#pragma incomplete Mouse

//----------------------------------------------------------------------------
// Data structures and constants
//----------------------------------------------------------------------------

⟨struct Point 31a⟩

⟨type Vector 31e⟩

⟨struct Rectangle 32e⟩


⟨enum Colors 41g⟩
⟨type rgba 41f⟩

⟨struct RGB 247g⟩


⟨enum ImageChan 40c⟩
⟨function __DC 40d⟩
⟨function CHAN1 40g⟩
⟨function CHAN2 40h⟩
⟨function CHAN3 40i⟩
⟨function CHAN4 41a⟩
⟨function NBITS 40e⟩
⟨function TYPE 40f⟩
⟨enum ImageType 41b⟩
⟨type channels 40b⟩


⟨type Errorfn 352a⟩

⟨struct Display 34e⟩

⟨struct Image 37c⟩

⟨struct Font 137a⟩



⟨enum constants1 386c⟩

⟨enum drawop 44f⟩
typedef enum drawop Drawop;

⟨enum Refresh 309h⟩

⟨enum Endline 113b⟩

⟨function ARROW 257a⟩

//----------------------------------------------------------------------------
// Globals
//----------------------------------------------------------------------------

/*
 * Set up by initdraw()
```

```
 */
extern  Display *display;
extern  Image   *view; // was called 'screen' before
extern  Font    *font;

/*
 * Predefined
 */
extern  Point       ZP;
extern  Rectangle   ZR;

//-----------------------------------------------------------------------------
// Functions
//-----------------------------------------------------------------------------

//
// Display
//
// This sets the following globals: display, view, font (and screen).
extern int  initdraw(Errorfn, char*, char*);
extern int  geninitdraw(char*, Errorfn, char*, char*, char*, int);
// to call after a resize event
extern int  getwindow(Display*, int);

extern Display* initdisplay(char*, char*, Errorfn);
extern void     closedisplay(Display*);

extern int      flushimage(Display*, bool);

extern void drawerror(Display*, char*);

/*
 * Image management
 */
extern Image*   allocimage(Display*, Rectangle, channels, bool, rgba);
extern Image*   allocimagemix(Display*, rgba, rgba);
extern int      freeimage(Image*);

extern int      loadimage(Image*, Rectangle, byte*, int);
extern int      unloadimage(Image*, Rectangle, byte*, int);
extern Image*   readimage(Display*, fdt, bool);
extern int      writeimage(fdt, Image*, bool);
// compressed variants
extern int      cloadimage(Image*, Rectangle, byte*, int);
extern Image*   creadimage(Display*, int, int);

extern Image*   namedimage(Display*, char*);
extern int      nameimage(Image*, char*, bool);

//
// Channels
//
extern  int     chantodepth(channels);
extern  char*       chantostr(char*, channels);
extern  channels    strtochan(char*);

/*
 * Colors
 */
extern  void    readcolmap(Display*, RGB*);
```

```
extern  void    writecolmap(Display*, RGB*);

extern int      rgb2cmap(int, int, int);
extern int      cmap2rgb(int);
extern int      cmap2rgba(int);

/*
 * Geometry
 */
extern Point    Pt(int, int);
extern int      eqpt(Point, Point);

extern Point    addpt(Point, Point);
extern Point    subpt(Point, Point);
extern Point    divpt(Point, int);
extern Point    mulpt(Point, int);

extern Rectangle    Rect(int, int, int, int);
extern Rectangle    Rpt(Point, Point);
extern int          eqrect(Rectangle, Rectangle);
```

⟨*function* Dx 33e⟩
⟨*function* Dy 33f⟩

```
extern Rectangle    rectaddpt(Rectangle, Point);
extern Rectangle    rectsubpt(Rectangle, Point);
extern Rectangle    insetrect(Rectangle, int);
extern Rectangle    canonrect(Rectangle);
extern int      rectXrect(Rectangle, Rectangle);
extern int      rectinrect(Rectangle, Rectangle);
extern void     combinerect(Rectangle*, Rectangle);
extern int      ptinrect(Point, Rectangle);

extern void     icossin(int, int*, int*);
extern void     icossin2(int, int, int*, int*);

// Clipping and replication
extern void     replclipr(Image*, int, Rectangle);
extern int      rectclip(Rectangle*, Rectangle);
extern int      drawreplxy(int, int, int);  /* used to be drawsetxy */
extern Point    drawrepl(Rectangle, Point);

/*
 * Graphics
 */
extern void draw(Image*, Rectangle, Image*, Image*, Point);
extern void drawop(Image*, Rectangle, Image*, Image*, Point, Drawop);
extern void gendraw(Image*, Rectangle, Image*, Point, Image*, Point);
extern void gendrawop(Image*, Rectangle, Image*, Point, Image*, Point, Drawop);

extern void border(Image*, Rectangle, int, Image*, Point);
extern void borderop(Image*, Rectangle, int, Image*, Point, Drawop);

extern void line(Image*, Point, Point, int, int, int, Image*, Point);
extern void lineop(Image*, Point, Point, int, int, int, Image*, Point, Drawop);

extern void poly(Image*, Point*, int, int, int, int, Image*, Point);
extern void polyop(Image*, Point*, int, int, int, int, Image*, Point, Drawop);
extern void fillpoly(Image*, Point*, int, int, Image*, Point);
extern void fillpolyop(Image*, Point*, int, int, Image*, Point, Drawop);
```

```
extern void ellipse(Image*, Point, int, int, int, Image*, Point);
extern void ellipseop(Image*, Point, int, int, int, Image*, Point, Drawop);
extern void fillellipse(Image*, Point, int, int, Image*, Point);
extern void fillellipseop(Image*, Point, int, int, Image*, Point, Drawop);

extern void arc(Image*, Point, int, int, int, Image*, Point, int, int);
extern void arcop(Image*, Point, int, int, int, Image*, Point, int, int, Drawop);
extern void fillarc(Image*, Point, int, int, Image*, Point, int, int);
extern void fillarcop(Image*, Point, int, int, Image*, Point, int, int, Drawop);

extern int  bezier(Image*, Point, Point, Point, Point, int, int, int, Image*, Point);
extern int  bezierop(Image*, Point, Point, Point, Point, int, int, int, Image*, Point, Drawop);
extern int  bezspline(Image*, Point*, int, int, int, int, Image*, Point);
extern int  bezsplineop(Image*, Point*, int, int, int, int, Image*, Point, Drawop);
extern int  fillbezier(Image*, Point, Point, Point, Point, int, Image*, Point);
extern int  fillbezierop(Image*, Point, Point, Point, Point, int, Image*, Point, Drawop);
extern int  fillbezspline(Image*, Point*, int, int, Image*, Point);
extern int  fillbezsplineop(Image*, Point*, int, int, Image*, Point, Drawop);

extern int  bezsplinepts(Point*, int, Point**);


extern Point    string(Image*, Point, Image*, Point, Font*, char*);
extern Point    stringop(Image*, Point, Image*, Point, Font*, char*, Drawop);
extern Point    stringn(Image*, Point, Image*, Point, Font*, char*, int);
extern Point    stringnop(Image*, Point, Image*, Point, Font*, char*, int, Drawop);
extern Point    runestring(Image*, Point, Image*, Point, Font*, Rune*);
extern Point    runestringop(Image*, Point, Image*, Point, Font*, Rune*, Drawop);
extern Point    runestringn(Image*, Point, Image*, Point, Font*, Rune*, int);
extern Point    runestringnop(Image*, Point, Image*, Point, Font*, Rune*, int, Drawop);
extern Point    stringbg(Image*, Point, Image*, Point, Font*, char*, Image*, Point);
extern Point    stringbgop(Image*, Point, Image*, Point, Font*, char*, Image*, Point, Drawop);
extern Point    stringnbg(Image*, Point, Image*, Point, Font*, char*, int, Image*, Point);
extern Point    stringnbgop(Image*, Point, Image*, Point, Font*, char*, int, Image*, Point, Drawop);
extern Point    runestringbg(Image*, Point, Image*, Point, Font*, Rune*, Image*, Point);
extern Point    runestringbgop(Image*, Point, Image*, Point, Font*, Rune*, Image*, Point, Drawop);
extern Point    runestringnbg(Image*, Point, Image*, Point, Font*, Rune*, int, Image*, Point);
extern Point    runestringnbgop(Image*, Point, Image*, Point, Font*, Rune*, int, Image*, Point, Drawop);


extern Point    stringsize(Font*, char*);
extern int      stringwidth(Font*, char*);
extern int      stringnwidth(Font*, char*, int);

extern Point    runestringsize(Font*, Rune*);
extern int      runestringwidth(Font*, Rune*);
extern int      runestringnwidth(Font*, Rune*, int);


/*
 * Font management
 */
extern Font*    buildfont(Display*, char*, char*);
extern Font*    openfont(Display*, char*);
extern void     freefont(Font*);

/*
 * One of a kind
 */
```

```
extern int      mousescrollsize(int);

extern int  bytesperline(Rectangle, int);
extern int  wordsperline(Rectangle, int);

// seems related to font
extern void     lockdisplay(Display*);
extern void     unlockdisplay(Display*);

//
// Dumpers
//
#pragma varargck    type    "R" Rectangle
#pragma varargck    type    "P" Point
extern  int Rfmt(Fmt*);
extern  int Pfmt(Fmt*);
```

G.1.2 include/graphics/window.h

⟨include/graphics/window.h 391a⟩≡

```
#pragma src "/sys/src/libdraw"
#pragma lib "libdraw.a"
// This file assumes you have included draw.h before.

⟨struct Screen 197a⟩

// set by initdraw() automatically.
extern  Screen  *screen; // was called _screen before

// called by initdraw()
extern int  gengetwindow(Display*, char*, Image**, Screen**, int);

//
// Base layer
//
extern Screen*  allocscreen(Image*, Image*, int);
extern int      freescreen(Screen*);
extern Screen*  publicscreen(Display*, int, ulong);

/*
 * Windows
 */
extern Image*   allocwindow(Screen*, Rectangle, int, ulong);

extern int  originwindow(Image*, Point, Point);
extern void bottomnwindows(Image**, int);
extern void bottomwindow(Image*);
extern void topnwindows(Image**, int);
extern void topwindow(Image*);

extern int  newwindow(char*);
```

G.1.3 include/graphics/internals/font.h

⟨include/graphics/internals/font.h 391b⟩≡

```
#pragma src "/sys/src/libdraw"
```

```
#pragma lib "libdraw.a"
// This file assumes you have included draw.h before.

typedef struct Cachefont Cachefont;
typedef struct Cacheinfo Cacheinfo;
typedef struct Cachesubf Cachesubf;
typedef struct Fontchar Fontchar;
typedef struct Subfont Subfont;

// The main type, Font, is defined in draw.h.

⟨struct Fontchar 138c⟩

⟨struct Subfont 138b⟩

⟨struct Cachefont 137e⟩

⟨struct Cacheinfo 155e⟩

⟨struct Cachesubf 165b⟩

⟨enum misc 154d⟩


// internal to font code
extern Subfont* allocsubfont(char*, int, int, int, Fontchar*, Image*);
extern void     freesubfont(Subfont*);
extern Subfont* lookupsubfont(Display*, char*);
extern void installsubfont(char*, Subfont*);
extern void uninstallsubfont(Subfont*);
extern Subfont* readsubfont(Display*, char*, int, int);
extern char* subfontname(char*, char*, int);
extern Subfont* _getsubfont(Display*, char*);
// for subfont designers
extern int writesubfont(int, Subfont*);

extern int  cachechars(Font*, char**, Rune**, ushort*, int, int*, char**);
extern void  agefont(Font*);
extern Point strsubfontwidth(Subfont*, char*);
extern int  loadchar(Font*, Rune, Cacheinfo*, int, int, char**);
extern Subfont* getdefont(Display*);

// used also by libmemdraw/
extern void  _unpackinfo(Fontchar*, byte*, int);
extern byte defontdata[];
extern int  sizeofdefont;
```

G.1.4 include/graphics/internals/marshal.h

```
⟨include/graphics/internals/marshal.h 392⟩≡
  ⟨function BGSHORT 69f⟩
  ⟨function BGLONG 69g⟩
  ⟨function BPSHORT 69h⟩
  ⟨function BPLONG 69i⟩

  extern byte* bufimage(Display*, int);
```

G.1.5 include/graphics/internals/draw_private.h

⟨include/graphics/internals/draw_private.h 393a⟩≡

```
extern int  drawlsetrefresh(ulong, int, void*, void*);

// dead?
extern int  _cursorfd;

extern bool _drawdebug; /* set to true to see errors from flushimage */

// used also by window.c
extern Image* _allocimage(Image*, Display*, Rectangle, ulong, int, ulong, int, int);
extern int     _freeimage1(Image*);

extern void _setdrawop(Display*, Drawop);

// used also by libmemdraw/
void _twiddlecompressed(uchar *buf, int n);
int _compblocksize(Rectangle r, int depth);

⟨constant NMATCH 328b⟩
⟨constant NRUN 328c⟩
⟨constant NMEM 333f⟩
⟨constant NDUMP 333g⟩
⟨constant NCBLOCK 333h⟩

/* XXX backwards helps; should go */
extern ulong drawld2chan[];
extern void drawsetdebug(bool);

#include <marshal.h>
```

G.1.6 include/graphics/internals/memdraw.h

⟨include/graphics/internals/memdraw.h 393b⟩≡

```
#pragma src "/sys/src/libmemdraw"
#pragma lib "libmemdraw.a"
// This file assumes you have included draw.h before.

typedef struct Memimage Memimage;
typedef struct Memdata Memdata;
typedef struct Memsubfont Memsubfont;
typedef struct Memcmap Memcmap;
typedef struct Memdrawparam Memdrawparam;

// defined in other header files
typedef struct Memlayer Memlayer;
typedef struct Fontchar Fontchar;
#pragma incomplete Memlayer
#pragma incomplete Fontchar

⟨struct Memdata 38h⟩

⟨enum fxxx 38a⟩

⟨struct Memimage 37e⟩
```

⟨*struct* Memcmap 247i⟩

⟨*struct* Memsubfont 340a⟩

⟨*enum* _anon_ (include/memdraw.h)2 98c⟩
//TODO: internal to draw.c?
⟨*struct* Memdrawparam 93a⟩

```
/*
 * Memimage management
 */
extern Memimage* allocmemimage(Rectangle, ulong);
extern Memimage* allocmemimaged(Rectangle, ulong, Memdata*);
extern void   freememimage(Memimage*);

extern Memimage* readmemimage(fdt);
extern int   writememimage(int, Memimage*);
extern int   loadmemimage(Memimage*, Rectangle, uchar*, int);
extern int   unloadmemimage(Memimage*, Rectangle, uchar*, int);
extern Memimage* creadmemimage(int);
extern int   cloadmemimage(Memimage*, Rectangle, uchar*, int);

// misc utilities
extern ulong* wordaddr(Memimage*, Point);
extern uchar* byteaddr(Memimage*, Point);

extern int  drawclip(Memimage*, Rectangle*, Memimage*, Point*, Memimage*, Point*, Rectangle*, Rectangle*);
extern void  memfillcolor(Memimage*, ulong);
extern int  memsetchan(Memimage*, ulong);

/*
 * Graphics
 */
extern void memimageinit(void);

// actually in memlayer
extern void memdraw(Memimage*, Rectangle, Memimage*, Point, Memimage*, Point, int);
extern void memimagedraw(Memimage*, Rectangle, Memimage*, Point, Memimage*, Point, int);

// actually in memlayer/
extern void memline(Memimage*, Point, Point, int, int, int, Memimage*, Point, int);


extern void mempoly(Memimage*, Point*, int, int, int, int, Memimage*, Point, int);
extern void memfillpoly(Memimage*, Point*, int, int, Memimage*, Point, int);


extern void memellipse(Memimage*, Point, int, int, int, Memimage*, Point, int);

extern void memarc(Memimage*, Point, int, int, int, Memimage*, Point, int, int, int);

extern Point memimagestring(Memimage*, Point, Memimage*, Point, Memsubfont*, char*);

// TODO: cruft private?
// !!!
extern int hwdraw(Memdrawparam*);


extern Rectangle memlinebbox(Point, Point, int, int, int);
extern int   memlineendsize(int);
```

```
extern void   _memmkcmap(void);

/*
 * Subfont management
 */
extern Point  memsubfontwidth(Memsubfont*, char*);
extern Memsubfont* getmemdefont(void);
extern Memsubfont* allocmemsubfont(char*, int, int, int, Fontchar*, Memimage*);

/*
 * Predefined
 */
extern Memimage* memwhite;
extern Memimage* memblack;
extern Memimage* memopaque;
extern Memimage* memtransparent;

extern Memcmap *memdefcmap;


// Forward decl? or should be fixed?
extern void _memimageline(Memimage*, Point, Point, int, int, int, Memimage*, Point, Rectangle, int);
//todo: remove this one and rename the previous one
//extern void memimageline(Memimage*, Point, Point, int, int, int, Memimage*, Point, int);

extern void _memfillpolysc(Memimage*, Point*, int, int, Memimage*, Point, int, int, int, int);

// used by color.c and alphadraw.c now (and test files)
⟨function RGB2K 247c⟩

/*
 * Kernel interface
 */
void  memimagemove(void*, void*);

/*
 * Kernel cruft
 */
extern void rdb(void);
extern int (*iprint)(char*, ...);
#pragma varargck argpos iprint 1

extern int  drawdebug;

/*
 * doprint interface: numbconv bit strings
 */
#pragma varargck type "llb" vlong
#pragma varargck type "llb" uvlong
#pragma varargck type "lb" long
#pragma varargck type "lb" ulong
#pragma varargck type "b" int
#pragma varargck type "b" uint
```

G.1.7 include/graphics/internals/memlayer.h

⟨include/graphics/internals/memlayer.h 395⟩≡

```
#pragma src "/sys/src/libmemlayer"
```

```
#pragma lib "libmemlayer.a"
// This file assumes you have included draw.h before.


typedef struct Memscreen Memscreen;
typedef void (*Refreshfn)(Memimage*, Rectangle, void*);

⟨struct Memscreen 199a⟩

⟨struct Memlayer 199e⟩

/*
 * These functions accept local coordinates
 */
int   memload(Memimage*, Rectangle, uchar*, int, int);
int   memunload(Memimage*, Rectangle, uchar*, int);

/*
 * All these functions accept screen coordinates, not local ones.
 */

Memimage*  memlalloc(Memscreen*, Rectangle, Refreshfn, void*, ulong);
void   memlfree(Memimage*);
void   memldelete(Memimage*);

int    memlorigin(Memimage*, Point, Point);

void   memltofront(Memimage*);
void   memltofrontn(Memimage**, int);
void   memltorear(Memimage*);
void   memltorearn(Memimage**, int);
void   memlhide(Memimage*, Rectangle);
void   memlexpose(Memimage*, Rectangle);

int    memlsetrefresh(Memimage*, Refreshfn, void*);
void   memlnorefresh(Memimage*, Rectangle, void*);

void   _memlayerop(void (*fn)(Memimage*, Rectangle, Rectangle, void*, int), Memimage*, Rectangle, Rectangle, vo

void   _memltofrontfill(Memimage*, int);
void   _memlsetclear(Memscreen*);
```

G.1.8 include/graphics/input/keyboard.h

⟨include/graphics/input/keyboard.h 396⟩≡

```
#pragma src "/sys/src/libdraw"
#pragma lib "libdraw.a"

typedef struct  Keyboardctl Keyboardctl;
typedef struct Channel Channel;

⟨struct Keyboardctl 194a⟩

extern Keyboardctl* initkeyboard(char*);
extern int    ctlkeyboard(Keyboardctl*, char*);
extern void   closekeyboard(Keyboardctl*);

⟨enum Kxxx 193c⟩
```

G.1.9 include/graphics/input/mouse.h

⟨include/graphics/input/mouse.h 397a⟩≡

```
#pragma src "/sys/src/libdraw"

typedef struct Channel Channel;
typedef struct Cursor Cursor;
typedef struct Menu Menu;
typedef struct  Mousectl Mousectl;

⟨enum Click 183c⟩

⟨type buttons 183b⟩

⟨struct Mouse 183a⟩

⟨struct Mousectl 183d⟩

//TODO: mv in menu.h?
⟨struct Menu 272d⟩

/*
 * Mouse
 */
extern Mousectl* initmouse(char*, Image*);
extern void   closemouse(Mousectl*);

extern int   readmouse(Mousectl*);
extern void   moveto(Mousectl*, Point);
extern void   setcursor(Mousectl*, Cursor*);

//TODO? mv in menu.h?
extern void   drawgetrect(Rectangle, int);
extern Rectangle getrect(int, Mousectl*);

extern int    menuhit(int, Mousectl*, Menu*, Screen*);
```

G.1.10 include/graphics/cursor.h

⟨include/graphics/cursor.h 397b⟩≡

```
#pragma src "/sys/src/libdraw"

typedef struct Cursor Cursor;

⟨struct Cursor 186e⟩
```

G.1.11 include/graphics/input/event.h

⟨include/graphics/input/event.h 397c⟩≡

```
#pragma src "/sys/src/libdraw"
#pragma lib "libdraw.a"

typedef struct Cursor Cursor;
typedef struct Event Event;
typedef struct Menu Menu;
//typedef struct Mouse Mouse;

⟨type keys 283d⟩
```

⟨*enum* Key 283c⟩

⟨*enum* _anon_ (include/event.h)2 284d⟩

⟨*type* buttons 183b⟩

⟨*struct Mouse (include/event.h)* 284b⟩

⟨*enum* Click 183c⟩

⟨*struct* Event 284a⟩

⟨*struct Menu (include/event.h)* 284c⟩

```
/*
 * Events
 */
extern void  einit(ulong);
extern ulong estart(ulong, int, int);
extern ulong estartfn(ulong, int, int, int (*fn)(int, Event*, uchar*, int));
extern ulong  event(Event*);
extern ulong  eread(ulong, Event*);
extern int  ecanread(ulong);

extern Mouse emouse(void);
extern int   ekbd(void);
extern ulong etimer(ulong, int);

extern int   ecanmouse(void);
extern int   ecankbd(void);

//ugly!!!
extern void  eresized(int); /* supplied by user */

extern int   emenuhit(int, Mouse*, Menu*);
extern void   esetcursor(Cursor*);
extern void   emoveto(Point);
extern void  edrawgetrect(Rectangle, int);
extern int  ereadmouse(Mouse*);
extern int  eatomouse(Mouse*, char*, int);

extern Rectangle getrect(int, Mouse*);
extern Rectangle egetrect(int, Mouse*);

// from plan9front
extern int  eenter(char*, char*, int, Mouse*);
```

G.2 lib_graphics/libdraw/

G.2.1 lib_graphics/libdraw/arith.c

⟨lib_graphics/libdraw/arith.c 398⟩≡

```
#include <u.h>
#include <libc.h>
#include <draw.h>
```

⟨*function* Pt 31b⟩

⟨*function* Rect 33a⟩

⟨*function* Rpt 33b⟩

⟨*function* addpt 32a⟩

⟨*function* subpt 32b⟩

⟨*function* insetrect 90b⟩

⟨*function* divpt 32d⟩

⟨*function* mulpt 32c⟩

⟨*function* rectsubpt 34a⟩

⟨*function* rectaddpt 33g⟩

⟨*function* eqpt 31c⟩

⟨*function* eqrect 33c⟩

⟨*function* rectXrect 34d⟩

⟨*function* rectinrect 34c⟩

⟨*function* ptinrect 34b⟩

⟨*function* canonrect 32f⟩

⟨*function* combinerect 82a⟩

⟨*global* ZP 31d⟩
⟨*global* ZR 33d⟩

G.2.2 lib_graphics/libdraw/icossin.c

⟨lib_graphics/libdraw/icossin.c 399a⟩≡

```
#include <u.h>
#include <libc.h>
#include <draw.h>
```

⟨*global* sinus 354⟩

⟨*function* icossin 355⟩

G.2.3 lib_graphics/libdraw/icossin2.c

⟨lib_graphics/libdraw/icossin2.c 399b⟩≡

```
#include <u.h>
#include <libc.h>
#include <draw.h>
```

⟨*global* sinus*(*(lib_graphics/libdraw/icossin2.c)*)* 356⟩

⟨*global* cosinus 358⟩

⟨*function* icossin2 360⟩

G.2.4 lib_graphics/libdraw/resolution.c

⟨lib_graphics/libdraw/resolution.c 400a⟩≡

```
#include <u.h>
#include <libc.h>
#include <draw.h>
```

⟨*function* unitsperline 52g⟩

⟨*function* wordsperline 52e⟩

⟨*function* bytesperline 52f⟩

G.2.5 lib_graphics/libdraw/chan.c

⟨lib_graphics/libdraw/chan.c 400b⟩≡

```
#include <u.h>
#include <libc.h>
#include <draw.h>
```

⟨*global* channames 179c⟩
⟨*function* chantostr 350a⟩

⟨*function* isspace 179e⟩

⟨*function* strtochan 179d⟩

⟨*function* chantodepth 41c⟩

G.2.6 lib_graphics/libdraw/concurrency.c

⟨lib_graphics/libdraw/concurrency.c 400c⟩≡

```
#include <u.h>
#include <libc.h>
#include <draw.h>
```

⟨*global* debuglockdisplay 348c⟩

⟨*function* lockdisplay 311b⟩

⟨*function* unlockdisplay 311c⟩

G.2.7 lib_graphics/libdraw/init.c

⟨lib_graphics/libdraw/init.c 400d⟩≡

```
#include <u.h>
#include <libc.h>
#include <draw.h>
#include <draw_private.h>
#include <window.h>
#include <font.h>
```

⟨*global* deffontname 143g⟩

```
static void _closedisplay(Display*, int);
```

⟨*function* drawshutdown 59b⟩

⟨*function* geninitdraw 55b⟩

⟨*function* initdraw 55a⟩

⟨*constant* NINFO 55e⟩

⟨*function* initdisplay 56⟩

⟨*function* closedisplay 59a⟩

⟨*function* _closedisplay 59c⟩

G.2.8 lib_graphics/libdraw/globals.c

⟨lib_graphics/libdraw/globals.c 401a⟩≡

```
#include <u.h>
#include <libc.h>
#include <draw.h>
```

⟨*global* display 34f⟩
⟨*global* view 232a⟩
⟨*global* font 137b⟩

G.2.9 lib_graphics/libdraw/error.c

⟨lib_graphics/libdraw/error.c 401b⟩≡

```
#include <u.h>
#include <libc.h>
#include <draw.h>
```

⟨*function* drawerror 352c⟩

G.2.10 lib_graphics/libdraw/flush.c

⟨lib_graphics/libdraw/flush.c 401c⟩≡

```
#include <u.h>
#include <libc.h>
#include <draw.h>
#include <draw_private.h>
```

⟨*function* doflush 70d⟩

⟨*function* flushimage 80d⟩

⟨*function* bufimage 70a⟩

G.2.11 lib_graphics/libdraw/alloc.c

⟨lib_graphics/libdraw/alloc.c 402a⟩≡

```
#include <u.h>
#include <libc.h>
#include <draw.h>
#include <draw_private.h>

⟨function allocimage 72a⟩

⟨function _allocimage 72c⟩

⟨function _freeimage1 74d⟩

⟨function freeimage 72b⟩

⟨function allocimagemix 274c⟩
```

G.2.12 lib_graphics/libdraw/naming.c

⟨lib_graphics/libdraw/naming.c 402b⟩≡

```
#include <u.h>
#include <libc.h>
#include <draw.h>
#include <draw_private.h>

⟨function namedimage 83a⟩

⟨function nameimage 82c⟩
```

G.2.13 lib_graphics/libdraw/misc.c

⟨lib_graphics/libdraw/misc.c 402c⟩≡

```
#include <u.h>
#include <libc.h>
#include <draw.h>

⟨global drawld2chan 180d⟩
```

G.2.14 lib_graphics/libdraw/debug.c

⟨lib_graphics/libdraw/debug.c 402d⟩≡

```
#include <u.h>
#include <libc.h>
#include <draw.h>
#include <draw_private.h>

⟨global _drawdebug 348a⟩

⟨function drawsetdebug 348e⟩
```

G.2.15 lib_graphics/libdraw/fmt.c

⟨lib_graphics/libdraw/fmt.c 403a⟩≡

```
#include <u.h>
#include <libc.h>
#include <draw.h>

⟨function Rfmt 349g⟩

⟨function Pfmt 349f⟩
```

G.2.16 lib_graphics/libdraw/defont.c

⟨*global* defontdata 403b⟩≡ (409a)

```
/*
 * lucm/latin1.9, in uncompressed form
 */
byte
defontdata[] =
{
0x20,0x20,0x20,0x20,0x20,0x20,0x20,0x20,0x20,0x20,0x30,0x20,0x20,0x20,0x20,0x20,
0x20,0x20,0x20,0x20,0x20,0x20,0x30,0x20,0x20,0x20,0x20,0x20,0x20,0x20,0x20,0x20,
0x20,0x20,0x30,0x20,0x20,0x20,0x20,0x20,0x20,0x20,0x20,0x32,0x33,0x30,0x34,0x20,
0x20,0x20,0x20,0x20,0x20,0x20,0x20,0x20,0x20,0x31,0x35,0x20,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0xff,0xff,0xff,0xff,
0xff,0xff,0xff,0xff,0xff,0xff,0xff,0xff,0xff,0xff,0xff,0xff,0xff,0xff,0xff,0xff,
0xff,0xff,0xff,0xff,0xff,0xff,0xff,0xff,0xff,0xff,0xff,0xff,0xff,0xff,0xff,0xff,
0xff,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x30,0x06,0x06,0x03,0x42,0x40,0x00,0x00,0x00,0x18,0x03,0x03,
0x02,0x43,0x00,0x60,0x60,0x48,0x00,0x0d,0x0c,0x01,0x81,0x80,0xd0,0x90,0x00,0x00,
0x18,0x01,0x81,0x81,0x40,0x60,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x18,0x7f,0x9c,0x1c,
0x0e,0x07,0x01,0x80,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0xe0,0x70,
0x38,0x1c,0x0e,0x04,0x81,0xc1,0xc0,0x70,0x00,0x1c,0x1c,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0xaa,0x80,0xc0,0x63,0xe3,
0xf1,0xf8,0xfe,0x7f,0xff,0xff,0xff,0xff,0xff,0xff,0xfc,0x7f,0xff,0xff,0x1f,0x8f,
0xc7,0xe3,0xf1,0xfb,0x7e,0x3e,0x3f,0x8f,0xff,0xe3,0xe3,0xff,0xff,0xff,0xff,0xff,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x0c,0x18,0x09,0x05,0x82,0x40,0xc0,0x00,0x00,0x06,0x0c,0x04,
0x82,0x40,0xc1,0x80,0x90,0x48,0x00,0x16,0x03,0x06,0x02,0x41,0x60,0x90,0x00,0x00,
0x06,0x06,0x02,0x41,0x41,0x80,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
```

```
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x3e,0x7f,0xa0,0x10,
0x08,0x04,0x02,0x40,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x90,0x48,
0x24,0x12,0x09,0x06,0x82,0x01,0x00,0x90,0x00,0x20,0x10,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x04,0x80,0x00,0x40,0x00,0x00,0x38,0x06,0x18,0x00,0x00,0x00,0x00,0x00,
0x00,0x06,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x07,0xc6,0x01,0xf0,0x00,0x00,0x0c,0x00,0x18,0x00,0x00,0x30,0x00,0x3c,
0x00,0x60,0x06,0x01,0x8c,0x07,0xc0,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x01,0xe0,0xc3,0xc0,0x01,0x54,0x9c,0xc0,0x5f,0xef,
0xf7,0xfb,0xfd,0xbf,0xff,0xff,0xff,0xff,0xff,0xff,0xfb,0x7f,0xff,0xff,0x6f,0xb7,
0xdb,0xed,0xf6,0xf9,0x7d,0xfe,0xff,0x6f,0xff,0xdf,0xef,0xff,0xff,0xff,0xff,0xff,
0xff,0x00,0x01,0x00,0x00,0x00,0x00,0x30,0x00,0x14,0x00,0x00,0x00,0x00,0x00,0x00,
0x00,0x38,0x00,0x00,0x00,0x00,0x00,0x60,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x01,0x20,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x78,0x30,0x06,0x06,0x06,0x82,0x80,0xc0,0x00,
0x00,0x18,0x03,0x03,0x02,0x41,0x80,0x30,0x30,0x24,0x76,0x0d,0x0c,0x00,0xc0,0xc0,
0xd0,0x50,0x00,0x00,0x18,0x01,0x81,0x81,0x40,0x30,0x00,0x28,0x0f,0x7f,0xbc,0x1c,
0x0e,0x07,0x03,0xc0,0x10,0x70,0x24,0x10,0x09,0x07,0x03,0x80,0xe0,0x70,0x90,0x48,
0x24,0x12,0x09,0x05,0x81,0x81,0xc0,0x80,0x70,0x18,0x1c,0x07,0x01,0xc1,0xc0,0x90,
0x00,0x0c,0x04,0x84,0x83,0xe1,0xc0,0xe0,0x38,0x0c,0x0c,0x02,0x00,0x00,0x00,0x00,
0x00,0x06,0x1c,0x06,0x0f,0x87,0xc0,0x63,0xf8,0x78,0xfe,0x3e,0x0e,0x00,0x00,0x00,
0x00,0x00,0x00,0x7c,0x1c,0x0c,0x1f,0x03,0xc7,0xc3,0xf1,0xf8,0x3c,0x63,0x3f,0x0f,
0x8c,0x66,0x06,0x19,0x84,0x78,0x7e,0x1e,0x1f,0x07,0xcf,0xf3,0x1b,0x0d,0x86,0x63,
0x61,0x9f,0xc6,0x06,0x00,0x30,0x00,0x00,0x10,0x00,0x18,0x00,0x00,0x30,0x00,0x60,
0x00,0x60,0x06,0x01,0x8c,0x00,0xc0,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x01,0x80,
0x00,0x00,0x00,0x00,0x00,0x00,0x03,0x00,0xc0,0x60,0x00,0xaa,0xb6,0xc0,0x43,0xe3,
0xf1,0xf8,0xfc,0x3f,0xef,0x8f,0xdb,0xef,0xf6,0xf8,0xfb,0xff,0x1f,0x8f,0x6f,0xb7,
0xdb,0xed,0xf6,0xfa,0x7e,0x7e,0x3f,0x7f,0x8f,0xe7,0xe3,0xf8,0xfe,0x3e,0x3f,0x6f,
0x00,0x00,0x01,0x01,0xc8,0x0b,0x0c,0x30,0x7c,0x14,0x0f,0x0f,0x00,0x00,0x00,0x00,
0x78,0x00,0x1c,0x00,0x0f,0x07,0x81,0x80,0x00,0x7c,0x00,0x00,0x1c,0x0f,0x80,0x04,
0x42,0x23,0x90,0x00,0x18,0x0c,0x06,0x03,0x01,0x80,0xc0,0x3c,0x3c,0x3f,0x1f,0x8f,
0xc7,0xe7,0xe3,0xf1,0xf8,0xfc,0x7c,0x30,0x8f,0x07,0x83,0xc1,0xe0,0xf0,0x00,0x3d,
0x31,0x98,0xcc,0x66,0x36,0x19,0x80,0xcc,0x0c,0x18,0x09,0x0b,0x02,0x81,0x20,0x00,
0x00,0x06,0x0c,0x04,0x82,0x40,0x60,0xc0,0x48,0x24,0x18,0x16,0x03,0x03,0x01,0x21,
0x60,0x50,0x00,0x00,0x06,0x06,0x02,0x41,0x40,0xc1,0x80,0x28,0x87,0x7f,0x84,0x10,
0x08,0x04,0x02,0x40,0x38,0x48,0x24,0x10,0x09,0x04,0x04,0x81,0x00,0x80,0x90,0x48,
0x24,0x12,0x09,0x04,0x80,0x41,0x00,0x80,0x40,0x04,0x10,0x04,0x02,0x01,0x20,0x90,
0x00,0x0c,0x04,0x84,0x86,0x53,0x65,0xb0,0x08,0x18,0x06,0x0a,0x80,0x00,0x00,0x00,
0x00,0x0c,0x36,0x0e,0x19,0xcc,0xe0,0xe3,0xf8,0xcc,0xfe,0x63,0x1b,0x00,0x00,0x00,
0x00,0x00,0x00,0xc6,0x62,0x0c,0x19,0x86,0x66,0x63,0x01,0x80,0x66,0x63,0x0c,0x01,
0x8c,0xc6,0x06,0x19,0xc4,0xcc,0x63,0x33,0x19,0x8c,0x61,0x83,0x1b,0x0d,0x86,0x63,
0x61,0x80,0xc6,0x03,0x00,0x30,0x30,0x00,0x1c,0x00,0x18,0x00,0x00,0x30,0x00,0x60,
0x00,0x60,0x00,0x00,0x0c,0x00,0xc0,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x01,0x80,
0x00,0x00,0x00,0x00,0x00,0x00,0x03,0x00,0xc0,0x60,0x01,0x54,0x86,0xc0,0x7b,0xef,
0xf7,0xfb,0xfd,0xbf,0xc7,0xb7,0xdb,0xef,0xf6,0xfb,0xfb,0x7e,0xff,0x7f,0x6f,0xb7,
0xdb,0xed,0xf6,0xfb,0x7f,0xbe,0xff,0x7f,0xbf,0xfb,0xef,0xfb,0xfd,0xfe,0xdf,0x6f,
0xff,0x00,0x07,0x83,0x24,0x13,0x0c,0x30,0xc6,0x00,0x10,0x81,0x80,0x00,0x00,0x00,
0x84,0x00,0x22,0x00,0x01,0x80,0xc0,0x00,0x00,0xf4,0x00,0x00,0x2c,0x18,0xc0,0x0c,
0x46,0x20,0x90,0x00,0x18,0x0c,0x06,0x03,0x01,0x80,0xc0,0x70,0x66,0x30,0x18,0x0c,
0x06,0x01,0x80,0xc0,0x60,0x30,0x66,0x38,0x99,0x8c,0xc6,0x63,0x31,0x98,0x00,0x66,
0x31,0x98,0xcc,0x66,0x36,0x19,0x80,0xcc,0x00,0x00,0x00,0x00,0x00,0x00,0xc0,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x6c,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x01,0x80,0x00,0xff,0x7f,0xb8,0x1c,
0x0e,0x07,0x02,0x40,0x7c,0x70,0x3c,0x10,0x09,0x07,0x04,0x00,0xc0,0x60,0xe0,0x70,
0x38,0x1c,0x0e,0x04,0x83,0x81,0xc0,0x70,0x70,0x38,0x1c,0x07,0x02,0xc1,0xc0,0x90,
0x00,0x0c,0x00,0x04,0x86,0x43,0x69,0xb0,0x30,0x18,0x06,0x07,0x01,0x00,0x00,0x00,
0x00,0x0c,0x63,0x16,0x00,0xc0,0x61,0x62,0x01,0x80,0x06,0x63,0x31,0x80,0x00,0x00,
```

```
0x60,0x00,0xc0,0x06,0x43,0x16,0x19,0x8c,0x06,0x33,0x01,0x80,0xc0,0x63,0x0c,0x01,
0x8c,0x86,0x07,0x39,0xc5,0x86,0x63,0x61,0x99,0x8c,0x01,0x83,0x1b,0x0d,0xb6,0x63,
0x31,0x01,0x86,0x03,0x00,0x30,0x30,0x00,0x1c,0x3e,0x1b,0x03,0xc1,0xf0,0xf0,0x60,
0x3e,0x6e,0x3e,0x0f,0x8c,0x60,0xc5,0xb1,0xb8,0x38,0x6c,0x0f,0x8c,0xc7,0xc1,0x83,
0x19,0x8d,0x82,0x63,0x31,0x9f,0xc1,0x80,0xc0,0xc0,0x00,0xaa,0x86,0xc0,0x47,0xe3,
0xf1,0xf8,0xfd,0xbf,0x83,0x8f,0xc3,0xef,0xf6,0xf8,0xfc,0xff,0x3f,0x9f,0x1f,0x8f,
0xc7,0xe3,0xf1,0xfb,0x7c,0x7e,0x3f,0x8f,0x8f,0xc7,0xe3,0xf8,0xfd,0x3e,0x3f,0x6f,
0x00,0x0c,0x0d,0x43,0x03,0xe1,0x88,0x30,0xc0,0x00,0x27,0x41,0x80,0x00,0x00,0x01,
0x72,0x00,0x22,0x04,0x01,0x80,0xc0,0x03,0x18,0xf4,0x00,0x00,0x0c,0x18,0xc0,0x04,
0x82,0x43,0x20,0x18,0x2c,0x16,0x0b,0x05,0x82,0xc1,0x60,0xb0,0xc0,0x30,0x18,0x0c,
0x06,0x01,0x80,0xc0,0x60,0x30,0x63,0x38,0xb0,0xd8,0x6c,0x36,0x1b,0x0c,0x00,0xc7,
0x31,0x98,0xcc,0x66,0x33,0x11,0xf8,0xc8,0x7c,0x3e,0x1f,0x0f,0x87,0xc3,0xe1,0xd8,
0x3c,0x1e,0x0f,0x07,0x83,0xc7,0xc3,0xe1,0xf0,0xf8,0x06,0x37,0x07,0x03,0x81,0xc0,
0xe0,0x70,0x10,0x1d,0x31,0x98,0xcc,0x66,0x33,0x19,0xb0,0xc6,0x8f,0x7f,0x87,0x03,
0x81,0x80,0x90,0x30,0x6c,0x48,0x24,0x10,0x06,0x04,0x04,0x80,0x20,0x10,0x10,0x0e,
0x07,0x03,0x81,0xc0,0x60,0x88,0x38,0x0c,0x40,0x09,0x03,0x84,0x02,0x41,0x40,0x90,
0x00,0x0c,0x00,0x1f,0xe7,0x41,0xd1,0xa0,0x00,0x30,0x03,0x0a,0x81,0x00,0x00,0x00,
0x00,0x18,0x63,0x06,0x00,0xc0,0xc2,0x62,0x01,0xb0,0x0c,0x72,0x31,0x86,0x03,0x00,
0xc0,0x00,0x60,0x06,0x8f,0x16,0x19,0x0c,0x06,0x33,0x01,0x80,0xc0,0x63,0x0c,0x01,
0x8d,0x06,0x07,0x39,0x65,0x86,0x63,0x61,0x99,0x0e,0x01,0x83,0x19,0x89,0xb6,0x32,
0x33,0x03,0x06,0x01,0x80,0x30,0x78,0x00,0x00,0x03,0x1d,0x86,0x23,0x31,0x99,0xfc,
0x66,0x77,0x06,0x01,0x8c,0x40,0xc6,0xd9,0xdc,0x6c,0x76,0x19,0x8d,0xcc,0x27,0xf3,
0x19,0x8d,0x82,0x63,0x31,0x80,0xc0,0x80,0xc0,0x80,0x01,0x54,0x8c,0xc0,0x78,0xfc,
0x7e,0x7f,0x6f,0xcf,0x93,0xb7,0xdb,0xef,0xf9,0xfb,0xff,0xff,0xdf,0xef,0xef,0xf1,
0xf8,0xfc,0x7e,0x3f,0x9f,0x77,0xc7,0xf3,0xbf,0xf6,0xfc,0x7b,0xfd,0xbe,0xbf,0x6f,
0xff,0x0c,0x19,0x03,0x03,0x61,0x98,0x30,0x78,0x00,0x28,0x4f,0x83,0x30,0x00,0x01,
0x4a,0x00,0x1c,0x04,0x03,0x03,0x80,0x03,0x18,0xf4,0x00,0x00,0x0c,0x18,0xd9,0x84,
0x82,0x40,0xa0,0x18,0x2c,0x16,0x0b,0x05,0x82,0xc1,0x60,0xb0,0xc0,0x30,0x18,0x0c,
0x06,0x01,0x80,0xc0,0x60,0x30,0x63,0x2c,0xb0,0xd8,0x6c,0x36,0x1b,0x0c,0x64,0xcb,
0x31,0x98,0xcc,0x66,0x33,0x31,0x8c,0xd8,0x06,0x03,0x01,0x80,0xc0,0x60,0x30,0x6c,
0x62,0x33,0x19,0x8c,0xc6,0x60,0xc0,0x60,0x30,0x18,0x1e,0x3b,0x8d,0x86,0xc3,0x61,
0xb0,0xd8,0x10,0x36,0x31,0x98,0xcc,0x66,0x33,0x19,0xd8,0xc6,0x0f,0x7f,0x82,0x01,
0x02,0x40,0xd0,0x40,0x6c,0x70,0x24,0x1c,0x06,0x04,0x03,0x01,0xc0,0xe0,0x10,0x12,
0x09,0x04,0x82,0x40,0x90,0x50,0x10,0x12,0x70,0x09,0x04,0x04,0x01,0xc1,0x20,0x60,
0x00,0x0c,0x00,0x04,0x83,0xc0,0x20,0xcc,0x00,0x30,0x03,0x02,0x01,0x00,0x00,0x00,
0x00,0x18,0x63,0x06,0x01,0x83,0x84,0x63,0xf1,0xd8,0x18,0x3c,0x31,0x86,0x03,0x01,
0x83,0xf8,0x30,0x1c,0x9b,0x33,0x1e,0x0c,0x06,0x33,0xe1,0x80,0xc0,0x7f,0x0c,0x01,
0x8f,0x06,0x07,0x79,0x65,0x86,0x66,0x61,0x9e,0x07,0x81,0x83,0x19,0x89,0xb6,0x1c,
0x1a,0x03,0x06,0x01,0x80,0x30,0x48,0x00,0x00,0x03,0x18,0xcc,0x06,0x33,0x18,0x60,
0xc6,0x63,0x06,0x01,0x8c,0x80,0xc6,0xd9,0x8c,0xc6,0x63,0x31,0x8e,0x4c,0x01,0x83,
0x19,0x8d,0x92,0x32,0x31,0x81,0x87,0x00,0xc0,0x70,0xe4,0xaa,0x98,0xc0,0x7d,0xfe,
0xfd,0xbf,0x2f,0xbf,0x93,0x8f,0xdb,0xe3,0xf9,0xfb,0xff,0x1e,0x3f,0x1f,0xef,0xed,
0xf6,0xfb,0x7d,0xbf,0x6f,0xaf,0xef,0xed,0x8f,0xf6,0xfb,0xfb,0xfe,0x3e,0xdf,0x9f,
0x00,0x00,0x19,0x0f,0xc6,0x30,0xd0,0x00,0xcc,0x00,0x28,0x59,0x86,0x67,0xf0,0x01,
0x72,0x00,0x00,0x3f,0x86,0x00,0xc0,0x03,0x18,0xf4,0x00,0x00,0x0c,0x18,0xcc,0xc5,
0x32,0x83,0x4c,0x00,0x66,0x33,0x19,0x8c,0xc6,0x63,0x31,0xbc,0xc0,0x3e,0x1f,0x0f,
0x87,0xc1,0x80,0xc0,0x60,0x30,0xfb,0x2c,0xb0,0xd8,0x6c,0x36,0x1b,0x0c,0x38,0xcb,
0x31,0x98,0xcc,0x66,0x31,0xa1,0x8c,0xcc,0x06,0x03,0x01,0x80,0xc0,0x60,0x30,0x6c,
0xc0,0x63,0x31,0x98,0xcc,0x60,0xc0,0x60,0x30,0x18,0x37,0x31,0x98,0xcc,0x66,0x33,
0x19,0x8c,0x00,0x67,0x31,0x98,0xcc,0x66,0x33,0x19,0x8c,0xc6,0x1f,0x7f,0x82,0x01,
0x02,0x40,0xb0,0x40,0x6c,0x07,0x03,0x83,0x80,0xe0,0xe0,0x00,0x18,0x0e,0x10,0x10,
0x08,0x04,0x02,0x00,0xf0,0x20,0x10,0x1e,0x08,0x89,0x03,0x00,0xe0,0x38,0x1c,0x0e,
0x00,0x0c,0x00,0x04,0x81,0xe0,0x41,0x6c,0x00,0x30,0x03,0x00,0x0f,0xe0,0x03,0xf8,
0x00,0x30,0x63,0x06,0x03,0x00,0xc7,0xf0,0x39,0x8c,0x30,0x3e,0x1b,0x80,0x00,0x03,
0x00,0x00,0x18,0x30,0x9b,0x23,0x19,0x0c,0x06,0x33,0x01,0xf8,0xc6,0x63,0x0c,0x01,
0x8d,0x86,0x05,0xd9,0x35,0x86,0x7c,0x61,0x9b,0x01,0xc1,0x83,0x19,0x99,0xb4,0x1c,
0x0c,0x06,0x06,0x00,0xc0,0x30,0xcc,0x00,0x00,0x3f,0x18,0xcc,0x06,0x33,0xf8,0x60,
0xc6,0x63,0x06,0x01,0x8f,0x00,0xc6,0xd9,0x8c,0xc6,0x63,0x31,0x8c,0x0f,0x81,0x83,
0x18,0xd9,0xba,0x1c,0x1b,0x03,0x00,0x80,0xc0,0x81,0x75,0x54,0x98,0xc0,0x7d,0xfe,
0xfd,0xbf,0x4f,0xbf,0x93,0xf8,0xfc,0x7c,0x7f,0x1f,0x1f,0x6f,0xe7,0xf1,0xef,0xef,
```

```
0xf7,0xfb,0xfd,0xff,0x0f,0xdf,0xef,0xe1,0xf7,0x76,0xfc,0xff,0x1f,0xc7,0xe3,0xf1,
0xff,0x08,0x19,0x03,0x06,0x31,0xf8,0x00,0xc6,0x00,0x28,0x5b,0x8c,0xc0,0x11,0xf1,
0x4a,0x00,0x00,0x04,0x0c,0x00,0xc0,0x03,0x18,0x74,0x38,0x00,0x0c,0x18,0xc6,0x65,
0x52,0xb8,0x54,0x18,0x46,0x23,0x11,0x88,0xc4,0x62,0x31,0x30,0xc0,0x30,0x18,0x0c,
0x06,0x01,0x80,0xc0,0x60,0x30,0x63,0x26,0xb0,0xd8,0x6c,0x36,0x1b,0x0c,0x10,0xd3,
0x31,0x98,0xcc,0x66,0x30,0xc1,0x8c,0xc6,0x7e,0x3f,0x1f,0x8f,0xc7,0xe3,0xf1,0xfc,
0xc0,0x7f,0x3f,0x9f,0xcf,0xe0,0xc0,0x60,0x30,0x18,0x63,0x31,0x98,0xcc,0x66,0x33,
0x19,0x8c,0xfe,0x6b,0x31,0x98,0xcc,0x66,0x31,0xb1,0x8c,0x6c,0x0e,0x7f,0x82,0x01,
0x01,0x80,0x90,0x30,0xc6,0x08,0x01,0x02,0x00,0x40,0x80,0xe0,0x24,0x04,0x1c,0x10,
0x08,0x04,0x02,0x00,0x90,0x20,0x10,0x12,0x0d,0x86,0x00,0x81,0x00,0x40,0x20,0x10,
0x00,0x04,0x00,0x1f,0xe1,0x70,0xbb,0x28,0x00,0x30,0x03,0x00,0x01,0x00,0x00,0x00,
0x00,0x30,0x63,0x06,0x06,0x00,0x67,0xf0,0x19,0x8c,0x30,0x67,0x0d,0x80,0x00,0x01,
0x83,0xf8,0x30,0x30,0x9b,0x7f,0x19,0x8c,0x06,0x33,0x01,0x80,0xc6,0x63,0x0c,0x01,
0x8c,0xc6,0x05,0xd9,0x35,0x86,0x60,0x61,0x99,0x80,0xe1,0x83,0x18,0xd0,0xdc,0x26,
0x0c,0x0c,0x06,0x00,0xc0,0x30,0x84,0x00,0x00,0x63,0x18,0xcc,0x06,0x33,0x00,0x60,
0xc6,0x63,0x06,0x01,0x8d,0x80,0xc6,0xd9,0x8c,0xc6,0x63,0x31,0x8c,0x03,0xe1,0x83,
0x18,0xd9,0xba,0x1c,0x1b,0x06,0x01,0x80,0xc0,0xc1,0x38,0xaa,0x80,0xc0,0x7d,0xfe,
0xfe,0x7f,0x6f,0xcf,0x39,0xf7,0xfe,0xfd,0xff,0xbf,0x7f,0x0f,0xdb,0xfb,0xe3,0xef,
0xf7,0xfb,0xfd,0xff,0x6f,0xdf,0xef,0xed,0xf2,0x79,0xff,0x7e,0xff,0xbf,0xdf,0xef,
0x00,0x0c,0x19,0x03,0x03,0x60,0x60,0x30,0x66,0x00,0x28,0x4d,0xc6,0x60,0x10,0x00,
0x84,0x00,0x00,0x04,0x0f,0x87,0x80,0x03,0x18,0x14,0x38,0x00,0x3f,0x0f,0x8c,0xc2,
0x90,0x84,0xa4,0x18,0xfe,0x7f,0x3f,0x9f,0xcf,0xe7,0xf1,0xf0,0xc0,0x30,0x18,0x0c,
0x06,0x01,0x80,0xc0,0x60,0x30,0x63,0x26,0xb0,0xd8,0x6c,0x36,0x1b,0x0c,0x38,0xd3,
0x31,0x98,0xcc,0x66,0x30,0xc1,0x98,0xc6,0xc6,0x63,0x31,0x98,0xcc,0x66,0x33,0x60,
0xc0,0x60,0x30,0x18,0x0c,0x00,0xc0,0x60,0x30,0x18,0x63,0x31,0x98,0xcc,0x66,0x33,
0x19,0x8c,0x00,0x6b,0x31,0x98,0xcc,0x66,0x31,0xb1,0x8c,0x6c,0x1c,0x7f,0x81,0x20,
0x90,0x38,0x18,0x0b,0x83,0x06,0x01,0x03,0x80,0x40,0xe0,0x90,0x24,0x04,0x03,0x8e,
0x86,0xc3,0x61,0x90,0x24,0x12,0x0e,0x04,0x8a,0x81,0xc7,0x70,0xc0,0x30,0x18,0x0c,
0x00,0x00,0x00,0x04,0x81,0x31,0x6f,0x30,0x00,0x18,0x06,0x00,0x01,0x00,0x00,0x00,
0x00,0x60,0x63,0x06,0x0c,0x00,0x60,0x60,0x19,0x8c,0x60,0x63,0x01,0x80,0x00,0x00,
0xc0,0x00,0x60,0x00,0x4d,0xe1,0x99,0x8c,0x06,0x33,0x01,0x80,0xc6,0x63,0x0c,0x01,
0x8c,0xc6,0x04,0x99,0x1d,0x86,0x60,0x61,0x99,0x80,0x61,0x83,0x18,0xd0,0xdc,0x63,
0x0c,0x0c,0x06,0x00,0x60,0x30,0x84,0x00,0x00,0x63,0x18,0xcc,0x06,0x33,0x00,0x60,
0x6e,0x63,0x06,0x01,0x8c,0xc0,0xc6,0xd9,0x8c,0xc6,0x63,0x31,0x8c,0x00,0x61,0x83,
0x18,0xd0,0xcc,0x26,0x0e,0x0c,0x03,0x00,0xc0,0x60,0x01,0x54,0x98,0xc0,0x7e,0xdf,
0x6f,0xc7,0xe7,0xf4,0x7c,0xf9,0xfe,0xfc,0x7f,0xbf,0x1f,0x5f,0xdb,0xfb,0xfc,0x71,
0x79,0x3c,0x9e,0x6f,0xdb,0xed,0xf1,0xfb,0x75,0x7e,0x38,0x8f,0x3f,0xcf,0xe7,0xf3,
0xff,0x0c,0x0d,0x03,0x03,0xe1,0xf8,0x30,0x3c,0x00,0x27,0x40,0x03,0x30,0x00,0x00,
0x78,0x00,0x00,0x00,0x00,0x00,0x00,0x03,0x18,0x14,0x00,0x00,0x00,0x00,0x19,0x82,
0xf8,0x98,0xbe,0x70,0xc3,0x61,0xb0,0xd8,0x6c,0x36,0x1b,0x30,0xc0,0x30,0x18,0x0c,
0x06,0x01,0x80,0xc0,0x60,0x30,0x63,0x23,0xb0,0xd8,0x6c,0x36,0x1b,0x0c,0x4c,0xe3,
0x31,0x98,0xcc,0x66,0x30,0xc1,0xf0,0xc6,0xc6,0x63,0x31,0x98,0xcc,0x66,0x33,0x60,
0xc0,0x60,0x30,0x18,0x0c,0x00,0xc0,0x60,0x30,0x18,0x63,0x31,0x98,0xcc,0x66,0x33,
0x19,0x8c,0x10,0x73,0x31,0x98,0xcc,0x66,0x30,0xe1,0x8c,0x38,0x1c,0x7f,0x80,0xa0,
0x50,0x10,0x24,0x0d,0xff,0x01,0x01,0x02,0x00,0x40,0x80,0xf0,0x24,0x04,0x02,0x01,
0x81,0x20,0x10,0x30,0x28,0x1a,0x09,0x06,0x8a,0x81,0x20,0x90,0x20,0x08,0x04,0x02,
0x00,0x0c,0x00,0x04,0x85,0x32,0x6f,0xb8,0x00,0x18,0x06,0x00,0x01,0x01,0xc0,0x00,
0x70,0x60,0x36,0x06,0x1f,0xcc,0xe0,0x63,0x30,0xd8,0x60,0x63,0x33,0x06,0x03,0x00,
0x60,0x00,0xc0,0x30,0x60,0x61,0x99,0x86,0x66,0x63,0x01,0x80,0x66,0x63,0x0c,0x03,
0x0c,0x66,0x04,0x19,0x1c,0xcc,0x60,0x33,0x18,0xcc,0x61,0x81,0xb0,0x60,0xcc,0x63,
0x0c,0x18,0x06,0x00,0x60,0x30,0x00,0x00,0x00,0x67,0x19,0x86,0x23,0x71,0x88,0x60,
0x36,0x63,0x06,0x01,0x8c,0x60,0xc6,0xd9,0x8c,0x6c,0x66,0x1b,0x8c,0x08,0x61,0x83,
0xb8,0x70,0xcc,0x63,0x0c,0x18,0x03,0x00,0xc0,0x60,0x00,0xaa,0x98,0xc0,0x7f,0x5f,
0xaf,0xef,0xdb,0xf2,0x00,0xfe,0xfe,0xfd,0xff,0xbf,0x7f,0x6f,0xdb,0xfb,0xfd,0xfe,
0x7e,0xdf,0xef,0xcf,0xd7,0xe5,0xf6,0xf9,0x75,0x7e,0xdf,0x6f,0xdf,0xf7,0xfb,0xfd,
0x00,0x0c,0x07,0xc6,0x04,0x10,0x60,0x30,0x06,0x00,0x10,0x80,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x3f,0x80,0x00,0x00,0x03,0xb8,0x14,0x00,0x00,0x00,0x00,0x00,0x04,
0x11,0x21,0x04,0xc0,0xc3,0x61,0xb0,0xd8,0x6c,0x36,0x1b,0x30,0x66,0x30,0x18,0x0c,
0x06,0x01,0x80,0xc0,0x60,0x30,0x66,0x23,0x99,0x8c,0xc6,0x63,0x31,0x98,0x00,0x66,
0x1b,0x0d,0x86,0xc3,0x60,0xc1,0x80,0xc6,0xce,0x67,0x33,0x99,0xcc,0xe6,0x73,0x74,
```

```
0x62,0x31,0x18,0x8c,0x46,0x20,0xc0,0x60,0x30,0x18,0x36,0x31,0x8d,0x86,0xc3,0x61,
0xb0,0xd8,0x10,0x36,0x3b,0x9d,0xce,0xe7,0x70,0xc1,0x98,0x30,0x00,0x7f,0x80,0xc0,
0x60,0x10,0x24,0x0c,0x38,0x0e,0x01,0x02,0x00,0x40,0x80,0xa0,0x18,0x0e,0x03,0x00,
0x80,0x40,0x60,0x50,0x30,0x16,0x0e,0x05,0x88,0x81,0xc0,0x81,0xc0,0x70,0x38,0x1c,
0x00,0x0c,0x00,0x04,0x83,0xe0,0x39,0xcc,0x00,0x0c,0x0c,0x00,0x00,0x01,0xc0,0x00,
0x70,0xc0,0x1c,0x06,0x1f,0xc7,0xc0,0x61,0xe0,0x70,0x60,0x3e,0x1e,0x06,0x03,0x00,
0x00,0x00,0x00,0x30,0x1e,0x61,0x9f,0x03,0xc7,0xc3,0xf1,0x80,0x3e,0x63,0x3f,0x1e,
0x0c,0x67,0xe4,0x19,0x0c,0x78,0x60,0x1e,0x18,0xc7,0xc1,0x80,0xe0,0x60,0xcc,0x63,
0x0c,0x1f,0xc6,0x00,0x30,0x30,0x00,0x00,0x00,0x3b,0x9f,0x03,0xc1,0xb0,0xf0,0x60,
0x06,0x63,0x06,0x01,0x8c,0x70,0xc6,0xd9,0x8c,0x38,0x7c,0x0d,0x8c,0x07,0xc0,0xf1,
0xd8,0x60,0xcc,0x63,0x0c,0x1f,0xc3,0x00,0xc0,0x60,0x01,0x54,0x80,0xc0,0x7f,0x3f,
0x9f,0xef,0xdb,0xf3,0xc7,0xf1,0xfe,0xfd,0xff,0xbf,0x7f,0xff,0xe7,0xf1,0xfc,0xff,
0x7f,0xbf,0x9f,0xaf,0xcf,0xe9,0xf1,0xfa,0x77,0x7e,0x3f,0x7e,0x3f,0x8f,0xc7,0xe3,
0xff,0x0c,0x01,0x0f,0xe8,0x08,0x60,0x30,0xc6,0x00,0x0f,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x03,0xd8,0x14,0x00,0x00,0x00,0x00,0x00,0x04,
0x11,0x3d,0x04,0xc0,0xc3,0x61,0xb0,0xd8,0x6c,0x36,0x1b,0x3c,0x3c,0x3f,0x1f,0x8f,
0xc7,0xe7,0xe3,0xf1,0xf8,0xfc,0x7c,0x21,0x8f,0x07,0x83,0xc1,0xe0,0xf0,0x00,0xbc,
0x0e,0x07,0x03,0x81,0xc0,0xc1,0x80,0xcc,0x77,0x3b,0x9d,0xce,0xe7,0x73,0xb9,0x98,
0x3c,0x1e,0x0f,0x07,0x83,0xc0,0xc0,0x60,0x30,0x18,0x1c,0x31,0x87,0x03,0x81,0xc0,
0xe0,0x70,0x00,0x5c,0x1d,0x8e,0xc7,0x63,0xb0,0xc1,0xf0,0x30,0x00,0x7f,0x81,0x40,
0xa0,0x10,0x28,0x0a,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x90,0x00,0x00,0x02,0x00,
0x80,0x80,0x10,0xf8,0x28,0x12,0x09,0x04,0x80,0x01,0x20,0x80,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x01,0x00,0x00,0x00,0x00,0x06,0x18,0x00,0x00,0x00,0x40,0x00,
0x00,0xc0,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x01,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x03,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x07,0xc0,0x31,0xf0,0x01,0xff,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0xcc,0x00,0x00,0x01,0x80,0x00,0x00,0x00,0x00,0x00,0x60,0x01,0x80,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x18,0x00,0x01,0xe0,0xc3,0xc0,0x00,0x00,0xff,0xc0,0x7e,0xbf,
0x5f,0xef,0xd7,0xf5,0xff,0xff,0xff,0xff,0xff,0xff,0xff,0xff,0xff,0xff,0xfd,0xff,
0x7f,0x7f,0xef,0x07,0xd7,0xed,0xf6,0xfb,0x7f,0xfe,0xdf,0x7f,0xff,0xff,0xff,0xff,
0x00,0x0c,0x01,0x00,0x00,0x00,0x00,0x30,0x7c,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x03,0x00,0x14,0x00,0x08,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0xc6,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x08,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x08,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x01,0x81,0x80,0x60,0x00,0x7f,0x81,0x20,
0x90,0x10,0x1c,0x0a,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x03,0x80,
0x81,0xe0,0x60,0x10,0x24,0x12,0x0e,0x04,0x80,0x01,0xc0,0x70,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x01,0x80,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x02,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x01,0x80,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x78,0x00,0x00,0x1f,0x00,0x00,0x00,0x00,0x00,0x00,0x60,0x01,0x80,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x18,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x7f,0xfe,0xdf,
0x6f,0xef,0xe3,0xf5,0xff,0xff,0xff,0xff,0xff,0xff,0xff,0xff,0xff,0xff,0xfc,0x7f,
0x7e,0x1f,0x9f,0xef,0xdb,0xed,0xf1,0xfb,0x7f,0xfe,0x3f,0x8f,0xff,0xff,0xff,0xff,
0x00,0x0c,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x03,0x00,0x00,0x00,0x30,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x7c,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x30,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x30,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x01,0x81,0x80,0x60,0x20,0x20,0x20,0x20,
0x20,0x20,0x20,0x20,0x32,0x35,0x36,0x20,0x20,0x20,0x20,0x20,0x20,0x20,0x20,0x20,
0x20,0x31,0x35,0x20,0x20,0x20,0x20,0x20,0x20,0x20,0x20,0x20,0x20,0x31,0x33,0x20,
0x00,0x00,0x01,0x0c,0x00,0x09,0x09,0x00,0x01,0x0f,0x00,0x09,0x12,0x00,0x01,0x0f,
0x00,0x09,0x1b,0x00,0x01,0x0f,0x00,0x09,0x24,0x00,0x01,0x0f,0x00,0x09,0x2d,0x00,
```

```
0x01,0x0f,0x00,0x09,0x36,0x00,0x01,0x0f,0x00,0x09,0x3f,0x00,0x03,0x0d,0x00,0x09,
0x48,0x00,0x03,0x0d,0x00,0x09,0x51,0x00,0x03,0x0d,0x00,0x09,0x5a,0x00,0x03,0x0d,
0x00,0x09,0x63,0x00,0x03,0x0d,0x00,0x09,0x6c,0x00,0x03,0x0d,0x00,0x09,0x75,0x00,
0x03,0x0e,0x00,0x09,0x7e,0x00,0x03,0x0d,0x00,0x09,0x87,0x00,0x03,0x0d,0x00,0x09,
0x90,0x00,0x01,0x0f,0x00,0x09,0x99,0x00,0x01,0x0f,0x00,0x09,0xa2,0x00,0x01,0x0f,
0x00,0x09,0xab,0x00,0x01,0x0f,0x00,0x09,0xb4,0x00,0x01,0x0f,0x00,0x09,0xbd,0x00,
0x01,0x0f,0x00,0x09,0xc6,0x00,0x01,0x0f,0x00,0x09,0xcf,0x00,0x01,0x0f,0x00,0x09,
0xd8,0x00,0x01,0x0f,0x00,0x09,0xe1,0x00,0x03,0x0d,0x00,0x09,0xea,0x00,0x01,0x0f,
0x00,0x09,0xf3,0x00,0x01,0x0f,0x00,0x09,0xfc,0x00,0x03,0x0d,0x00,0x09,0x05,0x01,
0x03,0x0d,0x00,0x09,0x0e,0x01,0x03,0x0d,0x00,0x09,0x17,0x01,0x03,0x0d,0x00,0x09,
0x20,0x01,0x00,0x00,0x00,0x09,0x29,0x01,0x03,0x0d,0x00,0x09,0x32,0x01,0x02,0x05,
0x00,0x09,0x3b,0x01,0x03,0x0d,0x00,0x09,0x44,0x01,0x02,0x0e,0x00,0x09,0x4d,0x01,
0x03,0x0d,0x00,0x09,0x56,0x01,0x03,0x0d,0x00,0x09,0x5f,0x01,0x02,0x06,0x00,0x09,
0x68,0x01,0x02,0x0e,0x00,0x09,0x71,0x01,0x02,0x0e,0x00,0x09,0x7a,0x01,0x03,0x08,
0x00,0x09,0x83,0x01,0x05,0x0c,0x00,0x09,0x8c,0x01,0x0b,0x0f,0x00,0x09,0x95,0x01,
0x08,0x09,0x00,0x09,0x9e,0x01,0x0b,0x0d,0x00,0x09,0xa7,0x01,0x02,0x0e,0x00,0x09,
0xb0,0x01,0x03,0x0d,0x00,0x09,0xb9,0x01,0x03,0x0d,0x00,0x09,0xc2,0x01,0x03,0x0d,
0x00,0x09,0xcb,0x01,0x03,0x0d,0x00,0x09,0xd4,0x01,0x03,0x0d,0x00,0x09,0xdd,0x01,
0x03,0x0d,0x00,0x09,0xe6,0x01,0x03,0x0d,0x00,0x09,0xef,0x01,0x03,0x0d,0x00,0x09,
0xf8,0x01,0x03,0x0d,0x00,0x09,0x01,0x02,0x03,0x0d,0x00,0x09,0x0a,0x02,0x06,0x0d,
0x00,0x09,0x13,0x02,0x06,0x0f,0x00,0x09,0x1c,0x02,0x05,0x0c,0x00,0x09,0x25,0x02,
0x07,0x0a,0x00,0x09,0x2e,0x02,0x05,0x0c,0x00,0x09,0x37,0x02,0x03,0x0d,0x00,0x09,
0x40,0x02,0x03,0x0d,0x00,0x09,0x49,0x02,0x03,0x0d,0x00,0x09,0x52,0x02,0x03,0x0d,
0x00,0x09,0x5b,0x02,0x03,0x0d,0x00,0x09,0x64,0x02,0x03,0x0d,0x00,0x09,0x6d,0x02,
0x03,0x0d,0x00,0x09,0x76,0x02,0x03,0x0d,0x00,0x09,0x7f,0x02,0x03,0x0d,0x00,0x09,
0x88,0x02,0x03,0x0d,0x00,0x09,0x91,0x02,0x03,0x0d,0x00,0x09,0x9a,0x02,0x03,0x0d,
0x00,0x09,0xa3,0x02,0x03,0x0d,0x00,0x09,0xac,0x02,0x03,0x0d,0x00,0x09,0xb5,0x02,
0x03,0x0d,0x00,0x09,0xbe,0x02,0x03,0x0d,0x00,0x09,0xc7,0x02,0x03,0x0d,0x00,0x09,
0xd0,0x02,0x03,0x0d,0x00,0x09,0xd9,0x02,0x03,0x0f,0x00,0x09,0xe2,0x02,0x03,0x0d,
0x00,0x09,0xeb,0x02,0x03,0x0d,0x00,0x09,0xf4,0x02,0x03,0x0d,0x00,0x09,0xfd,0x02,
0x03,0x0d,0x00,0x09,0x06,0x03,0x03,0x0d,0x00,0x09,0x0f,0x03,0x03,0x0d,0x00,0x09,
0x18,0x03,0x03,0x0d,0x00,0x09,0x21,0x03,0x03,0x0d,0x00,0x09,0x2a,0x03,0x03,0x0d,
0x00,0x09,0x33,0x03,0x02,0x0e,0x00,0x09,0x3c,0x03,0x02,0x0e,0x00,0x09,0x45,0x03,
0x02,0x0e,0x00,0x09,0x4e,0x03,0x04,0x0b,0x00,0x09,0x57,0x03,0x0d,0x0e,0x00,0x09,
0x60,0x03,0x02,0x06,0x00,0x09,0x69,0x03,0x05,0x0d,0x00,0x09,0x72,0x03,0x02,0x0d,
0x00,0x09,0x7b,0x03,0x05,0x0d,0x00,0x09,0x84,0x03,0x02,0x0d,0x00,0x09,0x8d,0x03,
0x05,0x0d,0x00,0x09,0x96,0x03,0x02,0x0d,0x00,0x09,0x9f,0x03,0x05,0x0f,0x00,0x09,
0xa8,0x03,0x02,0x0d,0x00,0x09,0xb1,0x03,0x02,0x0d,0x00,0x09,0xba,0x03,0x02,0x0f,
0x00,0x09,0xc3,0x03,0x02,0x0d,0x00,0x09,0xcc,0x03,0x02,0x0d,0x00,0x09,0xd5,0x03,
0x05,0x0d,0x00,0x09,0xde,0x03,0x05,0x0d,0x00,0x09,0xe7,0x03,0x05,0x0d,0x00,0x09,
0xf0,0x03,0x05,0x0f,0x00,0x09,0xf9,0x03,0x05,0x0f,0x00,0x09,0x02,0x04,0x05,0x0d,
0x00,0x09,0x0b,0x04,0x05,0x0d,0x00,0x09,0x14,0x04,0x03,0x0d,0x00,0x09,0x1d,0x04,
0x05,0x0d,0x00,0x09,0x26,0x04,0x05,0x0d,0x00,0x09,0x2f,0x04,0x05,0x0d,0x00,0x09,
0x38,0x04,0x05,0x0d,0x00,0x09,0x41,0x04,0x05,0x0f,0x00,0x09,0x4a,0x04,0x05,0x0d,
0x00,0x09,0x53,0x04,0x02,0x0e,0x00,0x09,0x5c,0x04,0x02,0x0e,0x00,0x09,0x65,0x04,
0x02,0x0e,0x00,0x09,0x6e,0x04,0x07,0x0a,0x00,0x09,0x77,0x04,0x01,0x0d,0x00,0x09,
0x80,0x04,0x00,0x0e,0x00,0x09,0x89,0x04,0x00,0x0f,0x00,0x09,0x92,0x04,0x00,0x0f,
0x00,0x09,0x9b,0x04,0x00,0x0f,0x00,0x09,0xa4,0x04,0x00,0x0f,0x00,0x09,0xad,0x04,
0x00,0x0f,0x00,0x09,0xb6,0x04,0x00,0x0f,0x00,0x09,0xbf,0x04,0x00,0x0f,0x00,0x09,
0xc8,0x04,0x00,0x0f,0x00,0x09,0xd1,0x04,0x00,0x0f,0x00,0x09,0xda,0x04,0x00,0x0f,
0x00,0x09,0xe3,0x04,0x00,0x0f,0x00,0x09,0xec,0x04,0x00,0x0f,0x00,0x09,0xf5,0x04,
0x00,0x0f,0x00,0x09,0xfe,0x04,0x00,0x0f,0x00,0x09,0x07,0x05,0x00,0x0f,0x00,0x09,
0x10,0x05,0x00,0x0f,0x00,0x09,0x19,0x05,0x00,0x0f,0x00,0x09,0x22,0x05,0x00,0x0f,
0x00,0x09,0x2b,0x05,0x00,0x0f,0x00,0x09,0x34,0x05,0x00,0x0f,0x00,0x09,0x3d,0x05,
0x00,0x0f,0x00,0x09,0x46,0x05,0x00,0x0f,0x00,0x09,0x4f,0x05,0x00,0x0f,0x00,0x09,
0x58,0x05,0x00,0x0f,0x00,0x09,0x61,0x05,0x00,0x0f,0x00,0x09,0x6a,0x05,0x00,0x0f,
0x00,0x09,0x73,0x05,0x00,0x0f,0x00,0x09,0x7c,0x05,0x00,0x0f,0x00,0x09,0x85,0x05,
0x00,0x0f,0x00,0x09,0x8e,0x05,0x00,0x0f,0x00,0x09,0x97,0x05,0x00,0x0f,0x00,0x09,
0xa0,0x05,0x00,0x0d,0x00,0x09,0xa9,0x05,0x05,0x0f,0x00,0x09,0xb2,0x05,0x02,0x0e,
0x00,0x09,0xbb,0x05,0x03,0x0d,0x00,0x09,0xc4,0x05,0x03,0x0d,0x00,0x09,0xcd,0x05,
```

```
  0x03,0x0d,0x00,0x09,0xd6,0x05,0x02,0x0e,0x00,0x09,0xdf,0x05,0x03,0x0e,0x00,0x09,
  0xe8,0x05,0x02,0x04,0x00,0x09,0xf1,0x05,0x03,0x0d,0x00,0x09,0xfa,0x05,0x03,0x0a,
  0x00,0x09,0x03,0x06,0x06,0x0b,0x00,0x09,0x0c,0x06,0x07,0x0a,0x00,0x09,0x15,0x06,
  0x08,0x09,0x00,0x09,0x1e,0x06,0x03,0x0b,0x00,0x09,0x27,0x06,0x02,0x03,0x00,0x09,
  0x30,0x06,0x03,0x07,0x00,0x09,0x39,0x06,0x05,0x0c,0x00,0x09,0x42,0x06,0x03,0x0a,
  0x00,0x09,0x4b,0x06,0x03,0x0a,0x00,0x09,0x54,0x06,0x02,0x04,0x00,0x09,0x5d,0x06,
  0x05,0x0f,0x00,0x09,0x66,0x06,0x03,0x0e,0x00,0x09,0x6f,0x06,0x08,0x0a,0x00,0x09,
  0x78,0x06,0x0d,0x0f,0x00,0x09,0x81,0x06,0x03,0x0a,0x00,0x09,0x8a,0x06,0x03,0x0a,
  0x00,0x09,0x93,0x06,0x06,0x0b,0x00,0x09,0x9c,0x06,0x03,0x0d,0x00,0x09,0xa5,0x06,
  0x03,0x0d,0x00,0x09,0xae,0x06,0x03,0x0d,0x00,0x09,0xb7,0x06,0x05,0x0f,0x00,0x09,
  0xc0,0x06,0x00,0x0d,0x00,0x09,0xc9,0x06,0x00,0x0d,0x00,0x09,0xd2,0x06,0x00,0x0d,
  0x00,0x09,0xdb,0x06,0x00,0x0d,0x00,0x09,0xe4,0x06,0x00,0x0d,0x00,0x09,0xed,0x06,
  0x01,0x0d,0x00,0x09,0xf6,0x06,0x03,0x0d,0x00,0x09,0xff,0x06,0x03,0x0f,0x00,0x09,
  0x08,0x07,0x00,0x0d,0x00,0x09,0x11,0x07,0x00,0x0d,0x00,0x09,0x1a,0x07,0x00,0x0d,
  0x00,0x09,0x23,0x07,0x00,0x0d,0x00,0x09,0x2c,0x07,0x00,0x0d,0x00,0x09,0x35,0x07,
  0x00,0x0d,0x00,0x09,0x3e,0x07,0x00,0x0d,0x00,0x09,0x47,0x07,0x00,0x0d,0x00,0x09,
  0x50,0x07,0x03,0x0d,0x00,0x09,0x59,0x07,0x00,0x0d,0x00,0x09,0x62,0x07,0x00,0x0d,
  0x00,0x09,0x6b,0x07,0x00,0x0d,0x00,0x09,0x74,0x07,0x00,0x0d,0x00,0x09,0x7d,0x07,
  0x00,0x0d,0x00,0x09,0x86,0x07,0x00,0x0d,0x00,0x09,0x8f,0x07,0x06,0x0b,0x00,0x09,
  0x98,0x07,0x03,0x0d,0x00,0x09,0xa1,0x07,0x00,0x0d,0x00,0x09,0xaa,0x07,0x00,0x0d,
  0x00,0x09,0xb3,0x07,0x00,0x0d,0x00,0x09,0xbc,0x07,0x00,0x0d,0x00,0x09,0xc5,0x07,
  0x00,0x0d,0x00,0x09,0xce,0x07,0x03,0x0d,0x00,0x09,0xd7,0x07,0x02,0x0d,0x00,0x09,
  0xe0,0x07,0x02,0x0d,0x00,0x09,0xe9,0x07,0x02,0x0d,0x00,0x09,0xf2,0x07,0x02,0x0d,
  0x00,0x09,0xfb,0x07,0x02,0x0d,0x00,0x09,0x04,0x08,0x02,0x0d,0x00,0x09,0x0d,0x08,
  0x02,0x0d,0x00,0x09,0x16,0x08,0x05,0x0d,0x00,0x09,0x1f,0x08,0x05,0x0f,0x00,0x09,
  0x28,0x08,0x02,0x0d,0x00,0x09,0x31,0x08,0x02,0x0d,0x00,0x09,0x3a,0x08,0x02,0x0d,
  0x00,0x09,0x43,0x08,0x02,0x0d,0x00,0x09,0x4c,0x08,0x02,0x0d,0x00,0x09,0x55,0x08,
  0x02,0x0d,0x00,0x09,0x5e,0x08,0x02,0x0d,0x00,0x09,0x67,0x08,0x02,0x0d,0x00,0x09,
  0x70,0x08,0x02,0x0d,0x00,0x09,0x79,0x08,0x02,0x0d,0x00,0x09,0x82,0x08,0x02,0x0d,
  0x00,0x09,0x8b,0x08,0x02,0x0d,0x00,0x09,0x94,0x08,0x02,0x0d,0x00,0x09,0x9d,0x08,
  0x02,0x0d,0x00,0x09,0xa6,0x08,0x02,0x0d,0x00,0x09,0xaf,0x08,0x05,0x0c,0x00,0x09,
  0xb8,0x08,0x05,0x0d,0x00,0x09,0xc1,0x08,0x02,0x0d,0x00,0x09,0xca,0x08,0x02,0x0d,
  0x00,0x09,0xd3,0x08,0x02,0x0d,0x00,0x09,0xdc,0x08,0x02,0x0d,0x00,0x09,0xe5,0x08,
  0x02,0x0f,0x00,0x09,0xee,0x08,0x03,0x0f,0x00,0x09,0xf7,0x08,0x02,0x0f,0x00,0x09,
  0x00,0x09,0x00,0x00,0x00,0x00,
  };
```

⟨lib_graphics/libdraw/defont.c 409a⟩≡

```
  #include <u.h>
  #include <libc.h>
  #include <draw.h>
  #include <font.h>

  // This file is one of the very few files linked by the kernel, because
  // of defontdata. Do not add functions in this file with complex dependencies,
  // otherwise they will be linked also with the kernel.
```

⟨*global* defontdata 403b⟩

⟨*global* sizeofdefont 140b⟩

⟨*function* _unpackinfo 140d⟩

G.2.17 lib_graphics/libdraw/draw.c

⟨lib_graphics/libdraw/draw.c 409b⟩≡

```
  #include <u.h>
  #include <libc.h>
```

```
#include <draw.h>
#include <draw_private.h>

⟨function _setdrawop 90d⟩

⟨function draw1 90c⟩

⟨function draw 89a⟩

⟨function drawop 89b⟩

⟨function gendraw 89c⟩

⟨function gendrawop 89d⟩

⟨function border 89e⟩
```

G.2.18 lib_graphics/libdraw/line.c

```
⟨lib_graphics/libdraw/line.c 410a⟩≡
  #include <u.h>
  #include <libc.h>
  #include <draw.h>
  #include <draw_private.h>

  ⟨function line 113a⟩

  ⟨function lineop 113c⟩
```

G.2.19 lib_graphics/libdraw/poly.c

```
⟨lib_graphics/libdraw/poly.c 410b⟩≡
  #include <u.h>
  #include <libc.h>
  #include <draw.h>
  #include <draw_private.h>

  ⟨function addcoord 119d⟩

  ⟨function dopoly 118e⟩

  ⟨function poly 118a⟩

  ⟨function polyop 118b⟩

  ⟨function fillpoly 118c⟩

  ⟨function fillpolyop 118d⟩
```

G.2.20 lib_graphics/libdraw/ellipse.c

```
⟨lib_graphics/libdraw/ellipse.c 410c⟩≡
  #include <u.h>
  #include <libc.h>
  #include <draw.h>
  #include <draw_private.h>
```

⟨*function* doellipse 129a⟩

⟨*function* ellipse 128b⟩

⟨*function* ellipseop 128c⟩

⟨*function* fillellipse 128d⟩

⟨*function* fillellipseop 128e⟩

⟨*function* arc 134a⟩

⟨*function* arcop 134b⟩

⟨*function* fillarc 134c⟩

⟨*function* fillarcop 134d⟩

G.2.21 lib_graphics/libdraw/bezier.c

⟨lib_graphics/libdraw/bezier.c 411⟩≡

```
#include <u.h>
#include <libc.h>
#include <draw.h>

⟨constant PINC 261b⟩

typedef struct Plist Plist;
⟨struct Plist 261c⟩

⟨function appendpt 261d⟩

⟨function normsq 264c⟩

⟨function psdist 264d⟩

⟨function bpts1 264b⟩

⟨function bpts 264a⟩

⟨function bezierpts 263d⟩

⟨function _bezsplinepts 265⟩

⟨function bezsplinepts 263c⟩

⟨function bezier 261e⟩

⟨function bezierop 262a⟩

⟨function bezspline 262b⟩

⟨function bezsplineop 262c⟩

⟨function fillbezier 262d⟩

⟨function fillbezierop 262e⟩

⟨function fillbezspline 263a⟩
```

⟨*function* fillbezsplineop 263b⟩

G.2.22 lib_graphics/libdraw/fontcache.c

⟨lib_graphics/libdraw/fontcache.c 412a⟩≡

```
#include <u.h>
#include <libc.h>
#include <draw.h>
#include <draw_private.h>
#include <font.h>

static int fontresize(Font*, int, int, int);

⟨constant PJW 159a⟩

⟨function cachechars 156c⟩

⟨function agefont 164b⟩

⟨function cf2subfont 166d⟩

⟨function loadchar 158d⟩

⟨function fontresize 162a⟩
```

G.2.23 lib_graphics/libdraw/font.c

⟨lib_graphics/libdraw/font.c 412b⟩≡

```
#include <u.h>
#include <libc.h>
#include <draw.h>
#include <draw_private.h>
#include <font.h>

⟨function skip 145b⟩


⟨function openfont 142c⟩

⟨function buildfont 144⟩

⟨function freefont 146e⟩
```

G.2.24 lib_graphics/libdraw/colormap.c

⟨lib_graphics/libdraw/colormap.c 412c⟩≡

```
#include <u.h>
#include <libc.h>
#include <draw.h>
#include <bio.h>

⟨function getval 251b⟩

⟨function readcolmap 251a⟩

⟨function writecolmap 251c⟩
```

⟨*function* rgb2cmap 248h⟩

⟨*function* cmap2rgb 249a⟩

⟨*function* cmap2rgba 249b⟩

G.2.25 lib_graphics/libdraw/replclipr.c

⟨lib_graphics/libdraw/replclipr.c 413a⟩≡

```
#include <u.h>
#include <libc.h>
#include <draw.h>
#include <draw_private.h>
```

⟨*function* replclipr 79d⟩

⟨*function* rectclip 79c⟩

⟨*function* drawreplxy 97e⟩

⟨*function* drawrepl 98b⟩

G.2.26 lib_graphics/libdraw/string.c

⟨*enum* _anon_(lib_graphics/libdraw/string.c) 413b⟩≡ (413c)

```
enum
{
⟨constant Max 149c⟩
};
```

⟨lib_graphics/libdraw/string.c 413c⟩≡

```
#include <u.h>
#include <libc.h>
#include <draw.h>
#include <draw_private.h>
#include <font.h>
```

⟨*enum* _anon_(lib_graphics/libdraw/string.c) 413b⟩

```
Point
_string(Image *dst, Point pt, Image *src, Point sp, Font *f, char *s, Rune *r, int len, Rectangle clipr, Image
```

⟨*function* string 147c⟩

⟨*function* stringop 147d⟩

⟨*function* stringn 147e⟩

⟨*function* stringnop 147f⟩

⟨*function* runestring 147g⟩

⟨*function* runestringop 148a⟩

⟨*function* runestringn 148b⟩

⟨*function* runestringnop 148c⟩

⟨*function* _string 149d⟩

⟨*function* stringbg 148d⟩

⟨*function* stringbgop 148e⟩

⟨*function* stringnbg 148f⟩

⟨*function* stringnbgop 148g⟩

⟨*function* runestringbg 148h⟩

⟨*function* runestringbgop 148i⟩

⟨*function* runestringnbg 149a⟩

⟨*function* runestringnbgop 149b⟩

G.2.27 lib_graphics/libdraw/stringwidth.c

⟨lib_graphics/libdraw/stringwidth.c 414a⟩≡

```
#include <u.h>
#include <libc.h>
#include <draw.h>
#include <draw_private.h>
#include <font.h>
```

⟨*enum* _anon_*(*(lib_graphics/libdraw/stringwidth.c)*)* 153g⟩

⟨*function* _stringnwidth 154a⟩

⟨*function* stringnwidth 153b⟩

⟨*function* stringwidth 153a⟩

⟨*function* stringsize 153c⟩

⟨*function* runestringnwidth 153d⟩

⟨*function* runestringwidth 153e⟩

⟨*function* runestringsize 153f⟩

G.2.28 lib_graphics/libdraw/subfont.c

⟨lib_graphics/libdraw/subfont.c 414b⟩≡

```
#include <u.h>
#include <libc.h>
#include <draw.h>
#include <draw_private.h>
#include <font.h>

// Do not put this function in defont.c! Otherwise it will create a chain
// of dependencies that will force the kernel to link in lockdisplay().
```

```
// This in turn will generate a signature conflict because of the differences
// between QLock (used in Display) in libc.h and in the kernel (in lib.h).
⟨function getdefont 140c⟩


⟨function allocsubfont 141f⟩

⟨function freesubfont 147a⟩

⟨function _getsubfont 169a⟩

⟨function readsubfont 170e⟩

⟨function packinfo 172b⟩

⟨function writesubfont 171g⟩


⟨function subfontname 167c⟩

/*
 * Easy versions of the cache routines; may be substituted by fancier ones for other purposes
 */

⟨global lastname 169g⟩
⟨global lastsubfont 170a⟩

⟨function lookupsubfont 170b⟩

⟨function installsubfont 170c⟩

⟨function uninstallsubfont 170d⟩
```

G.2.29 lib_graphics/libdraw/desktop.c

```
⟨lib_graphics/libdraw/desktop.c 415a⟩≡
  #include <u.h>
  #include <libc.h>
  #include <draw.h>
  #include <window.h>
  #include <draw_private.h>

  ⟨global screenid 201e⟩


  ⟨function allocscreen 200c⟩

  ⟨function publicscreen 201c⟩

  ⟨function freescreen 201a⟩
```

G.2.30 lib_graphics/libdraw/window.c

```
⟨lib_graphics/libdraw/window.c 415b⟩≡
  #include <u.h>
  #include <libc.h>
```

```
#include <draw.h>
#include <window.h>
#include <draw_private.h>

// see getwindow()
⟨global screen 232b⟩

⟨function _allocwindow 206c⟩

⟨function allocwindow 206b⟩



⟨function gengetwindow 232e⟩

⟨function getwindow 232d⟩



⟨function topbottom 211f⟩

⟨function bottomwindow 211b⟩

⟨function topwindow 211a⟩

⟨function bottomnwindows 211d⟩

⟨function topnwindows 211c⟩

⟨function originwindow 217c⟩


⟨function newwindow 236a⟩
```

G.2.31 lib_graphics/libdraw/transfer.c

```
⟨lib_graphics/libdraw/transfer.c 416a⟩≡
  #include <u.h>
  #include <libc.h>
  #include <draw.h>
  #include <draw_private.h>

  ⟨function loadimage 173a⟩

  ⟨function unloadimage 176c⟩
```

G.2.32 lib_graphics/libdraw/io.c

```
⟨lib_graphics/libdraw/io.c 416b⟩≡
  #include <u.h>
  #include <libc.h>
  #include <draw.h>
  #include <draw_private.h>

  ⟨function readimage 180e⟩

  // writeimage() is in compressed.c. It does not support the
```

```
// regular (uncompressed) format.
```

G.2.33 lib_graphics/libdraw/compressed.c

⟨lib_graphics/libdraw/compressed.c 417a⟩≡

```
#include <u.h>
#include <libc.h>
#include <draw.h>
#include <draw_private.h>

⟨function _twiddlecompressed 332b⟩

⟨function _compblocksize 332a⟩

⟨function cloadimage 328d⟩

⟨function creadimage 330b⟩

⟨constant HSHIFT 333a⟩
⟨constant NHASH 333b⟩
⟨constant HMASK 333c⟩
⟨function hupdate 333d⟩

typedef struct Hlist Hlist;
⟨struct Hlist 333e⟩

⟨function writeimage 333i⟩
```

G.2.34 lib_graphics/libdraw/keyboard.c

⟨lib_graphics/libdraw/keyboard.c 417b⟩≡

```
#include <u.h>
#include <libc.h>
#include <draw.h>
#include <thread.h>
#include <keyboard.h>

⟨function _ioproc 195f⟩

⟨function ctlkeyboard 195b⟩

⟨function initkeyboard 194b⟩


⟨function closekeyboard 196b⟩
```

G.2.35 lib_graphics/libdraw/mouse.c

⟨lib_graphics/libdraw/mouse.c 417c⟩≡

```
#include <u.h>
#include <libc.h>
#include <draw.h>
#include <draw_private.h>
#include <thread.h>
```

```
#include <cursor.h>
#include <mouse.h>
```

⟨*function* moveto 186d⟩

⟨*function* readmouse 186b⟩

⟨*function* _ioproc*(*(lib_graphics/libdraw/mouse.c)*)* 185a⟩

⟨*function* initmouse 184c⟩

⟨*function* setcursor 272b⟩

⟨*function* closemouse 185c⟩

⟨*function* mousescrollsize 272c⟩

G.2.36 lib_graphics/libdraw/menuhit.c

⟨lib_graphics/libdraw/menuhit.c 418a⟩≡

```
#include <u.h>
#include <libc.h>
#include <draw.h>
#include <draw_private.h>
#include <window.h>
#include <thread.h>
#include <mouse.h>
```

⟨*enum* _anon_*(*(lib_graphics/libdraw/menuhit.c)*)* 273a⟩

⟨*global* menutxt*(*(lib_graphics/libdraw/menuhit.c)*)* 273g⟩
⟨*global* back*(*(lib_graphics/libdraw/menuhit.c)*)* 273b⟩
⟨*global* high*(*(lib_graphics/libdraw/menuhit.c)*)* 273c⟩
⟨*global* bord*(*(lib_graphics/libdraw/menuhit.c)*)* 273d⟩
⟨*global* text*(*(lib_graphics/libdraw/menuhit.c)*)* 273e⟩
⟨*global* htext*(*(lib_graphics/libdraw/menuhit.c)*)* 273f⟩

⟨*function* menucolors*(*(lib_graphics/libdraw/menuhit.c)*)* 273i⟩

⟨*function* menurect*(*(lib_graphics/libdraw/menuhit.c)*)* 276d⟩

⟨*function* menusel*(*(lib_graphics/libdraw/menuhit.c)*)* 279a⟩

⟨*function* paintitem*(*(lib_graphics/libdraw/menuhit.c)*)* 277d⟩

⟨*function* menuscan*(*(lib_graphics/libdraw/menuhit.c)*)* 278f⟩

⟨*function* menupaint*(*(lib_graphics/libdraw/menuhit.c)*)* 277c⟩

⟨*function* menuscrollpaint*(*(lib_graphics/libdraw/menuhit.c)*)* 280f⟩

⟨*function* menuhit 274f⟩

G.2.37 lib_graphics/libdraw/getrect.c

⟨lib_graphics/libdraw/getrect.c 418b⟩≡

```
#include <u.h>
#include <libc.h>
#include <draw.h>
#include <thread.h>
#include <cursor.h>
#include <mouse.h>

⟨constant W((lib_graphics/libdraw/getrect.c)) 281b⟩

⟨global tmp((lib_graphics/libdraw/getrect.c)) 281c⟩
⟨global red((lib_graphics/libdraw/getrect.c)) 281d⟩

⟨global sweep((lib_graphics/libdraw/getrect.c)) 281e⟩

⟨function brects((lib_graphics/libdraw/getrect.c)) 281f⟩

⟨function getrect 282a⟩

⟨function freetmp((lib_graphics/libdraw/getrect.c)) 283a⟩

⟨function max 283b⟩

⟨function drawgetrect 282b⟩
```

G.2.38 lib_graphics/libdraw/event.c

```
⟨lib_graphics/libdraw/event.c 419⟩≡
  #include <u.h>
  #include <libc.h>
  #include <draw.h>
  #include <draw_private.h>
  #include <cursor.h>
  #include <event.h>

  typedef struct Slave Slave;
  typedef struct Ebuf Ebuf;

  ⟨struct Slave 284e⟩

  ⟨struct Ebuf 284f⟩

  ⟨global eslave 284g⟩
  ⟨global Skeyboard 285b⟩
  ⟨global Smouse 285c⟩
  ⟨global Stimer 285d⟩
  ⟨global logfid 285f⟩

  ⟨global nslave 285a⟩
  ⟨global parentpid 285e⟩
  ⟨global epipe 285g⟩

  static int eforkslave(ulong);
  static void extract(void);
  static void ekill(void);
  static int enote(void *, char *);

  ⟨global mousefd 285h⟩
  ⟨global cursorfd 285i⟩
```

⟨*function* ebread 290a⟩

⟨*function* event 288c⟩

⟨*function* eread 288d⟩

⟨*function* ecanmouse 291⟩

⟨*function* ecankbd 292a⟩

⟨*function* ecanread 292b⟩

⟨*function* estartfn 288b⟩

⟨*function* estart 288a⟩

⟨*function* etimer 292c⟩

⟨*function* ekeyslave 286c⟩

⟨*function* einit 285j⟩

⟨*function* extract 290b⟩

⟨*function* eforkslave 287⟩

⟨*function* enote 286b⟩

⟨*function* ekill 286a⟩

⟨*function* emouse 289a⟩

⟨*function* ekbd 289b⟩

⟨*function* emoveto 293a⟩

⟨*function* esetcursor 293b⟩

⟨*function* ereadmouse 293c⟩

⟨*function* eatomouse 293d⟩

G.2.39 lib_graphics/libdraw/egetrect.c

⟨lib_graphics/libdraw/egetrect.c 420⟩≡

```
#include <u.h>
#include <libc.h>
#include <draw.h>
#include <cursor.h>
#include <event.h>

⟨constant W 294a⟩

⟨global tmp 294b⟩
⟨global red 294c⟩

⟨global sweep 294d⟩

⟨function brects 294e⟩
```

⟨*function* `egetrect` 294f⟩

⟨*function* `freetmp` 295a⟩

⟨*function* `edrawgetrect` 295b⟩

G.2.40 lib_graphics/libdraw/emenuhit.c

⟨`lib_graphics/libdraw/emenuhit.c` 421a⟩≡

```
#include <u.h>
#include <libc.h>
#include <draw.h>
#include <draw_private.h>
#include <event.h>

⟨enum _anon_(lib_graphics/libdraw/emenuhit.c) 296a⟩

⟨global menutxt 296b⟩
⟨global back 296c⟩
⟨global high 296d⟩
⟨global bord 296e⟩
⟨global text 296f⟩
⟨global htext 296g⟩

⟨function menucolors 296h⟩

⟨function menurect 297a⟩

⟨function menusel 297b⟩

⟨function paintitem 297c⟩

⟨function menuscan 298a⟩

⟨function menupaint 298b⟩

⟨function menuscrollpaint 298c⟩

⟨function emenuhit 299⟩
```

G.3 lib_graphics/libdraw/tests/

G.3.1 lib_graphics/libdraw/tests/drawtest.c

⟨*function* `main` 421b⟩≡ (421c)

```
void
main(int argc, char **argv)
{
        print("%dn", wordsperline(Rect(atoi(argv[1]), atoi(argv[2]), atoi(argv[3]), atoi(argv[4])), atoi(argv[5
}
```

⟨`lib_graphics/libdraw/tests/drawtest.c` 421c⟩≡

```
#include <u.h>
#include <libc.h>
#include <draw.h>

⟨function main 421b⟩
```

G.4 lib_graphics/libmemdraw/

G.4.1 lib_graphics/libmemdraw/resolution.c

⟨lib_graphics/libmemdraw/resolution.c 422a⟩≡

```
#include <u.h>
#include <libc.h>
#include <draw.h>
#include <memdraw.h>

⟨function byteaddr 39b⟩

⟨function wordaddr 44a⟩

⟨function pixelbits 99c⟩
```

G.4.2 lib_graphics/libmemdraw/chan.c

⟨lib_graphics/libmemdraw/chan.c 422b⟩≡

```
#include <u.h>
#include <libc.h>
#include <draw.h>
#include <memdraw.h>

⟨function memsetchan 53a⟩
```

G.4.3 lib_graphics/libmemdraw/alloc.c

⟨lib_graphics/libmemdraw/alloc.c 422c⟩≡

```
#include <u.h>
#include <libc.h>
#include <draw.h>
#include <memdraw.h>
#include <pool.h>

⟨function memimagemove 48e⟩

⟨function allocmemimaged 51e⟩


⟨function allocmemimage 76e⟩

⟨function freememimage 54a⟩
```

G.4.4 lib_graphics/libmemdraw/arc.c

⟨lib_graphics/libmemdraw/arc.c 422d⟩≡

```
#include <u.h>
#include <libc.h>
#include <draw.h>
#include <memdraw.h>
#include <memlayer.h>

⟨enum _anon_((lib_graphics/libmemdraw/arc.c)) 135a⟩
```

⟨*global* corners 135b⟩

⟨*global* p00 135c⟩

⟨*function* memarc 135d⟩

G.4.5 lib_graphics/libmemdraw/compressed.c

⟨lib_graphics/libmemdraw/compressed.c 423a⟩≡

```
#include <u.h>
#include <libc.h>
#include <draw.h>
#include <draw_private.h>
#include <memdraw.h>

// creadmemimage() could be in this file but then it creates a dependency
// to libdraw which ends up linking in lockdisplay in the kernel.
// To avoid this issue creadmemimage() is defined instead in io.c.
```

⟨*function* cloadmemimage 329b⟩

G.4.6 lib_graphics/libmemdraw/cmap.c

⟨*global* def 423b⟩≡ (428a)

```
static Memcmap def = {
/* cmap2rgb */ {
    0x00,0x00,0x00,0x00,0x00,0x44,0x00,0x00,0x88,0x00,0x00,0xcc,0x00,0x44,0x00,0x00,
    0x44,0x44,0x00,0x44,0x88,0x00,0x44,0xcc,0x00,0x88,0x00,0x00,0x88,0x44,0x00,0x88,
    0x88,0x00,0x88,0xcc,0x00,0xcc,0x00,0x00,0xcc,0x44,0x00,0xcc,0x88,0x00,0xcc,0xcc,
    0x00,0xdd,0xdd,0x11,0x11,0x11,0x00,0x00,0x55,0x00,0x00,0x99,0x00,0x00,0xdd,0x00,
    0x55,0x00,0x00,0x55,0x55,0x00,0x4c,0x99,0x00,0x49,0xdd,0x00,0x99,0x00,0x00,0x99,
    0x4c,0x00,0x99,0x99,0x00,0x93,0xdd,0x00,0xdd,0x00,0x00,0xdd,0x49,0x00,0xdd,0x93,
    0x00,0xee,0x9e,0x00,0xee,0xee,0x22,0x22,0x22,0x00,0x00,0x66,0x00,0x00,0xaa,0x00,
    0x00,0xee,0x00,0x66,0x00,0x00,0x66,0x66,0x00,0x55,0xaa,0x00,0x4f,0xee,0x00,0xaa,
    0x00,0x00,0xaa,0x55,0x00,0xaa,0xaa,0x00,0x9e,0xee,0x00,0xee,0x00,0x00,0xee,0x4f,
    0x00,0xff,0x55,0x00,0xff,0xaa,0x00,0xff,0xff,0x33,0x33,0x33,0x00,0x00,0x77,0x00,
    0x00,0xbb,0x00,0x00,0xff,0x00,0x77,0x00,0x00,0x77,0x77,0x00,0x5d,0xbb,0x00,0x55,
    0xff,0x00,0xbb,0x00,0x00,0xbb,0x5d,0x00,0xbb,0xbb,0x00,0xaa,0xff,0x00,0xff,0x00,
    0x44,0x00,0x44,0x44,0x00,0x88,0x44,0x00,0xcc,0x44,0x44,0x00,0x44,0x44,0x44,0x44,
    0x44,0x88,0x44,0x44,0xcc,0x44,0x88,0x00,0x44,0x88,0x44,0x44,0x88,0x88,0x44,0x88,
    0xcc,0x44,0xcc,0x00,0x44,0xcc,0x44,0x44,0xcc,0x88,0x44,0xcc,0xcc,0x44,0x00,0x00,
    0x55,0x00,0x00,0x55,0x00,0x55,0x4c,0x00,0x99,0x49,0x00,0xdd,0x55,0x55,0x00,0x55,
    0x55,0x55,0x4c,0x4c,0x99,0x49,0x49,0xdd,0x4c,0x99,0x00,0x4c,0x99,0x4c,0x4c,0x99,
    0x99,0x49,0x93,0xdd,0x49,0xdd,0x00,0x49,0xdd,0x49,0x49,0xdd,0x93,0x49,0xdd,0xdd,
    0x4f,0xee,0xee,0x66,0x00,0x00,0x66,0x00,0x66,0x55,0x00,0xaa,0x4f,0x00,0xee,0x66,
    0x66,0x00,0x66,0x66,0x66,0x55,0x55,0xaa,0x4f,0x4f,0xee,0x55,0xaa,0x00,0x55,0xaa,
    0x55,0x55,0xaa,0xaa,0x4f,0x9e,0xee,0x4f,0xee,0x00,0x4f,0xee,0x4f,0x4f,0xee,0x9e,
    0x55,0xff,0xaa,0x55,0xff,0xff,0x77,0x00,0x00,0x77,0x00,0x77,0x5d,0x00,0xbb,0x55,
    0x00,0xff,0x77,0x77,0x00,0x77,0x77,0x77,0x5d,0x5d,0xbb,0x55,0x55,0xff,0x5d,0xbb,
    0x00,0x5d,0xbb,0x5d,0x5d,0xbb,0xbb,0x55,0xaa,0xff,0x55,0xff,0x00,0x55,0xff,0x55,
    0x88,0x00,0x88,0x88,0x00,0xcc,0x88,0x44,0x00,0x88,0x44,0x44,0x88,0x44,0x88,0x88,
    0x44,0xcc,0x88,0x88,0x00,0x88,0x88,0x44,0x88,0x88,0x88,0x88,0x88,0xcc,0x88,0xcc,
    0x00,0x88,0xcc,0x44,0x88,0xcc,0x88,0x88,0xcc,0xcc,0x88,0x00,0x00,0x88,0x00,0x44,
    0x99,0x00,0x4c,0x99,0x00,0x99,0x93,0x00,0xdd,0x99,0x4c,0x00,0x99,0x4c,0x4c,0x99,
    0x4c,0x99,0x93,0x49,0xdd,0x99,0x99,0x00,0x99,0x99,0x4c,0x99,0x99,0x99,0x93,0x93,
    0xdd,0x93,0xdd,0x00,0x93,0xdd,0x49,0x93,0xdd,0x93,0x93,0xdd,0xdd,0x99,0x00,0x00,
```

```
    0xaa,0x00,0x00,0xaa,0x00,0x55,0xaa,0x00,0xaa,0x9e,0x00,0xee,0xaa,0x55,0x00,0xaa,
    0x55,0x55,0xaa,0x55,0xaa,0x9e,0x4f,0xee,0xaa,0xaa,0x00,0xaa,0xaa,0x55,0xaa,0xaa,
    0xaa,0x9e,0x9e,0xee,0x9e,0xee,0x00,0x9e,0xee,0x4f,0x9e,0xee,0x9e,0x9e,0xee,0xee,
    0xaa,0xff,0xff,0xbb,0x00,0x00,0xbb,0x00,0x5d,0xbb,0x00,0xbb,0xaa,0x00,0xff,0xbb,
    0x5d,0x00,0xbb,0x5d,0x5d,0xbb,0x5d,0xbb,0xaa,0x55,0xff,0xbb,0xbb,0x00,0xbb,0xbb,
    0x5d,0xbb,0xbb,0xbb,0xaa,0xaa,0xff,0xaa,0xff,0x00,0xaa,0xff,0x55,0xaa,0xff,0xaa,
    0xcc,0x00,0xcc,0xcc,0x44,0x00,0xcc,0x44,0x44,0xcc,0x44,0x88,0xcc,0x44,0xcc,0xcc,
    0x88,0x00,0xcc,0x88,0x44,0xcc,0x88,0x88,0xcc,0x88,0xcc,0xcc,0xcc,0x00,0xcc,0xcc,
    0x44,0xcc,0xcc,0x88,0xcc,0xcc,0xcc,0xcc,0x00,0x00,0xcc,0x00,0x44,0xcc,0x00,0x88,
    0xdd,0x00,0x93,0xdd,0x00,0xdd,0xdd,0x49,0x00,0xdd,0x49,0x49,0xdd,0x49,0x93,0xdd,
    0x49,0xdd,0xdd,0x93,0x00,0xdd,0x93,0x49,0xdd,0x93,0x93,0xdd,0x93,0xdd,0xdd,0xdd,
    0x00,0xdd,0xdd,0x49,0xdd,0xdd,0x93,0xdd,0xdd,0xdd,0xdd,0x00,0x00,0xdd,0x00,0x49,
    0xee,0x00,0x4f,0xee,0x00,0x9e,0xee,0x00,0xee,0xee,0x4f,0x00,0xee,0x4f,0x4f,0xee,
    0x4f,0x9e,0xee,0x4f,0xee,0xee,0x9e,0x00,0xee,0x9e,0x4f,0xee,0x9e,0x9e,0xee,0x9e,
    0xee,0xee,0xee,0x00,0xee,0xee,0x4f,0xee,0xee,0x9e,0xee,0xee,0xee,0xee,0x00,0x00,
    0xff,0x00,0x00,0xff,0x00,0x55,0xff,0x00,0xaa,0xff,0x00,0xff,0xff,0x55,0x00,0xff,
    0x55,0x55,0xff,0x55,0xaa,0xff,0x55,0xff,0xff,0xaa,0x00,0xff,0xaa,0x55,0xff,0xaa,
    0xaa,0xff,0xaa,0xff,0xff,0xff,0x00,0xff,0xff,0x55,0xff,0xff,0xaa,0xff,0xff,0xff,
},
/* rgb2cmap */ {
    0x00,0x00,0x11,0x01,0x01,0x12,0x23,0x34,0x02,0x13,0x24,0x35,0x03,0x14,0x25,0x36,
    0x00,0x11,0x11,0x01,0x01,0x12,0x23,0x34,0x02,0x13,0x24,0x35,0x03,0x14,0x25,0x36,
    0x11,0x11,0x11,0x01,0x01,0x12,0x23,0x34,0x02,0x13,0x24,0x35,0x03,0x14,0x25,0x36,
    0x04,0x04,0x04,0x05,0x05,0x05,0x05,0x06,0x06,0x06,0x17,0x07,0x07,0x18,0x18,0x29,
    0x04,0x04,0x04,0x05,0x05,0x05,0x16,0x06,0x06,0x17,0x28,0x07,0x07,0x18,0x29,0x3a,
    0x15,0x15,0x15,0x05,0x05,0x16,0x16,0x06,0x06,0x17,0x28,0x39,0x07,0x18,0x29,0x3a,
    0x26,0x26,0x26,0x05,0x16,0x16,0x27,0x27,0x38,0x28,0x28,0x39,0x39,0x29,0x29,0x3a,
    0x37,0x37,0x37,0x09,0x09,0x09,0x27,0x38,0x0a,0x0a,0x39,0x0b,0x0b,0x0b,0x1c,0x3a,
    0x08,0x08,0x08,0x09,0x09,0x09,0x38,0x0a,0x0a,0x0a,0x1b,0x0b,0x0b,0x1c,0x1c,0x2d,
    0x19,0x19,0x19,0x09,0x1a,0x1a,0x2b,0x0a,0x0a,0x1b,0x1b,0x0b,0x0b,0x1c,0x2d,0x3e,
    0x2a,0x2a,0x2a,0x1a,0x2b,0x2b,0x2b,0x3c,0x1b,0x1b,0x2c,0x2c,0x3d,0x2d,0x2d,0x3e,
    0x3b,0x3b,0x3b,0x0d,0x0d,0x3c,0x3c,0x0e,0x0e,0x0e,0x2c,0x3d,0x0f,0x0f,0x3e,0x3e,
    0x0c,0x0c,0x0c,0x0d,0x0d,0x0d,0x3c,0x0e,0x0e,0x0e,0x3d,0x0f,0x0f,0x0f,0x10,0x3e,
    0x1d,0x1d,0x1d,0x1e,0x1e,0x1e,0x2f,0x0e,0x1f,0x1f,0x20,0x0f,0x0f,0x10,0x10,0x21,
    0x2e,0x2e,0x2e,0x1e,0x2f,0x2f,0x2f,0x1f,0x1f,0x20,0x20,0x31,0x10,0x10,0x21,0x21,
    0x3f,0x3f,0x3f,0x2f,0x30,0x30,0x30,0x30,0x20,0x31,0x31,0x31,0x31,0x21,0x21,0x32,
    0x00,0x11,0x11,0x01,0x01,0x12,0x23,0x34,0x02,0x13,0x24,0x35,0x03,0x14,0x25,0x36,
    0x11,0x11,0x11,0x01,0x01,0x12,0x23,0x34,0x02,0x13,0x24,0x35,0x03,0x14,0x25,0x36,
    0x11,0x11,0x22,0x22,0x01,0x12,0x23,0x34,0x02,0x13,0x24,0x35,0x03,0x14,0x25,0x36,
    0x04,0x04,0x22,0x05,0x05,0x05,0x05,0x06,0x06,0x06,0x17,0x07,0x07,0x18,0x18,0x29,
    0x04,0x04,0x04,0x05,0x05,0x05,0x16,0x06,0x06,0x17,0x28,0x07,0x07,0x18,0x29,0x3a,
    0x15,0x15,0x15,0x05,0x05,0x16,0x16,0x06,0x06,0x17,0x28,0x39,0x07,0x18,0x29,0x3a,
    0x26,0x26,0x26,0x05,0x16,0x16,0x27,0x27,0x38,0x28,0x28,0x39,0x39,0x29,0x29,0x3a,
    0x37,0x37,0x37,0x09,0x09,0x09,0x27,0x38,0x0a,0x0a,0x39,0x0b,0x0b,0x0b,0x1c,0x3a,
    0x08,0x08,0x08,0x09,0x09,0x09,0x38,0x0a,0x0a,0x0a,0x1b,0x0b,0x0b,0x1c,0x1c,0x2d,
    0x19,0x19,0x19,0x09,0x1a,0x1a,0x2b,0x0a,0x0a,0x1b,0x1b,0x0b,0x0b,0x1c,0x2d,0x3e,
    0x2a,0x2a,0x2a,0x1a,0x2b,0x2b,0x2b,0x3c,0x1b,0x1b,0x2c,0x2c,0x3d,0x2d,0x2d,0x3e,
    0x3b,0x3b,0x3b,0x0d,0x0d,0x3c,0x3c,0x0e,0x0e,0x0e,0x2c,0x3d,0x0f,0x0f,0x3e,0x3e,
    0x0c,0x0c,0x0c,0x0d,0x0d,0x0d,0x3c,0x0e,0x0e,0x0e,0x3d,0x0f,0x0f,0x0f,0x10,0x3e,
    0x1d,0x1d,0x1d,0x1e,0x1e,0x1e,0x2f,0x0e,0x1f,0x1f,0x20,0x0f,0x0f,0x10,0x10,0x21,
    0x2e,0x2e,0x2e,0x1e,0x2f,0x2f,0x2f,0x1f,0x1f,0x20,0x20,0x31,0x10,0x10,0x21,0x21,
    0x3f,0x3f,0x3f,0x2f,0x30,0x30,0x30,0x30,0x20,0x31,0x31,0x31,0x31,0x21,0x21,0x32,
    0x11,0x11,0x11,0x01,0x01,0x12,0x23,0x34,0x02,0x13,0x24,0x35,0x03,0x14,0x25,0x36,
    0x11,0x11,0x22,0x22,0x01,0x12,0x23,0x34,0x02,0x13,0x24,0x35,0x03,0x14,0x25,0x36,
    0x11,0x22,0x22,0x22,0x33,0x33,0x23,0x34,0x02,0x13,0x24,0x35,0x03,0x14,0x25,0x36,
    0x04,0x22,0x22,0x33,0x33,0x33,0x05,0x06,0x06,0x06,0x17,0x07,0x07,0x18,0x18,0x29,
    0x04,0x04,0x33,0x33,0x33,0x05,0x16,0x06,0x06,0x17,0x28,0x07,0x07,0x18,0x29,0x3a,
    0x15,0x15,0x33,0x33,0x05,0x16,0x16,0x06,0x06,0x17,0x28,0x39,0x07,0x18,0x29,0x3a,
    0x26,0x26,0x26,0x05,0x16,0x16,0x27,0x27,0x38,0x28,0x28,0x39,0x39,0x29,0x29,0x3a,
    0x37,0x37,0x37,0x09,0x09,0x09,0x27,0x38,0x0a,0x0a,0x39,0x0b,0x0b,0x0b,0x1c,0x3a,
```

```
0x08,0x08,0x08,0x09,0x09,0x09,0x38,0x0a,0x0a,0x0a,0x1b,0x0b,0x0b,0x1c,0x1c,0x2d,
0x19,0x19,0x19,0x09,0x1a,0x1a,0x2b,0x0a,0x0a,0x1b,0x1b,0x0b,0x0b,0x1c,0x2d,0x3e,
0x2a,0x2a,0x2a,0x1a,0x2b,0x2b,0x2b,0x3c,0x1b,0x1b,0x2c,0x2c,0x3d,0x2d,0x2d,0x3e,
0x3b,0x3b,0x3b,0x0d,0x0d,0x3c,0x3c,0x0e,0x0e,0x0e,0x2c,0x3d,0x0f,0x0f,0x3e,0x3e,
0x0c,0x0c,0x0c,0x0d,0x0d,0x0d,0x3c,0x0e,0x0e,0x0e,0x3d,0x0f,0x0f,0x0f,0x10,0x3e,
0x1d,0x1d,0x1d,0x1e,0x1e,0x1e,0x2f,0x0e,0x1f,0x1f,0x20,0x0f,0x0f,0x10,0x10,0x21,
0x2e,0x2e,0x2e,0x1e,0x2f,0x2f,0x2f,0x1f,0x1f,0x20,0x20,0x31,0x10,0x10,0x21,0x21,
0x3f,0x3f,0x3f,0x2f,0x30,0x30,0x30,0x30,0x20,0x31,0x31,0x31,0x31,0x21,0x21,0x32,
0x4f,0x4f,0x22,0x40,0x40,0x40,0x40,0x41,0x41,0x41,0x52,0x42,0x42,0x53,0x53,0x64,
0x4f,0x22,0x22,0x22,0x40,0x40,0x40,0x41,0x41,0x41,0x52,0x42,0x42,0x53,0x53,0x64,
0x22,0x22,0x22,0x33,0x33,0x33,0x40,0x41,0x41,0x41,0x52,0x42,0x42,0x53,0x53,0x64,
0x43,0x22,0x33,0x33,0x33,0x44,0x44,0x45,0x45,0x45,0x56,0x46,0x46,0x46,0x57,0x68,
0x43,0x43,0x33,0x33,0x44,0x44,0x44,0x45,0x45,0x45,0x56,0x46,0x46,0x57,0x57,0x68,
0x43,0x43,0x33,0x44,0x44,0x44,0x55,0x45,0x45,0x56,0x56,0x46,0x46,0x57,0x68,0x68,
0x43,0x43,0x43,0x44,0x44,0x55,0x55,0x45,0x45,0x56,0x67,0x46,0x46,0x57,0x68,0x79,
0x47,0x47,0x47,0x48,0x48,0x48,0x48,0x49,0x49,0x49,0x49,0x4a,0x4a,0x4a,0x5b,0x79,
0x47,0x47,0x47,0x48,0x48,0x48,0x48,0x49,0x49,0x49,0x5a,0x4a,0x4a,0x4a,0x5b,0x6c,
0x47,0x47,0x47,0x48,0x48,0x59,0x59,0x49,0x49,0x5a,0x5a,0x4a,0x4a,0x5b,0x5b,0x6c,
0x58,0x58,0x58,0x59,0x59,0x59,0x6a,0x49,0x5a,0x5a,0x6b,0x6b,0x5b,0x5b,0x6c,0x7d,
0x4b,0x4b,0x4b,0x4c,0x4c,0x4c,0x4c,0x4d,0x4d,0x4d,0x6b,0x4e,0x4e,0x4e,0x6c,0x7d,
0x4b,0x4b,0x4b,0x4c,0x4c,0x4c,0x4c,0x4d,0x4d,0x4d,0x5e,0x4e,0x4e,0x4e,0x5f,0x5f,
0x5c,0x5c,0x5c,0x4c,0x5d,0x5d,0x5d,0x4d,0x4d,0x5e,0x5e,0x4e,0x4e,0x5f,0x5f,0x60,
0x5c,0x5c,0x5c,0x5d,0x5d,0x6e,0x6e,0x5e,0x5e,0x5e,0x6f,0x6f,0x5f,0x5f,0x60,0x60,
0x6d,0x6d,0x6d,0x6e,0x6e,0x6e,0x7f,0x7f,0x6f,0x6f,0x70,0x70,0x5f,0x60,0x60,0x71,
0x4f,0x4f,0x40,0x40,0x40,0x40,0x51,0x41,0x41,0x52,0x63,0x42,0x42,0x53,0x64,0x75,
0x4f,0x4f,0x22,0x40,0x40,0x40,0x51,0x41,0x41,0x52,0x63,0x42,0x42,0x53,0x64,0x75,
0x43,0x22,0x33,0x33,0x33,0x40,0x51,0x41,0x41,0x52,0x63,0x42,0x42,0x53,0x64,0x75,
0x43,0x43,0x33,0x33,0x44,0x44,0x44,0x45,0x45,0x45,0x56,0x46,0x46,0x57,0x57,0x68,
0x43,0x43,0x33,0x44,0x44,0x44,0x55,0x45,0x45,0x56,0x56,0x46,0x46,0x57,0x68,0x68,
0x43,0x43,0x43,0x44,0x44,0x55,0x55,0x45,0x45,0x56,0x67,0x46,0x46,0x57,0x68,0x79,
0x54,0x54,0x54,0x44,0x55,0x55,0x55,0x45,0x56,0x56,0x67,0x78,0x78,0x57,0x68,0x79,
0x47,0x47,0x47,0x48,0x48,0x48,0x48,0x49,0x49,0x49,0x49,0x4a,0x4a,0x4a,0x5b,0x79,
0x47,0x47,0x47,0x48,0x48,0x48,0x59,0x49,0x49,0x49,0x5a,0x4a,0x4a,0x5b,0x5b,0x6c,
0x58,0x58,0x58,0x48,0x59,0x59,0x59,0x49,0x49,0x5a,0x5a,0x4a,0x4a,0x5b,0x6c,0x6c,
0x69,0x69,0x69,0x59,0x59,0x6a,0x6a,0x49,0x5a,0x5a,0x6b,0x6b,0x5b,0x5b,0x6c,0x7d,
0x4b,0x4b,0x4b,0x4c,0x4c,0x4c,0x7b,0x4d,0x4d,0x4d,0x6b,0x4e,0x4e,0x4e,0x7d,0x7d,
0x4b,0x4b,0x4b,0x4c,0x4c,0x4c,0x7b,0x4d,0x4d,0x4d,0x5e,0x4e,0x4e,0x4e,0x5f,0x7d,
0x5c,0x5c,0x5c,0x5d,0x5d,0x5d,0x5d,0x4d,0x5e,0x5e,0x5e,0x4e,0x4e,0x5f,0x5f,0x60,
0x6d,0x6d,0x6d,0x5d,0x6e,0x6e,0x6e,0x5e,0x5e,0x6f,0x6f,0x70,0x5f,0x5f,0x60,0x60,
0x7e,0x7e,0x7e,0x6e,0x6e,0x7f,0x7f,0x7f,0x6f,0x6f,0x70,0x70,0x70,0x60,0x60,0x71,
0x50,0x50,0x50,0x40,0x40,0x51,0x51,0x41,0x41,0x52,0x63,0x74,0x42,0x53,0x64,0x75,
0x50,0x50,0x50,0x40,0x40,0x51,0x51,0x41,0x41,0x52,0x63,0x74,0x42,0x53,0x64,0x75,
0x50,0x50,0x33,0x33,0x40,0x51,0x51,0x41,0x41,0x52,0x63,0x74,0x42,0x53,0x64,0x75,
0x43,0x43,0x33,0x44,0x44,0x44,0x55,0x45,0x45,0x56,0x56,0x46,0x46,0x57,0x68,0x68,
0x43,0x43,0x43,0x44,0x44,0x55,0x55,0x45,0x45,0x56,0x67,0x46,0x46,0x57,0x68,0x79,
0x54,0x54,0x54,0x44,0x55,0x55,0x55,0x45,0x56,0x56,0x67,0x78,0x78,0x57,0x68,0x79,
0x54,0x54,0x54,0x55,0x55,0x55,0x66,0x66,0x56,0x67,0x67,0x78,0x78,0x68,0x68,0x79,
0x47,0x47,0x47,0x48,0x48,0x48,0x66,0x49,0x49,0x49,0x78,0x78,0x4a,0x4a,0x5b,0x79,
0x47,0x47,0x47,0x48,0x48,0x59,0x59,0x49,0x49,0x5a,0x5a,0x4a,0x4a,0x5b,0x6c,0x6c,
0x58,0x58,0x58,0x59,0x59,0x59,0x6a,0x49,0x5a,0x5a,0x6b,0x6b,0x5b,0x5b,0x6c,0x7d,
0x69,0x69,0x69,0x59,0x6a,0x6a,0x6a,0x7b,0x5a,0x6b,0x6b,0x6b,0x7c,0x6c,0x6c,0x7d,
0x7a,0x7a,0x7a,0x4c,0x4c,0x7b,0x7b,0x7b,0x4d,0x6b,0x6b,0x7c,0x7c,0x4e,0x7d,0x7d,
0x4b,0x4b,0x4b,0x4c,0x4c,0x7b,0x7b,0x4d,0x4d,0x5e,0x7c,0x7c,0x4e,0x5f,0x5f,0x7d,
0x5c,0x5c,0x5c,0x5d,0x5d,0x5d,0x6e,0x4d,0x5e,0x5e,0x6f,0x4e,0x5f,0x5f,0x60,0x60,
0x6d,0x6d,0x6d,0x6e,0x6e,0x6e,0x6e,0x5e,0x6f,0x6f,0x6f,0x70,0x5f,0x60,0x60,0x71,
0x7e,0x7e,0x7e,0x6e,0x7f,0x7f,0x7f,0x7f,0x6f,0x70,0x70,0x70,0x70,0x60,0x71,0x71,
0x61,0x61,0x61,0x40,0x51,0x51,0x62,0x62,0x73,0x63,0x63,0x74,0x74,0x64,0x64,0x75,
0x61,0x61,0x61,0x40,0x51,0x51,0x62,0x62,0x73,0x63,0x63,0x74,0x74,0x64,0x64,0x75,
0x61,0x61,0x61,0x40,0x51,0x51,0x62,0x62,0x73,0x63,0x63,0x74,0x74,0x64,0x64,0x75,
0x43,0x43,0x43,0x44,0x44,0x55,0x55,0x45,0x45,0x56,0x67,0x46,0x46,0x57,0x68,0x79,
```

```
0x54,0x54,0x54,0x44,0x55,0x55,0x55,0x45,0x56,0x56,0x67,0x78,0x78,0x57,0x68,0x79,
0x54,0x54,0x54,0x55,0x55,0x55,0x66,0x66,0x56,0x67,0x67,0x78,0x78,0x68,0x68,0x79,
0x65,0x65,0x65,0x55,0x55,0x66,0x66,0x66,0x77,0x67,0x78,0x78,0x78,0x78,0x79,0x79,
0x65,0x65,0x65,0x48,0x48,0x66,0x66,0x77,0x77,0x77,0x78,0x78,0x78,0x5b,0x79,0x79,
0x76,0x76,0x76,0x48,0x59,0x59,0x77,0x77,0x77,0x5a,0x5a,0x4a,0x4a,0x5b,0x6c,0x6c,
0x69,0x69,0x69,0x59,0x59,0x6a,0x6a,0x77,0x5a,0x5a,0x6b,0x6b,0x5b,0x6c,0x6c,0x7d,
0x69,0x69,0x69,0x6a,0x6a,0x6a,0x7b,0x7b,0x5a,0x6b,0x6b,0x7c,0x7c,0x6c,0x7d,0x7d,
0x7a,0x7a,0x7a,0x4c,0x7b,0x7b,0x7b,0x7b,0x4d,0x6b,0x7c,0x7c,0x7c,0x7c,0x7d,0x7d,
0x7a,0x7a,0x7a,0x4c,0x7b,0x7b,0x7b,0x7b,0x4d,0x5e,0x7c,0x7c,0x7c,0x5f,0x5f,0x7d,
0x6d,0x6d,0x6d,0x5d,0x5d,0x6e,0x7b,0x5e,0x5e,0x6f,0x6f,0x7c,0x5f,0x5f,0x60,0x60,
0x6d,0x6d,0x6d,0x6e,0x6e,0x6e,0x7f,0x7f,0x6f,0x6f,0x70,0x70,0x5f,0x60,0x60,0x71,
0x7e,0x7e,0x7e,0x7f,0x7f,0x7f,0x7f,0x7f,0x6f,0x70,0x70,0x70,0x70,0x60,0x71,0x71,
0x72,0x72,0x72,0x8f,0x8f,0x62,0x62,0x73,0x73,0x80,0x74,0x81,0x81,0x81,0x92,0x75,
0x72,0x72,0x72,0x8f,0x8f,0x62,0x62,0x73,0x73,0x80,0x74,0x81,0x81,0x81,0x92,0x75,
0x72,0x72,0x72,0x83,0x83,0x62,0x62,0x73,0x73,0x80,0x74,0x81,0x81,0x81,0x92,0x75,
0x82,0x82,0x82,0x83,0x83,0x83,0x83,0x84,0x84,0x84,0x84,0x85,0x85,0x85,0x96,0x79,
0x82,0x82,0x82,0x83,0x83,0x83,0x66,0x84,0x84,0x84,0x67,0x85,0x85,0x85,0x96,0x79,
0x65,0x65,0x65,0x83,0x83,0x66,0x66,0x66,0x84,0x84,0x78,0x78,0x85,0x85,0x96,0x79,
0x65,0x65,0x65,0x83,0x66,0x66,0x66,0x77,0x77,0x77,0x78,0x78,0x78,0x96,0x79,0x79,
0x76,0x76,0x76,0x87,0x87,0x66,0x77,0x77,0x77,0x88,0x78,0x89,0x89,0x89,0x89,0x79,
0x76,0x76,0x76,0x87,0x87,0x87,0x77,0x77,0x88,0x88,0x88,0x89,0x89,0x89,0x9a,0x9a,
0x86,0x86,0x86,0x87,0x87,0x87,0x77,0x88,0x88,0x88,0x6b,0x89,0x89,0x9a,0x9a,0x7d,
0x7a,0x7a,0x7a,0x87,0x6a,0x7b,0x7b,0x7b,0x88,0x6b,0x6b,0x7c,0x7c,0x9a,0x7d,0x7d,
0x8a,0x8a,0x8a,0x8b,0x8b,0x7b,0x7b,0x8c,0x8c,0x8c,0x7c,0x7c,0x8d,0x8d,0x7d,0x7d,
0x8a,0x8a,0x8a,0x8b,0x8b,0x8b,0x7b,0x8c,0x8c,0x8c,0x7c,0x8d,0x8d,0x8d,0x9e,0x9e,
0x8a,0x8a,0x8a,0x8b,0x8b,0x8b,0x9c,0x8c,0x8c,0x9d,0x9d,0x8d,0x8d,0x9e,0x9e,0x9e,
0x9b,0x9b,0x9b,0x9c,0x9c,0x9c,0x7f,0x8c,0x9d,0x9d,0x70,0x70,0x9e,0x9e,0x9e,0x71,
0x7e,0x7e,0x7e,0x7f,0x7f,0x7f,0x7f,0x7f,0x9d,0x70,0x70,0x70,0x9e,0x9e,0x71,0x71,
0x8e,0x8e,0x8e,0x8f,0x8f,0x8f,0x73,0x73,0x80,0x80,0x91,0x81,0x81,0x92,0x92,0xa3,
0x8e,0x8e,0x8e,0x8f,0x8f,0x8f,0x73,0x73,0x80,0x80,0x91,0x81,0x81,0x92,0x92,0xa3,
0x82,0x82,0x82,0x83,0x83,0x83,0x73,0x73,0x80,0x80,0x91,0x81,0x81,0x92,0x92,0xa3,
0x82,0x82,0x82,0x83,0x83,0x83,0x83,0x84,0x84,0x84,0x95,0x85,0x85,0x85,0x96,0xa7,
0x82,0x82,0x82,0x83,0x83,0x83,0x94,0x84,0x84,0x84,0x95,0x85,0x85,0x96,0x96,0xa7,
0x82,0x82,0x82,0x83,0x83,0x94,0x94,0x84,0x84,0x95,0x95,0x85,0x85,0x96,0xa7,0xa7,
0x76,0x76,0x76,0x83,0x94,0x94,0x77,0x77,0x77,0x95,0x95,0x85,0x85,0x96,0xa7,0xa7,
0x76,0x76,0x76,0x87,0x87,0x87,0x77,0x77,0x88,0x88,0x88,0x89,0x89,0x89,0x9a,0x9a,
0x86,0x86,0x86,0x87,0x87,0x87,0x77,0x88,0x88,0x88,0x99,0x89,0x89,0x9a,0x9a,0xab,
0x86,0x86,0x86,0x87,0x87,0x98,0x98,0x88,0x88,0x99,0x99,0x89,0x89,0x9a,0x9a,0xab,
0x97,0x97,0x97,0x98,0x98,0x98,0x98,0x88,0x99,0x99,0x99,0x89,0x9a,0x9a,0xab,0xab,
0x8a,0x8a,0x8a,0x8b,0x8b,0x8b,0x8b,0x8c,0x8c,0x8c,0x8c,0x8d,0x8d,0x8d,0xab,0xbc,
0x8a,0x8a,0x8a,0x8b,0x8b,0x8b,0x8b,0x8c,0x8c,0x8c,0x9d,0x8d,0x8d,0x8d,0x9e,0x9e,
0x9b,0x9b,0x9b,0x8b,0x9c,0x9c,0x9c,0x8c,0x9d,0x9d,0x9d,0x8d,0x8d,0x9e,0x9e,0xaf,
0x9b,0x9b,0x9b,0x9c,0x9c,0xad,0xad,0x9d,0x9d,0x9d,0xae,0xae,0x9e,0x9e,0xaf,0xaf,
0xac,0xac,0xac,0xad,0xad,0xad,0xad,0x9d,0xae,0xae,0xae,0xbf,0x9e,0xaf,0xaf,0xaf,
0x9f,0x9f,0x9f,0x8f,0x90,0x90,0xa1,0x80,0x80,0x91,0x91,0x81,0x81,0x92,0xa3,0xb4,
0x9f,0x9f,0x9f,0x8f,0x90,0x90,0xa1,0x80,0x80,0x91,0x91,0x81,0x81,0x92,0xa3,0xb4,
0x9f,0x9f,0x9f,0x83,0x90,0x90,0xa1,0x80,0x80,0x91,0x91,0x81,0x81,0x92,0xa3,0xb4,
0x82,0x82,0x82,0x83,0x83,0x94,0x94,0x84,0x84,0x95,0x95,0x85,0x85,0x96,0x96,0xa7,
0x93,0x93,0x93,0x83,0x94,0x94,0x94,0x84,0x84,0x95,0x95,0x85,0x85,0x96,0xa7,0xa7,
0x93,0x93,0x93,0x94,0x94,0x94,0xa5,0x84,0x95,0x95,0xa6,0xa6,0x96,0x96,0xa7,0xb8,
0xa4,0xa4,0xa4,0x94,0x94,0xa5,0xa5,0x77,0x95,0x95,0xa6,0xa6,0x96,0xa7,0xa7,0xb8,
0x86,0x86,0x86,0x87,0x87,0x87,0x77,0x88,0x88,0x88,0x99,0x89,0x89,0x9a,0x9a,0xb8,
0x86,0x86,0x86,0x87,0x87,0x98,0x98,0x88,0x88,0x99,0x99,0x89,0x89,0x9a,0x9a,0xab,
0x97,0x97,0x97,0x98,0x98,0x98,0x98,0x88,0x99,0x99,0x99,0x89,0x9a,0x9a,0xab,0xab,
0x97,0x97,0x97,0x98,0x98,0xa9,0xa9,0x99,0x99,0x99,0xaa,0xaa,0x9a,0xab,0xab,0xbc,
0x8a,0x8a,0x8a,0x8b,0x8b,0xa9,0xa9,0x8c,0x8c,0x8c,0xaa,0x8d,0x8d,0x8d,0xab,0xbc,
0x8a,0x8a,0x8a,0x8b,0x8b,0x9c,0x9c,0x8c,0x8c,0x9d,0x9d,0x8d,0x8d,0x9e,0x9e,0xbc,
0x9b,0x9b,0x9b,0x9c,0x9c,0x9c,0xad,0x9d,0x9d,0x9d,0xae,0x8d,0x9e,0x9e,0xaf,0xaf,
0xac,0xac,0xac,0x9c,0xad,0xad,0xad,0x9d,0x9d,0xae,0xae,0xae,0x9e,0xaf,0xaf,0xaf,
0xbd,0xbd,0xbd,0xad,0xad,0xbe,0xbe,0xbe,0xae,0xae,0xbf,0xbf,0xbf,0xaf,0xaf,0xb0,
```

```
0xa0,0xa0,0xa0,0x90,0xa1,0xa1,0xa1,0xb2,0x91,0x91,0xa2,0xa2,0xb3,0xa3,0xa3,0xb4,
0xa0,0xa0,0xa0,0x90,0xa1,0xa1,0xa1,0xb2,0x91,0x91,0xa2,0xa2,0xb3,0xa3,0xa3,0xb4,
0xa0,0xa0,0xa0,0x90,0xa1,0xa1,0xa1,0xb2,0x91,0x91,0xa2,0xa2,0xb3,0xa3,0xa3,0xb4,
0x93,0x93,0x93,0x94,0x94,0x94,0xa5,0x84,0x95,0x95,0xa6,0xa6,0x96,0x96,0xa7,0xb8,
0xa4,0xa4,0xa4,0x94,0x94,0xa5,0xa5,0x84,0x95,0x95,0xa6,0xa6,0x96,0x96,0xa7,0xb8,
0xa4,0xa4,0xa4,0x94,0xa5,0xa5,0xa5,0xb6,0x95,0xa6,0xa6,0xa6,0xb7,0xa7,0xa7,0xb8,
0xa4,0xa4,0xa4,0xa5,0xa5,0xa5,0xb6,0xb6,0x95,0xa6,0xa6,0xb7,0xb7,0xa7,0xb8,0xb8,
0xb5,0xb5,0xb5,0x87,0x87,0xb6,0xb6,0xb6,0x88,0x99,0xa6,0xb7,0xb7,0x9a,0xb8,0xb8,
0x97,0x97,0x97,0x98,0x98,0x98,0x98,0x88,0x99,0x99,0x99,0x89,0x9a,0x9a,0xab,0xab,
0x97,0x97,0x97,0x98,0x98,0xa9,0xa9,0x99,0x99,0x99,0xaa,0xaa,0x9a,0xab,0xab,0xbc,
0xa8,0xa8,0xa8,0xa9,0xa9,0xa9,0xa9,0xa9,0x99,0xaa,0xaa,0xaa,0xbb,0xab,0xab,0xbc,
0xa8,0xa8,0xa8,0xa9,0xa9,0xa9,0xba,0xba,0x8c,0xaa,0xaa,0xbb,0xbb,0xab,0xbc,0xbc,
0xb9,0xb9,0xb9,0x9c,0x9c,0xba,0xba,0xba,0x9d,0x9d,0xbb,0xbb,0xbb,0x9e,0x9e,0xbc,
0xac,0xac,0xac,0x9c,0x9c,0xad,0xad,0x9d,0x9d,0xae,0xae,0xae,0x9e,0x9e,0xaf,0xaf,
0xac,0xac,0xac,0xad,0xad,0xad,0xbe,0xbe,0xae,0xae,0xae,0xbf,0x9e,0xaf,0xaf,0xb0,
0xbd,0xbd,0xbd,0xbe,0xbe,0xbe,0xbe,0xbe,0xae,0xbf,0xbf,0xbf,0xbf,0xaf,0xb0,0xb0,
0xb1,0xb1,0xb1,0xce,0xce,0xb2,0xb2,0xcf,0xcf,0xa2,0xa2,0xb3,0xb3,0xc0,0xb4,0xb4,
0xb1,0xb1,0xb1,0xce,0xce,0xb2,0xb2,0xcf,0xcf,0xa2,0xa2,0xb3,0xb3,0xc0,0xb4,0xb4,
0xb1,0xb1,0xb1,0xc2,0xc2,0xb2,0xb2,0xc3,0xc3,0xa2,0xa2,0xb3,0xb3,0xc0,0xb4,0xb4,
0xc1,0xc1,0xc1,0xc2,0xc2,0xc2,0xa5,0xc3,0xc3,0xc3,0xa6,0xc4,0xc4,0xc4,0xa7,0xb8,
0xc1,0xc1,0xc1,0xc2,0xc2,0xa5,0xb6,0xc3,0xc3,0xc3,0xa6,0xc4,0xc4,0xc4,0xb8,0xb8,
0xb5,0xb5,0xb5,0xc2,0xa5,0xb6,0xb6,0xb6,0xc3,0xa6,0xa6,0xb7,0xb7,0xc4,0xb8,0xb8,
0xb5,0xb5,0xb5,0xa5,0xb6,0xb6,0xb6,0xb6,0xc3,0xa6,0xb7,0xb7,0xb7,0xb7,0xb8,0xb8,
0xc5,0xc5,0xc5,0xc6,0xc6,0xb6,0xb6,0xc7,0xc7,0xc7,0xb7,0xb7,0xc8,0xc8,0xb8,0xb8,
0xc5,0xc5,0xc5,0xc6,0xc6,0xc6,0xc6,0xc7,0xc7,0xc7,0xaa,0xc8,0xc8,0xc8,0xab,0xbc,
0xa8,0xa8,0xa8,0xc6,0xc6,0xa9,0xa9,0xc7,0xc7,0xaa,0xaa,0xaa,0xc8,0xc8,0xab,0xbc,
0xa8,0xa8,0xa8,0xa9,0xa9,0xa9,0xba,0xba,0xaa,0xaa,0xaa,0xbb,0xbb,0xab,0xbc,0xbc,
0xb9,0xb9,0xb9,0xca,0xca,0xba,0xba,0xba,0xcb,0xaa,0xbb,0xbb,0xbb,0xcc,0xbc,0xbc,
0xb9,0xb9,0xb9,0xca,0xca,0xba,0xba,0xcb,0xcb,0xcb,0xbb,0xbb,0xcc,0xcc,0xcc,0xbc,
0xc9,0xc9,0xc9,0xca,0xca,0xca,0xba,0xcb,0xcb,0xcb,0xae,0xcc,0xcc,0xcc,0xaf,0xaf,
0xbd,0xbd,0xbd,0xad,0xbe,0xbe,0xbe,0xbe,0xae,0xae,0xbf,0xbf,0xcc,0xaf,0xaf,0xb0,
0xbd,0xbd,0xbd,0xbe,0xbe,0xbe,0xbe,0xbe,0xbf,0xbf,0xbf,0xbf,0xbf,0xaf,0xb0,0xb0,
0xcd,0xcd,0xcd,0xce,0xce,0xce,0xb2,0xcf,0xcf,0xcf,0xb3,0xb3,0xc0,0xc0,0xd1,0xb4,
0xcd,0xcd,0xcd,0xce,0xce,0xce,0xb2,0xcf,0xcf,0xcf,0xb3,0xb3,0xc0,0xc0,0xd1,0xb4,
0xc1,0xc1,0xc1,0xc2,0xc2,0xc2,0xb2,0xc3,0xc3,0xc3,0xb3,0xb3,0xc0,0xc0,0xd1,0xb4,
0xc1,0xc1,0xc1,0xc2,0xc2,0xc2,0xc2,0xc3,0xc3,0xc3,0xd4,0xc4,0xc4,0xc4,0xd5,0xd5,
0xc1,0xc1,0xc1,0xc2,0xc2,0xc2,0xb6,0xc3,0xc3,0xc3,0xd4,0xc4,0xc4,0xc4,0xd5,0xb8,
0xc1,0xc1,0xc1,0xc2,0xc2,0xb6,0xb6,0xc3,0xc3,0xd4,0xb7,0xb7,0xc4,0xd5,0xd5,0xb8,
0xb5,0xb5,0xb5,0xc2,0xb6,0xb6,0xb6,0xb6,0xc3,0xd4,0xb7,0xb7,0xb7,0xd5,0xd5,0xb8,
0xc5,0xc5,0xc5,0xc6,0xc6,0xc6,0xb6,0xc7,0xc7,0xc7,0xb7,0xc8,0xc8,0xc8,0xd9,0xd9,
0xc5,0xc5,0xc5,0xc6,0xc6,0xc6,0xc6,0xc7,0xc7,0xc7,0xd8,0xc8,0xc8,0xc8,0xd9,0xd9,
0xc5,0xc5,0xc5,0xc6,0xc6,0xd7,0xd7,0xc7,0xc7,0xd8,0xd8,0xc8,0xc8,0xd9,0xd9,0xbc,
0xb9,0xb9,0xb9,0xd7,0xd7,0xba,0xba,0xba,0xd8,0xd8,0xbb,0xbb,0xbb,0xd9,0xd9,0xbc,
0xb9,0xb9,0xb9,0xca,0xca,0xba,0xba,0xcb,0xcb,0xcb,0xbb,0xbb,0xcc,0xcc,0xcc,0xbc,
0xc9,0xc9,0xc9,0xca,0xca,0xca,0xba,0xcb,0xcb,0xcb,0xbb,0xcc,0xcc,0xcc,0xdd,0xdd,
0xc9,0xc9,0xc9,0xca,0xca,0xdb,0xdb,0xcb,0xcb,0xdc,0xdc,0xcc,0xcc,0xdd,0xdd,0xdd,
0xda,0xda,0xda,0xdb,0xdb,0xdb,0xdb,0xdc,0xdc,0xdc,0xdc,0xcc,0xdd,0xdd,0xdd,0xb0,
0xbd,0xbd,0xbd,0xdb,0xbe,0xbe,0xbe,0xdc,0xdc,0xbf,0xbf,0xbf,0xdd,0xdd,0xb0,0xb0,
0xde,0xde,0xde,0xdf,0xdf,0xdf,0xe0,0xcf,0xd0,0xd0,0xe1,0xc0,0xc0,0xd1,0xd1,0xe2,
0xde,0xde,0xde,0xdf,0xdf,0xdf,0xe0,0xcf,0xd0,0xd0,0xe1,0xc0,0xc0,0xd1,0xd1,0xe2,
0xde,0xde,0xde,0xdf,0xdf,0xdf,0xe0,0xc3,0xd0,0xd0,0xe1,0xc0,0xc0,0xd1,0xd1,0xe2,
0xd2,0xd2,0xd2,0xc2,0xd3,0xd3,0xd3,0xc3,0xc3,0xd4,0xd4,0xc4,0xc4,0xd5,0xd5,0xe6,
0xd2,0xd2,0xd2,0xd3,0xd3,0xd3,0xd3,0xc3,0xd4,0xd4,0xd4,0xc4,0xc4,0xd5,0xd5,0xe6,
0xd2,0xd2,0xd2,0xd3,0xd3,0xd3,0xe4,0xc3,0xd4,0xd4,0xe5,0xc4,0xd5,0xd5,0xe6,0xe6,
0xe3,0xe3,0xe3,0xd3,0xd3,0xe4,0xb6,0xd4,0xd4,0xe5,0xe5,0xb7,0xd5,0xd5,0xe6,0xe6,
0xc5,0xc5,0xc5,0xc6,0xc6,0xc6,0xd7,0xc7,0xc7,0xd8,0xd8,0xc8,0xc8,0xd9,0xd9,0xd9,
0xd6,0xd6,0xd6,0xc6,0xd7,0xd7,0xd7,0xc7,0xd8,0xd8,0xd8,0xc8,0xc8,0xd9,0xd9,0xea,
0xd6,0xd6,0xd6,0xd7,0xd7,0xd7,0xe8,0xd8,0xd8,0xd8,0xe9,0xc8,0xd9,0xd9,0xea,0xea,
0xe7,0xe7,0xe7,0xd7,0xd7,0xe8,0xe8,0xd8,0xd8,0xe9,0xe9,0xe9,0xd9,0xd9,0xea,0xea,
0xc9,0xc9,0xc9,0xca,0xca,0xca,0xba,0xcb,0xcb,0xcb,0xe9,0xcc,0xcc,0xcc,0xea,0xea,
```

```
    0xc9,0xc9,0xc9,0xca,0xca,0xdb,0xdb,0xcb,0xcb,0xdc,0xdc,0xcc,0xcc,0xdd,0xdd,0xdd,
    0xda,0xda,0xda,0xdb,0xdb,0xdb,0xdb,0xdc,0xdc,0xdc,0xdc,0xcc,0xdd,0xdd,0xdd,0xee,
    0xda,0xda,0xda,0xdb,0xdb,0xec,0xec,0xdc,0xdc,0xed,0xed,0xed,0xdd,0xdd,0xee,0xee,
    0xeb,0xeb,0xeb,0xec,0xec,0xec,0xec,0xdc,0xed,0xed,0xed,0xed,0xdd,0xee,0xee,0xee,
    0xef,0xef,0xef,0xdf,0xe0,0xe0,0xe0,0xd0,0xd0,0xe1,0xe1,0xf2,0xd1,0xd1,0xe2,0xe2,
    0xef,0xef,0xef,0xdf,0xe0,0xe0,0xe0,0xd0,0xd0,0xe1,0xe1,0xf2,0xd1,0xd1,0xe2,0xe2,
    0xef,0xef,0xef,0xdf,0xe0,0xe0,0xe0,0xd0,0xd0,0xe1,0xe1,0xf2,0xd1,0xd1,0xe2,0xe2,
    0xd2,0xd2,0xd2,0xd3,0xd3,0xe4,0xe4,0xd4,0xd4,0xd4,0xe5,0xe5,0xd5,0xd5,0xe6,0xe6,
    0xe3,0xe3,0xe3,0xd3,0xe4,0xe4,0xe4,0xd4,0xd4,0xe5,0xe5,0xf6,0xd5,0xd5,0xe6,0xe6,
    0xe3,0xe3,0xe3,0xe4,0xe4,0xe4,0xe4,0xd4,0xe5,0xe5,0xe5,0xf6,0xd5,0xe6,0xe6,0xf7,
    0xe3,0xe3,0xe3,0xe4,0xe4,0xe4,0xf5,0xf5,0xe5,0xe5,0xf6,0xf6,0xd5,0xe6,0xe6,0xf7,
    0xd6,0xd6,0xd6,0xd7,0xd7,0xd7,0xf5,0xc7,0xd8,0xd8,0xf6,0xc8,0xd9,0xd9,0xd9,0xf7,
    0xd6,0xd6,0xd6,0xd7,0xd7,0xe8,0xe8,0xd8,0xd8,0xd8,0xe9,0xe9,0xd9,0xd9,0xea,0xea,
    0xe7,0xe7,0xe7,0xd7,0xe8,0xe8,0xe8,0xd8,0xd8,0xe9,0xe9,0xe9,0xd9,0xea,0xea,0xea,
    0xe7,0xe7,0xe7,0xe8,0xe8,0xe8,0xf9,0xf9,0xe9,0xe9,0xe9,0xfa,0xd9,0xea,0xea,0xfb,
    0xf8,0xf8,0xf8,0xe8,0xf9,0xf9,0xf9,0xcb,0xe9,0xe9,0xfa,0xfa,0xcc,0xea,0xea,0xfb,
    0xda,0xda,0xda,0xdb,0xdb,0xdb,0xdb,0xdc,0xdc,0xdc,0xdc,0xcc,0xdd,0xdd,0xdd,0xee,
    0xda,0xda,0xda,0xdb,0xdb,0xec,0xec,0xdc,0xdc,0xed,0xed,0xed,0xdd,0xdd,0xee,0xee,
    0xeb,0xeb,0xeb,0xec,0xec,0xec,0xec,0xdc,0xed,0xed,0xed,0xed,0xdd,0xee,0xee,0xee,
    0xeb,0xeb,0xeb,0xec,0xec,0xfd,0xfd,0xfd,0xed,0xed,0xfe,0xfe,0xee,0xee,0xee,0xff,
    0xf0,0xf0,0xf0,0xe0,0xf1,0xf1,0xf1,0xf1,0xe1,0xf2,0xf2,0xf2,0xf2,0xe2,0xe2,0xf3,
    0xf0,0xf0,0xf0,0xe0,0xf1,0xf1,0xf1,0xf1,0xe1,0xf2,0xf2,0xf2,0xf2,0xe2,0xe2,0xf3,
    0xf0,0xf0,0xf0,0xe0,0xf1,0xf1,0xf1,0xf1,0xe1,0xf2,0xf2,0xf2,0xf2,0xe2,0xe2,0xf3,
    0xe3,0xe3,0xe3,0xe4,0xe4,0xe4,0xf5,0xf5,0xe5,0xe5,0xf6,0xf6,0xd5,0xe6,0xe6,0xf7,
    0xf4,0xf4,0xf4,0xe4,0xe4,0xf5,0xf5,0xf5,0xe5,0xe5,0xf6,0xf6,0xf6,0xe6,0xe6,0xf7,
    0xf4,0xf4,0xf4,0xe4,0xf5,0xf5,0xf5,0xf5,0xe5,0xf6,0xf6,0xf6,0xf6,0xe6,0xf7,0xf7,
    0xf4,0xf4,0xf4,0xf5,0xf5,0xf5,0xf5,0xf5,0xe5,0xf6,0xf6,0xf6,0xf6,0xe6,0xf7,0xf7,
    0xf4,0xf4,0xf4,0xf5,0xf5,0xf5,0xf5,0xf5,0xd8,0xf6,0xf6,0xf6,0xd9,0xd9,0xf7,0xf7,
    0xe7,0xe7,0xe7,0xe8,0xe8,0xe8,0xe8,0xd8,0xe9,0xe9,0xe9,0xfa,0xd9,0xea,0xea,0xea,
    0xf8,0xf8,0xf8,0xe8,0xe8,0xf9,0xf9,0xf9,0xe9,0xe9,0xfa,0xfa,0xfa,0xea,0xea,0xfb,
    0xf8,0xf8,0xf8,0xf9,0xf9,0xf9,0xf9,0xf9,0xe9,0xfa,0xfa,0xfa,0xfa,0xea,0xfb,0xfb,
    0xf8,0xf8,0xf8,0xf9,0xf9,0xf9,0xf9,0xf9,0xfa,0xfa,0xfa,0xfa,0xfa,0xea,0xfb,0xfb,
    0xf8,0xf8,0xf8,0xdb,0xf9,0xf9,0xf9,0xdc,0xdc,0xfa,0xfa,0xfa,0xdd,0xdd,0xee,0xfb,
    0xeb,0xeb,0xeb,0xec,0xec,0xec,0xec,0xdc,0xed,0xed,0xed,0xed,0xdd,0xee,0xee,0xee,
    0xeb,0xeb,0xeb,0xec,0xec,0xfd,0xfd,0xfd,0xed,0xed,0xfe,0xfe,0xee,0xee,0xee,0xff,
    0xfc,0xfc,0xfc,0xfd,0xfd,0xfd,0xfd,0xfd,0xed,0xfe,0xfe,0xfe,0xfe,0xee,0xff,0xff,
  }
  };
```

⟨lib_graphics/libmemdraw/cmap.c 428a⟩≡

```
  /*
   * generated by mkcmap.c
   */
  #include <u.h>
  #include <libc.h>
  #include <draw.h>
  #include <memdraw.h>

  ⟨global def 423b⟩
  ⟨global memdefcmap 248a⟩
  ⟨function _memmkcmap 248c⟩
```

G.4.7 lib_graphics/libmemdraw/defont.c

⟨lib_graphics/libmemdraw/defont.c 428b⟩≡

```
  #include <u.h>
  #include <libc.h>
  #include <draw.h>
  #include <draw_private.h>
```

```
#include <font.h>
#include <memdraw.h>

⟨function getmemdefont 338f⟩
```

G.4.8 lib_graphics/libmemdraw/init.c

```
⟨lib_graphics/libmemdraw/init.c 429a⟩≡
  #include <u.h>
  #include <libc.h>
  #include <draw.h>
  #include <memdraw.h>
  #include <pool.h>

  extern Pool* imagmem;

  ⟨global memones 38b⟩
  ⟨global memzeros 38c⟩
  ⟨global memwhite 38d⟩
  ⟨global memblack 38e⟩
  ⟨global memtransparent 38f⟩
  ⟨global memopaque 38g⟩

  int _ifmt(Fmt*);

  extern int replmul[];

  extern uchar replbit[1+8][256];

  ⟨function mktables 267c⟩


  ⟨function memimageinit 48a⟩
```

G.4.9 lib_graphics/libmemdraw/alphadraw.c

```
⟨lib_graphics/libmemdraw/alphadraw.c 429b⟩≡
  #include <u.h>
  #include <libc.h>
  #include <draw.h>
  #include <memdraw.h>

  ⟨macro DBG1 349b⟩

  ⟨function MUL 112a⟩
  ⟨constant MASK13 324a⟩
  ⟨constant MASK02 324c⟩
  ⟨function MUL13 323d⟩
  ⟨function MUL02 324b⟩
  ⟨function MUL0123 323c⟩

  ⟨global ones 109a⟩

  /*
   * General alpha drawing case.  Can handle anything.
   */
  typedef struct Buffer Buffer;
```

⟨*struct* Buffer 107d⟩

```
typedef struct ParamDraw Param;
typedef Buffer Readfn(Param*, uchar*, int);
typedef void Writefn(Param*, uchar*, Buffer);
typedef Buffer Calcfn(Buffer, Buffer, Buffer, int, int, int);
```

⟨*enum* _anon_*(*(lib_graphics/libmemdraw/draw.c)*)* 315d⟩

⟨*struct* ParamDraw 106g⟩

```
static Readfn greymaskread, replread, readptr;
```

⟨*global* nullwrite 317d⟩

```
static Calcfn alphacalc0, alphacalc14, alphacalc2810, alphacalc3679, alphacalc5, alphacalc11, alphacalcS;
static Calcfn boolcalc14, boolcalc236789, boolcalc1011;

static Readfn* readfn(Memimage*);
static Readfn* readalphafn(Memimage*);
static Writefn* writefn(Memimage*);

static Calcfn* boolcopyfn(Memimage*, Memimage*);
static Readfn* convfn(Memimage*, Param*, Memimage*, Param*, int*);
static Readfn* ptrfn(Memimage*);
```

⟨*global* alphacalc 111d⟩

⟨*global* boolcalc 320a⟩

```
/*
 * Avoid standard Lock, QLock so that can be used in kernel.
 */
typedef struct Dbuf Dbuf;
```
⟨*struct* Dbuf 105b⟩
⟨*global* dbuf 105e⟩

⟨*function* allocdbuf 105f⟩

⟨*function* getparam 107b⟩

⟨*function* clipy 105a⟩

⟨*function* alphadraw 103⟩

⟨*function* alphacalc0 267e⟩

⟨*function* alphacalc14 268a⟩

⟨*function* alphacalc2810 268b⟩

⟨*function* alphacalc3679 269⟩

⟨*function* alphacalc5 271a⟩

⟨*function* alphacalc11 111e⟩

⟨*function* alphacalcS 323a⟩

⟨*function* boolcalc14 320c⟩

⟨*function* boolcalc236789 321⟩

⟨*function* boolcalc1011 322⟩
⟨*function* replread 315e⟩

⟨*function* greymaskread 111c⟩

⟨*global* replbit 267b⟩

⟨*function* readnbit 241d⟩

⟨*function* writenbit 243b⟩

⟨*function* readcmap 249e⟩

⟨*function* writecmap 250⟩

⟨*function* readbyte 108b⟩

⟨*function* writebyte 109c⟩

⟨*function* readfn 108a⟩

⟨*function* readalphafn 111b⟩

⟨*function* writefn 109b⟩

⟨*function* nullwrite 317e⟩

⟨*function* readptr 318a⟩

⟨*function* boolmemmove 318c⟩

⟨*function* boolcopy8 318d⟩

⟨*function* boolcopy16 319a⟩

⟨*function* boolcopy24 319b⟩

⟨*function* boolcopy32 319c⟩

⟨*function* genconv 317c⟩

⟨*function* convfn 316c⟩

```
// in resolution.c now
extern ulong pixelbits(Memimage *i, Point pt);
```

⟨*function* boolcopyfn 318b⟩

G.4.10 lib_graphics/libmemdraw/color.c

⟨lib_graphics/libmemdraw/color.c 431⟩≡

```
#include <u.h>
#include <libc.h>
```

```
#include <draw.h>
#include <memdraw.h>

⟨macro DBG1 349b⟩

⟨function imgtorgba 42⟩

⟨function rgbatoimg 43a⟩

// could put memfillcolor in draw.c so no need those extern decls.
extern void memsetl(void *vp, ulong val, int n);
extern void memset24(void *vp, ulong val, int n);

⟨function memfillcolor 43b⟩
```

G.4.11 lib_graphics/libmemdraw/draw.c

```
⟨lib_graphics/libmemdraw/draw.c 432⟩≡
#include <u.h>
#include <libc.h>
#include <draw.h>
#include <memdraw.h>

⟨macro DBG1 349b⟩
⟨macro DBG 349c⟩

⟨global drawdebug 348g⟩

typedef int Subdraw(Memdrawparam*);
static Subdraw chardraw, memoptdraw;

// in alphadraw.c
Subdraw alphadraw;
// in color.c
ulong imgtorgba(Memimage*, ulong);
ulong rgbatoimg(Memimage*, ulong);
// in resolution.c
ulong pixelbits(Memimage*, Point);

⟨function memimagedraw 93b⟩

⟨function drawclip 95c⟩



⟨function memsets 245e⟩

⟨function memsetl 44b⟩

⟨function memset24 246c⟩

⟨function memoptdraw 100c⟩

⟨function chardraw 313⟩
```

G.4.12 lib_graphics/libmemdraw/ellipse.c

⟨lib_graphics/libmemdraw/ellipse.c 433a⟩≡

```
#include <u.h>
#include <libc.h>
#include <draw.h>
#include <memdraw.h>
#include <memlayer.h>

/*
 * ellipse(dst, c, a, b, t, src, sp)
 *   draws an ellipse centered at c with semiaxes a,b>=0
 *   and semithickness t>=0, or filled if t<0.  point sp
 *   in src maps to c in dst
 *
 *   very thick skinny ellipses are brushed with circles (slow)
 *   others are approximated by filling between 2 ellipses
 *   criterion for very thick when b<a: t/b > 0.5*x/(1-x)
 *   where x = b/a
 */

typedef struct ParamEllipse Param;
typedef struct State State;

static void bellipse(int, State*, Param*);
static void erect(int, int, int, int, Param*);
static void eline(int, int, int, int, Param*);

⟨struct ParamEllipse 130c⟩

⟨struct State 130d⟩

⟨function newstate 130e⟩

⟨function step 133a⟩

⟨function memellipse 131⟩

⟨global p00((lib_graphics/libmemdraw/ellipse.c)) 132a⟩

⟨function bellipse 132b⟩

⟨function erect 132c⟩

⟨function epoint 133b⟩

⟨function eline 133c⟩
```

G.4.13 lib_graphics/libmemdraw/fillpoly.c

⟨lib_graphics/libmemdraw/fillpoly.c 433b⟩≡

```
#include <u.h>
#include <libc.h>
#include <draw.h>
#include <memdraw.h>
#include <memlayer.h>

typedef struct Seg Seg;
```

⟨*struct* Seg 122c⟩

```
static void zsort(Seg **seg, Seg **ep);
static int ycompare(void*, void*);
static int xcompare(void*, void*);
static int zcompare(void*, void*);
static void xscan(Memimage *dst, Seg **seg, Seg *segtab, int nseg, int wind, Memimage *src, Point sp, int, int,
static void yscan(Memimage *dst, Seg **seg, Seg *segtab, int nseg, int wind, Memimage *src, Point sp, int, int)
```

⟨*function* fillline 126a⟩

⟨*function* fillpoint 261a⟩

⟨*function* memfillpoly 122b⟩

⟨*function* _memfillpolysc 122d⟩

⟨*function* mod 126b⟩

⟨*function* sdiv 126c⟩

⟨*function* smuldivmod 126d⟩

⟨*function* xscan 123h⟩

⟨*function* yscan 258d⟩

⟨*function* zsort 127a⟩

⟨*function* ycompare 127b⟩

⟨*function* xcompare 258c⟩

⟨*function* zcompare 128a⟩

G.4.14 lib_graphics/libmemdraw/hooks.c

⟨*function* memdraw_hwdraw 434a⟩≡ (434d)

```
int
hwdraw(Memdrawparam*)
{
    return 0; /* could not satisfy request */
}
```

⟨*function* memdraw_iprint 434b⟩≡ (434d)

```
int
memdraw_iprint(char*,...)
{
    return -1;
}
```

⟨*global* iprint 434c⟩≡ (434d)

```
int  (*iprint)(char*, ...) = &memdraw_iprint;
```

⟨lib_graphics/libmemdraw/hooks.c 434d⟩≡

```
#include <u.h>
#include <libc.h>
#include <draw.h>
#include <memdraw.h>
```

⟨*function* memdraw_hwdraw 434a⟩

⟨*function* memdraw_iprint 434b⟩

⟨*global* iprint 434c⟩

G.4.15 lib_graphics/libmemdraw/line.c

⟨lib_graphics/libmemdraw/line.c 435a⟩≡

```
#include <u.h>
#include <libc.h>
#include <draw.h>
#include <memdraw.h>
#include <memlayer.h>
```

⟨*enum* _anon_(lib_graphics/libmemdraw/line.c) 257b⟩

⟨*function* membrush 256d⟩

⟨*function* discend 256c⟩

⟨*function* arrowend 257e⟩

⟨*function* _memimageline 115c⟩

⟨*function* addbbox 325c⟩

⟨*function* memlineendsize 325a⟩

⟨*function* lmax 325b⟩

⟨*function* memlinebbox 324g⟩

G.4.16 lib_graphics/libmemdraw/transfer.c

⟨lib_graphics/libmemdraw/transfer.c 435b⟩≡

```
#include <u.h>
#include <libc.h>
#include <draw.h>
#include <memdraw.h>
```

⟨*function* loadmemimage 175d⟩

⟨*function* unloadmemimage 178h⟩

G.4.17 lib_graphics/libmemdraw/poly.c

⟨lib_graphics/libmemdraw/poly.c 435c⟩≡

```
#include <u.h>
#include <libc.h>
#include <draw.h>
#include <memdraw.h>
#include <memlayer.h>
```

⟨*function* mempoly 121c⟩

G.4.18 lib_graphics/libmemdraw/io.c

⟨lib_graphics/libmemdraw/io.c 436a⟩≡

```
#include <u.h>
#include <libc.h>
#include <draw.h>
#include <draw_private.h>
#include <memdraw.h>

⟨function readmemimage 341b⟩

⟨function creadmemimage 336⟩



⟨constant CHUNK 343a⟩

⟨constant HSHIFT((lib_graphics/libmemdraw/write.c)) 343b⟩
⟨constant NHASH((lib_graphics/libmemdraw/write.c)) 343c⟩
⟨constant HMASK((lib_graphics/libmemdraw/write.c)) 343d⟩
⟨function hupdate((lib_graphics/libmemdraw/write.c)) 343e⟩
typedef struct Hlist Hlist;
⟨struct Hlist((lib_graphics/libmemdraw/write.c)) 343f⟩

⟨function writememimage 344⟩
```

G.4.19 lib_graphics/libmemdraw/string.c

⟨lib_graphics/libmemdraw/string.c 436b⟩≡

```
#include <u.h>
#include <libc.h>
#include <draw.h>
#include <memdraw.h>
#include <memlayer.h>
#include <font.h>

⟨function memimagestring 341a⟩

⟨function memsubfontwidth 340c⟩
```

G.4.20 lib_graphics/libmemdraw/subfont.c

⟨lib_graphics/libmemdraw/subfont.c 436c⟩≡

```
#include <u.h>
#include <libc.h>
#include <draw.h>
#include <memdraw.h>

⟨function allocmemsubfont 340b⟩
```

G.5 lib_graphics/libmemdraw/scripts/

G.5.1 lib_graphics/libmemdraw/scripts/mkcmap.c

⟨function mkcmap 436d⟩≡ (438a)

```
/*
struct Memcmap
{
    uchar cmap2rgb[3*256];
    uchar rgb2cmap[16*16*16];
};
*/

static Memcmap*
mkcmap(void)
{
    static Memcmap def;

    int i, rgb, r, g, b;

    for(i=0; i<256; i++){
        rgb = cmap2rgb(i);
        r = (rgb>>16)&0xff;
        g = (rgb>>8)&0xff;
        b = rgb&0xff;
        def.cmap2rgb[3*i] = r;
        def.cmap2rgb[3*i+1] = g;
        def.cmap2rgb[3*i+2] = b;
    }

    for(r=0; r<16; r++)
    for(g=0; g<16; g++)
    for(b=0; b<16; b++)
        def.rgb2cmap[r*16*16+g*16+b] = rgb2cmap(r*0x11, g*0x11, b*0x11);
    return &def;
}
```

⟨*function* main*(*(lib_graphics/libmemdraw/scripts/mkcmap.c)*)* 437⟩≡ (438a)

```
void
main(int argc, char **argv)
{
    Memcmap *c;
    int i, j;

    ARGBEGIN{
    }ARGEND

    memimageinit();
    c = mkcmap();

    print("#include <u.h>\n#include <libc.h>\n");
    print("#include <draw.h>\n");
    print("#include <memdraw.h>\n\n");
    print("static Memcmap def = {\n");
    print("/* cmap2rgb */ {\n");
    for(i=0; i<sizeof(c->cmap2rgb); ){
        print("\t");
        for(j=0; j<16; j++, i++)
            print("0x%2.2ux,", c->cmap2rgb[i]);
        print("\n");
    }
    print("},\n");
    print("/* rgb2cmap */ {\n");
    for(i=0; i<sizeof(c->rgb2cmap);){
        print("\t");
```

```
        for(j=0; j<16; j++, i++)
            print("0x%2.2ux,", c->rgb2cmap[i]);
        print("\n");
    }
    print("}\n");
    print("};\n");
    print("Memcmap *memdefcmap = &def;\n");
    print("void _memmkcmap(void){}\n");
    exits(0);
}
```

⟨lib_graphics/libmemdraw/scripts/mkcmap.c 438a⟩≡

```
#include <u.h>
#include <libc.h>
#include <draw.h>
#include <memdraw.h>

⟨function mkcmap 436d⟩

⟨function main((lib_graphics/libmemdraw/scripts/mkcmap.c)) 437⟩
```

G.6 lib_graphics/libmemdraw/tests/

G.6.1 lib_graphics/libmemdraw/tests/arctest.c

⟨function main((lib_graphics/libmemdraw/tests/arctest.c)) 438b⟩≡ (439a)

```
void
main(int argc, char **argv)
{
    Memimage *x;
    Point c = {208,871};
    int a = 441;
    int b = 441;
    int thick = 0;
    Point sp = {0,0};
    int alpha = 51;
    int phi = 3;
    vlong t0, t1;
    int i, n;
    vlong del;

    if (argc != 2) {
        fprint(2, "usage: arctest number\n");
        exits("usage");
    }
    memimageinit();

    x = allocmemimage(Rect(0,0,1000,1000), CMAP8);
    n = atoi(argv[1]);

    t0 = nsec();
    t0 = nsec();
    t0 = nsec();
    t1 = nsec();
    del = t1-t0;
    t0 = nsec();
    for(i=0; i<n; i++)
        memarc(x, c, a, b, thick, memblack, sp, alpha, phi, SoverD);
```

```
    t1 = nsec();
    print("%lld %lld\n", t1-t0-del, del);
  }
```

⟨lib_graphics/libmemdraw/tests/arctest.c 439a⟩≡

```
  #include <u.h>
  #include <libc.h>
  #include <draw.h>
  #include <memdraw.h>
  #include <memlayer.h>

  ⟨function main((lib_graphics/libmemdraw/tests/arctest.c)) 438b⟩
```

G.6.2 lib_graphics/libmemdraw/tests/memdrawtest.c

⟨function RGB2K((lib_graphics/libmemdraw/tests/memdrawtest.c)) 439b⟩≡ (456)

```
  #define RGB2K(r,g,b) ((299*((ulong)(r))+587*((ulong)(g))+114*((ulong)(b)))/1000)
```

⟨global drawdebug((lib_graphics/libmemdraw/tests/memdrawtest.c)) 439c⟩≡ (456)

```
  //old: used to have static in front, but at some point I got
  //some errors (I probably introduced some regressions in 8c of kencc)
  int drawdebug=0;
```

⟨global seed 439d⟩≡ (456)

```
  static int seed;
```

⟨global niters 439e⟩≡ (456)

```
  static int niters = 100;
```

⟨global dbpp 439f⟩≡ (456)

```
  static int dbpp; /* bits per pixel in destination */
```

⟨global sbpp 439g⟩≡ (456)

```
  static int sbpp; /* bits per pixel in src */
```

⟨global mbpp 439h⟩≡ (456)

```
  static int mbpp; /* bits per pixel in mask */
```

⟨global dpm 439i⟩≡ (456)

```
  static int dpm; /* pixel mask at high part of byte, in destination */
```

⟨global nbytes 439j⟩≡ (456)

```
  static int nbytes; /* in destination */
```

⟨global Xrange 439k⟩≡ (456)

```
  static int Xrange = 64;
```

⟨global Yrange 439l⟩≡ (456)

```
  static int Yrange = 8;
```

⟨global dst 439m⟩≡ (456)

```
  static Memimage *dst;
```

⟨global src 439n⟩≡ (456)

```
  static Memimage *src;
```

⟨global mask 439o⟩≡ (456)

```
  static Memimage *mask;
```

⟨*global* stmp 440a⟩≡ (456)

```
static Memimage *stmp;
```

⟨*global* mtmp 440b⟩≡ (456)

```
static Memimage *mtmp;
```

⟨*global* ones*(*(lib_graphics/libmemdraw/tests/memdrawtest.c)*)* 440c⟩≡ (456)

```
static Memimage *ones;
```

⟨*global* dstbits 440d⟩≡ (456)

```
static uchar *dstbits;
```

⟨*global* srcbits 440e⟩≡ (456)

```
static uchar *srcbits;
```

⟨*global* maskbits 440f⟩≡ (456)

```
static uchar *maskbits;
```

⟨*global* savedstbits 440g⟩≡ (456)

```
static ulong *savedstbits;
```

⟨*function* main*(*(lib_graphics/libmemdraw/tests/memdrawtest.c)*)* 440h⟩≡ (456)

```
void
main(int argc, char *argv[])
{
    memimageinit();
    seed = time(0);

    ARGBEGIN{
    case 'x':
        Xrange = atoi(ARGF());
        break;
    case 'y':
        Yrange = atoi(ARGF());
        break;
    case 'n':
        niters = atoi(ARGF());
        break;
    case 's':
        seed = atoi(ARGF());
        break;
    }ARGEND

    dchan = "r8g8b8";
    schan = "r8g8b8";
    mchan = "r8g8b8";
    switch(argc){
    case 3: mchan = argv[2];
    case 2: schan = argv[1];
    case 1: dchan = argv[0];
    case 0: break;
    default: goto Usage;
    Usage:
        fprint(2, "usage: dtest [dchan [schan [mchan]]]\n");
        exits("usage");
    }

// fmtinstall('b', numbconv); /* binary! */

    fprint(2, "%s -x %d -y %d -s 0x%x %s %s %s\n", argv0, Xrange, Yrange, seed, dchan, schan, mchan);
```

```
    srand(seed);

    dst = allocmemimage(Rect(0, 0, Xrange, Yrange), strtochan(dchan));
    src = allocmemimage(Rect(0, 0, Xrange, Yrange), strtochan(schan));
    mask = allocmemimage(Rect(0, 0, Xrange, Yrange), strtochan(mchan));
    stmp = allocmemimage(Rect(0, 0, Xrange, Yrange), strtochan(schan));
    mtmp = allocmemimage(Rect(0, 0, Xrange, Yrange), strtochan(mchan));
    ones = allocmemimage(Rect(0, 0, Xrange, Yrange), strtochan(mchan));
// print("chan %lux %lux %lux %lux %lux %lux\n", dst->chan, src->chan, mask->chan, stmp->chan, mtmp->chan, ones
    if(dst==0 || src==0 || mask==0 || mtmp==0 || ones==0) {
    Alloc:
        fprint(2, "dtest: allocation failed: %r\n");
        exits("alloc");
    }
    nbytes = (4*Xrange+4)*Yrange;
    srcbits = malloc(nbytes);
    dstbits = malloc(nbytes);
    maskbits = malloc(nbytes);
    savedstbits = malloc(nbytes);
    if(dstbits==0 || srcbits==0 || maskbits==0 || savedstbits==0)
        goto Alloc;
    dbpp = dst->depth;
    sbpp = src->depth;
    mbpp = mask->depth;
    dpm = 0xFF ^ (0xFF>>dbpp);
    memset(ones->data->bdata, 0xFF, ones->width*sizeof(ulong)*Yrange);


    fprint(2, "dtest: verify single pixel operation\n");
    verifyone();

    fprint(2, "dtest: verify full line non-replicated\n");
    verifyline();

    fprint(2, "dtest: verify full rectangle non-replicated\n");
    verifyrect();

    fprint(2, "dtest: verify full rectangle source replicated\n");
    verifyrectrepl(1, 0);

    fprint(2, "dtest: verify full rectangle mask replicated\n");
    verifyrectrepl(0, 1);

    fprint(2, "dtest: verify full rectangle source and mask replicated\n");
    verifyrectrepl(1, 1);

    exits(0);
}
```

⟨*function* Bprintr5g6b5 441⟩≡ (456)

```
/*
 * Dump out an ASCII representation of an image.  The label specifies
 * a list of characters to put at various points in the picture.
 */
static void
Bprintr5g6b5(Biobuf *bio, char*, ulong v)
{
    int r,g,b;
    r = (v>>11)&31;
    g = (v>>5)&63;
```

```
    b = v&31;
    Bprint(bio, "%.2x%.2x%.2x", r,g,b);
}
```

⟨*function* Bprintr5g5b5a1 442a⟩≡ (456)

```
static void
Bprintr5g5b5a1(Biobuf *bio, char*, ulong v)
{
    int r,g,b,a;
    r = (v>>11)&31;
    g = (v>>6)&31;
    b = (v>>1)&31;
    a = v&1;
    Bprint(bio, "%.2x%.2x%.2x%.2x", r,g,b,a);
}
```

⟨*function* dumpimage 442b⟩≡ (456)

```
void
dumpimage(char *name, Memimage *img, void *vdata, Point labelpt)
{
    Biobuf b;
    uchar *data;
    uchar *p;
    char *arg;
    void (*fmt)(Biobuf*, char*, ulong);
    int npr, x, y, nb, bpp;
    ulong v, mask;
    Rectangle r;

    fmt = nil;
    arg = nil;
    switch(img->depth){
    case 1:
    case 2:
    case 4:
        fmt = (void(*)(Biobuf*,char*,ulong))Bprint;
        arg = "%.1ux";
        break;
    case 8:
        fmt = (void(*)(Biobuf*,char*,ulong))Bprint;
        arg = "%.2ux";
        break;
    case 16:
        arg = nil;
        if(img->chan == RGB16)
            fmt = Bprintr5g6b5;
        else{
            fmt = (void(*)(Biobuf*,char*,ulong))Bprint;
            arg = "%.4ux";
        }
        break;
    case 24:
        fmt = (void(*)(Biobuf*,char*,ulong))Bprint;
        arg = "%.6lux";
        break;
    case 32:
        fmt = (void(*)(Biobuf*,char*,ulong))Bprint;
        arg = "%.8lux";
        break;
    }
```

```
    if(fmt == nil){
        fprint(2, "bad format\n");
        abort();
    }

    r  = img->r;
    Binit(&b, 2, OWRITE);
    data = vdata;
    bpp = img->depth;
    Bprint(&b, "%s\t%d\tr %R clipr %R repl %d data %p *%P\n", name, r.min.x, r, img->clipr, (img->flags&Frepl)
    mask = (1ULL<<bpp)-1;
// for(y=r.min.y; y<r.max.y; y++){
    for(y=0; y<Yrange; y++){
        nb = 0;
        v = 0;
        p = data+(byteaddr(img, Pt(0,y))-(uchar*)img->data->bdata);
        Bprint(&b, "%-4d\t", y);
//  for(x=r.min.x; x<r.max.x; x++){
        for(x=0; x<Xrange; x++){
            if(x==0)
                Bprint(&b, "\t");

            if(x != 0 && (x%8)==0)
                Bprint(&b, " ");

            npr = 0;
            if(x==labelpt.x && y==labelpt.y){
                Bprint(&b, "*");
                npr++;
            }
            if(npr == 0)
                Bprint(&b, " ");

            while(nb < bpp){
                v &= (1<<nb)-1;
                v |= (ulong)(*p++) << nb;
                nb += 8;
            }
            nb -= bpp;
//   print("bpp %d v %.8lux mask %.8lux nb %d\n", bpp, v, mask, nb);
            fmt(&b, arg, (v>>nb)&mask);
        }
        Bprint(&b, "\n");
    }
    Bterm(&b);
}
```

⟨*function* checkone 443⟩≡ (456)

```
/*
 * Verify that the destination pixel has the specified value.
 * The value is in the high bits of v, suitably masked, but must
 * be extracted from the destination Memimage.
 */
void
checkone(Point p, Point sp, Point mp)
{
    int delta;
    uchar *dp, *sdp;

    delta = (uchar*)byteaddr(dst, p)-(uchar*)dst->data->bdata;
```

```
    dp = (uchar*)dst->data->bdata+delta;
    sdp = (uchar*)savedstbits+delta;

    if(memcmp(dp, sdp, (dst->depth+7)/8) != 0) {
        fprint(2, "dtest: one bad pixel drawing at dst %P from source %P mask %P\n", p, sp, mp);
        fprint(2, " %.2ux %.2ux %.2ux %.2ux should be %.2ux %.2ux %.2ux %.2ux\n",
            dp[0], dp[1], dp[2], dp[3], sdp[0], sdp[1], sdp[2], sdp[3]);
        fprint(2, "addresses dst %p src %p mask %p\n", dp, byteaddr(src, sp), byteaddr(mask, mp));
        dumpimage("src", src, src->data->bdata, sp);
        dumpimage("mask", mask, mask->data->bdata, mp);
        dumpimage("origdst", dst, dstbits, p);
        dumpimage("dst", dst, dst->data->bdata, p);
        dumpimage("gooddst", dst, savedstbits, p);
        abort();
    }
}
```

⟨*function* RECTPTS 444a⟩≡ (456)

```
/*
 * Verify that the destination line has the same value as the saved line.
 */
#define RECTPTS(r) (r).min.x, (r).min.y, (r).max.x, (r).max.y
```

⟨*function* checkline 444b⟩≡ (456)

```
void
checkline(Rectangle r, Point sp, Point mp, int y, Memimage *stmp, Memimage *mtmp)
{
    ulong *dp;
    int nb;
    ulong *saved;

    dp = wordaddr(dst, Pt(0, y));
    saved = savedstbits + y*dst->width;
    if(dst->depth < 8)
        nb = Xrange/(8/dst->depth);
    else
        nb = Xrange*(dst->depth/8);
    if(memcmp(dp, saved, nb) != 0){
        fprint(2, "dtest: bad line at y=%d; saved %p dp %p\n", y, saved, dp);
        fprint(2, "draw dst %R src %P mask %P\n", r, sp, mp);
        dumpimage("src", src, src->data->bdata, sp);
        if(stmp) dumpimage("stmp", stmp, stmp->data->bdata, sp);
        dumpimage("mask", mask, mask->data->bdata, mp);
        if(mtmp) dumpimage("mtmp", mtmp, mtmp->data->bdata, mp);
        dumpimage("origdst", dst, dstbits, r.min);
        dumpimage("dst", dst, dst->data->bdata, r.min);
        dumpimage("gooddst", dst, savedstbits, r.min);
        abort();
    }
}
```

⟨*function* fill 444c⟩≡ (456)

```
/*
 * Fill the bits of an image with random data.
 * The Memimage parameter is used only to make sure
 * the data is well formatted: only ucbits is written.
 */
void
fill(Memimage *img, uchar *ucbits)
{
```

```
    int i, x, y;
    ushort *up;
    uchar alpha, r, g, b;
    void *data;

    if((img->flags&Falpha) == 0){
        up = (ushort*)ucbits;
        for(i=0; i<nbytes/2; i++)
            *up++ = lrand() >> 7;
        if(i+i != nbytes)
            *(uchar*)up = lrand() >> 7;
    }else{
        data = img->data->bdata;
        img->data->bdata = ucbits;

        for(x=img->r.min.x; x<img->r.max.x; x++)
        for(y=img->r.min.y; y<img->r.max.y; y++){
            alpha = rand() >> 4;
            r = rand()%(alpha+1);
            g = rand()%(alpha+1);
            b = rand()%(alpha+1);
            putpixel(img, Pt(x,y), rgbatopix(r,g,b,alpha));
        }
        img->data->bdata = data;
    }

}
```

⟨*function* verifyonemask 445⟩≡ (456)

```
/*
 * Mask is preset; do the rest
 */
void
verifyonemask(void)
{
    Point dp, sp, mp;

    fill(dst, dstbits);
    fill(src, srcbits);
    memmove(dst->data->bdata, dstbits, dst->width*sizeof(ulong)*Yrange);
    memmove(src->data->bdata, srcbits, src->width*sizeof(ulong)*Yrange);
    memmove(mask->data->bdata, maskbits, mask->width*sizeof(ulong)*Yrange);

    dp.x = nrand(Xrange);
    dp.y = nrand(Yrange);

    sp.x = nrand(Xrange);
    sp.y = nrand(Yrange);

    mp.x = nrand(Xrange);
    mp.y = nrand(Yrange);

    drawonepixel(dst, dp, src, sp, mask, mp);
    memmove(mask->data->bdata, maskbits, mask->width*sizeof(ulong)*Yrange);
    memmove(savedstbits, dst->data->bdata, dst->width*sizeof(ulong)*Yrange);

    memmove(dst->data->bdata, dstbits, dst->width*sizeof(ulong)*Yrange);
    memimagedraw(dst, Rect(dp.x, dp.y, dp.x+1, dp.y+1), src, sp, mask, mp, SoverD);
    memmove(mask->data->bdata, maskbits, mask->width*sizeof(ulong)*Yrange);
```

```
    checkone(dp, sp, mp);
}
```

⟨*function* verifyone 446a⟩≡ (456)

```
void
verifyone(void)
{
    int i;

    /* mask all zeros */
    memset(maskbits, 0, nbytes);
    for(i=0; i<niters; i++)
        verifyonemask();

    /* mask all ones */
    memset(maskbits, 0xFF, nbytes);
    for(i=0; i<niters; i++)
        verifyonemask();

    /* random mask */
    for(i=0; i<niters; i++){
        fill(mask, maskbits);
        verifyonemask();
    }
}
```

⟨*function* verifylinemask 446b⟩≡ (456)

```
/*
 * Mask is preset; do the rest
 */
void
verifylinemask(void)
{
    Point sp, mp, tp, up;
    Rectangle dr;
    int x;

    fill(dst, dstbits);
    fill(src, srcbits);
    memmove(dst->data->bdata, dstbits, dst->width*sizeof(ulong)*Yrange);
    memmove(src->data->bdata, srcbits, src->width*sizeof(ulong)*Yrange);
    memmove(mask->data->bdata, maskbits, mask->width*sizeof(ulong)*Yrange);

    dr.min.x = nrand(Xrange-1);
    dr.min.y = nrand(Yrange-1);
    dr.max.x = dr.min.x + 1 + nrand(Xrange-1-dr.min.x);
    dr.max.y = dr.min.y + 1;

    sp.x = nrand(Xrange);
    sp.y = nrand(Yrange);

    mp.x = nrand(Xrange);
    mp.y = nrand(Yrange);

    tp = sp;
    up = mp;
    for(x=dr.min.x; x<dr.max.x && tp.x<Xrange && up.x<Xrange; x++,tp.x++,up.x++)
        memimagedraw(dst, Rect(x, dr.min.y, x+1, dr.min.y+1), src, tp, mask, up, SoverD);
    memmove(savedstbits, dst->data->bdata, dst->width*sizeof(ulong)*Yrange);
```

```
    memmove(dst->data->bdata, dstbits, dst->width*sizeof(ulong)*Yrange);

    memimagedraw(dst, dr, src, sp, mask, mp, SoverD);
    checkline(dr, drawrepl(src->r, sp), drawrepl(mask->r, mp), dr.min.y, nil, nil);
}
```

⟨*function* verifyline 447a⟩≡ (456)

```
void
verifyline(void)
{
    int i;

    /* mask all ones */
    memset(maskbits, 0xFF, nbytes);
    for(i=0; i<niters; i++)
        verifylinemask();

    /* mask all zeros */
    memset(maskbits, 0, nbytes);
    for(i=0; i<niters; i++)
        verifylinemask();

    /* random mask */
    for(i=0; i<niters; i++){
        fill(mask, maskbits);
        verifylinemask();
    }
}
```

⟨*function* verifyrectmask 447b⟩≡ (456)

```
/*
 * Mask is preset; do the rest
 */
void
verifyrectmask(void)
{
    Point sp, mp, tp, up;
    Rectangle dr;
    int x, y;

    fill(dst, dstbits);
    fill(src, srcbits);
    memmove(dst->data->bdata, dstbits, dst->width*sizeof(ulong)*Yrange);
    memmove(src->data->bdata, srcbits, src->width*sizeof(ulong)*Yrange);
    memmove(mask->data->bdata, maskbits, mask->width*sizeof(ulong)*Yrange);

    dr.min.x = nrand(Xrange-1);
    dr.min.y = nrand(Yrange-1);
    dr.max.x = dr.min.x + 1 + nrand(Xrange-1-dr.min.x);
    dr.max.y = dr.min.y + 1 + nrand(Yrange-1-dr.min.y);

    sp.x = nrand(Xrange);
    sp.y = nrand(Yrange);

    mp.x = nrand(Xrange);
    mp.y = nrand(Yrange);

    tp = sp;
    up = mp;
    for(y=dr.min.y; y<dr.max.y && tp.y<Yrange && up.y<Yrange; y++,tp.y++,up.y++){
```

```
        for(x=dr.min.x; x<dr.max.x && tp.x<Xrange && up.x<Xrange; x++,tp.x++,up.x++)
            memimagedraw(dst, Rect(x, y, x+1, y+1), src, tp, mask, up, SoverD);
        tp.x = sp.x;
        up.x = mp.x;
    }
    memmove(savedstbits, dst->data->bdata, dst->width*sizeof(ulong)*Yrange);

    memmove(dst->data->bdata, dstbits, dst->width*sizeof(ulong)*Yrange);

    memimagedraw(dst, dr, src, sp, mask, mp, SoverD);
    for(y=0; y<Yrange; y++)
        checkline(dr, drawrepl(src->r, sp), drawrepl(mask->r, mp), y, nil, nil);
}
```

⟨*function* verifyrect 448a⟩≡ (456)

```
void
verifyrect(void)
{
    int i;

    /* mask all zeros */
    memset(maskbits, 0, nbytes);
    for(i=0; i<niters; i++)
        verifyrectmask();

    /* mask all ones */
    memset(maskbits, 0xFF, nbytes);
    for(i=0; i<niters; i++)
        verifyrectmask();

    /* random mask */
    for(i=0; i<niters; i++){
        fill(mask, maskbits);
        verifyrectmask();
    }
}
```

⟨*function* randrect 448b⟩≡ (456)

```
Rectangle
randrect(void)
{
    Rectangle r;

    r.min.x = nrand(Xrange-1);
    r.min.y = nrand(Yrange-1);
    r.max.x = r.min.x + 1 + nrand(Xrange-1-r.min.x);
    r.max.y = r.min.y + 1 + nrand(Yrange-1-r.min.y);
    return r;
}
```

⟨*function* tilexy 448c⟩≡ (456)

```
/*
 * Return coordinate corresponding to x withing range [minx, maxx)
 */
int
tilexy(int minx, int maxx, int x)
{
    int sx;

    sx = (x-minx) % (maxx-minx);
```

```
    if(sx < 0)
        sx += maxx-minx;
    return sx+minx;
}
```

⟨*function* replicate 449⟩≡ (456)

```
void
replicate(Memimage *i, Memimage *tmp)
{
    Rectangle r, r1;
    int x, y, nb;

    /* choose the replication window (i->r) */
    r.min.x = nrand(Xrange-1);
    r.min.y = nrand(Yrange-1);
    /* make it trivial more often than pure chance allows */
    switch(lrand()&0){
    case 1:
        r.max.x = r.min.x + 2;
        r.max.y = r.min.y + 2;
        if(r.max.x < Xrange && r.max.y < Yrange)
            break;
        /* fall through */
    case 0:
        r.max.x = r.min.x + 1;
        r.max.y = r.min.y + 1;
        break;
    default:
        if(r.min.x+3 >= Xrange)
            r.max.x = Xrange;
        else
            r.max.x = r.min.x+3 + nrand(Xrange-(r.min.x+3));

        if(r.min.y+3 >= Yrange)
            r.max.y = Yrange;
        else
            r.max.y = r.min.y+3 + nrand(Yrange-(r.min.y+3));
    }
    assert(r.min.x >= 0);
    assert(r.max.x <= Xrange);
    assert(r.min.y >= 0);
    assert(r.max.y <= Yrange);
    /* copy from i to tmp so we have just the replicated bits */
    nb = tmp->width*sizeof(ulong)*Yrange;
    memset(tmp->data->bdata, 0, nb);
    memimagedraw(tmp, r, i, r.min, ones, r.min, SoverD);
    memmove(i->data->bdata, tmp->data->bdata, nb);
    /* i is now a non-replicated instance of the replication */
    /* replicate it by hand through tmp */
    memset(tmp->data->bdata, 0, nb);
    x = -(tilexy(r.min.x, r.max.x, 0)-r.min.x);
    for(; x<Xrange; x+=Dx(r)){
        y = -(tilexy(r.min.y, r.max.y, 0)-r.min.y);
        for(; y<Yrange; y+=Dy(r)){
            /* set r1 to instance of tile by translation */
            r1.min.x = x;
            r1.min.y = y;
            r1.max.x = r1.min.x+Dx(r);
            r1.max.y = r1.min.y+Dy(r);
            memimagedraw(tmp, r1, i, r.min, ones, r.min, SoverD);
```

```
        }
    }
    i->flags |= Frepl;
    i->r = r;
    i->clipr = randrect();
// fprint(2, "replicate [[%d %d] [%d %d]] [[%d %d][%d %d]]\n", r.min.x, r.min.y, r.max.x, r.max.y,
//  i->clipr.min.x, i->clipr.min.y, i->clipr.max.x, i->clipr.max.y);
    tmp->clipr = i->clipr;
}
```

⟨*function* verifyrectmaskrepl 450⟩≡ (456)

```
/*
 * Mask is preset; do the rest
 */
void
verifyrectmaskrepl(int srcrepl, int maskrepl)
{
    Point sp, mp, tp, up;
    Rectangle dr;
    int x, y;
    Memimage *s, *m;

// print("verfrect %d %d\n", srcrepl, maskrepl);
    src->flags &= ~Frepl;
    src->r = Rect(0, 0, Xrange, Yrange);
    src->clipr = src->r;
    stmp->flags &= ~Frepl;
    stmp->r = Rect(0, 0, Xrange, Yrange);
    stmp->clipr = src->r;
    mask->flags &= ~Frepl;
    mask->r = Rect(0, 0, Xrange, Yrange);
    mask->clipr = mask->r;
    mtmp->flags &= ~Frepl;
    mtmp->r = Rect(0, 0, Xrange, Yrange);
    mtmp->clipr = mask->r;

    fill(dst, dstbits);
    fill(src, srcbits);

    memmove(dst->data->bdata, dstbits, dst->width*sizeof(ulong)*Yrange);
    memmove(src->data->bdata, srcbits, src->width*sizeof(ulong)*Yrange);
    memmove(mask->data->bdata, maskbits, mask->width*sizeof(ulong)*Yrange);

    if(srcrepl){
        replicate(src, stmp);
        s = stmp;
    }else
        s = src;
    if(maskrepl){
        replicate(mask, mtmp);
        m = mtmp;
    }else
        m = mask;

    dr = randrect();

    sp.x = nrand(Xrange);
    sp.y = nrand(Yrange);

    mp.x = nrand(Xrange);
```

```
    mp.y = nrand(Yrange);

    //DBG print("smalldraws\n");
    for(tp.y=sp.y,up.y=mp.y,y=dr.min.y; y<dr.max.y && tp.y<Yrange && up.y<Yrange; y++,tp.y++,up.y++)
        for(tp.x=sp.x,up.x=mp.x,x=dr.min.x; x<dr.max.x && tp.x<Xrange && up.x<Xrange; x++,tp.x++,up.x++)
            memimagedraw(dst, Rect(x, y, x+1, y+1), s, tp, m, up, SoverD);
    memmove(savedstbits, dst->data->bdata, dst->width*sizeof(ulong)*Yrange);

    memmove(dst->data->bdata, dstbits, dst->width*sizeof(ulong)*Yrange);

    //DBG print("bigdraw\n");
    memimagedraw(dst, dr, src, sp, mask, mp, SoverD);
    for(y=0; y<Yrange; y++)
        checkline(dr, drawrepl(src->r, sp), drawrepl(mask->r, mp), y, srcrepl?stmp:nil, maskrepl?mtmp:nil);
}
```

⟨*function* verifyrectrepl 451a⟩≡ (456)

```
void
verifyrectrepl(int srcrepl, int maskrepl)
{
    int i;

    /* mask all ones */
    memset(maskbits, 0xFF, nbytes);
    for(i=0; i<niters; i++)
        verifyrectmaskrepl(srcrepl, maskrepl);

    /* mask all zeros */
    memset(maskbits, 0, nbytes);
    for(i=0; i<niters; i++)
        verifyrectmaskrepl(srcrepl, maskrepl);

    /* random mask */
    for(i=0; i<niters; i++){
        fill(mask, maskbits);
        verifyrectmaskrepl(srcrepl, maskrepl);
    }
}
```

⟨*function* replbits 451b⟩≡ (456)

```
/*
 * Convert v, which is nhave bits wide, into its nwant bits wide equivalent.
 * Replicates to widen the value, truncates to narrow it.
 */
ulong
replbits(ulong v, int nhave, int nwant)
{
    v &= (1<<nhave)-1;
    for(; nhave<nwant; nhave*=2)
        v |= v<<nhave;
    v >>= (nhave-nwant);
    return v & ((1<<nwant)-1);
}
```

⟨*function* pixtorgba 451c⟩≡ (456)

```
/*
 * Decode a pixel into the uchar* values.
 */
void
pixtorgba(ulong v, uchar *r, uchar *g, uchar *b, uchar *a)
```

```
{
    *a = v>>24;
    *r = v>>16;
    *g = v>>8;
    *b = v;
}
```

⟨*function* rgbatopix 452a⟩≡ (456)

```
/*
 * Convert uchar channels into ulong pixel.
 */
ulong
rgbatopix(uchar r, uchar g, uchar b, uchar a)
{
    return (a<<24)|(r<<16)|(g<<8)|b;
}
```

⟨*function* getpixel 452b⟩≡ (456)

```
/*
 * Retrieve the pixel value at pt in the image.
 */
ulong
getpixel(Memimage *img, Point pt)
{
    uchar r, g, b, a, *p;
    int nbits, npack, bpp;
    ulong v, c, rbits, bits;

    r = g = b = 0;
    a = ~0; /* default alpha is full */

    p = byteaddr(img, pt);
    v = p[0]|(p[1]<<8)|(p[2]<<16)|(p[3]<<24);
    bpp = img->depth;
    if(bpp<8){
        /*
         * Sub-byte greyscale pixels.
         *
         * We want to throw away the top pt.x%npack pixels and then use the next bpp bits
         * in the bottom byte of v.  This madness is due to having big endian bits
         * but little endian bytes.
         */
        npack = 8/bpp;
        v >>= 8 - bpp*(pt.x%npack+1);
        v &= (1<<bpp)-1;
        r = g = b = replbits(v, bpp, 8);
    }else{
        /*
         * General case.  We need to parse the channel descriptor and do what it says.
         * In all channels but the color map, we replicate to 8 bits because that's the
         * precision that all calculations are done at.
         *
         * In the case of the color map, we leave the bits alone, in case a color map
         * with less than 8 bits of index is used.  This is currently disallowed, so it's
         * sort of silly.
         */

        for(c=img->chan; c; c>>=8){
            nbits = NBITS(c);
            bits = v & ((1<<nbits)-1);
```

```
            rbits = replbits(bits, nbits, 8);
            v >>= nbits;
            switch(TYPE(c)){
            case CRed:
                r = rbits;
                break;
            case CGreen:
                g = rbits;
                break;
            case CBlue:
                b = rbits;
                break;
            case CGrey:
                r = g = b = rbits;
                break;
            case CAlpha:
                a = rbits;
                break;
            case CMap:
                p = img->cmap->cmap2rgb + 3*bits;
                r = p[0];
                g = p[1];
                b = p[2];
                break;
            case CIgnore:
                break;
            default:
                fprint(2, "unknown channel type %lud\n", TYPE(c));
                abort();
            }
        }
    }
    return rgbatopix(r, g, b, a);
}
```

⟨*function* getgrey 453a⟩≡ (456)

```
/*
 * Return the greyscale equivalent of a pixel.
 */
uchar
getgrey(Memimage *img, Point pt)
{
    uchar r, g, b, a;
    pixtorgba(getpixel(img, pt), &r, &g, &b, &a);
    return RGB2K(r, g, b);
}
```

⟨*function* getmask 453b⟩≡ (456)

```
/*
 * Return the value at pt in image, if image is interpreted
 * as a mask.  This means the alpha channel if present, else
 * the greyscale or its computed equivalent.
 */
uchar
getmask(Memimage *img, Point pt)
{
    if(img->flags&Falpha)
        return getpixel(img, pt)>>24;
    else
        return getgrey(img, pt);
}
```

⟨*function* putpixel 454⟩≡ (456)

```
/*
 * Write a pixel to img at point pt.
 *
 * We do this by reading a 32-bit little endian
 * value from p and then writing it back
 * after tweaking the appropriate bits.  Because
 * the data is little endian, we don't have to worry
 * about what the actual depth is, as long as it is
 * less than 32 bits.
 */
void
putpixel(Memimage *img, Point pt, ulong nv)
{
    uchar r, g, b, a, *p, *q;
    ulong c, mask, bits, v;
    int bpp, sh, npack, nbits;

    pixtorgba(nv, &r, &g, &b, &a);

    p = byteaddr(img, pt);
    v = p[0]|(p[1]<<8)|(p[2]<<16)|(p[3]<<24);
    bpp = img->depth;
    //DBG print("v %.8lux...", v);
    if(bpp < 8){
        /*
         * Sub-byte greyscale pixels.  We need to skip the leftmost pt.x%npack pixels,
         * which is equivalent to skipping the rightmost npack - pt.x%npack - 1 pixels.
         */
        npack = 8/bpp;
        sh = bpp*(npack - pt.x%npack - 1);
        bits = RGB2K(r,g,b);
        //DBG print("repl %lux 8 %d = %lux...", bits, bpp, replbits(bits, 8, bpp));
        bits = replbits(bits, 8, bpp);
        mask = (1<<bpp)-1;
        //DBG print("bits %lux mask %lux sh %d...", bits, mask, sh);
        mask <<= sh;
        bits <<= sh;
        //DBG print("(%lux & %lux) | (%lux & %lux)", v, ~mask, bits, mask);
        v = (v & ~mask) | (bits & mask);
    } else {
        /*
         * General case.  We need to parse the channel descriptor again.
         */
        sh = 0;
        for(c=img->chan; c; c>>=8){
            nbits = NBITS(c);
            switch(TYPE(c)){
            case CRed:
                bits = r;
                break;
            case CGreen:
                bits = g;
                break;
            case CBlue:
                bits = b;
                break;
            case CGrey:
                bits = RGB2K(r, g, b);
                break;
```

```
            case CAlpha:
                bits = a;
                break;
            case CIgnore:
                bits = 0;
                break;
            case CMap:
                q = img->cmap->rgb2cmap;
                bits = q[(r>>4)*16*16+(g>>4)*16+(b>>4)];
                break;
            default:
                SET(bits);
                fprint(2, "unknown channel type %lud\n", TYPE(c));
                abort();
            }

            //DBG print("repl %lux 8 %d = %lux...", bits, nbits, replbits(bits, 8, nbits));
            if(TYPE(c) != CMap)
                bits = replbits(bits, 8, nbits);
            mask = (1<<nbits)-1;
            //DBG print("bits %lux mask %lux sh %d...", bits, mask, sh);
            bits <<= sh;
            mask <<= sh;
            v = (v & ~mask) | (bits & mask);
            sh += nbits;
        }
    }
    //DBG print("v %.8lux\n", v);
    p[0] = v;
    p[1] = v>>8;
    p[2] = v>>16;
    p[3] = v>>24;
}
```

⟨*function* drawonepixel 455⟩≡ (456)

```
void
drawonepixel(Memimage *dst, Point dp, Memimage *src, Point sp, Memimage *mask, Point mp)
{
    uchar m, M, sr, sg, sb, sa, sk, dr, dg, db, da, dk;

    pixtorgba(getpixel(dst, dp), &dr, &dg, &db, &da);
    pixtorgba(getpixel(src, sp), &sr, &sg, &sb, &sa);
    m = getmask(mask, mp);
    M = 255-(sa*m)/255;

    //DBG print("dst %x %x %x %x src %x %x %x %x m %x = ", dr,dg,db,da, sr,sg,sb,sa, m);
    if(dst->flags&Fgrey){
        /*
         * We need to do the conversion to grey before the alpha calculation
         * because the draw operator does this, and we need to be operating
         * at the same precision so we get exactly the same answers.
         */
        sk = RGB2K(sr, sg, sb);
        dk = RGB2K(dr, dg, db);
        dk = (sk*m + dk*M)/255;
        dr = dg = db = dk;
        da = (sa*m + da*M)/255;
    }else{
        /*
         * True color alpha calculation treats all channels (including alpha)
```

```
         * the same.  It might have been nice to use an array, but oh well.
         */
        dr = (sr*m + dr*M)/255;
        dg = (sg*m + dg*M)/255;
        db = (sb*m + db*M)/255;
        da = (sa*m + da*M)/255;
    }

    //DBG print("%x %x %x %x\n", dr,dg,db,da);
    putpixel(dst, dp, rgbatopix(dr, dg, db, da));
}
```

⟨lib_graphics/libmemdraw/tests/memdrawtest.c 456⟩≡

```
#include <u.h>
#include <libc.h>
#include <bio.h>
#include <draw.h>
#include <memdraw.h>

⟨function RGB2K((lib_graphics/libmemdraw/tests/memdrawtest.c)) 439b⟩

/*
 * This program tests the 'memimagedraw' primitive stochastically.
 * It tests the combination aspects of it thoroughly, but since the
 * three images it uses are disjoint, it makes no check of the
 * correct behavior when images overlap.  That is, however, much
 * easier to get right and to test.
 */

void drawonepixel(Memimage*, Point, Memimage*, Point, Memimage*, Point);
void verifyone(void);
void verifyline(void);
void verifyrect(void);
void verifyrectrepl(int, int);
void putpixel(Memimage *img, Point pt, ulong nv);
ulong rgbatopix(uchar, uchar, uchar, uchar);

static char *dchan, *schan, *mchan;

⟨global drawdebug((lib_graphics/libmemdraw/tests/memdrawtest.c)) 439c⟩
⟨global seed 439d⟩
⟨global niters 439e⟩
⟨global dbpp 439f⟩
⟨global sbpp 439g⟩
⟨global mbpp 439h⟩
⟨global dpm 439i⟩
⟨global nbytes 439j⟩

⟨global Xrange 439k⟩
⟨global Yrange 439l⟩

⟨global dst 439m⟩
⟨global src 439n⟩
⟨global mask 439o⟩
⟨global stmp 440a⟩
⟨global mtmp 440b⟩
⟨global ones((lib_graphics/libmemdraw/tests/memdrawtest.c)) 440c⟩
⟨global dstbits 440d⟩
⟨global srcbits 440e⟩
⟨global maskbits 440f⟩
```

⟨*global* `savedstbits` 440g⟩

⟨*function* `main`*(*`(lib_graphics/libmemdraw/tests/memdrawtest.c)`*)* 440h⟩

⟨*function* `Bprintr5g6b5` 441⟩

⟨*function* `Bprintr5g5b5a1` 442a⟩

⟨*function* `dumpimage` 442b⟩

⟨*function* `checkone` 443⟩

⟨*function* `RECTPTS` 444a⟩
⟨*function* `checkline` 444b⟩

⟨*function* `fill` 444c⟩

⟨*function* `verifyonemask` 445⟩

⟨*function* `verifyone` 446a⟩

⟨*function* `verifylinemask` 446b⟩

⟨*function* `verifyline` 447a⟩

⟨*function* `verifyrectmask` 447b⟩

⟨*function* `verifyrect` 448a⟩

⟨*function* `randrect` 448b⟩

⟨*function* `tilexy` 448c⟩

⟨*function* `replicate` 449⟩

⟨*function* `verifyrectmaskrepl` 450⟩

⟨*function* `verifyrectrepl` 451a⟩

```
/*
 * Trivial draw implementation.
 * Color values are passed around as ulongs containing αα RRGGBB
 */
```

⟨*function* `replbits` 451b⟩

⟨*function* `pixtorgba` 451c⟩

⟨*function* `rgbatopix` 452a⟩

⟨*function* `getpixel` 452b⟩

⟨*function* `getgrey` 453a⟩

⟨*function* `getmask` 453b⟩

⟨*function* `putpixel` 454⟩

⟨*function* `drawonepixel` 455⟩

G.7 lib_graphics/libmemlayer/

G.7.1 lib_graphics/libmemlayer/draw.c

⟨lib_graphics/libmemlayer/draw.c 458a⟩≡

```
#include <u.h>
#include <libc.h>
#include <draw.h>
#include <memdraw.h>
#include <memlayer.h>

⟨macro DBG1 349b⟩
⟨macro DBG 349c⟩

⟨struct Draw 224d⟩

⟨function ldrawop 225a⟩

⟨function memdraw 92e⟩
```

G.7.2 lib_graphics/libmemlayer/lalloc.c

⟨lib_graphics/libmemlayer/lalloc.c 458b⟩≡

```
#include <u.h>
#include <libc.h>
#include <draw.h>
#include <memdraw.h>
#include <memlayer.h>

⟨function memlalloc 207e⟩
```

G.7.3 lib_graphics/libmemlayer/layerop.c

⟨lib_graphics/libmemlayer/layerop.c 458c⟩≡

```
#include <u.h>
#include <libc.h>
#include <draw.h>
#include <memdraw.h>
#include <memlayer.h>

⟨function RECUR 227b⟩

⟨function _layerop 227a⟩

⟨function _memlayerop 225f⟩
```

G.7.4 lib_graphics/libmemlayer/ldelete.c

⟨lib_graphics/libmemlayer/ldelete.c 458d⟩≡

```
#include <u.h>
#include <libc.h>
#include <draw.h>
#include <memdraw.h>
#include <memlayer.h>

⟨function memldelete 209g⟩
```

⟨*function* memlfree 210e⟩

⟨*function* _memlsetclear 214c⟩

G.7.5 lib_graphics/libmemlayer/lhide.c

⟨lib_graphics/libmemlayer/lhide.c 459a⟩≡

```
#include <u.h>
#include <libc.h>
#include <draw.h>
#include <memdraw.h>
#include <memlayer.h>
```

⟨*function* lhideop 216c⟩

⟨*function* memlhide 216b⟩

⟨*function* lexposeop 217b⟩

⟨*function* memlexpose 217a⟩

G.7.6 lib_graphics/libmemlayer/line.c

⟨lib_graphics/libmemlayer/line.c 459b⟩≡

```
#include <u.h>
#include <libc.h>
#include <draw.h>
#include <memdraw.h>
#include <memlayer.h>
```

⟨*struct* Lline 229b⟩

```
static void llineop(Memimage*, Rectangle, Rectangle, void*, int);
```

⟨*function* _memline 115a⟩

⟨*function* llineop 229d⟩

⟨*function* memline 114e⟩

G.7.7 lib_graphics/libmemlayer/load.c

⟨lib_graphics/libmemlayer/load.c 459c⟩≡

```
#include <u.h>
#include <libc.h>
#include <draw.h>
#include <memdraw.h>
#include <memlayer.h>
```

⟨*function* memload 175c⟩

G.7.8 lib_graphics/libmemlayer/lorigin.c

⟨lib_graphics/libmemlayer/lorigin.c 459d⟩≡

```
#include <u.h>
#include <libc.h>
```

```
#include <draw.h>
#include <memdraw.h>
#include <memlayer.h>

⟨function memlorigin 219b⟩

⟨function memlnorefresh 309a⟩
```

G.7.9 lib_graphics/libmemlayer/lsetrefresh.c

```
⟨lib_graphics/libmemlayer/lsetrefresh.c 460a⟩≡
#include <u.h>
#include <libc.h>
#include <draw.h>
#include <memdraw.h>
#include <memlayer.h>

⟨function memlsetrefresh 310e⟩
```

G.7.10 lib_graphics/libmemlayer/ltofront.c

```
⟨lib_graphics/libmemlayer/ltofront.c 460b⟩≡
#include <u.h>
#include <libc.h>
#include <draw.h>
#include <memdraw.h>
#include <memlayer.h>

⟨function _memltofront 213g⟩

⟨function _memltofrontfill 215a⟩

⟨function memltofront 213f⟩

⟨function memltofrontn 213d⟩
```

G.7.11 lib_graphics/libmemlayer/ltorear.c

```
⟨lib_graphics/libmemlayer/ltorear.c 460c⟩≡
#include <u.h>
#include <libc.h>
#include <draw.h>
#include <memdraw.h>
#include <memlayer.h>

⟨function _memltorear 215d⟩

⟨function memltorear 215c⟩

⟨function memltorearn 215b⟩
```

G.7.12 lib_graphics/libmemlayer/unload.c

```
⟨lib_graphics/libmemlayer/unload.c 460d⟩≡
#include <u.h>
#include <libc.h>
```

```
#include <draw.h>
#include <memdraw.h>
#include <memlayer.h>
```

⟨*function* memunload 178g⟩

G.8 windows/apps/

G.8.1 windows/apps/clock.c

⟨*function* circlept 461a⟩≡ (462c)

```
static Point
circlept(Point c, int r, int degrees)
{

    double rad;
    rad = (double) degrees * PI/180.0;
    c.x += cos(rad)*r;
    c.y -= sin(rad)*r;
    return c;
}
```

⟨*function* redraw 461b⟩≡ (462c)

```
static void
redraw(Image *view)
{
    static int tm, ntm;
    static Rectangle r;
    static Point c;
    static int rad;
    static Image *im;
    int i;
    int anghr, angmin;
    static Tm tms;
    static Tm ntms;

    ntm = time(0);
    if(ntm == tm && eqrect(view->r, r))
        return;

    ntms = *localtime(ntm);
    anghr = 90-(ntms.hour*5 + ntms.min/12)*6;
    angmin = 90-ntms.min*6;
    tm = ntm;
    tms = ntms;
    r = view->r;
    c = divpt(addpt(r.min, r.max), 2);
    rad = Dx(r) < Dy(r) ? Dx(r) : Dy(r);
    rad /= 2;
    rad -= 8;

    draw(view, view->r, back, nil, ZP);
    for(i=0; i<12; i++)
        fillellipse(view, circlept(c, rad, i*(360/12)), 2, 2, dots, ZP);

    line(view, c, circlept(c, (rad*3)/4, angmin), 0, 0, 1, minhand, ZP);
    line(view, c, circlept(c, rad/2, anghr), 0, 0, 1, hrhand, ZP);
```

```
    flushimage(display, true);
}
```

⟨*function* eresized 462a⟩≡ (462c)

```
void
eresized(bool new)
{
    if(new && getwindow(display, Refnone) < 0)
        fprint(2,"can't reattach to window");
    redraw(view);
}
```

⟨*function main (windows/apps/clock.c)* 462b⟩≡ (462c)

```
void
main(int, char**)
{
    Event e;
    Mouse m;
    Menu menu;
    char *mstr[] = {"exit", 0};
    int key, timer;
    int t;

    if (initdraw(nil, nil, "clock") < 0)
        sysfatal("initdraw failed");
    back = allocimagemix(display, DPalebluegreen, DWhite);

    hrhand  = allocimage(display, Rect(0,0,1,1), CMAP8, 1, DDarkblue);
    minhand = allocimage(display, Rect(0,0,1,1), CMAP8, 1, DPaleblue);
    dots    = allocimage(display, Rect(0,0,1,1), CMAP8, 1, DBlue);

    redraw(view);

    einit(Emouse);
    t = (30*1000);
    timer = etimer(0, t);

    menu.item = mstr;
    menu.lasthit = 0;
    // the event loop
    for(;;) {
        key = event(&e);
        if(key == Emouse) {
            m = e.mouse;
            if(m.buttons & 4) {
                if(emenuhit(3, &m, &menu) == 0)
                    exits(nil);
            }
        } else if(key == timer) {
            redraw(view);
        }
    }
}
```

⟨windows/apps/clock.c 462c⟩≡

```
#include <u.h>
#include <libc.h>

#include <draw.h>
#include <window.h>
```

```
#include <event.h>

static Image *hrhand, *minhand;
static Image *dots, *back;

⟨function circlept 461a⟩

⟨function redraw 461b⟩

⟨function eresized 462a⟩

⟨function main (windows/apps/clock.c) 462b⟩
```

G.8.2 windows/apps/colors.c

⟨*global* pixel 463a⟩≡ (466a)

```
static Image *pixel;
```

⟨*global* crect 463b⟩≡ (466a)

```
static Rectangle crect[256];
```

⟨*global* color 463c⟩≡ (466a)

```
static Image *color[256];
```

⟨*function eresized (windows/apps/colors.c)* 463d⟩≡ (466a)

```
void
eresized(bool new)
{
    int x, y, i, n, nx, ny;
    Rectangle r, b;

    if(new && getwindow(display, Refnone) < 0){
        fprint(2, "colors: can't reattach to window: %r\n");
        exits("resized");
    }
    if(view->depth > 8){
        n = 256;
        nx = 16;
    }else{
        n = 1<<view->depth;
        nx = 1<<(view->depth/2);
    }

    ny = n/nx;
    draw(view, view->r, display->white, nil, ZP);
    r = insetrect(view->r, 5);
    r.min.y+=20;
    b.max.y=r.min.y;
    for(i=n-1, y=0; y!=ny; y++){
        b.min.y=b.max.y;
        b.max.y=r.min.y+(r.max.y-r.min.y)*(y+1)/ny;
        b.max.x=r.min.x;
        for(x=0; x!=nx; x++, --i){
            b.min.x=b.max.x;
            b.max.x=r.min.x+(r.max.x-r.min.x)*(x+1)/nx;
            crect[i]=insetrect(b, 1);
            draw(view, crect[i], color[i], nil, ZP);
        }
    }
```

```
    flushimage(display, true);
}
```

⟨*global* buttons 464a⟩≡ (466a)

```
static char *menubuttons[] =
{
    "exit",
    0
};
```

⟨*global* menu 464b⟩≡ (466a)

```
static Menu menu =
{
    menubuttons
};
```

⟨*function* grey 464c⟩≡ (466a)

```
static ulong
grey(int i)
{
    if(i < 0)
        return grey(0);
    if(i > 255)
        return grey(255);
    return (i<<16)+(i<<8)+i;
}
```

⟨*global* dither 464d⟩≡ (466a)

```
static int
dither[16] =  {
    0, 8, 2, 10,
    12, 4, 14, 6,
    3, 11, 1, 9,
    15, 7, 13, 5
};
```

⟨*function main (windows/apps/colors.c)* 464e⟩≡ (466a)

```
void
main(int argc, char *argv[])
{
    Point p;
    Mouse m;
    bool ramp;
    int i, j, k, l, n, prev;
    char buf[100];
    char *fmt;
    Image *dark;
    ulong rgb;

    ramp = false;

    fmt = "index %3d r %3lud g %3lud b %3lud 0x%.8luX        ";
    ARGBEGIN{
    default:
        goto Usage;
    case 'x':
        fmt = "index %2luX r %3luX g %3luX b %3luX 0x%.8luX       ";
        break;
    case 'r':
        ramp = true;
```

```
    break;
}ARGEND

if(argc){
Usage:
    fprint(STDERR, "Usage: %s [-rx]\n", argv0);
    exits("usage");
}

if(initdraw(nil, nil, "colors") < 0)
    sysfatal("initdraw failed: %r");
einit(Emouse);

for(i=0; i<256; i++){
    if(ramp){
        if(view->chan == CMAP8){
            /* dither the fine grey */
            j = i-(i%17);
            dark = allocimage(display, Rect(0,0,1,1), view->chan, 1, (grey(j)<<8)+0xFF);
            color[i] = allocimage(display, Rect(0,0,4,4), view->chan, 1, (grey(j+17)<<8)+0xFF);
            for(j=0; j<16; j++){
                k = j%4;
                l = j/4;
                if(dither[j] > (i%17))
                    draw(color[i], Rect(k, l, k+1, l+1), dark, nil, ZP);
            }
            freeimage(dark);
        }else
            color[i] = allocimage(display, Rect(0,0,1,1), view->chan, 1, (grey(i)<<8)+0xFF);
    }else
        color[i] = allocimage(display, Rect(0,0,1,1), view->chan, 1,
                                (cmap2rgb(i)<<8)+0xFF);
    if(color[i] == nil)
        sysfatal("can't allocate image: %r");
}

eresized(false);

prev = -1;
for(;;){
    m = emouse();
    switch(m.buttons){
    case 1: // left click
        while(m.buttons){
            if(view->depth > 8)
                n = 256;
            else
                n = 1<<view->depth;
            for(i=0; i!=n; i++)
                if(i!=prev && ptinrect(m.xy, crect[i])){
                    if(ramp)
                        rgb = grey(i);
                    else
                        rgb = cmap2rgb(i);
                    sprint(buf, fmt,
                        i,
                        (rgb>>16)&0xFF,
                        (rgb>>8)&0xFF,
                        rgb&0xFF,
                        (rgb<<8) | 0xFF);
```

```
                        p = addpt(view->r.min, Pt(2,2));
                        draw(view, Rpt(p, addpt(p, stringsize(font, buf))), display->white, nil, p);
                        string(view, p, display->black, ZP, font, buf);
                        prev=i;
                        break;
                    }
                m = emouse();
            }
            break;

        case 4: // right click
            switch(emenuhit(3, &m, &menu)){
            case 0:
                exits(nil);
            }
        }
    }
}
```

⟨windows/apps/colors.c 466a⟩≡

```
#include <u.h>
#include <libc.h>

#include <draw.h>
#include <window.h>
#include <event.h>

static int nbit, npix;
⟨global pixel 463a⟩
⟨global crect 463b⟩
⟨global color 463c⟩

⟨function eresized (windows/apps/colors.c) 463d⟩

⟨global buttons 464a⟩

⟨function grey 464c⟩

⟨global menu 464b⟩

⟨global dither 464d⟩

⟨function main (windows/apps/colors.c) 464e⟩
```

G.8.3 windows/apps/screenlock.c

⟨global pic 466b⟩≡ (471b)

```
static char pic[] = "/lib/bunny.bit";
```

⟨global vgactl 466c⟩≡ (471b)

```
static int vgactl;
```

⟨global debug 466d⟩≡ (471b)

```
static int debug;
```

⟨global doblank 466e⟩≡ (471b)

```
static int doblank;
```

⟨global chatty 466f⟩≡ (471b)

```
static int chatty = 0;
```

⟨*global* user 467a⟩≡ (471b)

```
static char user[256];
```

⟨*function* blankscreen 467b⟩≡ (471b)

```
static void
blankscreen(bool blank)
{
    if(vgactl < 0)
        return;
    seek(vgactl, 0, 0);
    if(fprint(vgactl, blank? "blank": "unblank") < 0)
        fprint(2, "blankscreen: can't blank: %r\n");
}
```

⟨*function error (windows/apps/screenlock.c)* 467c⟩≡ (471b)

```
static void
error(char *fmt, ...)
{
    Fmt f;
    char buf[64];
    va_list arg;

    fmtfdinit(&f, 1, buf, sizeof buf);
    fmtprint(&f, "screenlock: ");
    va_start(arg, fmt);
    fmtvprint(&f, fmt, arg);
    va_end(arg);
    fmtprint(&f, "\n");
    fmtfdflush(&f);
    threadexitsall("fatal error");
}
```

⟨*function usage (windows/apps/screenlock.c)* 467d⟩≡ (471b)

```
static void
usage(void)
{
    fprint(2, "usage: %s\n", argv0);
    exits("usage");
}
```

⟨*function* readfile 467e⟩≡ (471b)

```
static void
readfile(char *name, char *buf, int nbuf, int addnul)
{
    int fd;

    fd = open(name, OREAD);
    if(fd == -1)
        error("%s - can't open: %r", name);
    nbuf = read(fd, buf, nbuf-addnul);
    close(fd);
    if(nbuf == -1)
        error("%s - can't can't read: %r", name);
    if(addnul)
        buf[nbuf] = '\0';
}
```

⟨*function* readline 468a⟩≡ (471b)

```
static void
readline(char *buf, int nbuf)
{
    char c;
    int i;

    i = 0;
    while(i < nbuf-1)
        if(read(0, &c, 1) != 1 || c == '\04' || c == '\177'){
            i = 0;
            break;
        } else if(c == '\n')
            break;
        else if(c == '\b' && i > 0)
            --i;
        else if(c == ('u' & 037))
            i = 0;
        else
            buf[i++] = c;
    buf[i] = '\0';
}
```

⟨*function* checkpassword 468b⟩≡ (471b)

```
static void
checkpassword(void)
{
    int fd, consctl, must;
    char buf[256];
    AuthInfo *ai;
    static int opened;

    must = 1;
    if(!opened){
        fd = open("/dev/cons", OREAD);
        if(fd == -1)
            error("can't open cons: %r");
        dup(fd, 0);
        close(fd);
        fd = open("/dev/cons", OWRITE);
        if(fd == -1)
            error("can't open cons: %r");
        dup(fd, 1);
        dup(1, 2);
        close(fd);
        consctl = open("/dev/consctl", OWRITE);
        if(consctl == -1)
            error("can't open consctl: %r");
        if(write(consctl, "rawon", 5) != 5)
            error("can't turn off echo\n");
        opened = 1;
    }

    for(;;){
        if(chatty || !must)
            fprint(2, "%s's screenlock password: ", user);
        memset(buf, 0, sizeof buf);
        readline(buf, sizeof buf);
        blankscreen(0);
        if(chatty || !must)
```

```
            fprint(2, "\n");
        if(buf[0] == '\0' || buf[0] == '\04'){
            if(must)
                continue;
            error("no password typed");
        }

        /* authenticate */
        ai = auth_userpasswd(user, buf);
        if(ai != nil && ai->cap != nil)
            break;
        auth_freeAI(ai);

        if(chatty || !must)
            fprint(2, "password mismatch\n");
        doblank = 1;
    }
    memset(buf, 0, sizeof buf);
    blankscreen(0);
}
```

⟨*function* blanker 469a⟩≡ (471b)

```
static void
blanker(void *)
{
    int tics;

    tics = 0;
    for(;;){
        if(doblank > 0){
            doblank = 0;
            tics = 10;
        }
        if(tics > 0 && --tics == 0)
            blankscreen(1);
        sleep(1000);
    }
}
```

⟨*function* grabmouse 469b⟩≡ (471b)

```
static void
grabmouse(void*)
{
    int fd, x, y;
    char ibuf[256], obuf[256];

    if(debug)
        return;
    fd = open("/dev/mouse", ORDWR);
    if(fd < 0)
        error("can't open /dev/mouse: %r");

    snprint(obuf, sizeof obuf, "m %d %d",
        view->r.min.x + Dx(view->r)/2,
        view->r.min.y + Dy(view->r)/2);
    while(read(fd, ibuf, sizeof ibuf) > 0){
        ibuf[12] = 0;
        ibuf[24] = 0;
        x = atoi(ibuf+1);
        y = atoi(ibuf+13);
```

```
        if(x != view->r.min.x + Dx(view->r)/2 ||
           y != view->r.min.y + Dy(view->r)/2){
            fprint(fd, "%s", obuf);
            doblank = 1;
        }
    }
}
```

⟨*function* screenstring 470a⟩≡ (471b)

```
/* lay down text at 'p' */
static void
screenstring(Point p, char *s)
{
    string(view, p, view->display->white, ZP, font, s);
    flushimage(display, 1);
}
```

⟨*function* lockscreen 470b⟩≡ (471b)

```
static void
lockscreen(void)
{
    enum { Nfld = 5, Fldlen = 12, Cursorlen = 2*4 + 2*2*16, };
    char *s;
    char buf[Nfld*Fldlen], *flds[Nfld], newcmd[128], cbuf[Cursorlen];
    int fd, dx, dy;
    Image *i;
    Point p;
    Rectangle r;
    Tm *tm;

    fd = open("/dev/screen", OREAD);
    if(fd < 0)
        error("can't open /dev/screen: %r");
    if(read(fd, buf, Nfld*Fldlen) != Nfld*Fldlen)
        error("can't read /dev/screen: %r");
    close(fd);
    buf[sizeof buf-1] = 0;
    if(tokenize(buf, flds, Nfld) != Nfld)
        error("can't tokenize /dev/screen header");
    snprint(newcmd, sizeof newcmd, "-r %s %s %d %d",
        flds[1], flds[2], atoi(flds[3]) - 1, atoi(flds[4]) - 1);
    newwindow(newcmd);
    if (initdraw(nil, nil, "screenlock") < 0)
        sysfatal("initdraw failed");
    if(display == nil)
        error("no display");

    /* screen is now open and covered.  grab mouse and hold on tight */
    procrfork(grabmouse, nil, 4096, RFFDG);
    procrfork(blanker, nil, 4096, RFFDG);
    fd = open(pic, OREAD);
    if(fd > 0){
        i = readimage(display, fd, 0);
        if(i){
    r = view->r;
            p = Pt(r.max.x / 2, r.max.y * 2 / 3);
            dx = (Dx(view->r) - Dx(i->r)) / 2;
            r.min.x += dx;
            r.max.x -= dx;
            dy = (Dy(view->r) - Dy(i->r)) / 2;
```

```
            r.min.y += dy;
            r.max.y -= dy;
            draw(view, view->r, display->black, nil, ZP);
            draw(view, r, i, nil, i->r.min);
            flushimage(display, 1);
        }
        close(fd);

        /* identify the user on screen, centered */
        tm = localtime(time(0));
        s = smprint("user %s at %d:%02.2d", getuser(), tm->hour, tm->min);
        p = subpt(p, Pt(stringwidth(font, "m") * strlen(s) / 2, 0));
        screenstring(p, s);
    }

    /* clear the cursor */
    fd = open("/dev/cursor", OWRITE);
    if(fd > 0){
        memset(cbuf, 0, sizeof cbuf);
        write(fd, cbuf, sizeof cbuf);
        /* leave it open */
    }
}
```

⟨*function threadmain (windows/apps/screenlock.c)* 471a⟩≡ (471b)

```
void
threadmain(int argc, char *argv[])
{
    readfile("#c/user", user, sizeof user, 1);

    if((vgactl = open("/dev/vgactl", OWRITE)) < 0)
        vgactl = open("#v/vgactl", OWRITE);

    ARGBEGIN{
    case 'd':
        debug++;
        break;
    default:
        usage();
    }ARGEND

    if(argc != 0)
        usage();

    doblank = 1;
    lockscreen();
    checkpassword();
    threadexitsall(nil);
}
```

⟨windows/apps/screenlock.c 471b⟩≡

```
/* screenlock - lock a terminal */
#include <u.h>
#include <libc.h>
#include <libsec.h>
#include <draw.h>
#include <window.h>
#include <thread.h>
#include <auth.h>
```

⟨*global* pic 466b⟩

⟨*global* vgactl 466c⟩
⟨*global* debug 466d⟩
⟨*global* doblank 466e⟩
⟨*global* chatty 466f⟩

⟨*global* user 467a⟩

⟨*function* blankscreen 467b⟩

⟨*function error (windows/apps/screenlock.c)* 467c⟩

⟨*function usage (windows/apps/screenlock.c)* 467d⟩

⟨*function* readfile 467e⟩

⟨*function* readline 468a⟩

⟨*function* checkpassword 468b⟩

⟨*function* blanker 469a⟩

⟨*function* grabmouse 469b⟩

⟨*function* screenstring 470a⟩

⟨*function* lockscreen 470b⟩

⟨*function threadmain (windows/apps/screenlock.c)* 471a⟩

G.9 kernel/devices/screen/

G.9.1 kernel/devices/screen/portscreen.c

⟨kernel/devices/screen/portscreen.c 472⟩≡

```
#include    "u.h"
#include    "../port/lib.h"
#include    "../port/error.h"
#include    "mem.h"
#include    "dat.h"
#include    "fns.h"

#include    <draw.h>
#include    <memdraw.h>
#include    <memlayer.h>
#include    <cursor.h>

// Many of the stuff below used to be in vgascreen.c, but they are
// quite VGA independent so better to have a generic portscreen.c.
```

⟨*global* gscreen 39d⟩
⟨*global* gscreendata 39e⟩
⟨*global* physgscreenr 368c⟩

```
// for software cursor look swcursor.c
// for software console look swconsole.c
```

G.9.2 kernel/devices/screen/portscreen.h

⟨kernel/devices/screen/portscreen.h 473⟩≡

```
// The content of this file used to be in pc/screen.h, but many prototypes were
// VGA independent, so it is better to have a generic portscreen.h interface
// and VGA-specific stuff in pc/screen.h in a separate file.

// Image

// portscreen.c, set in <arch>/screen.c (used by devmouse.c, swcursor.c)
extern Memimage* gscreen;
extern Memdata   gscreendata;
extern Rectangle physgscreenr;  /* actual monitor size */

/* <arch>/screen.c  (used by devdraw.c) */
extern byte* arch_attachscreen(Rectangle*, ulong*, int*, int*, int*);
extern void  arch_flushmemscreen(Rectangle);
extern void  arch_blankscreen(bool);
extern void  arch_getcolor(ulong, ulong*, ulong*, ulong*);
extern int  arch_setcolor(ulong, ulong, ulong, ulong);
extern bool  arch_ishwimage(Memimage*);

// Cursor

⟨struct Cursorinfo 187b⟩
typedef struct Cursorinfo Cursorinfo;

⟨global signature cursor 187c⟩

// swcursor.c (called from <arch>/screen.c from arch_cursorxxx)
extern void swcursor_init(void);
extern void swcursor_hide(void);
extern void swcursor_draw(void);
extern int  swcursor_move(Point p);
extern void swcursor_load(Cursor *curs);
extern void swcursor_avoid(Rectangle);
//extern Cursor swcursor_arrow;

/* <arch>/screen.c (needed by devmouse.c) */
extern Cursor   arch_arrow;

/* devmouse.c (needed by ??) */ // just enough Mouse getters/setters
extern Point  mousexy(void);

// <arch>/screen.c (called from devmouse.c)
extern void  arch_ksetcursor(Cursor*);
extern int   arch_cursoron(int);
extern void  arch_cursoroff(int);

// Graphical console

// swconsole.c (used from <arch>/screen.c)
extern Memsubfont *swconsole_memdefont;
extern Memimage *swconsole_conscol;
extern Point     swconsole_curpos;
extern Rectangle swconsole_window;
```

```
extern void swconsole_init(void);
extern void swconsole_screenputs(char *s, int n);
```

G.9.3 kernel/devices/screen/swcursor.c

⟨kernel/devices/screen/swcursor.c 474⟩≡

```
/*
 * Software cursor
 */
#include    "u.h"
#include    "../port/lib.h"
#include    "../port/error.h"
#include    "mem.h"
#include    "dat.h"
#include    "fns.h"

#include    <draw.h>
#include    <memdraw.h>
#include    <memlayer.h>
#include    <cursor.h>

#include    "../port/portscreen.h"

⟨global swcursor_arrow 187a⟩

⟨global swback 187d⟩
⟨global swimg1 187e⟩
⟨global swmask1 187f⟩

⟨global swpt 187g⟩
⟨global swvispt 187h⟩
⟨global swvers 187i⟩
⟨global swvisvers 187j⟩

⟨global swenabled 187k⟩
⟨global swvisible 187l⟩

⟨global swimg 189a⟩
⟨global swmask 189b⟩

⟨global swrect 190a⟩

⟨global swoffset 193a⟩


⟨function swcursorclock 189e⟩

⟨function swcursorinit 188f⟩

⟨function swcursordraw 190b⟩

⟨function swcursorhide 190c⟩

⟨function swcursoravoid 191a⟩

⟨function swload 192c⟩

⟨function swmove 193b⟩
```

G.9.4 kernel/devices/screen/swconsole.c

⟨kernel/devices/screen/swconsole.c 475⟩≡

```
/*
 * Text output using memdraw
 */
#include    "u.h"
#include    "../port/lib.h"
#include    "../port/error.h"
#include    "mem.h"
#include    "dat.h"
#include    "fns.h"

#include    <draw.h>
#include    <font.h>
#include    <memdraw.h>
#include    <memlayer.h>
#include    <cursor.h>

#include    "../port/portscreen.h"

// The main entry point of this module is the function
// swconsole_screenputs() which is assigned to screenputs in devcons.c and
// called from print() to display text on the screen.
// swconsole_screenputs() uses the default text font in memdraw
// and the memdraw functions to draw characters on the screen (via gscreen).
// See pc/cga.c for another screenputs function using the CGA hardware
// to display text.

enum {
    Scroll      = 8, // want this to be configurable?
    Tabstop     = 4,
};

⟨global swconsole_screenlock 301h⟩

⟨global curpos 301a⟩

⟨global window bis 301b⟩

⟨global conscol 301c⟩

⟨global back2 301d⟩

⟨global memdefont2 301e⟩

⟨global swconsole h w 301g⟩

⟨function swconsole_init 301f⟩

⟨function swconsole_scroll 301i⟩

⟨function swconsole_screenputc 302⟩

⟨function swconsole_screenputs 303a⟩
```

G.9.5 kernel/devices/screen/devdraw.h

⟨kernel/devices/screen/devdraw.h 476⟩≡

```
typedef struct Client Client;
typedef struct KDraw KDraw;
typedef struct DImage DImage;
typedef struct DName DName;
typedef struct DScreen DScreen;
typedef struct CScreen CScreen;
typedef struct FChar FChar;
typedef struct Refresh Refresh;
typedef struct Refx Refx;

⟨constant NHASH bis 35e⟩
⟨constant HASHMASK 35f⟩


⟨struct KDraw 36a⟩

⟨struct Client 35d⟩


⟨struct DName 86d⟩


⟨struct DImage 39f⟩

⟨struct FChar 139c⟩



⟨struct CScreen 198c⟩

⟨struct DScreen 197b⟩


⟨struct Refresh 306d⟩

⟨struct Refx 308g⟩

// drawerror.c
extern char Enodrawimage[];
extern char Enodrawscreen[];
extern char Eshortdraw[];
extern char Eshortread[];
extern char Eimageexists[];
extern char Escreenexists[];
extern char Edrawmem[];
extern char Ereadoutside[];
extern char Ewriteoutside[];
extern char Enotfont[];
extern char Eindex[];
extern char Enoclient[];
extern char Enameused[];
extern char Enoname[];
extern char Eoldname[];
extern char Enamed[];
extern char Ewrongname[];
```

```
//drawinit.c
extern Memimage    *screenimage;
error0 initscreenimage(void);

//devdraw.c
extern KDraw        sdraw;
void dlock(void);
int candlock(void);
void dunlock(void);
DImage* drawlookup(Client *client, int id, bool checkname);
void addflush(Rectangle r);
void dstflush(int dstid, Memimage *dst, Rectangle r);
void drawflush(void);

//drawalloc.c
DImage* allocdimage(Memimage *i);
void drawfreedimage(DImage *dimage);
Memimage* drawinstall(Client *client, int id, Memimage *i, DScreen *dscreen);
void drawuninstall(Client *client, int id);

//drawname.c
extern char    screenname[];
DName* drawlookupname(int n, char *str);
bool drawgoodname(DImage *d);
void drawaddname(Client *client, DImage *di, int n, char *str);
void drawdelname(DName *name);

//drawwindow.c
DScreen*   drawlookupscreen(Client *client, int id, CScreen **cs);
Memscreen* drawinstallscreen(Client*, DScreen*, int, DImage *, DImage *, bool);
void       drawuninstallscreen(Client*, CScreen*);
DScreen* drawlookupdscreen(int id);
void drawfreedscreen(DScreen*);

//drawmisc.c
int drawrefactive(void *a);
void drawrefresh(Memimage*, Rectangle r, void *v);
void drawrefreshscreen(DImage *l, Client *client);
```

G.9.6 kernel/devices/screen/drawinit.c

⟨kernel/devices/screen/drawinit.c 477⟩≡

```
#include    "u.h"
#include    "../port/lib.h"
#include    "../port/error.h"
#include    "mem.h"
#include    "dat.h"
#include    "fns.h"

#include    <draw.h>
#include    <memdraw.h>
#include    <memlayer.h>

#include    <cursor.h>
#include    "portscreen.h"

#include    "devdraw.h"
```

⟨*global* screenimage 39c⟩
⟨*global* screendimage 50a⟩

⟨*global* screenname 234g⟩
⟨*global* screennameid 234i⟩

⟨*function* makescreenimage 50b⟩

⟨*function* initscreenimage 49g⟩

⟨*function* deletescreenimage 371c⟩

⟨*function* resetscreenimage 371b⟩

G.9.7 kernel/devices/screen/drawname.c

⟨kernel/devices/screen/drawname.c 478a⟩≡

```
#include    "u.h"
#include    "../port/lib.h"
#include    "../port/error.h"
#include    "mem.h"
#include    "dat.h"
#include    "fns.h"

#include    <draw.h>
#include    <memdraw.h>
#include    <memlayer.h>

#include    "devdraw.h"
```

⟨*function* drawcmp 87a⟩

⟨*function* drawlookupname 86e⟩

⟨*function* drawgoodname 88h⟩

⟨*function* drawdelname 87d⟩

⟨*function* drawaddname 87b⟩

G.9.8 kernel/devices/screen/drawalloc.c

⟨kernel/devices/screen/drawalloc.c 478b⟩≡

```
#include    "u.h"
#include    "../port/lib.h"
#include    "../port/error.h"
#include    "mem.h"
#include    "dat.h"
#include    "fns.h"

#include    <draw.h>
#include    <memdraw.h>
#include    <memlayer.h>

#include    <cursor.h>
```

```
#include    "portscreen.h"

#include    "devdraw.h"

⟨function allocdimage 39g⟩

⟨function drawinstall 77e⟩

⟨function drawfreedimage 78c⟩

⟨function drawuninstall 78a⟩
```

G.9.9 kernel/devices/screen/drawwindow.c

⟨kernel/devices/screen/drawwindow.c 479a⟩≡

```
#include    "u.h"
#include    "../port/lib.h"
#include    "../port/error.h"
#include    "mem.h"
#include    "dat.h"
#include    "fns.h"

#include    <draw.h>
#include    <memdraw.h>
#include    <memlayer.h>

#include    "devdraw.h"

⟨global dscreen 197d⟩

⟨function drawlookupdscreen 198a⟩

⟨function drawlookupscreen 198d⟩

⟨function drawinstallscreen 203i⟩

⟨function drawfreedscreen 205d⟩

⟨function drawuninstallscreen 205c⟩
```

G.9.10 kernel/devices/screen/drawerror.c

⟨kernel/devices/screen/drawerror.c 479b⟩≡

```
⟨global Enodrawimage 352d⟩
⟨global Enodrawscreen 352e⟩
⟨global Eshortdraw 352f⟩
⟨global Eshortread 352g⟩
⟨global Eimageexists 352h⟩
⟨global Escreenexists 352i⟩
⟨global Edrawmem 353a⟩
⟨global Ereadoutside 353b⟩
⟨global Ewriteoutside 353c⟩
⟨global Enotfont 353d⟩
⟨global Eindex 353e⟩
```

⟨*global* Enoclient 353f⟩
⟨*global* Enameused 353g⟩
⟨*global* Enoname 353h⟩
⟨*global* Eoldname 353i⟩
⟨*global* Enamed 353j⟩
⟨*global* Ewrongname 353k⟩

G.9.11 kernel/devices/screen/drawmesg.c

⟨kernel/devices/screen/drawmesg.c 480a⟩≡

```
#include    "u.h"
#include    "../port/lib.h"
#include    "../port/error.h"
#include    "mem.h"
#include    "dat.h"
#include    "fns.h"

#include    <draw.h>
#include    <marshal.h>
#include    <memdraw.h>
#include    <memlayer.h>

#include    <cursor.h>
#include    "portscreen.h"

#include    "devdraw.h"
```

⟨*function* drawclientop 92c⟩

⟨*function* drawimage 92a⟩

⟨*function* drawrectangle 76b⟩

⟨*function* drawpoint 91f⟩

⟨*function* drawcoord 121b⟩

⟨*function* drawchar 152d⟩

⟨*function* printmesg 350b⟩

⟨*function* drawmesg 71⟩

G.9.12 kernel/devices/screen/drawmisc.c

⟨kernel/devices/screen/drawmisc.c 480b⟩≡

```
#include    "u.h"
#include    "../port/lib.h"
#include    "../port/error.h"
#include    "mem.h"
#include    "dat.h"
#include    "fns.h"

#include    <draw.h>
```

```
#include    <memdraw.h>
#include    <memlayer.h>
#include    <cursor.h>

#include    "portscreen.h"
#include    "devdraw.h"
```

⟨*global* `blanktime` 304e⟩

⟨*function* `drawrefactive` 306c⟩

⟨*function* `drawrefreshscreen` 309e⟩

⟨*function* `drawrefresh` 308e⟩

⟨*function* `drawcmap` 254b⟩

⟨*function* `drawblankscreen` 304c⟩

⟨*function* `drawactive` 305a⟩

⟨*function* `drawidletime` 305b⟩

G.9.13 kernel/devices/screen/devdraw.c

⟨`kernel/devices/screen/devdraw.c` 481⟩≡

```
#include    "u.h"
#include    "../port/lib.h"
#include    "../port/error.h"
#include    "mem.h"
#include    "dat.h"
#include    "fns.h"

#include    <draw.h>
#include    <marshal.h>
#include    <memdraw.h>
#include    <memlayer.h>
#include    <cursor.h>

#include    "portscreen.h"
#include    "devdraw.h"
```

⟨*enum* `QxxxDraw` 61b⟩

⟨*constant* `QSHIFT` 62a⟩

⟨*function QID bis* 62b⟩
⟨*function* `CLIENTPATH` 62c⟩

⟨*constant* `IOUNIT` 65b⟩

⟨*global* `sdraw` 36b⟩
⟨*global* `drawlock` 49c⟩

⟨*global* `flushrect` 81c⟩

⟨*global* waste 328a⟩

```
// forward decls
void        drawmesg(Client*, void*, int);
void        drawuninstall(Client*, int);
void        drawfreedimage(DImage*);
Client*     drawclientofpath(ulong);
DImage*  allocdimage(Memimage*);
```

⟨*function* dlock 49d⟩

⟨*function* candlock 49e⟩

⟨*function* dunlock 49f⟩

⟨*function* drawgen 62g⟩

⟨*function* addflush 327d⟩

⟨*function* dstflush 81e⟩

⟨*function* drawflush 81d⟩

⟨*function* drawlookup 35h⟩

⟨*function* drawnewclient 36d⟩

⟨*function* drawclientofpath 64c⟩

⟨*function* drawclient 66a⟩

⟨*function* drawattach 49b⟩

⟨*function* drawwalk 62d⟩

⟨*function* drawstat 62f⟩

⟨*function* drawopen 65a⟩

⟨*function* drawclose 65e⟩

⟨*function* drawread 66d⟩

⟨*function* drawwakeall 306e⟩

⟨*function* drawwrite 67a⟩

⟨*global* drawdevtab 61a⟩

G.10 kernel/devices/screen/arm/

G.10.1 kernel/devices/screen/arm/screen.c

⟨kernel/devices/screen/arm/screen.c 483⟩≡

```
/*
 * bcm2385 framebuffer
 */
#include "u.h"
#include "../port/lib.h"
#include "mem.h"
#include "dat.h"
#include "fns.h"

#include <draw.h>
#include <font.h>
#include <memdraw.h>
#include <cursor.h>

#include "../port/portscreen.h"

enum {
    Wid     = 1024,
    Ht      = 768,
    Depth       = 16,
};

static Memdata xgdata;

static Memimage xgscreen =
{
    .r = { 0, 0, Wid, Ht },
    .clipr = { 0, 0, Wid, Ht },
    .depth = Depth,
    .nchan = 3,
    .chan = RGB16,
    .cmap = nil,
    .data = &xgdata,
    .zero = 0,
    .width = 0,             /* width in words of a single scan line */
    .layer = nil,
    .flags = 0,
};
```

```
/*
 * Software cursor.
 */

// could factorize with swcursor_arrow
Cursor  arch_arrow = {
    { -1, -1 },
    { 0xFF, 0xFF, 0x80, 0x01, 0x80, 0x02, 0x80, 0x0C,
      0x80, 0x10, 0x80, 0x10, 0x80, 0x08, 0x80, 0x04,
      0x80, 0x02, 0x80, 0x01, 0x80, 0x02, 0x8C, 0x04,
      0x92, 0x08, 0x91, 0x10, 0xA0, 0xA0, 0xC0, 0x40,
    },
    { 0x00, 0x00, 0x7F, 0xFE, 0x7F, 0xFC, 0x7F, 0xF0,
      0x7F, 0xE0, 0x7F, 0xE0, 0x7F, 0xF0, 0x7F, 0xF8,
      0x7F, 0xFC, 0x7F, 0xFE, 0x7F, 0xFC, 0x73, 0xF8,
      0x61, 0xF0, 0x60, 0xE0, 0x40, 0x40, 0x00, 0x00,
    },
};

bool
arch_cursoron(bool dolock)
{
    bool retry;

    if (dolock)
        lock(&cursor);
    if (canqlock(&drawlock)) {
        retry = false;
        swcursor_hide();
        swcursor_draw();
        qunlock(&drawlock);
    } else
        retry = true;
    if (dolock)
        unlock(&cursor);
    return retry;
}

void
arch_cursoroff(bool dolock)
{
    if (dolock)
        lock(&cursor);
    swcursor_hide();
    if (dolock)
        unlock(&cursor);
}


/* called from devmouse */
void
arch_ksetcursor(Cursor* curs)
{
    arch_cursoroff(false);
    swcursor_load(curs);
    arch_cursoron(false);
}

/*
 * Drawing
```

```
 */

int
hwdraw(Memdrawparam *par)
{
    Memimage *dst, *src, *mask;

    if((dst=par->dst) == nil || dst->data == nil)
        return 0;
    if((src=par->src) == nil || src->data == nil)
        return 0;
    if((mask=par->mask) == nil || mask->data == nil)
        return 0;

    if(dst->data->bdata == xgdata.bdata)
        swcursor_avoid(par->r);
    if(src->data->bdata == xgdata.bdata)
        swcursor_avoid(par->sr);
    if(mask->data->bdata == xgdata.bdata)
        swcursor_avoid(par->mr);

    return 0;
}

static bool
screensize(void)
{
    char *p;
    char *f[3];
    int width, height, depth;

    p = getconf("vgasize");
    if(p == nil || getfields(p, f, nelem(f), 0, "x") != nelem(f) ||
        (width = atoi(f[0])) < 16 ||
        (height = atoi(f[1])) <= 0 ||
        (depth = atoi(f[2])) <= 0)
        return false;
    xgscreen.r.max = Pt(width, height);
    xgscreen.depth = depth;
    return true;
}

static void
screenwin(void)
{
    char *greet;
    Memimage *orange;
    Point p, q;
    int h;

    orange = allocmemimage(Rect(0, 0, 1, 1), RGB16);
    orange->flags |= Frepl;
    orange->clipr = gscreen->r;
    orange->data->bdata[0] = 0x40;      /* magic: colour? */
    orange->data->bdata[1] = 0xfd;      /* magic: colour? */

    h = swconsole_memdefont->height;

    memimagedraw(gscreen,
                 Rect(swconsole_window.min.x, swconsole_window.min.y,
```

```
                    swconsole_window.max.x, swconsole_window.min.y + h + 5 + 6),
                orange, ZP, nil, ZP, S);
    freememimage(orange);

    swconsole_window = insetrect(swconsole_window, 5);

    greet = " Plan 9 Console ";
    p = addpt(swconsole_window.min, Pt(10, 0));
    q = memsubfontwidth(swconsole_memdefont, greet);
    memimagestring(gscreen, p, swconsole_conscol, ZP, swconsole_memdefont,
                   greet);

    arch_flushmemscreen(gscreen->r); // was r before, but now in swconsole.c

    swconsole_window.min.y += h + 6;
    swconsole_curpos = swconsole_window.min;
    swconsole_window.max.y =
      swconsole_window.min.y +
      ((swconsole_window.max.y - swconsole_window.min.y) / h) * h;
}


void
arch__screeninit(void)
{
    uchar *fb;
    int set;
    ulong chan;

    set = screensize();
    fb = fbinit(set, &xgscreen.r.max.x, &xgscreen.r.max.y, &xgscreen.depth);
    if(fb == nil){
        print("can't initialise %dx%dx%d framebuffer \n",
            xgscreen.r.max.x, xgscreen.r.max.y, xgscreen.depth);
        return;
    }
    xgscreen.clipr = xgscreen.r;
    switch(xgscreen.depth){
    default:
        print("unsupported screen depth %d\n", xgscreen.depth);
        xgscreen.depth = 16;
        /* fall through */
    case 16:
        chan = RGB16;
        break;
    case 24:
        chan = BGR24;
        break;
    case 32:
        chan = ARGB32;
        break;
    }
    memsetchan(&xgscreen, chan);
    //conf.monitor = 1;
    xgdata.bdata = fb;
    xgdata.ref = 1;

    // set portscreen.c globals (used by draw.c, swcursor.c, swconsole
    gscreen = &xgscreen;
    gscreen->width = wordsperline(gscreen->r, gscreen->depth);
```

```
    memimageinit();
    swconsole_init();
    screenwin();

    screenputs = swconsole_screenputs;
}

uchar*
arch_attachscreen(Rectangle *r, ulong *chan, int* d, int *width, bool *softscreen)
{
    *r = gscreen->r;
    *d = gscreen->depth;
    *chan = gscreen->chan;
    *width = gscreen->width;
    *softscreen = false;

    return gscreen->data->bdata;
}

void
arch_blankscreen(bool blank)
{
    fbblank(blank);
}

void arch_flushmemscreen(Rectangle) { }
//old: #define ishwimage(i) 1       /* for ../port/devdraw.c */
bool arch_ishwimage(Memimage*) { return true; }
void arch_getcolor(ulong, ulong*, ulong*, ulong*) { }
int  arch_setcolor(ulong, ulong, ulong, ulong) { return 0; }
```

G.11 kernel/devices/screen/386/

G.11.1 kernel/devices/screen/386/screen.h

⟨kernel/devices/screen/386/screen.h 487⟩≡

⟨*enum* vgaports*(x86)* 363f⟩

⟨*enum* vgamisc*(x86)* 363c⟩

⟨*function* VGAMEM*(x86)* 362c⟩

⟨*function* vgao*(x86)* 364a⟩

```
//#define vgai(port)    inb(port)
extern int vgaxi(long, uchar);
extern int vgaxo(long, uchar, uchar);

// forward decl
typedef struct VGAdev VGAdev;
typedef struct VGAcur VGAcur;
typedef struct VGAscr VGAscr;
```

⟨*struct* VGAdev*(x86)* 365b⟩

⟨*struct* VGAcur*(x86)* 188b⟩

⟨*struct* VGAscr*(x86)* 362a⟩

```
//!!!
extern VGAscr vgascreen;
```

⟨*enum* _anon_ (kernel/devices/screen/386/screen.h)2 363d⟩

```
/* screen.c */
extern bool    hwaccel;  /* use hw acceleration; default on */
extern bool    hwblank;  /* use hw blanking; default on */
extern bool    panning;  /* use virtual screen panning; default off */

extern void  addvgaseg(char*, ulong, ulong);

extern int  screensize(int, int, int, ulong);
extern int  screenaperture(int, int);

/* devdraw.c */
extern void deletescreenimage(void);
extern void resetscreenimage(void);
extern int drawidletime(void);
extern void drawblankscreen(int);
extern ulong  blanktime;
extern QLock  drawlock;
//extern int drawhasclients(void);
//extern void setscreenimageclipr(Rectangle);
//extern void drawflush(void);

/* vga.c */
extern void vgascreenwin(VGAscr*);
extern void vgaimageinit(ulong);
extern void vgalinearpciid(VGAscr*, int, int);
extern void vgalinearaddr(VGAscr*, ulong, int);
extern void vgablank(VGAscr*, int);
extern Lock vgascreenlock;

void swcursorinit_wrapper(void);

// software cursors
extern VGAcur swcursor;
```

G.11.2 kernel/devices/screen/386/screen.c

⟨*global* curpos*(x86)* 488a⟩≡ (492)

```
static Point curpos;
```

⟨*global window bis(x86)* 488b⟩≡ (492)

```
static Rectangle window;
```

⟨screensize() *vga settings* 488c⟩+≡ (46a) ◁255a 491d▷

```
scr->memdefont = getmemdefont();
```

⟨*function* vgascreenwin*(x86)* 488d⟩≡ (492)

```
void
vgascreenwin(VGAscr* scr)
{
    int h, w;
```

```
    h = scr->memdefont->height;
    w = scr->memdefont->info[' '].width;

    window = insetrect(gscreen->r, 48);
    window.max.x = window.min.x+((window.max.x-window.min.x)/w)*w;
    window.max.y = window.min.y+((window.max.y-window.min.y)/h)*h;
    curpos = window.min;

    screenputs = vgascreenputs;
}
```

⟨*function* vgascreenputs*(x86)* 489⟩≡ (492)

```
static void
vgascreenputs(char* s, int n)
{
    int i;
    bool gotdraw;
    Rune r;
    char buf[4];
    VGAscr *scr;
    Rectangle flushr;

    scr = &vgascreen;

    if(!arch_islo()){
        /*
         * Don't deadlock trying to
         * print in an interrupt.
         */
        if(!canlock(&vgascreenlock))
            return;
    }
    else
        lock(&vgascreenlock);

    /*
     * Be nice to hold this, but not going to deadlock
     * waiting for it.  Just try and see.
     */
    gotdraw = canqlock(&drawlock);

    flushr = Rect(10000, 10000, -10000, -10000);

    while(n > 0){
        i = chartorune(&r, s);
        if(i == 0){
            s++;
            --n;
            continue;
        }
        memmove(buf, s, i);
        buf[i] = 0;
        n -= i;
        s += i;
        vgascreenputc(scr, buf, &flushr);
    }
    arch_flushmemscreen(flushr);

    if(gotdraw)
        qunlock(&drawlock);
```

```
    unlock(&vgascreenlock);
}
```

⟨*global* xbuf *(x86)* 490a⟩≡ (492)

```
static int xbuf[256];
```

⟨*global* xp *(x86)* 490b⟩≡ (492)

```
static int *xp;
```

⟨*function* vgascreenputc *(x86)* 490c⟩≡ (492)

```
static void
vgascreenputc(VGAscr* scr, char* buf, Rectangle *flushr)
{
    Point p;
    int h, w, pos;
    Rectangle r;

    if(xp < xbuf || xp >= &xbuf[sizeof(xbuf)])
        xp = xbuf;

    h = scr->memdefont->height;
    switch(buf[0]){

    case '\n':
        if(curpos.y+h >= window.max.y){
            vgascroll(scr);
            *flushr = window;
        }
        curpos.y += h;
        vgascreenputc(scr, "\r", flushr);
        break;

    case '\r':
        xp = xbuf;
        curpos.x = window.min.x;
        break;

    case '\t':
        p = memsubfontwidth(scr->memdefont, " ");
        w = p.x;
        if(curpos.x >= window.max.x-4*w)
            vgascreenputc(scr, "\n", flushr);

        pos = (curpos.x-window.min.x)/w;
        pos = 4-(pos%4);
        *xp++ = curpos.x;
        r = Rect(curpos.x, curpos.y, curpos.x+pos*w, curpos.y + h);
        memimagedraw(gscreen, r, back, back->r.min, nil, back->r.min, S);
        curpos.x += pos*w;
        break;

    case '\b':
        if(xp <= xbuf)
            break;
        xp--;
        r = Rect(*xp, curpos.y, curpos.x, curpos.y+h);
        memimagedraw(gscreen, r, back, back->r.min, nil, ZP, S);
        combinerect(flushr, r);
        curpos.x = *xp;
        break;
```

```
    case '\0':
        break;

    default:
        p = memsubfontwidth(scr->memdefont, buf);
        w = p.x;

        if(curpos.x >= window.max.x-w)
            vgascreenputc(scr, "\n", flushr);

        *xp++ = curpos.x;
        r = Rect(curpos.x, curpos.y, curpos.x+w, curpos.y+h);
        memimagedraw(gscreen, r, back, back->r.min, nil, back->r.min, S);
        memimagestring(gscreen, curpos, conscol, ZP, scr->memdefont, buf);
        combinerect(flushr, r);
        curpos.x += w;
    }
}
```

⟨*function* vgascroll*(x86)* 491a⟩≡ (492)

```
static void
vgascroll(VGAscr* scr)
{
    int h, o;
    Point p;
    Rectangle r;

    h = scr->memdefont->height;
    o = 8*h;
    r = Rpt(window.min, Pt(window.max.x, window.max.y-o));
    p = Pt(window.min.x, window.min.y+o);
    memimagedraw(gscreen, r, gscreen, p, nil, p, S);
    r = Rpt(Pt(window.min.x, window.max.y-o), window.max);
    memimagedraw(gscreen, r, back, ZP, nil, ZP, S);

    curpos.y -= o;
}
```

⟨*global* back2*(x86)* 491b⟩≡ (492)

```
static Memimage* back;
```

⟨*global* conscol*(x86)* 491c⟩≡ (492)

```
static Memimage *conscol;
```

⟨screensize() *vga settings* 491d⟩+≡ (46a) ◁488c

```
vgaimageinit(chan);
```

⟨*function* vgaimageinit*(x86)* 491e⟩≡ (492)

```
void
vgaimageinit(channels chan)
{
    if(back == nil){
        back = allocmemimage(Rect(0,0,1,1), chan);  /* RSC BUG */
        if(back == nil)
            panic("back alloc");        /* RSC BUG */
        back->flags |= Frepl;
        back->clipr = Rect(-0x3FFFFFF, -0x3FFFFFF, 0x3FFFFFF, 0x3FFFFFF);
        memfillcolor(back, DBlack);
    }
```

```
    if(conscol == nil){
        conscol = allocmemimage(Rect(0,0,1,1), chan);   /* RSC BUG */
        if(conscol == nil)
            panic("conscol alloc"); /* RSC BUG */
        conscol->flags |= Frepl;
        conscol->clipr = Rect(-0x3FFFFFF, -0x3FFFFFF, 0x3FFFFFF, 0x3FFFFFF);
        memfillcolor(conscol, DWhite);
    }
}
```

⟨kernel/devices/screen/386/screen.c 492⟩≡

```
#include "u.h"
#include "../port/lib.h"
#include "../port/error.h"
#include "mem.h"
#include "dat.h"
#include "fns.h"

#include "io.h"
#include <ureg.h>

#include <draw.h>
#include <font.h>
#include <memdraw.h>
#include <cursor.h>

#include "../port/portscreen.h"
#include "screen.h"

//TODO: can not reuse swcursor.c, WEIRD, kernel fault when run rio!
static void swcursorhide(void);
static void swcursoravoid(Rectangle);
static void swcursordraw(void);
static void swload(VGAscr*, Cursor *curs);
static int swmove(VGAscr*, Point p);
static void swcursorinit(void);

void swcursorinit_wrapper(void);

//-----------------------------------------------------------------------------
// vgax.c
//-----------------------------------------------------------------------------

⟨global vgaxlock(x86) 364b⟩

⟨function vgaxi(x86) 364c⟩

⟨function vgaxo(x86) 365a⟩

//-----------------------------------------------------------------------------
// vga.c
//-----------------------------------------------------------------------------

⟨global back2(x86) 491b⟩
⟨global conscol(x86) 491c⟩

⟨global curpos(x86) 488a⟩
⟨global window bis(x86) 488b⟩
⟨global xp(x86) 490b⟩
```

⟨*global* xbuf*(x86)* 490a⟩
⟨*global* vgascreenlock*(x86)* 46b⟩

⟨*function* vgaimageinit*(x86)* 491e⟩

⟨*function* vgascroll*(x86)* 491a⟩

⟨*function* vgascreenputc*(x86)* 490c⟩

⟨*function* vgascreenputs*(x86)* 489⟩

⟨*function* vgascreenwin*(x86)* 488d⟩

⟨*function* vgablank*(x86)* 306a⟩

⟨*function* addvgaseg*(x86)* 501a⟩

```
//-----------------------------------------------------------------------------
// vgascreen.c
//-----------------------------------------------------------------------------


//#define RGB2K(r,g,b)    ((156763*(r)+307758*(g)+59769*(b))>>19)

extern void vgalinearpci(VGAscr*);


//already in libdraw/arith.c
```

⟨*global* vgascreen*(x86)* 362b⟩

⟨*global* didswcursorinit*(x86)* 188e⟩

⟨*global* softscreen*(x86)* 47e⟩

⟨*function* ishwimage*(x86)* 363b⟩

⟨*function* screensize*(x86)* 46a⟩

⟨*function* screenaperture*(x86)* 372e⟩

⟨*function* attachscreen*(x86)* 54e⟩

⟨*function* flushmemscreen*(x86)* 82b⟩

⟨*function* getcolor*(x86)* 253b⟩

⟨*function* setpalette*(x86)* 255b⟩

⟨*function* setcolor*(x86)* 254c⟩

⟨*function* cursoron*(x86)* 191b⟩

⟨*function* cursoroff*(x86)* 191c⟩

⟨*function* ksetcursor*(x86)* 192b⟩

⟨*global* hwaccel*(x86)* 94c⟩

⟨*global* hwblank*(x86)* 305c⟩
⟨*global* panning*(x86)* 373a⟩

⟨*function* hwdraw*(x86)* 94d⟩

⟨*function blankscreen bis(x86)* 305g⟩

⟨*function* vgalinearpciid*(x86)* 375a⟩

⟨*function* vgalinearpci*(x86)* 375b⟩

⟨*function* vgalinearaddr*(x86)* 376⟩

```
/*
 * Software cursor.
 */
//TODO: reuse swcursor.c (but get kernel fault when run rio, WEIRD)

Cursor  arch_arrow = {
    .offset = { -1, -1 },
    .clr = {
      0xFF, 0xFF, 0x80, 0x01, 0x80, 0x02, 0x80, 0x0C,
      0x80, 0x10, 0x80, 0x10, 0x80, 0x08, 0x80, 0x04,
      0x80, 0x02, 0x80, 0x01, 0x80, 0x02, 0x8C, 0x04,
      0x92, 0x08, 0x91, 0x10, 0xA0, 0xA0, 0xC0, 0x40,
    },
    .set = {
      0x00, 0x00, 0x7F, 0xFE, 0x7F, 0xFC, 0x7F, 0xF0,
      0x7F, 0xE0, 0x7F, 0xE0, 0x7F, 0xF0, 0x7F, 0xF8,
      0x7F, 0xFC, 0x7F, 0xFE, 0x7F, 0xFC, 0x73, 0xF8,
      0x61, 0xF0, 0x60, 0xE0, 0x40, 0x40, 0x00, 0x00,
    },
};

extern bool swvisible;  /* is the cursor visible? */
extern bool swenabled;  /* is the cursor supposed to be on the screen? */
extern Memimage*   swback; /* screen under cursor */
extern Memimage*   swimg;  /* cursor image */
extern Memimage*   swmask; /* cursor mask */
extern Memimage*   swimg1;
extern Memimage*   swmask1;
extern Point   swoffset;
extern Rectangle   swrect; /* screen rectangle in swback */
extern Point   swpt;   /* desired cursor location */
extern Point   swvispt;    /* actual cursor location */
extern int swvers; /* incremented each time cursor image changes */
extern int swvisvers;  /* the version on the screen */


/*
 * called with drawlock locked for us, most of the time.
 * kernel prints at inopportune times might mean we don't
 * hold the lock, but memimagedraw is now reentrant so
 * that should be okay: worst case we get cursor droppings.
 */
static void
swcursorhide(void)
{
    if(!swvisible)
```

```
        return;
    if(swback == nil)
        return;

    swvisible = false;
    // restore what was under the cursor
    memimagedraw(gscreen, swrect, swback, ZP, memopaque, ZP, S);
    arch_flushmemscreen(swrect);
}

static void
swcursoravoid(Rectangle r)
{
    if(swvisible && rectXrect(r, swrect))
        swcursorhide();
}

static void
swcursordraw(void)
{
    if(swvisible)
        return;
    if(!swenabled)
        return;
    if(swback == nil || swimg1 == nil || swmask1 == nil)
        return;
    assert(!canqlock(&drawlock));

    swvispt = swpt;
    swvisvers = swvers;
    // cursor is 16x16 picture
    swrect = rectaddpt(Rect(0,0,16,16), swvispt);
    // save what is under the cursor
    memimagedraw(swback, swback->r, gscreen, swpt, memopaque, ZP, S);
    // draw cursor
    memimagedraw(gscreen, swrect, swimg1, ZP, swmask1, ZP, SoverD);
    arch_flushmemscreen(swrect);
    swvisible = true;
}
```

⟨*function* swenable*(x86)* 191d⟩

⟨*function* swdisable*(x86)* 192a⟩

```
static void
swload(VGAscr*, Cursor *curs)
{
    byte *ip, *mp;
    int i, j, set, clr;

    if(!swimg || !swmask || !swimg1 || !swmask1)
        return;
    /*
     * Build cursor image and mask.
     * Image is just the usual cursor image
     * but mask is a transparent alpha mask.
     *
     * The 16x16x8 memimages do not have
     * padding at the end of their scan lines.
     */
    ip = byteaddr(swimg, ZP);
```

```
    mp = byteaddr(swmask, ZP);

    for(i=0; i<32; i++){
        set = curs->set[i];
        clr = curs->clr[i];
        for(j=0x80; j; j>>=1){
            *ip++ = set & j ? 0x00 : 0xFF;
            *mp++ = (clr|set) & j ? 0xFF : 0x00;
        }
    }
    swoffset = curs->offset;
    swvers++;

    memimagedraw(swimg1,  swimg1->r,  swimg,  ZP, memopaque, ZP, S);
    memimagedraw(swmask1, swmask1->r, swmask, ZP, memopaque, ZP, S);
}

static int
swmove(VGAscr*, Point p)
{
    swpt = addpt(p, swoffset);
    return 0;
}

static void
swcursorclock(void)
{
    int x;

    if(!swenabled)
        return;
    if(swvisible && eqpt(swpt, swvispt) && swvers==swvisvers)
        return;

    x = arch_splhi();
    // check again, might have changed in between
    if(swenabled)
     if(!swvisible || !eqpt(swpt, swvispt) || swvers!=swvisvers)
      if(canqlock(&drawlock)){

        swcursorhide();
        swcursordraw();

        qunlock(&drawlock);
    }
    arch_splx(x);
}

void
swcursorinit_wrapper(void)
{
    static bool init;
    VGAscr *scr;
    static bool warned;

    didswcursorinit = true;
    if(!init){
        init = true;
        addclock0link(swcursorclock, 10);
    }
```

```
    scr = &vgascreen;

    if(scr == nil || gscreen == nil)
        return;
    if(scr->dev == nil || scr->dev->linear == nil){
        if(!warned){
            print("cannot use software cursor on non-linear vga screen\n");
            warned = true;
        }
        return;
    }

    if(swback){
        freememimage(swback);
        freememimage(swmask);
        freememimage(swmask1);
        freememimage(swimg);
        freememimage(swimg1);
    }

    swback  = allocmemimage(Rect(0,0,32,32), gscreen->chan);

    swmask  = allocmemimage(Rect(0,0,16,16), GREY8);
    swmask1 = allocmemimage(Rect(0,0,16,16), GREY1);
    swimg   = allocmemimage(Rect(0,0,16,16), GREY8);
    swimg1  = allocmemimage(Rect(0,0,16,16), GREY1);

    if(swback == nil || swmask == nil || swmask1 == nil || swimg == nil || swimg1 == nil){
        print("software cursor: allocmemimage fails");
        return;
    }

    memfillcolor(swmask,  DOpaque);
    memfillcolor(swmask1, DOpaque);
    memfillcolor(swimg,   DBlack);
    memfillcolor(swimg1,  DBlack);
}
```

⟨*global* swcursor*(x86)* 188c⟩

G.11.3 kernel/devices/screen/386/devvga.c

⟨kernel/devices/screen/386/devvga.c 497⟩≡

```
/*
 * VGA controller
 */
#include "u.h"
#include "../port/lib.h"
#include "mem.h"
#include "dat.h"
#include "fns.h"
#include "io.h"
#include "../port/error.h"

#include <draw.h>
#include <memdraw.h>
#include <cursor.h>
```

```
#include "../port/portscreen.h"
#include "screen.h"
```

⟨*enum* `_anon_ (kernel/devices/screen/386/devvga.c)` 366a⟩

⟨*global* `vgadir`*(x86)* 366b⟩

⟨*enum* `_anon_ (kernel/devices/screen/386/devvga.c)2` 366c⟩

⟨*global* `vgactlmsg`*(x86)* 367a⟩

⟨*function* `vgareset`*(x86)* 367b⟩

⟨*function* `vgaattach`*(x86)* 367c⟩

⟨*function* `vgawalk`*(x86)* 367d⟩

⟨*function* `vgastat`*(x86)* 367e⟩

⟨*function* `vgaopen`*(x86)* 367f⟩

⟨*function* `vgaclose`*(x86)* 368a⟩

⟨*function* `vgaread`*(x86)* 368d⟩

⟨*function* `vgactl`*(x86)* 369⟩

⟨*global* `Enooverlay`*(x86)* 374b⟩

⟨*function* `vgawrite`*(x86)* 374c⟩

⟨*global* `vgadevtab`*(x86)* 363e⟩

G.11.4 kernel/devices/screen/386/vgaclgd542x.c

⟨`kernel/devices/screen/386/vgaclgd542x.c` 498⟩≡

```
#include "u.h"
#include "../port/lib.h"
#include "../port/error.h"
#include "mem.h"
#include "dat.h"
#include "fns.h"

#include "io.h"

#include <draw.h>
#include <memdraw.h>
#include <cursor.h>

#include "../port/portscreen.h"
#include "screen.h"
```

⟨*function* `clgd542xpageset`*(x86)* 380b⟩

⟨*function* `clgd542xpage`*(x86)* 380a⟩

⟨*function* `clgd542xlinear`*(x86)* 379b⟩

⟨*function* clgd542xdisable*(x86)* 382a⟩

⟨*function* clgd542xenable*(x86)* 380d⟩

⟨*function* clgd542xinitcursor*(x86)* 382b⟩

⟨*function* clgd542xload*(x86)* 384⟩

⟨*function* clgd542xmove*(x86)* 383⟩

⟨*global* vgaclgd542xdev*(x86)* 379a⟩

⟨*global* vgaclgd542xcur*(x86)* 380c⟩

G.11.5 kernel/devices/screen/386/vgavesa.c

⟨*enum* _anon_ (kernel/devices/screen/386/vgavesa.c) 499a⟩≡ (501c)

```
enum {
    Usesoftscreen = 1,
};
```

⟨*global* hardscreen*(x86)* 499b⟩≡ (501c)

```
static void *hardscreen;
```

⟨*global* modebuf*(x86)* 499c⟩≡ (501c)

```
static uchar modebuf[0x1000];
```

⟨*function* WORD*(x86)* 499d⟩≡ (501c)

```
// pad: seems similar to macros in fcall.h, factorize?
#define WORD(p) ((p)[0] | ((p)[1]<<8))
```

⟨*function* LONG*(x86)* 499e⟩≡ (501c)

```
#define LONG(p) ((p)[0] | ((p)[1]<<8) | ((p)[2]<<16) | ((p)[3]<<24))
```

⟨*function* vbesetup*(x86)* 499f⟩≡ (501c)

```
static uchar*
vbesetup(Ureg *u, int ax)
{
    ulong pa;

    pa = PADDR(RMBUF);
    memset(modebuf, 0, sizeof modebuf);
    memset(u, 0, sizeof *u);
    u->ax = ax;
    u->es = (pa>>4)&0xF000;
    u->di = pa&0xFFFF;
    return modebuf;
}
```

⟨*function* vbecall*(x86)* 499g⟩≡ (501c)

```
static void
vbecall(Ureg *u)
{
    Chan *creg, *cmem;
    ulong pa;

    cmem = namec("/dev/realmodemem", Aopen, ORDWR, 0);
    if(waserror()){
        cclose(cmem);
```

```
        nexterror();
    }
    creg = namec("/dev/realmode", Aopen, ORDWR, 0);
    if(waserror()){
        cclose(creg);
        nexterror();
    }
    pa = PADDR(RMBUF);
    /* TODO: check read and write return values */
    devtab[cmem->type]->write(cmem, modebuf, sizeof modebuf, pa);
    u->trap = 0x10;
    devtab[creg->type]->write(creg, u, sizeof *u, 0);

    devtab[creg->type]->read(creg, u, sizeof *u, 0);
    if((u->ax&0xFFFF) != 0x004F)
        error("vesa bios error");
    devtab[cmem->type]->read(cmem, modebuf, sizeof modebuf, pa);

    poperror();
    cclose(creg);
    poperror();
    cclose(cmem);
}
```

⟨*function* vbecheck*(x86)* 500a⟩≡ (501c)

```
static void
vbecheck(void)
{
    Ureg u;
    uchar *p;

    p = vbesetup(&u, 0x4F00);
    strcpy((char*)p, "VBE2");
    vbecall(&u);
    if(memcmp((char*)p, "VESA", 4) != 0)
        error("bad vesa signature");
    if(p[5] < 2)
        error("bad vesa version");
}
```

⟨*function* vbegetmode*(x86)* 500b⟩≡ (501c)

```
static int
vbegetmode(void)
{
    Ureg u;

    vbesetup(&u, 0x4F03);
    vbecall(&u);
    return u.bx;
}
```

⟨*function* vbemodeinfo*(x86)* 500c⟩≡ (501c)

```
static uchar*
vbemodeinfo(int mode)
{
    uchar *p;
    Ureg u;

    p = vbesetup(&u, 0x4F01);
    u.cx = mode;
```

```
    vbecall(&u);
    return p;
}
```

⟨*function* addvgaseg*(x86)* 501a⟩≡ (492)

```
void
addvgaseg(char *name, ulong pa, ulong size)
{
    Physseg seg;

    memset(&seg, 0, sizeof seg);
    seg.attr = SG_PHYSICAL;
    seg.name = name;
    seg.pa = pa;
    seg.size = size;
    addphysseg(&seg);
}
```

⟨*function* vesaflush*(x86)* 501b⟩≡ (501c)

```
static void
vesaflush(VGAscr *scr, Rectangle r)
{
    int t, w, wid, off;
    ulong *hp, *sp, *esp;

    USED(scr); // now use gscreen directly
    if(hardscreen == nil)
        return;
    if(rectclip(&r, gscreen->r) == 0)
        return;
    sp = (ulong*)(gscreendata.bdata + gscreen->zero);
    t = (r.max.x * gscreen->depth + 2*BI2WD-1) / BI2WD;
    w = (r.min.x * gscreen->depth) / BI2WD;
    w = (t - w) * BY2WD;
    wid = gscreen->width;
    off = r.min.y * wid + (r.min.x * gscreen->depth) / BI2WD;

    hp = hardscreen;
    hp += off;
    sp += off;
    esp = sp + Dy(r) * wid;
    while(sp < esp){
        memmove(hp, sp, w);
        hp += wid;
        sp += wid;
    }
}
```

⟨kernel/devices/screen/386/vgavesa.c 501c⟩≡

```
/*
 * vga driver using just vesa bios to set up.
 *
 * note that setting hwaccel to zero will cause cursor ghosts to be
 * left behind.  hwaccel set non-zero repairs this.
 */
#include "u.h"
#include "../port/lib.h"
#include "../port/error.h"
#include "mem.h"
#include "dat.h"
```

```
#include "fns.h"

#include "io.h"
#include <ureg.h>

#include <draw.h>
#include <memdraw.h>
#include <cursor.h>

#include "../port/portscreen.h"
#include "screen.h"
```

⟨*enum* `_anon_ (kernel/devices/screen/386/vgavesa.c)` 499a⟩

⟨*global* `hardscreen`*(x86)* 499b⟩
⟨*global* `modebuf`*(x86)* 499c⟩

⟨*function* `WORD`*(x86)* 499d⟩
⟨*function* `LONG`*(x86)* 499e⟩
⟨*function* `vbesetup`*(x86)* 499f⟩

⟨*function* `vbecall`*(x86)* 499g⟩

⟨*function* `vbecheck`*(x86)* 500a⟩

⟨*function* `vbegetmode`*(x86)* 500b⟩

⟨*function* `vbemodeinfo`*(x86)* 500c⟩

⟨*function* `vesalinear`*(x86)* 378b⟩

⟨*function* `vesaflush`*(x86)* 501b⟩

⟨*global* `vgavesadev`*(x86)* 378a⟩

Glossary

```
API = Application Programming Interface
LOC = Lines Of Code
VGA = Video Graphics Array
RGB = Red Green Blue
GUI = Graphical User Interface
CAD = Computer Aided Design
OpenGL = Open Graphics Library
VGA = Video Graphics Array
```

Indexes

Here is a list of the identifiers used, and where they appear. Underlined entries indicate the place of definition. This index is generated automatically.

Bibliography

[FDFH90] James Foley, Andries Van Dam, Steven Feiner, and John Hughes. *Computer Graphics, Principles and Practice*. Addison-Wesley, 1990. cited page(s) 13, 17

[GRA89] James Gosling, David S.H. Rosenthal, and Michelle J. Arden. *The NeWS Book, an Introduction to the Network/Extensible Window System*. Springer-Verlag, 1989. cited page(s) 15

[Inc85] Adobe Systems Incorporated. *Postscript Language Reference Manual*. Addison-Wesley, 1985. cited page(s) 15

[Knu92] Donald E. Knuth. *Literate Programming*. Center for the Study of Language and Information, 1992. cited page(s) 17

[KR88] Brian W. Kernighan and Dennis M. Ritchie. *The C Programming Language*. Prentice Hall, 1988. cited page(s) 16

[Pad09] Yoann Padioleau. Syncweb, literate programming meets unison, 2009. https://github.com/aryx/syncweb. cited page(s) 17

[Pad14] Yoann Padioleau. *Principia Softwarica: The Plan 9 Kernel 9*. 2014. cited page(s) 14, 16, 17, 27, 29

[Pad16a] Yoann Padioleau. *Principia Softwarica: The Plan 9 Core Libraries*. 2016. cited page(s) 29

[Pad16b] Yoann Padioleau. *Principia Softwarica: The Plan 9 Windowing System rio*. 2016. cited page(s) 13, 14, 22, 29

[PGI84] Rob Pike, Leo Guibas, and Dan Ingalls. Bitmap graphics, course notes. In *SIGGRAPH*, 1984. Also available at lib_graphics/docs/pike-bitbltd-1984.pdf. cited page(s) 20

[Pik83a] Rob Pike. The blit: A multiplexed graphics terminal. Technical report, Bell Labs, 1983. Also available at lib_graphics/docs/blit-1983.pdf. cited page(s) 14

[Pik83b] Rob Pike. Graphics in overlapping bitmap layers. In *SIGGRAPH*, pages 331–356, 1983. Also available at lib_graphics/docs/pike-bitmap-1983.ps. cited page(s) 17

[Ros88] David Rosenthal. A simple X11 client program -or- how hard can it really be to write "hello, world"? In *USENIX Winter Conference*, pages 229–242, 1988. cited page(s) 15

[SG86] Robert W. Scheifler and Jim Gettys. The X Window system. In *ACM Transaction of Graphics*, pages 79–109, 1986. cited page(s) 14

[TML+79] C. P. Thacker, E. M. McCreight, B. W. Lampson, R. F. Sproull, and D. R. Boggs. Alto: A personal computer. Technical report, Xerox PARC, 1979. CSL-79-11. cited page(s) 13

[WNDS99] Mason Woo, Jackie Neider, Tom Davis, and Dave Shreiner. *OpenGL Programming Guide: The Official Guide to Learning OpenGL, Version 1.2*. Addison-Wesley, 3rd edition, 1999. cited page(s) 15

[WP03] Carl Worth and Keith Packard. Xr: Cross-device rendering for vector graphics. In *Ottawa Linux Symposium*, 2003. cited page(s) 15